# UF2404: Principios de la programación orientada a objetos

# Presentación

## Identificación de la Unidad Formativa

Bienvenido a la Unidad Formativa **UF2404: Principios de la programación orientada a objetos**. Esta Unidad Formativa pertenece al **Módulo Formativo MF0227_3: Programación orientada a objetos** que forma parte del Certificado de Profesionalidad **IFCD0112: Programación de lenguajes orientados a objetos y bases de datos relacionales**, de la familia de **Informática y comunicaciones**

## Presentación de los contenidos

La finalidad de esta Unidad Formativa es enseñar al alumno a implementar los componentes software encomendados de modo que cumplan las especificaciones del diseño, manipular bases de datos a través de interfaces para integrar el lenguaje de programación con el lenguaje de acceso a datos, probar los componentes software desarrollados para asegurar que cumplen las especificaciones recibidas, asi como utilizar los componentes orientados a objeto como base en el desarrollo de aplicaciones para el modelo de programación web, y por último, elaborar la documentación del código desarrollado según los estándares de la organización.

Para ello, se realizará en primer lugar una introducción al paradigma orientado a clases y objetos, se analizará la generalización/especialización, las relaciones entre clases, y se realizará un análisis del polimorfismo. También se aplicarán las técnicas de programación estructurada, la estructura de la información y los lenguajes de programación orientados a objetos, para terminar con la implementación del paradigma utilizando un lenguaje de programación orientado a objetos.

## Objetivos de la Unidad Formativa

Al finalizar esta Unidad Formativa aprenderás a:

– Dominar los conceptos fundamentales del paradigma orientado a objetos.

– Desarrollar clases aplicando los fundamentos del paradigma Orientado a Objetos.

# Índice

UD1. Introducción al paradigma orientado a objetos ..... 11

1.1. Ciclo de desarrollo del software bajo el paradigma de orientación a objetos: Análisis, diseño y programación orientada a objetos .... 13

1.2. Análisis del proceso de construcción de software: modularidad . 46

1.3. Distinción del concepto de módulo en el paradigma orientado a objetos ..... 70

1.4. Identificación de objetos como abstracciones de las entidades del mundo real que se quiere modelar..... 74

1.4.1. Descripción de objetos: Conjunto de datos que definen un objeto y conjunto comportamientos que pueden solicitarse a los objetos..... 75

1.4.2. Identificación del comportamiento de un objeto: Concepto del mensaje..... 83

UD2. Clases y objetos ..... 93

2.1. Distinguir el concepto de clase y sus atributos, métodos y mecanismo de encapsulación..... 95

2.1.1. Relación entre interfaz y clase..... 118

2.1.2. Distinción de los tipos de datos y clases..... 127

2.2. Análisis de los objetos: Estado, comportamiento e identidad 160

2.2.1. Análisis de mensajes..... 164

2.2.2. Tipos de métodos y su clasificación: Métodos de acceso, de selección o consulta, de construcción, de destrucción ..... 165

2.3. Uso de objetos como instancias de clase. Instancia actual (this, self, current)..... 175

2.4. Identificación del concepto de programa en el paradigma orientado a objetos. POO = objetos + mensajes ..... 179

UD3. Generalización/especialización: herencia ..... 189

3.1. Descripción del concepto de herencia: simple y múltiple ..... 191

3.1.1. Relación de herencia: características ..... 213

3.1.2. Reglas y características que definen una relación de herencia: regla “es–un” ..... 216

3.1.3. Transmisión de atributos y métodos..... 218

3.1.4. Regla de especialización de la superclase en la subclase ..... 223

3.1.5. Acceso a los atributos de una clase y acoplamiento entre las clases..... 227

3.1.6. Utilización de objetos this (current, self u otros) y super 231

3.1.7. Leyes de Demeter ..... 239

3.2. Distinción de la herencia múltiple ..... 243

3.2.1. Problemas: Conflictos de nombres, herencia repetida. 244

3.2.2. Soluciones ..... 250

3.3. Creación de objetos en la herencia..... 253

3.4. Clasificación jerárquica de las clases ..... 258

3.4.1. Clase raíz ..... 264

3.4.2. Clases abstractas ..... 266

3.4.3. Métodos virtuales..... 269

3.4.4. Redefinición de métodos..... 270

UD4. Relaciones entre clases ..... 283

4.1. Distinción entre agregación/composición ..... 296

4.2. Distinción entre generalización/especialización ..... 300

4.3. Identificación de asociaciones ..... 302

UD5. Análisis del polimorfismo ..... 311

5.1. Concepto ..... 313

5.2. Tipos ..... 318

5.3. Polimorfismo en tiempo de compilación (sobrecarga)..... 320

5.4. Polimorfismo en tiempo de ejecución (ligadura dinámica) ..... 322

5.5. Objetos polimórficos ..... 327

5.6. Comprobación estática y dinámica de tipos ..... 330

UD6. Técnicas de programación estructurada ..... 355

6.1. Identificación de elementos básicos: constantes, variables, operadores y expresiones ..... 357

6.2. Análisis de estructuras de control: secuencial, condicional y de repetición..... 385

6.3. Distinción entre funciones y procedimientos ..... 399

6.3.1. Interfaz ..... 402

6.3.2. Paso de parámetros: por valor y por referencia ..... 404

6.3.3. Parámetros actuales y formales ..... 410

6.3.4. Funciones: valor de retorno ..... 412

6.3.5. Procedimientos..... 414

6.3.6. Ámbito de las variables ..... 419

6.3.7. Almacenamiento de las variables ..... 425

6.4. Demostración de llamadas a funciones y procedimientos .... 429

6.5. Empleo de llamadas a funciones y procedimientos incluidos en las clases ..... 430

6.5.1. Llamadas calificadas y no calificadas (instancia actual) 433

6.5.2. Paso de parámetros ..... 437

6.5.3. Los atributos de la clase ..... 441

UD7. Estructura de la información ..... 451

7.1. Enumeración de datos simples: Numéricos (enteros y reales), lógicos, carácter, cadena de caracteres, puntero o referencia a memoria ..... 453

7.2. Datos estructurados: arrays ..... 467

7.2.1. Listas enlazadas, pilas y colas ..... 480

7.2.2. Estructuras ..... 486

7.2.3. Ficheros ..... 488

7.2.4. Otras estructuras complejas: tablas hash e introducción a los árboles y grafos ..... 493

7.3. Mecanismos de gestión de memoria ..... 496

7.3.1. Uso de la gestión automática de memoria ..... 500

7.3.2. Construcción y destrucción de objetos ..... 504

7.3.3. Objetos inalcanzables ..... 507

7.3.4. Recolección de «basura» ..... 508

7.3.5. Métodos constructores y destructores ..... 511

UD8. Lenguajes de programación orientados a objetos ..... 523

8.1. Análisis del lenguaje de programación orientado a objetos y paradigma orientado a objetos ..... 525

8.1.1. Lenguajes de programación orientados a objetos... 532

8.1.2. Lenguajes de programación basados en objetos ... 534

8.1.3. Lenguajes de programación que utilizan objetos..... 538

8.2. Comparación entre los lenguajes de programación orientados a objetos más habituales. Características esenciales ..... 539

8.3. Librerías de clases ..... 554

8.3.1. Definición de su estructura ..... 564

8.3.2. Creación y utilización ..... 568

UD9. Implementación del paradigma utilizando un lenguaje de programación orientado a objetos ..... 577

9.1. Elección del lenguaje ..... 581

9.2. Enumeración de los tipos de aplicaciones ..... 600

9.3. Herramientas de desarrollo ..... 601

9.4. Tipos de datos y elementos básicos característicos del lenguaje. Instrucciones ..... 630

9.5. Estudio y utilización de las clases básicas incluidas en la librería de clases ..... 633

9.6. Definición de clases ..... 647

9.7. Construcción de métodos. Sobrecarga ..... 648

9.8. Construcción de atributos ..... 650

9.9. Construcción de la interfaz de la clase ..... 652

9.10. Construcción de clases incluyendo relaciones de agregación/composición y asociación ..... 654

9.11. Construcción de clases con herencia ..... 658

9.12. Construcción de clases con herencia múltiple ..... 667

9.13. Definición de clases abstractas ..... 673

9.14. Construcción de clases con herencia incluyendo polimorfismo . 676

9.15. Empleo de excepciones ..... 678

9.16. Gestión de eventos ..... 688
9.16.1. Eventos, fuentes y auditores de eventos ..... 691
9.16.2. Tipos de eventos. Mecanismos de gestión de eventos 693
9.16.3. Librerías de clases asociadas ..... 696

9.17. Empleo de hilos ..... 699
9.17.1. Fundamentos ..... 702
9.17.2. Creación ..... 704
9.17.3. Prioridad ..... 707
9.17.4. Comunicación ..... 709
9.17.5. Sincronización ..... 716
9.17.6. Estados ..... 718
9.17.7. Creación y ejecución de hilos en el lenguaje ..... 722
9.17.8. Librerías de clases asociadas ..... 727
9.17.9. Programación multihilo ..... 740

9.18. Definición y análisis de programación en red ..... 743
9.18.1. Aplicación cliente servidor ..... 747
9.18.2. Sockets ..... 750

9.19. Acceso a bases de datos desde las aplicaciones. Librerías de clases asociadas ..... 763

Glosario ..... 783

Soluciones ..... 787

# UD1

## Introducción al paradigma orientado a objetos

1.1. Ciclo de desarrollo del software bajo el paradigma de orientación a objetos: Análisis, diseño y programación orientada a objetos

1.2. Análisis del proceso de construcción de software: modularidad

1.3. Distinción del concepto de módulo en el paradigma orientado a objetos

1.4. Identificación de objetos como abstracciones de las entidades del mundo real que se quiere modelar

1.4.1. Descripción de objetos: Conjunto de datos que definen un objeto y conjunto comportamientos que pueden solicitarse a los objetos

1.4.2. Identificación del comportamiento de un objeto: Concepto del mensaje

## 1.1. Ciclo de desarrollo del software bajo el paradigma de orientación a objetos: Análisis, diseño y programación orientada a objetos

### Un poco de historia

Todas las ideas elementales de la programación con el paradigma de la orientación a objetos nacen a principios de los años 60 en la universidad de Noruega. Con la idea de simular y obtener rendimientos de un motor, un grupo de programadores dirigido por el Dr. Nygaard se dedicaba a desarrollar sistemas informáticos para realizar dichas simulaciones de componentes físicos. La dificultad en la que se encontraban era doble. Por un lado los programas eran muy complejos y, por otro, forzosamente tenían que ser muy modificados. Este segundo punto era especialmente problemático, ya que la razón de ser de los programas era el cambio y no sólo se requerían varias iteraciones para obtener un producto con el rendimiento deseado, sino que muchas veces se querían obtener diversas alternativas viables cada una con sus ventajas e inconvenientes.

Una idea que se les ocurrió fue diseñar el programa de manera paralela y con todas las piezas que tuviera, si tenía 20 piezas el código también tendría 20 módulos, uno por cada pieza. Con esta total correspondencia entre el sistema físico y el sistema informático, el lenguaje también se enviaba mensajes. Era una manera de abstraer informáticamente cada pieza en un módulo independiente.

los dos problemas planteados, ya que el principio es lógico que al partir el programa en módulos como unidades informáticas, la descomposición es automática desde la visión de las unidades físicas. Y la forma natural de fragmentar algo muy complejo era una buena manera del resolverse. La reparación muy sencilla y controlable. El segundo punto también se resuelve ya que, a cada iteración de simulación, el analista querrá cambiar o bien piezas enteras o bien el comportamiento de alguna pieza. En ambos casos la localización de los cambios está perfectamente clara y su alcance se reduce a un componente, siempre y cuando el interfaz del mismo no cambie. Por ejemplo, si se estuviese simulando un motor de coche, puede que se quisiera modificar la caja de

cambios utilizado en la simulación anterior. Si la nueva caja de cambios tuviera la misma interfaz (mismos inputs y outputs) o se cambiase sólo su comportamiento interno, nada del sistema (fuera de la caja de cambios) estaría afectado por el cambio. Ningún elemento del motor necesitaría modificarse. Ningún elemento de la simulación con el programa debería rectificarse.




Otra razón muy importante apareció en el estudio de ese paradigma de programación, y su beneficio enorme, el concepto de reusabilidad. Poder rehusar piezas para futuros programas abrió grandes horizontes a la industria de la informática relacionada con la programación.

Construir y programas piezas que pueda utilizar programas de muy diversa índole y naturaleza. Esto llevó a unos niveles de reutilización de hasta un 80% del código. Para implementar estas ideas lo que se hizo fue crear un lenguaje para darle soporte, Simula–67, que continua utilizándose actualmente.

Otro paso muy importante se dio gracias a Xerox en su centro de investigación allá por los años 70, cuando contrataron a un joven llamado Alan Kay para que llevase a término las ideas que proponía en su tesis doctoral, la propuesta de construcción de un ordenador llamado Dynabook, adecuado para ser utilizado por niños. El ordenador no tenía teclado, la pantalla era sensible al tacto y la mayor parte de la comunicación era gráfica. Al desarrollar este proyecto se inventó el 'mouse' y los entornos gráficos. Al volver a encontrarse con una programación compleja y experimental, como en el caso de Nygaard, decidieron crear un entorno y lenguaje llamado Smalltalk.

Este lenguaje Smalltalk tuvo gran aceptación e intentaron crear uno que fuera sucesor de C en los años 80 por la ATT–Bell, así fue como incorporaron las principales ideas de Smalltalk y de Simula, creando el lenguaje C++. Puede afirmarse que se debe a este último la gran extensión de los conceptos de la programación orientada a objetos.

Conviene recalcar que el desarrollo que hemos sintetizado se refiere sólo a la evolución que ha tenido la orientación a objetos en el mundo de la ingeniería del software. Ha coexistido un desarrollo análogo de los mismos conceptos en el mundo de la inteligencia artificial, donde el lenguaje **CLOS**, una variedad de Lisp orientada a objetos, está enormemente extendido.

Evolución de lenguajes de programación OOP:

- Simula: 1967 → Introduce el concepto de Objeto.
- Smalltalk: 1980 → POO puro, desarrollado en Xerox Palo Alto Reasearch Center Learning Research Group.
- C++: 1983–1990 → por Bjarne Stroustrup (C con clases).
- Object Pascal: 1986 → Esqueleto simplificado de lenguaje POO por desarrolladores de Apple Computer y Niklaus Wirth.
- LISP/CLOS: 1988 → Diseñado por un comité presidido por Daniel Bobrow a partir de la ACM Lisp and Fuctional Programming Conference de 1986.
- Java: 1995, 1997, 1998 → Sun Microsystems, estándar de facto en el desarrollo de aplicaciones actuales.
- C# (C Sharp) → Microsoft (Scott Wiltamuth y Anders Hejlsberg). Plataforma ".Net".

## Principios de la programación orientada a objetos




La orientación a objetos se basa en tres pilares básicos:

- Encapsulación y ocultación de la información.
- Abstracción y clasificación
- Herencia y polimorfismo.

Por el término encapsulación se entiende como la agrupación en una entidad o caja de datos sobre esas operaciones. Está claro que en cualquier programa existe una encapsulación, pero por encapsulación queremos decir algo más preciso: en concreto, cada componente ha de tener nada más que los datos relacionados con un tema y su manipulación.

Por ejemplo cualquier programa de facturación puede leer directamente un registro de artículo y modificar el 'stock' al mismo tiempo que imprime la factura y modifica el saldo del cliente. Tenemos tres actividades sobre tres entidades diferentes en un único componente ('el programa'). Este programa no tiene una encapsulación adecuada.

La noción de ocultación de la información se describe a que cada componente sabe lo minúsculo posible de los otros y suministra la mínima información permisible sobre sí mismo. De muestra, si en una aplicación se obtiene el acceso al saldo de un cliente, es superfluo saber si el saldo está depositado como un entero o un real o si se ha de formar una rutina para obtenerlo. Sólo debe inquietar como pedirle al cliente su saldo y no como está representado internamente. Estos principios son fundamentales para una correcta programación.

Desde hace cuantiosos años, estas iniciaciones han estado utilizables en los lenguajes de programación, aunque su uso era voluntario. La diferencia es que los lenguajes orientados a objetos dan soporte sintáctico a los dos conceptos, y que su uso es obligado en muchos lenguajes puros orientados a objetos.

El consecutivo pilar es el de la abstracción y clasificación. Por abstracción deducimos que los módulos, a los que manejamos la encapsulación y la ocultación mencionadas, sean representaciones abstractas de los conceptos que nos interesan. Es decir, que si hacemos una aplicación para una autoescuela, deberemos crear una entidad informática, que de momento podemos llamar módulo, para abstraer los hechos más característicos de los conceptos que estamos tratando: coche, alumno, matrícula, semáforo, etc.

Hasta ahora, se habían considerado solamente los datos al diseñar la base de datos. Ahora deberemos hacerlo con todas las características de las entidades de nuestro modelo o dominio. Por ejemplo, si un cliente puede tener un descuento, deberemos asociar la rutina de cálculo al cliente y no a los programas aplicativos. Hay que señalar, que el módulo 'cliente' quedará definido no sólo por unos datos (nombre, DNI, dirección…) sino también por rutinas (cálculo del descuento, saldo…).

Desde el momento en que se realizan las abstracciones de esa forma, se están clasificando las entidades del dominio en clases. Es decir, la clase 'cliente' (la definición de la clase cliente) describe no solamente un cliente concreto sino a todos los clientes que cumplan unas características dadas. Podemos ver a 'cliente' como una fábrica de clientes, donde cada cliente concreto será una 'instancia' (un objeto) de una clase cliente.

En ocasiones se apunta que en la programación orientada a objetos se tratan tipos de usuario. Pretende exponerse que el programador (el usuario) logra definir unas formas abstractas (como ahora cliente o entero) a partir de los datos (atributos) que le son propios y las operaciones que sean válidas (cálculo- des-

cuento o suma) sobre la entidad. Es decir, además de unos tipos que vienen definidos por el lenguaje (enteros, reales, caracteres), el usuario puede crear otros nuevos y, a partir de éstos, crear instancias o variables de los tipos del usuario.

Conjuntamente puede beneficiarse diferentes prototipos de clientes (mayoristas, minoristas, etc.) que serán solo en parte diferentes. Con ello, todos poseerán nombre, DNI, dirección, etc.; pero la rutina de cálculo de descuento será distinta por el tipo de cliente y además, cada cliente poseerá una información adicional necesaria para calcular el descuento (un porcentaje, una tabla de descuentos, etc.) diferente según el tipo de cliente.

Cuando esto sucede, se puede definir un cliente abstracto como lo que tienen en común todos los clientes para asignar a continuación, en cada uno de los tipos de clientes que se tienen, sólo aquello que le es exclusivo. Es decir, cliente mayorista heredará toda la descripción de cliente abstracto y le sumará algunas de las características adicionales.




La herencia consiente de momento detallar entidades “clases” por contraste entre ellas; por eso se apunta que programar con orientación a objetos es programar para diversificar. La utilidad de la herencia es múltiple. En primer lugar hay que analizar, diseñar, codificar y poner a punto menos, ya que en cuanto se tiene un concepto definido, jamás lo repetimos.

Como mucho, si el concepto nuevo es muy similar a uno primero, se concreta el nuevo como la diferencia respecto al anterior. Si sale un error en la utilización de la nueva entidad, seguro que el error está en lo que se ha añadido porque lo que se ha heredado ya se había probado.

Existen varios tipos de herencia: los más significativos son simple y múltiple. Con ello pretendo recalcar que, al inverso de la herencia biológica donde eternamente se hereda de dos padres, en la orientación a objetos se puede heredar de uno, dos o más padres. La ilustración de cliente minorista a partir de cliente es una muestra de herencia simple. Y se podría definir un vehículo anfibio como una herencia (una subclase) de otras dos clases: coche y barco.

Queda como posterior noción a tratar el polimorfismo. En la muestra anterior de los clientes se tomaría conseguir que toda subclase de cliente poseyera una operación (“servicio” o “método”) de cálculo–descuento, concretando una operación diferida (o virtual) en la clase abstracta cliente.Para ello, que cliente pasa a ser una especie de plantilla de todos los clientes donde hay algunas cosas definidas (nombre, DNI) y otras pendientes a definir en cada subclase (cálculo–descuento). En caso de que esto suceda, podríamos garantizar que todo cliente tendría una operación de ‹calcular descuento› asociada, sea el tipo que sea de cliente. Si nos hallamos en esta situación quedaremos en disposición de usar lo que se llama polimorfismo, que resuelve mucho los programas y los hace extensibles a nuevos tipos sin necesidad siquiera de recompilar.

Como modelo, si pretendiéramos hacer un programa de inscribir facturas a partir de unos albaranes, lograríamos ir leyendo secuencialmente los albaranes a medida que los anduviésemos emparejando con clientes. En cada emparejamiento imprimiríamos una factura tras calcular el descuento aplicable. Entendemos que descuento es algo que está precisado en “cliente”, podemos mandar a cliente el mensaje “calculo_descuento” pedirle que ruede la rutina con “albarán” como parámetro. La instrucción sería algo parecido a: Descuento = cliente.calculo_descuento (albaran)

Esta instrucción es notable para todos los clientes sean del prototipo que sean. Por ello con un mismo recado y según sea el receptor, se establecen diferentes rutinas (diferentes representaciones para un mismo mensaje = polimorfismo). También y mientras se amplíe que todo cliente tiene una operación asociada de calculo_descuento, podremos crear nuevos tipos de clientes sin necesidad ni de recompilar el programa de facturas.



## El impacto de la orientación a objetos

Este paradigma de programación, ha supuesto una revolución en la industria del software ha sido enorme, algunas de sus razones son estas.

La primera y más trascendental razón es la reutilización. Hasta el presente sólo se podía obtener reutilización de dos formas: rutinas de bajo nivel (operar con datos) o subsistemas completos (la aplicación de cartera que enlaza con cualquier facturación). Esto limitaba fuertemente la reutilización, porque reaprovechar una aplicación quería decir aceptarla como era al 100%. Como mucho se podía intentar parametrizar una aplicación potencialmente reutilizable si se lograban prever futuros requerimientos al programa y se flexibilizaba su uso. El coste es envolver mucho más el programa y obstaculizar su mantenimiento, sin excluir del todo la necesidad de modificar el programa cuando las exigencias de los clientes no pueden conseguirse con la parametrización.




El paradigma (la forma de madurar y representar el contexto) de la orientación a objetos es mucho más potente que el estructurado y permite obtener más reusabilidad, por dos razones. En primer lugar porque se puede tener reusabilidad por separado, tanto del análisis como del diseño y la programación; no estamos obligados a coger un paquete entero: si sólo nos interesa el análisis, lo podemos reutilizar con un diseño diferente del que se había utilizado originalmente.

La segunda razón es la herencia. Si una aplicación tiene algunas partes que no se adecuan a nuestras necesidades, podemos modificarlos sin 'parchear' mediante la herencia. Es decir, la programación pasa a ser como el 'prêt–a–porter' en la industria textil. Podemos tener la mayor parte hecha industrialmente y adaptarla a cada cliente según sus necesidades.

La programación orientada a objetos además es profusa y más fidedigna por diversas razones. En primer lugar por el desarrollo incremental y la programación por diferencia, al poder ir añadiendo funcionalidad vía herencia. El tamaño medio de una rutina en entornos orientados a objetos es de 4 o 5 líneas; y se ha de tener en cuenta que sólo se tienen rutinas, ya que no existe el concepto de programa principal. La utilización masiva de librerías de clases garantiza la fiabilidad, ya que los componentes sólo se añaden a la librería cuando se ha verificado la corrección de su funcionamiento.

Algunos aspectos de la verificación formal de los programas, que hasta ahora era un propuesta totalmente teórica, se han añadido por vez primera a los lenguajes de producción.

Veamos a continuación el impacto de la orientación a objetos en la ingeniería del software.

En primer término los lenguajes, que es en el cual se forma la orientación a objetos. Como ya hemos señalado el lenguaje mas extendido es el C++. Este consiente la compatibilidad con todo el software C y, por ello, ANSI ha asumido su estandarización. También ANSI está ampliando en Cobol y Fortran las nociones de la orientación a objetos. Para Cobol ya existía el esbozo que se aprobó en 1997 y también un producto (Microfocus) que incorpora parte de las extensiones supuestas por ANSI. Este nuevo Cobol permitirá mantener la compatibilidad con Cobol–85 al mismo tiempo que ofrecerá una vía de acceso a la nueva tecnología para todos los grandes centros con "mainframes" que trabajen con Cobol. El entorno ADA también tendría extensiones orientadas a objetos ya que se han incorporado a la nueva definición ADA/9X .

Además es pertinente tener metodologías de exploración y diseño como existían en el mundo estructurado. En todo lo que respecta a los centros que trabajan con metodologías y herramientas CASE, es impensable pasar a una nueva tecnología sin metodologías. Los autores principales de los métodos estructurados (Martin, Mellor, Yourdon) se han pasado a la orientación a objetos y han realizado propuestas de análisis y diseño orientado a objetos. Los constructores de herramientas CASE, por su parte, han desarrollado nuevas herramientas o han añadido extensiones a les herramientas anteriores.

También el entorno global de las bases de datos está moviéndose hacia la orientación a objetos. Por un lado se tienen las BDOO, bases de datos orientadas a objetos puras (POET, Objectstore, etc.) y por otro las relacionales. En las BDOO, la organización ODMG representa el 100% de las BDOO industriales y ha establecido un estándar de definición (ODL) y manipulación (OQL) de bases de datos equivalente a SQL. En cambio lo relativo a las bbdd relacionales, como Oracle o Informix, están sumando en mayor o menor grado varios aspectos de la orientación a objetos.

Esta organización ODMG nació de un grupo más grande, llamado OMG, donde están personificadas todas las grandes compañías con influencia en el sector. Este grupo definió un estándar universal por objetos. Este desarrollo permitió que un objeto programado en cualquier lenguaje y sistema operativo acceda a cualquier otro objeto sin importar ni tener en cuenta lenguaje ni sistema operativo. Esto facilitará enormemente el desarrollo de sistemas abiertos cliente–servidor.

Ya se auguraba que los sistemas operativos además están floreciendo afectados por la orientación a objetos. En primer lugar ya se tiene un primer sistema operativo orientado a objetos, NextStep, que está integrado con Sun. Microsoft desde su segmento anunció para 1996 un sistema operativo orientado a objetos llamado de momento Cairo. IBM y Apple Además estaban implantando uno nuevo, a través de la empresa Taligent, que de en el instante de s creación se denominó Pink y que también estaría disponible a finales del 95 o principios del 96.




Cuando un software se planifica bien, se implementa y acaba teniendo éxito, sobre todo cuando satisface las necesidades de las personas que lo utilizan; cuando funciona de forma impecable durante mucho tiempo; cuando es fácil de modificar o incluso es más fácil de utilizar, puede cambiar todas las cosas y de hecho cambiar para mejor.

Pero cuando un software da problemas y falla, cuando los usuarios no quedan totalmente satisfechos, cuando es proclive a errores, cuando es dificultoso de cambiar e incluso más difícil de utilizar, pueden ocurrir y de hecho ocurren verdaderos desastres. Todos queremos desarrollar un software que haga bien las cosas, sorteando que los problemas aparezcan. Para tener éxito al diseñar y construir un software necesitaremos disciplina. Es decir, necesitaremos un enfoque de ingeniería.

## Término software

El término software tiene una especial transcendencia y relevancia. Entenderemos este concepto para poder avanzar en la definición de Ingeniería de Software.

Algunas definiciones de software:




El IEEE especifica el software como “programas, procedimientos y documentación y datos asociados, relacionados con la operación de un sistema informático”.

El software se puede definir como el conjunto de tres componentes:

Programas (instrucciones): este componente proporciona la funcionalidad deseada y el rendimiento cuando se ejecute.

– Datos: este elemento conlleva los datos necesarios para tratar y probar los programas y las estructuras necesarias para conservar y usar estos datos.

– Programas: Los programas están compuestos por sentencias e instrucciones que proporcionan la funcionalidad deseada cuando son arrancadas en el ordenador. Están tecleados usando lenguajes específicos que los ordenadores pueden leer y ejecutar, tales como lenguaje ensamblador, Basic, Java, C, Visual Basic, .Net, Fortran etc…Los programas también pueden ser generados usando generadores de programas.

  Los programas suministran la funcionalidad y operatividad necesaria para gestionar datos en grandes volúmenes y cálculos. Necesitan datos para aplicar el control apropiado en lo que realizan. El mantenimiento y las pruebas de los programas también necesitan datos. El diseño del programa asume la disponibilidad de las estructuras de datos tales como bases de datos y archivos que contienen la información.

– Documentos: Este componente describe la operación y uso del programa. Muy importantes son los anexos y manuales de cómo manejar y configurar los programas. Además de los programas y los datos, los usuarios necesitan también soporte y ayuda de cómo usar el programa.

  Los documentos también son requeridos por las personas encargadas de mantener el software para entender el interior del software y modificarlo, en el caso en que sea necesario.

A lo largo de los años, se han evolucionado muchas formas de producir bienes de mejor calidad en el sector de las empresas desarrolladoras de software. Este conocimiento puede extenderse a la construcción de productos software de mejor calidad si los profesionales del software entienden las características propias del software.

Para alcanzar a percibir lo que es el software (e inflexiblemente la ingeniería del software), es trascendente explorar las características del software que lo diferencian de otras entidades que el hombre puede construir.

El software es esencialmente un acumulado de instrucciones (programas) que facilitan la funcionalidad solicitada, los datos relacionados y documentos. Por ello el software es un componente lógico y etéreo y se diferencia del hardware, un elemento físico, en sus características.




El software se desarrolla, no se fabrica en el sentido clásico. Aunque existen similitudes entre el desarrollo del software y la construcción del hardware, ambas actividades son fundamentalmente distintas.

Cada producto software es desigual porque se levanta para efectuar los requisitos únicos de un cliente. Cada software requiere, por lo tanto, ser construido usando un enfoque de ingeniería.

Construir un producto software implica entender qué es necesario, diseñar el producto para que cumpla los requisitos, implementar el diseño usando un lenguaje de programación y comprobar que el producto cumple con los requisitos. Todas estas actividades se llevan a cabo mediante la ejecución de un proyecto software y requiere un equipo multidisciplinar trabajando de una forma coordinada.

El camino usado para levantar software es diferente de la fabricación del hardware, donde las aparatos se usan para originar partes y cada trabajador sólo requiere realizar la tarea estipulada o usar una máquina.




En el software, el recurso principal son las personas, las ideas y buenas herramientas de desarrollo y edición. Es muy difícil acelerar el diseño de software añadiendo personas porque la construcción de software demanda un esfuerzo en equipo. El equipo tiene que trabajar de forma coordinada y compartir un objetivo de proyecto común. Se necesita muchas dosis de comunicación efectiva dentro del equipo.

Un nuevo miembro del equipo no es prontamente productivo y requiere la iniciación proporcionada por el equipo y la formación para cumplir el trabajo para que encaje y se acomode progresivamente al proyecto. Esto demanda una inversión de tiempo y esfuerzo por parte de los órganos del equipo existentes y les puede distraer de su propio trabajo.

Otra particularidad del software es que no se estropea. Los deterioros no descubiertos harán que falle el programa durante los primeros períodos de su vida. Sin embargo, una vez que se corrigen partiendo de la premisa que no se introducen nuevos errores los fallos disminuyen.

El software difícilmente se deteriora, pero con el paso del tiempo sufre importantes desperfectos y desajustes. Durante su vida, el software sufre cambios relativos al mantenimiento. Conforme se hacen los cambios, es bastante probable que se introduzcan nuevos defectos, lo que hace que el software se vaya deteriorando debido a los cambios.

Más puntos del software y condicionantes es que debido a que la industria del software es nueva, el software se diferencia del hardware en el aspecto de uso de componentes. Sin embargo la mayoría de la industria tiende a ensamblar componentes, la mayoría del software se construye a medida.

Los módulos reutilizables se han fundado para que el ingeniero consiga concentrarse en componentes ciertamente innovadores de un esbozo. En el mundo del software es algo que únicamente ha comenzado a alcanzarse en productos lanzados a gran escala.






Cada vez hay más compañías de software que desarrollan utilidades y herramientas para el sector de la programación. Librerias, código fuente, clases y otras herramientas que ayudan al desarrollo.

El bloque y pack de software deberá desarrollarse e implementarse para que pueda volver a ser reutilizado en otros muchos programas diferentes. Hoy en día, se ha extendido la visión de la reutilización para abarcar tanto algoritmos como estructuras de datos, permitiendo al ingeniero del software crear nuevas aplicaciones a partir de las partes reutilizables.

El hardware usa módulos estándar con funciones e interfaces bien concretadas. El uso de estos dispositivos ayuda a evitar reinventar la rueda. La etapa de esquema en el ciclo de vida de un producto hardware involucra elegir los componentes adecuados más convenientes y decidir el enfoque para montarlos. Los componentes de hardware estándar son útiles porque conducen a:

- Bajar y reducir el coste.
- Bajar tiempo de lanzamiento al mercado.
- Buena calidad.
- Ingeniería rápida.
- Fácil mantenimiento.
- Fácil mejora.




El software se crea normalmente desde cero, aunque en muchos casos los desarrolladores se apoyan el librerías, clases y componentes de software ya implementados que ajustan y se configuran para desarrollar buen software a modo de piezas de puzzle.

Dichas herramientas necesitan de buenos programadores/as para su implementación, pues no resultan fáciles. Con frecuencia se construye de acuerdo a los requisitos específicos de un cliente y no se crea por la unión de componentes existentes.

Como la industria del hardware, la industria del software está intentando adoptar el mecanismo de reutilizar para hacer más fácil y más rápida la construcción. Las ventajas de la reutilización de software están siendo concebidas y evaluadas. Concurren algunos componentes reutilizables a través de librerías de funciones y objetos reutilizables que armonizan funciones y datos.

Mientras que la reutilización y el ajuste cimentado en componentes y clases se están incrementando, la mayoría del software sigue siendo construido de manera personalizada, y las cotas de reutilización actuales están lejos de los que corresponderían ser. Además, la tarea de identificar componentes reutilizables potenciales es difícil porque cada producto software es único y distinto.

La industria del software tiene procesos bien definidos para la reutilización de componentes. Esto incluye procesos para la construcción de componentes, almacenamiento de los mismos en librerías de donde se pueden extraer para su reutilización y entonces incorporarlos.

A lo largo de los años, la industria del software tiene en perspectiva crear componentes reutilizables específicos a dominios de aplicación particulares.




La expresión ingeniería de software se define en el DRAE (Diccionario de la Real Academia Española) como: Conjunto de conocimientos y técnicas que permiten aplicar el saber científico a la utilización de la materia y de las fuentes de energía.

Y el término ingeniero se define como: "Persona que profesa o ejerce la ingeniería".

Igualmente, la Real Academia de Ciencias Exactas, Físicas y Naturales de España define el término ingeniería de la siguiente forma: "Conjunto de conocimientos y técnicas cuya aplicación permite la utilización racional de los materiales y de los recursos naturales, mediante invenciones, construcciones u otras realizaciones provechosas para el hombre".

Otras definiciones:

- Ingeniería del Software es el estudio de los principios y metodologías para el desarrollo y mantenimiento de sistemas de software (Zelkovitz, 1978)
- Ingeniería del Software es la aplicación práctica del conocimiento científico al diseño y construcción de programas de computadora y a la documentación asociada requerida para desarrollar, operar (funcionar) y mantenerlos. Se conoce también como desarrollo de software o producción de software (Bohem, 1976).
- Ingeniería del Software trata del establecimiento de los principios y métodos de la ingeniería a fin de obtener software de modo rentable que sea fiable y trabaje en máquinas reales (Bauer, 1972).
- La aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación (funcionamiento) y mantenimiento del software; es decir, la aplicación de ingeniería al software. (IEEE, 1993).

La definición de IEEE describe la ingeniería del software como un enfoque sistemático cubriendo los aspectos del desarrollo, operación y mantenimiento. Este enfoque es disciplinado y cuantificable.

La ingeniería de software requiere llevar a cabo numerosas tareas, dentro de etapas como las siguientes:

### Análisis de requisitos

Uno de los apartados más importantes por no decir el más importante, es el momento del análisis de requisitos y de captar completamente que necesidades tiene el cliente y que debe hacer el software, quienes lo usarán, en que entorno y ambiente de trabajo. Esa forma de extraer los requisitos y necesidades de un producto software es la primera etapa para crearlo. Ya que los clientes creen que ellos saben lo que el software tiene que hacer, se requiere habilidad y experiencia en la ingeniería del software para identificar requisitos incompletos, ambiguos o contradictorios. El resultado del análisis funcional de los requisitos con el cliente se plasma en el documento Especificación de Requisitos. De igual manera, se expresa un diagrama de “Entidad/Relación”, en el que se moldean las principales entidades que participarán en el desarrollo de software. Este paso como comentaba es vital y muy importante.

La toma, análisis y enumeración de requisitos sobre todo pruebas de ellos, es una parte vital, de esta fase depende en gran medida el conseguir los objetivos finales. Se han ideado modelos y diversos procesos de trabajo para estos fines. Aunque aún no está formalizada, se habla de la Ingeniería de Requisitos.

La IEEE Std. 830–1998 normaliza la creación de las especificaciones de requisitos software.

### Especificación

Es el trabajo de codificar con pelos y señales el software a ser desplegado, en una representación matemáticamente rigurosa. En el contexto, la mayoría de las buenas especificaciones han sido escritas para entender y afinar aplicaciones que ya estaban desarrolladas. Las especificaciones son más necesarias para las interfaces externas, que deben persistir invariables.

### Diseño y arquitectura

Se describe en como establecer y cómo funcionará el software de forma general sin ingresar en pormenores. Radican en agregar consideraciones de la implementación tecnológica, como el hardware, la red, etc. Se concretan los argumentos de uso para cubrir las funciones que realizará el sistema, y se transformarán las entidades definidas en el análisis de requisitos en clases de diseño, obteniendo un modelo cercano a la programación orientada a objetos.

### Programación

Convertir, formar, reducir un esquema a código alcanza ser la parte más obvia del encargo de ingeniería del software, pero no forzosamente es la que solicita mayor trabajo ni la más complicada. La complicación y la duración de esta fase está profundamente relacionada al o a los lenguajes de programación utilizados, así como al diseño primeramente realizado.

### Prueba

Consiste en acreditar en este período que el software ejecute correctamente las labores indicadas en la declaración del problema. Una pericia de prueba es ensayar por separado cada módulo del software y luego probarlo de forma integral, para así llegar al objetivo. Se considera una buena práctica que las pruebas sean efectuadas por alguien distinto al desarrollador que la programó. Y se introduzcan todo tipo de datos extraños y absurdos para observar resultados y consecuencias de ello.

### Mantenimiento

Conservar y optimizar el software para solucionar faltas descubiertos y tratar con nuevas exigencias. El mantenimiento puede ser de cuatro tipos: perfectivo (regenerar la calidad interna de los sistemas), evolutivo (inscripciones, modificaciones y eliminaciones inevitables en un producto software para cubrir la expansión o cambio en las necesidades del usuario), adaptativo (modificaciones que afectan a los entornos en los que el sistema opera, por ejemplo, cambios de configuración del hardware, software de base, gestores de base de datos, comunicaciones) y correctivo (corrección de errores).

El proceso de ingeniería conlleva un conjunto de fundamentos que deberían seguirse constantemente. Incluyen actividades explícitas para el entendimiento del problema y la comunicación con el cliente, métodos establecidos para representar un diseño, mejores prácticas para la implementación de la solución y estrategias con tácticas robustas para las pruebas. Al implementar los principios básicos, esto resulta en productos de alta calidad.




Para obtener el objetivo de construir productos de alta calidad dentro de la programación, la ingeniería del software emplea una serie de prácticas para:

- Entender el problema.
- Diseñar una solución.
- Implementar la solución correctamente.
- Probar la solución.
- Gestionar las actividades anteriores para conseguir alta calidad.

Importante

La ingeniería del software simboliza un proceso formal que agrega una serie de métodos bien concretos para el análisis, diseño, implementación y pruebas del software y sistemas. Además, incluye una amplia colección de métodos y técnicas de gestión de proyectos para el aseguramiento de la calidad y la gestión de la configuración del software.

## Principios de la ingeniería del software

Entre los principios más importantes de la ingeniería del software, conseguimos señalar los siguientes:

– Primero hazlo correcto, luego hazlo rápido.

– El personal es la clave del éxito.

– El deber del cliente y como se vuelca es el factor más crítico en la calidad del software.

– Entregar productos al usuario lo más pronto posible.

– Haz de la calidad la razón de trabajar.

– Una buena gestión es más importante que una buena tecnología.

– Las personas y el tiempo no son intercambiables.

– Seleccionar el modelo de ciclo de vida adecuado.

– Determinar y acotar el problema antes de escribir los requisitos.

– Realizar un diseño.

– Minimizar la distancia intelectual.

– Documentar.

– Las técnicas son anteriores a las herramientas.

– Probar, probar y probar.

– Introducir las mejoras y modificaciones con cuidado.

– Asunción de responsabilidades.

– La entropía del software es creciente.

– Nunca dejes que tu jefe o cliente te convenza para hacer un mal trabajo.

– El personal necesita sentir que su trabajo es apreciado.

– La educación continua es responsabilidad de cada miembro del equipo.

- Tu mayor desafío es compartir la visión del producto final con el cliente.
- La mejora continua de tu proceso de desarrollo de software es posible y esencial.
- Tener procedimientos escritos de desarrollo de software puede ayudar a crear una cultura compartida de buenas prácticas.
- La calidad es trascendental como objetivo; la productividad a largo plazo es una consecuencia de una alta calidad.
- Haz que los errores los encuentre un colaborador, no un cliente.
- Una clave en la calidad en el tratamiento de software es efectuar iteraciones en todas las fases del proceso exceptuado en la codificación.
- La gestión de errores y cambios es esencial para controlar la calidad y el mantenimiento.
- Si mides lo que haces aprenderás a hacerlo mejor.
- Haz lo que tenga sentido, no recurras a los dogmas.
- No puedes cambiar todo de una vez. Localiza los cambios.

### Ciclos de vida

Las fases por las que tiene que pasar obligatoriamente el software que se ha desarrollado desde que comienza la idea hasta que es retirado del mercado, ese conjunto de fases se denomina Ciclos de Vida. También se denomina a veces paradigma.

Entre los requerimientos que debe poseer un ciclo de vida se deben destacar:

- Establecer el orden de las fases del proceso de software.
- Fijar los criterios de conversión para pasar de una fase a la siguiente.
- Fijar las entradas y salidas de cada fase.
- Detallar los estados por los que circula el producto.
- Describir las actividades a realizar para transformar el producto.
- Especificar un esquema que sirve como base para planificarlo, organizarlo, coordinarlo, desarrollarlo…




Un ciclo de vida para un plan se compone de fases periódicas compuestas por labores que se pueden planear.

Según el modelo de ciclo de vida, la sucesión de fases consigue ampliarse con bucles de realimentación, de forma que lo que conceptualmente se imagina en una misma fase se pueda implantar más de una vez a lo largo de un proyecto, recogiendo en cada pasada de ejecución contribuciones a los resultados intermedios que se van originando (realimentación).

Fases: se designa al grupo de actividades que tienen relación con un objetivo común en el desarrollo de un proyecto. Se puntualiza creando labores (actividades elementales) que consiguen compartir un tramo determinado del tiempo de vida de un proyecto. La agrupación temporal de trabajos impone exigencias temporales ajustadas a la asignación de recursos (humanos, financieros o materiales).

Entregables: se nombra así a los productos intermedios que componen las fases. Pueden ser materiales o inmateriales (documentos, software). Los entregables permiten evaluar la marcha del proyecto mediante comprobaciones de su adecuación o no a los requisitos funcionales y de condiciones de realización previamente establecidos.

## Tipos de modelo de ciclo de vida

Las principales discrepancias entre diferentes modelos de ciclo de vida residen en:

– El alcance del ciclo dependiendo de hasta dónde llegue el proyecto correspondiente. Un proyecto logra alcanzar un mero estudio de aptitud del progreso de un producto, o su desarrollo completo, toda la historia del producto con su desarrollo, producción y modificaciones posteriores hasta su retirada del mercado.

– Las particularidades y contenidos de las fases en que fraccionan el ciclo. Esto puede estribar por el propio fondo al que se refiere el proyecto, o de la organización.

– La estructura y la sucesión de las etapas, si hay realimentación entre ellas, y si poseemos libertad de repetirlas e iterar.

## Modelos de ciclos de vida

En este punto concreto hay que detallar que concurren diversos modelos y que la rama de la ingeniería se vale de una serie de diseños que constituyen y muestran las distintas etapas y fases por los que pasa un producto software, desde su concepción inicial, pasando por su desarrollo, puesta en marcha y ulterior mantenimiento, hasta la retirada del producto. A estos modelos se les denomina "Modelos de ciclo de vida del software". El primer modelo concebido fue el de Royce, más comúnmente conocido como Cascada o "Lineal Secuencial". Este modelo implanta que las diversas actividades que se van realizando al desarrollar un producto software, se suceden de forma lineal.

Los modelos de ciclo de vida del software describen las fases del ciclo de software y el orden en que se ejecutan las fases.




Un modelo de ciclo de vida de software es una panorámica de las acciones que suceden durante el desarrollo de software, consigue establecer el orden de las etapas involucradas y las razones de transición asociados entre estas etapas.

Un modelo de ciclo de vida del software:

- Define las fases fundamentales de desarrollo de software.
- Describe las fases primarias esperadas de ser ejecutadas durante esas fases.
- Apoya al administrar el progreso del desarrollo.
- Proporciona un lugar de trabajo para la enunciación de un proceso especificado de desarrollo de software.

  Es muy trascendental concretar objetivos inicialmente en cada una de las etapas del modelo de ciclo de vida, igualmente se pueden constituir una serie de tareas y actividades que lo caracterizan. Existen diversos modelos de ciclo de vida, y elegir bien cada modelo para cada tipo de proyecto es importante y el orden que se implementa. Existen varias alternativas de modelos de ciclo de vida. A continuación se muestran algunos de los modelos tradicionales y más utilizados.

## Modelo en cascada

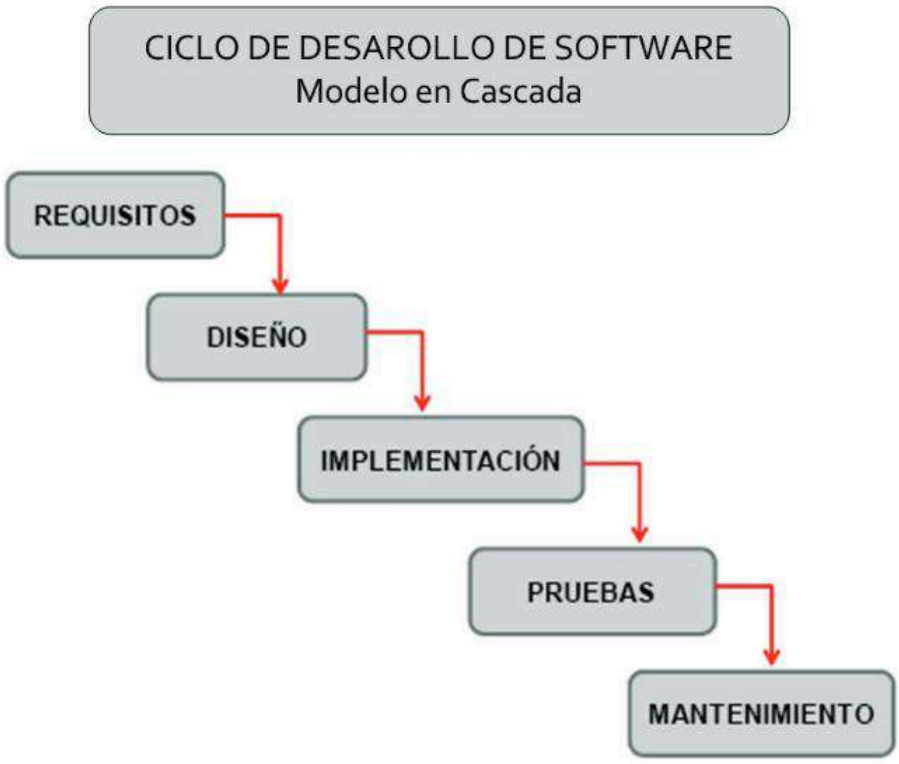
Es la perspectiva metodológica que ordena rigurosamente las etapas del ciclo de vida del software, de forma que el inicio de cada etapa debe esperar a la finalización de la inmediatamente anterior.

El modelo en cascada es un proceso de desarrollo secuencial, en el que el desarrollo se ve fluyendo hacia abajo (como una cascada) sobre las fases que componen el ciclo de vida.

Se cree que el modelo en cascada fue el primer modelo de proceso introducido y seguido ampliamente en la ingeniería el software. La innovación estuvo en la primera vez que la ingeniería del software fue dividida en fases separadas.

La primera definición seria del modelo en cascada se cree que fue en un artículo lanzado en 1970 por Winston W. Royce, aunque Royce no usó el término cascada en este artículo. Irónicamente, Royce estaba exhibiendo este modelo como un ejemplo de modelo que no funcionaba, incompleto.

En el modelo original de Royce, existían las siguientes fases:




*Esquema de Ciclo de desarrollo de Software. Modelo en Cascada*

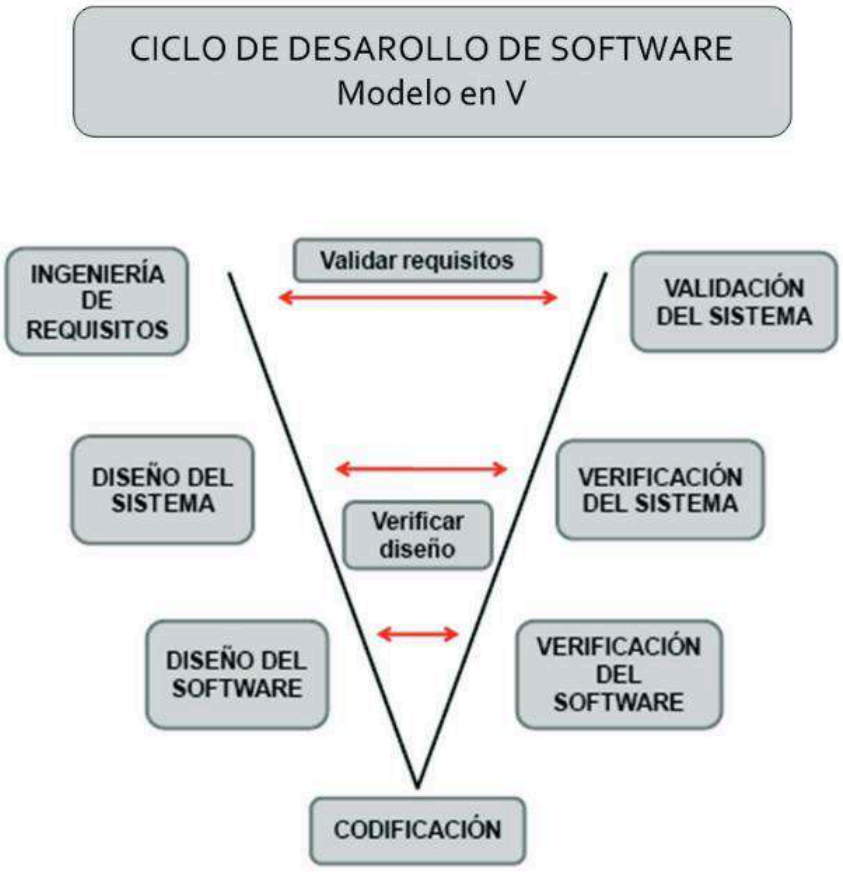
- Especificación de requisitos
- Diseño
- Construcción (Implementación o codificación)
- Integración
- Pruebas
- Instalación
- Mantenimiento

Para continuar el modelo en cascada, se progresa de una fase a la siguiente en una forma puramente secuencial.

### Modelo en V

Para finalizar y concluir con diferentes errores y problemas que producían el uso del modelo standart en cascada, el modelo en V se desarrolló para mejorar el anterior y corregir los defectos que estaban siendo encontrados demasiado tarde en el ciclo de vida, ya que las pruebas no se introducían hasta el final del proyecto. El modelo en v dice que las pruebas necesitan empezarse lo más pronto posible en el ciclo de vida. También muestra que las pruebas no son sólo una actividad basada en la ejecución. Hay una variedad de actividades que se han de realizar antes del fin de la fase de codificación. Estas actividades deberían ser llevadas a cabo en paralelo con las actividades de desarrollo, y los técnicos de pruebas necesitan trabajar con los desarrolladores y analistas de negocio de tal forma que puedan realizar estas actividades y tareas y producir una serie de entregables de pruebas. Los productos de trabajo generados por los desarrolladores y analistas de negocio durante el desarrollo son las bases de las pruebas en uno o más niveles. El modelo en v es un modelo que ilustra cómo las actividades de prueba (verificación y validación) se pueden integrar en cada fase del ciclo de vida. Internamente en el modelo en V, las pruebas de validación tienen lugar fundamentalmente durante las etapas prematuras, por ejemplo, examinando los requisitos de usuario y después por ejemplo, durante las pruebas de beneplácito de usuario.

El modelo en V representa la sucesión de caminos en el desarrollo del ciclo de vida de un proyecto. Detalla las actividades y resultados que han de ser elaborados durante el desarrollo del producto. La parte izquierda de la V representa la descomposición de los requisitos y la creación de las especificaciones del sistema. El lado derecho de la V representa la integración de partes y su verificación. V significa “Validación y Verificación”.



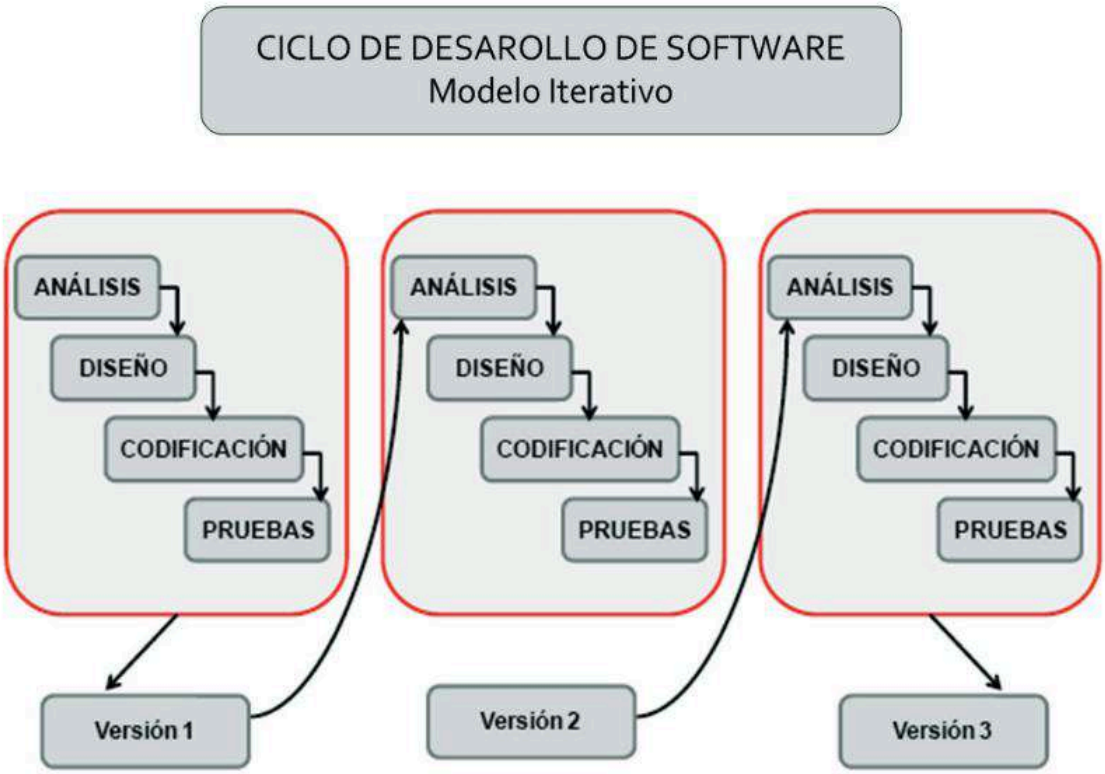

*Esquema de Ciclo de desarrollo de Software. Modelo en V*

## Modelo iterativo

Este modelo nace como derivación al modelo en cascada. Buscaba al implementarse disminuir el riesgo que existía entre las necesidades del usuario y el producto final por malas interpretaciones durante la etapa de recogida de requisitos.

Es una ampliación de diferentes ciclos en cascada unidos en una iteración. Al final de cada iteración se le entrega al cliente una versión mejorada o con mayores funcionalidades del producto. El cliente es quien después de cada iteración evalúa el producto y lo corrige o propone mejoras. Estas iteraciones se repetirán hasta obtener un producto que satisfaga las necesidades del cliente.

Este modelo se utiliza en proyectos en los que los requerimientos no están definidos y están algo oscuros por parte del usuario, por lo que se hace vital la formación de distintos prototipos para exhibirlos y conseguir la conformidad del cliente.




*Esquema de Ciclo de desarrollo de Software. Modelo Iterativo*

De las importantes avances que aporta este modelo es no necesitar de describir los requisitos en el inicio, y que paulatinamente se irán transcribiendo en las iteraciones.

Igual que otros modelos similares tiene las ventajas propias de realizar el desarrollo en pequeños ciclos, lo que permite gestionar mejor los riesgos, gestionar mejor las entregas.

### Modelo en espiral

Gracias a Barry Boehm que definió este modelo en los 85, se usa muchísimo y de manera general dentro del mundo del sector del desarrollo de software. Las actividades de este modelo se conforman en una espiral, cada bucle representa un conjunto de actividades. Las actividades no están fijadas a priori, sino que las siguientes se eligen en función del análisis de riesgos, comenzando por el bucle anterior.

Barry Boehm, escritor de varios artículos de ingeniería del software; modelos de cálculo de esfuerzos y tiempo que se gasta en hacer productos software, y modelos de ciclo de vida, inventó y promulgó un modelo desde una óptica distinta a la tradicional en Cascada: el Modelo Evolutivo Espiral.




Este modelo de ciclo de vida en espiral contempla abiertamente el riesgo que aparece a la hora de desarrollar software. Para ello, se comienza mirando las posibles alternativas de desarrollo, se opta por la de riesgos más asumibles y se hace un ciclo de la espiral.

Si el cliente quiere seguir haciendo mejoras en el software, se vuelven a evaluar las nuevas alternativas y riesgos y se realiza otra vuelta de la espiral, así hasta que llegue un momento en el que el producto software desarrollado sea aceptado y no necesite seguir mejorándose con otro nuevo ciclo.

Este modelo de ejecución asimila y une las características del modelo de prototipos y el modelo en cascada. El modelo en espiral está pensado para proyectos largos, caros y complicados.

Esto modelo no fue el primero en tratar el desarrollo iterativo, pero fue el primer modelo en explicar las iteraciones.

Este tipo fue planteado por Boehm en 1988 en su artículo “A Spiral Model of Software Development and Enhancement”. Radicalmente consiste en una serie de ciclos que se repiten en forma de espiral, comenzando desde el centro. Se suele explicar como que dentro de cada ciclo de la espiral se sigue un modelo en cascada, pero no forzosamente ha de ser así.

Este sistema es muy utilizado en proyectos grandes y complejos como puede ser, por ejemplo, software de gestión de multinacionales o la creación de un sistema operativo.

Al ser un modelo de ciclo de vida enfocado a la gestión de riesgos se dice que uno de los aspectos primordiales de su éxito radica en que el equipo que lo aplique tenga la necesaria experiencia y habilidad para detectar y catalogar correctamente riesgos.




*Esquema de Ciclo de desarrollo de Software. Modelo en espiral*

Para cada ciclo habrá cuatro actividades:

– Determinar o fijar objetivos.

  Establecer igualmente los productos definidos a obtener. Requerimientos, especificación, manual de ayuda. Fijar las limitaciones. Identificación de riesgos del proyecto y posibles estrategias alternativas para salvarlos. Hay una fase que solo se implementa una vez como es la planificación inicial o previa.

– Análisis del riesgo.

  Se analizan los riesgos que puedan darse y se eligen una o varias vías propuestas para disminuir o minimizar los riesgos.

– Desarrollar, verificar y testear.

  Actividades de la fase propia y de prueba.

  Análisis de otros caminos e identificación de resolución de riesgos.

  Dependiendo del resultado de la evaluación de riesgos, se elige un modelo para el desarrollo, que puede ser cualquiera de los otros existentes, como formal, evolutivo, cascada, etc. Así, por ejemplo, si los riesgos de la interfaz de usuario son dominantes, un modelo de desarrollo apropiado podría ser la construcción de prototipos evolutivos.

– Planificar.

  Comprobamos todo lo que hemos hecho, evaluando y con ello decidir si continuamos con las fases siguientes y planeamos la próxima actividad. El proceso comienza en la posición central. Desde esa posición se circula en el sentido de las agujas del reloj.

Fases en que se descompone el proceso de desarrollo OO:

– Planificación y Especificación de Requisitos: Organización, enunciación de exigencias, conocer los procesos del dominio, etc.

– Construcción: La construcción del sistema. Se subdivide en las siguientes:

  · Análisis: Se investiga el problema a solventar desde la perspectiva de los usuarios y de las entidades externas que van a requerir servicios al sistema.

  · Diseño: El sistema se especifica en detalle, describiendo cómo va a funcionar internamente para satisfacer lo especificado en el análisis.

  · Implementación: Se lleva lo especificado en el diseño a un lenguaje de programación.

  · Pruebas: Se llevan a cabo una serie de pruebas para corroborar que el software funciona correctamente y que satisface lo especificado en la etapa de Planificación y Especificación de Requisitos.

– Instalación: La puesta en marcha del sistema en el entorno previsto de uso.

## Análisis y diseño estructurado

Desde 0 es muy complicado asimilar en principio de manera completa algunos sistemas grandes y de gran complejidad. Se dispone de mecanismos de implementación del análisis para poder superar dicho problema, reduciendo y llegando a una simplificación de cada componente, y con el desarrollo de un modelo que capte esa visión de módulos, añadiendo cada vez más elementos de diseño como de análisis.

## Análisis estructurado

Un buen análisis estructurado busca que se defina transparentemente lo que busca que haga el sistema y sus necesidades de aplicación. No se puede tener en cuenta en este paso como se implantará el aplicativo ni todavía requisitos que deben cumplirse. Si en cambio necesita que las personas observen los elementos lógicos (lo que hará el sistema) separado de los componentes físicos (computadoras, terminales, sistemas de almacenamiento, etc.). Después de esto se puede desarrollar un diseño físico eficiente para la situación donde será utilizado.

## Elementos del análisis estructurado

Descripción Grafica: Maneja símbolos o iconos para establecer un modelo grafico del sistema. Sin meter procesos manuales o informatizados, archivos, entre otros.

Diagramas de Flujo de Datos: Poseen la misión de exponer las fuentes y destinos de los datos, Identificar y suministrar nombre a los procesos, Dar nombre a los grupos de datos que corresponden una función con otra, Señalar los almacenes de datos a los que se tiene acceso.

Diccionario de Datos: Se concretan con flujo de datos, procesos y almacenes de datos.

## Diseño estructurado

Este paso de diseño estructurado, se apoya en descripciones gráficas para enfocar en la implementación de la especificidad del software. Es otro componente cuya meta es definir programas compuestos modularmente de manera independiente desde la óptica operativa y de funcionamiento. Este enfoque no sólo conduce hacia mejores programas sino que facilita el mantenimiento de los mismos cuando surja la necesidad de hacerlo.

El diseño estructurado es un proceso específico para el diseño de programas y no un método de diseño de compresión. Es decir, no indica nada relacionado con el diseño de archivos o bases de datos, la presentación de entradas o salidas, la secuencia de procesamiento o el hardware que dará soporte a la aplicación. Esta técnica conduce a la especificación de módulos de programa que son funcionalmente independientes.

La herramienta importante de este tipo de diseño, es el diagrama de flujo de datos, los diagramas estructurados son de entorno gráfico y evitan referencias relacionada con el hardware o detalles físicos. Su fin no es mostrar la lógica de los programas que es la tarea de los diagramas de flujo. Los diagramas estructurados representan la interacción entre fases independientes junto con los datos que una fase pasa a otra cuando interacciona con ella. Estas relaciones funcionales para los módulos se suministran a los programadores antes que empiece la fase de escritura de código.

### Empleo del análisis y diseño estructurado con otros métodos

Se puede mezclar frecuentemente con métodos de ciclos de vida del software más clásico. Incluir diagramas de flujo por parte de un analista como manera de complementar y describir las relaciones entre elementos y durante la investigación al detalle de algún sistema existente. Asimismo, se pueden definir los archivos y datos en un diccionario centralizado de datos de acuerdo con las reglas del análisis estructurado.

### Análisis y diseño orientado a objetos

Los métodos orientados a objetos, permite que se desarrolle desde objetos para construir un software desde actitudes específicas de dichos objetos. Dichos componentes denominados objetos se pueden crear a partir de otros, que a su vez pueden estar formados por otros objetos.

El estudio de los objetos en su mundo y entorno enfocado a dichos elementos reiterativos pero no repetitivos y sencillos como son los objetos, como eventos que interactúan entre ellos y su ambiente entre los diversos elementos integradores. Su implementación del software se realiza al reutilizar modelos de clases de objetos ya existentes y, en caso necesario, al construir nuevas clases. Al modelar una empresa, los analistas deben identificar sus tipos de objetos y las operaciones que hagan que los objetos se comporten en determinada forma.

Las visión orientada a objetos se utilizan como medio encaminada de manera paradigmática para el diseño sencillo de sistemas muy complejos. el sistema se puede ver como una colección de objetos, donde cada uno de ellos puede llegar a tener varias posibilidades. Las operaciones que modifican el estado son relativamente sencillas. Los objetos se construyen a partir de otros objetos. Los sistemas se construyen a partir de otros componentes probados con un formato definido para las solicitudes de las operaciones del componente.

Los analistas que trabajan con este paradigma de orientación a objetos, observan el mundo como solo objetos (con estructuras de datos y métodos) y eventos que activan operaciones, las cuales modifican el estado de los objetos. Las operaciones aparecen como objetos que hacen solicitudes a otros objetos. El analista crea diagramas de la estructura de los objetos y de los eventos que los modifican. El modelo del diseñador es similar al modelo del analista, pero se toma con el detalle suficiente como para crear el código. El análisis y diseño enfocado a objetos intenta lograr la reutilización masiva de las clases de objetos. Modela el mundo en términos de objetos que tienen propiedades y comportamientos, y eventos que activan operaciones que modifican el estado de los objetos. Los objetos interactúan de manera formal con otros objetos.

## Programación orientada a objeto




Este paradigma de programar orientado a objetos no es una visión conceptual nueva, sus inicios y técnicas de programación se iniciaron a principios de los 70. Se puede explicar brevemente programación orientada a objetos (OOP) como una técnica o manera de programar con un lenguaje que utiliza objetos como bloques esenciales de construcción. La OOP, es un tipo de programación más cercana al razonamiento humano.

La OOP surge como una solución a la programación de grandes programas, y para resolver el mantenimiento de las aplicaciones, ya que en la programación estructura el más pequeño cambio implica la modificación de bastantes funciones relacionadas, en cambio con la OOP solo es cuestión de sumar o modificar métodos de una clase o mejor, crear una nueva clase a partir de otra (Herencia).

### Objeto

Los objetos son los elementos físicos y conceptuales que descubrimos en el mundo alrededor de nosotros. Hardware, software, documentos, seres humanos, los conceptos son todos los ejemplos de los objetos. Todo lo que nos rodea son objetos.

### Clases

Una clase en una especie de plantilla o modelo que describe como se crean ciertos tipos de Objeto. En el momento que se crea un Objeto de una Clase, se crea una instancia de esa Clase "Instancia". Una Clase es una colección de Objetos similares y un Objeto es una instancia de una Clase. Se puede explicar una Clase como un molde que se utiliza para describir uno o más Objetos del mismo tipo. La instancia es como si extrajésemos un clon de un objeto nuevo de esa clase en concreto.

### Herencia

Una particularidad muy importante de los Objetos y las Clases es la Herencia, una propiedad que permite construir nuevos Objetos "Clases" a partir de unos ya existentes. Esto permite crear "Sub–Clases" denominadas Clases Derivadas que comparten las propiedades de la Clase de la cual derivan "Clase base". Las Clases derivadas heredan código y datos de la clase base, asimismo incorporan su propio código y datos especiales. Se puede decir que la herencia permite definir nuevas Clases a partir de las Clases ya existentes.

### Polimorfismo

En un sentido exacto, Polimorfismo representa la cualidad de poseer más de una forma. En el contexto de OOP, el Polimorfismo se refiere al hecho de que una simple acción puede tener diferente comportamiento en otros objetos. En otras palabras, diferentes objetos reaccionan al similar mensaje de modo diferente. Los primeros lenguajes de OOP fueron interpretados, de forma que el Polimorfismo se contemplaba en tiempo de ejecución. Por ejemplo, en C++, al ser un lenguaje compilado, el Polimorfismo se admite tanto en tiempo de ejecución como en tiempo de compilación.

### Diferencias. Análisis y diseño estructurado vs. orientado a objeto

La metodología de análisis y diseño estructurado, visualizan los sistemas desde la óptica de las funciones o tareas que deben procesar, tareas que se van descomponiendo sucesivamente en otras tareas más pequeñas y que constituyen los bloques o módulos de las aplicaciones. En la orientación a objeto, por su parte, cobra mucho más importancia el aspecto de modelado del sistema, examinando el dominio del problema como un conjunto de objetos que interactúan entre sí.

Importante

En la metodología de análisis y diseño estructurado se produce una división entre los dos elementos de un sistema: funciones que llevan a cabo los programas y datos que se almacenan en archivos o bases de datos. Y por otro lado, la orientación al objeto da un enfoque unificador de ambos aspectos, que se unen en los objetos.

En la técnica de análisis y diseño estructurado las herramientas que utilizan para el análisis son: Esquemas de Flujos de Datos, Diccionarios de Datos, Diagramas Entidad/Relación, Diagramas de Transición de Estado, Descripciones de procesos. En las metodologías orientadas a objetos se utilizan diferentes modelos que depende de la metodología, entre los principales están Modelo de objetos, Modelo de Estado u Objeto–Estado, entre otros.

Incluso, se puede añadir otras diferencias secundarias tales como:

– Se borran límites entre fases debido a la naturaleza iterativa del desarrollo orientado al objeto.

– Aparece una nueva manera de idear los lenguajes de programación y su uso al añadirse bibliotecas de clases y otros componentes reutilizables.

– Hay un alto grado de iteración y solapamiento, lo que lleva a una forma de trabajo muy dinámica.

Es muy diferente el concepto de diseño Orientado a Objetos frente a diseño estructurado. Este último con una antigua perspectiva de programación y análisis de las aplicaciones.



Actualmente el diseño orientado a objetos representa el futuro de la programación por todas las bondades y ventajas que aportan al mundo de la programación y de cubrir las necesidades de aplicaciones informáticas en diferentes sectores y ámbitos de actuación y entornos industriales y empresariales.

## 1.2. Análisis del proceso de construcción de software: modularidad

Las grandes ventajas que encontramos a la hora de usar un proceso de programación modular, sobre todo que es más fácil resolver un problema complejo cuando se divide en partes manejables: técnica de divide y vencerás.

En un programa monolítico la cantidad de variables utilizadas y caminos que debe seguir el flujo de control hace imposible su correcta comprensión.

Se obstaculiza muchísimo la corrección de errores y el mantenimiento con posterioridad del programa.

Esta programación modular aporta un método para enfocar el uso de recursos abstractos y la programación descendente.

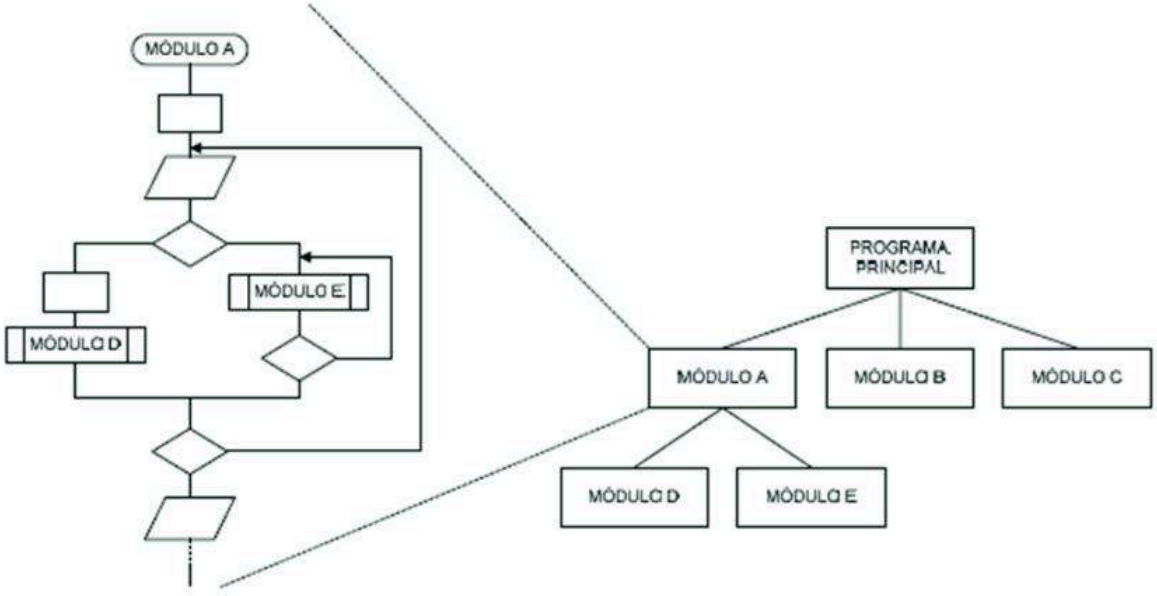
Reside en fragmentar un problema grande y complejo en partes más pequeñas: módulos, sub–algoritmos o subprogramas.

Cada módulo sería un programa normal pensado para ser integrado en una aplicación mayor.

Un programa modular estaría compuesto de:

– Un programa principal, encargado de coordinar la ejecución.

– Una serie de módulos que resolverían cada una de la tareas concretas del problema.




*Esquema de ejemplo de programación Modular*

En un programa Modular existen una serie de módulos o librerías que encajan como piezas de puzle, para resolver tareas concretas del programa. Según el lenguaje de programación existen módulos para casi cualquier tipo de procedimiento de programación que necesitemos.

### Ventajas

– Facilidad para desglosar el problema.

– División del trabajo entre un equipo de programadores. Si los módulos son independientes, cada programador del equipo de desarrollo puede encargarse de uno. El jefe del proyecto integrará los distintos módulos en la aplicación principal.

– Facilidad de mantenimiento y corrección de errores. Si cada módulo cumple una tarea completa es más fácil detectar donde se produce un error. Si se necesita realizar una mejora, sólo habrá que modificar un módulo.

– Reutilización del código. Un módulo que realice una tarea determinada podrá utilizarse en otro programa que precise de la misma tarea.

### Algunos criterios de descomposición modular

– Es vital un compromiso entre el tamaño de los módulos y la complicación de la aplicación:

  Si un programa se fragmenta en demasiadas unidades, se reduce la efectividad. Cuando el número de módulos se incrementa, decrece el esfuerzo para realizarlos, pero aumenta el esfuerzo de integración y la carga en memoria. Aunque hay muchos expertos que no están de acuerdo.

  Algunos criterios de fragmentación:

  · Descomposición por tamaño 50 líneas por módulo.

  · Complejidad del módulo, niveles de anidamiento menos de 7 niveles.

– Independencia funcional:

  Un módulo debe efectuar una única labor y comunicarse lo menos permisible con el resto de módulos.

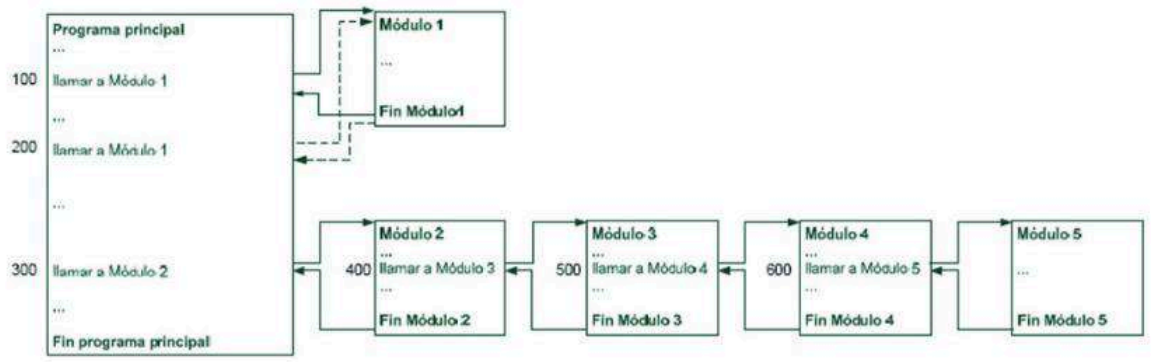
  Un módulo se debe fragmentar hasta que se obtenga un nivel mínimo tolerable de independencia funcional.

La independencia funcional se puede calcular según dos criterios:

- Cohesión. Mide la relación entre las partes internas de un módulo. Todas deben estar orientadas a realizar una única función.
- Acoplamiento. Mide la relación del módulo con el resto de los módulos. Debe comunicarse lo menos posible. Pocas veces se conseguirá un acoplamiento nulo. Un módulo debe tener mucha cohesión y poco acoplamiento.

Sabías que

En muchos casos estamos bastante en manos de las grandes compañías de software que desarrollan módulos para programación en diferentes lenguajes desde PHP, .NET, Java, etc… No tenemos otra posibilidad que implantar sus módulos y soluciones de software y con unas estrictas sentencias, librerías y parámetros. Solo desde los entornos de software libre estas librerías pueden ser programadas libremente, y sacarles mayor provecho y adaptabilidad a nuestros problemas y soluciones de programación.




*Esquema de ejemplo de llamadas a módulos en programación Modular*

Ejemplo de llamadas típicas a módulos en un programa Modular. Un programa modular contará con un programa principal y uno o varios módulos. El programa principal invoca o llama a los módulos, cediendo el control del flujo del programa.




Un programa modular contará con un programa principal y uno o varios módulos.

El programa principal llama o invoca a los módulos, cediendo a éstos el control del flujo del programa.

En el momento que se hace la invocación a un módulo se acarrea el control a la primera línea del módulo llamado.

- El programa llamador subsiste en espera hasta que finaliza el módulo llamado. En varios lenguajes no estructurados (COBOL) es posible entrar por otra línea. El módulo llamado carga en memoria su código y un espacio para sus propias variables y estructuras. Cuando el programa llamado termina, transfiere el control a la instrucción siguiente a la llamada.

- Se pierde la información concebida por el módulo sino se produce un intercambio de información entre el programa llamador y el llamado. En el momento de su ejecución el programa llamado actúa como si se tratara del programa principal.

- Puede realizar llamadas a otros módulos que funcionan de la misma forma antes expuesta.

- El posible realizar llamadas a otros módulos: llamadas anidadas.

- Es posible llamarse a si mismo (recursividad). ¿Cómo se retorna a la instrucción dónde se ha hecho la llamada?

- La dirección de la línea donde se efectúa la invocación y el estado de las variables del programa llamador se acumula en una estructura de tipo pila: la pila de llamadas (call stack). Los datos surgen de manera contraria a como han entrado. Cuando finaliza un módulo, el flujo de control se devuelve a la dirección de memoria acumulada en la parte superior de la pila, la ultima que ha entrado. Se retorna en orden inverso al orden de llamada. Cuando acaba un programa y la pila de llamadas está vacía, el control regresa el sistema operativo.




Para poder trabajar con NetBeans es necesario instalar la máquina virtual de Java y las variables del entorno necesarias.

## La máquina virtual de Java (JVM, Java Virtual Machine)

La máquina virtual de Java o entorno virtual que se maneja para descifrar los bytecodes de los binarios de Java, ya que como sabemos Java se hizo para funcionar en cualquier plataforma sin necesidad recompilar los binarios. De este modo este entorno virtual se puede obtener para nuestra arquitectura y sistema operativo sin modificaciones a nuestro programa original (esto no es cierto si utilizamos una mala dinámica de programación).

Conseguimos entonces generar un binario y este podrá Correr en Linux, Mac OSX, FreeBSD, Solaris o Windows, y para las arquitecturas disponibles en las que podamos obtener la JVM, como ser AMD64, SPARC, PIV, etc.

La máquina virtual de Java tiene la peculiaridad de ser un entorno de ejecución pensado en términos de recursos del procesador y memoria, que por medio de una administración estricta del sistema operativo estos podrían llegar a ser escasos y las aplicaciones ejecutarse de manera muy lenta. Esto no es cierto en la actualidad, existen alternativas a la JVM provista por Sun Microsystems que permiten una velocidad similar a una aplicación compilada en C oriunda en la arquitectura, un ejemplo de esto es Kaffe, que es una máquina de Java OpenSource que se compila sin mayores cambios en nuestra arquitectura disponible y funcionará mucho más rápida que la distribución estándar de JVM de Sun Microsystems y gastará muchos menos recursos.

## Kit de desarrollo y entorno de ejecución (JDK, JRE)

El Kit de desarrollo conocido como JDK (Java Development Kit) suministra un compilador, un elemento para comprimir un proyecto en un solo archivo de tipo JAR (que es compatible con ZIP) y un ambiente de ejecución para nuestros binarios.

Cuando nuestro proyecto concluido se prepara para su distribución, no es obligatorio tener el compilador y la mayoría de las herramientas que se suministran en el JDK, entonces conseguimos prescindir de dicho JDK y utilizar el entorno de ejecución que es más pequeño en cuestiones sólo de espacio en disco. Este JRE (Java Runtime Environment) también puede redistribuirse sin problemas de licencias.

En las plataformas establecidas en Linux, concurren mejores herramientas de desarrollo y por supuesto en casi todas las distribuciones de Linux, se encuentra disponible Kit de desarrollo o JDK. Así como recomendamos Kaffe como JVM para proyectos avanzados o de alto rendimiento, recomendamos Jikes como compilador de ByteCodes para Java, Jikes no es interpretado como el compilador proporcionado en el JDK de Sun Microsystems.

Comandos que inician el JDK, JRE:

– Comando Descripción. Java arranca el entorno de ejecución tomando como argumento el nombre del binario ejecutable en formato ByteCodes sin la extensión de archivo .class que identifica de manera visual un binario Java. Este comando es parte de JDK y JRE javac Inicia el compilador Java recibiendo como argumento todos los archivos de código fuente cuya terminación es .java incluida dicha extensión. Este comando no es parte de JRE.

– jar. Por medio de este comando iniciamos el empaquetador de clases y archivos de Java que nos permiten fabricar un único archivo contenedor de nuestras aplicaciones, multimedia y gráficos. Este comando es parte sólo de JDK.

## Las variables PATH y CLASSPATH en el sistema

El desarrollo y ejecución de aplicativos en lenguaje Java requiere que los instrumentos para compilar (javac.exe) y ejecutar (java.exe) se hallen fáciles. El ordenador, desde una ventana de comandos de Shell de Windows 7, sólo es capaz de ejecutar los programas que se encuentran en los directorios indicados en la variable PATH del ordenador (o en el directorio activo). Si se desea compilar o ejecutar código en Java, el directorio donde se encuentran estos programas (java.exe y javac.exe) deberá encontrarse en el PATH. Tecleando PATH en una ventana de comandos de la consola del sistema operativo, se muestran los nombres de directorios incluidos en dicha variable de entorno.

Java utiliza además una nueva variable de entorno denominada CLASSPATH, la cual determina dónde buscar tanto las clases o librerías de Java (el API de Java) como otras clases de usuario. A partir de la versión 1.6.5 del JDK no es necesario indicar esta variable, salvo que se desee añadir conjuntos de clases de usuario que no vengan con dicho JDK. La variable CLASSPATH puede incluir la ruta de directorios o ficheros *.zip o *.jar en los que se encuentren los ficheros *.class. En el caso de los ficheros *.zip hay que observar que los ficheros en él incluidos no deben estar comprimidos. En el caso de archivos *.jar existe una herramienta (jar.exe), incorporada en el JDK, que permite generar estos ficheros a partir de los archivos compilados *.class. Los ficheros *.jar son archivos comprimidos y por lo tanto ocupan menos espacio que los archivos *.class por separado o que el fichero *.zip equivalente.

Una forma general de aplicar estas dos variables es crear un fichero batch de Shell de windows (*.bat) donde se muestren los valores de dichas variables. Cada vez que se abra una ventana de Consola será necesario ejecutar este fichero *.bat para asignar adecuadamente estos valores. Un posible fichero llamado jdk175.bat, podría ser como sigue:

```
set JAVAPATH=C:\jdk1.7.5.

set PATH=.;%JAVAPATH%\bin;%PATH%

set CLASSPATH=.\;%JAVAPATH%\lib\classes.zip;%CLASSPATH%
```

Lo cual sería válido en el caso de que el JDK estuviera situado en el directorio C:\jdk1.7.5. Si no se desea tener que ejecutar este fichero cada vez que se abre una consola del sistema operativo es necesario indicar estos cambios de forma “permanente”. En Windows XP o Windows 7 es necesario modificar el fichero Autoexec.bat situado en C:\, añadiendo las líneas antes mencionadas. Una vez rearrancado el ordenador estarán presentes en cualquier consola de Shell de Windows que se cree. La modificación al fichero Autoexec.bat en Windows XP / 7 será la siguiente:

```
set JAVAPATH=C:\jdk1.7.5

set PATH=.;%JAVAPATH%\bin;%PATH%

set CLASSPATH=
```

Donde en la tercera línea debe contener la ruta de los ficheros donde existen las clases de Java. En el caso de utilizar Windows 7 se añadirá la variable PATH en el cuadro de diálogo que se abre con Start → Settings → Control Panel → System → Environment → User Variables for NombreUsuario. También es posible utilizar la opción -classpath en el momento de llamar al compilador javac.exe o al intérprete java.exe. En este caso los ficheros *.jar deben ponerse con el nombre completo en el CLASSPATH no basta poner el PATH o directorio en el que se encuentra. Por ejemplo, si se desea compilar y ejecutar el fichero ContieneMain.java, y éste necesitara la librería de clases G:\MyProject\OtherClasses.jar, además de las incluidas en el CLASSPATH, la forma de compilar y ejecutar sería:

```
javac -classpath .\;G:\MyProject\OtherClasses.jar ContieneMain.java

java -classpath .\;G:\MyProject\OtherClasses.jar ContieneMain
```

Es bueno consultar la ayuda correspondiente a la versión que se esté utilizando, debido a que concurren pequeñas variaciones entre las distintas versiones del JDK. Cuando un fichero filename.java se compila y en ese directorio existe ya un fichero filename.class, se comparan las fechas de los dos ficheros. Si el fichero filename.java es más antiguo que el filename.class no se produce un nuevo fichero filename.class. Esto sólo es válido para ficheros *.class que se corresponden con una clase public.

## Iniciar Netbeans

Antes de iniciar el estudio del lenguaje Java, vamos a familiarizarnos un poco con el entorno de desarrollo que utilizaremos a lo largo de la unidad: NetBeans 7.2.1. NetBeans es un entorno de desarrollo moderno y potente en el que se pueden desarrollar distintos tipos de aplicaciones Java.

Lo primero que nos encontramos al abrir NetBeans es la página de bienvenida o Welcome. Esta página se actualiza cada vez que se inicia el entorno de desarrollo con contenido descargado directamente desde Internet: noticias, artículos en blogs especializados, etc.




*Pantalla de ejemplo de Netbeans que es un IDE muy usado en POO*

Si queremos, podemos acceder a la información disponible en Internet sobre NetBeans y Java en general. Sin embargo, siempre se trabajará en un determinado proyecto, que podrá contener todos los archivos de código y otros recursos que necesitemos.

Según el tipo de proyecto, NetBeans generará un contenido u otro para facilitarnos su programación.

Importante

Para crear un nuevo proyecto, desplegamos el menú File (Archivo) de la ventana principal de NetBeans y elija New Project (Nuevo proyecto).

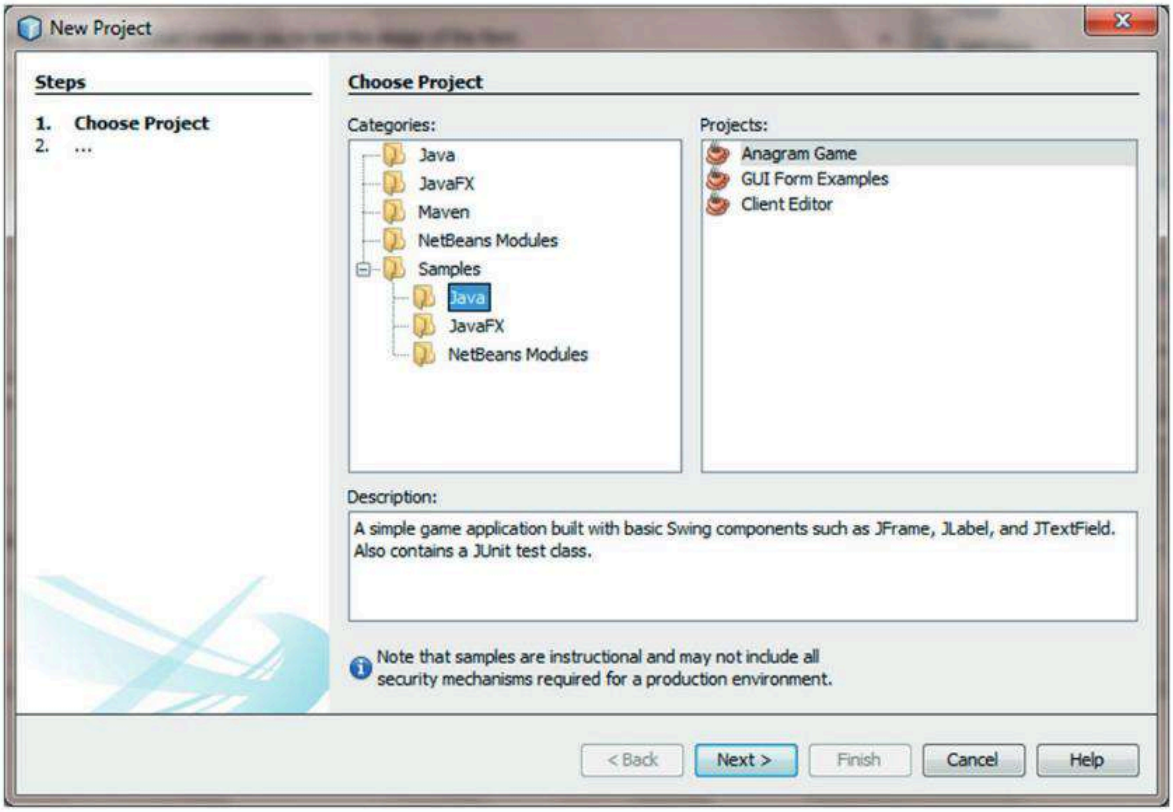
Aquí nos encontramos con los distintos proyectos disponibles en NetBeans. Cada uno de ellos tiene un determinado propósito, como puede ser una aplicación web o una aplicación Java estándar.

Nosotros trabajaremos con proyectos de Java estándar, que encontraremos en la carpeta General.

Dentro de esta carpeta o categoría volvemos a encontrar algunos tipos de proyectos. Principalmente utilizaremos dos: Java Application y Java Class Library.

El tipo de proyecto Java Application es una aplicación Java que puede ejecutarse como tal, mientras que Java Class Library sirve como recurso o biblioteca de clases que utilizan las aplicaciones Java del primer tipo. Lo iremos viendo.

Una vez elegido el tipo de proyecto (Java Application en este caso), pulsaremos en el botón Next e indicaremos el nombre del proyecto,la ubicación donde se guardará y alguna característica adicional que ahora no debe preocuparnos.

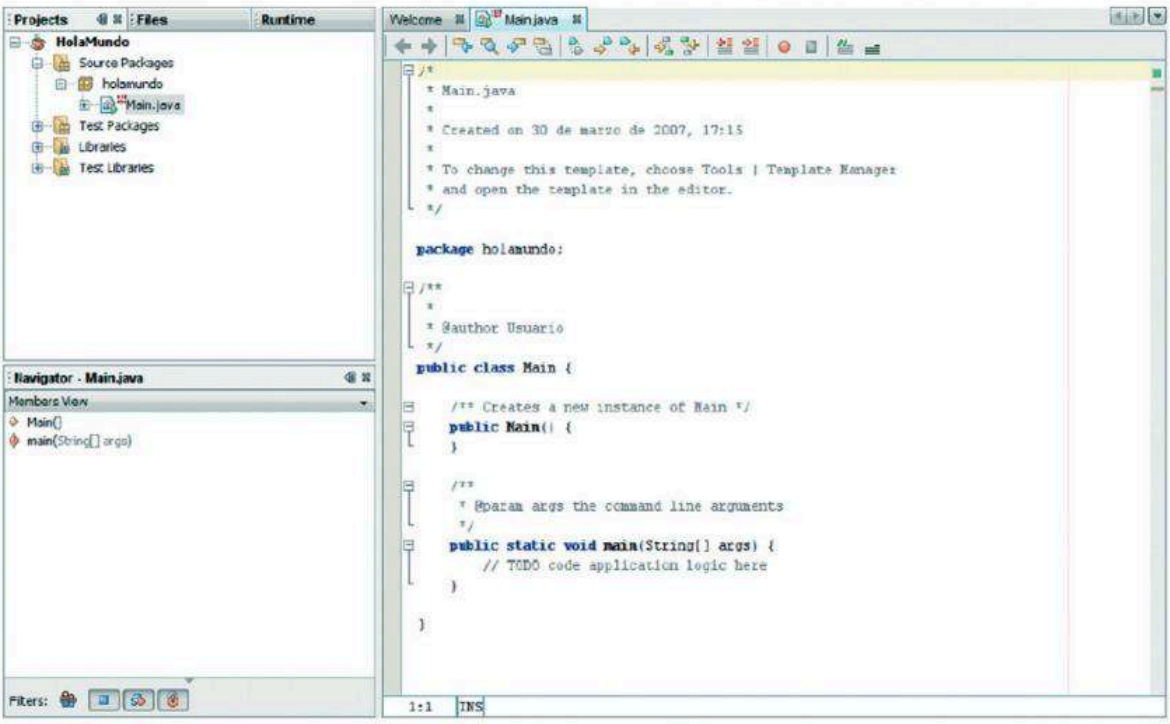



*Pantalla de ejemplo de Netbeans de Crear Nuevo Proyecto*

Desde la ventana de Nuevo Proyecto elegimos que tipo de Proyecto crearemos, si es una Java aplicación, si es una clase o cualquier otro diferente módulo.

Vamos a crear una pequeña aplicación que imprima el texto "Hola mundo" en pantalla. Por defecto, NetBeans maneja la carpeta personal del usuario para guardar los proyectos. Podemos satisfacer hacerlo aquí o en cualquier otra ubicación.

NetBeans concibe una estructura elemental para el proyecto, que en este caso consiste en un único archivo de código Java (Main.java), distintos archivos de configuración y una serie de carpetas que iremos conociendo. Además, vemos que la pantalla queda dividida en tres partes principales:




*Pantalla de ejemplo de Netbeans de código Java (main.java)*

Venta de Proyecto nuevo en NetBeans "Holamundo". NetBeans genera una estructura básica, en este caso consiste en un único archivo de código Java denominado Main.java.

La ventana de proyectos, surge compuesta por tres fichas distintas:

– Projects: muestra la estructura lógica del proyecto, es decir, cuáles son los paquetes de código fuente "Source Package", cuáles son las bibliotecas utilizadas "Libraries", etc.

– Files: donde se enseña la estructura física del proyecto, es decir la estructura de carpetas y los archivos que forman el conjunto del proyecto almacenados en la ubicación que le hemos indicado previamente.

  Por ejemplo, aquí encontraremos la carpeta src, donde se sitúan los archivos de código fuente o con la extensión .java.

– Runtime: donde configuraremos fácilmente las conexiones con servidores, como de base de datos, necesarios durante la ejecución de la aplicación.

  En la parte inferior nos muestra la ventana del navegador o Navigator, que lanza detalles sobre el elemento seleccionado en la ventana de proyectos.

  Al estar seleccionado el archivo Main.java, por lo que la ventana Navigator muestra que este archivo contiene una clase de nombre Main y un método main.

  Finalmente, en la parte derecha de la pantalla encontraremos el área de edición, es decir, donde escribiremos nuestro código Java.

La ventaja de esta área es que los diferentes archivos que vamos abriendo lo hacen en una ficha distinta, por lo que es cómodo trabajar con varios archivos a la vez.

Esencialmente hallamos en la parte superior de la ventana de NetBeans una barra de menús desplegables y una barra de herramientas con botones que proporcionan las labores más habituales en el entorno. Y, finalmente, en la parte inferior de la ventana está la barra de estado, donde irán abriéndose mensajes que nos suministrarán información adicional sobre la tarea que estamos llevando a cabo.

Se trata pues, de un entorno de desarrollo que no por ser gratuito o de "código libre" es de menores prestaciones que otros propietarios o de pago.

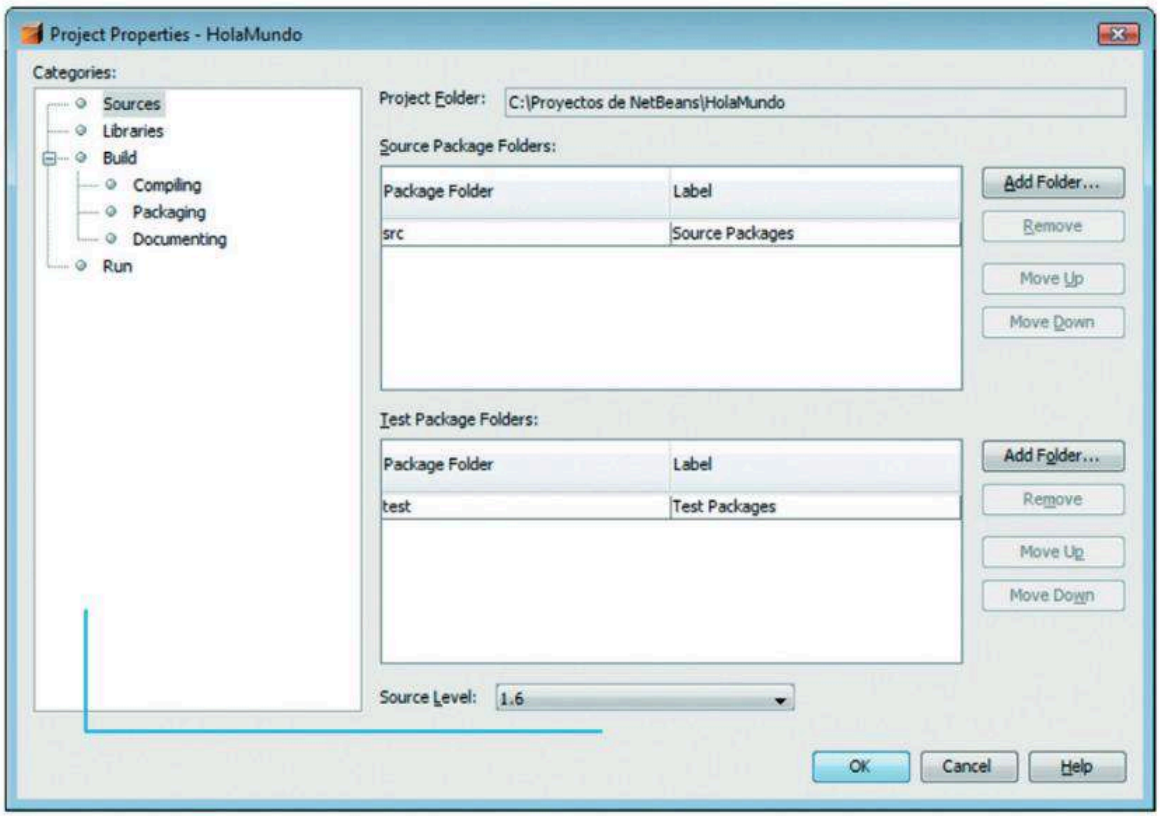
NetBeans es una buena elección, aunque es verdad que no la única, para programar en Java a nivel profesional.

## El primer proyecto con NetBeans

Es habitual encontrar el ejemplo de "Hola mundo" cuando se empieza a estudiar un lenguaje de programación y nosotros no íbamos a ser menos.

Veamos pues, cuáles serán las tareas más habituales a la hora de configurar un proyecto en NetBeans.

Lo primero ya lo hemos hecho al elegir el tipo de proyecto adecuado. En este caso, se trata de una aplicación estándar de Java.




*Pantalla de Netbeans con las propiedades del Proyecto*

En las propiedades del Proyecto debemos indicarle el nivel del código Java. Esto significa que compatibilidad tendrá nuestro código con respecto al JDK o plataforma Java instalada en el ordenador.

```
public class HolaMundo {

public static void main(String[] args) {

System.out.println("¡Hola, mundo!");

}

}
```

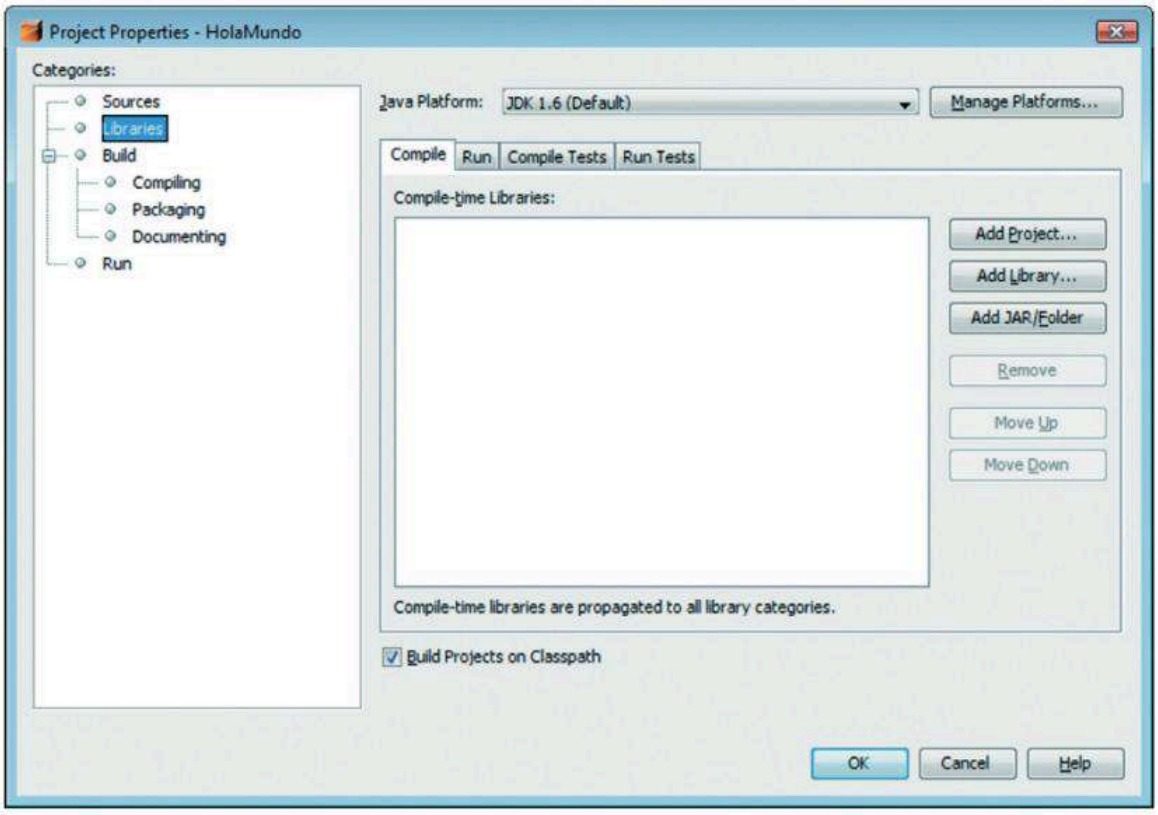
Posteriormente, debemos constituir el nivel de código Java. Esto es la compatibilidad que queremos que tenga nuestro código respecto al JDK o plataforma Java instalado en el equipo.

Para ello, hacemos clic con el botón derecho del ratón en el nombre del proyecto de la ventana Projects y elija Properties (Propiedades). En esta ventana podremos enumerar características específicas del proyecto. En la lista Source Level (Nivel del código fuente) mostraremos la compatibilidad del código del proyecto. Es decir, si queremos que sea compatible con el JDK 1.4, 1.5, 1.6, 1.7…

Es cierto que esto lo podemos instaurar en cualquier momento, pero si lo creamos desde el principio, podremos adquirir la ventaja de que NetBeans detecte, mientras trabajamos en él, problemas en nuestro código según este nivel de compatibilidad.

Distinguimos que aquí está seleccionado el nivel 1.6, que coincide con el JDK 1.6 instalado en nuestro equipo. Si es necesario, podemos indicar un nivel de compatibilidad anterior, aunque debemos entender que esto representará un requerimiento para el equipo en el que se ejecutará esta aplicación Java. Necesitará, por lo tanto, disponer del JDK 1.6 o posterior.

Podremos establecer un nivel de compatibilidad menor al de la plataforma instalada o seleccionada para el desarrollo en NetBeans, pero no mayor.




*Pantalla de Netbeans con las propiedades y el apartado Libraries*

En las propiedades del proyecto es habitual también indicar, que bibliotecas de clases se utilizan en nuestro programa y sean necesarias para nuestra correcta compilación y ejecución. Es lo que se conoce como el CLASSPATH del proyecto. Este proceso se realiza desde el apartado Libraries.

Otra tarea frecuente se realiza en el apartado Libraries, donde conseguiremos establecer qué bibliotecas de clases se manejan en nuestra aplicación y son necesarias para su correcta compilación y/o ejecución.

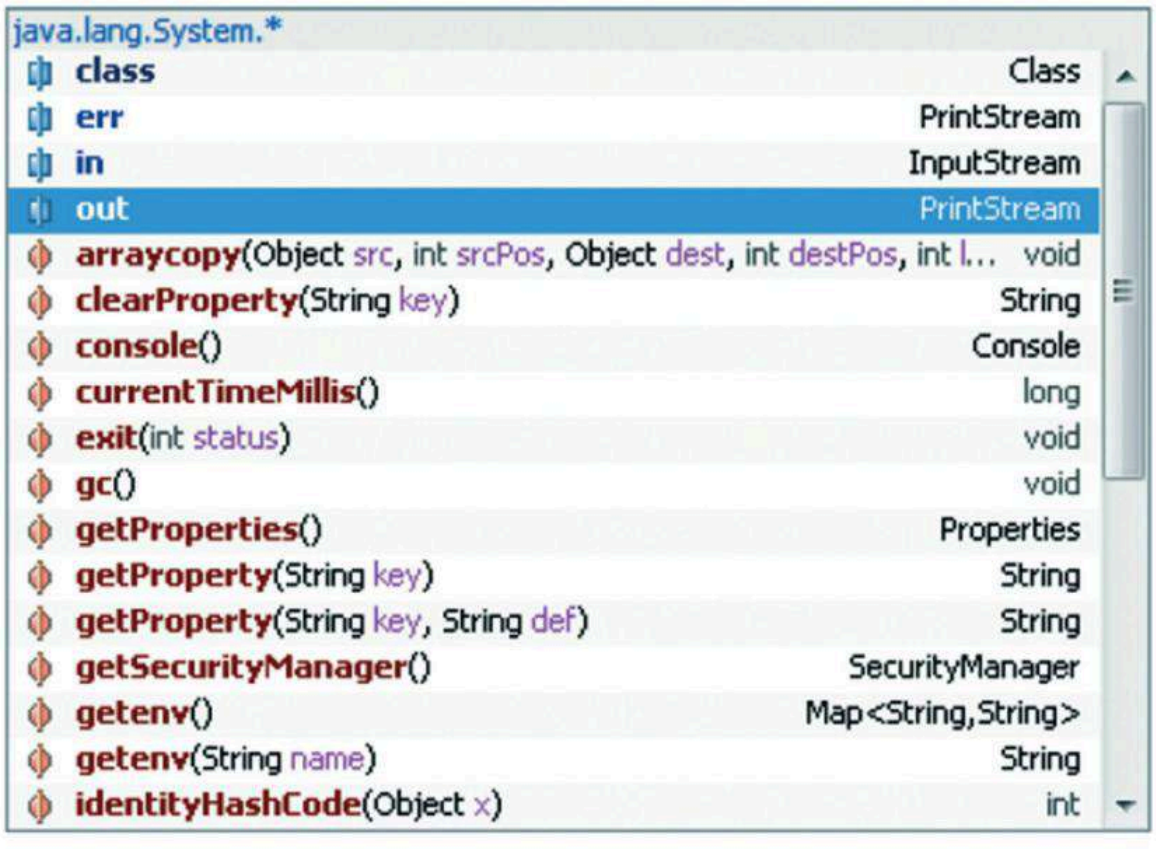



Esto es lo que se conoce como el Classpath del proyecto. Una vez especificado estos detalles, significativos para cualquier proyecto por sencillo que sea, empezamos a escribir algo de código.

Al hacerlo comprobaremos una de las características más útiles de un entorno de desarrollo como NetBeans. Se trata de la característica de completitud del código.

Es decir, que cuando NetBeans reconoce parte del código que está escribiendo, le sugiere cómo completarlo.

Si usted ve que es adecuado, no tendrá que seguir escribiendo, sino que será mucho más cómodo elegir el final de ese código entre una de las opciones sugeridas por NetBeans.




*Pantalla de Netbeans característica de autocompletar código*

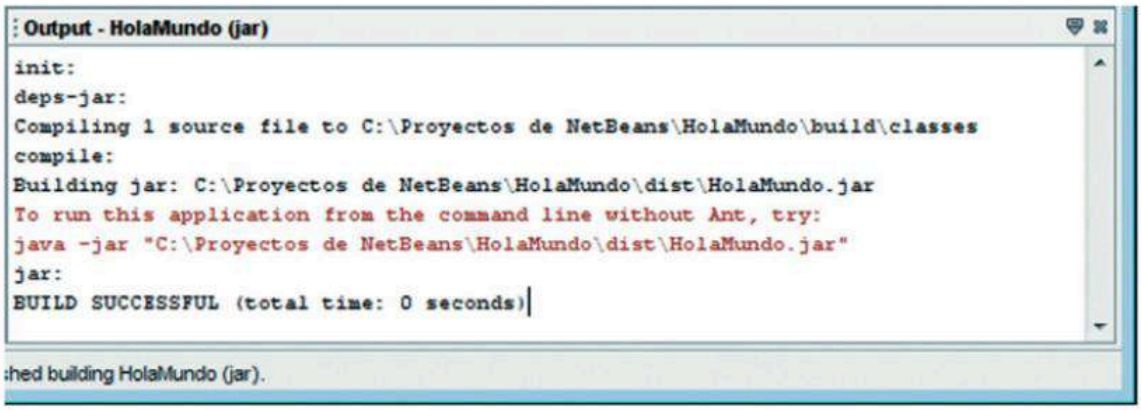
La característica de Completitud del código en NetBeans en una herramienta muy útil. Podemos elegir de una lista que nos proporciona el programa, sin necesidad de escribir toda la palabra o el código correspondiente.

Además, no sólo nos permite elegir una opción para completar el código, sino que logramos alcanzar incluso información adicional sobre nuestra elección.

Tomamos para adoptar la opción deseada de la lista que nos suministra el editor, sin necesidad de escribir el código correspondiente. Puede hacer doble clic en la entrada elegida o situarse en ella para obtener mayor información. Bueno, pues una vez completado nuestro código, es el momento de compilarlo. En NetBeans esto es muy sencillo, ya que nos oculta la utilización del comando javac del JDK de Java. Simplemente, elegimos la opción Build Main Project del menú Build.

El resultado de la compilación, proceso en el que el código del archivo .java se guarda como bytecode en un archivo .class, queda reflejado en la ventana de la parte inferior llamada Output o Salida.

Vemos que la compilación se ha efectuado correctamente (BUILD SUCCESSFUL) e incluso se nos indica cómo podríamos ejecutar la aplicación utilizando el comando java del JDK.




*Pantalla de Netbeans resultado de compilación*

Resultado de la compilación desde BUILT del Menú Built Main Project. El resultado de la compilación en el que el código Java del archivo fuente .java se convierte en bytecodes en un archivo .class. Queda reflejado en la ventana inferior de NetBeans denominada OUTPUT. Vemos que la compilación se ha producido correctamente.

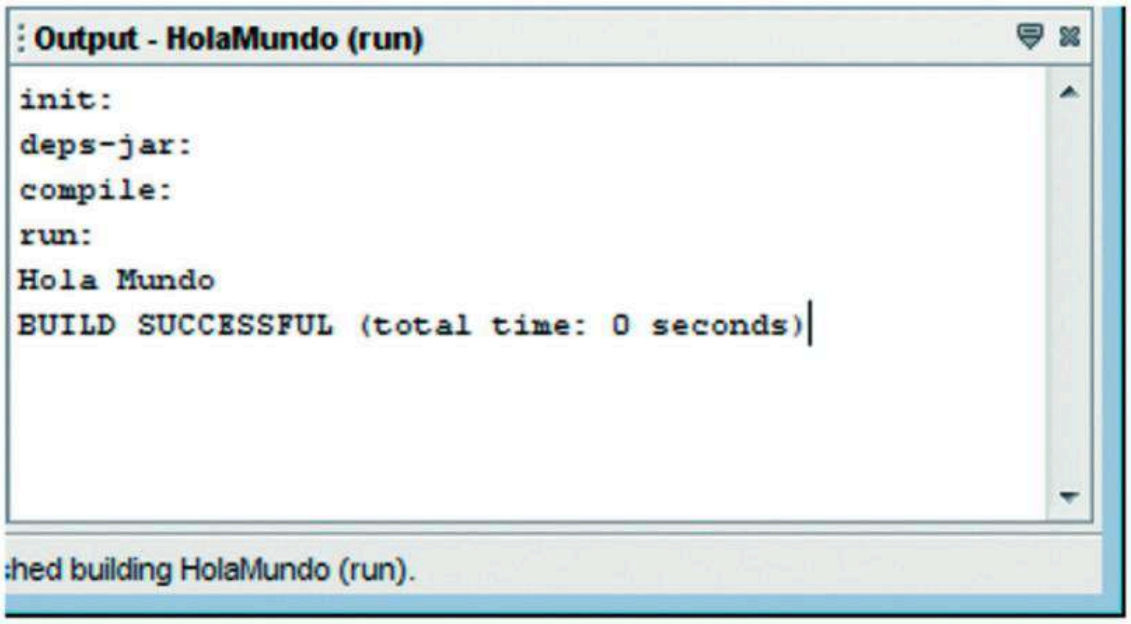
Sin embargo, no es necesario abandonar NetBeans para ello. En su lugar, conseguimos utilizar el comando Run Main Project (Ejecutar Proyecto principal) del menú Run (Ejecutar).

El resultado también se mostrará en la ventana Output.

Vemos que en la ventana Output ha aparecido, fruto de la ejecución, la cadena Hola Mundo.

Bueno, ¡pues acabas de crear, compilar y ejecutar su primer proyecto Java en NetBeans!

Aunque el resultado no es muy espectacular, ya conoce algunas de las tareas más habituales que se realizan en un entorno de desarrollo como NetBeans.




*Ejecución del fichero compilado en NetBeans*

Resultado en la misma ventana OUTPUT de la ejecución del pequeño programa. El resultado es la cadena Hola mundo.

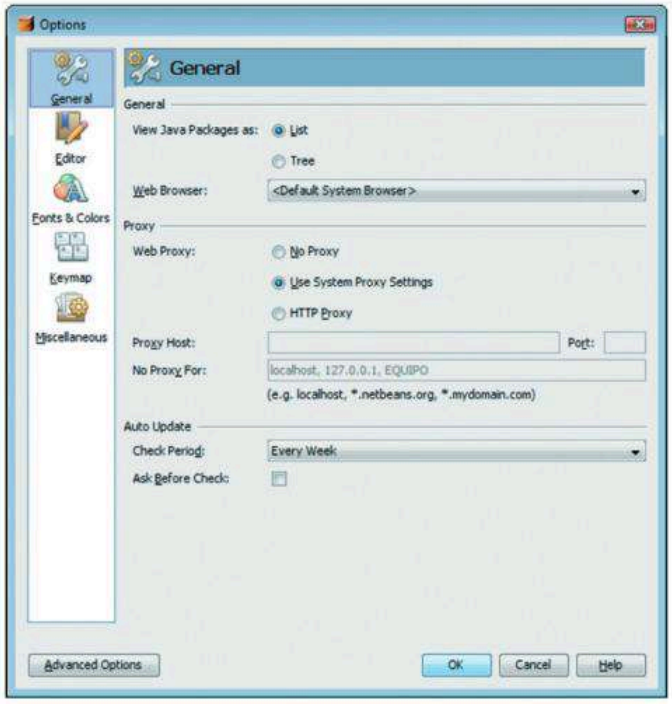
## Personalizando el editor

Podemos configurar NetBeans a nuestro antojo para prepararlo a nuestra forma personal de trabajar.

Esto también es útil en grandes proyectos en los que intervienen grupos de programadores, ya que facilita la "normalización" del código que escriben todos ellos.

Podemos personalizar características como la indentación utilizada, la fuente, el color del código, las plantillas de código que se utilizan al crear los proyectos, etc.

Todo ello a través del comando Options (Opciones) del menú Tools (Herramientas). Al hacerlo, accederá a un cuadro de diálogo como el de la figura siguiente.




*Cuadro de diálogo de configuración en NetBeans*

En este cuadro de diálogo tenemos todas las particularidades de disposición del editor. Para facilitarnos la labor, surgen clasificadas en varias categorías.

Además, disponemos de dos formas de verlas: la actual o básica y una más avanzada.

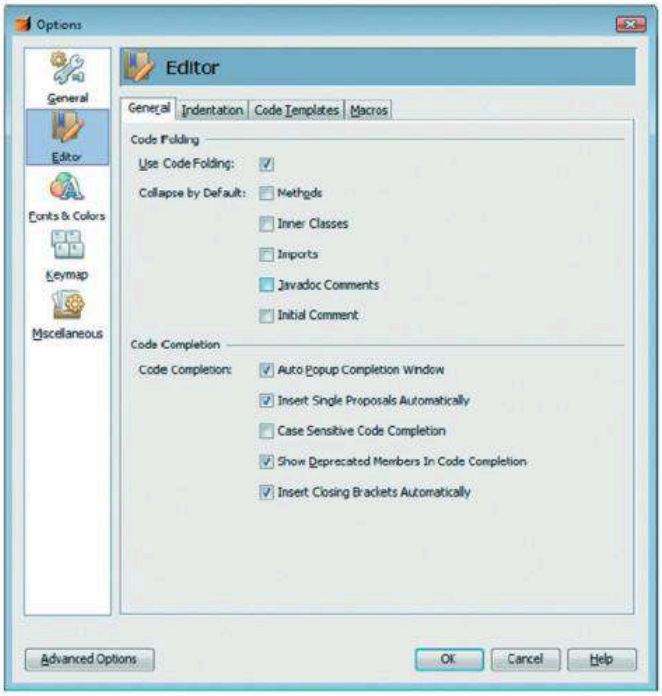
En la categoría General podemos:

– Indicar cómo queremos que se muestre el conjunto de paquetes de código fuente del proyecto: como una lista o como un árbol.

– Establecer el navegador que se utilizará por defecto, por ejemplo, al ejecutar applets o pequeñas aplicaciones Java que se insertan en las páginas web.

– Establecer cualquier ajuste para la correcta conexión con Internet si se realiza a través de un proxy.

– Establecer el período de actualización de NetBeans. Por defecto, se buscarán actualizaciones cada semana (every week).

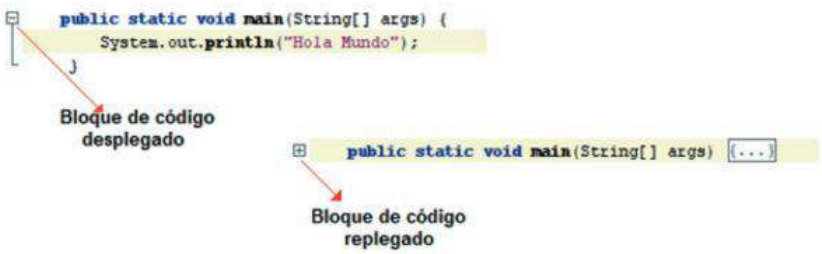



*Cuadro de diálogo de configuración en NetBeans opciones Editor*

En este cuadro de diálogo disponemos de todas las opciones de configuración del editor. Para facilitarnos la tarea viene agrupadas en varias pestañas de categorías. Podemos verlas en diferentes disposiciones, normal y avanzada.

En la categoría Editor se obtienen las propiedades que registran el aspecto y gestión del editor de código Java. Por ejemplo, si se utiliza code folding, podremos extender y replegar bloques de código para facilitar la lectura y reconocimiento de nuestros programas. Podremos indicar también qué bloques de código deben aparecer replegados inicialmente.

Fíjate que en el código fuente Java nos encontramos en su borde izquierdo líneas y cuadrados con un signo -. La idea es que pulsando en dichos cuadraditos podremos replegar esos bloques de código, así como desplegarlos después.




*Cuadro de diálogo de configuración en NetBeans opciones Code folding*

Si utilizamos code folding podremos replegar y desplegar bloques de código para inspeccionar y leer las líneas de código de nuestros programas. Podemos indicar además que bloques de código deben aparecer replegados inicialmente.

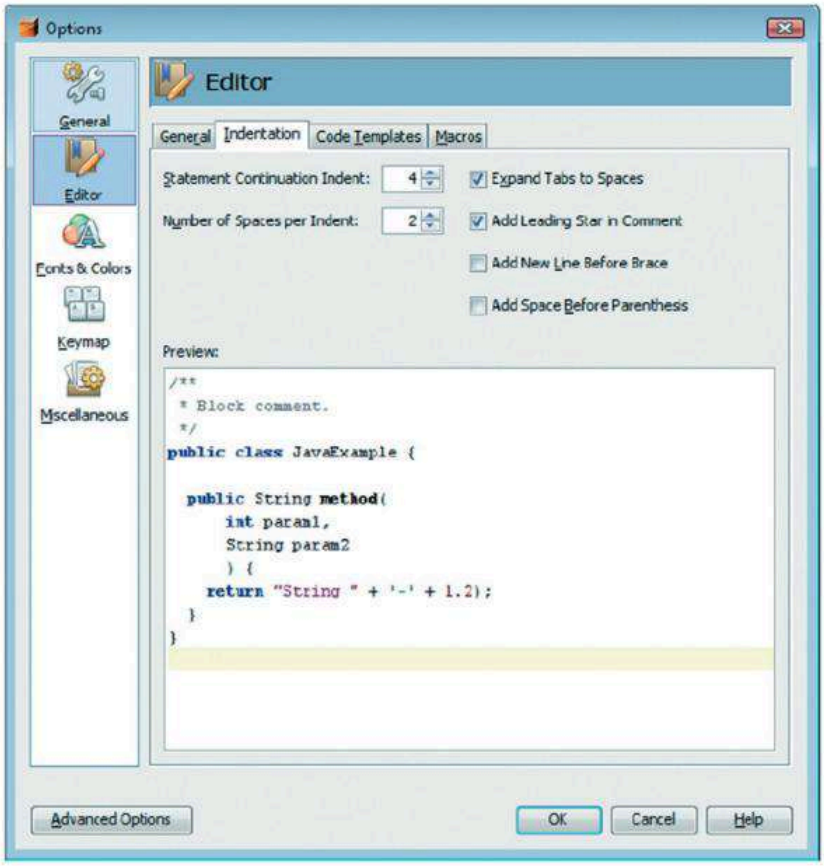
Esto puede facilitar mucho la inspección de código, al poder ocultar temporalmente, bloques de código que no nos interesan en un momento dado.

Esto quedará utilizable si activamos la característica Code Folding del editor.

En el apartado Code Completion podremos instaurar si deseamos la característica de completitud del código que ya hemos comentado anteriormente.

Normalmente es una buena idea disponer de esta característica, no sólo porque evita escribir parte de nuestro código, sino porque nos permite conocer las opciones que tenemos.

En la ficha Indentation podemos indicar cuántos espacios se utilizan para indentar el código cada vez que se pulsa la tecla. En la vista previa o preview podemos ver el resultado de ello.




*Cuadro de diálogo de configuración en NetBeans opciones Indentar*

Si utilizamos indentación podemos configurar que niveles de espacios y continuidad establecemos.

Es conveniente indentar el código para que no estén todas las líneas ajustadas al borde izquierdo de la ventana del editor, ya que esto facilita muchísimo la programación.

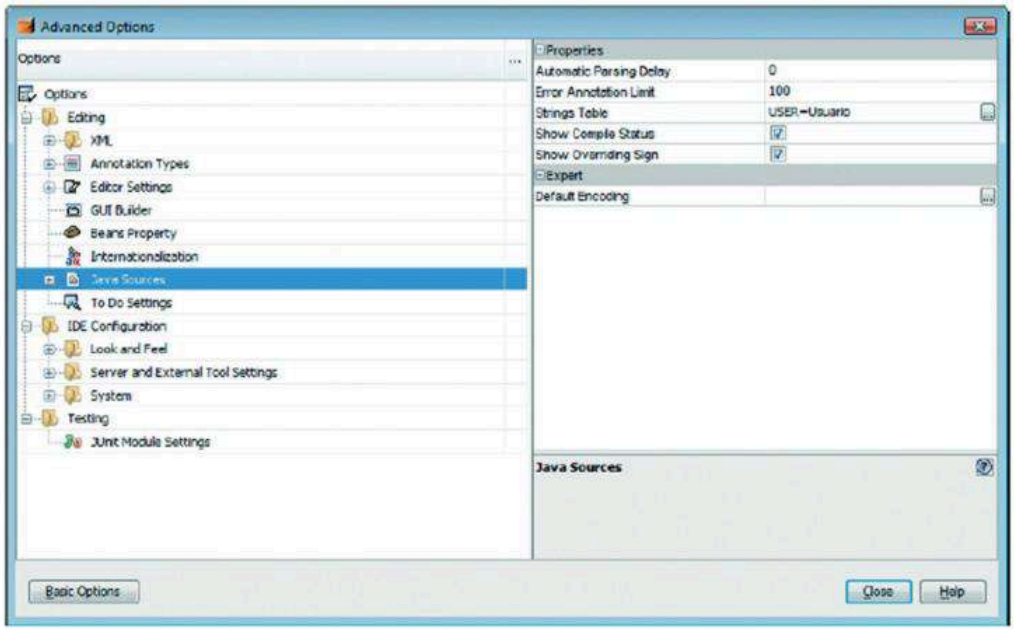
En la categoría Fonts & Colors podemos configurar el aspecto del código que escribimos. Elegiremos la fuente y color de cada tipo de código para que nos facilite posteriormente su inspección.

Podremos establecer este aspecto en puesto del lenguaje que estemos usando en nuestro proyecto. Para ello, se maneja la lista Language.

Igualmente, en la lista Profile podemos distinguir entre uno de los dos perfiles que proporciona originariamente NetBeans, aunque se consiguen crear más: el estándar o NetBeans y City Lights. En el perfil City Lights el código se escribe en colores sobre un fondo negro. Este aspecto es elegido por numerosos programadores ya que les cansa menos, pero las normas generales de ergonomía indican que es mejor escribir sobre un fondo blanco, que es la combinación de colores del perfil NetBeans o estándar.

Por consiguiente, esto es un factor muy particular que nosotros podremos elegir aquí.

Definitivamente, hay algunas casillas que sólo podremos configurar en la vista avanzada. Para ello, pulse en el botón Advanced Options. Por ejemplo, en el apartado Editor Settings podremos activar la opción Line Numbers para que se numeren las líneas de código.



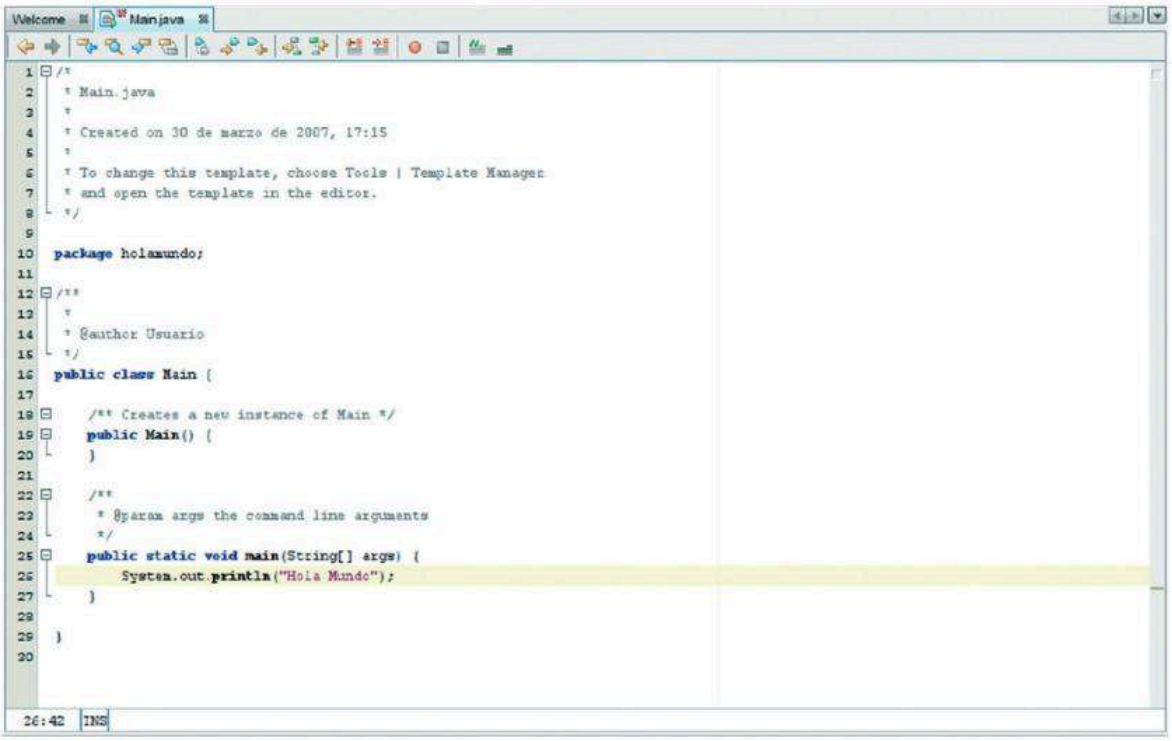

*Cuadro de diálogo de configuración en NetBeans opciones numerar líneas*

Desde estas opciones de configuración avanzada podemos incluso indicar la numeración automática de líneas en el código fuente.

Y en el apartado Java Sources podremos instaurar el valor de la propiedad Automatic Parsing Delay. Con un valor 0, los errores de sintaxis no surgen hasta que almacenamos el código.

Con estas propiedades que se han personalizado para seguir mejor el código que escribiremos a lo largo del manejo de Netbeans.

Finalmente podemos ubicar la distinta ventanas del editor para que nos permita escribir el código lo mejor posible. Para ello, utiliza el botón Minimize de las ventanas. Esto hará que no ocupen espacio en el área de edición, sino que aparezcan en la barra de la izquierda de la ventana.




*NetBeans con ejemplo de código fuente con diferentes características configuradas*

De esta manera conseguiremos mucho más espacio para escribir el código Java.

Siempre que necesitemos alguna de estas ventanas, simplemente tendremos que situar el puntero del ratón en el botón con su nombre que aparece ahora en la parte izquierda de la ventana de NetBeans.

Como puede ver, es fácil personalizar el entorno NetBeans a nuestra forma habitual de trabajar.

## Ayuda y actualizaciones

Continuamos en esta unidad conociendo el sistema de ayuda o documentación que nos facilita NetBeans, así como la posibilidad de aumentar (o disminuir) las funciones disponibles en NetBeans a través de los módulos.

Para acceder a la ayuda, despliega el menú Help.

En este menú vemos las distintas opciones de ayuda o documentación disponibles en NetBeans.

Algunas de estas opciones, como Quick Start Guide o Guía de inicio rápido, realmente son enlaces de Internet donde encontraremos dicha documentación.

Eligiendo la opción Help Contents Contenidos de la ayuda, accedemos a la ventana de ayuda principal.

El sistema de ayuda de NetBeans es muy parecido al que podremos encontrar en otras aplicaciones Windows. Queda dividido en tres fichas donde lo único que cambia es la forma de buscar la información que necesitamos en un momento dado:

– En la ficha Contents que es la ficha actual la información viene dividida en temas y subtemas que deberemos desplegar hasta encontrar la información requerida.

– En la ficha Index el contenido de la ayuda aparece ordenado alfabéticamente. Podemos utilizar el campo Buscar para realizar la búsqueda más fácilmente.

Posteriormente, la forma más expeditiva y habitual de alcanzar información es recurrir a la ficha Search, lugar donde escribiremos la palabra o frase para la que pretendemos conseguir detalles.

Indiscutiblemente, deberá escribir en inglés, ya que tanto la ayuda como el entorno están en este idioma.

En cualquier cuestión, únicamente tendrás que pulsar en el tema para conseguir la información deseada en el panel de la derecha. La ayuda disponible es igual en cualquier ficha, lo que cambia es la forma de localizarla.

Finalmente, indicarle que NetBeans es un entorno modular, es decir, que es fácil añadir (o eliminar) funciones. Para ello, disponemos de la herramienta Module Manager (Administrador de módulos) en el menú Tools. Aquí se nos muestra la lista de módulos instalados. Junto a cada módulo se nos indica la versión del mismo y si está activo o no. Por ejemplo, la versión del módulo principal de Java, es decir, del módulo J2SE Platform, es 1.6.22.2. Es decir, el JDK 6 de Java SE. Podremos activar o desactivar módulos desde aquí de forma que el editor los tenga en cuenta o los ignore. Esto no elimina el módulo.

También podremos desinstalar módulos con el botón Uninstall. Tenga en cuenta que al desinstalar un módulo, puede que ocurra lo mismo con otros dependientes del mismo. Adicionalmente, podremos buscar más módulos disponibles en Internet o versiones más actualizadas de los que ya disponemos. Para ello, utilizaremos el botón Update. Primero debemos indicar en qué lugares queremos buscar las actualizaciones. Estos lugares se conocen como Update Centers. Una vez elegido el centro o centros de actualización, pulsaremos en el botón Next y obtendremos la lista de módulos que podríamos descargar e instalar. Se nos indicará la versión disponible (Available Version) y, en su caso, la instalada actualmente en nuestro equipo (Installed Version).

## 1.3. Distinción del concepto de módulo en el paradigma orientado a objetos

### Programación modular

Desde los comienzos en que los programas se fueron creando todavía más complejos y, por lo tanto superiores, aún fue ineludible establecer un nivel más de división. Mismamente, ahora era preciso constituir todos los procedimientos y funciones de alguna forma. Los módulos agrupan, precisamente, procedimentos y funciones afines, y también los datos que manipulan, de manera que aquellos que no son empleados por el resto de módulos quedan ocultos la ellos. Modula–2 (desarrollado a finales de los 70, también por Niklaus Wirth, es el sucesor directo de Pascal) es un buen ejemplo de lenguaje de programación basado en módulos. Otro lenguaje que la soporta es Ada.




La meta de la programación modular es obtener una organización de la arquitectura del software, con un diseño no orientado a la simple obtención del programa final, sino también a su mantenimiento, a su reutilización y a poder probarlo y entenderlo de forma fácil.

Para ello, los algoritmos complejos y los programas se estructuran en módulos y éstos se estructuran en funciones y procedimientos. Al fin y al cabo, recordemos que las funciones y procedimientos, son porciones de código diseñadas para realizar tareas muy específicas.

Los módulos deben agrupar funcionalidades del software a desarrollar, de tal forma que, en el desarrollo de una aplicación de hoja de cálculo, por poner un ejemplo, todo el manejo de operaciones matemáticas resida en mismo módulo. Además todas las funciones que poseen tienen que ver con el manejo de fórmulas concretadas por el usuario, deben existir en otro módulo aparte (que seguramente necesite el módulo de matemáticas para andar), y así sucesivamente. El objetivo final sería poder reutilizar estos módulos en otras aplicaciones.

La programación modular persigue que cada módulo y función o procedimiento, cumpla las siguientes características:

– Pequeño tamaño: de esta forma el impacto de un cambio puede ser perfectamente aislado.

– Independencia modular: cuanto mayor es la independencia de los módulos y los procedimientos y funciones, más sencillo es trabajar con él. El diseño debe reducir la necesidad de compartir ficheros, datos, dispositivos y llamadas desde o hacia otros módulos y procedimientos y funciones.

– Características de caja negra: visión exclusiva de la funcionalidad de un módulo o procedimiento/función en base a sus entradas y salidas sin tener en cuenta como se realiza el proceso.

– Modelo conceptual: un diseño es más fácil de mantener si el modelo de diseño empleado se ha basado en conceptos lógicos de organización, los cuales sean más familiares y comprensibles para el personal de mantenimiento.

– Aislamiento de los detalles (encapsulación): en un sistema concurren fragmentos que muestran la filosofía y otras que reflejan los detalles. Ambas partes deben diseñarse por separado para evitar que una variación en los detalles afecte a la filosofía del sistema.

Para alcanzar los objetivos palpables se requieren estrategias de diseño. Una de las importantes se basa en la disposición de labores, es decir, descomponer un trabajo en otros más sencillos que se hayan identificando en la tarea de partida. Una vez descompuesto el software en tareas, habitualmente cada una de ellas será un procedimiento. Estos procedimientos se agruparán en módulos diferentes, conformando grupos que representen las distintas categorías basadas en el tipo de problema que resuelven conceptualmente.

La fragmentación en trabajos persigue la reducción del volumen de los procedimientos hasta alcanzar un sistema más factible de entender y modificar, menguando la duplicidad del código, así como la construcción de módulos y procedimientos o funciones útiles y reutilizables. De hecho, si se desarrolla un módulo con varios procedimientos y funciones para, por ejemplo, funciones matemáticas, este podrá ser reutilizado en aplicaciones futuras.

Para calcular la calidad de un diseño modular se utilizan las medidas de cohesión y acoplamiento que se pueden utilizar sobre módulos y procedimientos o funciones.

Cohesión: es la medida de la fuerza o relación funcional de los componentes de un módulo (entendiendo por componentes, los procedimientos o funciones que lo disponen, las definiciones de los datos y las llamadas a otros módulos). Un módulo coherente se encarga de una tarea comparativamente sencilla en un programa y demanda de poca interacción con el resto de módulos y/o procedimientos y funciones. Por ejemplo, en un módulo de matemáticas, las funciones comprendidas deben estar todas relacionadas con cálculos matemáticos, exclusivamente.

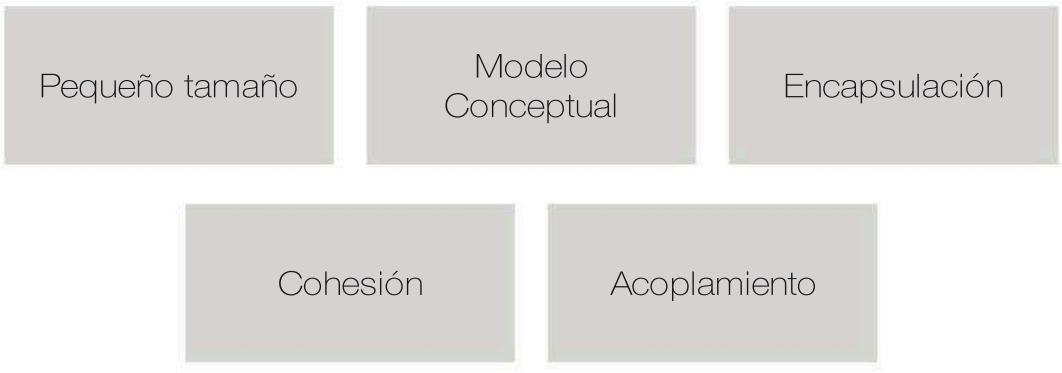
Acoplamiento: grado de interdependencia entre los distintos módulos. Por norma, dos módulos no pueden compartir estructuras de datos o variables. Por otro lado, cada módulo debe definir las funciones y procedimientos, estructuras de datos y variables que le resulten necesarias para la realización de su función específica. Así, el módulo de matemáticas ya comentado no debe necesitar en absoluto de otros módulos para trabajar. Otros módulos más complejos pueden necesitar a su vez de módulos que le sirvan para proveer de más funcionalidad, pero siempre de la manera más independiente posible.

Como ya se ha establecido, con respecto a las funciones y procedimientos, cada uno de ellos debe obedecer únicamente de los parámetros que intercambia con las cercanos procedimientos y funciones. Mejorar la calidad implica aumentar la cohesión y disminuir el acoplamiento, debido a las siguientes razones:

– En cuanto existan menor número de conexiones entre los módulos, menos oportunidades tendrá de que el efecto de un módulo sea contraproducente para otro.

– Si se quiere tener la posibilidad de cambiar un módulo por otro con igual funcionalidad con el mínimo riesgo, y que la modificación afecte lo menos posible a otros módulos por ello el enganche debe ser mínimo y la cohesión máxima.

– En el momento se está manteniendo un módulo, es deseable que no sea necesario preocuparse en los detalles internos de otros módulos.



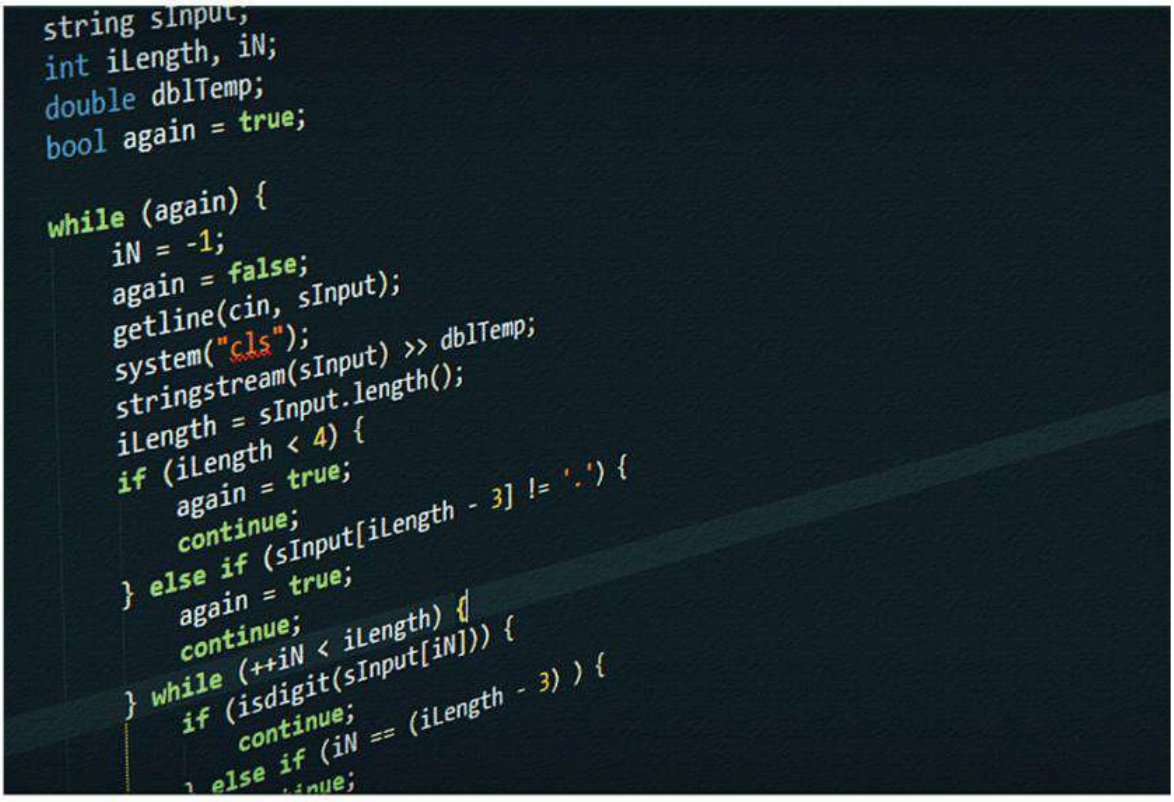
## Módulos de mayor nivel

Los lenguajes de programación actuales permiten la creación de módulos y procedimientos o funciones. Modula–2 fue sucesor de lenguaje Pascal,y fue el primero en soportar los módulos completamente empleando elementos de compilación autónomos, conocidas como Module estas encontrarían su lugar en Turbo Pascal de Borland, como Units.

Estos módulos se componen de funciones o procedimientos que permiten la realización de tareas específicas.

Se consiguen pensar que los módulos, que congregan procedimientos y funciones con características frecuentes, como de superior nivel con relación a las funciones y procedimientos, que agrupan instrucciones que cumplen algún cálculo o cuya composición soluciona un determinado problema.

Una vez identificadas las tareas necesarias para resolver un problema, estas se agrupan en módulos según criterios como los datos con los que operan, si sirven para manejar un dispositivo específico, o tienen una temática común, etc. Es decir, se agrupan por funcionalidad.



## 1.4. Identificación de objetos como abstracciones de las entidades del mundo real que se quiere modelar

Desde este momento nos centraremos en el lenguaje de programación orientado a objetos por naturaleza y el más utilizado a día de hoy que es Java independiente de la plataforma.

Aprenderemos cómo se utilizan los conceptos clave de la programación en Java y veremos que se trata de un lenguaje orientado a objetos.

Cuando tengamos cierta experiencia en el lenguaje, abordaremos el tema de la creación de aplicaciones o applets para las páginas web.

Empecemos, pues.




Sin lugar a dudas, la característica que mejor define al lenguaje Java es que es un lenguaje orientado a objetos. En un programa orientado a objetos, el sistema se divide en objetos o entidades que podemos encontrar en la realidad, lo que facilita no sólo la programación en sí sino también el diseño de la aplicación.

Veamos un ejemplo. Imagine que está realizando una aplicación que simula un coche.

Realizando una descomposición funcional del coche, en la que no se utiliza un enfoque orientado a objetos, hablaría de:

- Arrancar y apagar.
- Girar a la izquierda y a la derecha.
- Cambiar de marcha.
- Moverse hacia delante.
- Moverse hacia atrás.
- Aumentar la velocidad.
- Disminuir la velocidad.
- Etc.

Sin embargo, si piensa en objetos, podría describir el problema como:

"El coche dispone de un motor que se puede arrancar y apagar utilizando para ello el contacto. Además, el coche dispone de un volante que permite que nos desplacemos en una determinada dirección: derecha o izquierda. Al aumentar la velocidad utilizando el acelerador, debemos cambiar a una marcha más alta. Si reducimos la velocidad con el freno, puede que sea conveniente cambiar a una marcha inferior…"

Observar cómo en la descomposición funcional nos hemos fijado en lo que debería hacer el coche, mientras que en la descomposición en objetos nos hemos centrado en qué elementos podemos encontrar: motor, volante, acelerador, freno, caja de cambios, etc. y qué es lo que hacen.

Al pensar en objetos en lugar de en funciones, usted se acerca más a la realidad, ya que es la forma en que lo haría en cualquier otra situación cotidiana.

De esta forma, una vez encontrados los objetos que intervienen, podremos asignarles las tareas que éstos tienen que realizar.

### 1.4.1. Descripción de objetos: Conjunto de datos que definen un objeto y conjunto comportamientos que pueden solicitarse a los objetos

#### Clases

El elemento clave en todo lenguaje orientado a objetos es la clase. Una clase es como una plantilla o modelo que define los objetos.

Por ejemplo, en el caso del coche, piense en la clase como en la idea genérica que tiene usted de un coche. Sin embargo, un objeto sería un coche en particular que está viendo en la calle.

Está claro que en la vida real usted utilizará un coche determinado (un objeto) y que no puede hacer nada con la idea de coche que pueda tener (la clase).

Pues trasladamos esto al ambiente de programación y podemos seguir la misma lógica. En Java, el concepto de clase es fundamental ya que cualquier línea de código que escriba debe estar en el interior de una determinada clase.

Las clases sirven durante la programación de la aplicación, pero cuando la ejecutamos, estarán actuando los objetos.

Para agregar una clase en NetBeans:

– Cree un nuevo proyecto con el menú File – New Project. Elija el tipo Java Class Library en la categoría General.

– Pulse Next y escriba un nombre para el proyecto.

– Pulse Finish.

  Con ello, NetBeans crea la estructura de carpetas necesaria y nos muestra un proyecto vacío.

– En la ventana Projects, haga clic con el botón derecho del ratón en la carpeta <default package> (paquete predeterminado) de Source Packages.

– Elija New – Java Class.

– Establezca el nombre para la clase, por ejemplo Coche y pulse Finish.

NetBeans creará un archivo llamado Coche.java donde se guardará la clase. El nombre de la clase debe coincidir con el nombre del archivo .java donde aparece y en la que encontraremos la palabra clave class, que define una clase en Java.

Los archivos con la extensión .java son archivos de código fuente, es decir, archivos que pueden editarse en el entorno de desarrollo.

Volvamos al ejemplo del coche. Estoy seguro de que si le piden que describa un coche empezaría a indicar sus características físicas: tiene cuatro ruedas, puede ser de distintos colores, etc.

Estas características que definen el aspecto del coche son propiedades cuando se trasladan al ambiente de programación.

En la figura siguiente puede encontrar las propiedades color del coche, posX y posY para establecer su posición, la velocidad del mismo y num Ruedas para especificar el número de ruedas…

Siguiendo con la descripción de lo que es un coche, también indicaría que puede moverse en una determinada dirección, que puede tomar velocidad: aumentándola y reduciéndola..

Las características que definen lo que puede hacer el coche o, en definitiva, su comportamiento, son llamadas métodos en el ambiente de la programación.

En la figura siguiente puede encontrar el método mover para desplazar el coche, acelerar y frenar para cambiar su velocidad…

Observa lo directo que es trasladar una idea que puede manejar en la realidad a un programa orientado a objetos.

Recuerda: la clase define las características de los objetos. Estas características pueden describirlos (propiedades en el ambiente de programación) o ser de comportamiento (métodos en el ambiente de programación).

### Propiedades

Está claro que estamos escribiendo código Java que todavía no comprende. No te preocupes, por ahora sigue pensando en objetos.

La parte de los objetos que permite describirlos detalladamente son las propiedades.

Las propiedades no siempre serán características físicas. Por ejemplo, podría tener una propiedad que indicara la velocidad máxima que puede alcanzar un objeto coche.

Lógicamente, no todos los objetos coche van a tener la misma velocidad máxima.

Las propiedades se corresponden con variables en el interior de las clases de Java.

```
int numRuedas;
String color;
int posX;
int posY;
int velocidad;
```

Así, en nuestra clase Coche hemos definido varias propiedades:

```
numRuedas, color, posX, posY, velocidad…
```

Cada objeto se caracterizará por tener un determinado valor en dichas propiedades. Además, este valor podrá cambiar a lo largo de la "vida" del objeto.

Por ejemplo, un coche de color rojo y que está aparcado tendrá el valor rojo en la propiedad color y 0 en la de velocidad.

Sin embargo, un coche azul metalizado que va por la autopista podrá tener el valor azul metalizado en su propiedad color y 110 en su propiedad velocidad.

En cualquier caso ambos coches dispondrán del mismo número de ruedas: 4.

Esto nos hace ver que existen dos tipos de propiedades: aquellas que pueden tomar valores distintos en cada objeto y aquellas que son iguales para todos o que las comparten todos los objetos.

En este último caso se dice que son propiedades de clase en lugar de propiedades de objeto. En Java podrá indicar esta circunstancia antecediendo a su declaración la palabra static.

```
static int numRuedas = 4;
```

De esta forma indicamos que el valor de la propiedad numRuedas es compartido por todos los objetos que se creen de la clase Coche.

Esto quiere decir que si modifica esta propiedad en cualquiera de los objetos creados, la estará modificando para todos ellos. En la mayoría de los casos tendrá sentido indicar el valor que va a tomar las variables de clase en la propia declaración, como hemos hecho dándole el valor 4.

Lo importante es entender que definimos las propiedades en la clase pero que éstas sólo tendrán sentido cuando creemos objetos. Las propiedades se definen en la clase pero toman valor en los objetos.

### Metodos y constructores

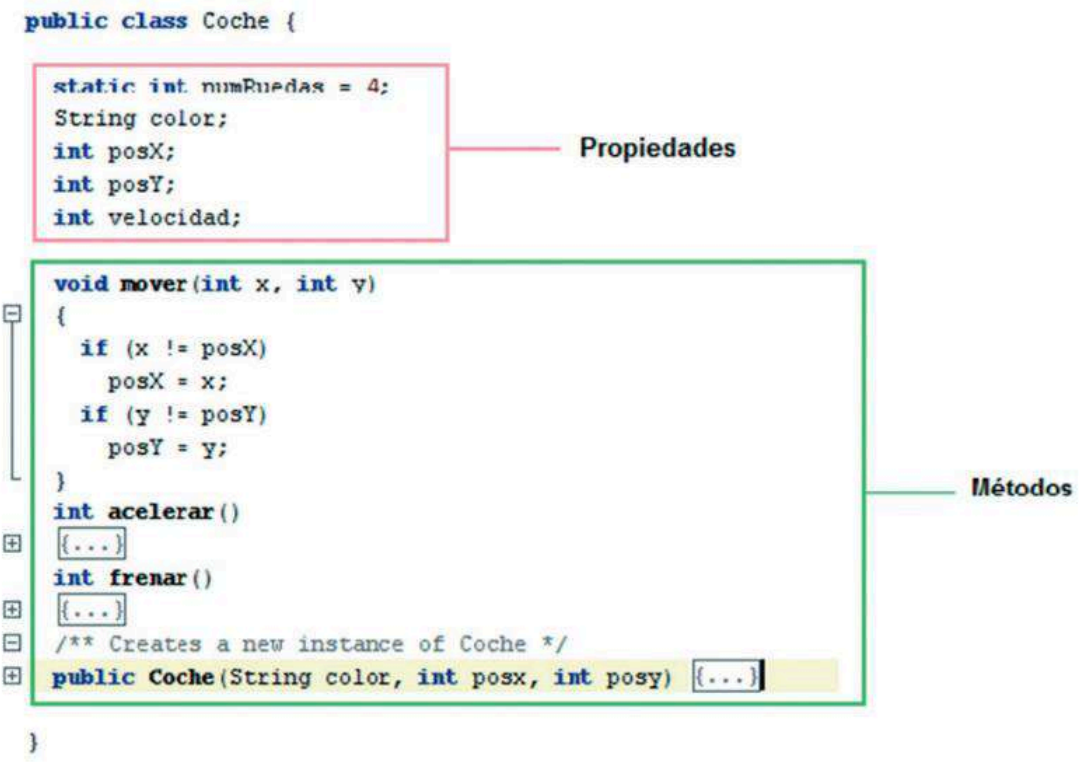
Si las propiedades describen estructuralmente los objetos, los métodos hacen lo propio con su comportamiento.

Los métodos se implementan en Java a través de las funciones. Las funciones en Java siempre aparecen en el interior de una determinada clase, por lo que no podrá encontrar funciones (o métodos) que no pertenezcan a una clase.

Veamos como ejemplo el método mover. La definición de un método indica el tipo de datos del valor que devuelve. Por ejemplo, este método devuelve un valor void.

```
void mover(int x, int y)
{
if (x != posX)
posX = x;
if (y != posY)
posY = y;
}
```

El tipo de datos void es especial ya que indica que el método no devuelve ningún valor.




*NetBeans con ejemplo de código fuente con diferentes características configuradas*

A continuación del nombre del método, se indica, entre paréntesis, la lista de parámetros que utiliza.

Un parámetro sirve para comunicar el método con el resto del código que lo utiliza. En este caso, el método mover permite desplazar el coche de una posición a otra. Lógicamente, para ello necesita conocer la posición final, que se le indica a través de los parámetros x e y.

Observa cómo los parámetros aparecen en el interior de los paréntesis indicando previamente su tipo de datos y separados entre comas.

También puede darse el caso de que el método no utilice parámetros, con lo que la lista aparecerá vacía (como el método acelerar).

```
int acelerar()

{

velocidad = velocidad + 10;

return velocidad;

}
```

Al igual que en el caso de las propiedades, podrías encontrar métodos de clase, es decir, que tienen sentido sólo cuando se piensa en la clase y no cuando estamos trabajando con un objeto en particular.

Existe un método especial que permite crear objetos de la clase.

Este método se llama el constructor.

NetBeans crea un método constructor por nosotros para cada clase. Sólo debemos escribir su código.

```
public Coche(String color, int posx, int posy) {

this.color = color;

this.posX = posx;

this.posY = posy;

this.velocidad = 0;

}
```

El método constructor tiene el mismo nombre que la clase y no devuelve ningún valor, por lo que no se debe especificar ningún tipo de datos delante de su nombre.

En el constructor es donde rellenamos los valores que deben tener las propiedades durante la creación del objeto. Por ejemplo, en este caso vamos a crear un objeto con un determinado color y posición inicial. La velocidad inicial será siempre de 0 kilómetros por hora. La forma de acceder a una determinada propiedad del objeto es a través de la sintaxis nombre_objeto.nombre_propiedad.

Además, la palabra clave this indica que estamos haciendo referencia al propio objeto que se está creando. Así, damos el valor del parámetro color a la propiedad color.

No es estrictamente necesaria la utilización de this ya que si utiliza el nombre de una propiedad o de un método sin especificar un nombre de objeto, entonces se entiende que es el propio objeto llamado o, lo que es lo mismo, this. Puede ver esta situación en los métodos mover, acelerar y frenar.

Observe cómo se utilizan las propiedades posX, posY y velocidad sin indicar el objeto. Es como si hubiese escrito this.posX, this.posY y this.velocidad.

Sabías que

Java diferencia entre mayúsculas y minúsculas. Por lo tanto, no es lo mismo posX que posx.

Sin embargo, en el constructor ha sido ineludible, al menos para constituir adecuadamente el valor de la propiedad color, de forma que se consiga diferenciar entre lo que es la propiedad color del objeto que se crea y el parámetro color del constructor.

La idea es que cuando creemos objetos en el código Java, indiquemos el valor de las propiedades.

Observa cómo no es obligatorio indicar el valor de la propiedad numRuedas ya que este valor se indica en la misma declaración de la propiedad, que es una variable estática.

Por lo tanto, ya sabemos que podremos utilizar el método constructor para crear objetos y, además, hemos visto cómo podemos acceder a las propiedades de los objetos. En la lección siguiente aprenderemos a crear objetos y a utilizar sus métodos.

Aunque no es un requisito de la programación en Java, es habitual seguir una serie de reglas a la hora de dar nombres a las clases, propiedades, métodos, etc. Nosotros las seguiremos.

Éstas son:

– El nombre de las clases debe empezar con una letra en mayúsculas. El mismo nombre sólo incluye una palabra, entonces el resto surge en minúsculas; si contiene más de una palabra, cada letra inicial se pondrá además en mayúsculas. Por ejemplo Coche, CocheUrbano, etc.

– El nombre de las propiedades debe empezar siempre con una letra minúscula. Si el nombre sólo incluye una palabra, toda estará en minúsculas; si incluye más de una palabra, la primera letra inicial de las siguientes palabras aparecerá en mayúscula.

  El nombre de las propiedades debe ser un sustantivo. Por ejemplo, velocidad, numRuedas, etc.

– El nombre de los métodos sigue la misma regla que la que se aplica a las propiedades. Sin embargo, en este caso, el nombre debe coincidir con un verbo.

Por ejemplo, mover, calcularNomina.

No debe utilizar acentos ni caracteres latinos, como la ñ.

Además, no se admiten los espacios en blanco, por lo que es habitual utilizar el carácter subrayado (_) en su lugar.

### 1.4.2. Identificación del comportamiento de un objeto: Concepto del mensaje

Como ya explicaba y conociendo el ejemplo del coche y sus piezas, un objeto como pieza elemental de la POO, es una representación o modelización de un objeto real concerniente a nuestro mundo, por ejemplo, conseguimos tener un objeto perro que simbolice a un perro dentro de nuestra realidad, o bien un objeto factura, cliente o pedido.




Los objetos en la vida real poseen todos dos características: estado y comportamiento. El estado de un objeto viene definido por una serie de parámetros que lo concretan y que lo diferencian de objetos del mismo tipo. En el caso de tener un objeto perro, su estado estaría definido por su raza, color de pelo, tamaño, etc. Y el comportamiento viene definido por las acciones que pueden realizar los objetos, por ejemplo, en el caso del perro su comportamiento sería: saltar, correr, ladrar, etc.

Si tenemos un ejemplo que posee una pertenencia más con el mundo de la informática se pueden ver más claros estos dos conceptos. Si poseemos un objeto pantalla que encarna la pantalla de nuestro ordenador, el estado de la misma quedaría definido por los consecutivos parámetros: encendida o apagada, tamaño, resolución, número de colores, etc.; y su comportamiento podría ser: imprimir, encender, apagar, etc.

Los parámetros o variables que definen el estado de un objeto se denominan atributos o variables miembro y las acciones que pueden realizar los objetos se denominan métodos o funciones miembro, y para indicar variables miembro y funciones miembro se utiliza el término general miembro.

Si lo comparamos con la programación estructurada los atributos o variables miembro serían variables y los métodos o funciones miembro procedimientos y funciones. A partir de ahora y a lo largo de todo el curso vamos a utilizar únicamente la nomenclatura de atributos y métodos.

Los atributos de un objeto deben encontrarse ocultos al resto de los objetos, es decir, no se va a poder acceder directamente a los atributos de un objeto para modificar su estado o consultarlo. Para acceder a los atributos de un objeto se deben utilizar métodos. Es decir, los métodos exponen toda la funcionalidad del objeto, mientras que los detalles del estado interno del objeto permanecen ocultos. Incluso algunos métodos también pueden permanecer ocultos.

El hecho de esconder la implementación interna de un objeto, es decir, como está compuesto y de que se compone se denomina encapsulación. La encapsulación es uno de los beneficios del paradigma de la Programación Orientada a Objetos. Normalmente un objeto ofrece una parte pública que será utilizada por otros objetos para interactuar entre sí, pero también permanece una parte oculta para encapsular los pormenores de la implementación del objeto.

Ya he comentado que un objeto está formado por atributos y métodos. Como la caja negra de un avión, el objeto envuelve la información que acumula y únicamente logramos alcanzar la información e indicarle que efectúa acciones por medio de lo que comúnmente se denomina interfaz del objeto.

Los datos y la implementación subsiste oculta a los demás objetos que interaccionan en el programa, lo que beneficia extraordinariamente la protección de los datos y las estructuras internas contra las modificaciones externas al objeto. Así es mucho más fácil localizar faltas y errores en los programas puesto que cada objeto está altamente especializado, y sólo se encarga de su tarea. Igual se puede observar, esto alcanza una mayor modularidad, que facilita además el diseño en equipo de programas y la reutilización de clases (componentes) creados por otros desarrolladores.

Hemos indicado que la encapsulación es un beneficio que nos aporta la POO, es un beneficio porque permite abstraernos de la utilización de los objetos, es decir, a mi me interesa realizar una determinada tarea con un objeto (por ejemplo imprimir una pantalla o rellenar una factura), pero a mí no me interesa como realiza este proceso internamente el objeto que estoy utilizando. En este momento entra en juego otro concepto importante de la POO y que es la abstracción.

La abstracción indica la capacidad de ignorar determinados aspectos de la realidad con el fin de facilitar la realización de una tarea. Nos permite ignorar aquellos aspectos de la realidad que no intervienen en el problema que deseamos abordar, y también nos permite ignorar los aspectos de implementación de los objetos en los pasos iniciales, con lo cual sólo necesitamos conocer qué es lo que hace un objeto, y no cómo lo hace, para definir un objeto y establecer las relaciones de éste con otros objetos.

Un objeto lo podríamos representar como dos circunferencias, una interna que permanece oculta al mundo exterior y que contendría todos los detalles de la implementación del objeto, y otra circunferencia concéntrica externa, que representa lo que el objeto muestra al mundo exterior y le permite utilizar para interactuar con él. En la figura 1 se puede ver dicha representación.

La encapsulación ofrecida a través de objetos tienen varios beneficios, entre los que destacan la modularidad y la ocultación de la información.

Por medio de la modularidad conseguimos escribir código de forma independiente de cómo se hallen construidos los diferentes objetos que vamos a utilizar. Y ocultando la información se permite efectuar cambios en el código interno de los objetos sin que afecte a otros objetos que los utilicen o dependan de ellos. No es preciso entender la implementación interna de un objeto para poder utilizarlo.

### Mensajes

Los mensajes son la manera que poseen de comunicarse diferentes objetos entre sí. Un objeto por sí sólo no es excesivamente útil, sino que se suele manejar dentro de una aplicación o programa que utiliza terceros objetos.

El comportamiento de un objeto está reflejado en los mensajes a los que dicho objeto puede responder. Representan las acciones que un determinado objeto puede realizar.

Un mensaje enviado a un objeto representa la invocación de un determinado método sobre dicho objeto, es decir, la ejecución de una operación sobre el objeto. Es la manera en la que un objeto utiliza a otro, el modo en el que dos objetos se comunican, ya que la ejecución de ese método retornará el estado del objeto invocado o lo modificará.

Los mensajes se utilizan para que distintos objetos puedan interactuar entre sí y den lugar a una funcionalidad más compleja que la que ofrecen por separado. Un objeto lanzará o enviará un mensaje a otro objeto si necesita utilizar un método del segundo objeto. De esta forma si el objeto A quiere utilizar un método del objeto B, le enviará un mensaje al objeto A.

Para mandar un mensaje se precisan tres componentes: el objeto al que se le va a enviar el mensaje, el nombre del método que se debe ejecutar y los parámetros obligatorios para el método en cuestión.

# UD1
# Lo más importante

– Otra razón muy importante apareció en el estudio de ese paradigma de programación, y su beneficio enorme, el concepto de reusabilidad. Poder reusar piezas para futuros programas abrió grandes horizontes a la industria de la informática relacionada con la programación.

– La orientación a objetos se basa en tres pilares básicos:

  · Encapsulación y ocultación de la información

  · Abstracción y clasificación

  · Herencia y polimorfismo

– Cuando un software se planifica bien, se implementa y acaba teniendo éxito, sobre todo cuando satisface las necesidades de las personas que lo utilizan; cuando funciona de forma impecable durante mucho tiempo; cuando es fácil de modificar o incluso es más fácil de utilizar, puede cambiar todas las cosas y de hecho cambiar para mejor.

– El software es esencialmente un conjunto de instrucciones (programas) que proporcionan la funcionalidad requerida, los datos relacionados y documentos. Por lo tanto, el software es un elemento lógico y etéreo y se diferencia del hardware, un elemento físico, en sus características.

– El software se crea normalmente desde cero, aunque en muchos casos los desarrolladores se apoyan el librerías, clases y componentes de software ya implementados que ajustan y se configuran para desarrollar buen software a modo de piezas de puzzle.Dichas herramientas necesitan de buenos programadores/as para su implementación, pues no resultan fáciles.

– La ingeniería del software representa un proceso formal que incorpora una serie de métodos bien definidos para el análisis, diseño, implementación y pruebas del software y sistemas.

– Las fases por las que tiene que pasar obligatoriamente el software que se ha desarrollado desde que comienza la idea hasta que es retirado del mercado, ese conjunto de fases se denomina Ciclos de Vida. También se denomina a veces paradigma.

– Modelo iterativo Este modelo nace como derivación al modelo en cascada. Buscaba al implementarse disminuir el riesgo que existía entre las necesidades del usuario y el producto final por malas interpretaciones durante la etapa de recogida de requisitos.

– Un buen análisis estructurado busca que se defina transparentemente lo que busca que haga el sistema y sus necesidades de aplicación. No se puede tener en cuenta en este paso como se implantará el aplicativo ni todavía requisitos que deben cumplirse.

– Se puede explicar brevemente programación orientada a objetos (OOP) como una técnica o manera de programar con un lenguaje que utiliza objetos como bloques esenciales de construcción.

– Una clase en una especie de plantilla o modelo que describe como se crean ciertos tipos de Objeto. Cada vez que se construye un Objeto de una Clase, se crea una instancia de esa Clase(“instance”). Una Clase es una colección de Objetos similares y un Objeto es una instancia de una Clase.

– En un sentido literal, Polimorfismo significa la cualidad de tener más de una forma. En el contexto de OOP, el Polimorfismo se refiere al hecho de que una simple operación puede tener diferente comportamiento en diferentes objetos.

# UD1
# Autoevaluación

1. ¿Qué es el término encapsulación en Programación Orientada a Objetos?
   a. Cada componente sabe lo mínimo posible de los otros.
   b. Nada puede funcionar sin su interacción.
   c. Tener nada más que los datos relacionados con un tema y su manipulación.
   d. Representaciones abstractas de los conceptos.

2. ¿Qué es el termino Herencia en POO?
   a. Tener nada más que los datos relacionados con un tema y su manipulación.
   b. Cada componente sabe lo mínimo posible de los otros.
   c. Representaciones abstractas de los conceptos.
   d. Define el nuevo como la diferencia respecto al anterior.

3. ¿Que significa el término Especificación?
   a. Es la tarea de escribir detalladamente el software a ser desarrollado.
   b. Enfoque sistemático cubriendo los aspectos del desarrollo.
   c. La captura, análisis y especificación de requisitos.
   d. Determinar cómo funcionará el software de forma general.

4. ¿No es un Principios de la ingeniería del software?
   a. Haz de la calidad la razón de trabajar.
   b. Cobrar lo antes posible el trabajo informático.
   c. Seleccionar el modelo de ciclo de vida adecuado.
   d. Probar, probar y probar.

5. ¿No es un modelo de ciclo de vida del software?
   a. Modelo en Cascada.
   b. Modelo en espiral.
   c. Modelo en V.
   d. Modelo intrínseco.

6. ¿No son elementos de la POO?
   a. Objetos.
   b. Función.
   c. Clases.
   d. Métodos.

7. ¿No es una ventaja de la POO?
   a. Facilidad para desglosar el problema.
   b. Reutilización del código.
   c. Programación toda en un solo bloque.
   d. Buena división del trabajo entre un equipo.

8. ¿La Máquina Virtual de Java (JVM)?
   a. Es el programa que usaremos para programar. Es gratis.
   b. Entorno virtual que se utiliza para interpretar los bytecodes.
   c. Un Sistema Operativo común para todas las plataformas.
   d. Espacio donde hacemos las pruebas de nuestro código fuente.

9. ¿Qué son las JDK (Java Development Kit)?
    a. Espacio donde hacemos las pruebas de nuestro código fuente.
    b. La máquina virtual de Java.
    c. Es el programa que usaremos para programar. Es gratis.
    d. El Kit de desarrollo y provee de un compilador y mecanismo para comprimir un proyecto.

10. ¿Los archivos con la extensión .java son?
    a. Son ficheros de Java empaquetados y se guardan además en .jar.
    b. Son ficheros que genera NetBeans.
    c. Archivos de código fuente de Java.
    d. Resultado de la compilación y se guarda además en .class.

Área: informática y comunicaciones

# UD2

## Clases y objetos

2.1. Distinguir el concepto de clase y sus atributos, métodos y mecanismo de encapsulación

2.1.1. Relación entre interfaz y clase

2.1.2. Distinción de los tipos de datos y clases

2.2. Análisis de los objetos: Estado, comportamiento e identidad

2.2.1. Análisis de mensajes

2.2.2. Tipos de métodos y su clasificación: Métodos de acceso, de selección o consulta, de construcción, de destrucción

2.3. Uso de objetos como instancias de clase. Instancia actual (this, self, current)

2.4. Identificación del concepto de programa en el paradigma orientado a objetos. POO = objetos + mensajes

## 2.1. Distinguir el concepto de clase y sus atributos, métodos y mecanismo de encapsulación

El concepto de clases y objetos en Java es muy simple de entender. La clase como eje principal del lenguaje Java donde todo forma parte de una clase, es una clase o describe como funciona una clase. Y el objeto como la instanciación de una clase. Entender bien este concepto es vital para comprender mejor la programación en Java. El bloque de la clase es donde se producen y realizan todas las acciones de la programación en Java. Y lo entendemos como una clonación. Todos los métodos se hacen en el interior de la clase, Java no resiste funciones o variables globales. Esto alcanza desorientar a los programadores de C++, que consiguen concretar métodos fuera del elemento de la clase, pero esta posibilidad es más una tentativa de no apartarse mucho y ser compatible con C, que un buen esquema orientado a objetos. Así pues, el armazón de cualquier aplicación Java se basa en la definición de una clase. El componente elemental de la programación orientada a objetos en Java es una clase. Una clase precisa la forma y comportamiento de un objeto. Todo en Java gira alrededor de estas percepciones, por ello, una vez que se han rectificado los mecanismos primordiales del lenguaje en cuestiones ya conocidas; esta unidad se penetra en la ampliación de las nociones teóricas conducentes hacia el estilo de programación orientada a objetos de Java.

### Objetos

Definiremos antes de entrar en el estudio de las Clases en Java, hay que presentar a los objetos, que son las instanciaciones de una clase. Un objeto tiene un estado y un funcionamiento. El estado está contenido en sus variables (variables miembro), y el funcionamiento viene determinado por sus métodos.

Las variables miembro consiguen ser variables de instancia o variables de clase. Se agiliza el funcionamiento del objeto solicitando a uno de sus métodos, que efectuará una tarea o modificará su estado, o ambas cosas. Cuando un objeto ya no se usa en C++, se destruye y la memoria que ocupaba se libera y

es devuelta al Sistema. Cuando un objeto ya no se usa en Java, simplemente se anula. Posteriormente, el reciclador de memoria (garbage collector) puede recuperar esa memoria para devolverla al Sistema.

Las afirmaciones anteriores son un compendio de lo que se puede esperar de un objeto.

### Creación de objetos

Un objeto es una instancia de una clase. En Java y en C++, la creación de un objeto se efectúa en tres movimientos (aunque se consiguen combinar):

– Declaración, proveer un nombre al objeto.

– Instanciación, establecer memoria al objeto.

– Inicialización, opcionalmente se consiguen facilitar valores iniciales a las variables de instancia del objeto.

Cuando se trata de Java, es trascendental examinar la diferencia entre objetos y variables de tipos básicos, porque en C++ se tratan de forma análoga, cosa que no ocurre en Java.

Tanto en Java como en C++ la declaración e instanciación de una variable de tipo básico, utiliza una sentencia que asigna un nombre a la variable y reserva memoria para almacenar su contenido:

```
int miVariable;
```

En los dos lenguajes se logra inicializar la variable con un valor establecido en el instante de declararla, es decir, podemos sintetizar los tres pasos primeramente referidos de declaración, instanciación e inicialización, en una sola sentencia:

```
int miVariable = 7;
```

Y, lo más importante, este sucede en tiempo de compilación.

C++ consigue declarar e instanciar variables de tipo primitivo u objetos, desde clases en tiempo de ejecución, de forma que el compilador no conoce, en tiempo de compilación, dónde va a estar almacenado el objeto o la variable. Esto es lo que se llama instanciación de un objeto o variable en memoria dinámica.

El ejercicio enseña la instanciación e inicialización de variables y objetos en memoria dinámica. También muestra la instanciación e inicialización de objetos y variables estáticas y la instanciación de arrays en memoria dinámica. En C++ cuando se instancia un array en memoria dinámica, no se consiente la inicialización.

Como ventaja decir que, Java es mucho más restringido que C++. Por ejemplo, Java no consiente la instanciación de variables de tipos básicos en memoria dinámica, aunque hay una serie de objetos que coinciden con los tipos básicos y que logran utilizarse para este propósito. Java tampoco aprueba la instanciación de objetos en memoria estática.

En lenguaje C++ si se instancia un array de objetos en memoria dinámica, lo que hace es instanciar el array de objetos y restituir un puntero al primer objeto. En Java, cuando se instancia un array (que siempre ha de ser en memoria dinámica), se instancia un array de referencias, o punteros (llámese como se quiera) a los objetos. En Java, es necesario utilizar múltiples veces el operador new para instanciar un array de objetos, mientras que en C++ sólo es necesario su uso una única vez. El siguiente código de ejemplo, corresponde a la aplicación Java que realiza el mismo trabajo que el programa C++ visto antes, pero con las restricciones que se acaban de enumerar. Se ha incorporado la presentación en pantalla no sólo del dato al que apunta un objeto, sino también de la dirección en la que se encuentra almacenado ese dato (la dirección del objeto). La salida del programa sería:

```
C:\> java pruebas
miVariableInt contiene 6
miPunteroObj contiene pruebas@208741
miPunteroObj apunta a 12
miArrayPunterosObj contiene pruebas@20875f pruebas@208760 pruebas@208761
miArrayPunterosObj apunta a 13 14 15
```

El formato en que se muestra un objeto en Java, cuando el compilador no conoce el tipo de que se trata, radica en el identificador de la clase y el valor hexadecimal de la dirección.

En Java no constantemente es obligatoria la declaración de un objeto (darle un nombre). En el sucesivo ejemplo, se instancia un nuevo objeto que se usa en una expresión, sin previamente haberlo declarado.

```
import java.util.*;

class pruebas {
    public static void main( String args[] ) {
        System.out.println( new Date() );
        }
    }
```

La inicialización de un objeto, ya sea en C++ como en lenguaje Java, se puede efectuar manejando las constantes de la clase, de forma que un objeto de un idéntico tipo puede ser declarado e inicializado de formas diferentes.

Todos los datos básicos, como los enteros, se deben declarar en las clases antes de hacer uso de ellos. En C la unidad fundamental son los ficheros con código fuente, en Java son las clases. De hecho son pocas las sentencias que se pueden colocar fuera del bloque de una clase. La palabra clave import (equivalente al #include) puede colocarse al principio de un fichero, fuera del bloque de la clase. Sin embargo, el compilador reemplazará esa sentencia con el contenido del fichero que se indique, que consistirá, como es de suponer, en más clases.

La definición de una clase consta de dos partes, la declaración y el cuerpo, según la siguiente sintaxis:

```
DeclaracionClase {
    CuerpoClase
    }
```

Sin el punto y coma (;) final característico de C++ para señalar el final de la definición de una clase.

La declaración de la clase muestra al compilador el nombre de la clase, la clase de la que procede (su superclase), los privilegios de acceso a la clase (pública, abstracta, final) y si la clase implementa o no, uno o varios interfaces. El nombre de la clase debe ser un identificador válido en Java. Por convención, muchos programadores Java empiezan el nombre de las clase Java con una letra mayúscula.

Las clases en Java derivan, directa o indirectamente, de la clase Object. La clase padre inmediatamente superior a la clase que se está expresando se conoce como superclass. Si no se detalla la superclase de la que deriva una clase, se concibe que deriva directamente de la clase Object (definida en el paquete java.lang).

En la declaración de una clase se maneja la palabra clave extends para detallar la superclase, de la forma:

```
class MiClase extends SuperClase {
    // cuerpo de la clase
    }
```

Los programadores C++ procuran mostrar atención a que no se muestra nada sobre los privilegios de acceso a la clase, anotando, si es pública, privada o protegida. O sea, la herencia de los métodos de acceso a una clase no se consigue utilizar en Java para transformar el control de acceso asignado a un miembro de la clase padre.

De forma similar que en C++, una clase hereda las variables y métodos de su superclase y de la superclase de esa clase, etc.; hay que señalar, de todo el árbol de jerarquía de clases desde la clase que estamos declarando hasta la raíz del árbol: Object. En otros términos, un objeto que es instanciado desde una clase determinada, contiene todas las variables y métodos de instancia específicos para esta clase y sus antecesores; no obstante los métodos pueden ser modificados (sobreescritos) en algún lugar.

Una clase puede implementar uno o más interfaces, declarándose esto utilizando la palabra clave implements, seguida de la lista de interfaces que implementa, separadas por coma (,), de la forma:

```
class MiClase extends SuperClase implements MiInterfaz,TuInterfaz {
    // cuerpo de la clase
    }
```

Si una clase muestra que implementa un interfaz, se puede certificar que provee una definición para todos y cada uno de los métodos revelados en ese interfaz; en caso inverso, el compilador generará errores al no poder solucionar los métodos del interfaz en la clase que lo implementa.

Hay cierta semejanza entre un interfaz y una clase abstracta, aunque las enunciaciones de métodos no están consentidas en un interfaz y sí se pueden definir en una clase abstracta. El propósito de los interfaces es proporcionar nombres, es decir, solamente declara lo que necesita implementar el interfaz, pero no cómo se ha de realizar esa implementación; es una forma de encapsulación de los protocolos de los métodos sin forzar al usuario a utilizar la herencia.

Cuando se implementa un interfaz, los nombres de los métodos de la clase deben coincidir con los nombres de los métodos que están declarados en es interfaz, todos y cada uno de ellos.

El cuerpo de la clase contiene las declaraciones, y potencialmente la inicialización, de todos los datos miembros, tanto variables de clase como variables de instancia, y la definición completa de todos los métodos.

Las variables pueden expresarse dentro del cuerpo de la clase o dentro del cuerpo de un método de la clase. Por ello, éstas últimas no son variables miembro de la clase, sino variables locales del método en la que están expresadas.

Un objeto instanciado de una clase tiene un momento definido por el valor actual de las variables de instancia y un entorno definido por los métodos de instancia. Es propio de la programación orientada a objetos el delimitar el acceso a las variables y proporcionar métodos que admitan el acceso a esas variables, aunque esto no es estrictamente necesario.

Alguna o todas las variables pueden declararse para que se comporten como si fuesen constantes, manejando la palabra clave final.

Los objetos instanciados a partir una clase poseen todos los métodos de instancia de esa clase y además todos los métodos heredados de la superclase y sus antecesores. Por ello, todos los objetos en Java heredan, directa o indirectamente, de la clase Object; inmediatamente todo objeto contiene los miembros de la clase Object, es decir, la clase Object define el esta y ambiente básicos para todo objeto Java. Esto difiere significativamente de C++, en donde no hay una clase central de la que se derive todo, por lo que no hay una definición de un estado básico en C++.

Las características más significativas que la clase Object cede a sus descendientes son:

– Posibilidad de cooperación consigo o con otro objeto

– Posibilidad de conversión automática a String

– Posibilidad de espera sobre una variable de condición

– Posibilidad de notificación a otros objetos del estado de la variable de condición

Hay muchas otras características que se verán a lo largo de los ejemplos y secciones de la unidad.

La sintaxis general de definición de clase se podría extender tal como se muestra en el siguiente diagrama, que debe tomarse solamente como referencia y no al pie de la letra:

```
NombreDeLaClase {
    // declaración de las variables de instancia
    // declaración de las variables de la clase
    metodoDeInstancia() {
        // variables locales
        // código
        }
    metodoDeLaClase() {
        // variables locales
        // código
        }
    }
```

## Tipos de clases

Hasta ahora sólo se ha utilizado la palabra clave public para calificar el nombre de las clases que hemos visto, pero hay tres modificadores más. Los tipos de clases que podemos definir son:

### public

Las clases public son accesibles desde nuevas clases, bien sea claramente o por herencia, desde clases declaradas externamente del paquete que contiene a esas clases públicas, ya que, por defecto, las clases únicamente son accesibles por otras clases declaradas adentro del mismo paquete en el que se han declarado. Para acceder desde otros paquetes, primeramente tienen que ser importadas. La sintaxis es:

```
public class miClase extends SuperClase implements miInterface,TuInterface {
    // cuerpo de la clase
    }
```

Aquí la palabra clave public se utiliza en un contexto diferente del que se emplea cuando se define internamente la clase, junto con private y protected.

### abstract

Una clase abstract tiene al menos un método abstracto. Una clase abstracta no se instancia, sino que se utiliza como clase base para la herencia. Es el equivalente al prototipo de una función en C++.

### final

Una clase final se declara como la clase que acaba una cadena de herencia, es lo inverso a una clase abstracta. Nada puede heredar de una clase final. Por ejemplo, la clase Math es una clase final. No obstante es técnicamente viable declarar clases con varias combinaciones de public, abstract y final, la declaración de una clase abstracta y a la vez final no tiene sentido, y el compilador no permitirá que se declare una clase con esos dos modificadores juntos.

### synchronizable

Este modificador detalla que todos los métodos específicos en la clase se sincronizan, es decir, que no se puede acceder al mismo tiempo a ellos a partir de diferentes threads; el sistema se faculta para ubicar los flags necesarios para evitarlo. Este componente hace que desde threads diferentes se puedan modificar las mismas variables sin que haya problemas de que se sobreescriban.

Si no se maneja alguno de los modificadores expuestos, por defecto, Java toma que una clase es:

– No final

– No abstracta

– Subclase de la clase Object

– No implementa interfaz alguno

En un lenguaje orientado al paradigma de la programación orientada a objetos, podemos sintetizar brevemente las siguientes expresiones con esta tabla:

| |
|---|
| Nombres:<br>**Objetos**<br>**Propiedades de objetos** |
| Adjetivos:<br>**Valores de las propiedades** |
| Verbos:<br>**Comportamiento de los objetos** |
| **“El coche tiene color rojo y se mueve”**<br>**“El documento tiene letra grande y se muestra”** |

Resumiendo varios conceptos En el mundo real, las personas identifican los objetos como cosas que pueden ser percibidas por los cinco sentidos. Los objetos tienen propiedades específicas, como posición, tamaño, color, forma, textura, etc. que definen su estado. Los objetos también poseen ciertos comportamientos que los hacen diferentes de otros objetos.

Se concreta un objeto como “algo que tiene un estado, un comportamiento y una identidad”. Supongamos una máquina de una fábrica. El estado de la máquina puede estar marchando/ parando (“on/off”), hay que tener en cuenta su potencia, velocidad máxima, velocidad actual, temperatura, etc. Su actuación puede contener acciones para arrancar y parar la máquina, alcanzar su temperatura, activar o desactivar nuevas máquinas, conocer las condiciones de señal de error o cambiar la velocidad. Su paralelismo se basa en el hecho de que cada instancia de una máquina es única, tal vez identificada por un número de serie. Las características que se eligen para enfatizar el estado y el comportamiento se apoyarán en cómo un objeto máquina se utilizará en una aplicación. En un diseño de un programa orientado a objetos, se crea una abstracción (un modelo simplificado) de la máquina basada en las propiedades y en el comportamiento que son útiles en el tiempo.

Martin y Odell concretan un objeto como “cualquier cosa, real o abstracta, en la que se acumulan datos y aquellos métodos (operaciones) que manejan los datos”. Para realizar esa actividad, se añaden a cada objeto de la clase los propios datos asociados con sus propios métodos miembro que corresponden a la clase.

Un mensaje es una instrucción que se manda a un objeto y que, cuando se toma, ejecuta sus acciones. Un mensaje envuelve un identificador que contiene la acción que ha de procesar el objeto junto con los datos que requiere el objeto para realizar su trabajo. Los mensajes, por consiguiente, forman una ventana del objeto al mundo exterior.

El usuario de un objeto se comunica con el objeto mediante su interfaz, un conjunto de operaciones definidas por la clase del objeto de modo que sean todas visibles al programa. Una interfaz se puede considerar como una vista simplificada de un objeto. Por ejemplo, un dispositivo electrónico como una máquina de fax tiene una interfaz de usuario bien definida; por ejemplo, esa interfaz incluye el mecanismo de avance del papel, botones de marcado, el receptor y el botón “enviar”. El usuario no tiene que conocer cómo está construida la máquina internamente, el protocolo de comunicaciones u otros detalles. De hecho, la apertura de la máquina durante el periodo de garantía puede anularla.

Desde una óptica práctica y de la programación se concibe como clase a un tipo definido por el usuario y que son bloques de construcción básicos dentro de la programación orientada a objetos. Booch define una clase como “un conjunto de objetos que comparten una estructura y un comportamiento comunes”.

Una clase contiene la especificación de los datos que representan un objeto junto con la descripción de las acciones que un objeto conoce cómo ha de ejecutar. Estas acciones se conocen como servicios o métodos. Una clase incluye asimismo todos los datos necesarios para representar los objetos creados a partir de la clase. Estos datos se conocen como atributos, variables o variables instancia. La expresión atributo se utiliza en análisis y diseño orientado a objetos, y el término variable instancia se suele utilizar en programas orientados a objetos.

Primeramente antes de que un programa consiga crear objetos de cualquier clase, ésta debe ser definida. La definición de una clase representa que se debe dar a la misma un nombre, dar nombre también a los elementos que acumulan sus datos y describir los métodos que realizarán las acciones consideradas en los objetos.

Las definiciones o especificaciones no son un código de programa ejecutable. Se utilizan para asignar almacenamiento a los valores de los atributos usados por el programa y reconocer los métodos que utilizará el programa. Normalmente, se sitúan en archivos formando los denominados packages, se utiliza un archivo para varias clases que están relacionadas.

Aunque ya comencemos a escribir algo de código fuente se irán describiendo a lo largo de las unidades todos los componentes, palabras clave, códigos, estructuras de datos y del lenguaje con mucho detenimiento. Creo importante para comprender mejor y explicar clases, objetos, métodos con un poco de código Java.

## Formato

```
class NombreClase
{
lista_de_miembros
}
```

– NombreClase: Nombre definido por el usuario que identifica la clase (puede incluir letras, números y subrayados).

– lista _ de _ miembros: métodos y datos miembros de la clase.

### Ejemplo

Definición de una clase llamada Punto que contiene las coordenadas x e y de un punto en un plano.

```
class Punto
{
private int x; // coordenada x
private int y; // coordenada y
public Punto(int x _ , int y _ ) // constructor
{
       x = x _ ;
       y = y _ ;
  }
public Punto() // constructor sin argumentos
{
      x = y = 0;
}
public int leerX() // devuelve el valor de x
{
return x;
}
public int leerY() // devuelve el valor de y
{
return y;
}
void fijarX(int valorX) // establece el valor de x
{
x = valorX;
}
void fijarY(int valorY) // establece el valor de y
{
y = valorY;
}
}
```

### Ejemplo

Declaración de la clase Edad.

```
class Edad
{
private int edadHijo, edadMadre, edadPadre; datos
public Edad(){…} método especial: constructor
public void iniciar(int h,int e,int p){…} métodos
public int leerHijo(){…}

public int leerMadre(){…}
public int leerPadre(){…}
}
```

### Objetos

En el momento que una clase ha sido concretada, un programa puede coger una instancia de la clase, nombrada objeto de la clase. Un objeto se crea con el operador new aplicado a un constructor de la clase. Un objeto de la clase Punto inicializado a las coordenadas (2,1) sería:

```
new Punto(2,1);
```

El operador new crea el objeto y devuelve una referencia al objeto elaborado. Esta referencia se asigna a una variable del tipo de la clase. El objeto permanecerá vivo siempre que esté referenciado por una variable de la clase que es instancia.

### Formato para definir una referencia

```
NombreClase varReferencia;
Formato para crear un objeto
varReferencia = new NombreClase(argmntos _ constructor);
```

Toda clase posee uno o mas métodos, llamados constructores, para inicializar el objeto cuando es creado; tienen el mismo nombre que el de la clase, no tienen tipo de retorno y pueden estar sobrecargados. En la clase Edad del Ejemplo de código, el constructor no tiene argumentos, se puede crear un objeto:

```
Edad f = new Edad();
```

El operador de acceso a un miembro (.) elige un miembro individual de un objeto de la clase. Por ejemplo:

```
Punto p;
p = new Punto();
p.fijarX (100);
System.out.println(“ Coordenada x es “ + p.leerX());
```

El operador punto (.) se utiliza con los nombres de los métodos y variables instancia para definir que son miembros de un objeto.

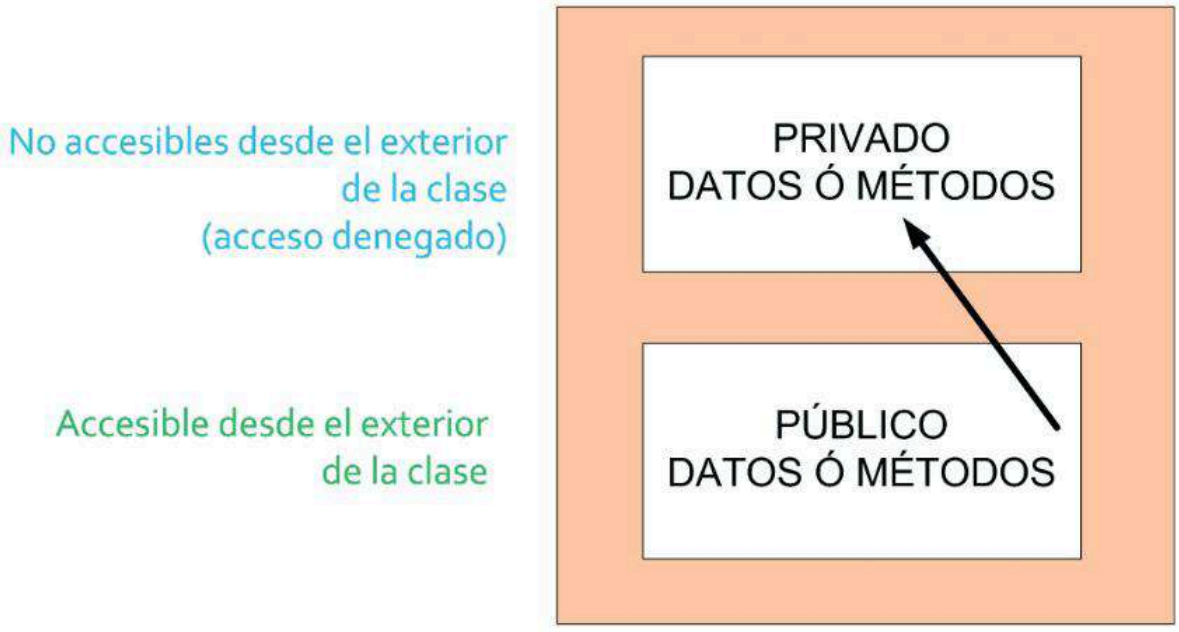
### Ejemplo

Clase DiaSemana, contiene el método visualizar()

```
DiaSemana hoy; // hoy es una referencia
hoy = new DiaSemana(); // se ha creado un objeto
hoy.visualizar(); // llama al método visualizar()
```

## Visibilidad de los miembros de la clase

Uno de los principios esenciales en programación orientada a objetos es la ocultación de la información, que quiere decir que a algunos datos del interior de una clase no se puede acceder por métodos externos a ella. El componente principal para ocultar datos es colocarlos en una clase y crearlos privados. A los datos o métodos privados únicamente se puede acceder desde dentro de la clase. Por el contrario, los datos o métodos públicos son accesibles desde el exterior de la clase.




*Estructura de Secciones pública y privada de una clase*

Para gestionar el acceso a los miembros de la clase se utilizan tres especificadores de acceso: public, private y protected. Cada miembro de la clase está precedido del especificador de acceso que le pertenece.

### Formato

```
class NombreClase
{
private declaración miembro privado; // miembros privados
protected declaración miembro protegido; // miembros protegidos
public declaración miembro público; // miembros públicos
}
```

El especificador public concreta como miembros públicos, aquellos a los que se puede acceder por cualquier método desde fuera de la clase. A los miembros que siguen al especificador private sólo se puede acceder por métodos miembros de la misma clase. A los miembros que siguen al especificador protected se puede acceder por métodos miembro de la semejante clase o de clases derivadas, así desde métodos de otras clases que se hallan en el mismo paquete.

Los especificadores public, protected y private pueden surgir en cualquier orden. Si no se detalla acceso (acceso por defecto) a un miembro de una clase, a éste se puede acceder desde los métodos de la clase y desde cualquier método de las clases del paquete en el que se encuentra.

### Ejemplo

Declaración de la clase Foto y Marco con miembros declarados con distinta visibilidad. Dichas clases forman parte del paquete soporte.

```
package soporte;



class Foto
{
    private int nt;
    private char opd;
   String q;
public Foto(String r) // constructor
  {
    nt = 0;
   opd = 'S';
   q = new String(r);
   }
public double mtd(){…}
}
class Marco
  {
   private double p;
   String t;
   public Marco() {…}
   public void poner()
       {
       Foto u = new Foto("Paloma");
       p = u.mtd();
       t = "**" + u.q + "**";
       }
}
```

| Modelo de visibilidad | | | | |
|---|---|---|---|---|
| Tipo de miembro | Miembro de la misma clase | Miembro de una clase derivada | Miembro de clase del paquete | Miembro de clase de otro paquete |
| Private | x | | | |
| En blanco | x | | x | |
| Protected | x | x | x | |
| Public | x | x | x | x |
| "x" muestra que el acceso está consentido | | | | |

No obstante las especificaciones públicas, privadas y protegidas pueden aparecer en cualquier orden, en Java los programadores suelen seguir ciertas reglas en el diseño que mencionamos a continuación, para que puedas reelegir la que consideres más eficaz.

– Poner los miembros privados primero, debido a que contienen los atributos (datos).

– Colocar los miembros públicos primero, debido a que los métodos y los constructores son la interfaz del usuario de la clase.

En realidad, tal vez el uso más importante de los especificadores de acceso es implementar la ocultación de la información. El principio de ocultación de la información muestra que toda la interacción con un objeto se debe limitar a utilizar una interfaz bien definida que permita ignorar los pormenores de implementación de los objetos. Por deducido, los datos y los métodos públicos constituyen la interfaz externa del objeto, mientras que los componentes privados son los aspectos internos que no requieren ser accesibles para usar el objeto. Los elementos de una clase sin especificador y los protected tienen las mismas propiedades que los públicos respecto a las clases del paquete.

El principio de encapsulamiento representa que las estructuras de datos internas utilizadas en la implementación de una clase no pueden ser accesibles directamente al usuario de la clase.

## Métodos de una clase

Los métodos en Java invariablemente son miembros de clases, no hay métodos o funciones fuera de las clases. La implementación de los métodos se circunscribe dentro del cuerpo de la clase. El código siguiente muestra la declaración completa de una clase.

```
Class Producto
{
prívate int numProd;
private String nomProd;
private Strig descripProd;
private double precioProd;
private int numUnidades;

public Producto() {…}
public Producto(int n, char[]nom,char[]des,double p, int nu) {   }
public void verProducto() {…}
public double obtenerPrecio() {…}
public void actualizarProd(int b)  {…}
}
```

## Código de definición típica de una clase

La clase Racional precisa el numerador y el denominador característicos de un número racional.

Por cada dato se facilita un método miembro que devuelve su valor y un método que establece numerador y denominador. Tiene un constructor que inicializa un objeto a 0/1.

```
class Racional
{
private int numerador;
private int denominador;
public Racional()
{
numerador = 0;
denominador = 1;
}

public int leerN() { return numerador; }
public int leerD() { return denominador; }
public void fijar (int n, int d)
{
numerador = n;
denominador = d;
}
}
```

## Ejercicio

Definir una clase DiaAnyo que contenga los atributos mes y día, el método igual() y el método visualizar(). El mes se registra como un valor entero (1, Enero; 2, Febrero; etc.). El día del mes se registra en la variable entera día. Escribir un programa que compruebe si una fecha es la de su cumpleaños.

El método main() de la clase principal, Cumple, crea un objeto DiaAnyo y llama al método igual() para determinar si coincide la fecha del objeto con la fecha de su cumpleaños, que se ha leído del dispositivo de entrada.

```
import java.io.*;
class DiaAnyo
{
private int mes;
private int dia;
public DiaAnyo(int d, int m)
{
dia = d;
mes = m;
}
public boolean igual(DiaAnyo d)
{
if ((dia == d.dia) && (mes == d.mes))
return true;
else
return false;
}
// muestra en pantalla el mes y el día
public void visualizar()
{
System.out.println("mes = " + mes + " , dia = " + dia);
}
}
// clase principal, con método main
public class Cumple
{
public static void main(String[] ar)throws IOException
```

```
{
DiaAnyo hoy;
DiaAnyo cumpleanyos;
int d, m;
BufferedReader entrada = new BufferedReader(
new InputStreamReader(System.in));
System.out.print("Introduzca fecha de hoy, dia: ");
d = Integer.parseInt(entrada.readLine());
System.out.print("Introduzca el número de mes: ");
m = Integer.parseInt(entrada.readLine());
hoy = new DiaAnyo(d,m);
System.out.print("Introduzca su fecha de nacimiento, dia: ");
d = Integer.parseInt(entrada.readLine());
System.out.print("Introduzca el número de mes: ");
m = Integer.parseInt(entrada.readLine());
cumpleanyos = new DiaAnyo(d,m);
System.out.print( " La fecha de hoy es ");
hoy.visualizar();
System.out.print( " Su fecha de nacimiento es ");
cumpleanyos.visualizar();
if (hoy.igual(cumpleanyos))
System.out.println( "¡Feliz cumpleaños ! ");
else
System.out.println( "¡Feliz dia ! ");
}
}
```

### 2.1.1. Relación entre interfaz y clase

#### ¿Qué es un interface?

Se denomina Interface al grupo de métodos declarados (sin que se definan). *Runnable* es un ejemplo de interface en el cual se declara, pero no se implemementa, una función miembro *run*.

Para explicarlo mejor es necesario indicar, que los métodos abstractos son útiles cuando se quiere que cada implementación de la clase parezca y funcione igual, pero necesita que se cree una nueva clase para utilizar los métodos abstractos. Los interfaces proporcionan un mecanismo para abstraer los métodos a un nivel superior, lo que permite simular la herencia múltiple de otros lenguajes.

Un interfaz sublima el concepto de clase abstracta hasta su grado más alto. Un interfaz podrá verse simplemente como una forma, es como un molde, solamente permite declarar nombres de métodos, listas de argumentos, tipos de retorno y adicionalmente miembros datos (los cuales podrán ser únicamente tipos básicos y serán tomados como constantes en tiempo de compilación, es decir, static y final).

Un interfaz contiene una repertorio de métodos que se implementan en otro terreno. Los métodos de una clase son public, static y final.

La primordial diferencia entre interface y abstract es que un interfaz suministra un mecanismo de encapsulación de los protocolos de los métodos sin forzar al usuario a utilizar la herencia. Por ejemplo:

```
public interface VideoClip {
        // comienza la reproduccion del video
        void play();
        // reproduce el clip en un bucle
        void bucle();
        // detiene la reproduccion
        void stop();
        }
```

Las clases que pretendan utilizar el interfaz VideoClip utilizarán la palabra implements y proporcionarán el código obligatorio para implementar los métodos que se han definido para el interfaz:

```
class MiClase implements VideoClip {
        void play() {
            <código>
            }
        void bucle() {
            <código>
            }
        void stop() {
            <código>
            }
```

Al utilizar implements para el interface es como si se formase una acción de copiar–y–pegar del código del interface, con lo cual no se hereda nada, solamente se pueden emplear los métodos.

La ventaja importante del uso de interfaces es que una clase interface puede ser implementada por cualquier número de clases, consintiendo a cada clase compartir el interfaz de programación sin tener que ser consciente de la implementación que hagan las otras clases que implementen el interface.

```
class MiOtraClase implements VideoClip {
        void play() {
            <código nuevo>
            }
        void bucle() {
            <código nuevo>
            }
        void stop() {
            <código nuevo>
            }
```

Cabe señalar, que el aspecto más importante del uso de interfaces es que combinados objetos de clases diferentes pueden ser tratados como si fuesen de un mismo tipo común, donde este tipo viene indicado por el nombre del interfaz.

Aunque se puede considerar el nombre del interfaz como un tipo de prototipo de referencia a objetos, no se pueden instanciar objetos en sí del tipo interfaz. La definición de un interfaz no tiene constructor, por lo que no es posible invocar el operador new sobre un tipo interfaz.

Conjuntamente entre ellos pueden consentirse la herencia de varios interfaces sin ningún problema. Por el contrario, una clase simplemente heredará de una clase base, pero puede implementar varios interfaces. También, el JDK brinda la posibilidad de crear un interfaz vacío, como es el caso de Serialize, que permite serializar un objeto. Un interfaz vacío se puede utilizar como un flag, un marcador para marcar a una clase con una propiedad determinada.

```
public interface Runnable {
    public abstract void run();
}
```

Las clases que implementen (implements) el interface Runnable han de definir obligatoriamente la función run.

```
class Animacion implements Runnable{
//..
        public void run(){
                //define la función run
        }
}
```

El papel del interface es el de representar algunas de las particularidades de una clase. Por ejemplo, el hecho de que una persona sea un futbolista no define su personalidad completa, pero hace que posea ciertas características que las diferencian de otras.

Clases que no están relacionadas pueden implementar el interface *Runnable*, por ejemplo, una clase que describa una animación, y también puede implementar el interface *Runnable* una clase que realice un cálculo intensivo.

## Diferencias entre un interface y una clase abstracta

Un interface es sencillamente un cuadro de métodos no implementados, también puede incluir la declaración de constantes. Una clase abstracta puede contener métodos implementados y no implementados o abstractos, miembros dato constantes y otros no constantes.

Sin embargo, la diferencia es mucho más profunda. Imaginemos que Runnable fuese una clase abstracta. Un applet definido por la clase MiApplet que moviese una figura por su área de trabajo, derivaría a la vez de la clase base Applet (que describe la funcionalidad mínima de un applet que se ejecuta en un navegador) y de la clase Runnable. Pero el lenguaje Java no tiene herencia múltiple.

En el lenguaje Java la clase MiApplet deriva de la clase base Applet e implementa el interface Runnable

```
class MiApplet extends Applet implements Runnable{
//…
//define la función run del interface
        public void run(){
                //…
        }
//redefine paint de la clase base Applet
        public void paint(Graphics g){
                //…
        }
//define otras funciones miembro
}
```

Una clase sin más puede derivar **extends** de una clase base, pero puede implementar diversos interfaces. Los nombres de los interfaces se ponen separados por una coma después de la palabra reservada **implements**.

El lenguaje Java no fuerza por tanto, una relación jerárquica, simplemente permite que clases no relacionadas puedan tener algunas características de su comportamiento similares.

## Los interfaces y el polimorfismo

En el lenguaje C++, es viable la herencia múltiple, pero este tipo de herencia presenta dificultades. Por ejemplo, cuando dos clases B y C derivan de una clase base A, y a su vez una clase D deriva de B y C. Este problema es conocido con el nombre de diamante.

En el lenguaje Java solamente existe la herencia simple, pero las clases pueden implementar interfaces. Comprobaremos en esta unidad que la importancia de los interfaces no reside en solventar los dificultades inherentes a la herencia múltiple sin forzar relaciones jerárquicas, sino es el de aumentar el polimorfismo del lenguaje más allá del que proporciona la herencia simple.

Para explicar este aspecto importante y novedoso del lenguaje Java adaptaremos los ejemplos que aparecen en el artículo del Bill Venners «Designing with interfaces» publicado en Java World (www.javaWorld.com) en Diciembre de 1998. Comparemos la herencia simple mediante un ejemplo similar al de la jerarquía de las figuras planas, con los interfaces.

## Herencia simple

Creamos una clase abstracta denominada Animal de la cual deriva las clases Gato y Perro. Ambas clases redefinen la función habla declarada abstracta en la clase base Animal.

```
public abstract class Animal {
    public abstract void habla();
}

class Perro extends Animal{
```

```
    public void habla(){
        System.out.println("¡Guau!");
    }
}

class Gato extends Animal{
    public void habla(){
        System.out.println("¡Miau!");
    }
}
```

El polimorfismo nos permite pasar la referencia a un objeto de la clase Gato a una función hazleHablar que conoce al objeto por su clase base Animal:

```
public class PoliApp {
  public static void main(String[] args) {
    Gato gato=new Gato();
    hazleHablar(gato);
 }

  static void hazleHablar(Animal sujeto){
    sujeto.habla();
  }
}
```

El compilador no sabe exactamente que objeto se le pasará a la función hazleHablar en el momento de la ejecución del programa. Si se pasa un objeto de la clase Gato se imprimirá ¡Miau!, si se pasa un objeto de la clase Perro se imprimirá ¡Guau!. El compilador solamente sabe que se le pasará un objeto de alguna clase derivada de Animal. Por tanto, el compilador no sabe que función habla será llamada en el momento de la ejecución del programa.

El polimorfismo nos ayuda a hacer el programa más flexible, por que en el futuro podemos añadir nuevas clases derivadas de Animal, sin que cambie para nada el método hazleHablar. Como ejercicio, se sugiere al lector añadir la clase Pajaro a la jerarquía, y pasar un objeto de dicha clase a la función hazleHablar para que se imprima ¡pio, pio, pio ..!.

## Interfaces

Vamos a crear un interface denominado Parlanchin que contenga la declaración de una función denominada habla.

```
public interface Parlanchin {
    public abstract void habla();
}
```

Hacemos que la jerraquía de clases que deriva de *Animal* implemente el interface *Parlanchin:*

```
public abstract class Animal implements Parlanchin{
    public abstract void habla();
}

class Perro extends Animal{
```

```
    public void habla(){
        System.out.println("¡Guau!");
    }
}

class Gato extends Animal{
    public void habla(){
        System.out.println("¡Miau!");
    }
}
```

Ahora veamos otra jerarquía de clases completamente distinta, la que deriva de la clase base Reloj. Una de las clases de dicha jerarquía Cucu implementa el interface Parlanchin y por tanto, debe de definir obligatoriamente la función habla declarada en dicho interface.

```
public abstract class Reloj {
}

class Cucu extends Reloj implements Parlanchin{
    public void habla(){
        System.out.println("¡Cucu, cucu, ..!");
    }
}
```

Definamos la función hazleHablar de modo que conozca al objeto que se le pasa no por una clase base, sino por el interface Parlanchin. A dicha función le podemos pasar cualquier objeto que implemente el interface Parlanchin, este o no en la misma jerarquía de clases.

```
public class PoliApp {

  public static void main(String[] args) {
    Gato gato=new Gato();
    hazleHablar(gato);
    Cucu cucu=new Cucu();
    hazleHablar(cucu);
  }

  static void hazleHablar(Parlanchin sujeto){
    sujeto.habla();
  }
}
```

Al ejecutar el programa, veremos que se imprime en la consola ¡Miau!, porque a la función hazleHablar se le pasa un objeto de la clase Gato, y después ¡Cucu, cucu…! por que a la función hazleHablar se le pasa un objeto de la clase Cucu.

Si solamente hubiese herencia simple, Cucu tendría que derivar de la clase Animal (lo que no es lógico) o bien no se podría pasar a la función hazleHablar. Con interfaces, cualquier clase en cualquier familia puede implementar el interface Parlanchin, y se podrá pasar un objeto de dicha clase a la función hazleHablar. Esta es la razón por la cual los interfaces proporcionan más polimorfismo que el que se puede obtener de una simple jerarquía de clases.

## 2.1.2. Distinción de los tipos de datos y clases

### Tipos de datos

- Un Tipo de Datos es una colección de valores.
- Han sido estudiados los tipos de datos que implementan lenguajes como ADA o Java (Boolean, Integer, Character…).
- Estos tipos son conocidos como “tipos simples”.
- Estos tipos pueden ser utilizados en nuestros programas sin necesidad de que los detalles sobre su implementación sean conocidos.

En la tabla siguiente se definen los Tipos de datos:

| Tipos de datos en Java | | | | |
|---|---|---|---|---|
| | **Nombre** | **Tipo** | **Ocupa** | **Rango aproximado** |
| **Tipos primitivos** (sin métodos; no son objetos; no necesitan una invocación para ser creados) | byte | Entero | 1 byte | -128 a 127 |
| | short | Entero | 2 bytes | -32768 a 32767 |
| | int | Entero | 4 bytes | 2*109 |
| | long | Entero | 8 bytes | Muy grande |
| | float | Decimal simple | 4 bytes | Muy grande |
| | double | Decimal doble | 8 bytes | Muy grande |
| | char | Carácter simple | 2 bytes | --- |
| | boolean | Valor true o false | 1 byte | --- |

| Tipos de datos en Java | | | | |
|---|---|---|---|---|
| | **Nombre** | **Tipo** | **Ocupa** | **Rango aproximado** |
| **Tipos objeto** (con métodos, necesitan una invocación para ser creados) | Tipos de la biblioteca estándar de Java | String (cadenas de texto)<br>Muchos otros (p.ej. Scanner, TreeSet, ArrayList…) | | |
| | Tipos definidos por el programador / usuario | Cualquiera que se nos ocurra, por ejemplo Taxi, Autobus, Tranvia | | |
| | arrays | Serie de elementos o formación tipo vector o matriz. Lo consideraremos un objeto especial que carece de métodos. | | |
| | Tipos envoltorio o wrapper (Equivalentes a los tipos primitivos pero como objetos.) | Byte | | |
| | | Short | | |
| | | Integer | | |
| | | Long | | |
| | | Float | | |
| | | Double | | |
| | | Character | | |
| | | Boolean | | |

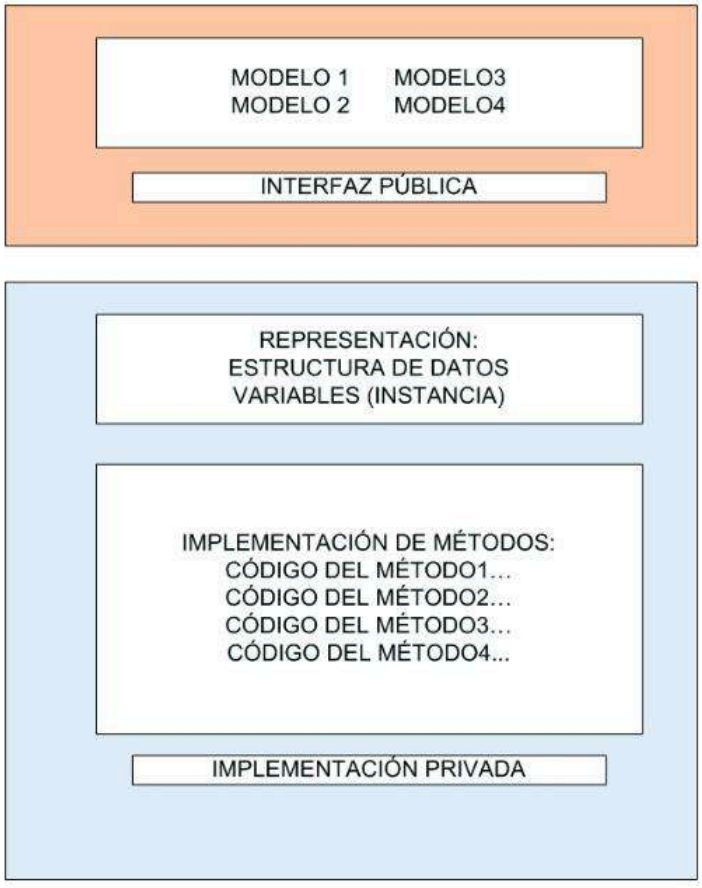
Algunos lenguajes de programación poseen características que nos permiten ampliar el lenguaje añadiendo sus propios tipos de datos. Un tipo de dato definido por el programador se denomina **tipo abstracto de datos (TAD)** para diferenciarlo del tipo fundamental (predefinido) de datos. Por ejemplo, en Java, el tipo Punto, que constituye las coordenadas x e y de un sistema de coordenadas rectangulares, no existe. Sin embargo, es viable implementar el tipo abstracto de datos, considerando los valores que se almacenan en las variables y qué operaciones están servibles para manejar estas variables. En esencia, un tipo abstracto es un tipo de dato que consta de datos (estructuras de datos propias) y operaciones que se pueden realizar sobre ellos. Un TAD se compone de estructuras de datos y los procedimientos o funciones que manipulan esas estructuras de datos.

Sabías que

Un tipo abstracto de datos puede definirse mediante la ecuación:

**TAD = Representación (datos) + Operaciones (funciones y procedimientos)**

La estructura de un tipo abstracto de dato (clase), desde un punto de vista global, se compone de la interfaz y de la implementación.




*Estructura de un tipo abstracto de datos (TAD)*

Las estructuras de datos reales preferidas para acumular la representación de un tipo abstracto de datos son invisibles a los usuarios o clientes. Los algoritmos utilizados para implementar cada una de las operaciones de los TAD están encapsuladas dentro de los propios TAD. La característica de ocultamiento de la información representa que los objetos poseen interfaces públicas. Sin embargo, las representaciones e implementaciones de esas interfaces son privadas.

### Ventajas de los tipos abstractos de datos

Un tipo abstracto de datos es un modelo (estructura) con un número de operaciones que afectan a ese modelo. Los tipos abstractos de datos suministran nutridos beneficios al programador, que se pueden simplificar en los siguientes:

– Admiten una mejor conceptualización y modelización del mundo real. Optimizan la representación y la comprensibilidad. Depuran los objetos basados en estructuras y comportamientos comunes.

– Mejoran la robustez del sistema. Si hay características subyacentes en los lenguajes, permiten la especificación del tipo de cada variable. Los tipos abstractos de datos permiten la comprobación de tipos para evitar errores de tipo en tiempo de ejecución.

– Mejoran el rendimiento (prestaciones). Para sistemas tipeados (tipificados), el conocimiento de los objetos permite la optimización de tiempo de compilación.

– Separan la implementación de la especificación. Permiten la modificación y la mejora de la implementación sin afectar la interfaz pública del tipo abstracto de dato.

– Permiten la extensibilidad del sistema. Los componentes de software reutilizables son más fáciles de crear y mantener.

– Recogen mejor la semántica del tipo. Los tipos abstractos de datos agrupan o localizan las operaciones y la representación de atributos.

Un programa que maneja un TAD lo hace teniendo en cuenta las operaciones o la funcionalidad que tiene, sin interesarse por la representación física de los datos. Es decir, los usuarios de un TAD se comunican con éste a partir de la interfaz que ofrece el TAD mediante funciones de acceso. Podría cambiarse la implementación del tipo de dato sin afectar al programa que usa el TAD ya que para el programa está oculta.

Las unidades de programación de lenguajes que pueden implementar un TAD reciben distintos nombres:

– Modula–2 módulo

– Ada paquete

– C++ clase

– Java clase

En estos lenguajes se definen la especificación del TAD, que declara las operaciones y los datos, y la implementación, que muestra el código fuente de las operaciones, que permanece oculto al exterior del módulo.

## Especificación de los TAD

El fin de la especificación es detallar el comportamiento del TAD; constituye dos partes, la descripción matemática del conjunto de datos y la de las operaciones concretadas en ciertos componentes de ese conjunto de datos.

La especificación del TAD puede poseer un enfoque informal, que detalla los datos y las operaciones relacionadas en lenguaje natural. Otro enfoque más riguroso, la especificación formal, supone suministrar un conjunto de axiomas que describen las operaciones en su aspecto sintáctico y semántico.

### Especificación informal de un TAD

Consta de dos partes:

– Detallar en los datos del tipo los valores que pueden tomar.

– Describir las operaciones relacionándolas con los datos.

El formato que generalmente se emplea, primero especifica el nombre del TAD y los datos: TAD nombre del tipo (valores y su descripción)

A continuación cada una de las operaciones con sus argumentos, y una descripción funcional en lenguaje natural, con este formato: Operación(argumentos)

## Descripción funcional

Como ejemplo, se va a especificar el tipo abstracto de datos Conjunto:

– TAD Conjunto (colección de elementos sin duplicidades, pueden estar en cualquier orden, se usa para representar los conjuntos matemáticos con sus operaciones).

– Operaciones, se ponen las operaciones básicas sobre conjuntos:

  · Conjuntovacio.

    Crea un conjunto sin elementos.

  · Añadir(Conjunto, elemento).

    Comprueba si el elemento forma parte del conjunto; en caso negativo, es añadido. La operación modifica al conjunto.

  · Retirar(Conjunto, elemento).

    Si el elemento pertenece al conjunto, es eliminado de éste. La operación modifica al conjunto.

  · Pertenece(Conjunto, elemento).

    Verifica si el elemento forma parte del conjunto, en cuyo caso devuelve cierto.

  · Esvacio(Conjunto).

    Verifica si el conjunto no tiene elementos, en cuyo caso devuelve cierto.

  · Cardinal(Conjunto).

    Devuelve el número de elementos del conjunto.

  · Union(Conjunto, Conjunto).

    Realiza la operación matemática de la unión de dos conjuntos. La operación devuelve un conjunto con los elementos comunes y no comunes a los dos conjuntos.

    Se pueden especificar más operaciones sobre conjuntos, todo dependerá de la aplicación que se quiera dar al TAD.




La especificación informal de un TAD tiene como objetivo describir los datos del tipo y las operaciones según la funcionalidad que tienen. No sigue normas rígidas al hacer la especificación, simplemente indica, de forma comprensible, la acción que realiza cada operación.

## Tipos abstractos de datos (TAD)

Encontramos varias definiciones para el significado de Tipo Abstracto de Datos (TAD):

- TAD: Conjunto de Operaciones.
- TAD: Modelo matemático con una serie de operaciones concretadas en ese modelo.
- TAD: Tipo de datos específico de forma única mediante un tipo y un conjunto de operaciones definidas sobre el tipo.

- Un Tipo Abstracto de Datos es una abstracción donde se localizan encapsulados los estados potenciales en los que se puede encontrar una entidad de ese tipo y las operaciones que pueden realizarse sobre ella.
- Abstraer: Separar por medio de una operación intelectual las condiciones de un objeto para considerarlas aisladamente o para considerar el mismo objeto en su pura esencia o noción.

  Como se ha mencionado, se trata de una abstracción.

  No se incluyen detalles sobre la implementación de las operaciones.
- Los TAD son autónomos por completo de la implementación.

  En muchos textos, pueden hallarse confundidos los términos TAD y Tipo de Datos, así como TAD y Estructura de Datos.
- Estructura de Datos: Conjunto de variables que se hallan relacionadas.
- Con Estructura de Datos, por tanto, nos referimos a la implementación física de un TAD.

– El encapsulamiento y la ocultación de información son atributos intrínsecos del diseño.

– Un TAD tiene estas propiedades:

  · Encapsulamiento: La información concerniente a la definición del tipo y todas las operaciones que pueden efectuarse sobre el mismo, se encuentran en el mismo lugar.

  · Ocultación de Información: La información acerca de la implementación se encuentra oculta al usuario.

### TAD: Diseño e implementación

Debido a todo lo expuesto, el diseñador de un TAD debe enfrentarse a tres pasos bien diferentes, pero íntimamente afines:

– Análisis de datos y operaciones

– Elección del TAD

– Elección de la implementación

Una especificación algebraica de un TAD tiene dos elementos:

– Signatura (Sintaxis): Se compone de:

  · Definición de los posibles valores del tipo

  · Operaciones definidas

– Axiomas (Semántica): Relaciones y restricciones que se crean sobre el modelo.

### Construcción de TAD en Java

– Esta construcción (la clase) admite, por tanto, encapsular datos y operaciones sobre los mismos, por lo que se revela idónea para crear TADs.

– Cada clase Java pertenece con un fichero, que simboliza la declaración e implementación de un TAD.

– En el archivo que contenga el TAD deben hallarse representados los atributos de la clase (privados), el nombre y forma de las operaciones que se exportan (su interfaz), y su implementación.

```
public class Conjunto {
private char contenedor[ ];
private int tam;
public Conjunto(){
contenedor = new char[256];
tam = 0;
}
public Conjunto(Conjunto S){
contenedor = new char[256];
for (int i = 0; i < S.length; i++)
contenedor[i] = S.contenedor[i];
tam = S.tam;
}
```

### Implementación de las clases

El código fuente de la definición de una clase, con todos sus métodos y variables instancia se recopilan en archivos de texto con extensión .java y el nombre de la clase, por ejemplo, Racional.

Habitualmente, se sitúa la implementación de cada clase en un archivo independiente.

Las clases pueden provenir de diferentes fuentes:

- Se pueden declarar e implementar sus propias clases. El código fuente siempre estará utilizable; pueden estar constituidas por paquetes.
- Se pueden usar clases que hayan sido escritas por otras personas o inclusive que se hayan comprado. En este caso, se puede disponer del código fuente o estar limitado a utilizar el bytecode de la implementación. Será necesario disponer del paquete donde se encuentran.
- Se pueden utilizar clases de los diversos packages que acompañan a su software de desarrollo Java.

### Clases públicas

La declaración de una clase puede incluir el modificador public como prefijo en la cabecera de la clase. Por ejemplo:

```
public class Examen
{
// miembros de la clase
}
```

La clase Examen puede ser utilizada por las clases que se encuentran en su mismo archivo (package), o por clases de cualquier otro paquete o archivo. Habitualmente, las clases se definen como public, a no ser que se quiera restringir su uso a las clases del paquete. Una clase declarada sin el prefijo public establece una restricción importante, y es que sólo podrá ser utilizada por las clases definidas en el mismo paquete.

### Advertencia

El especificador de acceso public es el único que se puede especificar en la cabecera de una clase.

### Paquetes

Los paquetes son la manera que tiene Java de organizar los archivos con las clases necesarias para desarrollar las aplicaciones. Java agrega varios paquetes, por ejemplo java.lang o java.io, con las clases básicas para construir programas: System, String, Integer…

## Sentencia package

¿Cómo se puede detallar un paquete? La sentencia package se utiliza para este labor. En primer lugar se debe incluir la sentencia package como inicial línea del archivo fuente de cada una de las clases del paquete. Por ejemplo, si las clases Lapiz, Boligrafo y Folio se van a establecer formando el paquete escritorio, el esquema a seguir es el siguiente:

```
// archivo fuente Lapiz.java
package escritorio;
public class Lapiz
{
// miembros de clase Lapiz
} // archivo fuente Boligrafo.java
package escritorio;
public class Boligrafo
{
// miembros de clase Boligrafo
}
// archivo fuente Folio.java
package escritorio;
public class Folio
{
// miembros de clase Folio
}
```

Tipos de datos: clases y objetos

### Formato

```
package NombrePaquete;
```

En segundo lugar, una vez elaborado el archivo fuente de cada clase del paquete, conviene situar cada uno en un subdirectorio con el mismo nombre que el del paquete. En el esquema anterior se ubicarán los archivos Lapiz.java, Boligrafo.java y Folio.java en el camino escritorio.

El uso de paquetes tiene dos beneficios importantes:

– Las restricciones de visibilidad son menores entre clases que están dentro del mismo paquete.

  Desde cualquier clase de un paquete, los miembros protected y los miembros sin modificador de visibilidad son accesibles, pero no lo son desde clases de otros paquetes.

– La selección de las clases de un paquete se puede abreviar con la sentencia import del paquete.

### Import

Las clases que se hallan en los paquetes se identifican utilizando el nombre del paquete, el selector punto (.) y, a continuación, el nombre de la clase. Por ejemplo, la declaración de la clase Arte con atributos de la clase PrintStream (paquete java.io) y Lapiz (paquete escritorio):

```
public class Arte
{
private java.io.PrintStream salida;
private escritorio.Lapiz p;
}
```

La sentencia import facilita la selección de una clase, permite escribir únicamente su nombre, evitando el nombre del paquete. La declaración anterior se puede abreviar:

```
import java.io.PrintStream;
import escritorio.*;
public class Arte
{
private PrintStream salida;
private Lapiz p;
}
```

La sentencia import debe aparecer primeramente de la declaración de las clases, a continuación de la sentencia package. Tiene dos formatos:

### Formato

```
import identificadorpaquete.nombreClase;
import identificadorpaquete.*;
```

El primer formato detalla una clase concreta. El segundo formato muestra que para todas las clases de un paquete no hace falta cualificar el nombre de la clase con el nombre del paquete.

## Estructuras de datos en Java

Con reiteración se utiliza el formato .*. Tiene la ventaja de conseguir reducir cualquier clase del paquete, pero se pueden señalar los siguientes problemas:

– Se desconoce qué clases concretas del paquete se están utilizando. Por contra, con una sentencia import por cada clase utilizada se conocen las clases empleadas.

– Puede haber colisiones entre nombres de clases declaradas en el archivo y nombres de clases del paquete.

– Mayor tiempo de compilación, debido a que el compilador busca la existencia de cualquier clase en el paquete.

No obstante si aparece la sentencia import paquete.*, el compilador genera bytecode sólo para las clases utilizadas.

## Ejemplo

Se establece el paquete numerico con la clase Random y se usa la clase en una aplicación.

```
package numerico;
public Random
{
// ...
}
```

Al utilizar la clase en otro archivo:

```
import java.util.*
import numerico.*;
```

En el paquete java.util se halla la clase Random, por ello se origina una ambigüedad con la clase Random del paquete numerico. Es forzoso calificar completamente el nombre de la clase Random de, por ejemplo, java.util.

```
java.util.Random aleatorio; // define una referencia
```

## Constructores

Un constructor es un método que se ejecuta automáticamente cuando se crea un objeto de una clase. Vale para inicializar los miembros de la clase.

El constructor posee el mismo nombre que la clase. Cuando se concreta no se puede especificar un valor de retorno, nunca devuelve un valor. Sin embargo, puede tomar cualquier número de argumentos.

### Reglas

– El constructor tiene el mismo nombre que la clase.

– Puede tener cero o más argumentos.

– No tiene tipo de retorno.

### Ejemplo

La clase Rectangulo tiene un constructor con cuatro parámetros.

```
public class Rectangulo
{
private int izdo;
private int superior;
```

```
private int dcha;
private int inferior;
// constructor
public Rectangulo(int iz, int sr, int d, int inf)
{
izdo = iz;
superior = sr;
dcha = d;
inferior = inf;
}
// definiciones de otros métodos miembro
}
```

Al establecer un objeto, se pasan los valores de los argumentos al constructor con la idéntica sintaxis que la de llamada a un método. Por ejemplo:

```
Rectangulo Rect = new Rectangulo(25, 25, 75, 75);
```

Se ha elaborado una instancia de Rectangulo traspasando valores concretos al constructor de la clase, y de esta manera queda inicializado.

### Constructor por defecto

Un constructor que no tiene parámetros se llama constructor por defecto. Un constructor por defecto normalmente inicializa los miembros dato de la clase con valores por defecto.

### Regla

Java establece automáticamente un constructor por defecto cuando no existen otros constructores.

Dicho constructor inicializa las variables de tipo numérico (int, float…) a cero, las variables de tipo boolean a true y las referencias a null.

### Ejemplo

El constructor por defecto inicializa x e y a 0.

```
public class Punto
{
private int x;
private int y;
public Punto() // constructor por defecto
{
x = 0;
44 Estructuras de datos en Java
y = 0;
}
}
```

Cuando se crea un objeto Punto, sus miembros dato se inicializan a 0.

```
Punto P1 = new Punto() ; // P1.x == 0, P1.y == 0
```

### Precaución

Hay que tener especial precaución con la escritura de una clase con sólo un constructor con argumentos. Si se prescinde un constructor sin argumento, no será viable utilizar el constructor por defecto.

La definición NomClase c = new NomClase() no será posible.

### Constructores sobrecargados

Al igual que se puede sobrecargar un método de una clase, igualmente se puede sobrecargar el constructor de una clase. De hecho, los constructores sobrecargados son bastante frecuentes y proporcionan diferentes alternativas para inicializar objetos.

### Regla

Para prevenir a los usuarios de la clase de crear un objeto sin parámetros, se puede:

– Omitir el constructor por defecto

– O bien, hacer el constructor privado.

### Ejemplo

La clase EquipoSonido se define con tres constructores: uno por defecto, otro con un argumento de tipo cadena y el tercero con tres argumentos.

```
public class EquipoSonido
{
private int potencia;
private int voltios;
```

```
private int numCd;
private String marca;
public EquipoSonido() // constructor por defecto
{
marca = “Sin marca”;
System.out.println(“Constructor por defecto”);
}
public EquipoSonido(String mt)
{
marca = mt;
System.out.println(“Constructor con argmto cadena “);
}
public EquipoSonido(String mt, int p, int v)
{
marca = mt;
potencia = p;
voltios = v;
numCd = 20;
System.out.println(“Constructor con tres argumentos “);
}
public double factura(){...}
};
```

La instanciación de un objeto EquipoSonido puede hacerse llamando a cualquier constructor:

```
EquipoSonido rt, gt, ht; // define tres referencias
rt = new EquipoSonido(); // llamada al constructor por defecto
gt = new EquipoSonido(“JULAT”);
rt = new EquipoSonido(“PARTOLA”,35,220)
```

## Recolección de objetos

En Java, un objeto continuamente ha de estar referenciado por una variable; en el instante en que un objeto deja de quedar referenciado, se activa la rutina de recolección de memoria, se puede expresar que el objeto es liberado, y la memoria que ocupa puede ser reutilizada. Por ejemplo:

```
Punto p = new Punto(1,2);
```

La sentencia p = null provoca que el objeto Punto sea liberado automáticamente.

El propio sistema se encarga de recolectar los objetos en desuso para aprovechar la memoria ocupada. Para ello, hay un proceso que se activa periódicamente y toma los objetos que no están referenciados por ninguna variable. El proceso lo realiza el método System.gc (garbage collection). Por ejemplo, el siguiente método crea objetos Contador que se liberan al perder su referencia.

```
void objetos()
{
Contador k, g, r, s;
// se crean cuatro objetos
k = new Contador();
g = new Contador();
r = new Contador();
s = new Contador();
/* la siguiente asignación hace que g referencie al mismo objeto que k,
además el objeto original de g será automáticamente recolectado. */
g = k;
/* ahora no se activa el recolector porque g sigue apuntando al objeto.
*/
k = null;
/* a continuación sí se activa el recolector para el objeto original de r. */
r = new Contador();
} // se liberan los objetos actuales apuntados por g, r, s
```

## Método finalize()

El método finalize() es exclusivo, se llama automáticamente si ha sido determinado en la clase, justo antes que la memoria del objeto recolectado vaya a ser restablecida al sistema. El método no es un destructor del objeto, no libera memoria; en algunas aplicaciones, se puede utilizar para liberar ciertos recursos del sistema.

### Regla

finalize() es un método especial con estas características:

– No devuelve valor, es de tipo void.

– No tiene argumentos.

– No puede sobrecargarse.

– Su visibilidad es protected.

### Ejercicio

Se expresan dos clases, cada una con su método finalize(). El método main() establece objetos de ambas clases; las variables que referencian a los objetos se cambian para que cuando se active la recolección automática de objetos se libere la memoria de estos; hay una llamada a System.gc() para no esperar a la llamada interna del sistema.

```
class Demo
{
private int datos;
public Demo(){datos = 0;}
protected void finalize()
{
System.out.println(“Fin de objeto Demo”);
}
}
class Prueba
{
private double x;
public Prueba (){x = -1.0;}
protected void finalize()
{
System.out.println(“Fin de objeto Prueba”);
}
}
public class ProbarDemo
{
public static void main(String[] ar)
{
Demo d1, d2;
Prueba p1, p2;
d1 = new Demo();
p1 = new Prueba();
System.gc(); // no se libera ningún objeto
Tipos de datos: Clases y objetos 47
p2 = p1;
p1 = new Prueba();
System.gc(); // no se libera ningún objeto
p1 = null;
d1 = new Demo();
System.gc(); // se liberan dos objetos
d2 = new Demo();
} // se liberan los objetos restantes
}
```

Miembros static de una clase

Cada instancia de una clase, cada objeto, tiene su propia copia de las variables de la clase. Cuando interese que haya miembros que no estén ligados a los objetos sino a la clase y, por tanto, sean comunes a todos los objetos, estos se declaran static.

## Variables static

Las variables de clase static son compartidas por todos los objetos de la clase. Se expresan de igual forma que otra variable, añadiendo como prefijo la palabra reservada static.

## Ejemplo

```
public class Conjunto
{
private static int k = 0;
static Totem lista = null;
// ...
}
```

Las variables miembro static no constituyen parte de los objetos de la clase sino de la propia clase.

Dentro de las clase, se accede a ellas de la manera tradicional, simplemente con su nombre. Desde fuera de la clase, se accede con el nombre de la clase, el selector y el nombre de la variable:

```
Conjunto.lista = ... ;
```

Además se puede acceder a través de un objeto de la clase pero no es recomendable, ya que los miembros static no corresponden a los objetos sino a las clases.

## Ejercicio

Dada una clase, se quiere conocer en todo momento los objetos activos en la aplicación. Se declara la clase Ejemplo con un constructor por defecto y otro con un argumento. Ambos incrementan la variable static cuenta en 1. De esa manera, cada nuevo objeto queda contabilizado. También se declara el método finalize(), de tal forma que al activarse cuenta decrece en 1.

El método main() crea objetos de la clase Ejemplo y visualiza la variable que contabiliza el número de sus objetos.

```
class Ejemplo
{
private int datos;
static int cuenta = 0;
public Ejemplo()
{
datos = 0;
cuenta++; // nuevo objeto
}
public Ejemplo(int g)
{
datos = g;
cuenta++; // nuevo objeto
}
// redefinición de finalize()
protected void finalize()
{
System.out.println("Fin de objeto Ejemplo");
cuenta--;
}
}
public class ProbarEjemplo
{
public static void main(String[] ar)
```

```
{
Ejemplo d1, d2;
System.out.println("Objetos Ejemplo: " + Ejemplo.cuenta);
d1 = new Ejemplo();
d2 = new Ejemplo(11);
System.out.println("Objetos Ejemplo: " + Ejemplo.cuenta);
50 Estructuras de datos en Java
d2 = d1;
System.gc();
System.out.println("Objetos Ejemplo: " + Ejemplo.cuenta);
d2 = d1 = null;
System.gc();
System.out.println("Objetos Ejemplo: " + Ejemplo.cuenta);
}
}
```

Una variable static suele inicializarse directamente en la definición. Sin embargo, existe una construcción de Java que permite inicializar miembros static en un bloque de código dentro de la clase; el bloque debe venir precedido de la palabra static. Por ejemplo:

```
class DemoStatic
{
private static int k;
private static double r;
private static String cmn;
static
{
k = 1;
r = 0.0;
cmn = "Bloque";
}
}
```

## Métodos static

Los métodos de las clases se invocan a través de los objetos. En ocasiones importa precisar métodos que queden controlados por la clase, de modo que no haga falta crear un objeto para llamarlos: son los métodos static. Muchos métodos de la biblioteca Java están definidos como static; es, por ejemplo, el caso de los métodos matemáticos de la clase Math: Math.sin(), Math.sqrt().

La llamada a los métodos static se realiza a través de la clase: Nombre-Clase.metodo(), respetando las reglas de visibilidad. También se pueden llamar con un objeto de la clase; pero no es recomendable debido a que son métodos dependientes de la clase y no de los objetos. Los métodos definidos como static no tienen asignada la referencia this, por lo que sólo pueden acceder a miembros static de la clase. Es un error que un método static acceda a miembros de la clase no static.

### Ejemplo

```
class Fiesta
{
int precio;
String cartel;
public static void main(String[] a)
{
cartel = " Virgen de los pacientes";
precio = 1;
…
```

Al compilar da dos errores debido a que desde el método main(), definido como static, se accede a miembros no static.

### Ejemplo

La clase SumaSerie define tres variables static, y un método static que calcula la suma cada vez que se llama.

```
class SumaSerie
{
private static long n;
private static long m;
static
{
n = 0;
m = 1;
}
public static long suma()
{
m += n;
n = m - n;
return m;
}
}
```

## Clase Object

Object es la superclase base de todas las clases de Java; toda clase definida en Java hereda de la clase Object y, en consecuencia, toda variable referencia a una clase se convierte, automáticamente, al tipo Object.

### Ejemplo

```
Object g;
String cd = new String("Barranco la Parra");
Integer y = new Integer(72); // objeto inicializado a 72
g = cd; // g referencia al mismo objeto que cd
g = y; // g ahora referencia a un objeto Integer
```

La clase Object tiene dos métodos importantes: equals() y toString(). Generalmente, se redefinen en las clases para especializarlos.

### equals()

Compara el objeto que hace la llamada con el objeto que se pasa como argumento, devuelve true si son iguales.

### boolean equals(Object k);

El siguiente ejemplo compara dos objetos; la comparación es true si contienen la misma cadena.

```
String ar = new String("Iglesia románica");
String a = "Vida sana";
if (ar.equals(a)) //...no se cumple, devuelve false
toString()
```

Este método construye una cadena, que es la representación del objeto, y devuelve la cadena.

Normalmente, se redefine en las clases para dar así detalles explícitos de los objetos de la clase.

### String toString()

Por ejemplo, un objeto Double llama al método toString() y asigna la cadena a una variable.

```
Double r = new Double(2.5);
String rp;
rp = r.toString();
```

## Operador instanceof

Con periodicidad, se requiere conocer la clase de la que es instancia un objeto. Hay que tener en cuenta que, en las jerarquías de clases, se dan modificaciones automáticas entre clases derivadas y su clase base; en particular, cualquier referencia se puede convertir a una variable de tipo Object.

Con el operador instanceof se establece la clase a la que corresponde un objeto, que tiene dos operandos: el primero es un objeto y, el segundo, una clase. Valora la expresión a true si el primero es una instancia del segundo. La siguiente función tiene una argumento de tipo Object, por lo que puede recibir cualquier referencia, seleccionando la clase a la que pertenece el objeto transmitido (String, Vector,...):

```
public hacer (Object g)
{
if (g instanceof String)
...
else if (g instanceof Vector)
...
```

El operador instanceof es un operador relacional, su evaluación da como resultado un valor de tipo boolean.

## Tipos abstractos de datos en Java

La implementación de un TAD en Java se ejecuta de manera natural con una clase. Dentro de la clase va a residir la representación de los datos junto a las operaciones (métodos de la clase). La interfaz del tipo abstracto queda perfectamente determinada con la etiqueta public, que se aplicará a los métodos de la clase que representen operaciones.

Por ejemplo, si se ha detallado el TAD Punto para constituir la abstracción punto en el espacio tridimensional, la siguiente clase implementa el tipo:

```
class Punto
{
// representación de los datos
private double x, y, z;
// operaciones
public double distancia(Punto p);
public double modulo();
public double anguloZeta();
…
};
```

### Implementación del TAD Conjunto

La implementación de un TAD se efectúa según la especificación realizada del tipo. La clase Conjunto implementa el TAD Conjunto la clase constituye los datos de manera genérica, usa un array para almacenar los elementos, de tipo Object.

Archivo conjunto.java

```
package conjunto;
public class Conjunto
{
static int M = 20; // aumento de la capacidad
private Object [] cto;
private int cardinal;
private int capacidad;
// operaciones
public Conjunto()
{
cto = new Object[M];
cardinal = 0;
capacidad = M;
}
// determina si el conjunto está vacío
public boolean esVacio()
{
return (cardinal == 0);
}
// añade un elemento si no está en el conjunto
public void annadir(Object elemento)
{
if (!pertenece(elemento))
{
/* verifica si hay posiciones libres, en caso contrario amplia el conjunto */
if (cardinal == capacidad)
{
Object [] nuevoCto;
nuevoCto = new Object[capacidad + M];
for (int k = 0; k < capacidad; k++)
nuevoCto[k] = cto[k];
```

```
capacidad += M;
cto = nuevoCto;
System.gc(); // devuelve la memoria no referenciada
54 Estructuras de datos en Java
}
cto[cardinal++] = elemento;
}
}
// quita elemento del conjunto
public void retirar(Object elemento)
{
if (pertenece(elemento))
{
int k = 0;
while (!cto[k].equals(elemento))
k++;
/* desde el elemento k hasta la última posición mueve los elementos
una posición a la izquierda */
for (; k < cardinal ; k++)
cto[k] = cto[k+1];
cardinal--;
}
}
//busca si un elemento pertenece al conjunto
public boolean pertenece(Object elemento)
{
int k = 0;
boolean encontrado = false;
while (k < cardinal && !encontrado)
{
```

```
encontrado = cto[k].equals(elemento);
k++;
}
return encontrado;
}
//devuelve el número de elementos
public int cardinal()
{
return this.cardinal;
}
//operación unión de dos conjuntos
public Conjunto union(Conjunto c2)
{
Conjunto u = new Conjunto();
// primero copia el primer operando de la unión
for (int k = 0; k < cardinal; k++)
u.cto[k] = cto[k];
u.cardinal = cardinal;
// añade los elementos de c2 no incluidos
for (int k = 0; k < c2.cardinal; k++)
u.annadir(c2.cto[k]);
return u;
}
public Object elemento(int n) throws Exception
{
if (n <= cardinal)
return cto[--n];
else
throw new Exception("Fuera de rango");
}
}
```

## 2.2. Análisis de los objetos: Estado, comportamiento e identidad

### Objetos

– Un objeto posee las siguientes características

– Identidad

– Estado

– Comportamiento

– La identidad de un objeto le consiente diferenciarse de otros objetos. El estado de un objeto está fijado por sus datos y el comportamiento del objeto está determinado por los métodos de la clase a la que pertenece.

### Identidad

La identidad de un objeto le identifica unívocamente y no cambia durante la vida del objeto. Es independiente del estado del objeto

### Estado

El estado de un objeto evoluciona en el tiempo, está determinado por los valores de sus atributos. Cada atributo toma un valor en un dominio.

### Comportamiento

Los métodos que concretan el comportamiento de un objeto describen sus acciones y agrupan sus responsabilidades Las acciones de un objeto dependen de su estado y de un estímulo externo, un mensaje remitido por otro objeto El estado y el comportamiento están interrelacionados. Por ejemplo, un avión no puede aterrizar si está en tierra.

Objetos: interfaz vs. implementación.

– Los objetos se relacionan a través de interfaces bien definidas. No es necesario que conozcan los detalles de la implementación de otros objetos.

– La interfaz del objeto está definida por los métodos públicos de la clase.

## Identidad de un objeto

En una clase no se habla de identidad ya que esta existe explícitamente dada por el nombre que le hemos asignado a la clase a la hora de definirla, así, la clase Persona es legítimamente identificada por ese nombre. Sin embargo, para los objetos, dado que pueden ser muchos de una misma clase (en efecto eso es lo que sucede) se hace necesario para el sistema identificar a cada uno de los objetos existentes por separado. De este modo la identidad de un objeto es única (al igual que la de una clase).

Como hablamos ya en Modula, la identidad de un objeto entonces es la dirección de memoria en donde está alojado, es decir, el contenido del puntero que lo registra. Nada tiene que ver la referencia que nosotros hemos asignado para llegar al objeto.

### Ejemplo

```
Persona p= new Persona();

Persona q;

q=p;
```

Entonces tenemos instanciado un objeto de tipo Persona referenciándolo con la variable p. Luego tenemos otra referencia de tipo Persona llamada q y a esta le asignamos el idéntico valor de p.

La instancia de la clase Persona que poseemos, es decir el nuevo objeto parece que tiene dos identidades. Sin embargo no es el caso. Si se puede acceder al objeto tanto con p como con q, quiere decir que es porque dichas variables conocen la identidad del objeto creado y por tanto ambas variables "miran" hacia él (son alias). Si yo cambio el valor de estas variables el objeto seguirá estando en el mismo lugar de memoria y por tanto su identidad no habrá cambiado. Así pues, las referencias no hacen más que decimos donde está un objeto y por tanto estas no son su identidad.

Otro punto significativo al respecto de la identidad, resaltando aún más que esta está dada únicamente por la dirección de memoria que ocupa el objeto, es que si dos objetos tienen los mismos valores en sus atributos (el mismo estado) no son el mismo objeto. Por ejemplo si yo tengo dos objetos de tipo Persona donde ambos tienen como nombre Lucas y como apellido Álvarez, no son el mismo objeto. Eso ustedes ya deberían manejarlo muy bien.

### Ejemplo

```
Persona p= new Persona(“Lucas”,”Álvarez”);
Persona q= new Persona(“Lucas”,”Álvarez”);
if (q==p)
System.out.println(“Son la misma persona!!!”);
else
System.out.println(“No son la misma persona.”);
```

Al compilar el código veremos como este ejercicio y su resultado, aclaran dudas que podemos tener sobre las variables, los objetos y las referencias de memoria. El resultado es que no son la misma persona. La razón es porque en la condición del if lo que estamos comparando son referencias a objetos, es decir, variables que contienen direcciones de memoria, por tanto esa condición será **true** si ambas direcciones son iguales, es decir si p y q son alias a un mismo objeto, y **false** en caso contrario.

De este modo, si nosotros queremos saber si son la misma persona deberíamos comparar el estado de ambos objetos, es decir, sus atributos uno a uno, y para eso deberíamos definirnos una operación que lo haga.

## Atributos de un objeto

Los atributos de un objeto son las variables que puntualizan su estado, comentar que, los datos que puede contener el objeto. En Java especialmente, los atributos son las variables que expresamos en la clase que concreta un tipo de objetos. Por ejemplo, la clase Persona tenía dos variables de tipo String: nombre y apellido. Dichas variables eran dos atributos. Por ello cada instancia de la clase Persona poseerá su propia variable nombre y su propia variable apellido.

En Modula los atributos correspondían a los campos del registro **(record)** que especificaba el tipo de datos que crearíamos. Esto ustedes ya lo tienen claro, y si no es así, repasen lo que vimos en Modula respecto a la teoría de objetos.

### Estado de un objeto

El estado de un objeto está proporcionado por los valores de sus atributos en un instante dado, es decir, es el grupo de valores de los atributos del objeto en un instante de tiempo. Por ejemplo, en el momento presente el estado de un objeto de tipo Persona es: Lucas Álvarez; pero más tarde podría ser Juan Benitez porque hemos cambiado el valor de sus atributos.

### Comportamiento de un objeto

El comportamiento de un objeto está proporcionado por el conjunto de operaciones que el objeto puede establecer. Estas operaciones quedan definidas en la clase que define al tipo de objetos y generalmente tienen como cometido el poder obtener el estado actual de un objeto para consultarlo y/o para modificarlo. Las operaciones también pueden realizar acciones sobre otros objetos. Si ustedes vuelven a la clase Persona y miran todas las operaciones que definimos tienen ahí todo lo que una instancia de esa clase puede hacer, es decir, como puede comportarse.

En Java, las operaciones declaradas en este instante son operaciones de instancia. De la misma manera que con los atributos, cada instancia de una clase poseerá su propio grupo de operaciones encapsulado dentro de sí misma. De este modo, si en la clase Persona yo defino una operación llamada obtenerNombrePersona(), cada instancia de esa clase tendrá una copia de esa operación para sí misma, y de este modo al hacer una invocación a la operación mediante una referencia.

### Ejemplo

```
p.obtenerNombrePersona();
```

El sistema conoce que debe llamar a la operación obtenerNombrePersona que está en el objeto apuntado por p. Esto es algo completamente nuevo porque en Modula no se había explicado ya que su funcionamiento era un tanto peculiar y en aquel en aquel momento no tenía mucho sentido. Ahora se tomará importante conocerlo. En este momento es vital saber que cada instancia tiene entonces sus propios atributos y sus propias operaciones, tomando siempre como base para crearse en memoria la clase que nosotros especificamos en Java.

### 2.2.1. Análisis de mensajes

Mensaje: Es la solicitud de un objeto a otro para requerir la ejecución de alguno de sus métodos o para conseguir el valor de un atributo público.

Estructuralmente, un mensaje consta de 3 partes:

– Identidad del receptor: Nombre del objeto que contiene el método a ejecutar. Objeto receptor.

– Nombre del método a ejecutar: Solo los métodos declarados públicos. Selector o identificador del método a aplicar.

– Lista de Parámetros que recibe el método (cero o mas parámetros). Argumentos.

Su sintaxis algorítmica es:

```
<Variable_Objeto>.<Nombre_Método> ( [<Lista de Parámetros> ] );
```

Cuando el objeto receptor recibe el mensaje, comienza la ejecución del algoritmo contenido dentro del método invocado, recibiendo y/o devolviendo los valores de los parámetros correspondientes, si los tiene ya que son opcionales: ( [ ] )

Mecanismo básico de la computación OO.

Invocación de la aplicación de un método sobre un objeto.

Método reintegro en la clase Cuenta:

```
public double reintegro (double cantidad) {
if (puedoSacar(cantidad))
saldo = saldo – cantidad;
}
```

Mensaje, aplica el método reintegro sobre un objeto cuenta:

```
cuenta.reintegro(600.0);
```

Un mensaje parece una invocación a procedimiento en la que sólo cambia el formato:

- Mensaje → unaCuenta.ingreso (100000)
- Procedimiento → ingreso (unaCuenta,100000)

En una llamada a procedimiento todos los argumentos se tratan del mismo modo.

En un mensaje un argumento posee una naturaleza especial: "objeto receptor"

### 2.2.2. Tipos de métodos y su clasificación: Métodos de acceso, de selección o consulta, de construcción, de destrucción

#### Tipos de métodos

Los métodos de instancia están relacionados con un objeto en particular, mientras que los métodos estáticos o de clase (también denominados métodos compartidos) están asociados a una clase en particular. En una implementación típica, a los métodos de instancia se les pasa una referencia oculta al objeto al que pertenecen, comúnmente denominada this o self (referencias a sí mismo por sus significados en inglés), para que puedan acceder a los datos asociados con el mismo. Un ejemplo típico de un método de clase sería uno que mantuviera la cuenta de la cantidad de objetos creados dentro de esa clase. Los llamados métodos obtener y métodos establecer (en inglés get y set) proveen un mecanismo para leer y modificar (respectivamente) los datos privados que se encuentran almacenados en un objeto o clase.

Algunos lenguajes de programación demandan la definición de constructores, siendo estos métodos de instancia especiales invocados automáticamente cuando se crea una instancia de alguna clase. En Java y C++ se distinguen por poseer el mismo nombre de la clases a la que están asociados. Lenguajes como Smalltalk no requieren constructores ni destructores.

Los métodos de acceso son un tipo de método normalmente pequeño y simple que se limita a proveer información acerca del estado de un objeto. Aunque introduce una nueva dependencia, la utilización de métodos es preferida a acceder directamente a la información para proveer de una nueva capa de abstracción (programación orientada a objetos). Por ejemplo, si una clase que modela una cuenta bancaria provee de un método de acceso "obtenerBalance()" en versiones posteriores de la clase se podría cambiar el código de dicho método substancialmente sin que el código dependiente de la clase tuviese que ser modificado (un cambio sería necesario siempre que el tipo de dato devuelto por el método cambie). Los métodos de acceso que pueden cambiar el estado de un objeto son llamados, frecuentemente, métodos de actualización ó métodos de mutación; a su vez, los objetos que proveen de dichos métodos son denominados óbjetos mutables.

### Métodos de clases

- Por norma, debe evitarse el uso de propiedades públicas. Es preferible definir métodos para modificar el valor de las propiedades de un objeto.
- Los métodos set se utilizan para modificar (escribir) el valor de los atributos de un objeto.
- Los métodos get se utilizan para consultar (leer) el valor de los atributos de un objeto.

```
getNombre()
setNombre()
```

```
public class Persona {
private String dni, nombre, apellidos;
private int añoNacimiento;
public void mostrarNombre() {
System.out.println(nombre + _ _ + apellidos);
}
public setNombre(String nombre) {
this.nombre = nombre;
}
```

```
public set añoNacimiento(int año) {
añoNacimiento = año;
}
}
```

En este ejemplo, el método setNombre(String nombre) recibe el parámetro nombre para modificar el atributo nombre de la clase. La palabra reservada this hace referencia a la instancia de la clase y permite diferenciar estas dos variables. El método setNombre asigna el valor del parámetro nombre al atributo nombre del objeto.

```
this.nombre = nombre;
```

– El método setAñoNacimiento no utiliza this porque el identificador del parámetro (año) es distinto del identificador del atributo de la clase (añoNacimiento).

### Métodos de clases

```
public class Cuenta {
private double saldo, double limiteCredito;
public void ingresar(double cantidad) {
saldo += cantidad;
}
public void retirar(double cantidad) {
saldo -= cantidad;
}
```

```
public void setLimiteCredito(double cantidad) {
limiteCredito = cantidad;
}
public double getLimiteCredito() {
return limiteCredito;
}
}
```

## Constructores

- El operador new crea una instancia de la clase invocando un método especial de la clase, denominado constructor.
- El método constructor inicializa el estado del objeto Cuenta miCuentaCorriente = new Cuenta();
- Por defecto, Java crea un constructor sin parámetros. El valor inicial de los atributos del objeto depende de la inicialización de las variables de instancia Java permite definir métodos constructores para especificar los valores iniciales de los objetos.
- El nombre de los métodos constructores coincide con el nombre de la clase.

```
public class Cuenta {
private double saldo, double limiteCredito;
private static final double LIMITE = 500;
public Cuenta() {
this.balance = 0;
this.limiteCredito = LIMITE;
}
public Cuenta(double limiteCredito) {
this.balance = 0;
```

```
this.limiteCredito = limiteCredito;
}
}

Otro ejemplo Constructores
public class MisCuentas {
public static void main(String[] args) {
Cuenta miCuenta1 = new Cuenta();
Cuenta miCuenta2 = new Cuenta(6000);
System.out.print(_Limite cuenta 1 (normal): _);
System.out.println(miCuenta1.getLimiteCredito());
System.out.print(_Limite cuenta 2 (vip): _);
System.out.println(miCuenta2.getLimiteCredito());
}
}
```

Java permite definir varios constructores diferentes para crear los objetos de la clase. La declaración de los constructores debe ser diferente y deben utilizar distintos parámetros.

En este ejemplo se han definido dos constructores, uno sin parámetros y otro con un parámetro para especificar el límite de crédito de la cuenta

### Referencias a objetos

- Cualquier tipo definido en Java con una clase es un tipo no primitivo.
- Cuando se declara una variable de tipo primitivo, se reserva un espacio de memoria para almacenar su valor.
- Cuando se declara una variable de tipo no primitivo, se reserva un espacio de memoria para almacenar la referencia al objeto, no el objeto en sí. Es por esto que es necesario utilizar el operador new.

## Referencias a objetos

– Cuando se utilizan tipos de datos primitivos las sentencias de asignación copian el valor de las variables:

```
int x = 100;
int y = 200;
int z;
z = y; // z almacena el valor inicial de y (200)
y = x; // y almacena el valor inicial de x (100)
x = z; // el valor de x es 200
```

– Cuando se utilizan tipos de datos no primitivos las sentencias de asignación copian la referencia al objeto:

```
Cuenta miCuenta1 = new Cuenta();
Cuenta miCuenta2 = new Cuenta(6000);
Cuenta miCuenta3;
miCuenta3 = miCuenta2; // se copia la referencia a miCuenta2
miCuenta3.retirar(2000); // hace un retiro de miCuenta2
```

## El uso de métodos

– Los métodos se utilizan para organizar un programa Java de forma lógica.

– El uso de métodos permite fragmentar un problema en problemas más pequeños que son más fáciles de solucionar (“Divide y vencerás”).

– Los métodos encapsulan un conjunto de instrucciones que se puede ejecutar tantas veces como sea necesario. El código del método queda oculto y sólo es necesario conocer su interfaz (parámetros y valor de retorno).

  Un método lleva a cabo una tarea específica de diversa naturaleza: input, output o cálculo

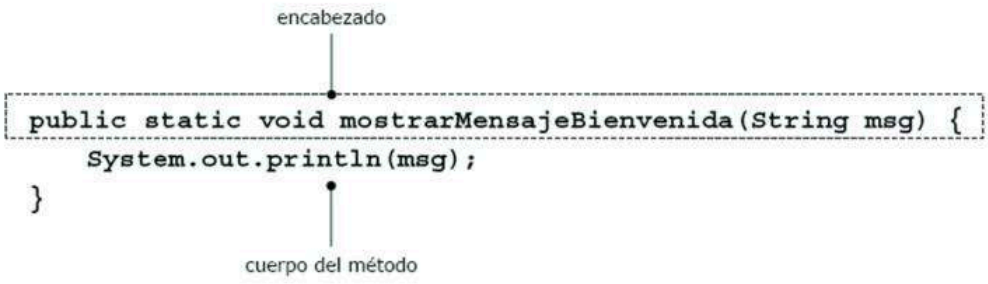
– Para crear un método se tiene que escribir la declaración del método, el encabezado y el cuerpo. Dentro del cuerpo pueden haber declaraciones de variables y constantes. Si las hay se conocen como variables locales y solo pueden ser utilizadas dentro del método

– En otros lenguajes, los métodos se denominan subrutinas o funciones Declaración de un método: Encabezado y cuerpo.

  · Encabezado

```
public static void mostrarMensajeBienvenida(String msg) {
System.out.println(msg);
}
```

  · Cuerpo del método




*Declaración de un Método. Encabezado y cuerpo*




*Encabezado de un método*

## El encabezado de un método

– Modificadores

  · public: el método puede invocarse fuera de la clase.

  · protected: el método puede invocarse fuera de la clase.

  · private: no se puede invocar fuera de la clase.

  · static: el método pertenece a la clase, no a un objeto.

– Tipo del valor de retorno (“Return type”)

  int | long | double | String o cualquier otro tipo de dato para métodos que devuelven valores, void si el método no devuelve un valor.

– Nombre del método

  Debe ser un nombre descriptivo de la tarea que realiza el método.

– Lista de parámetros

  La lista de parámetros es opcional. Contiene la declaración de parámetros (variables) que permiten al método recibir datos del exterior.

## Ejecución de métodos

– Un método se ejecuta cuando es invocado o llamado.

– El método main se ejecuta automáticamente al comenzar la ejecución de un programa.

– Para ejecutar un método basta indicar el nombre del método y la lista de parámetros.

```
mostrarMensajeBienvenida("Hola mundo");
```

Los valores que se pasan a un método cuando es invocado se denominan argumentos:

- Para invocar un método que tiene en su lista de parámetros un valor entero se debe pasar una constante entera, una variable entera o una expresión de tipo entero.

- Los métodos pueden ser de dos tipos: métodos sin valor de retorno o métodos con valor de retorno.

- Los métodos sin valor de retorno se declaran con el valor de retorno void. El método main devuelve void.

- Los métodos con valor de retorno se declaran con el tipo de dato correspondiente: int, char, long, double, String o cualquier otro tipo. Estos métodos utilizan la instrucción return para finalizar la ejecución del método, devolver el valor y el control del programa al punto donde se realizó la llamada.

```
public static int suma (int num1, int num2){
return (num1 + num2);
}
```

El valor de retorno se almacena en la variable entera c:

```
int a = 5, b = 10, c;
c = suma(5, 10);
c= suma(a, b)
```

## Parámetros y argumentos

- El tipo del argumento que se pasa a un método al invocarlo tiene que corresponder con el tipo del parámetro en la declaración.
- La correspondencia entre argumentos y parámetros es uno a uno según el orden.
- En el ejemplo anterior el 5 corresponde con num1 y el 10 con num2. Las constantes 5 y 10 son argumentos, mientras que num1 y num2 son parámetros. Los argumentos se usan en tiempo de ejecución y los parámetros en la definición del método.

Parámetros, argumentos y valor de retorno:

```
public static void main() {
int x, y;
Scanner teclado = new Scanner(System.in);
System.out.print(_Introduzca el valor de x: _);
x = teclado.nextInt();
System.out.print(_Introduzca el valor de y: _);
y = teclado.nextInt();
System.out.print(_La suma es: _ + suma(x, y));
}
```

Otro ejemplo:

```
public static int leerNumero() {
Scanner teclado = new Scanner(System.in);
System.out.printl(_Introduzca un numero: _);
int x = teclado.nextInt();
return x;
```

### Destrucción de objetos

En Java los objetos no se destruyen explícitamente.

Recolección automática de memoria de los objetos no referenciados (Garbage Collector).

Existe un método finalize()

Este método se invocará justo antes de la recolección de basura.

Interesa para liberar recursos (ej. conexión bases de datos).

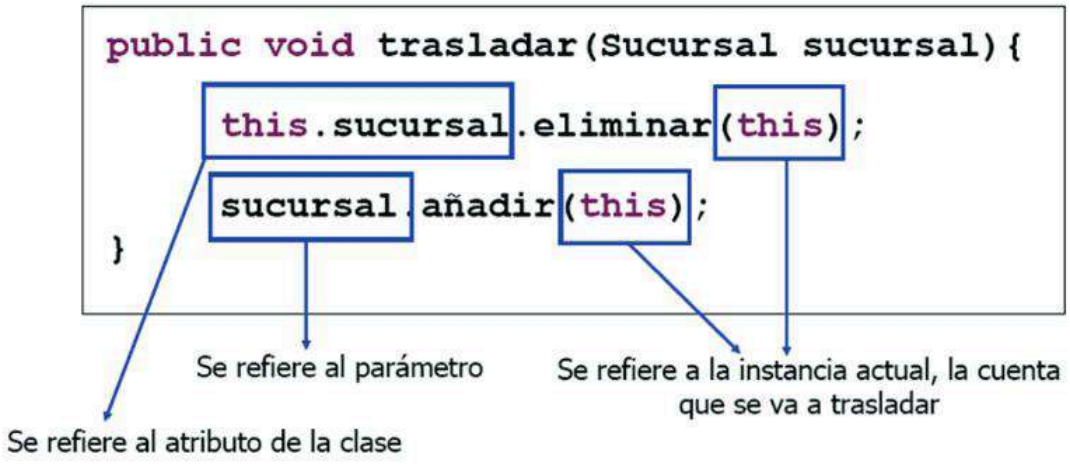
En C++ todos los objetos se destruyen (en un programa sin errores), mientras que en Java no siempre se "recolectan".

## 2.3. Uso de objetos como instancias de clase. Instancia actual (this, self, current)

El lenguaje Java proporciona la palabra clave this que referencia a la instancia actual.

Utilidad:

- Distinguir los atributos de los parámetros y variables locales dentro de la implementación de un método.
- Aplicar un mensaje a otro objeto estableciendo como parámetro la referencia al objeto actual.




*Referencia This*

## Objeto que envia el mensaje: this

this es una referencia al objeto que manda un mensaje, o sencillamente, una referencia al objeto que llama un método (este no debe ser static). Internamente se define:

```
final NombreClase this;
```

Por consiguiente, no puede modificarse. Las variables y los métodos de las clases están referenciados, implícitamente, por this. Pensemos, por ejemplo, en la siguiente clase:

```
class Triangulo
{
private double base;
private double altura;
public double area()
{
return base*altura/2.0 ;
}
}
```

En el método area() se crea referencia a las variables instancia base y altura. El método es común para todos los objetos Triangulo. Supuestamente no distingue entre un objeto u otro, pero cada variable instancia implícitamente está cualificada por this. Es como si estuviera escrito:

```
public double area()
{
return this.base*this.altura/2.0 ;
}
```

Fundamentalmente, this tiene dos usos:

– Seleccionar explícitamente un miembro de una clase con el fin de dar más claridad o de evitar colisión de identificadores.

  Por ejemplo:

```
class Triangulo
{
private double base;
private double altura;
public void datosTriangulo(double base, double altura)
{
this.base = base;
this.altura = altura;
}
// ...
}
```

  Se ha evitado con this la colisión entre argumentos y variables instancia.

– Que un método devuelva el mismo objeto que lo llamó. De esa manera, se pueden hacer llamadas en cascada a métodos de la misma clase. De nuevo se define la clase Triangulo:

```
class Triangulo
{
private double base;
private double altura;
public Triangulo datosTriangulo(double base, double altura)
{
this.base = base;
this.altura = altura;
return this;
}
public Triangulo visualizar()
{
System.out.println(“ Base = “ + base);
System.out.println(“ Altura = “ + altura);
return this;
}
public double area()
{
return base*altura/2.0 ;
}
}
```

Ahora se pueden concatenar llamadas a métodos:

```
Triangulo t = new Triangulo();
t.datosTriangulo(15.0,12.0).visualizar();
```

## 2.4. Identificación del concepto de programa en el paradigma orientado a objetos. POO = objetos + mensajes

### Diseño de programas Java

El diseño de la cabecera del programa ya está hecho, y su nueva oficina funciona. Pero cuando te sientas, observnado fijamente la esquina de la ventana y golpeando su nueva mesa, te cuestionas si es capaz de conservar su nueva posición.

El diseño de los programas en Java no es tarea fácil. Un buen diseño de programación involucra muchos aspectos, y echaremos un vistazo a algunos de ellos en este capítulo, antes de empezar a excavar en la sintaxis de Java. De hecho, uno de los aspectos más importantes de la creación de una nueva aplicación es diseñarla. Si no se selecciona bien puede que sea necesario hacer muchas revisiones del producto. Muchos libros se dedican al diseño de programas. Microsoft, que debería saber algo de ello, divide el proceso en cuatro áreas:

– Rendimiento. Responsabilidad y optimización global de la velocidad y **uso** de recursos.

– Mantenimiento. Capacidad de la aplicación para ser mantenida fácilmente.

– Extensibilidad. Capacidad de la aplicación para ser extendida de formas bien definidas.

– Disponibilidad. Robustez de la implementación de la aplicación y disponibilidad para su uso.

Veamos rápidamente estas cuatro áreas.

### Rendimiento

El rendimiento es un tema de diseño que es duro de argumentar. Si los usuarios no consiguen lo que quieren con su aplicación, esto se convierte claramente un problema. En general, el rendimiento depende de las necesidades de los usuarios. Para algunas personas, la velocidad es esencial; para otros, la robustez o el uso eficiente de los recursos es lo que están buscando. Globalmente, el rendimiento de una aplicación es una indicación de lo bien que responde a las necesidades de los usuarios. He aquí algunos aspectos generales de rendimiento que debería considerar cuando escriba programas Java:

– Eficiencia del algoritmo

– Velocidad de CPU

– Diseño y normalización eficiente de base de datos
– Limitación de accesos externos
– Velocidad de la red
– Temas de seguridad
– Uso de recursos
– Velocidad de acceso a la Web

### Mantenimiento

El mantenimiento es la medida de lo fácilmente que puede adaptarse su aplicación a necesidades futuras. Este asunto se deriva de buenas prácticas de programación, de las que hablaré a lo largo del libro. Buena parte de esto es de sentido común, simplemente tener en mente las necesidades de codificación futura al escribir el código. Algunos puntos de la "programación óptima" son los siguientes:

– Evitar el uso de bucles y condicionales anidados.
– Evitar el paso de variables globales a procedimientos.
– Ser modular cuando se escribe el código.
– Dividir el código en paquetes.
– Documentar los cambios de programa.
– Dar a cada procedimiento un único propósito.
– Asegurarse de que la aplicación puede extenderse sin problemas a más tareas y mayor número de usuarios.
– Planificar la reutilización de código.
– Programar de forma defensiva.
– Uso de procedimientos para el acceso a datos sensibles.
– Uso de comentarios.
– Uso de nombres de variables consistentes.
– Uso de constantes en lugar de números "mágicos".

### Extensibilidad

La extensibilidad es la capacidad de la aplicación para extenderse de una forma bien definida y relativamente fácil. Generalmente, supone una preocupación en las aplicaciones grandes y, con frecuencia, involucra a toda una interfaz especialmente diseñada para la extensión de módulos. De hecho, Java, en sí mismo, está diseñado para ser extendido, usando el framework de extensiones Java.

### Disponibilidad

La disponibilidad es calcular el tiempo que la aplicación puede utilizarse, en comparación con el tiempo que los usuarios quieren utilizarla. Esto lo engobla todo, desde que no se quede congelada cuando se ejecuta una tarea larga (al menos, dar al usuario el estado de la operación), hasta trabajar con técnicas y métodos que no se cuelguen, hacer backups de datos críticos y planificar el uso alternativo de recursos, si es posible, cuando el acceso al recurso deseado esté bloqueado. Globalmente el proceso de diseño es de los que más tiempo requiere. De hecho, todo el ciclo de desarrollo es tema de muchos estudios, puede resultar sorprendente saber que algunos de ellos consideran que el diseño es, al menos, el quince por ciento del proyecto total cuando se prueba un campo y se aumentan planificación, diseño y pruebas de interfaz de usuario.

# UD2
# Lo más importante

- El elemento básico de la programación orientada a objetos en Java es una clase. Una clase define la forma y comportamiento de un objeto.

- Tanto en Java como en C++ la declaración e instanciación de una variable de tipo básico, utiliza una sentencia que asigna un nombre a la variable y reserva memoria para almacenar su contenido.

- La declaración de la clase indica al compilador el nombre de la clase, la clase de la que deriva (su superclase), los privilegios de acceso a la clase (pública, abstracta, final) y si la clase implementa o no, uno o varios interfaces.

- Cada clase Java deriva, directa o indirectamente, de la clase Object. La clase padre inmediatamente superior a la clase que se está declarando se conoce como superclass. Si no se especifica la superclase de la que deriva una clase, se entiende que deriva directamente de la clase Object.

- El propósito de los interfaces es proporcionar nombres, es decir, solamente declara lo que necesita implementar el interfaz, pero no cómo se ha de realizar esa implementación; es una forma de encapsulación de los protocolos de los métodos sin forzar al usuario a utilizar la herencia.

- Las variables pueden declararse dentro del cuerpo de la clase o dentro del cuerpo de un método de la clase. Sin embargo, éstas últimas no son variables miembro de la clase, sino variables locales del método en la que están declaradas.

– Resumiendo varios conceptos En el mundo real, las personas identifican los objetos como cosas que pueden ser percibidas por los cinco sentidos. Los objetos tienen propiedades específicas, como posición, tamaño, color, forma, textura, etc. que definen su estado. Los objetos también poseen ciertos comportamientos que los hacen diferentes de otros objetos.

– Una vez que una clase ha sido definida, un programa puede contener una instancia de la clase, denominada objeto de la clase. Un objeto se crea con el operador new aplicado a un constructor de la clase.

– El operador new crea el objeto y devuelve una referencia al objeto creado. Esta referencia se asigna a una variable del tipo de la clase. El objeto permanecerá vivo siempre que esté referenciado por una variable de la clase que es instancia.

– Uno de los principios fundamentales en programación orientada a objetos es la ocultación de la información, que significa que a determinados datos del interior de una clase no se puede acceder por métodos externos a ella. Por el contrario, los datos o métodos públicos son accesibles desde el exterior de la clase.

# UD2
## Autoevaluación

1. Los objetos son:
    a. Bloques de código fuente.
    b. Funciones de Java.
    c. Todo en Java son objetos.
    d. Instanciamientos de una clase.

2. Una clase en Java:
    a. Define la forma y comportamiento de un objeto.
    b. Es una instancia de un objeto.
    c. Es una forma de variable.
    d. Determina el funcionamiento de un método.

3. La definición de una clase consta de:
    a. Dos partes, la declaración y el cuerpo.
    b. Tres partes declaración, variables y el cuerpo.
    c. Una parte el objeto.
    d. Dos partes el objeto y el método.

4. Todos los objetos en Java heredan, directa o indirectamente:
   a. De la clase Main.
   b. De la clase Object.
   c. De la subclase miembro del objeto.
   d. De si misma.

5. ¿Qué tipos de clases existen?
   a. Public – abstrac – final – syncrhonizable.
   b. Public – prívate – abstrac – final.
   c. Private – public – Object – interface.
   d. Ninguna de las ateriores.

6. Un objeto se crea con el operador:
   a. Create object.
   b. new.
   c. New object().
   d. This.object().

7. Para gestionar el acceso a los miembros de la clase se utilizan:
   a. Public, Object y Class.
   b. public, private y protected.
   c. Public, Private y abstract.
   d. Ninguna de las anteriores.

8. Que tipos de datos son double:
   a. Decimal doble de 8 bits.
   b. Decimal simple de 4 bits.
   c. Entero long de 8 bits.
   d. Entero de 4 bits.

9. No es una ventaja de los tipos abstractos de datos:
    a. Permiten una mejor conceptualización y modelización del mundo real.
    b. Evitan errores en tiempo de compilación.
    c. Mejoran la robustez del sistema.
    d. Separan la implementación de la especificación.

10. No define a un constructor:
    a. Un método que se ejecuta automáticamente.
    b. Sirve para inicializar los miembros de la clase.
    c. El constructor tiene el mismo nombre que la clase.
    d. No puede tomar cualquier número de argumentos.

Área: informática y comunicaciones

# UD3

## Generalización/ especialización: herencia

3.1. Descripción del concepto de herencia: simple y múltiple

- 3.1.1. Relación de herencia: características
- 3.1.2. Reglas y características que definen una relación de herencia: regla “es–un”
- 3.1.3. Transmisión de atributos y métodos
- 3.1.4. Regla de especialización de la superclase en la subclase
- 3.1.5. Acceso a los atributos de una clase y acoplamiento entre las clases
- 3.1.6. Utilización de objetos this (current, self u otros) y super
- 3.1.7. Leyes de Demeter

3.2. Distinción de la herencia múltiple

- 3.2.1. Problemas: Conflictos de nombres, herencia repetida
- 3.2.2. Soluciones

3.3. Creación de objetos en la herencia

3.4. Clasificación jerárquica de las clases

- 3.4.1. Clase raíz
- 3.4.2. Clases abstractas
- 3.4.3. Métodos virtuales
- 3.4.4. Redefinición de métodos

## 3.1. Descripción del concepto de herencia: simple y múltiple

La programación orientada a objetos implanta muchos conceptos que no están presentes en los lenguajes de programación procedimentales. Uno de estos conceptos es la herencia o jerarquía de clases. La herencia es, sin lugar a dudas, una de las características más recias de la orientación a objetos y una de las que permiten diferenciar claramente entre lo que es y no es un lenguaje orientado a objetos.

Veamos un ejemplo para que entienda este importante concepto. Imagine que le han propuesto preparar una aplicación para controlar el pago de las nóminas de los empleados de su empresa.

Usted ha analizado el problema y ha decidido escribir una clase Empleado donde se reflejen las características (propiedades y métodos) que estas personas (objetos) tienen.

```
public class Empleado {
//Propiedades
private String nombre;
private int salario;
private String fechaAlta;
private String fechaBaja;
//Constructor
public Empleado(String nombre, int salario,
String fecha) {
```

```
this.nombre = nombre;
this.salario = salario;
this.fechaAlta = fecha;
this.fechaBaja = null;
}
//Métodos
public void darDeBaja(String fecha)
{
fechaBaja = fecha;
salario = 0;
}
public int calcularSalario()
{
return salario;
}
}
```

Así, ha decidido incluir las propiedades nombre, salario, fechaAlta y fechaBaja, y, siguiendo las reglas de diseño de la orientación a objetos, las ha declarado como privadas para la clase.

También has escrito un método constructor en el que se inicializan las propiedades necesarias al crear un objeto Empleado. Obsérvese cómo, en este caso, es necesario utilizar la palabra this en el método constructor. De esta forma, puede diferenciar entre lo que es la propiedad del objeto (this .nombre) y lo que es el parámetro que se le pasa al constructor (nombre).

Finalmente, has decidido escribir dos métodos: uno que permita indicar que el empleado ha causado baja en la empresa y otro en el que se calcule el salario que hay que pagarle. Todos estos métodos los ha calificado como públicos. Usted está muy contento con la clase que ha escrito, pero ahora llega el jefe de contabilidad y le indica que su aplicación no le sirve para los vendedores que tiene la empresa.

*—¿Cómo que no? —contestas desafiante. —No me sirve porque no puedo calcular la parte de su salario que depende de las ventas efectuadas en un determinado mes, —replica el contable.*

*—Bueno, bueno, ya lo arreglaré, —respondes de forma más amable.*

Parece que no había tenido en cuenta este detalle. Existen vendedores en tu empresa y, como en la mayoría de los casos, tienen una parte variable en su salario que depende directamente de lo que han vendido en el mes.

Su primera reacción es la de crear una nueva clase que represente la existencia de los vendedores en la empresa. Empieza a pensar y decide que estos vendedores tendrán su nombre, su salario fijo, su fecha de alta, su fecha de baja…

Además, podrán darse de baja, deberá calcular su salario a la hora de pagar las nóminas…

En definitiva, se da cuenta de que tienen todas las características que ya había planificado para la clase Empleado. Por otra parte, tendrán otros detalles que los empleados no tienen: el número de ventas realizadas y la comisión que cobran.

¿Existe alguna forma de representar esta situación en la programación orientada a objetos? Sí, a través de la herencia.

## Crear subclases

Desde luego, podrías escribir de nuevo todas las propiedades y los métodos comunes en la clase Vendedor. Pero, ¿qué ocurre si después tiene que introducir más clases de empleados: Mensajero, Ingeniero, etc.?

La programación orientada a objetos permite utilizar el concepto de subclase.

Cuando tu encuentras una clase que especializa otra clase ya existente, entonces puede hacer que la primera sea subclase de la segunda. La ventaja que tiene esto es que la subclase hereda las características de la clase más sencilla y, además, añade nuevas características.

En Java, para indicar que una clase es subclase de otra, se utiliza la palabra clave extends.

```
public class Vendedor extends Empleado {
}
```

De esta forma estamos indicando que la clase Vendedor es una subclase de la clase Empleado.

Esto quiere decir que, automáticamente, los objetos de la clase Vendedor tienen todas las propiedades y métodos de la clase Empleado.

```
public class Vendedor extends Empleado {
private int ventas;
private int comision;
public void vender(int cantidad)
{
ventas = ventas + cantidad;
}
public Vendedor(String nombre, int salario,
String fecha, int comision) {
super(nombre, salario, fecha);
this.ventas = 0;
this.comision = comision;
}
```

Vemos que el vendedor *es un* empleado especial que tiene características que otros empleados no tienen. Por ejemplo, las propiedades **ventas** y **comisión** o el método **vender**.

En toda relación de herencia encontrarás que puede utilizar la expresión **“es un”.**

Obsérvese cómo esto ocurre en nuestro caso: un vendedor “es un” empleado que tiene una parte variable de su sueldo en comisiones. Sin embargo, al revés no es cierto: un empleado no es un vendedor… ya que no todos los empleados tienen por qué ser vendedores.

A la clase más sencilla se la llama clase base o superclase, mientras que aquella clase que la especializa o que le añade nuevas características se llama subclase.

## Crear subobjetos de las subclases

Un asunto delicado ocurre cuando desea crear objetos de una subclase. En este caso, es posible que necesite realizar lo mismo que hacía el constructor de la superclase, más otras tareas adicionales.

Java introduce la palabra clave super para poder acceder a las propiedades y métodos de la superclase.

```
public Vendedor(String nombre, int salario,
String fecha, int comision) {
super(nombre, salario, fecha);
this.ventas = 0;
this.comision = comision;
}
```

El código anterior es el constructor de la clase Vendedor. Obsérvese cómo lo primero que se hace es justamente llamar al constructor de la superclase.

Con la línea super(nombre, salario, fecha) estamos llamando al constructor de la superclase y le pasamos los parámetros que éste necesita. Después, se completa el trabajo para crear un objeto Vendedor.




Si no realizas una llamada explícita al constructor de la superclase, Java lo hará por ti antes de completar el constructor de la subclase. Sin embargo, esto sólo será así cuando el constructor de la superclase no necesite ningún parámetro.

Si en la superclase escribiste un constructor con parámetros, tendrás que llamar al constructor deseado de la superclase en la primera línea del de la subclase.

El lenguaje Java crea siempre un constructor sin parámetros para toda las clases si nosotros no hemos escrito ningún constructor. Es el constructor por defecto. Este constructor por defecto lo único que hace es asignar el valor predeterminado a cada propiedad de la clase. Este valor predeterminado está en función de su tipo de datos. Así, si se trata de una cadena de texto (String), inicializa la propiedad con la cadena vacía; mientras que si es un valor numérico, con el valor 0.

En el momento en que escribamos un constructor, ya no se crea el constructor por defecto.




La palabra clave super puede utilizarse para acceder a las propiedades y métodos de las superclases, no sólo para utilizar el constructor.

## Sobrescribir métodos

Hemos visto que una subclase puede incluir nuevas propiedades e incluso nuevos métodos de forma que amplían las características de su superclase. Sin embargo, en ocasiones esto no es suficiente.

Es posible que, en lugar de añadir nuevos métodos, tenga que modificar el funcionamiento de alguno de los heredados. Esto ocurre en nuestro ejemplo. Recuerda que, a la hora de calcular el salario de un vendedor, no es válido utilizar el método heredado de la clase Empleado, ya que este método no tiene en cuenta las comisiones del vendedor.

Es posible modificar el funcionamiento de un método heredado. Esto se conoce con el nombre sobrescribir o reemplazar los métodos.

```
public int calcularSalario()
{
return (salario + (ventas * comision / 100));
}
```

La cabecera o prototipo del método sobrescrito tiene que coincidir exactamente con el que sobrescribe; es decir, tendrá el mismo nombre, tipo de retorno y parámetros que el de la superclase.




En inglés se utiliza el término overriding para hacer referencia al proceso de sobrescribir métodos.

En el caso de un empleado, el método calcularSalario simplemente devuelve el valor de la propiedad salario. Ahora, sin embargo, el cálculo es más complejo ya que debemos tener en cuenta las comisiones que se lleva el vendedor. Lo que tienes que entender es que cuando creas un objeto Vendedor y utilizas su método calcularSalario, estará utilizando el método sobrescrito y no el heredado Lógicamente, esto no será siempre necesario; es decir, en la mayoría de los casos los métodos heredados serán válidos en las subclases y sólo necesitará sobrescribir aquellos que no deban funcionar del mismo modo.

El modificador de acceso de un método sobrescrito en la subclase puede ser igual o más permisivo que el método que sobrescribe en la superclase, pero no menos permisivo.

Por ejemplo, si el método en la superclase es public, no puede ser private en la subclase.

Sin embargo, el código del método anterior, que sobrescribe al de la clase Empleado no es correcto. Si intentas compilarlo, comprobarás que NetBeans encuentra un error. Nos indica que no podemos acceder a la propiedad salario desde la clase Vendedor porque esta propiedad es privada en la clase Empleado.




*Error de compilación en NetBeans relativo a un problema de ámbito de declaración de Superclase y subclase*

Si el método en la Superclase es Public, no puede ser Private en la subclase.

¿Pero no habíamos dicho que la clase Vendedor heredaba todas las propiedades y métodos de la clase Empleado? La respuesta sigue siendo afirmativa, pero debe tener en cuenta que no es lo mismo poseer una propiedad o método que poder acceder a ella.

## Acceso Protected

El error que se ha producido al compilar el proyecto se debe a que se ha intentado acceder a la propiedad salario de la clase Empleado desde la clase Vendedor.

Al declarar esta propiedad como private en la clase Empleado, sólo es posible acceder a ella desde la propia clase.




En este caso se da la paradoja de que un objeto de la clase Vendedor tiene una propiedad heredada a la que no puede acceder directamente.

Existen dos formas de resolver esta situación:

– Puedes crear un método public o protected en la clase Empleado de forma que dé acceso a la propiedad privada.

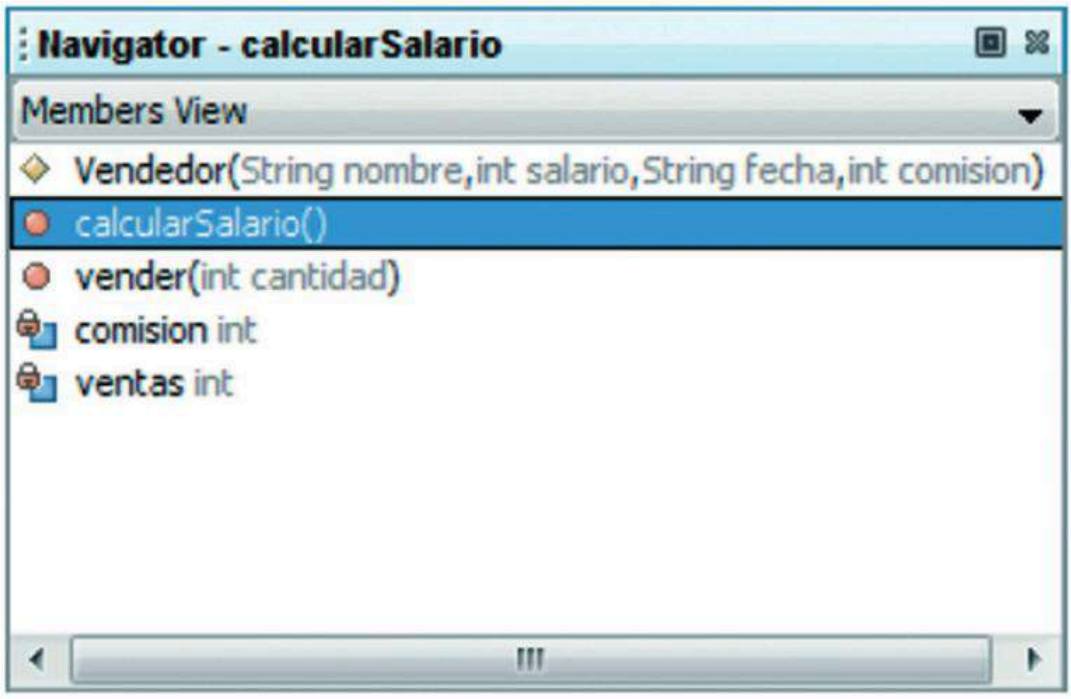
– Puedes declarar la propiedad como protected en lugar de private.

El modificador protected permite que la propiedad o método sea accesible tanto en la clase en la que se define como en sus subclases.

```
protected int salario;
```

De todas formas, es una buena costumbre ocultar las propiedades con el modificador private y utilizar métodos public o protected para acceder a estas propiedades.

NetBeans proporciona algunas herramientas que permiten manejar la jerarquía de clases de nuestra aplicación. Por ejemplo, en la ventana Navigator encontraremos los miembros de la clase en la que estamos trabajando, es decir, sus propiedades y métodos; pero también podremos hacer lo propio con sus superclases.

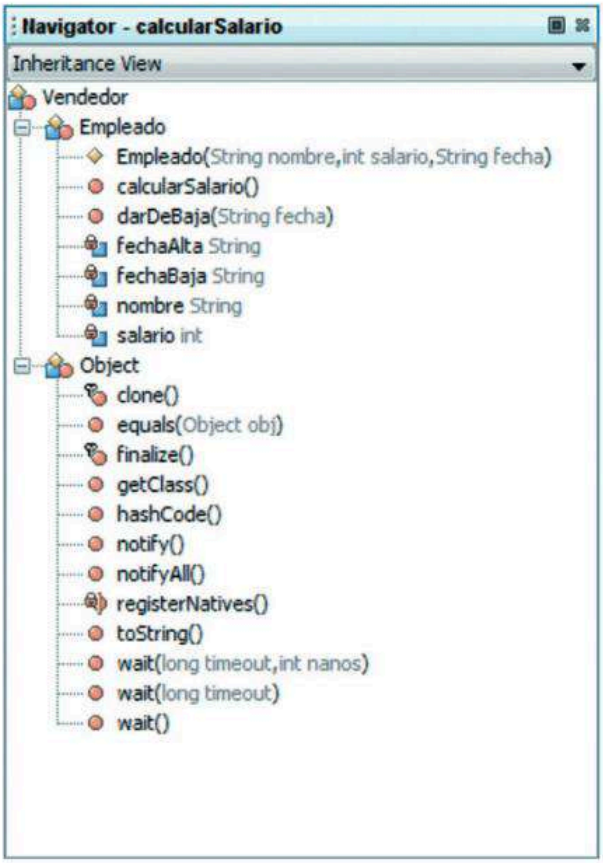



*Ventana Navigator de NetBeans donde encontramos los miembros de la clase*

En la ventana Navigator podemos encontrar los miembros de la clase en la que estemos trabajando.

Fíjate que aquí está la lista de los miembros de la clase Vendedor. Si queremos acceder rápidamente a alguno de las propiedades o métodos de esta clase, haremos doble clic sobre él.

Además de poder ver la lista de la clase en la que estamos trabajando, la ventana Navigator permite ver la jerarquía de clases involucrada. Para ello utilizaremos la lista que ahora pone Members View y elegiremos Inherintance View (Vista de herencia).



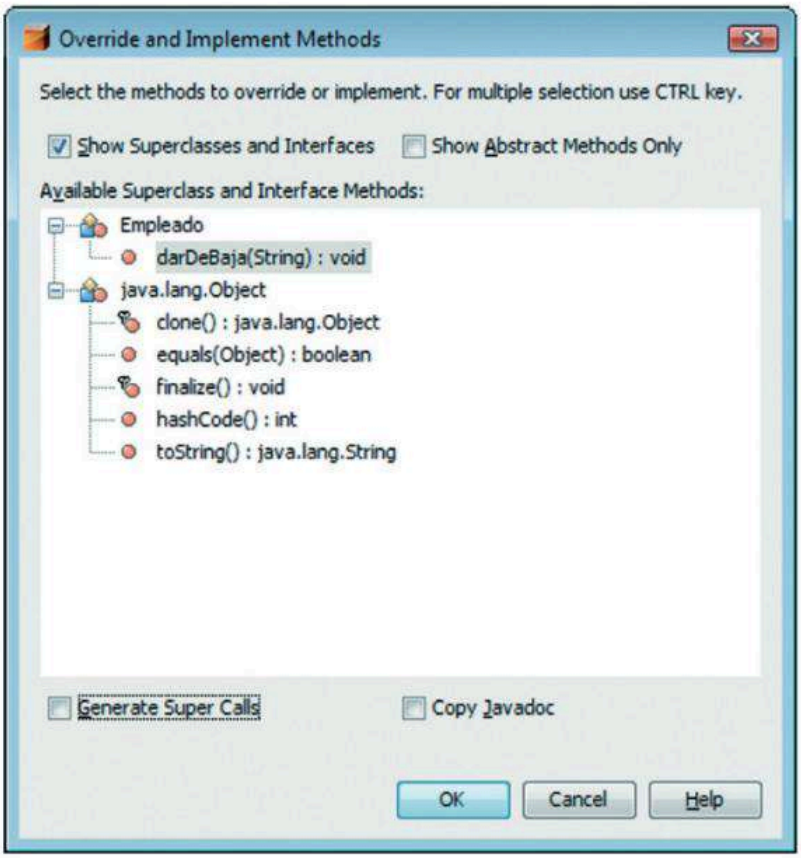

*Ventana Navigator de NetBeans con lista Members View y Inheritance View,*

Ahora disponemos de la lista de las clases que son superclase de la de Vendedor: por una parte la ya conocida clase Empleado y, por otra, la clase Object.

Object es la clase base del lenguaje Java, es decir, que cualquier otra clase hereda directa o indirectamente de Object.

Si escribe una clase que no es subclase de ninguna otra, lo que realmente está haciendo es una subclase de la clase genérica Object. Object proporciona la funcionalidad básica de cualquier clase.

La idea, en cualquier caso, es la misma. Así, podría acceder rápidamente a la declaración de una determinada propiedad o a la definición de un método de esta clase. Finalmente, veamos que NetBeans también facilita la tarea de sobrescribir métodos. Para ello, se puede utilizar el comando Override Methods del menú Source.




*Ventana Navigator de NetBeans con sobreescritura de Métodos*

Vemos que en la parte superior podremos seleccionar el método que queremos sobrescribir y en la parte inferior de este cuadro de diálogo encontramos la opción Generate Super Calls. Si está activada, NetBeans incluirá una llamada al método que sobrescribe de la superclase.

En este caso, en el método dar DeBaja de la clase Vendedor se incluiría una llamada al mismo método de la clase Empleado.

Si esto te interesa, manten activada la casilla; si no, desactívala.

Aceptando el cuadro de diálogo, NetBeans creará un nuevo método para que ahora escriba el correspondiente código, sobrescribiendo el mismo de la clase Empleado.

La ventaja de hacerlo así es que realmente no necesita conocer la definición del método en la superclase, ya que NetBeans se ha encargado de incluir el mismo nombre, tipo de datos que devuelve y los parámetros.

Nosotros sólo tenemos que escribir el cuerpo del método con el código correspondiente.

Vemos, por una parte, los conceptos teóricos de la programación orientada a objetos y su aplicación en Java; y, por otra, las herramientas que nos facilitan este trabajo en NetBeans.

Lo más importante es lo primero, es decir, asimilar los conceptos teóricos, ya que podrá implementarlos de una u otra forma en función de la herramienta de desarrollo que utilice.

## Comentarios en Java

En Java, como en la mayoría de lenguajes de programación, podemos escribir comentarios, es decir, líneas que el compilador no interpretará como código ejecutable sino simplemente como un texto descriptivo que hemos escrito. Los comentarios son necesarios para documentar adecuadamente nuestro código y son de extremada utilidad cuando hace tiempo que lo escribimos y estamos depurándolo. Escribe comentarios siempre que lo creas oportuno. Puede hacerlo de dos formas:

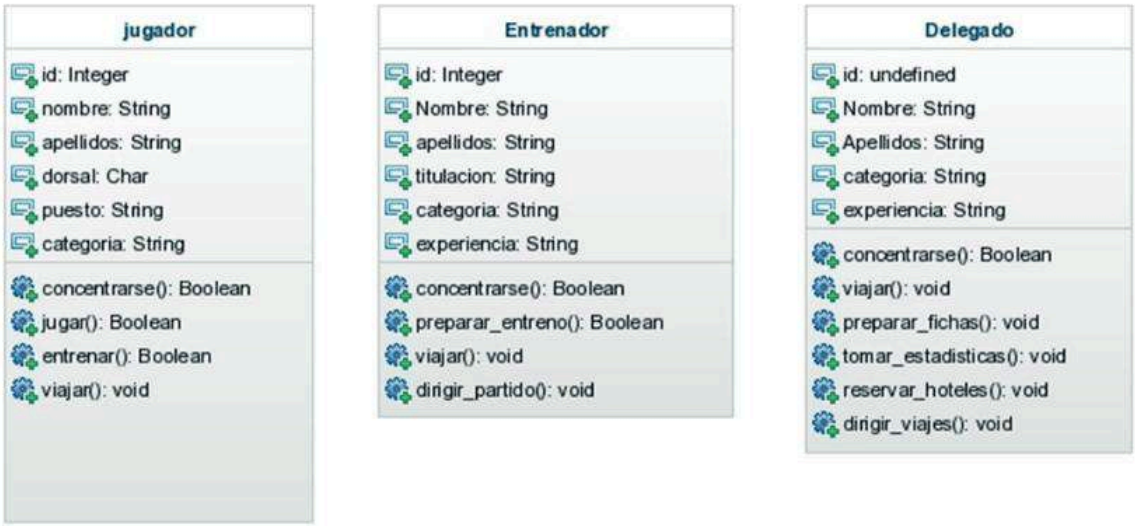
Comentario de una línea: utilice dos barras inclinadas hacia la derecha //. Todo lo que siga (en la misma línea) a estas dos barras se tomará como un comentario.

Comentario de más de una línea: encierra varias líneas de comentario utilizando /* al principio y */ al final. Todo lo escrito entre estos dos marcadores, independientemente del número de líneas que esto represente, se tomará como comentarios.

Importante

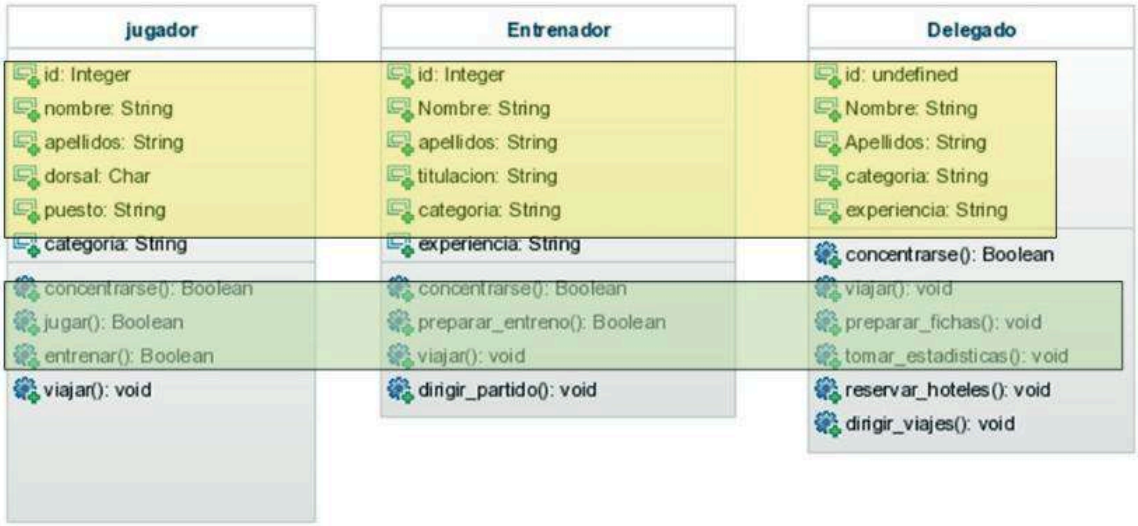
En relación a la herencia se tienen muchas definiciones como por ejemplo la siguiente: “La herencia es un mecanismo que permite la definición de una clase a partir de la definición de otra ya existente. La herencia permite compartir automáticamente métodos y datos entre clases, subclases y objetos“.

Así de primeras esta definición es un poco dificultosa de asimilar para aquellos que estéis empezando con la POO, así que vamos a intentar digerir esta definición con un ejemplo en el que veremos que la herencia no es más que un “copiar y pegar de una plantilla definida” o una forma de “sacar un factor clónico” al código que escribimos. El ejemplo que proponemos es un caso en el que vamos a simular el comportamiento que tendrían los diferentes integrantes de un equipo de baloncesto profesional; tanto los jugadores como el cuerpo técnico (Entrenadores, Delegado, etc.). Para simular este comportamiento vamos a definir tres clases que van a representaran a objetos jugador, Entrenador y Delegado. De cada unos de ellos vamos a necesitar algunos datos que reflejaremos en los *atributos* y una serie de acciones que reflejaremos en sus **métodos**. Estos atributos y métodos los mostramos en el siguiente diagrama de clases:




*Ejemplo de definición de diagrama de clases. Elaborado con Genmymodel*

Como se puede ver, con las tres clases poseemos atributos y métodos que son iguales ya que los tres tienen los atributos id, Nombre, Apellidos y Edad *;* y los tres poseen los métodos de Viajar *y* Concentrarse:




*Clases con atríbutos y métodos iguales. Elaborado con Genmymodel*

A nivel de código fuente tenemos las siguientes clases:

```
package model;


public class delegado
{
private int id;
private String nombre;
```

```
private String apellidos;
private String categoria;
private String experiencia;

                public delegado(){
                super();
        }
                public boolean concentrarse() {
                return false;
        }
                public void viajar() {

        }

                public void preparar_fichas() {

        }
                public void tomar_estadisticas() {

        }
                public void reservar_hoteles() {

        }
                public void dirigir_viajes() {

        }

}
```

```
package model;

public class entrenador
{
private int id;
private String nombre;
private String apellidos;
private String titulacion;
 private String categoria;
private String experiencia;

                public entrenador(){
                super();
        }
                public boolean concentrarse() {
                return false;
        }
                public boolean preparar_entreno() {
                return false;
        }

                public void viajar() {

        }
                public void dirigir_partido() {

        }

}
```

```
package model;

public class jugador
{

private int id;
private String nombre;
private String apellidos;
private char dorsal;
private String puesto;
private String categoria;

	public jugador(){
		super();
	}

		public boolean concentrarse() {
		return false;
	}

		public boolean jugar() {
		return false;
	}

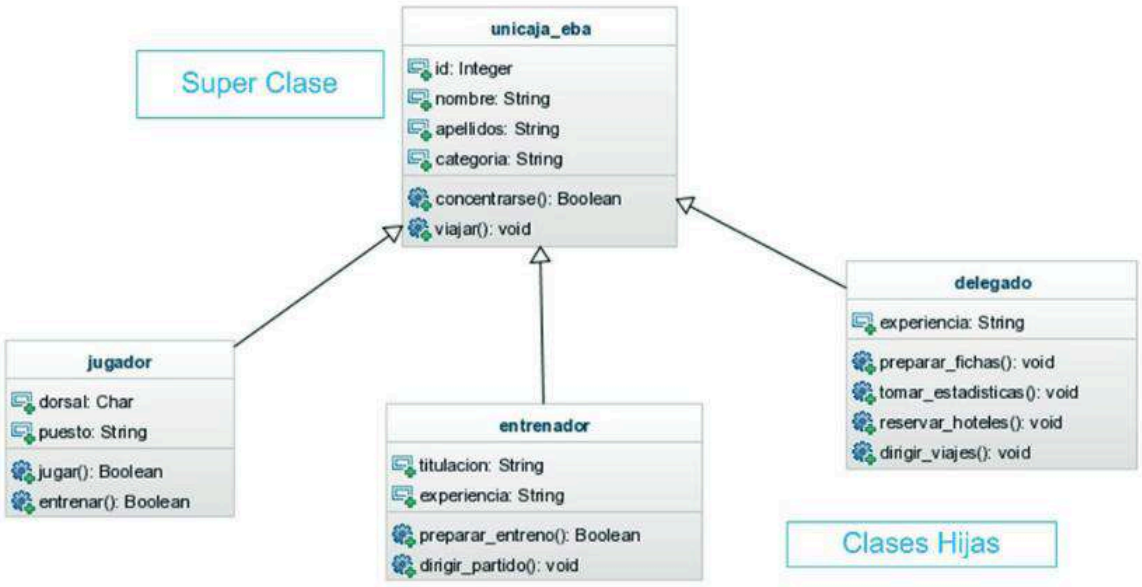
		public boolean entrenar() {
		return false;
	}

		public void viajar() {

	}

}
```

Lo que necesitamos ver en este punto es que estamos diseñando y picando considerable código repetido ya que las tres clases poseen métodos y atributos comunes, de ahí y como notaremos enseguida, exponemos que la herencia consiste en "sacar factor común " para no escribir código de más, por tanto lo que haremos será crearnos una clase con el "código que es común a las tres clases" (a esta clase se le denomina en la herencia como "SuperClase") y el código que es especifico de cada clase, lo dejaremos en ella, siendo denominadas estas clases como "Clases Hijas", las cuales *heredan de la clase padre todos los atributos y métodos públicos o protegidos*. Es muy importante decir que las clases hijas no van a heredar nunca los atributos y métodos privados de la clase padre, así que mucho cuidado con esto. En resumen para que veáis la ventaja de la herencia, tenemos ahora una clase padre con 'n' líneas de código y tres clases hijas con 'a', 'b' y 'c' líneas de códigos respectivamente, por tanto si calculamos, hemos reducido nuestro código en '2n' líneas menos ya que antes teníamos '(n+a)+(n+b)+(n+c)' líneas de código y ahora tras aplicar herencia tenemos 'n+a+b+c' líneas, aunque también es cierto que tenemos una clase más, pero veremos la superioridad de tener esa clase padre. Resumiendo, al "sacar factor común" y aplicar herencia, tenemos las siguientes clases:




*Superclase y clases hijas. Elaborado con Genmymodel*

En las siguientes clases podemos contemplar una mejor estructura y menos código fuente.

```
package model;


public class unicaja_eba
{
private int id;
private String nombre;
private String apellidos;
private String categoria;

                public unicaja_eba(){
                super();
        }

                public boolean concentrarse() {
                return false;
        }

                public void viajar() {

        }

}
```

```
package model;

public class delegado extends unicaja_eba
{
public String experiencia;

                public delegado(){
                super();
        }

                public void preparar_fichas() {

        }

                public void tomar_estadisticas() {

        }

                public void reservar_hoteles() {

        }

                public void dirigir_viajes() {

        }

}
```

```
package model;


public class entrenador extends unicaja_eba
{


private String titulacion;
private String experiencia;


                public entrenador(){
                super();
        }


                public boolean preparar_entreno() {
                return false;
        }


                public void dirigir_partido() {


        }


}
```

```
package model;

public class jugador extends unicaja_eba
{



private char dorsal;
private String puesto;

                public jugador(){
                super();
        }


                public boolean jugar() {
                return false;
        }

                public boolean entrenar() {
                return false;
        }

}
```

Diferentes clases que el alumno/a deberá teclear como ejercicios en su ordenador y compilar y ejecutar adecuadamente.

### 3.1.1. Relación de herencia: características

Según visto en la Unidad anterior ahora queda un código mucho más limpio, estructurado y con menos líneas de código, lo que lo hace más legible, cosa que es muy importante y lo que todavía lo hace más importante es que es un código reutilizable, lo que significa que ahora si queremos añadir más clases a nuestra aplicación como por ejemplo una clase Médico, Fisio, Jefe/a de prensa etc. que pertenezcan también al equipo técnico del equipo Unicaja EBA de baloncesto, lo podemos hacer de forma muy sencilla ya que en la clase padre (unicaja_eba) tenemos implementado parte de sus datos y de su comportamiento y solo habrá que implementar los atributos y métodos propios de esa clase. ¿Empezáis a ver la utilidad de la herencia?.

En este momento si os habéis fijado convenientemente en el código que se ha escrito y si no habéis tenido experiencia con la herencia en Java, habréis podido contemplar varias palabras reservadas "nuevas" como son "extends", "protected" y "super". Pues bien, ahora vamos a explicar el significado de ellas:

- extends: Esta palabra reservada, muestra a la clase hija cual va a ser su clase padre, es decir que por ejemplo en la clase Jugador al poner "public class jugador extends unicaja_eba" le estamos indicando a la clase 'jugador' que su clase padre es la clase 'unicaja_eba o dicho de otra manera para que se entienda mejor, al poner esto estamos haciendo un "copy–paste dinámico" diciendo a la clase 'jugador que se 'copie' todos los atributos y métodos públicos o protegidos de la clase 'unicaja_eba. De aquí viene esa 'definición' que dimos de que la herencia en un 'copy–paste dinámico'.

- protected: sirve para mostrar un tipo de visibilidad de los atributos y métodos de la clase padre y representa que cuando un atributo es 'protected' o protegido, solo es visible ese atributo o método desde una de las clases hijas y no desde otra clase.

- super: se usa para invocar al constructor de la clase padre. Quizás en el código que hemos puesto no se ha visto muy bien, pero a continuación lo mostramos de formas más clara, viendo el constructor de los objetos pasándole los atributos:

```
package Main;
import java.util.ArrayList;
import unicaja_eba.entrenador;
import unicaja_eba.jugador;
import unicaja_eba.delegado;
```

```
public class Main {

// ArrayList de objetos unciaja_eba. Idenpendientemente de la clase hija a la que pertenezca el objeto

public static ArrayList<unicaja_eba> integrantes = new ArrayList<unicaja_eba>();

public static void main(String[] args) {

Entrenador tomas = new entrenador(1, “Tomas”, “Lopez”, 40, “BA-MA234-JUNIOR91”);

Jugador caye= new jugador(2, “Cayetano”, “Sánchez”, 29, 9, “Base”);

Delegado arturo = new delegado(3, “Arturo”, “Martinez”, 41, “Licenciado en Empresariales”, 18);

integrantes.add(tomas);

integrantes.add(caye);

integrantes.add(arturo);


// CONCENTRACION

System.out.println(“Todos los integrantes comienzan una concentracion. (Todos ejecutan el mismo método)”);

for (unicaja_eba integrante : integrantes) {

System.out.print(integrante.getNombre()+” “+integrante.getApellidos()+” -> “);

integrante.Concentrarse();

}
```

```
// VIAJE
System.out.println("\nTodos los integrantes viajan para jugar un partido. (Todos ejecutan el mismo método)");
for (unicaja_eba integrante : integrantes) {
System.out.print(integrante.getNombre()+" "+integrante.getApellidos()+" -> ");
integrante.Viajar();
}

// ENTRENAMIENTO
System.out.println("\nEntrenamiento: Solamente el entrenador y el jugador tiene metodos para entrenar:");
System.out.print(tomas.getNombre()+" "+tomas.getApellidos()+" -> ");
tomas.dirigirEntrenamiento();
System.out.print(caye.getNombre()+" "+caye.getApellidos()+" -> ");
caye.entrenar();

// TOMA DATOS ESTADISTICOS
System.out.println("\estadisticas: Solo el Delegado tiene el método para tomar estadisticas:");
System.out.print(arturo.getNombre()+" "+arturo.getApellidos()+" -> ");
arturo.tomarestadisticas();

// PARTIDO DE LIGA EBA
System.out.println("\nPartido de Baloncesto: Solamente el entrenador y el jugador tiene metodos para el partido de baloncesto:");
System.out.print(tomas.getNombre()+" "+tomas.getApellidos()+" -> ");
tomas.dirigirPartido();
System.out.print(caye.getNombre()+" "+caye.getApellidos()+" -> ");
caye.jugarPartido();
}
}
```

### 3.1.2. Reglas y características que definen una relación de herencia: regla “es–un”




Herencia es el proceso que consiste en derivar una clase, llamada clase derivada, de otra, llamada clase base, y se pueden utilizar los métodos de la clase base en la clase derivada.

La herencia entre dos clases establece una relación entre las mismas de tipo “es un”, lo que significa que un objeto de una subclase “es un” objeto de la superclase.

Partimos de la premisa que un vehículo puede ser la superclase de la subclase coche, por lo que coche “es un” vehículo. Por ello una forma de saber si una relación de herencia entre dos clases está bien planteada, es comprobar si se cumple la relación “es un” entre la subclase y la superclase. Por ejemplo, para crear una clase **Línea** podríamos intentar heredar **Punto**, pensando que es una subclase de esta, sin embargo, “una línea es un punto?”, **no**, por lo tanto la herencia está mal planteada.

#### Test “es–un”

La clase A se debe relacionar mediante herencia con la clase B si “A **es–un** B”. Si la frase es coherente y suena bien, entonces la situación de herencia es la más probable para ese caso.

– Un pájaro es un animal

– Un perro es un mamífero

– Un pastel de fresas es un pastel

– Una matriz de enteros es un matriz

– Un coche es un vehículo

Sin embargo, si la frase suena rara por una razón u otra, es muy probable que la relación de herencia no sea lo más adecuado. Veamos unos ejemplos:

– Un pájaro es un mamífero

– Un pastel de fresas es una manzana

– Una matriz de enteros es un entero

– Un motor es un vehículo

Por ello, podemos encontramos casos en los que este test puede fracasar y sin embargo la correspondencia de herencia es evidente. Sin embargo, para la mayor parte de los casos, la aplicación de esta técnica es ajustada.

### Principales usos

La herencia como reutilización de código: Por ello una clase descendida puede heredar comportamiento de una clase base, por tanto, el código no requiere volver a ser escrito para la derivada.

La herencia como reutilización de conceptos: Esto ocurre cuando una clase descendida sobrescribe el comportamiento definido por la clase base. Aunque no se comparte ese código entre ambas clases, ambas comparten el prototipo del método (comparten el concepto).

La mente humana clasifica los conceptos de acuerdo a dos dimensiones:

– Pertenencia **(tiene–un)** → Relaciones todo–parte

– Variedad **(es–un)** → Herencia

La propiedad herencia logra catalogar los tipos de datos (abstracciones) por variedad, acercando un poco más el mundo de la programación al modo de razonar humano.

Este modo de razonar humano se denomina **generalización**, y da lugar a jerarquías de generalización/especialización.

La implementación de estas jerarquías en un lenguaje de programación da lugar a jerarquías de herencia.

### 3.1.3. Transmisión de atributos y métodos

Todos los campos y métodos de una clase son continuamente accesibles para el código de la idéntica clase.




Para controlar el acceso desde otras clases, y para inspeccionar la herencia por las subclase, los miembros (atributos y métodos) de las clases tienen tres modificadores posibles de control de acceso.

- Public: Los miembros declarados "public" son accesibles en cualquier parte en que sea posible la clase, y son heredados por las subclases.
- Private: Los miembros declarados private son accesibles únicamente en la propia clase.
- Protected: Los miembros declarados Protected son accesibles sólo para sus subclases.

Por ejemplo:

```
class Papa { // Hereda de Object
// Atributos
private int numFavorito, naceHace, dineroqueDispone;
// Métodos
public int getApuesta() {
```

```
return numeroFavorito;
}
protected int getEdad() {
return naceHace;
}
private int getSaldo() {
return dineroqueDisponible;
}
}
class Hija extends Papa {
// Definición
}
class VisitaCajero {
// Definición
}
```

En este ejemplo, un objeto de la clase Hija, hereda los tres atributos (numFavorito, naceHace y dineroqueDispone) y los tres métodos ( getApuestas(), getEdad() y dineroqueDispone() de la clase Papa, y podrá invocarlos. Cuando se llame al método getEdad() de un objeto de la clase Hija, se devolverá el valor de la variable de instancia nacido. Hace de ese objeto, y no de uno de la clase Padre.

Sin embargo, un objeto de la clase Hija, no podrá invocar al método getSaldo() de un objeto de la clase Papa, con lo que se evita que el Hijo conozca el estado de la cuenta corriente de un Padre.

La clase VisitaCajero, solo podrá acceder al método getApuestas(), para averiguar el número favorito de un Padre, pero de ninguna manera podrá conocer ni su saldo, ni su edad.

Las dos clases implementadas a continuación tienen una relación subclase/superclase simple con un único método que se sobrescribe en la subclase:

```
class claseAAAA {
void metodoDinamico() {
System.out.println("En el metodo dinamico de claseAAAA");
}
}
class claseBBBB extends claseAAAA {
void metodoDinamico() {
System.out.println("En el metodo dinamico de claseBBBB");
}
}
Por lo tanto si ejecutamos:
claseAAAA referenciaAAAA = new claseBBBB();
referenciaAAAA.metodoDinamico();
```

La salida de este programa es:

```
En el metodo dinamico de claseBBBB
```

Se expresa la variable de tipo claseA, y después se almacena una referencia a una instancia de la clase claseB en ella. Al invocar al método metodoDinamico() de claseA, el compilador de Java comprueba que claseA$ tiene un método denominado metodoDinamico() pero el intérprete de Java observa que la referencia es ciertamente una instancia de claseB, por lo que llama al método metodoDinamico() de claseB en vez de al de clasA.

## Sobreescritura de un método

Conforme en una jerarquía de herencia puede afectar volver a escribir el cuerpo de un método, para efectuar una funcionalidad de diferente manera dependiendo del nivel de abstracción en que nos encontremos. A esta modificación de operatividad se le llama sobrescritura de un método.

Por ejemplo, en una herencia entre una clase SerVivivіente y una clase hija Persona; si la clase SerViviente tuviese un método comer(), debería volver a escribirse en el nivel de Persona, puesto que una persona no come ni como un Animal, ni como una planta…

La excelente manera de observar la diferencia entre sobrescritura y sobrecarga es mediante un ejemplo. A continuación se puede observar la implementación de la sobrecarga de la distancia en 3D y la sobrescritura de la distancia en 2D.

```
class MiPoints3D extends MiPoints {
int x,y,z;
double distancia(int pX, int pY) { // Sobrescritura
int retoma=0;
retoma += ((x/z)-pX)*((x/z)-pX);
retoma += ((y/z)-pY)*((y/z)-pY);
return Math.sqrt( retoma );
}
}
```

Se inician los objetos mediante las sentencias:

```
MiPoints p3 = new MiPoints(1,1);
MiPoints p4 = new MiPoints3D(2,2);
```

Y llamando a los métodos de la siguiente forma:

```
p3.distancia(3,3); //Método MiPoints.distancia(pX,pY)
p4.distancia(4,4); //Método MiPoints3D.distancia(pX,pY)
```

Los métodos se seleccionan en función del tipo de la instancia en tiempo de ejecución, no a la clase en la cual se está ejecutando el método actual. A esto se le llama selección dinámica de método.

## Sobrecarga de método

Es posible que necesitemos crear más de un método con el mismo nombre, pero con listas de parámetros distintas. A esto se le llama sobrecarga del método. La sobrecarga de método se utiliza para proporcionar a Java un comportamiento polimórfico.

Un ejemplo de uso de la sobrecarga es por ejemplo, el crear constructores alternativos en función de las coordenadas, tal y como se hacía en la clase miPoints:

```
MiPoints( ) { //Constructor por defecto

inicia( -1, -1 );
}
MiPoints( int paramX, int paramY ) { // Parametrizado
this.x = paramX;
y = paramY;
}
```

Se llama a los constructores apoyándose en el número y tipo de parámetros que se les envía. Al número de parámetros con tipo de una sucesión específica se le llama signatura del tipo. Java utiliza estas signaturas de tipo para decidir a qué método llamar.

Para diferenciar entre dos métodos, no se consideran los nombres de los parámetros formales sino sus tipos:

```
MiPoints p1 = new MiPoints(); // Constructor por defecto
MiPoints p2 = new MiPoints( 5, 6 ); // Constructor parametrizado
```

### Limitación de la sobreescritura de métodos: final

Todos los métodos y las variables de instancia se pueden sobrescribir por defecto. Si se quiere expresar que ya no se quiere consentir que las subclases sobrescriban las variables o métodos, éstos se pueden expresar como final. Esto se utiliza a menudo para establecer el equivalente de una constante de C++.

Es un convenio de codificación habitual elegir identificadores en mayúsculas para las variables que sean final por ejemplo:

```
final int NUEVO_ARCHIVO = 1;
```

## 3.1.4. Regla de especialización de la superclase en la subclase

### Crear una subclase

Tenemos un buen ejemplo en el que se enseña cómo se crea una subclase utilizando le herencia. Tenemos que suponer que tenemos una clase llamada «vehículo» que tiene un método start, que se puede utilizar para arrancar el vehículo y que visualiza «Arrancar…»:

```
class vehiculo
public void start()
System.out.println(arrancar...");
```

Hay otro tipo de vehículos, por lo tanto si se quieres especializar la clase “vehículo” en una clase “coche”, se puede usar la herencia con la palabra clave extends. Así se declara “coche” como una subclase de “vehículo”:

```
class coche extends vehiculo
{
```

La sintaxis indica que “coche” es derivada de la clase “vehículo”, lo que significa que en este caso, “coche” heredará el método start de “vehículo”. Además se pueden añadir los miembros de datos y métodos propios en las subclases. Este es un ejemplo en el que se añade un método llamado drive a la clase “coche”:

```
class coche extends vehiculo

public void drive()
{
System.out.println(“Conducir...”);
}
```

Ahora se puede acceder a ambos métodos start y drive en los objetos de la clase “coche”, como se ve en el ejemplo:

```
public class app
{
public static void main(String[] args)

System.out.println(“Crear un coche...”);
```

```
Coche c = new coche();
c.start();
c.drive();

}
```

Esta es la salida del código anterior:

```
C:\>java app
Crear un coche. . .
Arrancar ...
Conducir ...
```

Este es un ejemplo esencial de subclase. Hay cuantiosa materia más en todo este proceso. Por ejemplo, si se define un método en una subclase con el nombre semejante que un método de la superclase ¿Cómo se pasan los datos al constructor de una superclase?

### Acceso a la propia clase: this

Además de hacer continua referencia a la clase en la que se invoque, también vale para sustituir a sus constructores, utilizándola como método:

```
this(); // Constructor por defecto
this( int paramX, int paramY ); // Constructor parametrizado
```

## Acceso a la superclase: super

Ya hemos visto el funcionamiento de la referencia this como referencia de un objeto hacia sí mismo. En Java conocemos otra referencia llamada “super”, que se refiere directamente a la “superclase”.

La referencia super usa para acceder a métodos o atributos de la superclase.

Podíamos haber implementado el constructor de la clase MiPoints3D_(hija de miPoints) de la siguiente forma:

```
MiPoints3D( int x, int y, int z ) {
super( x, y ); // Aquí se llama al constructor de MiPoints
this.z = super.metodoSuma( x, y ); // Método de la superclase
}
```

Con una sentencia super.metodoSuma(x, y) se llamaría al método metodoSuma() de la superclase de la instancia this. Por el contrario con super() llamamos al constructor de la superclase.

## La composición

Otro ejemplo de relación muy frecuente en los diseños de los programas es la composición. Los objetos suelen existir compuestos de conjuntos de objetos más pequeños; un coche es un conjunto de motor y carrocería, un motor es un conjunto de piezas, y así sucesivamente. Esta noción es lo que se conoce como composición.

La forma de implementar una relación de composición en Java es incluyendo una referencia a objeto de la clase componedora en la clase compuesta.

Por ejemplo, una clase AreaRectangular, quedaría definida por dos objetos de la clase MiPoints, que representasen dos puntas contrarias de un rectángulo:

```
class AreaRectangular {
MiPoinst extreme1; //extremo inferior izquierdo
MiPoints extreme2; //extremo superior derecho
AreaRectangular() {
extreme1=new MiPoints();
extreme2=new MiPoints();
}
boolean estaEnElArea( MiPoinst p ){
if ( ( p.x>=extreme1.x && p.x<=extreme2.x ) &&
( p.y>=extreme1.y && p.y<=extreme2.y ) )
return true;
else
return false;
}
}
```

Puede observarse que las referencias a objeto (extreme1 y extreme2) son iniciadas, instanciando un objeto para cada una en el constructor. Así esta clase mediante dos puntos, referenciados por extreme1 y extreme2, establece unos límites de su área, que serán utilizados para comprobar si un punto está en su área en el método estaEnElArea().

### 3.1.5. Acceso a los atributos de una clase y acoplamiento entre las clases

Los especificadores de acceso se emplean con clases, miembros de datos y métodos para enumerar la visibilidad de esos componentes en el resto del programa. Esta es la manera general de declarar y definir las clases, que muestra cómo se aprovechan los especificadores de acceso:

```
access class nombre de clase [extends ... ] [implements ... ]
(
[access] [staticl tipo variable-de-instancial;
[access] [staticl tipo variable-de-instanciaN;
[access] [staticl tipo método1 (lista de parámetros)
[access] [static1 tipo métodoN (lista de parámetros)
{
```

Los valores permisibles de access son public, private y protected. Cuando se expresa un miembro de una clase como public, es accesible desde cualquier lugar del programa. Si se declara como private, sólo es accesible desde la clase de la que es miembro. Si se declara como protected, está disponible para la clase actual, otras clases del mismo paquete, y clases que son derivadas de esa clase. Si no se usa un especificador de acceso, el acceso por defecto es que el miembro de la clase es visible a la clase de la que es miembro, a las clases derivadas de la misma que están en su mismo paquete y a otras clases del mismo paquete. Por ejemplo, echemos un vistazo al código, en el que el método start se declara como private pero además se accede a él en main:

```
class vehiculo
{
private void start()

System.out.println("Arrancar ...") ;

class coche extends vehiculo
{
public void drive ( )
```

```
System.out.println("Conducir ..." ) ;

public class app

public static void main(String [ ] args)

System.out.println("Crear un coche...");
coche c = new coche ( ) ;
c.start();
c.drive () ;
```

| Ubicación | Private | Sin modificador | Protected | Public |
|---|---|---|---|---|
| Misma clase | x | x | x | x |
| Subclase en el mismo paquete | | x | x | x |
| No subclase en el mismo paquete | | x | x | x |
| Subclase en otro paquete | | | x | x |
| No subclase en otro paquete | | | | x |

*Alcance para el especificador de acceso (x = en el alcance)*

Dado que la declaración de un método, usando private, restringe ese miembro a su clase, Java dice que no puede encontrar el método start cuando se usa en main:

```
c:\>c:\jdkl.2.2\bin\javac app.java -deprecation
app.java:47: No method matching start() found in class coche
c. start ( ) ;

1 error
```

Por otro lado, la declaración de un miembro como protected restringe su alcance al código del mismo paquete y de la subclase de la clase en la que está declarado. Sin embargo, el siguiente código funciona:

```
class vehiculo
{
protected void start()
{
System.out.println("Arrancar ..." ) ;
}
}
class coche extends vehiculo
{
public void drive ( )
{
System.out.println("Conducir . . . ") ;
}
}
public class app
```

```
{
public static void main(String [ ] args)
{
System.out.println("Crear un coche...");
coche c = new coche ( ) ;
c.driver();
c.start();
}
}
```

Esta es la salida de la aplicación:

```
C:\>java app
Crear un coche...
Arrancar ...
Conducir ...
```

### 3.1.6. Utilización de objetos this (current, self u otros) y super

#### La referencia this

Java incluye un valor de referencia exclusivo llamado this, que se usa dentro de cualquier método para referirse al objeto actual. El valor this se refiere al objeto sobre el que ha sido invocado el método actual. Se puede usar this siempre que se solicite una referencia a un objeto del tipo de una clase actual. Si hay dos objetos que utilicen el mismo código, seleccionados a través de otras instancias, cada uno tiene su propio valor único de this.

Una distinción habitual es que un constructor llame a otro para generar la instancia adecuadamente. El siguiente constructor llama al constructor parametrizado MiPoints(x,y) para finalizar de iniciar la instancia:

```
MiPoints() {
this( -1, -1 ); // Llama al constructor parametrizado
}
```

En Java se puede declarar variables locales, incluyendo parámetros formales de métodos, que se solapen con los nombres de las variables de instancia.

No se usan x e y como nombres de parámetro para el método inicia, porque esconderían las variables de instancia x e y reales del ámbito del método. Si lo hubiésemos hecho, entonces x se hubiera referido al parámetro formal, ocultando la variable de instancia x.

```
void inicia2( int x, int y ) {
x = x; // Ojo, no modificamos la variable de instancia!!!
this.y = y; // Modificamos la variable de instancia!!!
}
```

Muchas veces es bueno disponer de una referencia que apunte al propio objeto que se está manipulando. Esto se consigue con la palabra reservada this. this es una referencia implícita que tienen todos los objetos y que apunta a si mismo. Por ejemplo:

```
class Circulo {
    Punto centro;
    int radio;
    . . .
    Circulo elqueMayor(Circulo c) {
        if (radio > c.radio) return this;
        else return c;
    }
}
```

El método elqueMayor retorna una referencia al círculo que tiene mayor radio, comparando los radios del Círculo c que se toma como argumento y el propio. En caso de que el propio resulte mayor el método debe retornar una referencia a si mismo. Esto se obtiene con la expresión return this.

Otro ejemplo:

Si pensamos en la clase Operario y definirla como:

```
class Operario {
    String nombre;
    int numOperario , sueldo;

    static private int contador = 0;

    Operario(String nombre, int sueldo) {
        this.nombre = nombre;
```

```
        this.sueldo = sueldo;
        numOperario = ++contador;
    }

    public void aumentarSueldo(int porcentaje) {
        sueldo += (int)(sueldo * aumento / 100);
    }

    public String toString() {
        return “Num. operario “ + numOperario + “ Nombre: “ + nombre +
                “ Sueldo: “ + sueldo;
    }
}
```

En la muestra el Operario se define por un nombre (String) y por un número de empleado y sueldo “enteros”. La clase define un constructor que fija los valores de nombre y sueldo y calcula el número de empleado a partir de un contador (variable estática que continuamente irá aumentando), y dos métodos, uno para calcular el nuevo sueldo cuando se produce un aumento de sueldo (método aumentarSueldo) y un segundo que devuelve una representación de los datos del empleado en un String.(método toString).

Con esta representación podemos enfocar en otra clase que aglutine todas las características de Operario y sume alguna propia. Por ejemplo, la clase Responsable. En los objetos de esta clase se les podría aplicar todos los datos y métodos de la clase Operario y completar algunos, como por ejemplo el hecho de que un Responsable disfruta un presupuesto.

Así expresaríamos que la clase Responsable extiende o hereda la clase Operario. Esto en Java se forma con la clausula extends que se incorpora en la definición de la clase, de la siguiente manera:

```
class Ejecutivo extends Empleado {
    int presupuesto;
    void asignarPresupuesto(int p) {
        presupuesto = p;
    }
}
```

Con esta definición un Responsable es un Operario que también tiene algún rasgo distintivo propio. El cuerpo de la clase Responsable incorpora sólo los miembros que son concretos de esta clase, pero tácitamente tiene todo lo que tiene la clase Operario.

A Operario se le llama clase base o superclase y a Responsable clase derivada o subclase.




Los objetos de las clases derivadas se crean igual que los de la clase base y pueden acceder tanto sus datos y métodos como a los de la clase base.

Por ejemplo:

```
Responsable jefe = new Responsable( “Salvador Martinez”, 1000);
jefe.asignarPresupuesto(1500);
jefe.aumentarSueldo(5);
```

Un Responsable **es** un Operario, pero lo contrario no es cierto. Si escribimos:

```
Operario curri = new Operario ( “Esteban Gutierrez” , 100) ;
curri.asignarPresupuesto(5000);  // error
```

Se producirá un error de compilación pues en la clase Operario no existe ningún método llamado asignarPresupuesto.

### Redefinición de métodos. El uso de super

Además se podría pensar en redefinir algunos métodos de la clase base pero haciendo que métodos con el mismo nombre y características se comporten de forma distinta. Por ejemplo podríamos pensar en rediseñar el método toString de la clase Operario añadiendo las características propias de la clase Responsable. Así se podría poner:

```
class Responsable extends Operario {
    int presupuesto;

    void asignarPresupuesto(int p) {
        presupuesto = p;
    }

    public String toString() {
        String s = super.toString();
        s = s + “ Presupuesto: “ + presupuesto;
        return s;
    }
}
```

De esta forma cuando se invoque jefe.toString() se usará el método toString de la clase Responsable en lugar del existente en la clase Operario.

Obsérvese en el ejemplo el uso de super, que representa referencia interna implícita a la clase base (superclase). Mediante super.toString() se invoca el método toString de la clase Operario

### Inicialización de clases derivadas

Cuando se crea un objeto de una clase derivada se crea implícitamente un objeto de la clase base que se inicializa con su constructor correspondiente. Si en la creación del objeto se usa el constructor no–args, entonces se produce una llamada implícita al constructor no–args para la clase base. Pero si se usan otros constructores es necesario invocarlos explícitamente.

En nuestro ejemplo dado que la clase método define un constructor, necesitaremos también un constructor para la clase Responsable.

```
class Responsable extends Operario {
    int presupuesto;

    Responsable (String n, int s) {
        super(n,s);
    }

    void asignarPresupuesto(int p) {
        presupuesto = p;
    }

    public String toString() {
        String s = super.toString();
        s = s + " Presupuesto: " + presupuesto;
        return s;
    }
}
```

Observese que el constructor de Responsable invoca directamente al constructor de Operario mediante super(argumentos). En caso de resultar vital la llamada al constructor de la superclase tendría que ser la primera sentencia del constructor de la subclase.

### En que ocasiones podemos utilizar super

Super: Sirve para acceder a miembros de la superclase desde la subclase:

```
super.metodoquesea(parámetros)
super.atributo
super
```

También sirve para llamar al constructor desde el constructor de la subclase super(parámetros): llamada al constructor de la superclase:

```
class Motocicleta extends Vehiculo{
Motocicleta() {
super();
}
}
Super
```

Si no se incluye la llamada desde una subclase, el compilador siempre incluye una llamada super() por defecto.

Si el constructor de la superclase recibe parámetros, super tiene que llamarse con el tipo de parámetros adecuado.

La referencia null

Para asignar a una referencia el valor nulo se utiliza la constante null. El ejemplo del caso anterior se podría completar con:

```
class Circulo {
    Punto centro;
    int radio;
    . . .
    Circulo elMayor(Circulo c) {
        if (radio > c.radio) return this;
        else if (c.radio > radio) return c;
              else return null;
    }
}
```

## 3.1.7. Leyes de Demeter

La ley de Demeter es un principio elemental de la programación orientada a objetos. Sorprende al entender su gran utilidad, sus resultados en proyectos de todos los tamaños y el poco conocimiento que la comunidad de desarrolladores tiene de su existencia.

Su enunciado básico es:

Formalmente, la Ley de Demeter dice que un método de un objeto solo puede llamar a métodos de:

- El propio objeto.
- Un argumento del método.
- Cualquier objeto creado dentro del método.
- Cualquier propiedad/campo directo del propio objeto.

Mejora de forma considerable la encapsulación y la cambiabilidad de nuestro código. Como detectar los lugares en nuestro código donde no se cumple.

Una forma de comprender mejor esta ley podemos dar la vuelta al enunciado y enumerar los casos prohibidos: no se debe llamar a métodos de los objetos devueltos por otros métodos.

El caso más común que tendremos que resolver son las cadenas de métodos, de la forma:

```
a.getA().getB().getValue();
```

y sustituirlas por funciones que realicen dicha acción:

```
a.getABValue();
```

Otro de los casos comunes por lo que no se cumple esta ley es en el que una función necesite muchas variables temporales. Se trata del mismo caso anterior, pero escrito de forma diferente, el código erróneo es de la forma:

```
Aa = a.getA();
Bb = x.getB();
b.getABValue();
```

Y por último, un síntoma muy común de que no se cumple esta función es que una clase requiera muchas otras para realizar su cometido y en consecuencia, en su cabecera se encuentren muchas instrucciones import.

Siguiendo estas sencillas normas conseguiremos mejorar en gran medida la calidad de nuestro código.

Un ejemplo puedes ser más explicativo:

```
Public class Persona {
Pirvate string nombre;
Private string dni;

/* ejercicio de Ley  de Demeter */
Public void comer (churrasco carne);
/*
// 2 -Podemos llamar a métodos de objetos pasados a nuestro método
Boolean = esDeTernera = filete.llevaBarbacoa();
If (esDeTernera) {

// 4 - podemos acceder a propiedades del propio objeto
System.out.println("A "+.this.nombre" Le gustan los churrascos con
salsa!. A comer!!");
  }

// 3- podemos llamar a métodos de objetos creados dentro del método
Postre postre = new postre();
Postre.preparar();

// 1-podemos llamar a nuestros propios métodos
tomarCafe();
  }
prívate void tomarCafe();
// tras la comida pega una buena taza de café
  }
}
```

### Incumpliendo la ley de Demeter

Una de las principales cuestiones que la Ley de Demeter tiene que evitar que hagas esto:

```
objetoA.metodoDeA().metodoDeB().metodoDeC();
```

Puede que pienses que algo así está bien, es mucho más legible y ocupa menos líneas. No es cierto da ciertos problemas.

Cuando picas código de este tipo, estás expuesto a los posibles cambios que puedan ocurrir en la clase del objetoA, del objetoB y del objetoC.

– ¿Qué pasa si el objetoA en el futuro cambia, y ya no necesita una referencia al objetoB?

– ¿Y si luego el objetoB ya no necesita una referencia al objetoC?

– Además, esta clase no es reutilizable, porque para usarla necesitas también las clases B y C.

### Beneficios de cumplir la ley de Demeter

– Se reducen las dependencias entre clases y el acoplamiento.

– Se vuelve más sencillo reutilizar las clases.

– El código es más fácil de probar.

– El código es más mantenible, más flexible a los cambios.

## 3.2. Distinción de la herencia múltiple

### Herencia múltiple

En la orientación a objetos, se consideran dos tipos de herencia, simple y múltiple. En el caso de la primera, una clase sólo puede emanar de una única superclase. Para el segundo tipo, una clase puede provenir de varias superclases.




En Java sólo se dispone de herencia simple, para una mayor sencillez del lenguaje, si bien se subsana de cierta manera la inexistencia de herencia múltiple con un concepto denominado interface.

### Declaración

Para indicar que una clase deriva de otra, heredando sus propiedades (métodos y atributos), se usa el término extends, como en el siguiente ejemplo:

```
public class SubClase extends SuperClase {
// Contenido de la clase
}
```

Por ejemplo, creamos una clase MiCoordenada3D, hija de la clase ya mostrada MiCoordenada:

```
class MiCoordenada3D extends MiCoordenada {
int z;
MiCoordenada3D( ) {
x = 0; // Heredado de MiCoordenada
y = 0; // Heredado de MiCoordenada
z = 0; // Nuevo atributo
}
}
```

La palabra clave extends se utiliza para decir que deseamos crear una subclase de la clase que es nombrada a continuación, en nuestro caso miCoordenada3D es hija de MiCoordenada.

La clase "Object" es la superclase de todas las clases da Java. Todas las clases descienden, directa o indirectamente de ella. Si al crear una nueva clase, no aparece la cláusula extends, Java considera que dicha clase desciende directamente de Objects.

Esta clase Object lleva una serie de funciones elementales a todas las clases:

– Public boolean equals (Object obj): Se utiliza para comparar, en valor, dos objetos. Devuelve true si el objeto que recibe por parámetro es igual, en valor, que el objeto desde el que se llama al método. Si se desean comparar dos referencias a objeto se pueden utilizar los operadores de comparación == y !=.

– Public int hashCode(): Devuelve un código hash para ese objeto, para poder almacenarlo en una Hashtable.

– Protected Object clone () throws CloneNotSupportedException Devuelve una copia de ese objeto.

– Public final Class getClass(): Devuelve el objeto concreto, de tipo Class, que representa la clase de ese objeto.

– Protected void finalize() throws Trowable: Realiza acciones durante la recogida de basura.

### 3.2.1. Problemas: Conflictos de nombres, herencia repetida

Para la resolución al problema con un Interfaz que no es solamente una forma más pura de denominar a una clase abstracta, su propósito es mucho más amplio que eso. Como un interfaz no tiene ninguna implementación, es decir, no hay ningún tipo de almacenamiento asociado al interfaz, no hay nada que obstaculice la unión de varios interfaces. Esto tiene mucho valor porque hay veces en que es necesario que un objeto X sea a y b y c. En C es donde esto se acuñó como herencia múltiple y no es sencillo de hacer porque cada clase puede conllevar su propio desarrollo. En Java, se puede hacer lo mismo, pero solamente una de las clases puede ser implementada, con lo cual los problemas que se presentan en C no suceden con Java cuando se unen varios interfaces.

En una clase descendiente, el programador no está forzado a tener una clase base sea esta abstracta o concreta, es decir, una sin métodos abstractos. Si se hereda desde una clase no interfaz, únicamente se puede heredar de una. El resto de los elementos base deben ser interfaces. Todos los nombres de interfaces se ponen después de la palabra clave implements y separados por comas. Podemos tener tantos interfaces como se quiera y cada uno de ellos será un tipo independiente. El ejemplo siguiente, muestra una clase concreta combinada con varios interfaces para producir una nueva clase.

```
import java.util.*;

interface Pelear {
    void pelear();
    }
interface Correr {
    void correr();
    }
interface Volar {
    void volar();
    }
class Accion {
    public void pelear() {}
    }

class Heroe extends Accion implements Pelear,Correr,Volar {
    public void correr() {}
    public void volar() {}
    }

public class Comic {
```

```
static void a( Pelear x ) {
    x.pelear();
    }
static void b( Correr x ) {
    x.correr();
    }
static void c( Volar x ) {
    x.volar();
    }
static void d( Accion x ) {
    x.pelear();
    }

public static void main( String args[] ) {
    Heroe i = new Heroe();
    a( i ); // Trata al Heroe como Pelear
    b( i ); // Trata al Heroe como Correr
    c( i ); // Trata al Heroe como Volar
    d( i ); // Trata al Heroe como Accion
    }
}
```

Se puede ver que Heroe combina la clase concreta Accion con los interfaces Pelear, Correr y Volar. Cuando se combina la clase concreta con interfaces de este modo, la clase debe ir la primera y luego los interfaces; en caso contrario, el compilador dará un error.

La autentificación, signature, para pelear() es la misma en el interfaz Pelear y en la clase Accion, y pelear() no está proporcionado con la definicion de Heroe. Si se quiere crear un objeto del nuevo tipo, la clase debe tener todas las definiciones que necesita, y aunque Heroe no proporciona una definicion explicita para pelear(), la definición es proporcionada automáticamente por Accion y así es posible crear objetos de tipo Heroe.

En la clase de arriba Comic, hay cuatro métodos que toman como argumentos los diferentes interfaces y la clase concreta. Cuando un objeto Heroe es creado, puede ser pasado a cualquiera de estos métodos, lo que significa que se está realizando un casting al interfaz de que se trate. Debido a la forma en que han sido diseñados los interfaces en Java, esto funciona perfectamente sin ninguna dificultad ni esfuerzo extra por parte del programador.

La razón principal de la presencia de los interfaces en el ejemplo anterior es el poder efectuar un casting a más de un tipo base. Sin embargo, hay una segunda razón que es la misma por la que se usan clases abstractas: evitar que el programador cliente tenga que establecer un objeto de esta clase y establecer que solamente es un interfaz. Y esto plantea la cuestión de qué se debe utilizar pues, un interfaz o una clase abastacta. Un interfaz aporta las bondades de una clase abstracta y, además, los beneficios mismos del interfaz, así que es posible crear una clase base sin la definición de ningún método o variable miembro, por lo que se deberían usar mejor los interfaces que las clases abstractas. De hecho, si se necesita disponer de una clase base, la primera elección debería ser un interfaz, y usar clases abstractas únicamente si es necesario poseer alguna variable miembro o algun método implementado.

Como Java es un lenguaje que permite herencia simple para implementar y que puede aportar herencia múltiple a través de interfaz. Esto permite la herencia múltiple en el diseño de los programas y aplicaciones Java.

Una interfaz consigue heredar de más de una interfaz anterior a ella.

```
interface InterfazMultiple extends Interfaz1,Interfaz2{ }
```

Una clase no puede poseer más que una clase madre, pero puede implementar más de una interfaz.

```
class MiClase extends SuMadre implements Interfaz1,Interfaz2{ }
```

El ejemplo típico de herencia múltiple es el que se presenta con la herencia en diamante.

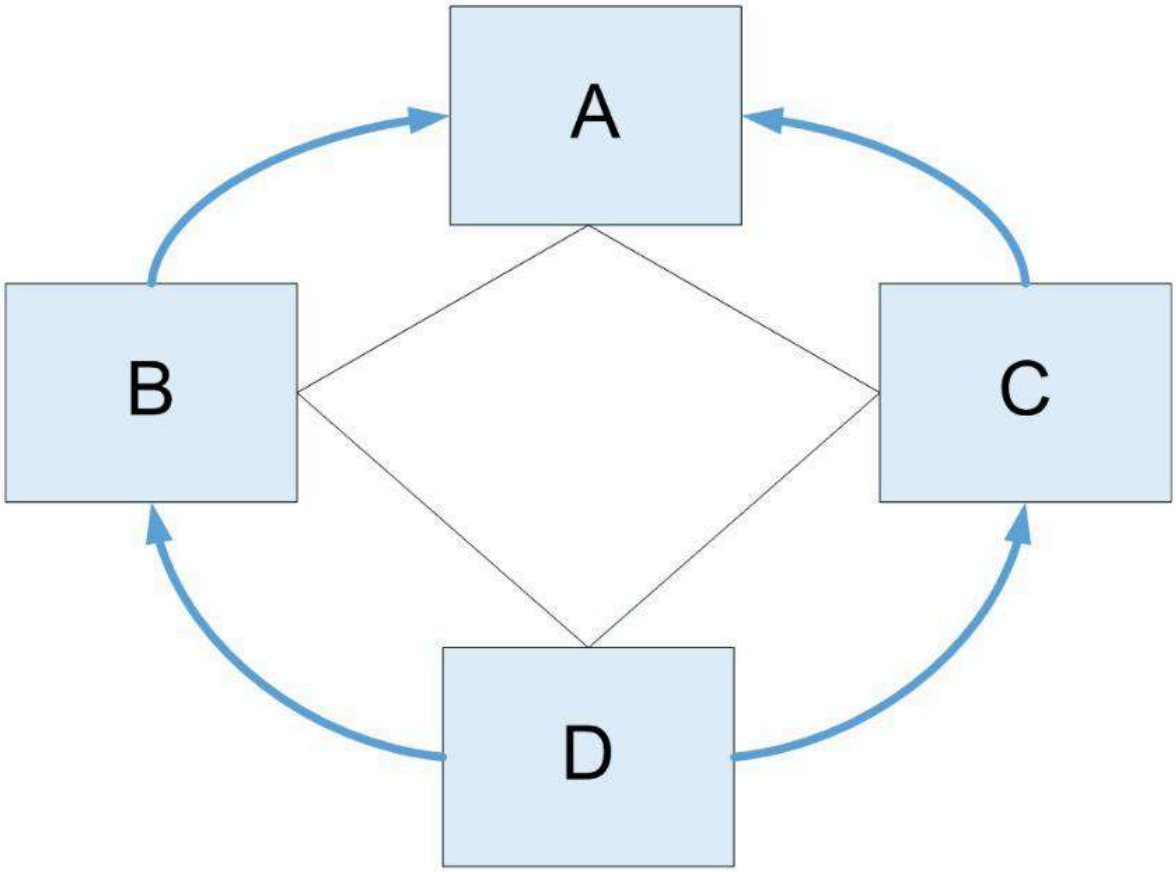



*Diagrama del problema del diamante*

El **problema del diamante** es un problema anómalo y ambiguo que aparece cuando dos clases B y C heredan de A, y la clase D hereda de B y C. Con esta premisa si un método en D invoca a un método definido en A, ¿por qué clase lo hereda, B o C?

### Solución desde Java al problema del diamante basada en interface

Para poder llevar a cabo un esquema como el anterior en Java es necesario que las clases A, B y C de la figura sean interfaces, y que la clase D sea una clase (que recibe la herencia múltiple):

```
interface A{ }
interface B extends A{ }
interface C extends A{ }
class D implements B,C{ }
```

Una interfaces hace lo siguiente: “Todas las clases que implementan esta interfaz especifica tendrán este aspecto”. La interfaz se utiliza para establecer un “protocolo” entre las clases.

Pero una interfaz es algo más que una clase totalmente abstracta ya que permite realizar una variante del mecanismo de “herencia múltiple”, creando una clase que pueda generalizarse a más de un tipo base.

Si necesitamos construir una interfaz usaremos el mandato “interface” en lugar de “class”. Podemos incluir la palabra public antes de interface, sólo si la interfaz está definida en un archivo del mismo nombre. Si no se incluye la palabra public, se obtiene acceso de tipo paquete. La interfaz puede contener campos, éstos serán static y final.

Cuando necesitemos crear una clase que se adapte a una o varias interfaces concretas, necesitaremos usar la palabra clave “implements”.

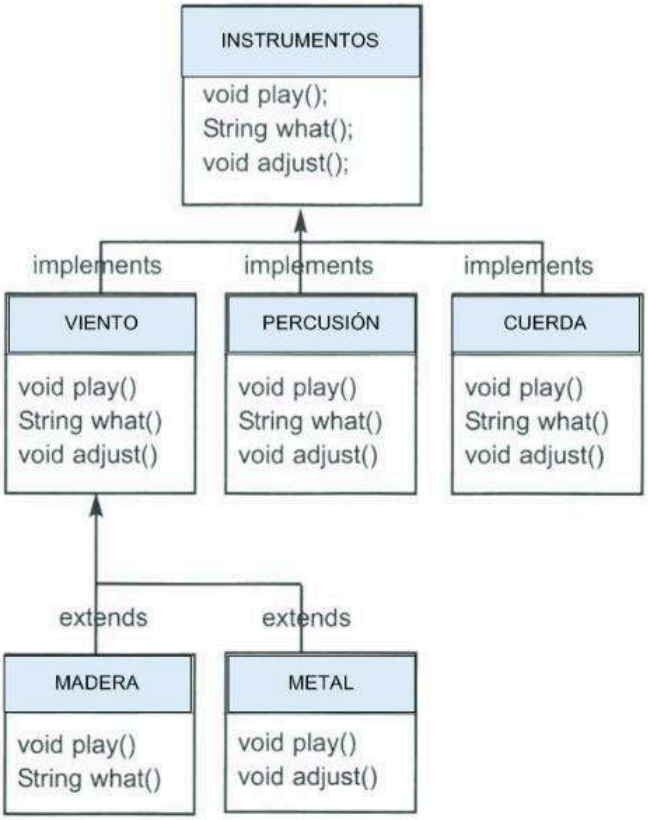



*Diagrama de solución al problema del diamante con Interface*

## 3.2.2. Soluciones

### Colisiones con herencia múltiple

En una herencia múltiple, los identificadores de algunos métodos o atributos pueden concordar en la clase que hereda, si dos de las interfaces padres poseen algún método o atributo que coincida en nombre. A esto se le llama colisión.

Esto se dará cuando las clases padre (en el ejemplo anterior B y C) tienen un atributo o método que se llame igual. Java solventa la complicación instaurando una serie de reglas.

Para la colisión de nombres de atributos, se obliga a especificar a qué interfaz base pertenecen al utilizarlos.

Para la colisión de nombres en métodos:

– Si tienen el mismo nombre y diferentes parámetros: se produce sobrecarga de métodos permitiendo que existan varias maneras de llamar al mismo.

– Si sólo cambia el valor devuelto: se da un error de compilación, indicando que no se pueden implementar los dos.

– Si coinciden en su declaración: se elimina uno de los dos, con lo que sólo queda uno.

### Envolturas de los tipos simples

Los tipos de datos de Java no forman parte de la jerarquía de objetos. Sin embargo a veces es necesario crear una representación como objeto de alguno de los tipos de datos simples de Java.

La API de Java contiene un conjunto de interfaces especiales para modificar el comportamiento de los tipos de datos simple. A estas interfaces se las conoce como envolturas de tipo simple.

Todas ellas son hijas de la clase abstracta Number y son:

| |
|---|
| Double: Da soporte al tipo double. |
| Float: Da soporte al tipo float. |
| Integer: Da soporte a los tipos int, short y byte. |
| Long: Da soporte al tipo long. |
| Character: Envoltura del tipo char. |
| Boolean: Envoltorio al tipo boolean. |

### ¿Por qué las interfaces?

Java no admite herencia múltiple, con las interfaces podemos conseguir el mismo efecto que si aplicáramos el paradigma de herencia múltiple propia de otros lenguajes como C++. . Supongamos que se quiere crear una applet que gestione los clics que se hacen en un botón.

Para crear una applet estándar, se puede derivar una clase de la clase java.applet.Applet y para gestionar los clics que se hacen en un botón, se usa otra clase, ActionListener. Por lo tanto, la applet se tendrá que basar en ambas clases Applet y ActionListener.

Basar una subclase en dos o más superclases se llama herencia múltiple, y Java no soporta la herencia múltiple (aunque sí lo hacen lenguajes como C++). En la práctica, esto quiere decir que sólo se puede usar la palabra clave extends con una clase. Para resolver este problema, Java implementa las clases como ActionListener como interfaces.

En el siguiente caso, eso significa que la applet se puede extender de la clase Applet y usar la palabra clave implements para añadir la gestión del clic en los botones. Así quedaría la applet:

```
import java.applet.Applet;
import j ava . awt . * ;
import java.awt.event.*;
public class clickar extends Applet implements ActionListener {
TextField texto1;
Button boton1;
public void init ( ) (
texto1 = new TextField(20);
add(texto1) ;
boton1 = new Button("¡Hacer clic aquí!");
add(boton1) ;
boton1.addActionListener(this);
}
public void actiopnerformed(ActionEvent event) {
String msg = new String ("Bienvenido a Java");
if (event .getSource() == boton1) {
texto1 . SetText (msg) ;
}
```

En este ejemplo podemos observar como podemos implementar tantas interfaces como necesitemos; en este fragmento de programa implementamos tres listeners para permitir que el programa utilice los clics en los botones y las acciones del ratón:

```
import java.awt.Graphics;
import java.awt.*;
import java.awt.event.*;
import java.lang.Math;
import java.applet.Applet;
public class dauber extends Applet implements ActionListener,
MouseListener, MouseMotionListener {
```

Este ejemplo no hay que compilarlo ya que no funcionará al faltar parte del código fuente que no viene al caso picar. Es solo muestra de un fragmento donde se implementan muchas interfaces.

## 3.3. Creación de objetos en la herencia

Siguiendo estos consejos y reglas. Cuando una clase implemente una interface, esto quiere decir que se deben implementar todos los métodos invocados en la interface. Son utilizadas para construir una interface pública estándar para puntos iguales. Esto posibilita que el código sea más fragmentado, sencillo de interpretar y muy modular.

```
public interface Telefono {
    public void marcaNumero(int numero);
    public boolean siLlamadaCurso();
}
```

El siguiente ejemplo demuestra esta interface siendo implementada en una clase Movil y una clase Prefijo. La sentencia implements es usada para implementar la interface.

```
public class Prefijo implements Telefono{
    private boolean siLlamadaCurso;
    public void marcaNumero (int numero) {
    }

    public boolean siLlamadaCurso() {
        return siLlamadaCurso;
    }
}
```

```
public class Movil implements Telefono{
    private boolean siLlamadaCurso;
    public void marcaNumero(int numero) {
    }

    public boolean siLlamadaCurso() {
        return siLlamadaCurso;
    }
}
```

En el momento que una interface es implementada, todos los métodos expuestos serán implementados en esta clase. Es viable implementar múltiples interfaces como ya hemos comprobado. Cuando mas de una interface está siendo usada, todos estos métodos definidos en cada uno debe ser implementados. Cualquier método sin implementar causará errores al compilar.

Lo siguiente es un ejemplo de una clase que implementa 2 interfaces:

```
public class VideoLlamada implements Telefono, VideoReproductor{
  ...
}
```




El gran avance de usar interfaces es que cualquier clase puede usar la misma interface y tendrán los mismos métodos públicos.

## ¿Por qué las clases internas?

Con otro ejemplo nuevo para este apartado, hay que decir que Java consiente establecer clases dentro de clases, y la clase encerrada se llama clase interna. Empezaremos a trabajar con ellas en esta unidad. Quizás no vea mucha necesidad de definir clases dentro de clases en este momento, pero lo verá más claro cuando se empiecen a gestionar los eventos de la interfaz de usuario, como, por ejemplo, cuando el usuario cierra una ventana.

Los eventos se gestionan con las interfaces, y cuando se implementa una interfaz, también se debe proporcionar la implementación de varios métodos abstractos en el interior de la misma. Para que este proceso sea más fácil, Java proporciona las clases adaptador, que ya tienen implementaciones vacías de los métodos requeridos. Para usar un evento de interfaz de usuario de cualquier tipo, es común poseer una subclase de las clases adaptador como clase interna, sobrescribiendo los métodos que se quiera de una forma muy compacta. Este es un ejemplo en el que el código finaliza una aplicación cuando el usuario cierra la ventana de la misma:

```
public class app
{
public static void main(String [ ] args)

AppFrame f = new AppFrameO;
f.setsize( 200, 200);
f.addWindovlistener(neor windowAdapter0 {public void
windoffClosing(WindarrEvent e) <System.exit(O);>>);
f.show();
}
}
```

```
public class Pato extends Animal
```

En este caso podemos decir que un Pato es un Animal y podremos usar un Pato en cualquier lugar donde se espere un animal.

El implements por otro lado se usa cuando una clase debe tener los métodos declarados en una interfaz.

Por ejemplo:

```
public interface Parlante {
public void decirAlgo();
}
```

Luego:

```
public class Persona implements Parlante{
public void decirAlgo(){
System.out.println("Hola");
}
}

public class Pato implements Parlante {
public void decirAlgo(){
System.out.println("Cuac");
}
}
```

El implements obliga a la clase a (justamente) implementar los métodos que estaban definidos en la interfaz. Como se puede ver cada clase implementa la interfaz a su manera.

Además se pueden hacer cosas como:

```
Parlante p = new Pato();
Parlante p1 = new Persona();
y hacer
p.decirAlgo();
p1.decirAlgo();
```

De esta manera cada uno se permitirá de manera distinta aunque hayas declarado ambos como parlantes. De la misma forma que antes en cada posición donde se espere un Parlante se podrá pasar un objeto de cualquier clase que implemente dicha interfaz. Las interfaces se utilizan para definir comportamiento.

## 3.4. Clasificación jerárquica de las clases

### La jerarquía

La verdadera potencia de la programación orientada a objetos radica en su capacidad para mostrar la abstracción que el cerebro humano efectúa automáticamente durante el proceso de aprendizaje y el proceso de análisis de información.

Las personas percibimos la circunstancias como un conjunto de objetos interrelacionados. Dichas interrelaciones, consiguen verse como un conjunto de abstracciones y generalizaciones que se han ido asimilando desde la niñez. Así, los defensores de la programación orientada a objetos afirman que esta técnica se ajusta mejor al funcionamiento del cerebro humano, al consentir descomponer un problema de cierta dimensión en un reducido conjunto de problemas menores dependientes del primero.

La capacidad de fragmentar un problema o concepto en un conjunto de objetos relacionados entre sí, y cuyo proceder es fácilmente identificable, puede ser muy viable y provechoso para el desarrollo de programas informáticos.




La herencia puede ser el componente básico de relación entre clases en la orientación a objetos. Relaciona las clases de forma jerárquica, una clase padre o superclase sobre otras clases hijas o subclases.

Los sucesores de una clase heredarán todas las variables y métodos que sus ascendientes hayan detallado como heredables, también de crear los suyos propios.

La peculiaridad de la herencia, es que nos posibilita definir nuevas clases derivadas de otra ya existente, que la especializan y perfeccionan de alguna forma. Así logramos definir una jerarquía de clases, que se puede mostrar mediante un árbol de herencia.

En toda la diversidad de lenguajes orientados a objetos existe siempre una jerarquía, por ella las clases se relacionan en términos de herencia. En Java, el punto más alto de la jerarquía es la clase “Object” de la cual cuelgan todas las demás clases.

## Ejemplo demostrativo y uso de jerarquía de las clases

Consideramos las figuras planas cerradas como el rectángulo, y el círculo. Dichas figuras comparten particularidades comunes como es la colocación de la figura, de su centro, y el área de la figura, aunque el proceder para calcular dicha área sea completamente distinto. Conseguimos por tanto, definir una jerarquía de clases, en la que la clase base se llama Figura, tenga las particularidades comunes y cada clase derivada de las específicas.

La clase Figura posee las características como común denominador a dichas figuras concretas por tanto, no tiene forma ni tiene área. Esto lo enunciamos declarando Figura como una clase abstracta, declarando la función miembro area abstract.




Las clases abstractas son muy útiles ya se pueden usar como clases base para otras clases. No se pueden instanciar objetos pertenecientes a una clase abstracta. Sin embargo, se pueden declarar variables de dichas clases.

En el juego del ajedrez definimos una clase base llamada Ficha, con las características comunes a todas las piezas, como es su colocación en el tablero, y descender de ella las características específicas de cada pieza particular. Así pues, la clase Ficha será una clase abstracta con una función abstract denominada mover, y cada tipo de pieza definirá dicha función según las reglas del juego de cada pieza y su movimiento sobre el tablero.

## La clase Ficha

La definición de la clase abstracta Ficha, lleva la posición x e y de la figura en concreto, de su centro, y la función area, que se va a aplicar en las clases derivadas para calcular el área de cada figura en particular.

```
public abstract class Ficha {
    protected int x;
    protected int y;
    public Ficha(int x, int y) {
        this.x=x;
        this.y=y;
    }
    public abstract double area();
}
```

## La clase Casilla

Las clases derivadas heredan los miembros dato x e y de la clase base, y definen la función area, declarada abstract en la clase base Ficha, ya que cada figura particular tiene una fórmula distinta para calcular su área. Por ejemplo, la clase derivada Casilla, tiene como datos, aparte de su posición (x, y) en el plano, sus dimensiones, es decir, su anchura ancho y altura alto.

```
class Casilla extends Ficha{
    protected double ancho, alto;
    public Casilla(int x, int y, double ancho, double alto){
        super(x,y);
        this.ancho=ancho;
        this.alto=alto;
    }
    public double area(){
        return ancho*alto;
    }
}
```

El primer bloque de código en el constructor de la clase descendida es una llamada al constructor de la clase base, para ello se emplea la palabra reservada **super**. El constructor de la clase derivada llama al constructor de la clase base y le pasa las coordenadas del punto *x* e *y*. Después inicializa sus miembros dato *ancho* y *alto*.

En la explicación de la función area, se calcula el área del rectángulo como producto de la anchura por la altura, y se devuelve el resultado.

## La clase Circunferencia

```
class Circunferencia extends Ficha{
    protected double radio;
    public Circunferencia(int x, int y, double radio){
        super(x,y);
        this.radio=radio;
    }
    public double area(){
        return Math.PI*radio*radio;
    }
}
```

En el bloque de código la primera sentencia en el constructor de la clase descendida, es una llamada al constructor de la clase base empleando la palabra reservada **super**. Posteriormente, se inicializa el miembro dato *radio*, de la clase derivada *Circunferencia*.

En la definición de la función area, se calcula el área del círculo mediante la conocida fórmula p*r2, o bien p*r*r.

## Uso de la jerarquía de clases

Creamos un objeto *c* de la clase *Circunferencia* situado en el punto (0, 0) y de 5.5 unidades de radio. Calculamos y mostramos el valor de su área.

```
Circunferencia c=new Circunferencia(0, 0, 5.5);
System.out.println("Area de la círcunferencia "+c.area());
```

Creamos un objeto *r* de la clase *Casilla* situado en el punto (0, 0) y de dimensiones 5.5 de anchura y 2 unidades de largo. Calculamos y mostramos el valor de su área.

```
Casilla r=new Casilla(0, 0, 5.5, 2.0);
System.out.println("Area de la casilla "+r.area());
```

Veamos ahora, una forma alternativa, guardamos el valor devuelto por **new** al crear objetos de las clases derivadas en una variable *f* del tipo *Ficha* (clase base).

```
Ficha f=new Circunferecia(0, 0, 5.5);
System.out.println("Area de la círcunferencia "+f.area());
f=new Casilla(0, 0, 5.5, 2.0);
System.out.println("Area de la casilla "+f.area());
```

### 3.4.1. Clase raíz

A veces necesitamos construir un método o una variable que se usará fuera del contenido de cualquier instancia, es decir, de una manera general a un programa. Todo lo que se tiene que hacer es declarar estos elementos como "static". Esta es la manera que tiene Java de poder implementar funciones y variables globales.

Por ejemplo:

```
static int a = 3;
static void metodoGlobal() {
// implementación del método
}
```

No se puede hacer referencia a this. o a super dentro de una método static.

Con atributos estáticos, todas las instancias de una clase contanto con el espacio propio para variables de instancia, cooperan en un espacio común. Esto es útil para modelizar casos de la vida real.

Otra faceta en la que es útil static es en la construcción de métodos a los que se puede invocar directamente escribiendo el nombre de la clase en la que están declarados. Se puede llamar a cualquier método static, o enfocar a cualquier variable static usando el operador punto con el nombre de la clase, sin necesidad de crear un objeto de ese tipo:

```
class ClaseStatic {
int atribNoStatic = 21;
static int atribStatic = 55;
```

```
static void metodoStatic() {
System.out.println("Met. static = " + atribStatic);
}
static void metodoNoStatic() {
System.out.println("Met. no static = " + atribNoStatic);
}
}
```

El siguiente código es capaz de llamar a metodoStatic y atribStatic nombrando directamente la clase (sin objeto, sin new), por haber sido declarados Static.

```
System.out.println("At. static = " + ClaseStatic.atribStatic);
ClaseStatic.metodoStatic(); // Sin instancia
new ClaseStatic().metodoNoStatic(); // Hace falta instancia
```

Si ejecutamos este programa obtendríamos:

```
At. static = 55
Met. static = 55
Met. no static = 21
```

Hay que tener en cuenta que en un método estático tan sólo puede hacerse referencia a variables estáticas.

### 3.4.2. Clases abstractas

Hay situaciones en las que se precisa definir una clase que simbolice un concepto abstracto, y por lo tanto no se consiga proporcionar una implementación completa de varios de sus métodos.

Se puede expresar que ciertos métodos han de ser sobrescritos en las subclases, usando el modificador de tipo abstract. A estos métodos además se les llama responsabilidad de subclase. Cualquier subclase de una clase abstract debe implementar todos los métodos abstract de la superclase o bien ser declarada también como abstract

Cualquier clase que contenga métodos declarados como abstract también se tiene que declarar como abstract, y no se podrán crear instancias de dicha clase (operador new).

Por último se pueden declarar constructores abstract o métodos abstract static.

Veamos un ejemplo de clases abstractas:

```
Veamos un ejemplo de clases abstractas:
abstract class claseAA {
abstract void metodoAbstracto();
void metodoConcreto() {
System.out.println("En el metodo preciso de claseAA");
}
}
class claseBB extends claseAA {
void metodoAbstracto(){
System.out.println("En el metodo abstracto de claseBB");
}
}
```

La clase abstracta claseAA ha implementado el método concreto metodoConcreto(), pero el método metodoAbstracto() era abstracto y por eso ha tenido que ser redefinido en la clase hija claseBB.

```
claseAA referenciaAA = new claseBB();
referenciaAA.metodoAbstracto();
referenciaAA.metodoConcreto();
```

La salida de la ejecución del programa es:

```
En el metodo abstracto de claseBB
En el metodo concreto de claseAA
```

En este otro ejemplo se puede apreciar mejor en el bloque de código:

```
abstract class FiguraCeramica {
    . . .
    abstract void pintar();
    . . .
}

class Plato extends FiguraCeramica {
    . . .
    void pintar() {
        // codigo para pintar Plato
        . . .
    }
}
```

La clase que es abstracta se define unicamente con el modificador “abstract” al escribirse. Los métodos abstractos se definene también de la misma manera, declarando el método pero sin implementarlo sin poner el bloque de código encerrado entre los corchetes {}. La clase derivada se define y aplica de forma normal, como cualquier otra. Sin embargo si no se define y aplican los métodos abstractos de la clase base, el compilador genera un error indicando que no se han implementado todos los métodos abstractos o se aplican o bien se declara la clase abstracta.

### Referencias y objetos abstractos

Se pueden crear referencias a clases abstractas como cualquier otra. No hay ningún problema en poner:

```
FiguraCeramica figura;
```

Por ello una clase abstracta no puede instanciarse, no se pueden crear objetos de una clase abstracta. El compilador producirá un error si se intenta:

```
FiguraCeramica figura = new FiguraCeramica();
```

Es lógico dado que una clase abstracta no tiene completa su implementación y encaja bien con la idea de que algo abstracto no puede materializarse.

Sin embargo utilizando el casting se puede escribir:

```
FiguraCeramica figura = new Plato(. . .);
figura.pintar();
```

La invocación al método pintarse resolverá en tiempo de ejecución y la JVM llamará al método de la clase adecuada.

### 3.4.3. Métodos virtuales

Para exponer mejor este punto de Métodos Virtuales y sobrescritura, usaremos un ejemplo de código en otro lenguaje distinto a Java.

```
TuCuentaVirtual* tucv;
tucv = new TuCuentaJoven;
tucv->abonarInteresMensuales();
// Llamada a TuCuentaJoven::abonarInteresMensuales()
delete tucv; // ¿?
```

La clase descendida redefine el comportamiento de la clase base.

Se intenta invocar a ciertos métodos redefinidos desde referencias a objetos de la clase base es elprincipio de sustitución.

En C++ es necesario indicar explícitamente que dicha sustitución es posible mediante la definición de dichos métodos como virtuales es mediante la palabra clave virtual.

En muchos lenguajes OO este rasgo es soportado de forma natural.

Por ejemplo, en Java, los métodos son virtuales por defecto.

La Sobrescritura la entenderemos como la redefinición de métodos virtuales en clases derivadas.

Y en Java el ejemplo seria:

```
class TuCuentaVirtual {
virtual void abonarInteresMensuales();
virtual void setInteres(float i);
virtual ~TuCuentaVirtual();
}
```

Si una clase tiene métodos virtuales:

– Es posible utilizar el principio de sustitución.

– Heredar de ella y sobrescribir los métodos virtuales

– El destructor de esta clase base debe declararse como método virtual.

## 3.4.4. Redefinición de métodos

### Redefinición de métodos

Se puede modificar localmente el comportamiento de los métodos heredados

Para ello con redeclarar un método declarado en la clase padre, se podría cambiar el comportamiento




De esta manera objetos de distintos subclases pueden responder de forma diferente a la misma llamada.Permite programar de manera más general.

Una clase puede redefinir( volver a definir ) cualquiera de los métodos heredados de su superclase que no sea final. El nuevo método sustituye al heredado para todos los efectos en la clase que lo ha redefinido.

Los métodos de la superclase que han sido rediseñados, pueden accederse por medio de la palabra reservada “super” desde los métodos de la clase derivada, pero con este sistema solo se puede subir un nivel en la jerarquía de clases.

Los métodos redefinidos pueden ampliar los derechos de accesos la superclases (por ejemplo ser public en vez de protected o package), pero nunca restringirles.

Los métodos de clase static no pueden ser redefinidos en las clases derivadas.




Redefinir métodos no es lo mismo que sobrecargar métodos. Pero si es lo mismo que sobrescribir.

Una subclase podría rediseñar un método heredado con la asignación de una nueva definición del método que posea el mismo nombre, el mismo número y tipo de parámetros, y el mismo tipo de resultado de retorno que el método heredado. El método heredado se oculta al entorno de la subclase. Cuando el método es invocado por un objeto de la subclase, el método rediseñado es arrancado usando ligadura dinámica. Un método rediseñado es asignado en una subclase como otro método, pero debe tener el mismo nombre, el mismo número de parámetros del mismo tipo, y el mismo tipo de retorno que el método de la superclase. Los métodos privados no pueden ser redefinidos, así que un método coincidente de la subclase se considera completamente separado. Por otro lado, el acceso de un método redefinido público (public), protegido (protected) o por defecto (si no hay modificador) debe tener el mismo acceso que el método de la superclase, o hacerse más accesible (por ejemplo, de protected a public). Un método redefinido no puede hacerse menos accesible.

Los métodos estáticos no pueden ser redefinidos. Los métodos de instancia no pueden redefinirse por un método estático. La redefinición de métodos depende del ligado dinámico (dynamic binding), de tal modo que el tipo del objeto para el cual un método es invocado determina cuál versión de un método redefinido se llama. Así, en una expresión como:

```
x.moldear(); // donde f es un método redefinido
```

La versión de moldear que se llama depende de la clase del objeto que se referencia por x cuando la llamada al método es evaluada en tiempo de ejecución, y no en el tipo de la variable x. La mayor consecuencia de esto es que una variable de tipo de una superclase puede asignarse a una referencia a un objeto de la subclase, pero cuando se llama a los métodos, el ligado dinámico asegura que los métodos redefinidos de la subclase sean llamados. Por tanto, la misma expresión evaluada a diferentes momentos durante la ejecución de un programa puede resultar en llamados a diferentes métodos. Los métodos estáticos no pueden usar ligado dinámico.




Recordar que el nombre, los tipos de los parámetros y el tipo de retorno frecuentemente llamado “signatura del método”, de un método rediseñado debe concordar exactamente. Si no, entonces ocurre la sobrecarga de métodos, en la que el método tiene más de un significado entre las dos clases.

La regla que la accesibilidad de un método redefinido debe ser igual o incrementarse es necesaria para preservar la sustitutibilidad. Si el objeto de una subclase debe ser usado donde un tipo de la superclase ha sido especificado, entonces debe soportar al menos la misma interfaz pública (o protegida, o por defecto). Remover un método de la interfaz prevendrá la sustitución.

El uso del ligado dinámico es parte esencial de la programación Orientada a Objetos, pero puede hacer difícil averiguar cuál método sería llamado al leer el código fuente. Dada una expresión como x.moldear() o sólo moldear() (que realmente es this.moldear()), el programador debe primero determinar el tipo de x o this, y entonces inspeccionar la clase correspondiente, superclases y subclases, para averiguar qué clase de método es moldear, y si es redefinido. El programador debe también determinar los tipos de objetos que pueden ser referenciados por x. Esto puede ser diferido e impreciso, pero considerar esto es importante. Es claro que un buen ambiente de desarrollo puede dar soporte para esta búsqueda, pero esto no significa que el comentario no deba hacerse.

El siguiente bloque de código Java muestra una variedad de métodos redefinidos y como usar el ligado dinámico. Es un ejercicio muy práctico y claro de entender.

```
class Figura {
// Metodo que puede ser redefinido
public void moldear(int x){
System.out.println(“Figura moldear: “ + x);
// Siempre llama al metodo pintar declarado en esta clase,
// ya que pintar es privada
pintar();
}
// No se puede redefinir el siguiente metodo
private void pintar(){
System.out.println(“Figura pintar”);
}
// Metodo que puede ser redefinido
protected void hornear(){
System.out.println(“Figura hornear”);
}
public void embalar(){
System.out.println(Figura embalar”);
// Siempre llama a Figura pintar, ya que pintar no puede ser
// redefinido
pintar();
// Llama a etiquetar dependiendo del tipo de objeto (el tipo de
// this) usando ligado dinamico
etiquetar();
// Siempre llama a almacenar en esta clase, ya que s es
// estatico
almacenar();
}
```

```
public static void almacenar() {
System.out.println("Figura static almacenar");
}
}
class Botijo extends Figura {
// Metodo redefinido. Debe ser publico
public void moldear(int x) {
System.out.println("Subclass moldear: " + x);
// Llama a pintar en esta clase
pintar();
}
// Nueva version de pintar, que no redefine la version de
// Figura
private void pintar(){
System.out.println("Botijo pintar");
}
// Metodo redefinido del heredado almacenar con acceso incrementado
// El hacer el acceso de este metodo privado o por defecto
// generaria un error
public void almacenar(){
System.out.println("Botijo hornear");
}
public static void embalar() {
System.out.println("Botijo static embalar");
}
}
class Override {
public static void main (String[] args) {
```

```
Figura figura = new Figura();
// Llama a la version de moldear en Figura
figura.moldear(1);
figura.hornear();
figura.embalar();
figura.etiquetar();
Botijo botijo = new Botijo();
// Llama a la version redefinida de moldear y hornear en Botijo
botijo.moldear(2);
botijo.hornear();
// Llama a la version de embalar en Figura, ya que no
// esta redefinida
botijo.embalar();
// Llama a la version de etiquetar en Botijo
botijo.etiquetar();
// Ahora, pone una variable de tipo Figura a una
// referencia a objeto de tipo Botijo
figura = botijo;
// Llama a las versiones de moldear y hornear en Botijo
figura.moldear(3);
figura.hornear();
// Llama a embalar para un objeto de Botijo
botijo.embalar();
// Llama a la version de etiquetar en Figura, ya que este
// metodo es estatico y no puede ser dinamicamente
// ligado, por lo que el metodo depende el tipo de la
// referencia
figura.etiquetar();
}
}
```

# UD3
## Lo más importante

- La **herencia** es, sin lugar a dudas, una de las características más potentes de la orientación a objetos y una de las que permiten distinguir claramente entre lo que es y no es un lenguaje orientado a objetos.

- La programación orientada a objetos permite utilizar el concepto de **subclase**. Cuando usted encuentra una clase que especializa otra clase ya existente, entonces puede hacer que la primera sea subclase de la segunda. La ventaja que tiene esto es que la subclase hereda las características de la clase más sencilla y, además, añade nuevas características.

- Un asunto delicado ocurre cuando desea crear objetos de una subclase. En este caso, es posible que necesite realizar lo mismo que hacía el constructor de la superclase, más otras tareas adicionales. Java introduce la palabra clave **super** para poder acceder a las propiedades y métodos de la superclase.

- Es posible modificar el funcionamiento de un método heredado. Esto se conoce con el nombre **sobrescribir** o reemplazar los métodos.

- En inglés se utiliza el término overriding para hacer referencia al proceso de sobrescribir métodos.

- De todas formas, es una buena costumbre ocultar las propiedades con el modificador private y utilizar métodos public o protected para acceder a estas propiedades.

- NetBeans proporciona algunas herramientas que permiten manejar la jerarquía de clases de nuestra aplicación.

– Herencia es el proceso que consiste en derivar una clase, llamada clase derivada, de otra, llamada clase base, y se pueden utilizar los métodos de la clase base en la clase derivada.

– Todos los campos y métodos de una clase son siempre accesibles para el código de la misma clase.

– Durante una jerarquía de herencia puede interesar volver a escribir el cuerpo de un método, para realizar una funcionalidad de diferente manera dependiendo del nivel de abstracción en que nos encontremos. A esta modificación de funcionalidad se le llama sobrescritura de un método.

– Java incluye un valor de referencia especial llamado this, que se utiliza dentro de cualquier método para referirse al objeto actual. El valor this se refiere al objeto sobre el que ha sido llamado el método actual.

– Super: Sirve para acceder a miembros de la superclase desde la subclase.

# UD3
## Autoevaluación

1. En Java, para indicar que una clase es subclase de otra:
   a. Se usa la palabra clave implements.
   b. Se utiliza la palabra clave subclass.
   c. Se utiliza la palabra clave extends.
   d. Ninguna de las anteriores.

2. Java introduce una palabra clave para poder acceder a las propiedades y métodos de la superclase:
   a. Super.
   b. New.
   c. This.
   d. Constructor.

3. De las expresiones cual no es cierta?
   a. En Java no es posible sobrescribir o reemplazar los métodos.
   b. El modificador de acceso de un método sobrescrito en la subclase puede ser igual.
   c. Es posible modificar el funcionamiento de un método heredado.
   d. Ninguna de las anteriores.

4. Para añadir comentarios en Java al código fuente:
   a. Utilizando /* al principio y */ al final.
   b. \\ y el texto que queremos comentar.
   c. \\ y al final del texto comentado //.
   d. ** y detrás el texto a comentar.

5. ¿Qué palabra reservada, muestra a la clase hija cual va a ser su clase padre?
   a. Super.
   b. protected.
   c. extend.
   d. Ninguna de las anteriores.

6. ¿Sirve para mostrar un tipo de visibilidad de los atributos y métodos de la clase?
   a. Super.
   b. extend.
   c. protected.
   d. Ninguna de las anteriores.

7. De las expresiones cual no es cierta?
   a. Pertenencia (tiene–un) → Herencia.
   b. Variedad (es–un) → Relaciones todo–parte.
   c. Una clase derivada puede heredar comportamiento de una clase base.
   d. Ninguna de las anteriores.

8. El valor this se refiere:
   a. Al objeto sobre el que ha sido invocado el método actual.
   b. Se usa dentro de cualquier método para referirse a la clase actual.
   c. Ninguna de las anteriores.

9. ¿Se usa para invocar al constructor de la clase padre?
    a. Super.
    b. extend.
    c. protected.
    d. Ninguna de las anteriores.

10. ¿Cual no es un enunciado de Las Leyes de Demeter? Un método de un objeto solo puede llamar a métodos de:
    a. El propio objeto.
    b. Un argumento de la clase.
    c. Cualquier objeto creado fuera del método.
    d. Cualquier propiedad/campo directo del propio método.

Área: informática y comunicaciones

# UD4

## Relaciones entre clases

4.1. Distinción entre agregación/composición

4.2. Distinción entre generalización/especialización

4.3. Identificación de asociaciones

Dentro del paradigma POO de programación orientada a objetos, la clase es el eje principal donde pivota la propiedad de modularidad. Tenemos una virtud natural de agrupar y clasificar todos los objetos por sus cualidades comunes, es decir por clase. Por ejemplo, las personas que asisten a la universidad se pueden clasificar en estudiante, docente, empleado e investigador.

La clase puede definirse como la agrupación ó colección de objetos que comparten una estructura común y un comportamiento común.

Es una estructura como una plantilla que posee la descripción general de una colección de objetos. Consta de atributos y métodos que resumen las características y el comportamiento comunes de un conjunto de objetos.

Todo objeto (asimismo denominado instancia de una clase), pertenece a alguna clase. Mientras un objeto es una forma concreta que existe en el tiempo y en el espacio, una clase simboliza solo una abstracción.

Todos aquellos objetos que pertenecen a la misma clase son descritos o comparten el mismo conjunto de atributos y métodos.

Todos los objetos de una clase tienen el mismo formato y comportamiento, son diferentes únicamente en los valores que contienen sus atributos. Todos ellos responden a los mismos mensajes.

*Su sintaxis algorítmica es:*

```
Clase <Nombre de la Clase>
…
FClase <Nombre de la Clase>;
```

## Características generales

Una clase está en un nivel de abstracción alto. La clase permite la descripción de un conjunto de características comunes para los objetos que representa. Ejemplo: La clase Avión se consigue utilizar para concretar los atributos (tipo de avión, distancia, altura, velocidad de crucero, capacidad, país de origen, etc.) y los métodos (calcular posición en el vuelo, calcular velocidad de vuelo, estimar tiempo de llegada, despegar, aterrizar, volar, etc.) de los objetos particulares Avión que representa.

Un objeto es una instancia de una clase. Cada objeto concreto intrínsecamente de un sistema es miembro de una clase específica y posee el conjunto de atributos y métodos especificados en la misma.

Las clases se relacionan entre sí mediante una jerarquía. Entre las clases se establecen diferentes tipos de relaciones de herencia, en las cuales la clase hija (subclase) hereda los atributos y métodos de la clase padre (superclase), además de incorporar sus propios atributos y métodos.

Ejemplos, Superclase: Clase Avión Subclases de Avión: Clase Avión Comercial, Avión de Combate, Avión de Transporte.

Los nombres o identificadores de las clases deben colocarse en singular (clase Animal, clase Carro, clase Alumno).

## Relación entre clase y objeto

Desde una óptica de la matemática y la algoritmia, las clases son descripciones netamente estáticas o plantillas que describen objetos. Su rol es definir nuevos tipos conformados por atributos y operaciones.

Por el contrario, los objetos son instancias particulares de una clase. Las clases son una especie de molde de fábrica, en base al cual son construidos los objetos. Durante la ejecución de un programa sólo existen los objetos, no las clases.

La declaración de una variable de una clase **no** crea el objeto.

La asociación subsiguiente: <Nombre_Clase> <Nombre_Variable>; (por ejemplo, Rectángulo R), no concibe o no crea automáticamente un objeto Rectángulo. Sólo indica que R será una referencia o una variable de objeto de la clase Rectángulo.

La creación de un objeto, debe ser mostrada explícitamente por el programador, de manera análoga a como inicializamos las variables con un valor dado, sólo que para los objetos se hace a través de un método Constructor (ver punto Métodos).

### Atributo

Se denominan a los datos o variables que definen al objeto y cuyos valores en un instante dado muestran su estado.

Un atributo es una particularidad de un objeto. Mediante los atributos se concreta información oculta dentro de un objeto, la cual es manejada simplemente por los métodos definidos sobre dicho objeto Un atributo consta de un nombre y un valor. Cada atributo está asociado a un tipo de dato, que puede ser simple (entero, real, lógico, carácter, string) o estructurado (arreglo, registro, archivo, lista, etc.).

Su sintaxis algorítmica es:

```
<Modo de Acceso> <Tipo de dato> <Nombre del Atributo>;
```

Los modos de acceso son:

- Público: Atributos (o Métodos) que son penetrables desde fuera de la clase. Consiguen ser invocados por cualquier clase, aun si no está relacionada con ella. Esta cualidad de acceso igualmente se puede representar con el símbolo +.

- Privado: Atributos (o Métodos) que sólo son accesibles dentro de la implementación de la clase. También se puede simbolizar con el símbolo -.

- Protegido: Atributos (o Métodos) que son accesibles para la propia clase y sus clases hijas (subclases). También se puede representar con el símbolo #.

### Método

Son las operaciones (acciones o funciones) que se usan sobre los objetos y que consienten crearlos, modificar su estado o examinar el valor de sus atributos.

Los métodos establecen la secuencia de acciones que implementan las operaciones relativo los objetos. La implementación de los métodos no es visible fuera de objeto.

La sintaxis algorítmica de los métodos expresados como funciones y acciones es:

– Para funciones se pueden usar cualquiera de estas dos sintaxis:

<Modo de Acceso> Función <Nombre> [(Lista Parámetros)]:
<Descripción del Tipo de datos>

– Para acciones:

<Modo de Acceso> Acción <Nombre> [(Lista Parámetros)]
donde los parámetros son opcionales

Ejemplo: Un rectángulo es un objeto caracterizado por los atributos Largo y Ancho, y por varios métodos, entre otros Calcular su área y Calcular su perímetro.

### Características generales

Cada método posee un nombre, cero o más parámetros (por valor o por referencia) que recoge o devuelve y un algoritmo con el proceso del mismo.

En exclusivo se acentúa el método constructor, que no es más que el método que se ejecuta cuando el objeto es elaborado. Este constructor suele tener el mismo nombre de la clase/ objeto, pero aunque es una práctica común, el método constructor no necesariamente tiene que llamarse igual a la clase (al menos, no en pseudos–código). Es un método que recibe cero o más parámetros y lo usual es que inicialicen los valores de los atributos del objeto.

En lenguajes como Java y C++ se puede definir más de un método constructor, que normalmente se diferencian entre sí por la cantidad de parámetros que reciben.

Los métodos se ejecutan o activan cuando el objeto recibe un recado, remitido por un objeto o clase externo al que lo contiene, o por el mismo objeto de manera local.

### Creación de objetos y métodos constructores:

Cada objeto o instancia de una clase se obliga a ser creada a través de un método u operación especial denominado Constructor. Los atributos de un objeto toman valores iniciales dados por el constructor. Por convención el método constructor.

Tiene el mismo nombre de la clase y no se le asocia un modo de acceso (es público).

Algunos lenguajes proveen un método constructor por defecto para cada clase y/o permiten la definición de más de un método constructor.

### Método de destructores de objetos

Los objetos que ya no son usados en un programa, invaden inútilmente espacio de memoria, que es beneficioso rescatar en un momento dado. Según el lenguaje de programación utilizado esta tarea es dada al programador o es tratada automáticamente por el procesador o soporte de ejecución del lenguaje.

En la notación algorítmica **no** tomaremos en cuenta ese problema de administración de memoria, por lo tanto no definiremos formas para destruir objetos. En cambio al utilizar lenguajes de programación si debemos conocer los métodos destructores suministrados por el lenguaje y utilizarlos a fin de eliminar objetos una vez no sean útiles.

La herencia es otra de las aportaciones fundamentales de la programación orientada a objetos.

Consideremos las dos clases Coche y Ornitorrinco. Probablemente son tan disjuntas que nos parecen no tener nada en común (podríamos discutir esto…). Cada clase se definiría por su cuenta y ya está.

Pero pensemos en las clases Coche y Camión. Seguro que muchos de los atributos de Camión serán parecidos o iguales a los de Coche, y lo mismo ocurrirá con los métodos. Hasta ahora, con programación convencional, tendríamos que rescribir de nuevo todo el código y datos para Camión y gran parte coincidiría con los de Coche.

Como la herencia es un componente que consiente reutilizar código mostrando cómo una clase se diferencia de otra especificada previamente, tomando parte de sus características y cambiando o ampliando el resto. Aunque en la orientación a objetos coexisten muy diversos tipos de herencia, para abreviar diremos que cuando una clase hereda de otra, lo que está enunciando es una relación «es un». Por ejemplo si la clase Perro hereda de la clase Mamifero simbolizará que Perro es un Mamifero, es decir, todo lo que haga un Mamifero lo podrá hacer también Perro. En Java lo que queremos decir es que cuando una clase hereda de otra, sin que nosotros lo tengamos que especificar en su definición,

automáticamente tiene todos los atributos(datos) y métodos (funcionalidad) de la clase de la que hereda. Como hemos comentado esta clase hija podrá cambiar la especificación de estos miembros o añadir tantos como desee.

La herencia actúa de un modo estrictamente jerárquico. Cada clase tiene una superclase o clase padre (de la que hereda atributos y métodos) y varias (de 0 a n) subclases o clases hijas (que heredan de ella atributos y métodos). La herencia transfiere atributos y métodos hacia abajo en la jerarquía a través de todos los niveles.

En otros lenguajes orientados a objetos no es así, pero en Java se consuma:

– Sólo una clase es padre (es decir, sólo se hereda de una clase). Esto se conoce como herencia simple (otros lenguajes, como C++, tienen herencia múltiple, es decir una clase hereda de varias clases).

– Toda clase (excepto la especial Object) es subclase de otra. Object marca la cima de toda la jerarquía. Todas las clases heredan, directa o indirectamente, de ella.

La ventaja fundamental de la herencia es la reutilización de código. Es importante, no obstante, que el diseño de la jerarquía sea muy preciso para conseguir que esta reutilización sea útil.

Por ejemplo, ¿cómo rediseñaríamos jerárquicamente nuestro Coche y nuestro Camión? Aunque se escapa de esta unidad definir el proceso de diseño, pero un buen principio puede ser preguntarse ¿qué es más general? Si todo lo que tiene y hace un coche también lo hace un camión, y además más cosas, coche sería una clase padre de camión.

En este supuesto no es así (hay cosas de los camiones que no tienen los coches y viceversa): no se incluyen. Pero lo que sí podemos es concretar una clase más general que recoja el comportamiento común de coches y camiones, y que consiga ser padre de ambas clases. ¿Cómo la citamos? Vehículo podría ser un buen nombre (no siempre tiene que haber un buen nombre para este tipo de clases).

La idea sería concretar la clase vehículo, incluir en ella la información común a todos los vehículos; y después definir las clases Coche y Camión, ambas como hijas de Vehículo.

Esta jerarquía debería complicarse mucho más para constituir adecuadamente una entorno más complejo. Por ejemplo, podríamos fraccionar los vehículos en vehículos de tracción mecánica y vehículos de tracción animal, o bien en aéreos, terrestres, marinos y anfibios… no siempre hay una única manera de diseñar una jerarquía (más bien, casi nunca hay sólo una manera de crear una jerarquía). Se habla de estas jerarquías igualmente como clasificaciones, por la similitud que tiene con las clasificaciones de especies que a menudo se realizan en las ciencias naturales.

En cualquier caso, lo importante es recordar que una vez que ampliamos una característica (un atributo o un método) en una clase de la jerarquía, esa característica se propaga a todos sus hijos sin necesidad de rescribirla (aunque sí se podrá transformar o eliminar).

### Notación para la herencia

Vamos a creer que en nuestro programa queremos trabajar con un tipo especial de coche, un coche eléctrico. Podríamos crear la clase CocheElectrico desde cero; pero más lógico será pensar que un coche eléctrico hace todo lo que ya hacía un coche cualquiera, luego podemos aprovechar la clase Coche ya definida.

Para ello definimos la clase CocheElectrico como descendiente de Coche, del siguiente modo:

```
class CocheElectrico extends Coche {
}
```

Con esto no añadimos nada más, si queremos redefinir algún método de Coche o añadir alguna información podemos hacerlo de la misma forma que antes:

```
class CocheElectrico extends Coche {
        int autonomia;
        void visualizar() {
                System.out.println( “Este coche es eléctrico!!!!” );
                System.out.println( “Autonomía: “ + autonomia + “ kilómetros”
);
               super.visualizar();
        }
}
```

A partir de ahí podría acceder a la siguiente funcionalidad:

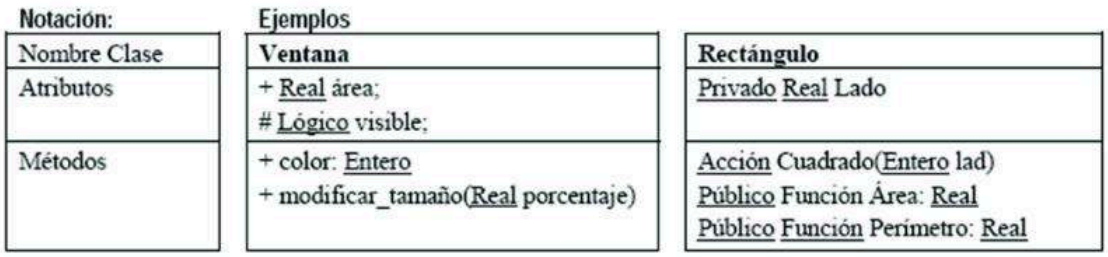
```
CocheElectrico miCocheElectrico = new CocheElectrico("azul", "Toyota Prius", 30);
miCocheElectrico.autonomia = 100;
miCocheElectrico.arrancar();
miCocheElectrico.visualizar();
```

## Diagramas de clase

La representación gráfica de una o diversas clases se formará mediante los nombrados Diagramas de Clase. Para los diagramas de clase se usará la simbología que provee el Lenguaje de Modelación Unificado (UML). Las clases se muestran como rectángulos divididos en tres partes. La primera contiene el nombre de la clase, la segunda contiene los atributos y la tercera los métodos.

Los modificadores de acceso a datos y operaciones, a saber: público, protegido y privado; se representan con los símbolos +, # y - respectivamente, al lado derecho del atributo. (+ público, # protegido, - privado).

En la figura se muestra el método "color" que no tiene ningún parámetro y retorna un valor entero y el método "modificar_tamaño" que tiene un real como parámetro y no retorna nada (es una acción).




*Ejemplo de representación gráfica de clase con diagramas de clase en UML*

Las clases no se fundan para trabajar de manera aislada, la idea es que ellas se consigan relacionar entre sí, de forma que puedan compartir atributos y métodos sin necesidad de rescribirlos.

La posibilidad de establecer jerarquías entre las clases es una característica que diferencia esencialmente la programación orientada a objetos de la programación tradicional, ello debido fundamentalmente a que permite extender y reutilizar el código existente sin tener que rescribirlo cada vez que se necesite.

Los cuatro tipos de relaciones entre clases estudiados en esta unidad serán:

– Herencia (Generalización / Especialización o Es–un)

– Agregación (Todo / Parte o Forma–parte–de)

– Composición (Es parte elemental de)

– Asociación (entre otras, la relación Usa–a)

## Modelo de Clases

### Introducción

Un diagrama de clases vale para concebir las relaciones entre las clases que envuelven el sistema, las cuales logran ser asociativas, de herencia, de uso.

Un diagrama de clases está compuesto por los siguientes componentes:

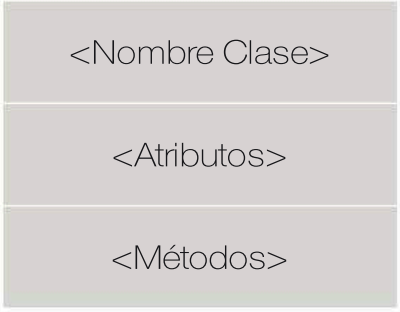
– Clase: atributos, métodos y visibilidad.

– Relaciones: Herencia, Composición, Agregación, Asociación y Uso.

### Elementos

Clase: es el elemento básico que encapsula toda la información de un Objeto (un objeto es una instancia de una clase). A través de ella conseguimos modelar el entorno en estudio (una Casa, un Auto, una Cuenta Corriente, etc.).

En UML, una clase es representada por un rectángulo que posee tres divisiones:



Donde:

– Superior: Contiene el nombre de la Clase

– Intermedio: Contiene los atributos (o variables de instancia) que caracterizan a la Clase (pueden ser private, protected o public).

– Inferior: Contiene los métodos u operaciones, los cuales son la forma como interactúa el objeto con su entorno (dependiendo de la visibilidad: private, protected o public).

Ejemplo:

Una Cuenta Corriente que posee como característica: Balance

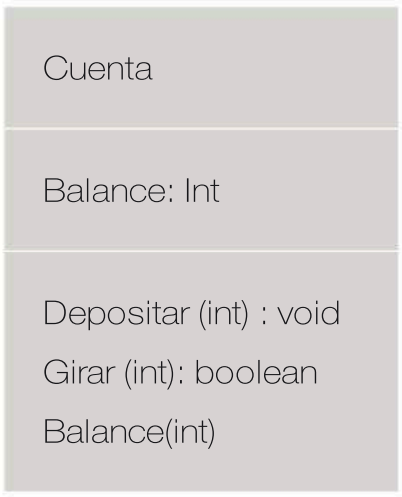
Puede realizar las operaciones de:

– Depositar

– Girar

– Balance

El diseño asociado es:



### Atributos y métodos:

– Atributos:

  Los atributos o características de una Clase son de tres tipos, los que concretan el grado de comunicación y visibilidad de ellos con el ambiente, estos son:

  · public (+,): Muestra que el atributo será visible tanto dentro como fuera de la clase, es accsesible desde todos lados.

  · private (-,): Indica que el atributo sólo es accesible desde dentro de la clase (sólo sus métodos lo pueden accesar).

  · protected (#,): Indica que el atributo no será accesible desde fuera de la clase, pero si podrá ser accesado por métodos de la clase además de las subclases que se deriven (ver herencia).

– Métodos:

  Los métodos u operaciones de una clase son la manera de como ésta interactúa con su entorno, éstos pueden poseer las características:

  · public (+,): Indica que el método será visible tanto dentro como fuera de la clase, es decir, es accsesible desde todos lados.

  · private (-,): Indica que el método sólo será accesible desde dentro de la clase (sólo otros métodos de la clase lo pueden accesar).

  · protected (#,): Indica que el método no será accesible desde fuera de la clase, pero si podrá ser accesado por métodos de la clase asimismo de métodos de las subclases que se deriven.

## Relaciones entre clases

Una vez definido la significación de Clase, es ineludible exponer como se pueden interrelacionar dos o más clases (cada uno con características y objetivos diferentes).

Antes es necesario explicar el concepto de cardinalidad de relaciones: En UML, la cardinalidad de las relaciones indica el grado y nivel de dependencia, se anotan en cada extremo de la relación y éstas pueden ser:

– Uno o muchos: 1..* (1..n).

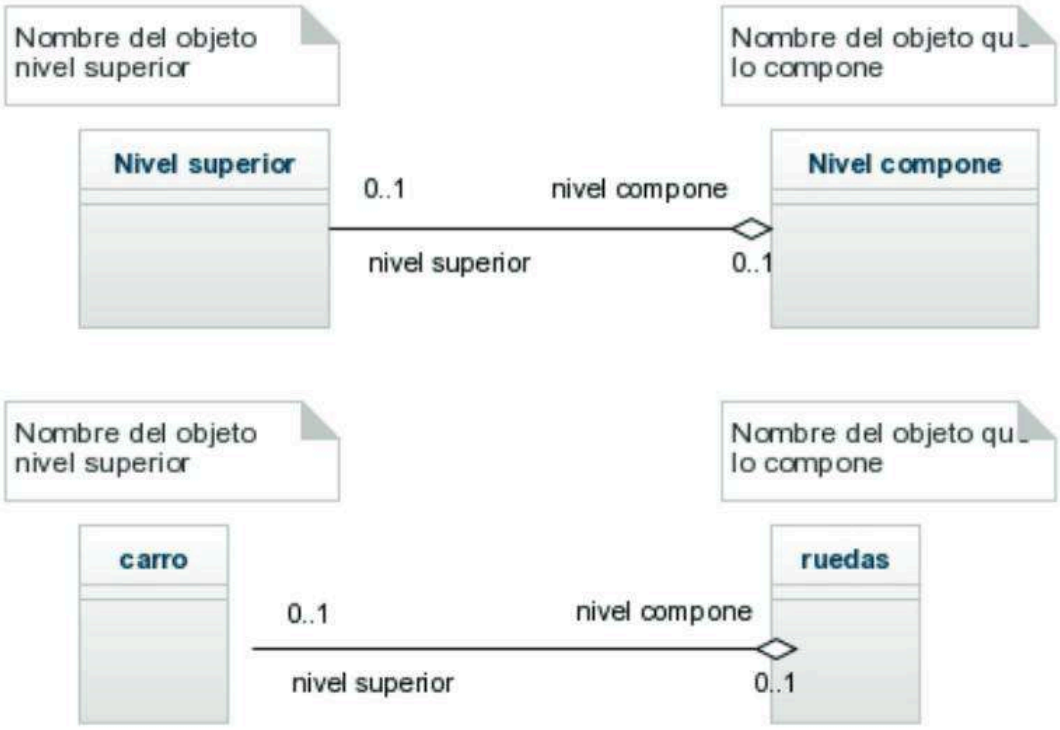
– 0 o muchos: 0..* (0..n).

– Número fijo: m (m denota el número).

## 4.1. Distinción entre agregación/composición

### Relación de agregación (todo/parte, forma parte de)

Es una relación que simboliza a los objetos compuestos por otros objetos. Muestra Objetos que a su vez están constituidos por otros. El objeto en el nivel superior de la jerarquía es el todo y los que están en los niveles inferiores son sus partes o componentes.




*Ejemplo de representación*

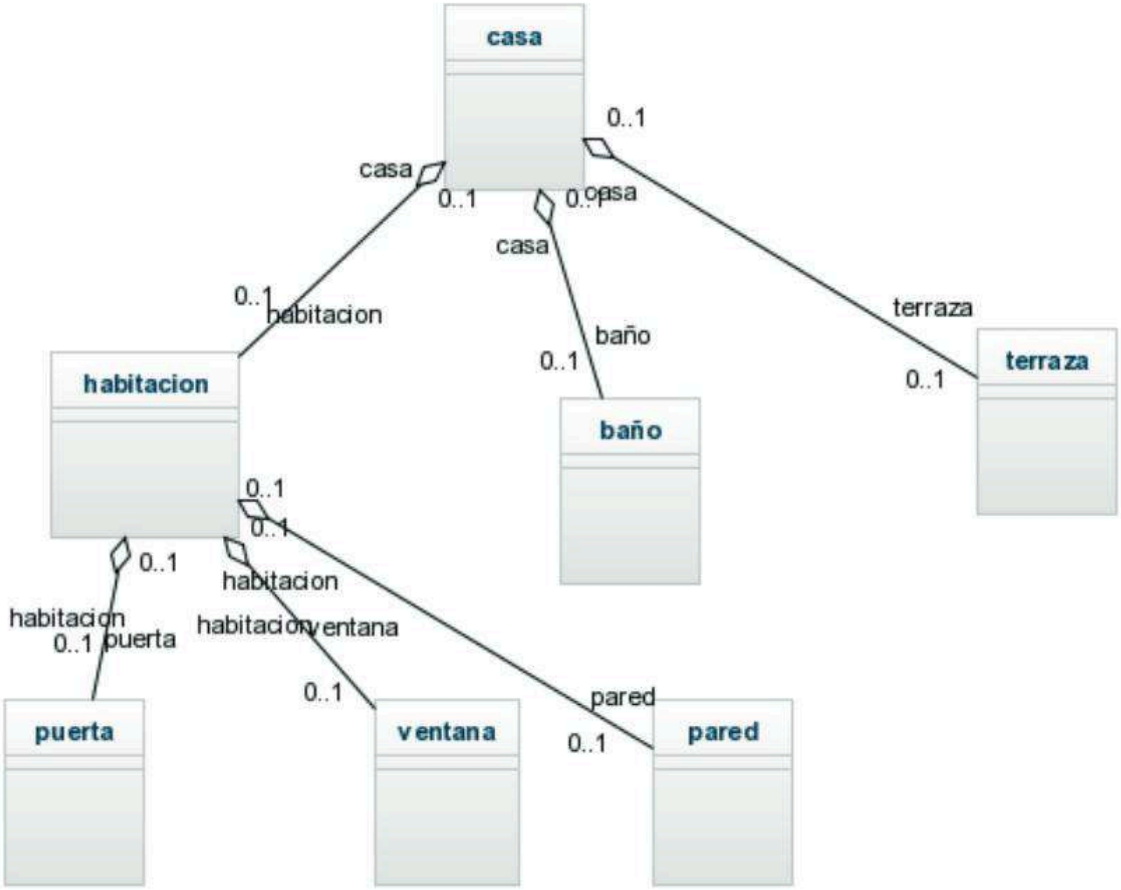
### Representación en el diagrama de clase

La relación "**forma parte de",** no es más que una asociación, que se denota:

Si motor forma parte de carro, la flecha apunta a la clase motor, y el diamante va pegado a carro.

La multiplicidad es el rango de cardinalidad consentido que puede tomar la asociación, se denota LI..LS. Se puede usar * en el límite superior para representar una cantidad ilimitada (ejemplo: 3..*).

Ejemplo: Objeto Casa descrito en términos de sus componentes:



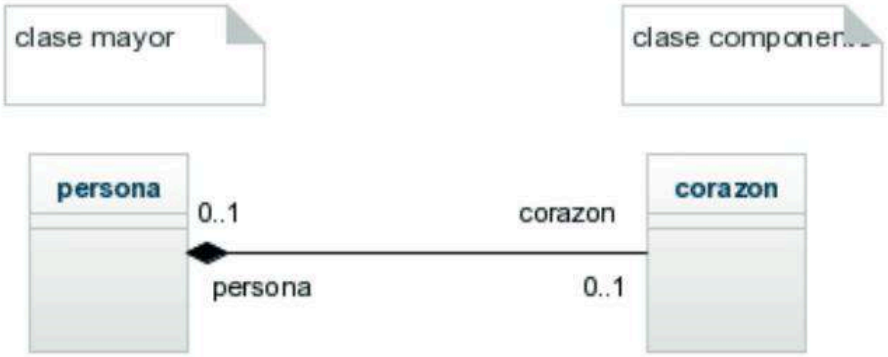

*Objeto casa descrito en términos de sus componentes*

## Frente a relación de composición

Un componente es parte esencial de un elemento. La relación es más enérgica que el caso de agregación, al punto que si el componente es eliminado o desaparece, la clase mayor deja de existir.

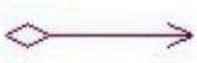
Representación en el Diagrama de Clases.

La relación de composición, se denota de la siguiente forma:




*Relación de composición en el diagrama de clases*

## Agregación

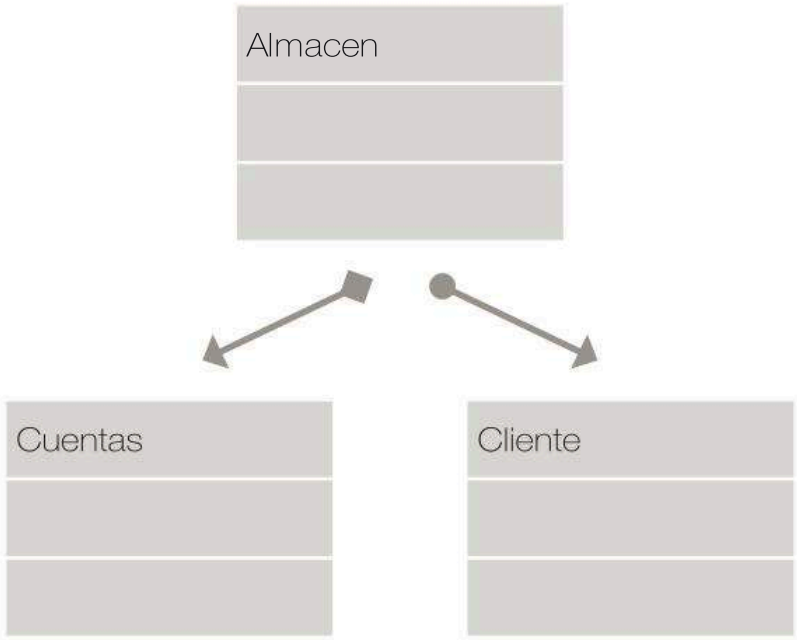


Para componer objetos complicados, no bastan los tipos de datos básicos que suministran los lenguajes: enteros, reales y secuencias de caracteres. Cuando se solicita componer objetos que son instancias de clases definidas por el desarrollador de la aplicación, poseemos dos posibilidades:

– Por Valor: Es un ejemplo de relación estática, en donde el período de vida del objeto contenido está condicionado por el tiempo de vida del que lo incluye. Este tipo de relación es frecuentemente llamada Composición (el Objeto base se construye a partir del objeto incluido, es decir, es “parte/todo”).

– Por Referencia: Es un tipo de relación dinámica, en donde el período de vida del objeto incluido es independiente del que lo incluye. Este tipo de relación es comúnmente llamada Agregación (el objeto base utiliza al incluido para su funcionamiento).

Un ejemplo es el siguiente:




En donde se acentúa que:

– Un Almacen posee Clientes y Cuentas (los rombos van en el objeto que ostenta las referencias).

– Cuando se destruye el Objeto Almacen igualmente son destruidos los objetos Cuenta asociados, en cambio no son afectados los objetos Cliente asociados.

– La composición (por Valor) se matiza por un rombo relleno.

– La agregación (por Referencia) se destaca por un rombo transparente.

La flecha en este tipo de relación indica la navegabilidad del objeto referenciado. Cuando no existe este tipo de particularidad la flecha se elimina.

## 4.2. Distinción entre generalización/especialización

Herencia simple: una subclase tiene exactamente un padre. Herencia múltiple: una subclase hereda de varios padres, sin cambio de estructura o comportamiento Semánticamente la relación se puede verificar comprobando si la subclase es un tipo de su padre Es un tipo de jerarquía de clases, en la que cada subclase contiene los atributos y métodos de una (herencia simple) o más superclases (herencia múltiple).

Mediante la herencia las instancias de una clase hija (o subclase) pueden acceder tanto a los atributos como a los métodos públicos y protegidos de la clase padre (o superclase). Cada subclase o clase hija en la jerarquía es siempre una extensión (esto es, conjunto estrictamente más grande) de la(s) superclase(s) o clase(s) padre(s) y además incorporar atributos y métodos propios, que a su vez serán heredados por sus hijas.

Los miembros declarados como protected, public o package son accesibles en las subclases. Sin embargo, los atributos no pueden declararse como public o package (encapsulamiento) Los métodos declarados como final no pueden redefinirse en las subclases Los métodos declarados como abstract, son implementados en las subclases.

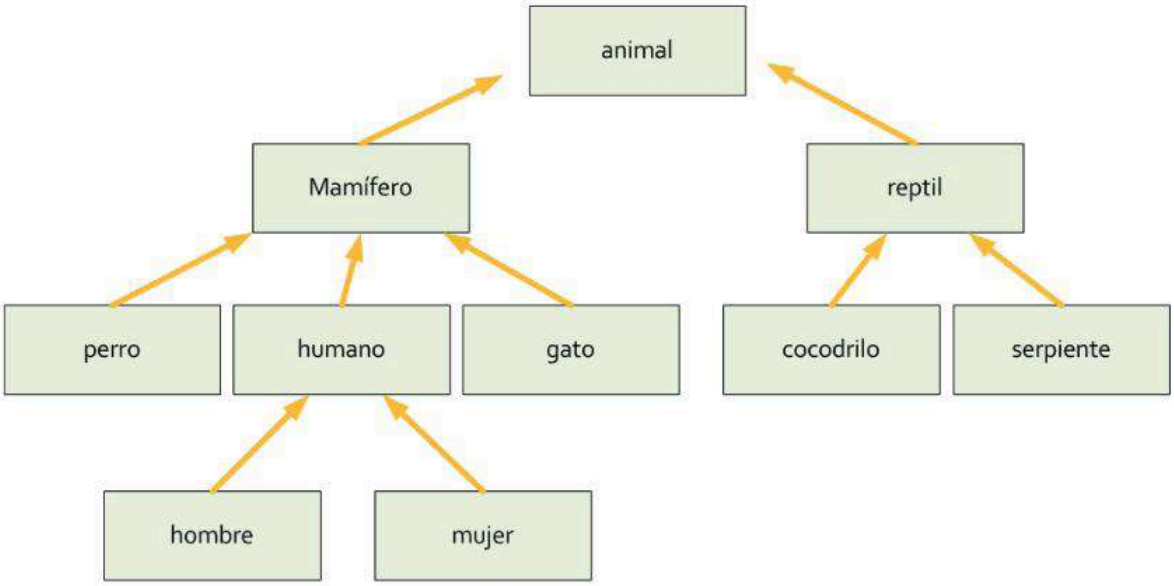
### Representación

En la notación algorítmica: Se coloca el nombre de la clase padre después de la frase Hereda de del encabezado de la clase y se usan sus atributos y métodos públicos o protegidos.

Ejemplo: Clase Punto3D Hereda de Punto2D

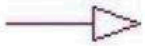
En el diagrama de clases: La herencia se representa mediante una relación de generalización/especificación, que se denota de la siguiente forma:

clase hija -- → clase madre subclase - → superclase

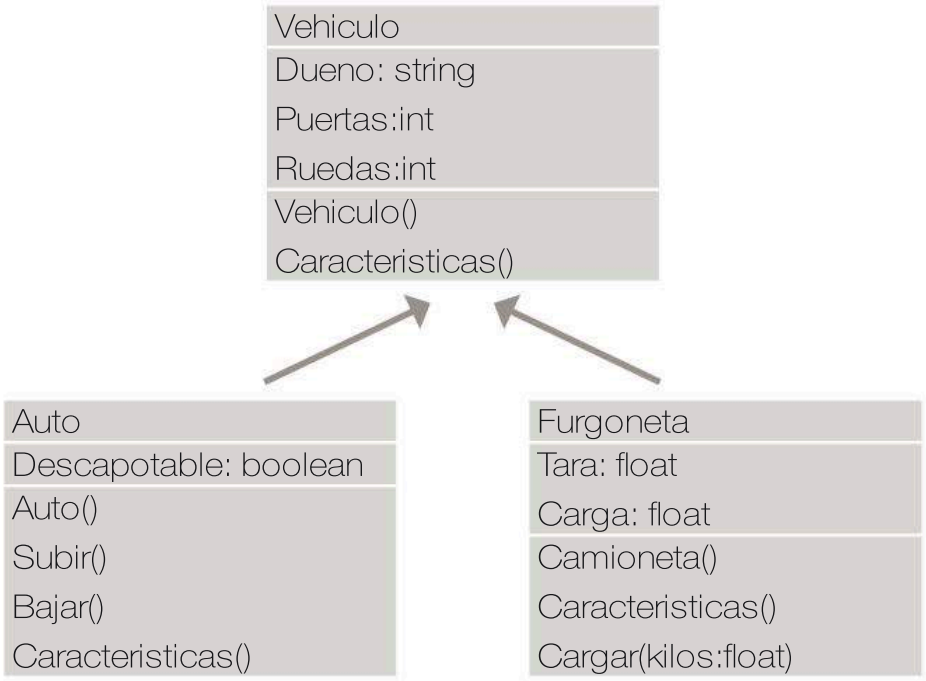



*Ejemplo de representación gráfica de clase animal y sus hijas*

## Herencia (especialización/generalización)



Enseña que una subclase hereda los métodos y atributos descritos por una Super Clase, por ende la Subclase además de poseer sus oportunos métodos y atributos, tendrá las características y atributos perceptibles de la Super Clase (public y protected), ejemplo:



En la figura se especifica que Auto y Camión heredan de Vehículo, es decir, Auto posee las Características de Vehículo (Precio, VeloMaxim, etc) además posee algo particular que es Descapotable, en cambio Camión también hereda las características de Vehiculo (Precio, VeloMaxim, etc) pero posee como particularidad propia Acoplado, Tara y Carga.

Cabe destacar que fuera de este entorno, lo único "visible" es el método Caracteristicas aplicable a instancias de Vehículo, Auto y Camión, pues tiene definición publica, en cambio atributos como Descapotable no son visibles por ser privados.

## 4.3. Identificación de asociaciones

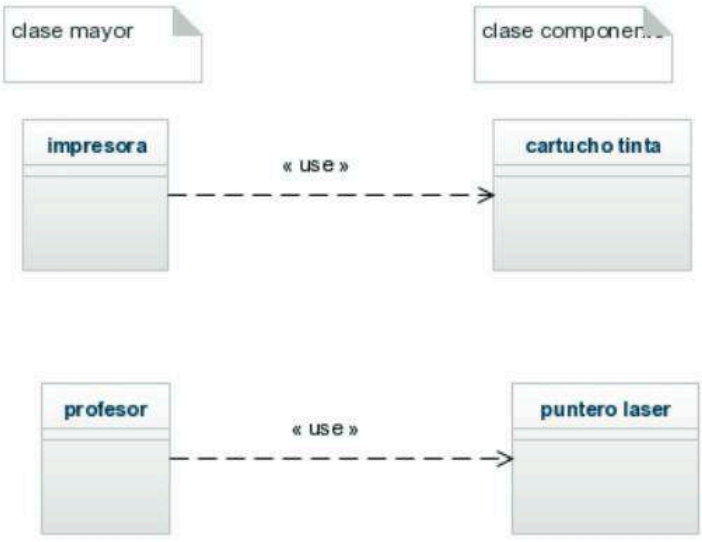
### Relación de asociación («uso», usa, cualquier otra relación)

Es una asociación que se constituye cuando dos clases poseen una dependencia de uso, es decir, una clase utiliza atributos y/o métodos de otra para funcionar. Estas dos clases no forzosamente están en jerarquía, es decir, no necesariamente una es clase padre de la otra, a diferencia de las otras relaciones de clases.

El ejemplo más habitual de esta relación es de objetos que son utilizados por los humanos para alguna función, como Lápiz (se usa para escribir), tenedor (se usa para comer), silla (se usa para sentarse), etc. Otro ejemplo son los objetos Cajero y Cuenta.

El Cajero "usa a" la cuenta para hacer las transacciones de consulta y retiro y verificar la información del usuario.

Representación en el Diagrama de Clases La relación de uso, se denota con una dependencia estereotipada:




*Ejemplo de representación gráfica de una asociación de dos clases*

## Dependencia

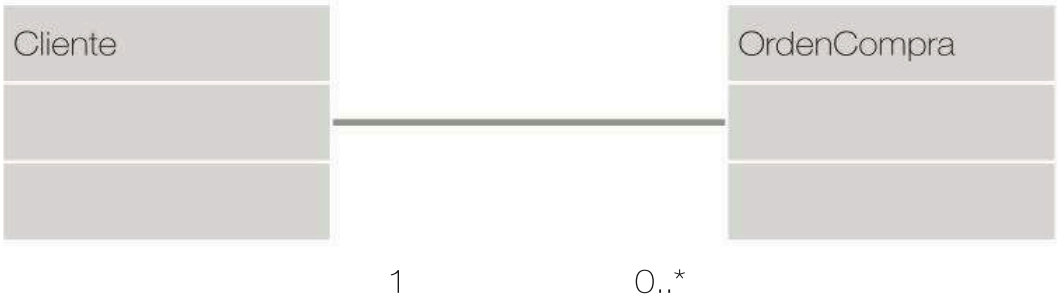
Es una relación de uso en donde los cambios en la especificación de una clase, pueden afectar a otra clase que la usa.

## Asociación



La relación entre clases conocida como Asociación, admite asociar objetos que colaboran entre si. Cabe subrayar que no es una relación fuerte, es decir, el período de vida de un objeto no depende del otro.

Ejemplo:



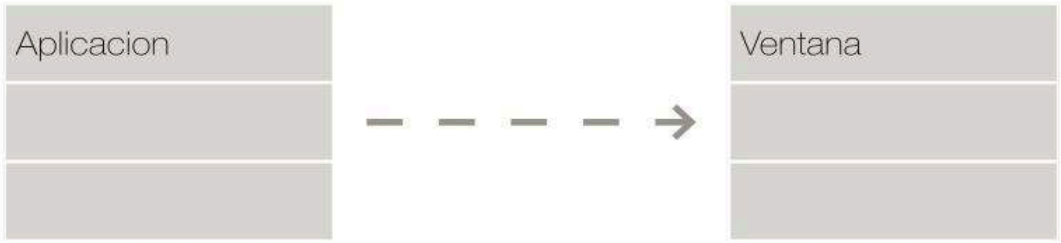

Un cliente puede tener asociadas muchas Ordenes de Compra, en cambio una orden de compra solo puede tener asociado un cliente.

## Dependencia o instanciación (uso)



Simboliza un tipo de relación muy específico, en la que una clase es instanciada (su instanciación es subordinado de otro objeto/clase). Se revela por una flecha punteada.

El uso más exclusivo de este tipo de relación es para expresar la dependencia que tiene una clase de otra, como por ejemplo una aplicación grafica que instancia una ventana (la creación del Objeto Ventana esta condicionado a la instanciación proveniente desde el objeto Aplicacion):

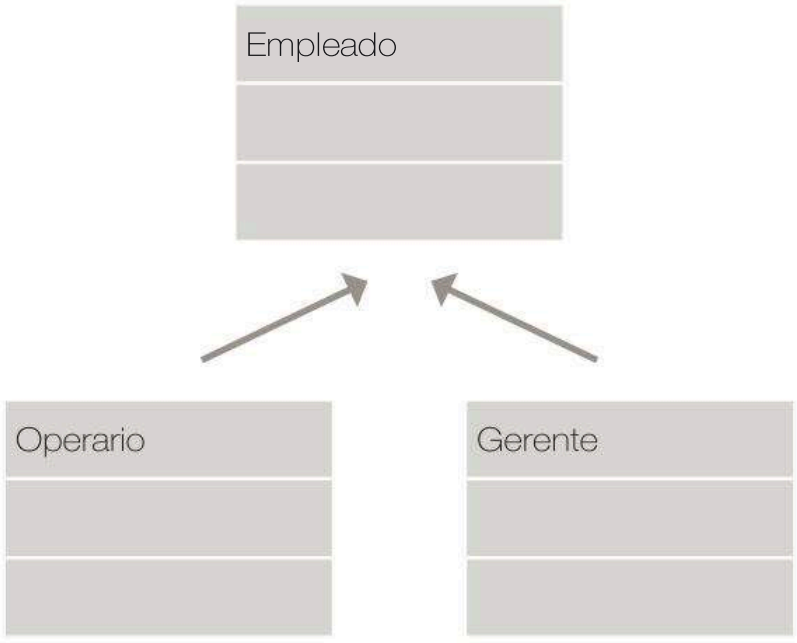



Cabe recalcar que el objeto elaborado (en este caso la Ventana gráfica) no se acumula dentro del objeto que lo crea (en este caso la Aplicación).

## Casos particulares

### Clase abstracta

Una clase abstracta se expresa con el nombre de la clase y de los métodos con letra "itálica". Esto muestra que la clase concretada no puede ser instanciada pues tiene métodos abstractos (aún no han sido definidos, es decir, sin implementación). La única forma de utilizarla es concretando subclases, que implementan los métodos abstractos definidos.



# UD4
# Lo más importante

- Dentro del paradigma PPO de programación orientada a objetos, la clase es el eje principal donde pivota la propiedad de modularidad.

- Es una estructura como una plantilla que posee la descripción general de una colección de objetos. Consta de atributos y métodos que resumen las características y el comportamiento comunes de un conjunto de objetos.

- Un atributo es una característica de un objeto. Mediante los atributos se define información oculta es manipulada solamente por los métodos definidos sobre dicho objeto Un atributo consta de un nombre y un valor.

- Los métodos son las operaciones (acciones o funciones) que se aplican sobre los objetos y que permiten crearlos, cambiar su estado o consultar el valor de sus atributos.

- Cada objeto o instancia de una clase debe ser creada explícitamente a través de un método u operación especial denominado Constructor. Los atributos de un objeto toman valores iniciales dados por el constructor.

- La herencia es un mecanismo que permite reutilizar código indicando cómo una clase se diferencia de otra definida previamente, tomando parte de sus características (o todas) y cambiando o ampliando el resto. Aunque en la orientación a objetos existen muy diversos tipos de herencia, para simplificar diremos que cuando una clase hereda de otra, lo que está expresando es una relación “es un”.

– La ventaja fundamental de la herencia es la reutilización de código. Es importante, no obstante, que el diseño de la jerarquía sea muy preciso para conseguir que esta reutilización sea útil.

– La representación gráfica de una o varias clases se hará mediante los denominados Diagramas de Clase. Para los diagramas de clase se utilizará la notación que provee el Lenguaje de Modelación Unificado (UML).

– En UML las clases se denotan como rectángulos divididos en tres partes. La primera contiene el nombre de la clase, la segunda contiene los atributos y la tercera los métodos.

– Mediante la herencia las instancias de una clase hija (o subclase) pueden acceder tanto a los atributos como a los métodos públicos y protegidos de la clase padre (o superclase).

# UD4
## Autoevaluación

1. Las clases se relacionan entre sí mediante:
   a. La Asociación.
   b. Una jerarquía.
   c. Los objetos.
   d. La Herencia.

2. ¿Los modos de acceso a un atributo son?
   a. Público – Privado y Protegido.
   b. Público – Abstract – Private.
   c. Final – Private – Protect.
   d. Ninguno de los anteriores.

3. En una notación con Herencia en Java escribiremos:
   a. Class CocheElectrico import Coche.
   b. Class CocheElectrico implement Coche.
   c. Class CocheElectrico extends Coche.
   d. Ninguna de las anteriores.

4. En Lenguaje de Modelación Unificado (UML). Las clases se definen como:
   a. Rectángulos divididos en tres partes.
   b. Solo Rectángulos.
   c. Rectángulos añadiendo partes según sus atributos.
   d. Ninguna de las anteriores.

5. Los cuatro tipo de relaciones entre clases son:
   a. Herencia – Agregación – Composición – Asociación.
   b. Herencia – Asociación – Polimorfismo – Especialización.
   c. Agregación – Especialización – Herencia – Asociación.
   d. Ninguna de las anteriores.

6. En una relación entre clases la ordinalidad (1 a 0*) cual seria la respuesta correcta?
   a. Ciudadano – DNI.
   b. Coche – Marca.
   c. Cliente – Orden de Compra.
   d. Aulas – Estudiantes.

7. En una Relación de Agregación:
   a. Es una relación que representa a los objetos compuestos por otros objetos.
   b. Se establece cuando dos clases tienen una dependencia de utilización.
   c. Permite asociar objetos que colaboran entre si.
   d. Ninguna de las anteriores.

8. En el diagrama de clases: La herencia se representa mediante una relación de:
   a. Clase Main → Superclase.
   b. Clase Hija → superclase.
   c. Clase hija → clase madre subclase → superclase.
   d. Ninguna de las anteriores.

9. En Herencia los métodos declarados como abstract:
    a. Son implementados en las subclases.
    b. Son heredados con la clase Padre.
    c. Son heredados de la Superclase.
    d. Ninguna de las anteriores.

10. En una relación de Asociación:
    a. Dos clases son necesariamente hija y padre.
    b. No necesariamente una es clase padre de la otra.
    c. Es una relación fuerte.
    d. Ninguna de la anteriores.

Área: informática y comunicaciones

# UD5

## Análisis del polimorfismo

5.1. Concepto

5.2. Tipos

5.3. Polimorfismo en tiempo de compilación (sobrecarga)

5.4. Polimorfismo en tiempo de ejecución (ligadura dinámica)

5.5. Objetos polimórficos

5.6. Comprobación estática y dinámica de tipos

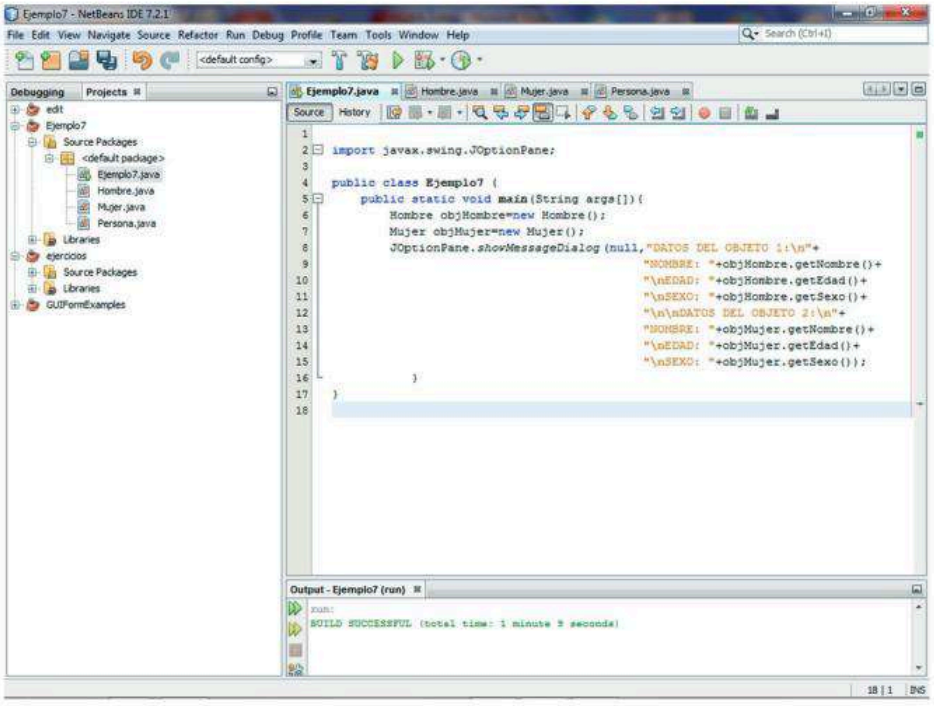
## 5.1. Concepto

En esta unidad trataré de documentar con unos ejercicios y unas explicaciones la propiedad Polimorfismo en el lenguaje Java. Una peculiaridad que incrementa la posibilidad de hacer una programación más transparente, limpia y a la vez potente en la reutilización de software.

Lo podemos considerar como otra denominación como la capacidad de una entidad de referenciar distintos elementos en distintos instantes de tiempo.

El polimorfismo nos permite programar de manera general en lugar de programar de manera específica.

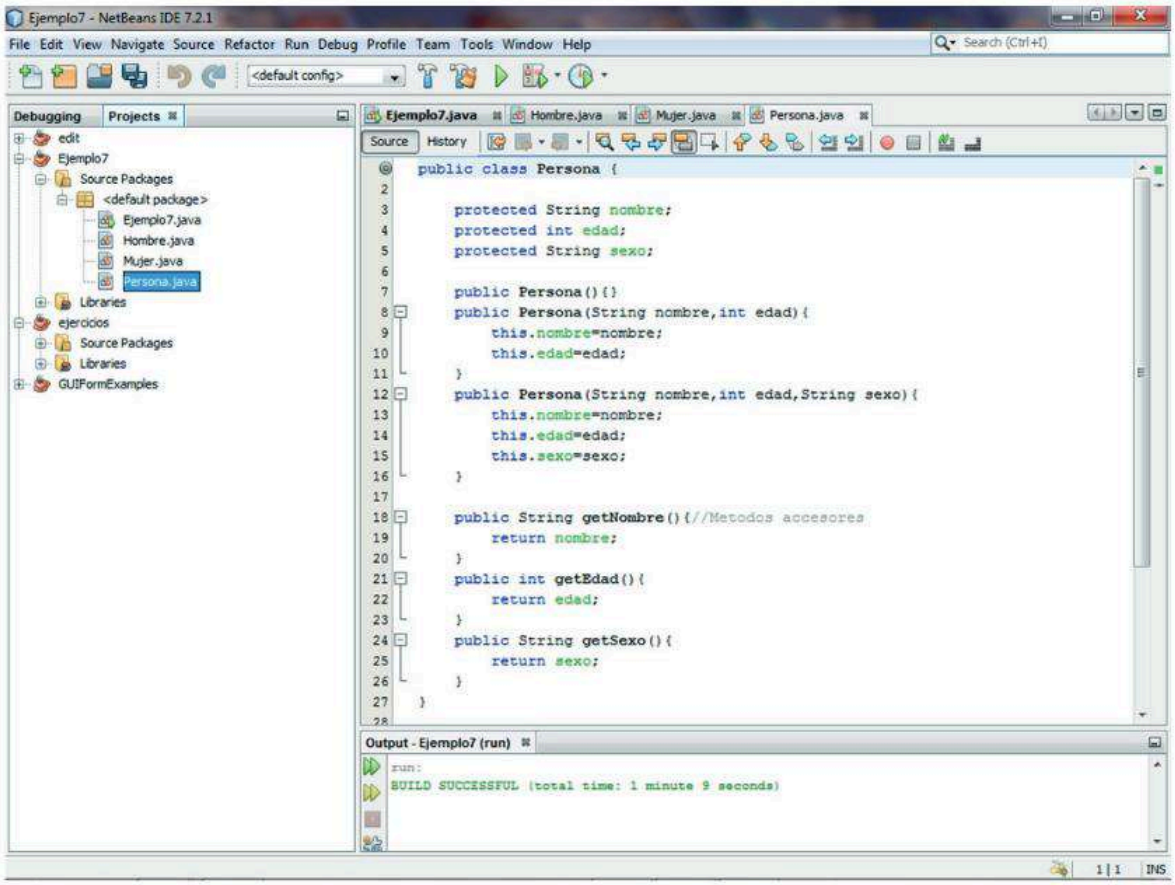
Pondré un ejemplo más genérico para luego entrar en detalles con las cuatro técnicas distintas.




*Ejemplo de código del Main Project denominado ejemplo7*

En la pantalla de la figura se muestra como en un ejemplo de Polimorfismo en código fuente Java, se observa cómo se consigue que un objeto de una clase, se comporte como un objeto de cualquiera de sus subclases Hombre, Mujer, en este caso la Superclase es Persona.

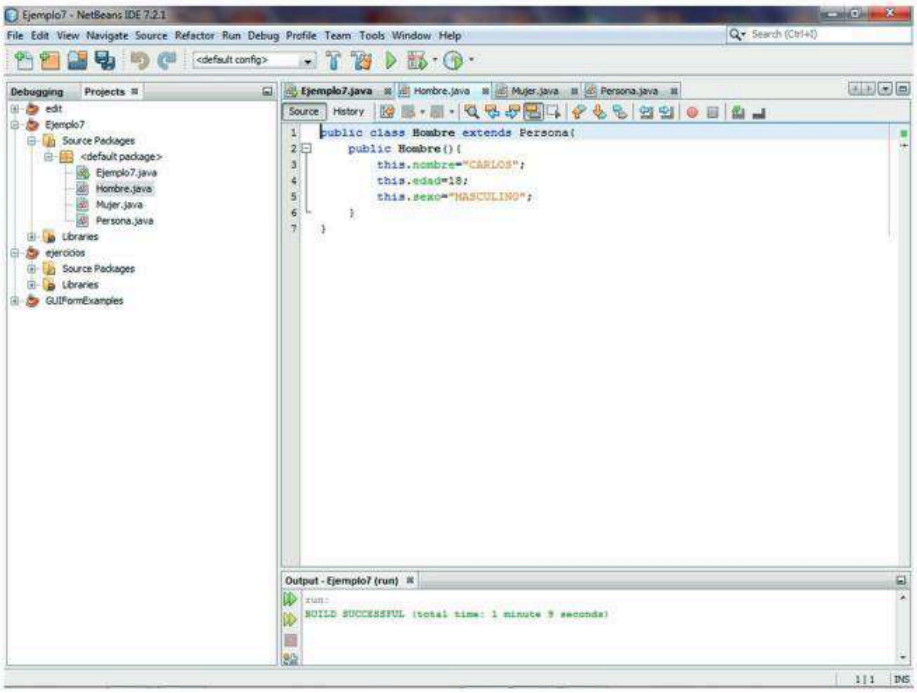
La siguiente serie de imágenes detallan la modularidad en subclases a partir de una superclase denominada Persona, una clase principal del proyecto y dos subclases denominadas Hombre y mujer correlativamente. Es un ejemplo muy didáctico para comprender la propiedad Polimorfismo en Java.




*Ejemplo de código fuente de la superclase Persona*

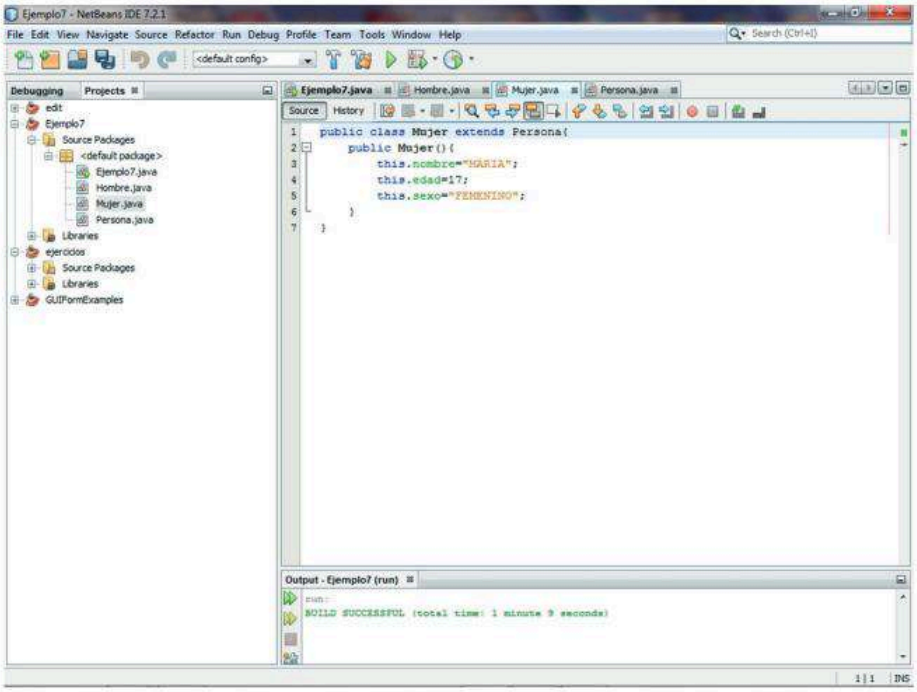
Ejemplo de polimorfismo con la Superclase Persona. Con la declaación de sus atributos y la referencia y constructores .this y los get.

Las siguientes pantallas describen las subclases Hombre y Mujer y puede observarse la simplicidad del código fuente.



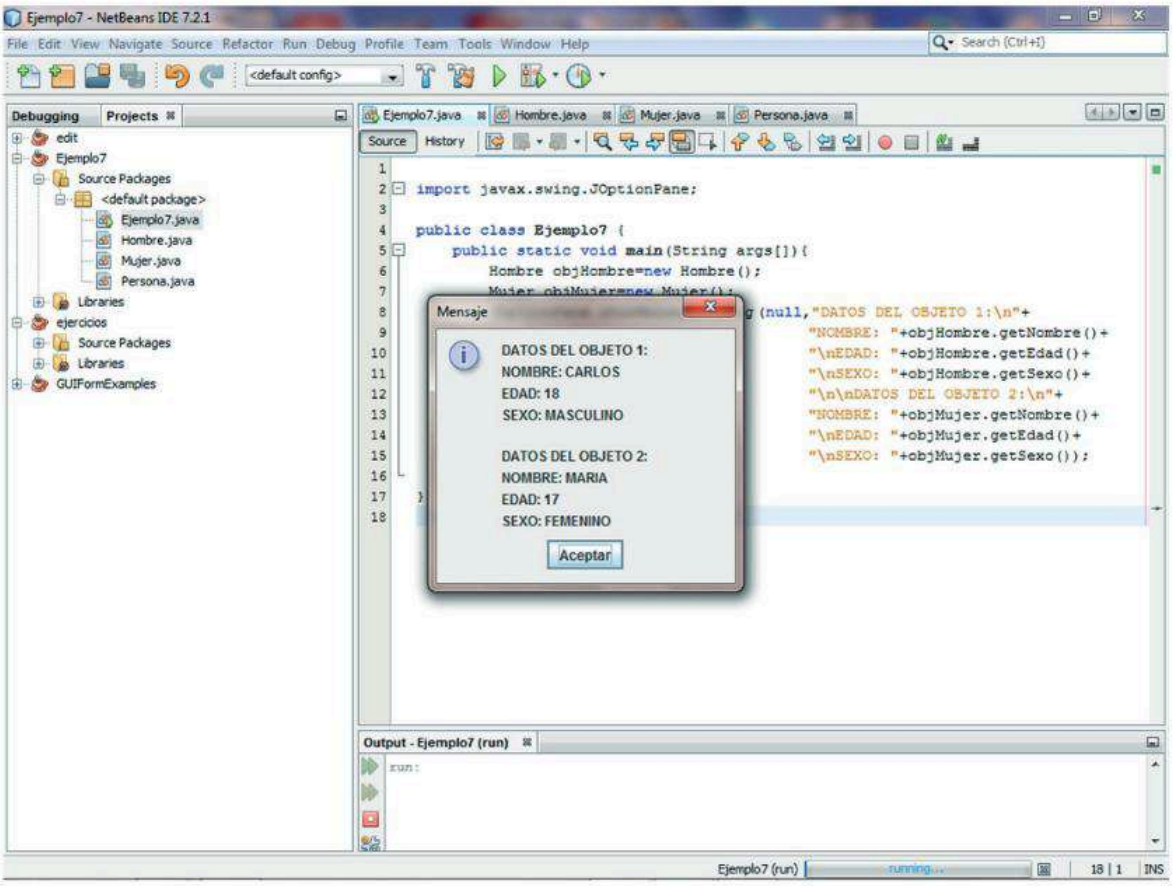

*Ejemplo de código fuente de la subclase Hombre*

Ejemplo de Polimorfismo con la subclase Hombre con los .this y valores de los atributos.




*Ejemplo de código fuente de la subclase Mujer*

Ejemplo de Polimorfismo con la subclase Mujer con los .this y valores de los atributos.




*Ejemplo del resultado final una vez compilado y ejecutado el pequeño proyecto*

La siguiente pantalla puede verse el resultado una vez que se ha compilado y ejecutado el programa dentro del proyecto en Netbeans. Para que se vea algo más vistoso se ha utilizado la clase de Java Swing,JOptionPane con algún método para lanzar un cuadro de diálogo en primer plano.

Hay cuatro técnicas, cada una de las cuales permite una forma distinta de reutilización de software, que facilita a su vez el desarrollo rápido, la confianza y la facilidad de uso y mantenimiento.

- Sobrecarga
- Sobreescritura
- Variables polimórficas
- Genericidad

El polimorfismo es el modo en que los lenguajes OO implementan el concepto de polisemia del mundo real: Un único nombre para muchos significados, según el contexto.

En programación orientada a objetos se denomina polimorfismo a la capacidad que tienen los objetos de una clase de responder al mismo mensaje o evento en función de los parámetros utilizados durante su invocación. Un objeto polimórfico es una entidad que puede contener valores de diferentes tipos durante la ejecución del programa.




Concretando, el polimorfismo consiste en permitir que un objeto de una clase se comporte como un objeto de cualquiera de sus subclases, obedeciendo la forma de llamar a los métodos de dicha clase o subclases.

Una forma de conseguir objetos polimórficos es mediante el uso de referencias (como si de punteros se tratase, que son más comunes en lenguajes como C) a la superclase. De esta forma podemos tener dentro de una misma estructura (arrays, listas, pilas, colas, ...) objetos de distintas subclases, haciendo que el tipo base de dichas estructuras sea una referencia a la superclase. Otros lenguajes nos dan la posibilidad de crear objetos polimórficos con el operador new. La forma de utilizarlo, por ejemplo en Java, sería:

```
Superclase sup = new (Subclase);
```

En la práctica esto quiere explicar que una referencia a un tipo puede contener varios tipos diferentes, no solo el construido. De esta manera podemos tener una referencia a un objeto de la clase Persona, pero esta referencia puede estar apuntando a un objeto subclase de la anterior como podría ser Alumno, Profesor o empleado (todas ellas deberían ser subclase de Persona).

La noción de polimorfismo se puede utilizar tanto a funciones como a tipos de datos. Así surgen los conceptos de funciones polimórficas y tipos polimórficos. Las primeras son aquellas funciones que permiten valorarse o ser empleadas a diferentes tipos de datos de forma indiferente; los tipos polimórficos, mientras tanto, son aquellos tipos de datos que poseen al menos un elemento cuyo tipo no está especificado.

## 5.2. Tipos

– **Sobrecarga denominado Overloading, o Polimorfismo ad–hoc**: un solo nombre de método y muchas implementaciones diferentes. Es una característica muy utilizada y que ahorra memoria al no tener que declarar demasiadas implementaciones distintas de variables y métodos. La sobrecarga permite en un solo método nombrado de una misma manera, bastantes aplicaciones y utilizaciones en el código para ejecutar los métodos apropiados. Si son funciones en sobrecarga, hay una diferencia y es que las funciones sobrecargadas habitualmente se diferencian en tiempo de compilación por poseer distintos parámetros de entrada y/o salida. Si fueran los mismos no tendría sentido pues estaríamos reiterando información y variables y datos.

– **Sobreescritura llamada Overriding, o Polimorfismo de inclusión**: Tipo especial de sobrecarga que sucede dentro de relaciones de herencia. El Overriding tiene un papel muy especial cuando usamos herencia en las clases y acceder a sus métodos y subclases. El polimorfismo de inclusión con esta propiedad la signatura es la misma, partimos de refinamiento o reemplazo del método del padre pero los métodos se hallan en dos clases distintas relacionadas mediante herencia. El acceso desde las subclases hijas a los métodos sobrescritos permite que haya un código fuente claro, preciso y acortado en métodos sin necesidad de duplica código.

– **Variables polimórficas o Polimorfismo de asignación**: variable que se expresa como de un tipo pero que referencia en realidad un valor de un tipo distinto. Es una peculiaridad y tipo muy usado el aplicar variables polimórficas cuando una variable polimórfica se emplea como argumento, la función proveniente se dice que presenta un polimorfismo puro.

  Una variable que apunta a un objeto admite distintas formas de ese objeto: las formas definidas por la superclase y las clases que extienden a la superclase. Es decir, una variable que apunta a un objeto es polimórfica porque admite distintos tipos de objetos, no solo uno. Una variable de tipo Persona admite tanto objetos de tipo Persona, como objetos de tipo Profesor, como objetos de tipo ProfesorGimnasia. O visto de otra manera, podríamos decir que un ProfesorGimnasia es una forma de Profesor, pero también una forma de Persona, y también una forma de Object. Las variables pueden ser polimórficas.

La posibilidad de uso de variables polimórficas reduce la cantidad de código que es necesario escribir y facilita la reusabilidad. Por ejemplo no necesitamos dos bucles:

```
for (ProfesorGimnasia tmp : listado) {    ...     }
for (ProfesorCiencias tmp : listado) {   ...    }
```

Sino solo uno:

```
for (Profesor tmp: listado) {   ...   }
```

- **Genericidad como plantillas o templates:** forma de establecer herramientas de propósito general (clases, métodos) y especializarlas para situaciones específicas.

  · Sobrecarga

```
Listado::imprimir()
Listado::imprimir(int numCopias)
ListaVentas::imprimir()
```

  · Sobreescritura

```
CarnetBiblioteca::prestarLibro()
CarnetJoven::ofertarLibro()
```

- Variables polimórficas

```
CarnetBiblioteca *pc=new CarnetJoven();
```

- Genericidad

```
Lista<Alumno>
Lista<Aula>
Lista<Curso>
Lista<Habitacion>
```

## 5.3. Polimorfismo en tiempo de compilación (sobrecarga)

Un mismo nombre de mensaje asociado a varias implementaciones.

La sobrecarga se realiza en tiempo de compilación (enlace estático) en función de la signatura completa del mensaje.

### Dos tipos de sobrecarga

- **Basada en ámbito:** Métodos con diferentes ámbitos de definición, independientemente de sus signaturas de tipo. Permitido en todos los lenguajes OO.

  Un mismo método puede ser utilizado en dos o más clases.

  Por ejemplo. Sobrecarga de operadores como funciones miembro.

- **Basada en signatura:** Métodos con diferentes signaturas de tipo en el propio ámbito de definición. No permitido en todos los lenguajes OO.

  En este caso por ejemplo tomamos cualquier conjunto de funciones no miembro en el ámbito de definición global que comparten nombre.

Dos o más métodos en la propia clase pueden tener el mismo nombre con la condición que tengan distinta signatura de tipos.

Diferentes ámbitos conllevan que el mismo nombre de método pueda presentarse en ellos sin ambigüedad ni pérdida de precisión.

La sobrecarga por ámbito no precisa que las funciones relacionadas con un nombre sobrecargado tengan similitud semántica, ni la misma signatura de tipo.

### Ejemplos

¿Son Profesor y Alumno de ámbitos diferentes?

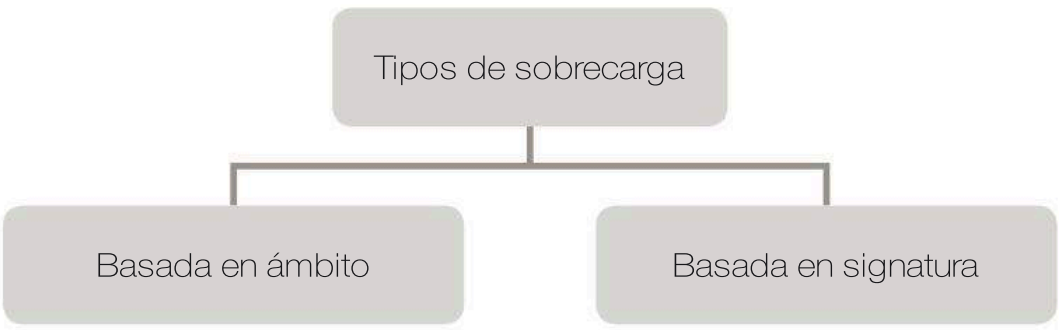
¿Y Profesor y Persona ?

Diversos métodos en el propio ámbito podrían compartir el mismo nombre siempre que tengan distinto número orden y en lenguajes con tipado estático, el tipo de los argumentos que precisen.

Por ello esta forma ocurre en diferentes lenguajes de programación como en algunos como ADA y en lenguajes Orientados a Objetos como C, C++, Java, Delphi, Pascal, etc…

C++ soporta esta sobrecarga de manera contenida siempre que la elección del método necesario por el usuario pueda establecerse de manera no ambigua en tiempo de compilación.

Esto con lleva que la signatura no puede diferenciarse sólo por el tipo de retorno.



## 5.4. Polimorfismo en tiempo de ejecución (ligadura dinámica)

### El polimorfismo

El polimorfismo es una concepción de la Programación Orientada a Objetos que nos posibilita programar de manera general, en lugar de hacerlo de forma específica. Habitualmente nos interesa para programar objetos con características comunes y que todos estos compartan la misma superclase en una jerarquía de clases, como si todas fueran objetos de la superclase. Esta característica facilita en gran medida la programación.

### Existen varias formas de polimorfismo

– Cuando llamamos el mismo nombre de método relativo a instancias de distinta clase.

– Cuando establecemos múltiples constructores.

– Cuando vía subtipo determinamos una instancia de una subclase con una referencia básica a la clase base.

Para situar en la teoría haremos como práctica un ejemplo bastante sencillo:

```
public class Coche {
private String dueno;
private String matricula;
private double cuentaKilometros ;

public void vender (String elDueno) {
dueno = elDueno;
}
```

```
public void matricular(String laMatricula) {

matricula = laMatricula;
}
public void recorrer(double kms ) {
cuentaKilometros = cuentaKilometros + kms;
}
public void printInfo() {
String ficha =
“Dueno: “+dueno+”;”+
“Matricula: “+matricula+”;”+
“Kms recorridos: “+cuentaKilometros+”;” ;
System.out.println(ficha) ;
}
}
```

```
public class CocheConGPS extends Coche {

private double latitud = 0;

private double longitud = 0;

public void cambiarCoordenadas (double alfaLatitud, double
alfaLongitud) {

latitud = latitud+alfaLatitud;

longitud = longitud+alfaLongitud;

}
```

```
public double latitud() {

return latitud;
}
public double longitud() {
return longitud;
}
public void printInfoPosicion() {
String posicion = “Latitud: “ + latitud + “; Longitud: “ + longitud;
System.out.println(posicion);
}
}
```

```
public class Taxi extends Coche {
/*  hereda de Coche  */
public void recorrer(double kms) {
System.out.println(“Taxi: comienza carrera” );
printInfo();
/ * metodo “recorrer” hereda de Coche */
super.recorrer(kms);
System.out.println(“Taxi: finaliza de carrera” );
}
}
```

```
/*  Ejemplo de asignar a la misma variable objetos de distintas clases. */

public class Ejemplo {
public static void main (String[ ] args ) {
Coche coche = null;
/ * A coche se le pueden asignar coches de diferentes tipos. */
coche = new Coche ();
coche = new CocheConGPS();
coche = new Taxi();
/*  metodos en Coche  */
coche.vender("€€€€€€€€€" );
coche.matricular("00000-AAAA" );
coche.printInfo();
}
}
```

Conocemos dos formas de ligadura: estática y dinámica.

- **Ligadura estática:**
  - Radica en efectuar el proceso de ligadura en tiempo de compilación, según el tipo del objeto al que se manda el mensaje.
  - La utiliza Java con los métodos de clase, y los métodos de instancia que son privados o final ya que estos finales no pueden ser sobrescritos.

– Ligadura dinámica:

  La ligadura dinámica consiste de ligar o relacionar la llamada a un método con el cuerpo del método que se establece finalmente.

  · Consiste en realizar el proceso de ligadura en tiempo de ejecución siendo la forma dinámica del objeto la que determina la versión del método a ejecutar.

  · Se utiliza en todos los métodos de instancia de Java que no son privados ni final.

### Funcionamiento de la ligadura dinámica

Resolver conflictos entre Superclases y Subclases:

– Cuando encontramos un conflicto entre un método de una superclase y un método de la subclase, el comportamiento adecuado es que el método de la subclase sobrescriba al de la superclase.

– Si estamos invocando a un método de la subclase desde una variable que ha sido declarada del tipo de la superclase.

### Ligadura dinámica:

– Quiere decir que la forma dinámica del objeto determina la versión de la operación que se aplicará.

– Esta posibilidad de las operaciones para adaptarse automáticamente a los objetos a los cuales se destinan es una de las propiedades más importantes de la orientación a objetos.

### Características de la ligadura dinámica

– Puede cambiar de un lenguaje a otro, pero básicamente presentan unas características comunes. Los métodos que necesitan ligadura dinámica.

– Deben mostrar ligadura dinámica solo aquellos que pueden ser redefinidos.

– Por ejemplo, en Java, los métodos de clase y los métodos de instancia privados y/o finales no presentan ligadura dinámica.

– En Java, si no se especifica nada se concebirá que el método puede ser redefinido y por tanto debe presentar ligadura dinámica.

## 5.5. Objetos polimórficos

Recordando el concepto Polimorfismo se describe como la posibilidad para presentarse en varias formas.

Polimorfismo en programas Java significa:

- La habilidad de una variable referencia para cambiar su comportamiento como arreglo a la instancia del objeto que contiene.

- Esto posibilita que diversos objetos de diferentes subclases sean tratados como objetos de una superclase única, mientras que automáticamente se escoge los métodos apropiados a emplear a un objeto en particular basado en la subclase a la que corresponde.

- Una variable polimórfica es la que se puede referenciar más de un tipo de objeto Puede mantener valores de distintos tipos en distintos momentos de ejecución del programa.

- En un lenguaje con debilidad en el tipado todas las variables son eventualmente polimórficas. En un lenguaje fuertemente tipado la variable polimórfica es la plasmación del principio de sustitución.

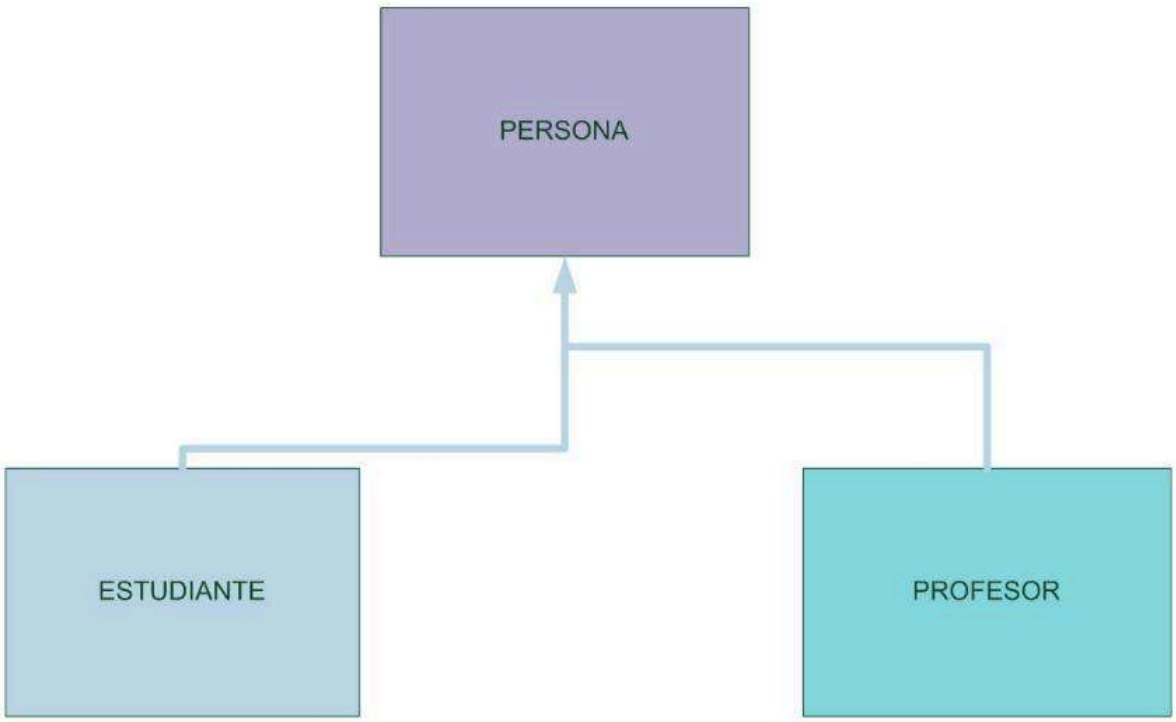
- Por ejemplo si disponemos de una clase base figura, el polimorfismo permite al programador definir diferentes métodos área para cualquier número de clases derivadas como circunferencias, rectángulos, y triángulos.

- El método área de circunferencia, rectángulo y triangulo se aplica de manera distinta.

  No importa qué propiedades y características tiene un objeto, aplicándole el método área devolverá el resultado correcto.

Dada la clase padre Persona y la clase hija Alumno se añade otra subclase Persona que es Profesor.

La gráfica de la jerarquía de clases es:




*Ejemplo de Diagrama de Polimorfismo*

Un ejemplo claro de Polimorfismo, dada la clase Persona y la clase hija Estudiante, se añade otra subclase Persona que es Profesor.

Se puede crear una referencia que es del tipo de la superclase, Persona, hacia un objeto de su subclase Alumno.

```
public static main( String[] args ) {
Alumno alumnoObject = new Alumno();
Profesor profesorObject = new Profesor();
Persona refere = empleadoObject; //referencia Persona
// al objeto Alumno
// Llamada de getName() de la instancia de objeto Alumno
String nombre = refere.getName();
}
```

Partimos de la premisa que hay un método getName en la superclase Persona y que este método es sobrescrito en las subclases Alumno y Profesor:

```
public class Alumno { public String getName() {
System.out.println("Nombre alumno:" + nombre);
return nombre;
}
}
public class Profesor { public String getName() {
System.out.println("Nombre profesor:" + nombre);
return nombre;
}
}
```

Volviendo al método main, cuando se invoca el método getName de la referencia Persona ref, el método getName del objeto Alumno será llamado.

Si se asigna ref a un objeto Profesor, el método getName de Profesor será llamado.

```
public static main (String[] args) {
Alumno alumnoObject = new Alumno();
Profesor profesorObject = new profesor();
//refere apunta a un objeto Alumno
Persona refere = alumnoObject;
//metodo getName() de la clase Alumno es invocado
```

```
String temp= refere.getName();
System.out.println(temp);
//refere apunta ahora a un objeto profesor
refere = profesorObject;
//metodo getName() de la clase Profesor es invocado
String temp = refere.getName();
System.out.println(temp);
}
```

## 5.6. Comprobación estática y dinámica de tipos

### Clasificación

Se puede catalogar el polimorfismo en dos grandes clases:

- **Polimorfismo Dinámico**, es aquél en el que el código no existe ningún tipo de detalle referente al tipo de datos sobre el que se se ocupa. Así, puede ser empleado a todo tipo de datos compatible.

- **Polimorfismo Estático,** es aquél en el que los tipos a los que se enfoca el polimorfismo serán expuestos y definidos uno por uno para que puedan ser empleados.

El polimorfismo dinámico combinado con la herencia es lo que en ocasiones se conoce como programación genérica.

Asimismo se clasifica en herencia por redefinición de métodos abstractos y por método sobrecargado. El segundo hace referencia al mismo método con diferentes parámetros.

Otro tipo de ordenamiento congrega los polimorfismo en dos tipos: “Ad–Hoc” que incluye a su vez sobrecarga de operadores y coerción, “Universal” inclusión o controlado por la herencia, paramétrico o genericidad.

## Polimorfismo. Métodos virtuales

El Polimorfismo muestra que una variable pasada o esperada puede acoger múltiples formas. Cuando se habla de polimorfismo en programación orientada a objetos se suelen entender dos cosas:

– La primera se refiere a que se puede trabajar con un objeto de una clase sin interesar de qué clase se trata. Es decir, se trabajará igual sea cual sea la clase a la que pertenece el objeto. Esto se consigue mediante jerarquías de clases y clases abstractas.

– La segunda suele referirse a la posibilidad de expresar métodos con el mismo nombre que pueden tener diferentes argumentos dentro de una misma clase.




La capacidad de un programa de trabajar con más de un tipo de objeto se conoce con el nombre de polimorfismo.

Incluso ahora la herencia se ha utilizado solamente para heredar los miembros de una clase base, pero también se halla la posibilidad de que un método de una clase derivada se llame como método de la clase base pero posea un funcionamiento diferente.

Otro ejemplo que ilustra el polimorfismo es cuando se trata de pasar una referencia a métodos como Parámetros.

Supongamos que poseemos un método estático printInformacion que toma una referencia Persona como parámetro.

```
public static printInformacion ( Persona p ){
// Se invocara el metodo getName() de la
// instancia actual del objeto que es pasado
p.getName();
}
```

Si se pasa una referencia de tipo Empleado y tipo Alumno al método printInformacion en tanto que es una subclase de la clase Persona.

```
public static main( String[] args ){
Alumno alumnoObject = new Alumno();
profesor profesorObject = new profesor();
printInformacion( alumnoObject );
printInformacion( profesorObject );
}
```

### Simplicidad

– Si se necesita picar código que se relaciona con una familia de subtipos, el código puede ignorar los detalles concretos de tipo y sólo interactuar con el tipo base de la familia.

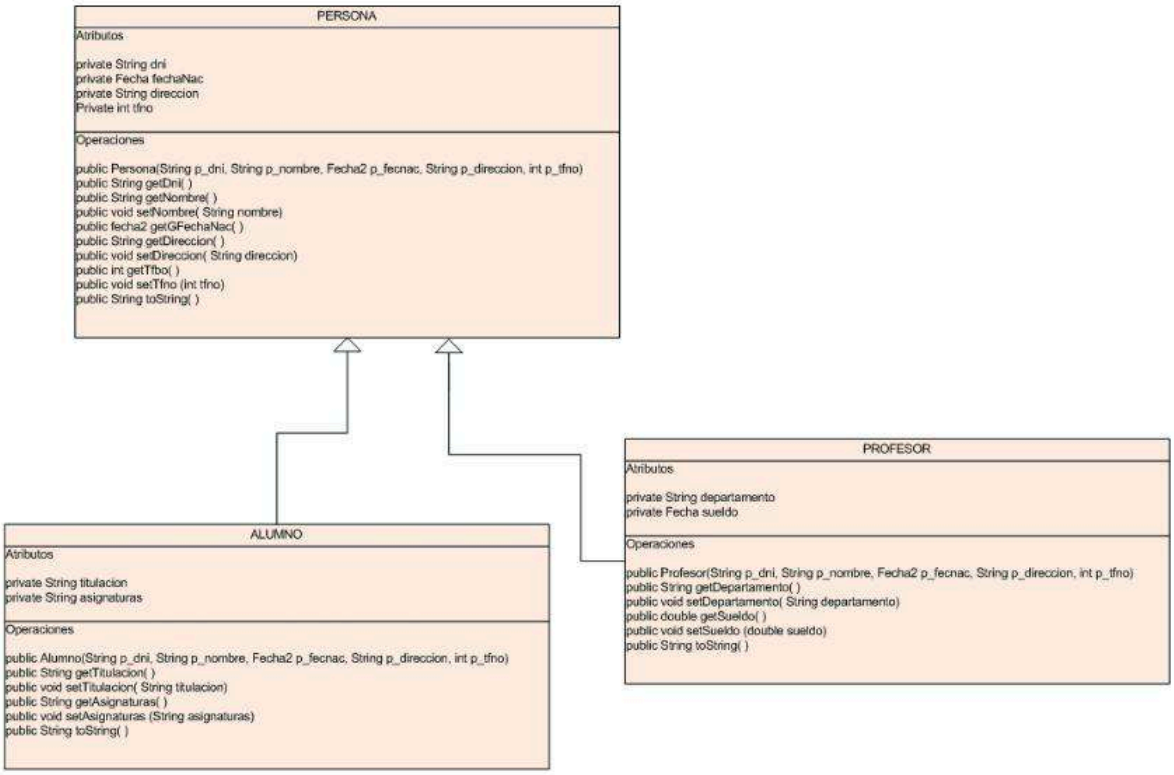
– Aún cuando el código piense que está usando un objeto de la clase base, la clase del objeto podría ser la clase base o cualquiera de sus subclases.

## Extensibilidad

Se pueden añadir subclases posteriormente a la familia de tipos, y los objetos de estas nuevas subclases podría trabajar con el código existente.

Con este ejercicio se pretende mostrar diferentes aspectos de Polimorfismo tocados en esta unidad y pode reflejarse en código fuente.

A continuación realizaremos un programa capaz de gestionar un censo de un Instituto de Formación Profesional de alumnos y profesores.




*Ejemplo de Diagrama de Clases para ejercicio de Polimorfismo*

Diagrama de Clases en un ejemplo de Polimorfismo. Dada la clase padre Persona y la clase hija Alumno, se añade otra subclase Persona denominada Profesor. Su gráfica de jeraquia de Clases.

El código de cada una de las clases y una aplicación capaz de manejarlas es el siguiente:

## Clase Persona

```
package censoinstitu;
public class Persona {
private String numdni;
private String nombre;
private Fecha fechaNacim;
private String direccion;
private int telefono;
//constructor
public Persona(String p_numdni, String p_nombre, Fecha p_
fechaNacim,
String p_direccion, int p_telefono) {
numdni = p_numdni;
nombre = p_nombre;
fechaNac = p_fechaNacim;
direccion = p_direccion;
telefono = p_telefono;
}
//métodos
public String getNumdni() {
return this.numdni;
}
public String getNombre() {
return this.nombre;
}
public void setNombre(String nombre) {
```

```
this.nombre = nombre;
}
public Fecha getFechaNac() {

return this.fechaNac;
}
public void setFechaNac(Fecha fechaNac) {
this.fechaNac = fechaNac;
}
public String getDireccion() {
return this.direccion;
}
public void setDireccion(String direccion) {
this.direccion = direccion;
}
public int getTelefono() {
return this.telefono;
}
public void setTelefono(int telefono) {
this.telefono = telefono;
}
public String toString() {
return numdni + " " + nombre;
}
}
```

Esta clase hace uso con una asociación de otra llamada Fecha, esta clase se encarga de manejar fechas. El código de la clase se muestra a continuación:

Clase Fecha:

```
//Clase para manejar fechas (DD/MM/AAAA)
package censoinstitu;
import java.util.Calendar;
public class Fecha {
//atributos
private int año;
private int mes;
private int dia;
//constructores
public Fecha(int dia, int mes, int año) {
this.dia = dia;
this.mes = mes;
this.año = año;
}
public Fecha(String cadenaFecha) throws FechaMal {
String[] st = cadenaFecha.split("/");
if (st.length < 3) {
throw new FechaMal();
}
String dd = st[0];
String mm = st[1];
String aa = st[2];
this.dia = Integer.parseInt(dd);
this.mes = Integer.parseInt(mm);
this.año = Integer.parseInt(aa);
}
public Fecha() {
Calendar hoy = Calendar.getInstance();
año = hoy.get(Calendar.YEAR);
mes = hoy.get(Calendar.MONTH) + 1;
dia = hoy.get(Calendar.DAY_OF_MONTH);
```

```
}
//métodos
public String toString() {
return dia + “/” + mes + “/” + año;
}
public int getAño() {
return this.año;
}
public void setAño(int año) {
this.año = año;
}
public int getDia() {
return this.dia;
}
public void setDia(int dia) {
this.dia = dia;
}
public int getMes() {
return this.mes;
}
public void setMes(int mes) {
this.mes = mes;
}
}
class FechaMal extends Exception {
public FechaMal() {
super(“FECHA INCORRECTA.”);
}
}
```

## Clase Alumno

```
package censoinstitu;
public class Alumno extends Persona {
private String titulacion;
private String asignaturas;
//constructor
public Alumno(String p_numdni, String p_nombre, Fecha p_
fechaNacim,
String p_direccion, int p_telefono, String titulacion, String asignaturas) {
super(p_numdni, p_nombre, p_fechaNacim, p_direccion, p_telefono);
this.titulacion = titulacion;
this.asignaturas = asignaturas;
}
//métodos
public String getTitulacion() {
return this.titulacion;
}
public void setTitulacion(String titulacion) {
this.titulacion = titulacion;
}
public String getAsignaturas() {
return this.asignaturas;
}
public void setAsignaturas(String asignaturas) {
this.asignaturas = asignaturas;
}
public String toString() {
```

```
return super.toString()+" # Alumno # - Titulación: "+ this.titulacion;
}
}
- Clase Profesor:
package censoinstitu;
public class Profesor extends Persona {
private String departamento;
private double sueldo;
//constructor
public Profesor(String p_numdni, String p_nombre, Fecha p_
fechaNacim,
String p_direccion, int p_telefono, String departamento, double sueldo) {
super(p_numdni, p_nombre, p_fechaNacim, p_direccion, p_telefono);
this.departamento = departamento;
this.sueldo = sueldo;
}
//métodos
public String getDepartamento() {
return this.departamento;
}
public void setDepartamento(String departamento) {
this.departamento = departamento;
}
public double getSueldo() {
return this.sueldo;
}
public void setSueldo(double sueldo) {
this.sueldo = sueldo;
}
public String toString() {
return super.toString()+" # Profesor # - Departamento: "
+ this.departamento;
}
}
```

## Clase Main

En este caso se muestra una clase llamada Main encargada de efectuar las operaciones elementales para gestionar un censo universitario. Estos procedimientos son las de alta, bajas, modificaciones y consulta de los datos de las personas censadas.

```
/* Aplicación CensoInstituto: Permite introducir, borrar y buscar
personas */

package censoinstitu;

import java.io.BufferedReader;

import java.io.IOException;

import java.io.InputStreamReader;

import java.util.HashMap;

public class Main {

/** Creates a new instance of Main */

public Main() {

}

private static String res, res2, numdni, nombre, direccion, titulacion,

asignaturas, departamento;

//Tabla hash

private static HashMap<String, Persona> personas = new
HashMap<String, Persona>();

private static BufferedReader entrada =

new BufferedReader(new InputStreamReader(System.in));

private static int telefono, op;

private static double sueldo;
```

```
private static Fecha fechaNacim;
private static Persona objper;
private static Alumno alumno;
private static Profesor profe;
private static String clase;
public static void main(String[] args) {
// TODO code application logic here
while (true) {
try {
do {
System.out.println("\n\n--- Operaciones del Censo Universitario ---");
System.out.println(" 1 - Alta. ");
System.out.println(" 2 - Baja. ");
System.out.println(" 3 - Consulta. ");
System.out.println(" 4 - Modificación. ");
System.out.println(" 5 - Salir. ");
System.out.println(" Seleccione la operación: (1,2,3,4,5): ");
res = entrada.readLine();
} while (!res.equals("1") && !res.equals("2") && !res.equals("3")
&& !res.equals("4") && !res.equals("5"));
// alta
if (res.equals("1")) {
alta();
} // baja
else if (res.equals("2")) {
baja();
} // consulta
```

```
else if (res.equals("3")) {
consulta();
} // modificación
else if (res.equals("4")) {
modifica();
} //salir
else if (res.equals("5")) {
System.exit(0);
}
} catch (NumberFormatException nfe) {
System.out.println("\nError de formato numérico: " + nfe.toString());
} catch (FechaMal fm) {
System.out.println("\nError en formato de fecha.");
} catch (IOException ioe) {
System.out.println("\nError de entrada/salida: " + ioe.toString());
} catch (Exception e) {
// Captura cualquier otra excepción
System.out.println(e.toString());
}
}
}
// alta
private static void alta() throws IOException, FechaMal {
System.out.println("\n -- Alta -- ");
System.out.println("Introduzca el DNI: ");
```

```
numdni = entrada.readLine();
System.out.println("Introduzca nombre: ");
nombre = entrada.readLine();
System.out.println("Introduzca fecha de nacimiento (DD/MM/AAAA): ");
fechaNacim = new Fecha(entrada.readLine());
System.out.println("Introduzca dirección: ");
direccion = entrada.readLine();
System.out.println("Introduzca teléfono: ");
telefono = Integer.parseInt(entrada.readLine());
do {
System.out.println("Introduzca categoría (1-alumno, 2-profesor): ");
res = entrada.readLine();
} while (!res.equals("1") && !res.equals("2"));
if (res.equals("1")) {
System.out.println("Introduzca titulación: ");
titulacion = entrada.readLine();
System.out.println("Introduzca asignaturas: ");
asignaturas = entrada.readLine();
//instanciamos un alumno
alumno = new Alumno(numdni, nombre, fechaNacim, direccion,
telefono,
titulacion, asignaturas);
//introducimos en la tabla hash.
personas.put(numdni, alumno);
System.out.println("\nAlumno dado de alta: " + alumno.toString());
} else {
System.out.println("Introduzca departamento: ");
departamento = entrada.readLine();
```

```
System.out.println("Introduzca sueldo: ");
sueldo = Double.parseDouble(entrada.readLine());
//instanciamos un profesor
profe = new Profesor(numdni, nombre, fechaNacim, direccion, telefono,
departamento, sueldo);
//introducimos en la tabla hash.
personas.put(numdni, profe);
System.out.println("\nProfesor dado de alta: " + profe.toString());
}
}
// baja
private static void baja() throws IOException {
System.out.println("\n -- Baja -- ");
System.out.println("Introduzca el DNI de la persona a borrar: ");
numdni = entrada.readLine();
//obtenemos la persona de la tabla hash.
objper = personas.get(numdni);
if (objper != null) {
personas.remove(numdni);
System.out.println("Persona borrada: " + objper.toString());
} else {
System.out.println(" Error: - Persona no encontrada - ");
}
}
```

```
// consulta
private static void consulta() throws IOException {
System.out.println("\n -- Consulta -- ");
System.out.println("Introduzca el DNI de la persona: ");
numdni = entrada.readLine();
//obtenemos la persona de la tabla hash.
objper = personas.get(numdni);
if (objper != null) {
System.out.println("- Nombre: " + objper.getNombre());
System.out.println("- Fecha de nacimiento: " + objper.getFechaNac().
toString());
System.out.println("- Dirección: " + objper.getDireccion());
System.out.println("- Teléfono: " + objper.getTelefono());
//sacamos la categoría de la persona
clase = objper.getClass().getName();
if (clase.equals("censoinstitu.Alumno")) {
alumno = (censoinstitu.Alumno) objper;
System.out.println("- Titulación: " + alumno.getTitulacion());
System.out.println("- Asignaturas: " + alumno.getAsignaturas());
} else {
profe = (censoinstitu.Profesor) objper;
System.out.println("- Departamento: " + profe.getDepartamento());
System.out.println("- Sueldo: " + profe.getSueldo());
}
} else {
System.out.println(" Error: - Persona no encontrada - ");
}
}
```

```
// modifica
private static void modifica() throws IOException, FechaMal {
System.out.println("\n -- Modificación -- ");
System.out.println("Introduzca el DNI de la persona: ");
numdni = entrada.readLine();
//obtenemos la persona de la tabla hash.
objper = personas.get(numdni);
if (objper != null) {
System.out.println("Introduzca el dato a modificar (1-Nombre, 2-FecNac,
3-Dir,
4-Telefono, 5-Otro): ");
op = Integer.parseInt(entrada.readLine());
switch (op) {
case 1:
System.out.println("Introduzca nuevo nombre: ");
nombre = entrada.readLine();
objper.setNombre(nombre);
break;
case 2:
System.out.println("Introduzca nueva fecha de nacimiento: ");
fechaNacim = new Fecha(entrada.readLine());
objper.setFechaNac(fechaNacim);
break;
case 3:
System.out.println("Introduzca nueva dirección: ");
direccion = entrada.readLine();
objper.setDireccion(direccion);
break;
```

```
case 4:
System.out.println("Introduzca nuevo teléfono: ");
telefono = Integer.parseInt(entrada.readLine());
objper.setTelefono(telefono);
break;
case 5: //sacamos la categoría de la persona
clase = objper.getClass().getName();
if (clase.equals("censoinstitu.Alumno")) {
alumno = (censoinstitu.Alumno) objper;
System.out.println("Introduzca el dato a modificar (1-Titulación,
2-Asignaturas): ");
res2 = entrada.readLine();
if (res2.equals("1")) {
System.out.println("Introduzca nueva titulación: ");
titulacion = entrada.readLine();
alumno.setTitulacion(titulacion);
} else if (res2.equals("2")) {
System.out.println("Introduzca nuevas asignaturas: ");
asignaturas = entrada.readLine();
alumno.setAsignaturas(asignaturas);
}
} else {
profe = (censoinstitu.Profesor) objper;
System.out.println("Introduzca el dato a modificar (1-Dpto, 2-Sueldo): ");
res2 = entrada.readLine();
if (res2.equals("1")) {
System.out.println("Introduzca nuevo departamento: ");
departamento = entrada.readLine();
```

```
profe.setDepartamento(departamento);
} else if (res2.equals("2")) {
System.out.println("Introduzca nuevo sueldo: ");
sueldo = Double.parseDouble(entrada.readLine());
profe.setSueldo(sueldo);
}
}
break;
default:
System.out.println("Opción incorrecta");
}
System.out.println("Persona Editada: " + objper.toString());
} else {
System.out.println(" Error: - Persona no encontrada - ");
}
}
}
```

# UD5
# Lo más importante

- El polimorfismo nos permite programar de manera general en lugar de programar de manera específica.
- Hay cuatro técnicas, cada una de las cuales permite una forma distinta de reutilización de software, que facilita a su vez el desarrollo rápido, Sobrecarga, Sobreescritura, Variables polimórficas y Genericidad.
- El polimorfismo es el modo en que los lenguajes OO implementan el concepto de polisemia del mundo real: Un único nombre para muchos significados, según el contexto.
- Un objeto polimórfico es una entidad que puede contener valores de diferentes tipos durante la ejecución del programa.
- Una forma de conseguir objetos polimórficos es mediante el uso de referencias a la superclase.
- El concepto de polimorfismo se puede aplicar tanto a funciones como a tipos de datos. Así nacen los conceptos de funciones polimórficas y tipos polimórficos.
- Sobrecarga (Overloading, Polimorfismo ad–hoc): un solo nombre de método y muchas implementaciones distintas.
- Sobreescritura (Overriding, Polimorfismo de inclusión): Tipo especial de sobrecarga que ocurre dentro de relaciones de herencia.
- Un mismo método puede ser utilizado en dos o más clases.
- La ligadura dinámica se encarga de ligar o relacionar la llamada a un método con el cuerpo del método que se ejecuta finalmente.

# UD5
## Autoevaluación

1. Una peculiaridad del Polimorfismo en Java es?
    a. La reutilización de software.
    b. Hace el software más seguro.
    c. Ambas son correctas.
    d. Ninguna es correcta.

2. Se obtienen objetos polimórficos mediante:
    a. Según tipado del método.
    b. El uso de referencias a la superclase.
    c. Como objeto de la misma clase.
    d. Ninguna de las anteriores.

3. Sobrecarga (Overloading, Polimorfismo ad–hoc):
    a. Un solo nombre de método y muchas implementaciones distintas.
    b. Tipo especial de sobrecarga que ocurre dentro de relaciones de herencia.
    c. Muchos métodos muchas implementaciones.
    d. Es una referencia en realidad a un valor de un tipo distinto.

4. Sobreescritura (Overriding, Polimorfismo de inclusión):
    a. Un solo nombre de método y muchas implementaciones distintas.
    b. Los métodos se encuentran en dos clases distintas relacionadas mediante herencia.
    c. Es una referencia en realidad a un valor de un tipo distinto.
    d. Forma de crear herramientas de propósito general (clases, métodos).

5. Variables polimórficas (Polimorfismo de asignación):
    a. Variable que se declara como de un tipo pero que referencia en realidad un valor de un tipo distinto.
    b. Variables que se declaran y cambian de estado y valor en el procedimiento.
    c. Tipo especial de sobrecarga que ocurre dentro de relaciones de herencia.
    d. Forma de crear herramientas de propósito general (clases, métodos).

6. Genericidad (plantillas o templates):
    a. Forma de crear herramientas de propósito general (clases, métodos).
    b. Es una referencia en realidad a un valor de un tipo distinto.
    c. Tipo especial de sobrecarga que ocurre dentro de relaciones de herencia.
    d. Los métodos se encuentran en dos clases distintas relacionadas mediante herencia.

7. Polimorfismo en tiempo de compilación:
    a. Basado en ligadura estática y dinámica.
    b. Basado en ámbito y signatura.
    c. Basado en Ligadura dinámica.
    d. Ninguna de las anteriores.

8. Polimorfismo en tiempo de ejecución:
    a. Basado en ámbito, signatura y ligadura estática.
    b. Basado en ámbito y signatura.
    c. Basado en Ligadura dinámica y estática.
    d. Ninguna de las anteriores.

9. Polimorfismo dinámico es:
    a. Tipos declarados uno por uno antes de poder ser utilizados.
    b. El código no incluye ningún tipo de especificación sobre el tipo de datos.
    c. Los tipos a los que se aplica el polimorfismo deben ser explicitados.
    d. Ninguna de las anteriores.

10. Polimorfismo estático es:
    a. Puede ser utilizado a todo tipo de datos compatible.
    b. El código no incluye ningún tipo de especificación sobre el tipo de datos.
    c. Aquél en el que los tipos deben ser explicitados y declarados uno por uno antes de poder ser utilizados.
    d. Ninguna de las anteriores.

Área: informática y comunicaciones

# UD6

## Técnicas de programación estructurada

6.1. Identificación de elementos básicos: constantes, variables, operadores y expresiones

6.2. Análisis de estructuras de control: secuencial, condicional y de repetición

6.3. Distinción entre funciones y procedimientos

- 6.3.1. Interfaz
- 6.3.2. Paso de parámetros: por valor y por referencia
- 6.3.3. Parámetros actuales y formales
- 6.3.4. Funciones: valor de retorno
- 6.3.5. Procedimientos
- 6.3.6. Ámbito de las variables
- 6.3.7. Almacenamiento de las variables

6.4. Demostración de llamadas a funciones y procedimientos

6.5. Empleo de llamadas a funciones y procedimientos incluidos en las clases

- 6.5.1. Llamadas calificadas y no calificadas (instancia actual)
- 6.5.2. Paso de parámetros
- 6.5.3. Los atributos de la clase

## 6.1. Identificación de elementos básicos: constantes, variables, operadores y expresiones

No hay nada mejor en programación que tener un buen hábito de trabajo y de escribir código de manera efectiva, limpia. Existen muchas técnicas de programación estructurada y sintaxis, que permiten una buena legibilidad por parte de distintas personas que en algún momento necesiten acceder al código fuente.

### La calidad del código

La especificación de la calidad de los desarrollos y la evaluación de la misma son factores clave para lograr un nivel de calidad de código correcto.

Se recomienda que cada necesidad de calidad se valore a través de herramientas de medida que estén validadas o ampliamente aceptadas y se apliquen reglas de codificación que garanticen las características de mantenimiento y gran facilidad para aplicar modificaciones en el código, eficiencia y capacidad para originar un buen uso de los recursos, capacidad para rodar en diferentes plataformas y fiabilidad y capacidad para ejecutarse correctamente como programas y aplicativos.

### ¿Porque medir?

Las herramientas de métrica y medidas permiten detectar tendencias y anticipar problemas y, por lo tanto, posibilitan implantar un mayor control y seguimiento de las aplicaciones que conlleve una mejora de la calidad. De la misma manera, permiten diferenciar entre situaciones reales de riesgo de las que no lo son, lo cual es vital para una correcta y acertada toma de decisiones.

## Principios de calidad del código

A continuación se concretan una serie de condiciones generales que se sugiere tener presentes para lograr alcanzar un nivel de calidad óptimo en el código de las aplicaciones a desarrollar:

– Medición la particularidad de las aplicaciones con métricas notoriamente delimitadas para conservar un control constante de los desarrollos.

– Instaurar reglas de codificación y emplearlas en todos las implementaciones aplicadas dentro de la organización para asegurar el mantenimiento, eficiencia, portabilidad y fiabilidad de las aplicaciones.

– Envolver tanto a desarrolladores como a gestores para lograr el éxito, en la consecución de los objetivos de calidad de código fuente en los programas y aplicativos.

## Volumen

El volumen de las aplicaciones hace mención al tamaño de las mismas medido en líneas de código, lo que afecta directamente a su mantenibilidad. Éste se puede medir en base a dos métricas:

– Volumen total

– Volumen por unidad (por método)

Aparte del tamaño del sistema completo, es importante estudiar el tamaño de los componentes individuales que forman el sistema. Un gran volumen del tamaño del sistema no implica necesariamente complicaciones en el diseño, simplemente puede poseer mucha funcionalidad implementada. Sin embargo, detectar unidades de código excesivamente grandes sí puede indicar un mal diseño, por ejemplo, debido a unidades con una cohesión muy baja.

Se puede calcular como el número completo de líneas de código que constituyen los métodos de un determinado tamaño y el porcentaje que forman del volumen total. Los tamaños a considerar para realizar el cálculo son:

– Muy grande: Número de líneas de código por unidad: > 100

– Grande: Número de líneas de código por unidad: (50–100]

– Medio: Número de líneas de código por unidad: (10–50]

## Complejidad

La complejidad es el número de posibles vías lógicas que tiene cada componente, es decir, las sentencias selectivas (if, case), sentencias iterativas (while, for), lanzamiento y tratamiento de excepciones (throw, catch), etc. A mayor complejidad, un sistema es más difícil de modificar y de probar.

De acuerdo a la complejidad, esta se establece en base a las estas métricas:

– Complejidad total

  Tiene que ver con la complejidad del sistema total. Es la media de la complejidad de todos los componentes de la aplicación.

– Complejidad por unidad (método)

  Esta medida ayuda a averiguar aquellos métodos que están incrementando la complicación del sistema en mayor medida.

  Se calcula obteniendo los valores de complejidad ciclomática para cada uno de los métodos, se ordenan de mayor a menor nivel de riesgo y se calcula el porcentaje de líneas de código para cada uno de los niveles definidos en función de los siguientes rangos:

  · Riesgo muy alto:Complejidad por unidad: >50

  · Riesgo alto:Complejidad por unidad: (20–50]

  · Riesgo moderado:Complejidad por unidad: (10–20]

## Reglas de codificación

Para ayudar al objetivo de lograr implementar código eficiente, mantenible, portable y fiable se recomienda seguir una serie de criterios de codificación a emplear en todos los desarrollos llevados a cabo. Dependiendo del argumento de cada una de las aplicaciones se puede hacer más ahínco en unas que en otras aunque se recomienda contar con un conjunto de reglas de codificación base a tener en cuenta en todos los desarrollos llevados a cabo dentro de una organización.

A continuación se definen algunas de las pautas de codificación más relevantes aplicadas al lenguaje Java.

## Eficiencia

La eficacia de una aplicativo es la posibilidad que tiene dicha aplicación para mantener un buen uso de los recursos que maneja. Ésta comprende los siguientes puntos:

– Comportamiento temporal: Como la posibilidad del producto software para suministrar tiempos de respuesta, lapsos de proceso y potencia apropiados, bajo circunstancias determinadas.

– Utilización de recursos: La posibilidad del software para usar las conjuntos y variantes de recursos adecuados cuando el software lleva a cabo su función bajo circunstancias establecidas.

El código fuente y el estilo que se sirven en su desarrollo intervienen directamente en la eficiencia de la aplicación, puesto que pueden afectar a la velocidad de ejecución de la aplicación, así como al uso de memoria de la misma.

A continuación se detallan una serie de reglas o pautas cuyo cumplimiento ayuda a mejorar la eficiencia de la aplicación:

– Métodos que enlazan strings utilizando '+' dentro de un bucle. Dicha regla trata sobre la presencia de construcciones de Strings mediante el uso de concatenación ('+') dentro de un bucle. En cada iteración, el String es transformado a StringBuffer/StringBuilder, concatenado y convertido de nuevo a String. Este proceso agranda considerablemente el coste, disminuyendo el rendimiento. Por tanto, para optimar la eficiencia, se recomienda utilizar StringBuffer o StringBuilder. Por ejemplo, dado el siguiente método:

```
String s = "";
for (int i = 0; i < dato.length; ++i) {
s = s + dato[i];
}
```

Optimizado ahora bastante mejor:

```
StringBuffer buffer = new StringBuffer();
for (int i = 0; i < dato.length; ++i) {
buffer.append(dato[i]);
}
```

- Instanciación de Short. La invocación a new Short() provoca asignación de memoria. Es mejor y más viable sustituir esta llamada por Short.valueOf(). Sobre todo si hay varias asignaciones.

- Instanciación de Integer. Las llamadas a new Integer() provocan asignación de memoria. Se recomienda suplir estas llamadas por Integer.valueOf( ) también como el supuesto anterior.

- Instanciación de String. Se recomienda evadir la creación de instancias de String, que a menudo suele ser redundante, para reducir el consumo de memoria. Por ejemplo, dada la siguiente clase:

- Instanciación de objetos Boolean. Se debe evitar instanciar objetos de la clase Boolean. En vez de ello, se podría referenciar a Boolean.TRUE o Boolean.FALSE.

- Uso de indexOf(char). A raiz esta regla se observa que cuando se utiliza “indexOf()” para comprobar el índice de un carácter simple dentro de un String, el parámetro que se le pase a dicha función sea de tipo char. Por ejemplo, en este ejemplo la clase:

```
public class Demo {
void bar() {
String s = “hola que tal?”;
if (s.indexOf(“a”) {}
}
```

Se podría cambiar por:

```
public class Demo {
void bar() {
String s = “hola que tal?”;
if (s.indexOf(’a’) {}
}
```

## Escribir un buen código Java

Nosotros tenemos ya sintaxis Java acumulada para poder escribir pequeños programas Java básicos, lo que significa que la primera mitad de esta unidad estás preparado para adquirir estos hábitos. Esta sección presenta algunas de las mejores prácticas que le ayudarán a escribir códigos Java más limpios y más plausibles de ser mantenidos.

### Mantenga pequeñas las clases

La creación de clases es vital hacerlo correctamente. Luego ten presente que para generar pares de getters y setters incluso para la pequeña cantidad (de acuerdo a los estándares de una clase Java del mundo real) de atributos, la clase Person tiene 150 líneas de código. Esta es una clase pequeña. No es raro ver clases con 50 o 100 métodos y mil líneas de origen (o más). El punto de los métodos es mantener solo los que necesita. Si necesita varios métodos ayudantes que hagan esencialmente lo mismo pero que tomen parámetros diferentes (tales como el método printAudit()), esa es una buena elección. Solo asegurate de limitar la lista de métodos a lo que necesita, no más. Técnica divide y vencerás.




De manera genérica, las clases simbolizan alguna entidad conceptual en su aplicación y sus tamaños deberían reflejar solo la funcionalidad para hacer lo que sea que la entidad necesite hacer.Deberían permanecer muy centradas para hacer una pequeña cantidad de actividades y hacerlas bien.

### Nombrar a los métodos con cuidado

El mejor patronaje a la hora de codificar es cuando se trata de nombres y métodos, es el patrón de nombres de métodos revelador de intenciones. Este patrón es más fácil de entender con un simple ejemplo. ¿Cuál de los siguientes nombres de métodos es más fácil de descifrar a la vista?

```
a()
```

```
computeInterest()
```

La consulta debe ser clara, por algún motivo los programadores tienen una tendencia a darle a los métodos y variables como costumbre nombres pequeños, abreviados. Sin duda, un nombre enormemente largo no será práctico, pero un nombre que informa lo que un método hace, no necesita ser largo. Seis meses después de que escribes un grupo de códigos, tal vez no recuerde lo que quiso hacer con un método llamado a() pero es evidente que un método llamado computeInterest(), bueno, probablemente compute interés.

### Mantenga pequeños los métodos

Los métodos pequeños son tan deseables como las clases pequeñas y por motivos iguales. Un modismo que intento seguir es mantener el tamaño de un método en una página como lo veo en mi pantalla. Esto hace que mis clases de aplicaciones sean más plausibles de ser mantenidas.

Si un método crece más allá de una página, lo refactorizo. Refactorear significa cambiar el diseño de un código existente sin cambiar sus resultados. Netbeans y Eclipse tiene un maravilloso conjunto de herramientas de refactorización. Normalmente, un método largo contiene subgrupos de funcionalidad agrupados. Tome esta funcionalidad y muévela a otro método nombrándola como corresponde y pásale los parámetros como se necesite.

Reduzca y limite cada método a un solo trabajo. Hay un estudio que argumenta que un método que hace solo una cosa bien, normalmente no toma más de alrededor de 30 líneas de códigos.

## Use los comentarios

Es vital y necesario usar los comentarios. Las personas que siguen trabajando en ese código o incluso tú mismo varios meses más adelante lo agradecerán. No es cierto el dicho que Un código bien escrito es de autodocumentación, por lo tanto, ¿quién necesita comentarios? Le daré dos razones por lo que esto es falso:

– La mayoría de los códigos no están bien escritos.

– Tu código probablemente no está tan bien escrito como piensas.

– Así que hay que hacer siempre comentarios en el código.

En este punto se representan normas de codificación que se encarga manejar para mejorar la mantenibilidad de la aplicación.

– Bloques vacíos. Esta regla justifica la existencia de bloques de tipo if, while o finally vacíos, es decir, en los que no se ejecuta ninguna acción. En el momento de desarrollar, se deberían omitir este tipo de construcciones.

– Bloques if incondicionales. Por esta norma se verifica la existencia de bloques if que son continuamente verdaderos o siempre falsos. Se confía no utilizar este tipo de construcciones que obstaculizan la mantenibilidad del código.

– Bloques sin llaves. Se debería impedir el uso de bloques if, if..else o for sin llaves (‘{‘). El uso de llaves ayuda a depurar el código y lo hace más comprensible, por eso se encomienda utilizarlo en todos los bloques existentes en la aplicación.

– Longitud de línea. Se confía que la longitud de las líneas de código no supere los 80 caracteres. El uso de líneas de código que superen esta longitud dificulta la mantenibilidad de la aplicación, por ello, es recomendable querer reducirla siempre que sea posible.

– Comentarios Javadoc de métodos y variables. Con esta regla se comprueba la presencia de comentarios Javadoc en métodos, constructores y variables. La documentación de una aplicación es fundamental para garantizar la mantenibilidad de la misma. Sin ella, se obstaculiza en gran medida la realización de cambios, modificaciones de funcionalidad, resolución de posibles fallos, etc. Por ello, se debería añadir comentarios a todos los métodos y variables.

- Variables cortas. Se recomienda obviar el uso de variables con un nombre muy corto, si dichos nombres no contribuyen a dar un significado al lector en función del contexto en el que se declaran. Por ejemplo, i, j que suelen utilizarse como índices, son muy cortos, pero sin embargo muy informativos. En caso de que estos nombres corto no aporten un significado al lector, deberían ser sustituidos por otros que sí lo hagan, ayudando así a la comprensión del código fuente y, por tanto, a la mantenibilidad del mismo.

- Convenio de nombres de variables. Esta regla instaura las siguientes convenciones:

- El nombre de las variables finales debe estar íntegramente en mayúsculas.

- Las variables no finales no deben contener guiones bajos.

- Usar estas formalidades con lo nombres de variables es muy necesario, puesto que posibilitan lograr un código homogéneo y más entendible para el lector a la hora de realizar cualquier modificación, detectar errores, etc.

- Modificar la visibilidad. Esta formalidad versa sobre la visibilidad de los componentes de las clases. Sólo los componentes static final deberán ser públicos, los demás deberían ser privados.

- Múltiples cadenas literales. En lugar de usar un mismo literal varias veces en el interior de un mismo fichero, se debe usar una constante.

### Siguiendo los pasos de Fowler

Un gran libro sobre estas materias es Refactoring: Improving the Design of Existing Code de Martin Fowler et al. Fowler y sus coautores departen sobre “olores de códigos” (esta expresión es muy usada por otros autores y desarrolladores), que solicitan refactorización y profundizan mucho sobre las diversas técnicas para arreglarlos.

La refactorización según ellos y la habilidad para escribir códigos de prueba son las aptitudes más importantes para que los programadores nuevos aprendan. Si todos fueran buenos en ambos aspectos, revolucionaría la industria. Si te vuelves bueno en ambos, en última instancia producirá códigos más limpios y aplicaciones más funcionales que muchos de sus compañeros/as.

En Java debemos tener en cuenta que existen palabras reservadas y hay que respetar en su entorno, y no podemos utilizarlas para nombrar nuestros objetos, variables, textos o cualquier componente de nuestra programación.

Cuando estes escribiendo código Java, debería saber que Java reserva ciertas palabras clave como parte del lenguaje. No hay muchas, de cualquier modo. A continuación se muestran:

– abstract: Define la clase o método que se va a implementar más tarde en una subclase.

– boolean: Tipo de dato que sólo consigue tomar los valores verdadero o falso.

– break: Sentencia de control para salir de los bucles.

– byte: Tipo de dato formado por cadena de 8 bits.

– Byvalue: Reservada en una utilización posterior en un futuro.

– Case: Se usa en las sentencias switch para expresar bloques de texto elegibles.

– cast: Reservada para uso futuro.

– catch: Sirve para capturar las excepciones creadas por las sentencias try.

– char: Tipo de dato que puede tener caracteres unicode sin signo de 16 bits.

– class: Prepara una nueva clase.

– const: Reservada para uso futuro.

– continue: Retorna el control a la salida de un bucle.

– default: Indica el bloque de código por defecto en una sentencia switch.

– do: Inicia un bucle do–while.

– double: Es un tipo de dato que conlleva números en coma flotante de 64 bits.

– else: Indica la salida alternativa en una sentencia if.

– extends: Sirve para indicar que una clase es descendida de otra o de una interfaz.

– final: Indica que una variable permite un valor constante o que un método no se va a sobrescribir.

- finally: Indica un bloque de código en una estructura “Try–catch” que siempre se va a ejecutar.
- flota: Sirve para indicar un tipo de dato que lleva un número en coma flotante de 32 bits.
- for: Utilizado para crear un bucle for.
- future: Reserva para un futuro.
- generic: Reserva para un futuro.
- goto: Reserva para uno futuro.
- if: Evalúa si una expresión es verdadera o falsa y la dirige adecuadamente.
- implements: Especifica que una clase efectúa una interfaz.
- import: Referencia a otras clases.
- inner: Reserva para un futuro.
- instance of: Indica si un objeto es una instancia de una clase determinada o efectúa una interfaz específica.
- int: Tipo de dato que puede acarrear un entero con signo de 32 bits.
- interface: Declara una interfaz.
- long: Tipo de dato que puede conllevar un entero de 64 bits.
- native: Sirve para indicar que un método está implementado con código nativo.
- new: Crea objetos nuevos.
- null: Indica que una referencia no se refiere a nada sino a nulo.
- operator: Reserva para un futuro.
- outer: Reserva para un futuro.
- package: Sirve para declarar un paquete Java.

- private: Indica que un método o variable sólo puede ser accesible desde la clase en la que está declarado.
- protected: Indicador de acceso que muestra que un método o variable sólo puede ser accesible desde la clase en la que está declarado, o una subclase de la clase en la que está declarada ú otras clases del mismo paquete.
- public: Indicador de acceso utilizado para clases, interfaces, métodos y variables que indican que un argumento es accesible desde la aplicación o desde donde la clase defina que es accesible.
- rest: Reserva para un futuro.
- return: Manda control y posiblemente retorna un valor desde el método que fue invocado.
- short: Tipo de dato que puede conllevar un entero de 16 bits.
- static: Sirve para indicar que una variable o método es de una clase más que estar limitado a un objeto particular.
- super: Se usa para indicar a una clase base de la clase utilizada en un método o constructor de clase.
- switch: Instrucción que ejecuta código basándose en un valor.
- synchronized: Enumera secciones o métodos examinadores de código multihilo.
- this: Se reseña y apunta al objeto actual en un método o constructor.
- throw: Crea una excepción.
- throws: Muestra qué excepción puede proporcionar un método.
- transient: Detalla que una variable no es parte del estado constante de un objeto.
- try: Forma un bloque de código que es probado para las excepciones.
- var: Reserva para un futuro.
- void: Detalla que un método no retorna ningún valor.
- volatile: Detalla que una variable podría cambiar de manera asíncrona.
- while: Crea e inicia un bucle while.

## Operadores para enteros

Los operadores que proporciona el lenguaje Java permiten escribir expresiones de cierta complejidad.

Estos operadores pueden ser unarios, es decir, los que trabajan con un único operando; o binarios, cuando lo hacen con dos operandos a la vez.

Nosotros ya hemos utilizado algún operador de ambos tipos.

Por ejemplo, el operador punto (.) permite acceder a los métodos y propiedades de un objeto.

Por otra parte, el operador + permite realizar la suma aritmética de dos valores numéricos o bien la concatenación de dos cadenas de texto.




Recuerda que Java es un lenguaje rígido en cuanto a las comprobaciones de tipo; es decir, una expresión en la que aparezca un operador sólo tendrá éxito cuando los operandos pertenezcan a un tipo adecuado para dicho operador.

Los operadores de Java se dividen en categorías según la naturaleza de la operación que llevan a cabo.

En el caso de operadores para enteros podrá encontrar:

- Operadores de asignación: =, +=, -=, *=, /=
- Operadores de comparación: ==, !=, <, >, <=, >=
- Operadores de signo unitarios: -
- Operadores aritméticos: +, -, *, /, %
- Operadores de incremento y decremento: ++, --
- Operadores de desplazamiento a nivel de bit: >>, <<
- Operador negación lógica a nivel de bit: ~
- Operadores AND, OR, XOR a nivel de bit: &, |, ^

Observando este código fuente. Hemos declarado tres variables de tipo short, es decir, que podrán contener valores enteros (en el rango [-32768 a 32767]). A las dos primeras se les asigna el valor 100 y a la tercera se le asigna el resultado de realizar la operación de adición.

Éste es un ejemplo típico en el que puede ver el uso de varios operadores. El más utilizado es el de asignación (=).

Al mismo tiempo, puede ver cómo se utiliza el operador + para realizar la operación de adición o suma.

```
public class Enteros {
public static void main(String[] args)
{
short operando1 = 150;
short operando2 = 150;
short resultado;
resultado = (operando1 + operando2);
System.out.println(resultado);
}
}
```

Si intenta compilar este sencillo ejemplo, obtendrá un error. El problema surge cuando se asigna el resultado de la operación suma a una variable short.

¿Dónde está el error? Muy sencillo, debe saber las siguientes reglas sobre los operadores para enteros:

Los operadores de igualdad y comparación siempre producen un resultado boolean (true o false).

Los otros operadores binarios producen un resultado de tipo int o de tipo long (nunca byte o short). Si alguno de los operandos es un long, entonces el resultado es un long. También será un long si el resultado desborda la capacidad de una variable int. En otro caso, el resultado será un int.

Por lo tanto, el error se ha producido al asignar un valor int a una variable short, ya que el resultado de la suma, aunque los operandos sean de tipo short, es un valor int.

Java introduce dos tipos de conversiones entre tipos. Las conversiones pueden forzar a que una expresión de cierto tipo sea de un tipo diferente.

Java puede producir conversiones implícitas, es decir, sin escribir código para ello, si el tipo objetivo es mayor (en capacidad) que el tipo inicial.

Por ejemplo, podrás convertir implícitamente un short a un int sin necesidad de indicarlo.

Sin embargo, si el tipo objetivo es menor que el tipo inicial, entonces debes indicarlo explícitamente en el código, demostrando “que sabe lo que hace”.

Por ejemplo, podríamos convertir explícitamente el resultado de la suma de nuestro ejemplo (un int) al tipo short, pero deberíamos tener en cuenta que se podría perder información, ya que short admite un rango de valores inferior a int.

Para ello se utiliza la siguiente sintaxis: (tipo objetivo) expresión.

Donde el tipo objetivo es el nombre del tipo de datos al que queremos convertir la expresión.

```
resultado = (short)(operando1 + operando2);
```

Lo que acabas de hacer es crear una expresión de conversión explícita (casting en inglés). Está indicando que desea convertir el valor de la suma en un valor de tipo short.

Es importante entender que simplemente estamos obteniendo una expresión "equivalente" con el tipo de datos que ponemos entre paréntesis.

Si la expresión siguiente al casting fuera una variable, como en (short)a, entonces esa expresión se evaluaría como del tipo short, pero la variable a seguiría siendo del tipo original, ya que no se produciría ningún cambio en ella.

Sin embargo, debe saber muy bien lo que está haciendo ya que, en caso contrario, podría producir resultados inesperados.

Ejecutando ahora el código obtendremos el resultado 300. Como el valor 300 está dentro del rango de valores válidos para el tipo short, todo va bien.

Sin embargo, modifiquemos el valor inicial de los operandos, de forma que el resultado de la operación no sea un valor de tipo short,como ve a continuación:

```
short operando1 = 32767;
short operando2 = 1;
```

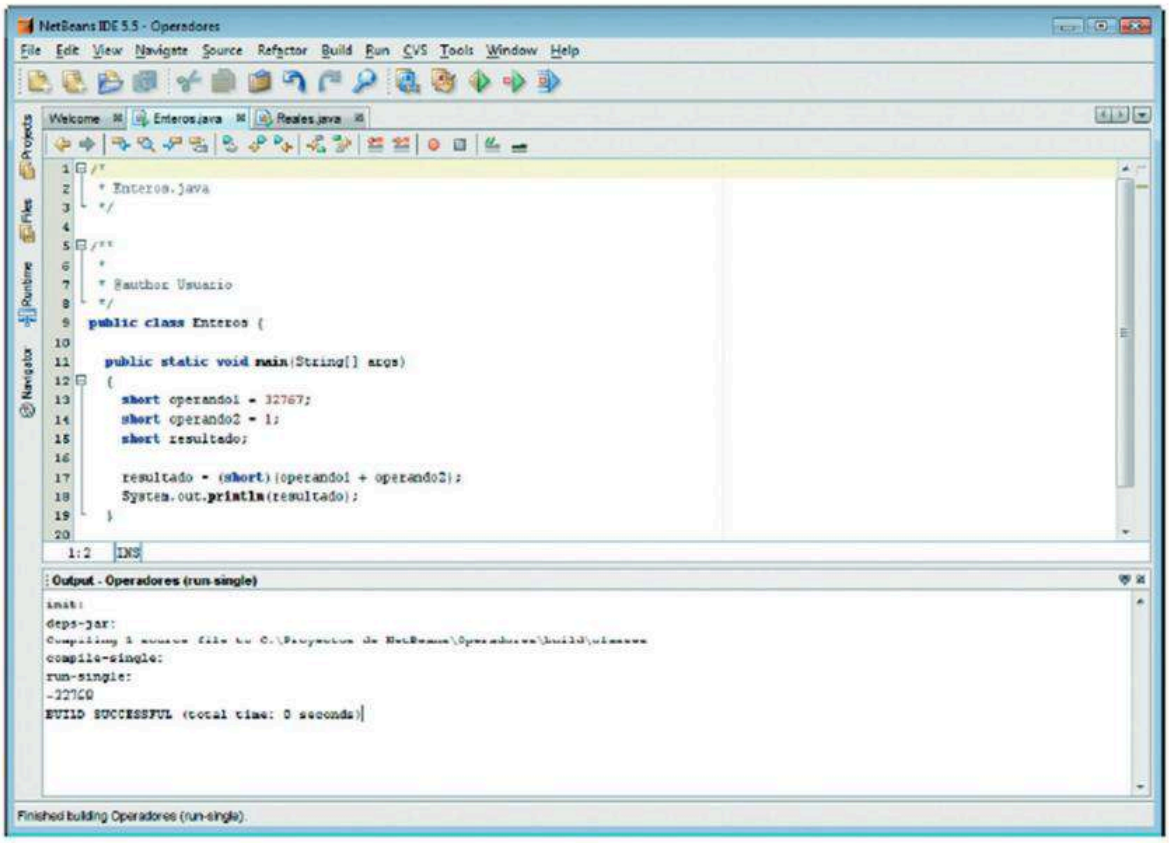
Observa cómo ahora el resultado de sumar los dos operandos ya no es un valor de tipo short, sino que se supera en 1 el valor máximo permitido en este tipo.

Es importante saber, que no se puede cambiar el tipo de datos de una variable posteriormente a su declaración.

Al ejecutar el nuevo código obtendremos el valor negativo -32768, pero ningún error.

De la cosas más imprevisibles que nos puede pasar en programación. Los errores de cálculo.

A continuación en la imagen veremos lo que ha ocurrido.




*Ejemplo de resultado erróneo por el tipo de datos*

Ejemplo de pantalla en Netbeans donde un ejercicio no produce ningún error de compilación pero sin embargo produce el ejercicio algo peor, como es un error de cálculo.

## Desbordamiento (overflow y underflow)

Se llama desbordamiento u overflow al hecho de que el valor que recoge una variable de cierto tipo de datos supera el rango de valores válidos de ese tipo.

Volviendo al ejemplo que nos ocupa, vemos que el tipo short tiene un rango de valores válidos que va desde el valor negativo -32768 hasta el valor positivo 32767.

Sin embargo, el resultado de la operación 32767 + 1 es 32768, que supera este rango.

Cuando esto sucede, Java no produce ningún error sino que empieza a contar desde el número negativo más pequeño, dando resultados inesperados y difíciles de detectar si no se han establecido las correspondientes comprobaciones.

El desbordamiento es un asunto a tener muy en cuenta y se relaciona directamente con la elección de los tipos de datos adecuados.

Podríamos pensar que lo mejor es utilizar siempre los tipos de datos más grandes para no tener este tipo de problemas.

Sin embargo, a mayor capacidad del tipo de datos, también mayor necesidad de memoria es requerida cada vez que declara una variable de ese tipo.

Por ello, su código debe ser lo suficientemente robusto y eficiente para tratar este tipo de situaciones.

Por otra parte, la situación opuesta al overflow es el underflow.

Por ejemplo, si hiciera una operación como -32768 - 1 con variables de tipo short, produciría underflow.

En el ejemplo que estamos siguiendo, lo lógico sería utilizar el tipo int para la variable suma y disponer, de esta forma, de un rango de valores mucho mayor.

```
public class Enteros {
public static void main(String[] args)
{
short operando1 = 32767;
short operando2 = 10;
int resultado;
resultado = (operando1 + operando2);
System.out.println(resultado);
}
}
```

## Operadores reales

Java permite trabajar con números reales, es decir, con aquellos que tienen una parte entera y otra decimal. Para ello, el lenguaje aporta dos tipos de datos: float y double.

Trabajar con números reales es más difícil que hacerlo con enteros ya que, al realizar operaciones, pueden darse resultados erróneos debido a redondeos o pérdidas de información.

Los valores float y double son siempre valores aproximados y no exactos como los enteros, ya que dependen de la exactitud con la que se han almacenado.

```
public class OperadoresReales {
public static void main(String[] args)
{
float sumandos = 0;
for (byte i=0; i<10; i++)
{
sumandos = sumandos + (float)0.1;
}
System.out.println("La Suma es: " + sumandos);
}
}
```

Estudia el código de este archivo. Observe que se declara una variable local de tipo float que se inicializa al valor o (recuerde que todas las variables locales deben inicializarse).

Después se utiliza un bucle, tema que estudiaremos más tarde, de forma que se suma 10 veces la cantidad 0.1, es decir, que al final del bucle la variable suma debería tener el valor 0.1 * 10 = 1.0.

Si ejecuta este archivo, comprobará que el resultado es ligeramente distinto al esperado: 1.0000001.

Este ejemplo nos debe servir para trabajar de forma muy precavida con los números reales, es decir de tipo float o de tipo double. Por ejemplo, no es adecuado realizar una comparación exacta entre dos números reales ya que el resultado puede verse afectado por la pérdida de exactitud.

Cuando desee comparar dos números reales, lo mejor es comparar la diferencia de ambos respecto a una constante que determine la exactitud de la operación.

En nuestro ejemplo, podría decir que la expresión suma == 1.0 es cierta si la diferencia entre ambos es menor de 0.0000001.

Los símbolos utilizados para los operadores que trabajan con reales son los mismos que para los que trabajan con enteros.

Operadores con números reales:

– Operadores de asignación.

– Operadores de comparación.

– Operadores de signo unitarios.

– Operadores aritméticos.

– Operadores incremento y decremento.

Por otra parte, debe tener en cuenta las siguientes reglas:

– El resultado de una operación en la que aparece al menos un valor double es siempre del tipo double.

– En una operación binaria en la que aparece un entero y un número real, el resultado será del tipo de datos del número real.

– Las operaciones binarias que involucren dos números float será un float.

– Un valor literal que incluye decimales es, por defecto, un double.

– La última regla es la que ha requerido de la conversión explícita del valor 1.0 en float: suma = suma + (float)0.1;

Si no lo hubiéramos hecho, el compilador habría detectado un error al no poder convertir implícitamente un valor double (el resultado de la operación) en un valor float (el tipo de la variable suma).

## Operador de concatenación

Un operador especial es el signo + ya que puede actuar tanto con valores numéricos como con valores de texto.

Cuando los dos operandos son numéricos, entonces se realiza la suma aritmética de ambos, mientras que, cuando uno o ambos son cadenas de texto, entonces se realiza lo que se llama la concatenación de las cadenas.

Por ejemplo la operación “Hasta “ + “luego” produce una tercera cadena “Hasta luego” (compruebe cómo en la primera cadena existe un espacio en blanco al final).

Las cadenas de texto deben aparecer siempre entre comillas dobles.

```
public class Silla {
private String color;
private String modelo;
private double precio;
public Silla() {
this.color = “Blanco”;
this.modelo = “Livanco”;
this.precio = 0.0;
}
public Silla(String color, String modelo,double precio)
{
this.color = color;
this.modelo = modelo;
this.precio = precio;
}
}
```

En este caso se ha definido una clase Silla que consta de tres propiedades: color, modelo y precio.

Además, se han escrito dos constructores para la clase: el primero se llama sin parámetros, por lo que toma los valores por defecto; el segundo utiliza parámetros, por lo que se puede indicar el valor de dichas propiedades.

Fíjate que se puede crear más de un constructor para una misma clase siempre que puedan diferenciarse entre ellos, es decir, que la lista de parámetros sea distinta.

Todas las clases de Java poseen un método heredado de la clase Object que permite devolver la representación, en forma de cadena de texto, de un objeto. Este método se llama toString.

En la mayoría de los casos, usted deseará sobrescribir este método para la clase que está escribiendo, ya que, por defecto, lo único que hace es devolver el nombre de la clase.

```
public String toString() {
return (“Color: “ + color + “, “ + “Modelo: “ + modelo
+ “, “ + “Precio: “ + precio);
}
```

Estamos sobrescribiendo dicho método para que devuelva una representación en forma de cadena de texto de un objeto Silla. Para ello, utilizamos el operador de concatenación.

Por lo tanto, al aplicar el método toString a un objeto de la clase Silla, obtendremos una cadena de texto del estilo de:

```
“Color: Rojo, Modelo: MP021, Precio: 125.48”
```

Este método puede servir para aportar la descripción de la Silla en un momento dado. Lo importante es darse cuenta de cómo se va construyendo la cadena final a partir de otras cadenas.

Así, la operación "Color: " + color proporciona una cadena en la que el valor de la propiedad color se inserta al final de la primera cadena. De la misma forma, ocurre con el resto de la expresión.

Observa cómo no importa el tipo de datos de las propiedades ya que se convierte implícitamente a texto al realizar la concatenación.

Ahora utilicemos dicho método:

```
public class NewClass {
public static void main(String[] args)
{
Puerta p1 = new Silla();
Puerta p2 = new Silla("Gris", "P025", 100.5);
System.out.println(p1.toString());
System.out.println(p2.toString());
}
```

Creamos un objeto p1 de la clase Silla, utilizando el constructor por defecto y otro objeto, p2, indicando el valor exacto de sus propiedades.

Observa lo que conseguimos: si deseamos especificar un determinado color, modelo y precio, utilizaremos este constructor; en caso contrario, utilizaremos el constructor por defecto.

Seguidamente imprimiremos en pantalla el resultado de invocar el método toString.

## Precedencia de los operadores

La precedencia de los operadores determina el orden en el que se evalúan las expresiones, lo que repercute en el resultado obtenido.

Por ejemplo, en la expresión 4 + 5 * 2, el resultado que obtenemos es 14, ya que se realiza primero la multiplicación y después, al resultado, la suma. De esta forma, la expresión se evalúa en los siguientes pasos: 4 + 5 * 2 = 4 + 10 = 14.

Esto es así porque el operador * tiene mayor precedencia que el operador +. A continuación se detalla la precedencia de los operadores, de mayor a menor:

```
. [] ()
++ -- (sufijo)
++ -- (prefijo) ! ~
* / %
+ -
<< >> >>>
< > <= >= instanceof
== !=
&
^|
&&
||
? :
= += -= *= /= %= &= ^= |= <<= >>= >>>=
```

Siempre puede utilizar los paréntesis para cambiar el orden de evaluación. Por ejemplo, la expresión (4 + 5) * 2 dará como resultado 18. Además, ante dos operadores de la misma precedencia, el orden de evaluación se realiza de izquierda a derecha.

Conocidos también como operandos, indican una evaluación o computación para ser realizada en objetos o datos, y en definitiva sobre identificadores o constantes. Los operadores admitidos por Java son:

| | | | | |
|---|---|---|---|---|
| + | ^ | <= | ++ | %= |
| >>>= | - | ≈ | >= | - |
| &= | . | * | && | << |
| == | <<= | [ | / | \|\| |
| >> | +- | ^= | ] | % |
| ! | >>> | = | != | ( |
| & | ¿!! | /= | >> | |

## Separadores

Son utilizados para notificar al compilador de Java de cómo están congregadas las sentencias en el código. Los separadores permitidos por Java son: { } ,: ;

## Comentarios y espacios en blanco

En Java su compilador examina y elimina los espacios, retornos de carro, tabuladores y comentarios durante la exploración del código fuente.

Los comentarios se representan en tres formatos de comentarios en Java distintos:

| Formato | Uso |
|---|---|
| **/*comentario*/** | Se ignoran todos los caracteres entre /* */ |
| **//comentario** | Se ignoran todos los caracteres detrás de // hasta el fin de línea. Proviene del C++ |
| **/**comentario*/** | Lo mismo que /* */ pero se podrán utilizar para documentación automática. |

Este código presenta un comentario.

Ejemplo:

```
int alturaMinima = 160; // Mínimo de 160 centímetros
```

## Expresiones

Los operadores, variables y las llamadas a métodos pueden ser mezclados en secuencias conocidas como expresiones. El comportamiento real de un programa Java se logra a través de expresiones, que se agrupan para establecer sentencias.

Una expresión es una serie de variables, operadores y llamadas a métodos (programada conforme a la sintaxis del lenguaje) que se evalúa a un único valor.

Partimos de la premisa que las expresiones son usadas para realizar cálculos, para asignar valores a variables, y para ayudar a controlar la ejecución del flujo del programa. La tarea de una expresión está formada de dos partes: realizar el cálculo requerido por los elementos de la expresión y retorna el valor obtenido como resultado del cálculo. Los operadores devuelven un valor, por lo que el uso de un operador es una expresión. Por ejemplo, la siguiente sentencia es una expresión:

```
int contador=1;
contador++;
```

La expresión contador++ en este caso peculiar se evalúa al valor 1 que era el valor de la variable contador antes de que la operación suceda, pero la variable contador adquiere un valor de 2.

El tipo de datos del valor retornado por una expresión tiene que ver con los elementos usados en la expresión. La sentencia contador++ retorna un valor entero ya que ++ retorna un valor de identico tipo que su operando y contador es un entero. Otras expresiones retornan valores cadenas, booleanos.

Una sentencia de invocación a un método se evalúa al valor de retorno del método; así el tipo de dato de la sentencia de llamada a un método es identico que el tipo de dato del valor de retorno de ese método.

Otra sentencia interesante sería:

```
in.read( ) != -1 // in es un flujo de entrada
```

Esta sentencia se compone de dos expresiones:

- La primera expresión es una llamada al método in.read() El método in.read() es creado para retornar un entero, por lo que la expresión in.read() se ajusta a un entero.

- La segunda expresión contenida en la sentencia utiliza el operador !=, que comprueba si dos operandos son distintos. En la sentencia en cuestión, los operandos son in.read() y -1. El operando in.read() es válido para el operador !=, porque in.read() es una expresión que se evalúa a un entero, así que la expresión in.read() !=-1 compara dos enteros. El valor devuelto por esta expresión será verdadero o falso dependiendo del resultado de la lectura del fichero in.




Como se puede contemplar, Java soporta crear sentencias “expresiones compuestas” a partir de diferentes expresiones más pequeñas, con tal que los tipos de datos necesarios por una parte de la expresión coincidan con los tipos de datos de la otra.

## Bloques y ámbito

En Java el código fuente está agrupado en partes incluidas entre llaves, denominas bloques. Cada bloque se halla de forma independiente entre bloques de llaves, agrupando en su interior sentencias y expresiones referidas.

Desde un bloque de código exterior se entiende que todo lo que está dentro de llaves se ejecuta como una sentencia. Pero, ¿qué es un bloque externo?. Esto tiene explicación si comprendemos que existe una jerarquía de bloques, y que un bloque puede poseer uno o más sub–bloques anidados.

Ámbito es un concepto que está estrictamente relacionado con el concepto de bloque y es muy importante cuando se trabaja con variables en Java. El ámbito se relaciona cómo las secciones de un programa y bloques afectan el tiempo de vida de las variables.

Toda variable tiene un ámbito, en el que se usa y viene determinado por los bloques. Una variable definida en un bloque interno no es visible por el bloque exterior.

Las llaves de separación son vitales no sólo en un sentido lógico, ya que son la forma de que el compilador diferencia dónde acaba una sección de código y dónde comienza otra, sino que tienen una connotación estética que facilita la lectura de los programas a los desarrolladores y programadores.

De igual manera para el programador, no para el compilador. La sangría también conocida como indentación, es más apropiada para la vista y legibilidad de un programa Java son dos espacios:

```
{
// Bloque exterior
int x = 1000;
{
// Bloque interno invisible al exterior
int y = 2000;
}
x = y; // Es  erroneo: Fuera de ambito y retorna un error.
}
```

## 6.2. Análisis de estructuras de control: secuencial, condicional y de repetición

En el proceso de ejecución de un programa concurren acciones que se han de redundar un número determinado de veces. Por ejemplo, leer 3 caracteres de un flujo de entrada in se codificaría:

```
in.read();
in.read();
in.read();
```

Este fragmento de código además de poco refinado no es factible para una repetición de 3000 lecturas. Por eso necesitamos las estructuras de control, que permiten que distintas acciones se ejecuten varias veces, desde que una condición se cumpla, y en definitiva tomar decisiones de qué hacer en función de las condiciones que se den en el programa en un momento dado de su ejecución.

Nuestro ejemplo puede ser el siguiente con un bucle for:

```
int i=0;
for ( i=0 ; i <= 3 ; i++ )
in.read();
```

Con solo variar el 0 por otro número para que la lectura se reiterase ese número de veces. El lenguaje Java permite distintas estructuras de control:

| Sentencia | Clave |
|---|---|
| **Toma de decisión** | if–else, switch–case |
| **Bucle** | for, while, do–while |
| **Miscelaneo** | break, continue, label:, return |

## La sentencia if–else

La sentencia If–Else de Java confiere a los programas de la práctica de rodar distintos bloques de instrucciones según algún criterio.

La sintaxis de la sentencia *If–Else* es:

```
if ( condición )
Bloque de código a ejecutar si la condición es cierta
else
Bloque de código a ejecutar si la condición es falsa
```

La parte del *else* es opcional, y un bloque de código puede ser meramente la sentencia vacía ; para representar que en ese caso no se ha de ejecutar nada. Partimos de la premisa que un programa debe efectuar diferentes acciones según si el usuario oprime el botón aceptar o el botón cancelar en una ventana de dialogo. Nuestro programa puede realizar esto usando la sentencia *If–Else*:

```
// La respuesta es Aceptar ó Cancelar
if (respuesta == Aceptar) {
// código para realizar la acción Aceptar
System.out.println( “Su peticion esta siendo atendida” );
}
else {
// código para realizar la acción Cancelar
System.out.println( “Cancelando accion” );
}
```

Además If–Else, se usa para poder efectuar aquellos casos con múltiples acciones. Esto es lo que se suele nombrar como sentencias *Else–If*. Por ejemplo, suponemos que se desea escribir un programa que clasifique según el contenido de una variable *valor*, asigne una letra a una variable *clasificación* para un valor del 100–91, B de 90–81, C para 80–71 y F si no es ninguno de los anteriores:

```
int valor;
char clasificacion;
if (valor > 90)
{clasificacion='A';}
else
if (valor > 80)
{clasificacion='B';}
else
if (valor > 70)
{clasificacion='C';}
else
{clasificacion='F';}
```

Se pueden escribir los If en las mismas líneas que los *else*, pero desde este tutorial se insta a utilizar la forma indentada (como se ha podido ver en el ejemplo), pues es más clara para el lector.

Este sistema de programación (Else–If ) no es demasiado recomendable, y por ello el lenguaje Java incluye la sentencia *Switch* que veremos a continuación, para dirigir el flujo de control de variables con múltiples valores.

## Sentencia switch

Mediante la sentencia *Switch* se puede seleccionar entre varias sentencias según el valor de cierta expresión.

La forma general de *Switch* es la siguiente:

```
switch ( expresionMultivalor ) {
case valor1 : conjuntoDeSentencias; break;
case valor2 : conjuntoDeSentencias; break;
case valor3: conjuntoDeSentencias; break;
default: conjuntoDeSentencias; break;
}
```

La sentencia switch chequea la expresiónMultivalor y ejecuta el conjuntoDeSentencias que aparece junto a la cláusula case cuyo valor corresponda con el de la expresión Multivalor.

Cada sentencia case debe ser única y el valor que evalúa debe ser del mismo tipo que el devuelto por la expresiónMultivalor de la sentencia switch.

La sentencia break que se colocan tras cada conjunto de sentencias, incita a que el proceso salga del switch y vaya a la siguiente instrucción del Switch. La sentencia break es necesaria porque sin ella se ejecutarían secuencialmente las sentencias case siguientes. Existen ciertas situaciones en las que se necesita arrancar secuencialmente algunas o todas las sentencias case, para lo que tendríamos que eliminar algún break.

Por último, se puede utilizar la sentencia "default" para operar con los valores que no son explícitamente consentidos por alguna de las sentencias case. Su uso es muy importante.

Por ejemplo, partimos de un bloque de código con una variable entera "months" cuyo valor indica el mes actual, y se necesita lanzar el nombre del mes en la actualidad. Se puede usar la sentencia switch para realizar esta operación:

```
int months;
switch ( months ){
case first: System.out.println( “January” ); break;
case second: System.out.println( “February” ); break;
case third: System.out.println( “March” ); break;
//Otros meses
// . . .
case twelve: System.out.println( “December” ); break;
default: System.out.println( “Month not valid !!” ); break;
}
```

Por supuesto, se puede implementar esta estructura como una sentencia if else if

```
int months;
if ( months == 1 ) {
System.out.println( “January” );
}
else
if ( meses == 2 ) {
System.out.println( “February” );
}
// Y así para los demás meses
```

El pensar si utilizar la sentencia if o Switch tendrá que ver con el razonamiento de cada caso. Se puede elegir cuál usar basándonos en la legibilidad, aunque es preferible utilizar switch para sentencias con más de tres o cuatro posibilidades.

Sentencias de iteración o bucles: for, do, while

## Bucle while

Este bucle “while” es elemental para acciones repetitivas. Se usa para hacer una acción sucesivamente mientras se cumple una condición establecida.

El formato general del bucle while es la siguiente:

```
while ( expresiónBooleana ) {
sentencias;
}
```

Las sentencias se ejecutan mientras la expresiónBooleana tenga un valor de verdadero/falso. Se utiliza, por ejemplo para estar en un bucle del que no hay que salir hasta que no se cumpla una determinada condición. Por ejemplo, multiplicar un número por 5 hasta que sea mayor que 100.

```
int i = 1;
while ( i <= 100 ) {
i = i * 5;
}
```

Con esta instrucción se podrían eliminar los bucles “do while” y “for” por ser ampliaciones de éste, pero que se tienen en el lenguaje para mejorar la programación.

## Bucle do–while

El bucle "Do–while" es análogo al bucle "while", pero en el bucle while *el* término se evalúa al principio del bucle y en el bucle *do*–while *la* evaluación se realiza al final.

El formato genérico del bucle *do*–while es el siguiente:

```
do {
sentencias;
} while ( expresiónBooleana );
```

La sentencia do–while suele usarse poco aunque tiene su utilidad en casos en los que el bucle será ejecutado mínimo una vez. Por ejemplo, cuando se lee información de un archivo, se sabe que siempre se debe leer por lo menos un carácter: por ejemplo Escribir un programa que solicite la carga de un número entre 0 y 999, y nos muestre un mensaje de cuántos dígitos tiene el mismo. Finalizar el programa cuando se cargue el valor 0.

En este ejercicio se carga por lo menos un valor. Si se carga un valor mayor o igual a 100 se tratará de un dígito de tres cifras, si es mayor o igual a 10 se trata de un valor de dos dígitos, en caso contrario se trata de un valor de un dígito. Este bloque es repetitivo hasta que se introduce en la variable valor el número 0 con lo que la condicional del bloque do while retorna falso y sale del bloque repetitivo finalizando el programa.

Programa:

```
import java.util.Scanner;


public class Estructura2 {
    public static void main(String[] ar) {
```

```
        Scanner teclado=new Scanner(System.in);
        int valor;
        do {
           System.out.print("Teclea un valor entre 0 y 999 (ó pulsa Fin):");
            valor=teclado.nextInt();
            if (valor>=100) {
               System.out.println("Tiene 3 dígitos.");
            } else {
               if (valor>=10) {
                  System.out.println("Tiene 2 dígitos.");
               } else {
                  System.out.println("Tiene 1 dígito.");
               }
            }
        } while (valor!=0);
    }
}
```

## Bucle for

Por medio de la sentencia "For" se puede resumir un bucle "Do while" con una iniciación previa. Es muy corriente que en un bucle while y do–while se inicie la variable de control de número de recorridos por el bucle, inmediatamente antes de comenzar los bucles. Por eso el bucle for está tan extendido.

El formato general de la instrucción for es esta:

```
for ( iniciación ; finalización ; incremento )
bloque de código;
```

La iniciación es una instrucción que se ejecuta una primera vez antes de entrar en el bucle.

La finalización es una expresión que define cuándo se debe terminar el bucle. Esta expresión se evalúa al final de cada ciclo repetitivo del bucle. Si la expresión se evalúa a falso, el bucle finaliza.

El incremento es una expresión que es invocada en cada iteración del bucle. En realidad puede ser una acción cualquiera, aunque se suele utilizar para incrementar una variable contador:

Algunos o todos estos elementos podrían obviarse, sin embargo los puntos y coma deberán aparecer no obstante sea sin nada más entre sí.

```
for ( i = 1 ; i < 100 ; i++ )
```

Para usar el bucle “for” aún se conozcan las limitaciones del bucle su instrucción de iniciación, criterio de finalización e instrucción de incremento.

Por ejemplo, los bucles for son usados corrientemente para iterar sobre los elementos de una matriz, o los caracteres de una cadena:

```
// frase es una cadena (String)
for ( int i = 0; i < frase.length() ; i++){
// aplicar accion con el elemento i-ésimo de frase
}
```

Por ejemplo par explicar a partir de un problema con bucle for vamos a escribir un programa que lea 10 notas de alumnos y nos informe cuántos tienen notas mayores o iguales a 5 y cuántos menores.

Para resolver este problema se requieren tres contadores:

– aprobados (Cuenta la cantidad de alumnos aprobados)

– suspensos (Cuenta la cantidad de suspendidos)

– rastreaalumno (es el contador del for)

Dentro de la estructura repetitiva debemos hacer la carga de la variable nota y verificar con una estructura condicional si el contenido de la variable nota es mayor o igual a 5 para incrementar el contador aprobados, en caso de que la condición retorne falso debemos incrementar la variable suspensos.

Los contadores aprobados y suspensos deben imprimirse al margen de la estructura repetitiva.

Es fundamental inicializar los contadores aprobados y suspensos en cero antes de entrar a la estructura for.

Para evitar un fallo común como es inicializar los contadores dentro de la estructura del blucle. En caso de hacer esto, los contadores se establecen a cero en cada ciclo del for, por lo que al terminar el for como máximo el contador puede tener el valor 1.

Programa:

```
import java.util.Scanner;
public class OrdenaNotasAlumnos {
    public static void main(String[] ar) {
        Scanner teclado=new Scanner(System.in);
        int aprobados,suspensos,rastreaalumno,nota;
        aprobados=0;
        suspensos=0;
        for(rastreaalumno=1;rastreaalumno<=10;rastreaalumno++) {
            System.out.print("Teclea por favor la nota:");
            nota=teclado.nextInt();
```

```
            if (nota>=5) {
                aprobados=aprobados+1;
            } else {
                        suspensos=suspensos+1;
            }
        }
        System.out.print("Cantidad de aprobados:");
        System.out.println(aprobados);
        System.out.print("Cantidad de suspensos:");
        System.out.print(suspensos);
    }
}
```

Sentencias de salto: break, continue y return

### Sentencia break

La sentencia Break provoca que el flujo de inspección salte a la sentencia inmediatamente posterior al bloque en curso. Ya se ha comprobado anteriormente la sentencia Break dentro de la sentencia switch.

El uso de la sentencia break con sentencias etiquetadas es una alternativa al uso de la sentencia goto, que no es soportada por el lenguaje Java.

Se puede etiquetar una sentencia colocando una identificador Java válido seguido por dos puntos antes de la instrucción:

```
nombreSentencia: instruccionEtiquetada
```

El mandato break es usado para salir de una instrucción etiquetada, trasladando el flujo del bloque de código al final de la sentencia de programa que indique:

```
break nombreSentencia2;
```

Un ejemplo de esto sería el programa:

```
void gotoBreak() {
System.out.println("Ejemplo de uso de break como si fuera un goto ");
recontarySalta: for( int inneas=1; inneas<10; inneas++ ){
System.out.print(" inneas="+inneas);
for( int jaramillo=1; jaramillo<10; jaramillo++ ){
if ( jaramillo==5 )
break recontarySalta; //Sale de los dos bucles!!!
System.out.print(" jaramillo="+jaramillo);
}
System.out.print("El programa no llega aquí");
}
}
```

Al llegar al break recontarySalta, aborta la ejecución del bucle interior de jaramillo, sino que se salta al final del bucle, obteniéndose:

```
inneas=1 jaramillo=1 jaramillo=2 jaramillo=3
```

No es recomendable esta forma de programación, basada en goto, y con saltos de flujo no controlados. Por eso goto no se usa ni existe en Java.

## Sentencia continue

De la misma manera que en un bucle se puede abortar la iteración, también podemos continuar con el bucle, pero permitiendo pasar una establecida iteración.

Podemos usar la instrucción “continue” dentro de los bucles para saltar a otra instrucción, aunque no puede ser invocada fuera de un bucle.

Tras la petición a una sentencia “continue” se pasa el control a la condición de finalizar del bucle, que vuelve a ser examinada en ese momento, y el bucle continúa o no dependiendo del resultado de la evaluación. En los bucles for además en ese momento se ejecuta la cláusula de incremento antes de la examinación. Por ejemplo el siguiente fragmento de código imprime los números del 0 al 9 no divisibles por 3:

```
for ( int jaramillo = 0 ; jaramillo < 10 ; jaramillo++ ) {
if ( ( jaramillo % 3 ) == 0 )
continue;
System.out.print( “ “ + jaramillo );
}
```

Del mismo modo que break, en la instrucción “continue” se indicaría una etiqueta de bloque al que hace alusión. Con ello podemos referirnos a un bloque superior, si estamos en bucles anidados. Si dicha etiqueta no es asignada, se presupone que nos referimos al bucle en el que la sentencia continue aparece.

Por ejemplo, el siguiente fragmento de código:

```
void gotoContinue( ) {
tomandoRamillete: for ( int jaramillo=1; jaramillo <5; jaramillo++ ) {
for ( int petunias=1; petunias<5; petunias++ ) {
if ( petunias>jaramillo ) {
```

```
System.out.println(“ “);
continue tomandoRamillete;
}
System.out.print( “ “ + (petunias*jaramillo) );
}
}
}
```

En este código la sentencia continue termina el bucle de petunias y continua el flujo en la siguiente iteración de jaramillo. Ese método imprimiría:

```
1
2 4
3 6 9
4 8 12 16
```

Reitero y desaconsejo esta manera de programación, basada en goto, y con saltos de flujo no controlados.

### Sentencia return

La actual sentencia de salto es la sentencia return, que se puede utilizar para salir del método en curso y volver a la sentencia dentro de la cual se realizó la llamada.

Para restituir un valor, simplemente se debe colocar el valor o una expresión que calcule el valor a continuación de la palabra Return. El valor devuelto por return debe coincidir con el tipo declarado como valor de retorno del método.

En el momento que un método es declarado como “void” se debe utilizar la forma de return sin asignarle ningún valor. Esto se realiza para no tener que ejecutar todo el código del bloque.

```
int cronos;
boolean condicion;
int retornaCronos(){
return ++cronos;
}
void metodoReturn(){
//Sentencias
if ( condicion == true )
return;
//Más sentencias a ejecutar si condicional no toma true
}
```

## 6.3. Distinción entre funciones y procedimientos

La diferencia primordial está en que:

### Funciones

Una función es un módulo de un programa apartado del cuerpo principal, que ejecuta una tarea específica y que puede retornar un valor a la parte principal del programa u otra función o procedimiento que la invoque.

El formato generico de una función:

```
datoretornable Nombre_funcion(parametros)
{ cuerpo de instrucciones;
intruccion return;
};
```

El tipo declara el valor que la funcion retornara utlizando la instrucción return. Si no se especifica un tipo se asume de default que el tipo regresado es int. La lista de parámetros formales es una lista de variables separadas por comas (,) que acumularan los valores que recoja la funcion, estas variables se comportan como locales internamente en el cuerpo de la funcion.

Aunque no se ocupen parametros los paréntesis son requeridos. La declaracion de parametros es la especificacion de cada tipo de parametro recibido.

### Instrucción return

Internamente en el cuerpo de la función tiene que haber una instrucción "return" para devolver el valor, es esta instrucción quien regresa el dato.

```
import java.lang.*;
import java.io.*;
class retornando {
// variable global
static int variaglobal=1000;
public static void main(String[] args) {
// llamando a funcionejemplo y pasando algunos parametros
double alfa = funcionejemplo(25+31);
System.out.println(" la suma es =" + alfa );
}; // cierra main
public static double funcionejemplo (int totaliza )
{
return totaliza + 1240;
}; // cierra funcionejemplo
} // cierra clase retornando suma
```

## Procedimientos

Recordar que un objeto presenta tres aspectos, propiedades, métodos y eventos, en este punto se analizan algunos elementos que intervienen en la definición de un método. Estamos hablando de los llamados procedimientos y funciones, que quede claro que procedimientos y funciones son solo algunos aspectos importantes de la definición de un método, pero que existen elementos tan o mas importantes que los analizados en esta unidad.

Un procedimiento es un grupo de instrucciones, variables, constantes, etc, que están definidos con una idea particular y tiene su nombre propio. Resumiendo un procedimiento es un modulo de un programa que efectúa tareas declaradas y que no puede retornar valores al programa principal u a otro procedimiento que lo este invocando.

Tras diseñar un procedimiento se debe usar su nombre propio como una sola instrucción o invocación al procedimiento.

Su estructura es:

```
static void NombreProc() { instrucciones; } ;
```

Un programa puede poseer tantos procedimientos como se deseen, para hacer una llamada o invocación al procedimiento durante la ejecución de un programa se deberá escribir el nombre de la misma y los paréntesis en blanco.

```
import java.lang.*;
import java.io.*;
class reconocimientoProc {
public static void main(String[] args) {
// llamando procedimiento uno
```

```
reconoProceduno();
}; // cierra main
public static void reconoProceduno()
{
System.out.println("soy procedimiento uno ");
//enlazando reconoProceddos()
reconoProceddos();
}; // cierra reconoProceduno
public static void reconoProcedos()
{
System.out.println(" y yo soy procedimiento dos ");
}; //cierra reconoProceddos
} // cierra clase reconocimientoProc
```

Los procedimientos se diseñan como parte de la clase pero al margen de main(){};

### 6.3.1. Interfaz

Las interfaces Java son expresiones puras de diseño. Son como genuinas conceptualizaciones no implementadas que valen de guía para concretar una determinada noción (clase) y lo que debe hacer, pero sin desarrollar un mecanismo de solución.

Consiste en que declaramos métodos abstractos y constantes que a continuación puedan ser utilizados de diferentes formas según las necesidades de un programa.

Como vemos una interfaz podría ser desarrollada en una versión beta de prueba de forma experimental, y ser mejorada convenientemente en la versión decisiva tras conocer más a fondo el problema.

Ejemplo de declarar una Interfaz:

Podemos declarar una interfaz si usamos la instrucción "interface", de la misma forma que se usa la sentencia class:

Ejemplo de bloque de código con interface:

```
interface PrimeraInterfaz {
int Contante = 1000;
int metodoAbstracto( int parametros );
}
```

Se comprueba en la implementación que las variables adoptan la afirmación en mayúsculas, pues en realidad actuarán como constantes finales. En ningún caso dichas variables se comportarán como variables de instancia.

De ahí que los métodos tras su definición presentan un punto y coma, en lugar de su cuerpo entre llaves. Son métodos abstractos, por tanto, métodos sin implementar.

Para Implementar una interfaz hay que decir que las interfaces carecen de funcionalidad por no estar implementados sus métodos, es necesario algún elemento para dar cuerpo a sus métodos.

El mandato “implements” usada en la definición de una clase refleja como la clase implementa la interfaz, que asume las constantes de la interfaz, y codifica sus métodos:

Ejemplo de código implementando una interfaz:

```
class ImpleInterfaz implements Uninterfaz{
int multip=Constante;
int metAbstracto( int parametros ){
return ( parametros * multip );
}
}
```

Vemos en el ejemplo que se han de codificar todos los métodos que define la interfaz metAbstracto() y la validez de la constante que define la interfaz durante toda la declaración de la clase.

Una interfaz no puede implementar otra interfaz, aunque sí extenderla Extends si la amplia.




La aplicación de varias interfaces redunda en la solución dentro del paradigma de programación orientada a objetos y en Java del problema del diamante. Donde la herencia múltiple no soportada en Java es faclmente resuelta con varias interfaces.

### 6.3.2. Paso de parámetros: por valor y por referencia

Este apartado en Java tiene un aporte de polémica sobre el paso de parámetros. Así como en otros lenguajes que cumplen con el paradigma OOP, donde el pase de parámetros puede ser por valor o por referencia muy claramente definidos, en Java hay que explicarlo bien.

En lenguaje Java a primera vista parece que se pasen los pasen los parámetros por referencia; aunque claramente el paso de parámetros es por valor. La confusión suele provenir por el hecho que Java no pasa el valor del objeto a los métodos sino que pasa por valor la referencia de dicho objeto.

#### Explicaciones

Cuando llamamos un método y le pasamos un objeto como parámetro se forma una copia de la referencia y se pasa la misma al método. Poseemos dos referencias distintas apuntando a un mismo objeto. La referencia inicial no se verá modificada por nada de lo que hagamos en el método que trabajará con la copia. Si se ejecuta un método sobre el objeto enviado al parámetro, como la referencia que hay enfoca al objeto originario, el objeto se verá cambiado y por eso parece y solo es eso que el objeto se pase por referencia.

Por ello es sencillo demostrar que esto no es como se piensa. Efectuamos una definición de un parámetro de entrada en un método y comprobamos que al termianr el método el objeto enviado al método no ha sido cambiado. Porque el parámetro se pasó por valor. La asignación está cambiando la referencia para que apunte al nuevo objeto. Tenemos dos objetos diferentes con sus respectivas referencias. Al finalizar el método perdemos el acceso al nuevo objeto que hemos creado ya que perdemos la referencia, como si quisiéramos acceder a una variable local desde el exterior de su ámbito.

Vemos con este código claro como funciona:

```
public class Prueba extends EntrandoLogin {

/**
 * @parametros y argumentos
 */
 public static void main(String[] args) {
 Prueba Prueba = new Prueba();
 Prueba.run();
 }

public void run() {
PruebaObject object = new PruebaObject("Primer Nombre");
 logger.error(object.toString());
 transformer(object);
 logger.error(object.toString());
 badAssigner(object);
 logger.error(object.toString());
 object = goodAssigner(object);
 logger.error(object.toString());

}

public void transformer(PruebaObject PruebaObject) {
 PruebaObject.setName("Segundo Nombre");
```

```
 }

public void badAssigner(PruebaObject PruebaObject) {
 PruebaObject = new PruebaObject("Tercer Nombre");
 }

public PruebaObject goodAssigner(final PruebaObject PruebaObject) {
 return new PruebaObject("Tercer Nombre");
 }
}

class PruebaObject {
 private String name;
 private Double value;

public PruebaObject(String name) {
 this.name = name;
 }

public String getName() {
 return name;
 }

public void setName(String name) {
 this.name = name;
 }

@Override
 public String toString() {
 return "PruebaObject [name=" + name + ", value=" + value + "]";
 }

}
```

Y su salida:

```
ERROR: Prueba.EntrandoLogin - PruebaObject [name=first name, value=null]
ERROR: Prueba.EntrandoLogin - PruebaObject [name=second name, value=null]
ERROR: Prueba.EntrandoLogin - PruebaObject [name=second name, value=null]
ERROR: Prueba.EntrandoLogin - PruebaObject [name=third name, value=null]
```

### El modificador “final”

Esta herramienta el modificar final es muy usada para evitar rompemos la cabeza descubriendo porque los objetos no tienen los valor que deberían. El modificador final no imposibilita en absoluto cambiar un objeto, lo que hace es impedir su asignación. Siempre que se pueda deberemos declarar los parámetros como “final” para evitar realizar asignaciones por error o descuido que resultarían no modificar nada.

Eso no quiere decir que todos los parámetros tienen que ser final, en momentos necesitaremos crear asignaciones de parámetros en el interior de un método, porque necesitamos el nuevo valor “dentro” de ese método, en todo momento sin perder de vista que esa modificación no será efectiva fuera del ámbito donde la realizamos.

Otro ejemplo aclarativo sobre el pase de parámetros es:

En C, se expresa un método en una clase y definirlo luego dentro de la clase o definirlo fuera de la clase. En el momento de declararlo, es preciso mostrar el tipo de argumentos que necesita, pero no se requiere indicar sus nombres. A la hora de concretar el método sí tiene que indicarse el nombre de los argumentos que necesita el método.

En Java, todos los métodos tienen que estar visibles y definidos dentro de la clase, y hay que especificar el tipo y nombre de los argumentos o parámetros que toma. Los argumentos son como variables locales definidas en el cuerpo del método, que se tienen que inicializar con el dato que se pasa como parámetro en la invocación del método.

En Java, todos los argumentos de formatos primitivos deben traspasarse por valor, mientras que los objetos deberán hacerlo por via referencia. Cuando se pasa un objeto por referencia, se está pasando la dirección de memoria en la que se encuentra almacenado el objeto.

Si se cambia una variable que se haya pasado por valor, no se modificará la variable originaria que se haya utilizado para invocar al método, mientras que si se modifica una variable pasada por referencia, la variable original del método de llamada se verá afectada de los cambios que se produzcan en el método al que se le ha pasado como argumento.

El ejemplo se muestra el paso de parámetros de tipo primitivo y también el paso de objetos, por valor y por referencia, respectivamente.

```
// Esta clase se usa para instanciar un objeto referencia
class PasandoP {
    int varInstancia = 100;
    }
  // Clase principal
class pasaycorta {

    // Función para mostrar un ejemplo de paso de parámetros
    void pasoVariables( int variaPrime,PasandoP variableReferen ) {
        System.out.println( "--> Entrada en la funcion pasoVariables" );
        System.out.println( "Valor de la variable primitiva: "+variaPrime );
        System.out.println( "Valor contenido en el objeto: "+
            varRef.variabInstancia );

        System.out.println( "-> Modificamos los valores" );
        variableReferen.variabInstancia = 101;
        variaPrime = 201;

        System.out.println( "--> Todavia en la funcion pasoVariables" );
        System.out.println( "Valor de la variable primitiva: "+varPrim );
```

```
    System.out.println( “Valor contenido en el objeto: “+
        variableReferen.variabInstancia );
    }

public static void main( String args[] ) {
    // Instanciamos un objeto para acceder a sus métodos
    pasaycorta aObj = new pasaycorta();

    // Instanciamos un objeto normal
    PasandoP obj = new PasandoP();
    // Instanciamos una variable de tipo primitivo
    int variaPrime = 200;

    System.out.println( “> Estamos en main()” );
    System.out.println( “Valor de la variable primitiva: “+variaPrime );
    System.out.println( “Valor contenido en el objeto: “+
        obj.variabInstancia );
    // Llamamos al método del objeto
    aObj.pasoVariables( variaPrime,obj );

    System.out.println( “> Volvemos a main()” );
    System.out.println( “Valor de la variable primitiva, todavia : “+
        variaPrime );
    System.out.println( “Valor contenido ahora en el objeto: “+
        obj.variabInstancia );
    }
}
```

En C, se puede pasar como parámetro un puntero que señala una función dentro de otra función, y utilizar este puntero en la segunda función para llamar a la primera. Esta capacidad no está soportada en Java. Sin embargo, se puede conseguir casi lo mismo encapsulando la primera función como un método de instancia de un objeto, y luego pasando el objeto a otro método, donde el método primero se puede arrancar a través del objeto.




Tanto en Java como en C, los métodos tienen acceso directo a las variables miembro de la clase. El nombre de un argumento puede tener identico el nombre que una variable miembro de la clase. Por ello, la variable local que sale del argumento del método, esconde a la variable miembro de la clase.

En el momento que se instancia un método, se pasa en todo momento una reseña al propio objeto que ha invocado al método, es la referencia "this."

### 6.3.3. Parámetros actuales y formales

- **Parámetros actuales:** Son los argumentos que aparecen en la invocación a un método. Poseen los valores que se le envían al método. Un parámetro actual puede ser una variable, un objeto, un valor literal válido, etc.
- **Parámetros formales:** Son los argumentos que aparecen en la cabecera del método. Recogen los valores que se envían en la invocación al método. Se usan como variables normales dentro del método.

Los parámetros actuales y los formales deben concordar en número, orden y tipo. Si el tipo de un parámetro actual no coincide con su correspondiente parámetro formal, el sistema lo convertirá al tipo de este último, siempre que se trate de tipos compatibles. Si no es viable la conversión, el compilador dará los mensajes de error correspondientes.

Si el método retorna un valor, la llamada al método puede estar contenida en una expresión que recoja el valor devuelto.

Se ha estudiado como un método (o constructor) puede demandar un parámetro, con un tipo y nombre determinados. Por ejemplo tipo String y nombre del parámetro valorDistrito. igualmente valorDistrito podría ir tomando distintos valores a lo largo de una ejecución del método, por ejemplo “Norte”, “Oeste”, “Sur”, etc.

Para explicar dos conceptos relacionados con los parámetros diremos que:

– Parámetro formal: como nombre de un parámetro de un constructor o método tal y como se declara en su cabecera, por ejemplo valorDistrito es un parámetro formal.

– Parámetro actual: es el dato determinado que posee un parámetro en un momento concreto.

```
//Esta clase es un ejemplo
public class PruebaParamet {
    public String resultado (String dameCadena) {
        System.out.println (“dameCadena inicialmente vale “ +
dameCadena);
        dameCadena = dameCadena + “ recibido”;
        System.out.println (“dameCadena ahora vale “ + dameCadena);
        return “El método devuelve “ + dameCadena;
    }
}
```

En este momento si creamos un objeto de tipo PruebaParamet y arrancamos el método tecleando una cadena, por ejemplo “demostración”. El resultado será algo similar a esto:

```
dameCadena inicialmente vale demostración
dameCadena ahora vale demostración recibido
```

El parámetro formal dameCadena es de tipo String y primeramente asume el valor que le hayamos pasado al constructor cuando creamos el objeto. Posteriormente su valor pasa a ser el valor pasado al constructor + "recibido". Finalmente el método nos retorna una cadena de la que forma parte el parámetro.




Los parámetros de métodos o constructores también almacenan valores que pueden ir cambiando. Por eso diremos que también son variables al igual que los campos, aunque conceptualmente sean cosas distintas.

Ten presente que, un campo es una variable que está de forma permanente asociada a un objeto y cuyo ámbito es toda la clase. Su duración es indefinida mientras exista el objeto al que está ligado. Para ello un parámetro formal posee un ámbito limitado al método o constructor, y tiempo de vida limitado al tiempo durante el cual se ejecuta el método o constructor. De esta manera, afirmamos que un parámetro es un tipo de variable local solo la usamos dentro del método y temporal se define al comienzo de la ejecución del método y se destruye al finalizar su ejecución. Para impedir confusiones, en general trataremos de impedir modificar el contenido de un parámetro en el interior del código de un método.

### 6.3.4. Funciones: valor de retorno

#### Valor de retorno de un método

En Java es fundamental que en la declaración de un método, se muestre el tipo de dato que ha de retornar. Si no devuelve ningún valor, se indicará el tipo void como retorno.

Los métodos y funciones en C++ consiguen devolver una variable u objeto, bien sea por valor, por puntero o por referencia. Como se comentaba en Java no se soportan los punteros, así que no puede devolver nada por puntero. Todos los tipos primitivos en Java se devuelven por valor y todos los objetos se devuelven por referencia. El devolver la referencia a un objeto en Java, es similar que retornar un puntero a un objeto situado en memoria dinámica en C++, a excepción que la sintaxis es mucho más sencilla en Java, en donde el item que se retorna es la dirección de la colocación en memoria dinámica donde se haya almacenado el objeto.

Se usa el mandato “return” si queremos retornar un valor. La palabra clave return va seguida de una expresión que será valorada para conocer el valor a devolver. Esta expresión puede ser compleja o puede ser sencillamente el nombre de un objeto, una variable de tipo primitivo o una constante.

```
// Un objeto de esta clase sera devuelto por referencia
class RetValores {
    int variabInstancia = 1000;
    }
```

Si un programa en Java retorna una referencia a un objeto y esa referencia no es aplicada a ninguna variable, o usada en una expresión, el objeto se marca rápidamente para que el reciclador de memoria en su próxima ejecución retome la memoria ocupada por el objeto al sistema, tomando que la dirección no se halla ya almacenada en ninguna otra variable. En C, si un programa devuelve un puntero a un objeto situado en memoria dinámica y el valor de ese puntero no se asigna a una variable, la posibilidad de devolver la memoria al sistema se volatiliza y se efectuará un “memory leak”, entendiendo que la dirección no está ya disponible para almacenar ninguna otra variable.

De igual manera en Java y en C, el tipo del dato de retorno debe combinar con el tipo de retorno que se ha indicado en la declaración del método; aunque en Java, el tipo actual de retorno alcanza ser una subclase del tipo que se ha insertado en la declaración del método. En Java esto es viable porque todas las clases heredan desde un objeto raíz común a todos ellos: Object.

En términos generales, es posible guardar una referencia a un objeto en una variable de referencia que sea una superclase de ese objeto. Además podemos utilizar un interfaz como tipo de retorno, en cuyo caso, el objeto retornado debe implementar dicho interfaz.




Mostrar siempre tipo de valor a retornar en la función en la declaración de un método. Sin tenemo prevcisto que no devuelve nada se declarará como Void.

### 6.3.5. Procedimientos

El término método lo comprendemos como un bloque de código, que puede ser invocado por el programa principal o por otro método para ejecutar alguna tarea específica. El término método en Java es equivalente al de subprograma, rutina, subrutina, procedimiento o función en otros lenguajes de programación. El método es invocado por su nombre o identificador, precedido por una secuencia o grupo de parámetros o argumentos utilizados por el propio método para sus cálculos, y estos irán entre paréntesis.

En el momento que el método termina sus operaciones, retorna usualmente un valor simple al programa que lo llama, que utiliza dicho valor de la forma que le venga mejor. El tipo de dato retornado por la sentencia return deberá coincidir con el tipo de dato definido en la cabecera del método.

Sintaxis de declaración de un método:

```
[modificadores] tipoValor identificadorMetodo (parametros formales) {
declaraciones de variables locales;
sentencia_1;
sentencia_2;
…
sentencia_n;
// dentro de estas instrucciones se inserta por lo menos un return
}
```

La primera línea de código tiene que ver con la cabecera del método. Los modificadores indican cómo puede invocarse al método, el tipo de dato señala el tipo de valor que retorna la llamada al método y los parámetros introducen información para la ejecución del método. Si no hay parámetros aplicados se dejan los paréntesis vacíos. A continuación, las sentencias entre llaves componen el cuerpo del método. Dentro del cuerpo del método se localiza, al menos, una sentencia return.

En este punto se enseña un ejemplo de declaración y uso de un método que retorna el cubo de un valor numérico real con una instrucción return:

```
/**
* Ejemplo del metodo calculaCubo
*/
public class calculaCubo {
public static void main (String [] args){
System.out.println("El cubo de 14.5 es: " + Cubo(14.5)); // llamada
}
public static double cubo (double x) { // declaracion
return x*x*x;
}
}
```

En oposición a otros lenguajes de programación, como Pascal, en Java, la declaración del método se realiza en el código fuente posteriormente de la llamada al propio método. En el caso anterior, public y static son los modificadores especificados en la cabecera del método. El manejo de estos dos modificadores permite que el tipo de método sea similar al de una función global de Pascal o C. El identificador double hace alusión al tipo de dato que retorna la llamada al método, calculaCubo es el identificador del método y x es el identificador del parámetro en la declaración de la cabecera del método parámetro formal. Ejemplo de ejecución del código anterior y salida correspondiente por pantalla:

```
$>java PruebaCubo
El cubo de 14.5 es: 3048625
```

En Java, los métodos suelen ir asociados con los objetos o instancias en particular a los que operan (métodos de instancia). Los métodos que no necesitan o trabajan con objetos (y sí con números, por ejemplo) se denominan métodos estáticos o de clase y se declaran con el modificador static. Los métodos estáticos o de clase son equivalentes a las rutinas (funciones o procedimientos) de los lenguajes que no emplean la programación orientada a objetos. Por ejemplo, el método sqrt de la clase Math es un método estático. También lo es el método cubo del ejemplo anterior. Por otro lado, todo programa o aplicación Java que no sea un applet tiene un método principal main que será siempre un método estático.

Es habitual emplear la palabra static para los métodos de clase. El significado o la acepción más común de la palabra estático (que permanece quieto en un lugar) parece no tener nada que ver con lo que hacen los métodos estáticos. Java emplea la palabra static porque C++ lo utiliza en el mismo contexto.

Para designar métodos de clase, aprovechando su uso en variables que poseen una única localización en memoria para diferentes llamadas a métodos, C++ lo comenzó a usar en la designación de los métodos de clase para diferenciarlos de los métodos de instancia y no enredar al compilador. El inconveniente es que nadie pensó en que el uso de la palabra static pudiera ocasionar cierta confusión.




En lenguaje Java un procedimiento es parecido a una función, se distingue en que no devuelve ningún resultado. Su uso e interés radica en las acciones que se realizan durante su invocación.

Un ejemplo de procedimiento es println(…). Éste siempre se invoca en forma aislada no como parte derecha en una asignación:

```
println( ... );
```

La acción que realiza la llamada de “println” consiste en mostrar en pantalla el argumento que recibe. No está permitido definir asignaciones como:

```
x= println( ... );
```

La linea anterior da error ya que println no devuelve ningún valor.

Por ello un procedimiento se define y se comporta de la misma forma que una función. La única diferencia es que el tipo retornado por un procedimiento es void. Por ejemplo, el siguiente procedimiento despliega los elementos de un array con valores reales:

```
void mostrar(double[] tab, int primero, int ultimo) {
  int itera= primero;
  while (itera<=ultimo) {
    println(itera+” -> “+tab[itera]);
    itera= itera+1;
  }
  // no hay return
}
```

Ejemplos de uso del procedimiento mostrar son los siguientes:

```
mostrar(tab, 0, 3);
```

Y que lanza el siguiente resultado en pantalla:

```
0 -> 4.5
1 -> 15.0
2 -> 2.2
3 -> 4.3
```

Definimos un procedimiento denominado “captar” que recibe como argumentos un array y el nombre de un fichero con 4 líneas, e inicializa el array con los valores reales contenidos en un archivo. Por ejemplo:

```
double[ ] tabBasica= new double[4];
captar(tabBasica, “resultados.txt”);
mostrar(tabBasica, 0, 3);
```

Tendrá que lanzar el este resultado en pantalla:

```
0 -> 4.5
1 -> 15.0
2 -> 2.2
3 -> 4.3
```

El ejercicio en sí está definido en el siguiente bloque de código.

```
void captar(double[ ] tab, String nombre) {
  TextoLeer lectura= new TextLeer(nombre);
  int i= 0;
  while (true) {
    double x= lectura.readDouble();
    if (lectura.eofReached())
       break;
    tab[i]= x;
    i= i+1;
  }
  lectura.close();
}
```

### 6.3.6. Ámbito de las variables

El alcance de una variable es la región del bloque de código del programa en el cual la variable es accesible.

El ámbito lo define el sitio en el cual se declara la variable.

En Java las variables clasifican por su ámbito en:

#### Variables miembro o atributos de una clase

Estos atributos son los definidos dentro de una clase y fuera de cualquier método.

Sin embargo pueden definirse al principio de la clase, y se pueden declarar en cualquier sitio siempre que sea fuera de un método.

Son accesibles para cualquier método de la clase.

Puede ser inicializada una variable miembro. Si no se les asigna un valor inicial, el compilador les asigna uno por defecto:

– 0 para las numéricas

– '\0' para las de tipo char

– null para String y resto de referencias a objetos.

### Variables locales

Son las declaradas dentro de un método. Su ámbito se inicia en el punto donde se expone la variable.

Están disponibles desde su declaración hasta el final del método donde se exponen. No son visibles desde otros métodos. Distintos métodos de la clase pueden contener variables con el mismo nombre. Se trata de variables distintas. El nombre de una variable local debe ser único dentro de su ámbito.

Si se declara una variable local con el mismo nombre que una variable miembro de la clase, la variable local oculta a la miembro. La variable miembro queda inaccesible en el ámbito de la variable local con el mismo nombre.

Se crean en memoria cuando se declaran y se destruyen cuando finaliza la ejecución del método.

No poseen un valor inicial por defecto. El programador es el encargado de fijarles valores iniciales válidos.

Los parámetros formales son variables locales en el método.

### Variables de bloque

Son las declaradas dentro de un conjunto de instrucciones definidas por llaves { }.

Su ámbito comienza en el punto donde se declara la variable.

Están disponibles desde su declaración hasta el final del bloque donde se declaran.

No son visibles desde otros bloques.

Estas variables permiten que diferentes bloques puedan poseer variables con el mismo nombre. Se trataría de variables distintas.

Si un bloque de instrucciones contiene dentro otro bloque de instrucciones, en el bloque interior no se puede declarar una variable con el mismo nombre que otra del bloque exterior.

Se crean en memoria cuando se declaran y se borran cuando acaba la ejecución del bloque.

No poseen un valor inicial por defecto.

La declaración de una variable miembro surge dentro del cuerpo de la clase, pero externamente del cuerpo de cualquier método de esa clase. Si se define dentro de un método, será una variable local del método y no una variable miembro de la clase. El tipo de una variable define los valores que se le pueden aplicar y las operaciones que se podrían efectuar con la misma.

El nombre de la variable tiene que ser un identificador permitido en Java. Por ello, los programadores Java comienzan a nombrar las variables con una letra minúscula, pero no es imprescindible. Los nombres de las variables no pueden haber dos con el mismo nombre dentro de la clase y se posibilita que haya variables y métodos con el mismo nombre aunque externamente en otros métodos y clases dentro del mismo programa, a diferencia de C++, en donde se produce un error de compilación si se da este caso.

Para los desarrolladores de C++, encuentran diversas discrepancias entre Java y C++ respecto a la sintaxis e interpretación en la definición de variables miembro de una clase, como son:

En los dos lenguajes se posibilita la declaración de variables miembro estáticas con una sintaxis parecida, aunque en C++, se deben declarar estas variables fuera de la definición de la clase usando el nombre de la clase, el nombre de la variable y el correspondiente operador de ámbito.

En ambos lenguajes de programación orientados a objetos, se puede utilizar los modificadores de acceso a las variables miembro con el uso de los términos clave public, private y protected, aunque la sintaxis e interpretación son distintos para cada lenguaje.

Los dos lenguajes reconocen la palabra clave volatile, aunque Java la ignora C++ no reconoce los términos clave transient y final, utilizadas en la declaración de variables miembro en Java. Separadamente de las diferencias de sintaxis anteriores, Java soporta el reseteo de variables miembro de tipos primitivos en el instante de la declaración y esto no es posible en C++.

La sintaxis para la definición de un atributo de clase o variable miembro de una clase en Java es:

```
[especificador_acceso] [static] [final] [transient] [volatile]
    tipo nombreaVariable [= valor_inicial];
```

## Ámbito de una variable

Los conjuntos de sentencias compuestas en Java se demarcan con dos llaves. Las variables de Java exclusivamente son válidas desde el punto donde están declaradas hasta el final de la sentencia compuesta que la engloba. Se deben enlazar estas instrucciones compuestas, y cada una puede poseer su propio grupo de declaraciones de variables locales. Pero no se puede definir una variable con el identico nombre que una de ámbito exterior.

El siguiente ejercicio define dos variables separadas con el mismo nombre. En C y C++ son distintas, porque están declaradas dentro de ámbitos diferentes. En Java, esta declaración no está permitida.

```
class Exterior {
        int variablei = 1;     // ámbito exterior
        {              // crea un nuevo ámbito
        int variablei = 2; // error de compilación
        }
    }
```

## Variables de instancia

La declaración de una variable miembro dentro de la enunciación de una clase sin anteponer la palabra clave static, hace que sea una variable de instancia en todos los objetos de la clase. El significado de variable de instancia sería, más o menos, un objeto instanciado de esa clase posee su misma copia de variable de instancia. Si se inspeccionara la zona de memoria reservada a cada objeto de la clase, se hallaría la reserva efectuada para todas las variables de instancia de la clase. Dicho y explicado de otra manera, como un objeto es una instancia de una clase, y como cada objeto tiene su misma copia de un dato miembro particular de la clase, entonces se puede designar a ese dato miembro como variable de instancia.

En Java, se puede acceder a las variables de instancia vinculadas a un objeto predefinido usando el nombre del objeto, el operador punto (.) y el nombre de la variable:

```
primerObjeto.primerVariableInstancia;
```

Sin embargo En C++, se podría llegar a las variables de instancia de un objeto por medio de la misma sintaxis, además utilizando el operador flecha (->), que usa como origen una variable puntero que enfoca al objeto:

```
primeraVariableDeclarada->primeraVariableInstancia;
```

### Variables estáticas

La declaración de un dato miembro de una clase aplicando static, implanta una variable de clase o variable estática de la clase. El significado de variable estática se entiende como que todas las instancias de la clase (todos los objetos instanciados de la clase) contienen las equivalentes variables de clase o estáticas. Dicho de otra manera, en un instante determinado se puede pretender crear una clase conteniendo y con el valor de una variable de instancia sea el mismo (y de hecho sea la misma variable) para todos los objetos instanciados a partir de esa clase. Necesario que se halle una única copia de la variable de instancia, en ese momento es cuando se usará la palabra clave “static”.

```
class Libro extends Pagina {
        static int pasador = 100;
        }
```

El valor de la variable pasador existirá y tendrá el mismo para cualquier objeto instanciado de la clase Libro. Siempre que un objeto instanciado de Libro cambie la variable pasador, ésta cambiará para todos los objetos.

En C++, hay que declarar nuevamente todas las variables estáticas fuera de la definición de la clase, provocando una variable pseudo–global, una variable que es global para los objetos de esa clase se refiere.

Si se inspeccionara en este caso la franja de memoria reservada por el sistema para cada objeto, se hallaría con que todos los objetos comparten la misma zona de memoria para cada una de las variables estáticas, por ello se llaman también variables de clase, porque son comunes a la clase, a todos los objetos instanciados de la clase.

Tanto en Java como en C++, se podría acceder a las variables de clase usando el nombre de la clase y el nombre de la variable, no hay que instanciar ningún objeto de la clase para acceder a las variables de clase.

En C++, a las variables de clase se accede usando el operador de ámbito unido al nombre de la clase y el nombre de la variable:

```
primeraVariable::primeraVariableClase;
```

En Java, a las variables de clase se accede manejando el nombre de la clase, el nombre de la variable y el operador punto (.). La siguiente línea de código, se usa para acceder a la variable out de la clase System. En el procedimiento, se accede al método println() de la variable de clase que enseña una cadena en el dispositivo estándar de salida con la famosa frase.

```
System.out.println( “Hola, Mundo” );
```




Es necesario acordarse de que todos los objetos de la clase comparten las misma variables de clase, porque si alguno de ellos cambia alguna de esas variables de clase, permanecerán modificadas para todos los objetos de la clase. Esto puede utilizarse como una forma de comunicación entre objetos.

### Constantes

En Java, se utiliza el término clave final para mostrar que una variable debe comportarse como si fuese constante, simbolizando con esto que no se permite su modificación una vez que haya sido declarada e inicializada.

Como es una constante, se le ha de proporcionar un valor en el momento en que se declare, por ejemplo:

```
class ConocePi {
    final float PI = 3.14159;
    . . .
    }
```

Si se pretende modificar el valor de una variable final desde el código del programa, se producirá un error de compilación.

Si se usa la palabra clave final con una variable o clase estática, se permite crear constantes de clase, haciendo de esta manera un uso altamente eficiente de la memoria, porque no se precisarían múltiples copias de las constantes.

El término clave “final” además se podría emplear en métodos, dando lugar en este caso que los métodos no pueden ser sobreescritos.

## 6.3.7. Almacenamiento de las variables

### Variables

Las variables se usan en la desarrollos de Java para almacenar datos que varían durante la ejecución del programa. Para emplear una variable, hay que enseñarle al compilador el tipo y nombre de esa variable, con la declaración de la variable. El tipo de la variable determinará el grupo de valores que se lograrán almacenar en la variable y el tipo de operaciones que se podrán realizar con ella.

Por ello, el tipo int exlusivamente puede poseer números completos enteros. En Java, todas las variables de tipo int contienen valores con signo. En lenguaje C++, por el contrario soporta el uso de tipos enteros con y sin signo todos positivos.

## Expresiones

Los programas en Java, al igual que en C y C++, se componen de sentencias, que a su vez están compuestas en base a expresiones. Una expresión es una determinada combinación de operadores y operandos que se evalúan para obtener un resultado particular. Los operandos pueden ser variables, constantes o llamamientos a métodos.

Una llamada a un método valora el valor devuelto por el método y el tipo de una llamada a un método es el tipo retornado por ese método.

En lenguaje Java se permite declarar constantes con nombre y la forma de construirlas es:

```
final float numPI = 3.14159;
```

En este ejemplo de código se genera un valor que se permtie referenciar en el programa, pero no debe ser modificado. El término clave final es el que evita que esto provoque un error.

En muchos supuestos, el orden en que se realizan las operaciones es concluyente para el resultado que se consiga. Al igual que en C++, Pascal y otros lenguajes, en Java se puede controlar el orden de evaluación de las expresiones mediante el uso de paréntesis. Si no se facilitan estos paréntesis, el orden estará determinado por la precedencia de operadores, de tal modo que las operaciones en las que estén envueltos operadores con más alta precedencia serán los que primero se evalúen.

En C++ los desarrolladores prestarán especial atención, excepto en el caso de la primera cláusula de una instrucción for, Java no soporta el operador coma (,), que en C++ está disponible para controlar el orden en que se evalúan las expresiones.

## Variables miembro

En Java una clase puede poseer variables y métodos. Las variables son de tipos primitivos como int, char…. Los métodos son como funciones.

En el siguiente bloque de código podemos observarlo:

```
public class primeraClase {
      int variablea;

      public primeraMiClase() {
          variablea = 100;
          }
      public void Sumar( int variableb ) {
     // método sumar
          int suma;
          suma = variablea + variableb;
          }
       }
```

La clase primeraClase contiene una variable (variablea) y dos métodos, primeraClase() que es el constructor de la clase y Sumar( int variableb ).

La declaración de una variable miembro aparece dentro del cuerpo de la clase, pero fuera del cuerpo de cualquier método de esa clase. Si se expresa dentro de un métodos, será una variable local del método y no una variable miembro de la clase. En el ejemplo anterior, variablea es una variable miembro de la clase y suma es una variable local del método Sumar().

El tipo de una variable determina los valores que se le pueden asignar y las operaciones que se pueden realizar con ella.

El nombre de una variable ha de ser un identificador válido en Java. Por acuerdo, los programadores Java comienza los nombres de variables con una letra minúscula, pero no es imprescindible. Los nombres de las variables tienen forzosamente que ser únicos, pero se permite que haya variables y métodos con el mismo nombre, a diferencia de C++, en donde se produce un error de compilación si se da este caso.

Para los desarrolladores del lenguaje C++, se podría marcar diferentes divergencias entre Java y C++ de acuerdo a la sintaxis e interpretación en la declaración de variables miembro de una clase, como son:

– En ambos lenguajes se puede declarar una variable miembro estática con un juego de instrucciones parecido aunque en C++, se tienen que declarar estas variables fuera de la definición de la clase utilizando el nombre de la clase, el nombre de la variable y el operador de ámbito.

– En ambos lenguajes se puede utilizar modificadores de acceso a las variables miembro usando los términos clave public, private y protected, con la sintaxis e interpretación diferentes en los dos lenguajes.

– En ambos lenguajes se conocenel término clave volatile, pero en Java se desprecia por desuso.

– C++ no permite las palabras clave transient y final, usadas en la declaración de variables miembro en Java.




Al margen de las diferencias de sintaxis anteriores, Java consiente la inicialización de variables miembro de tipos primitivos en el instante de la declaración y esto no es posible en C++.

La instrución al completo de la definición de una variable miembro de una clase en Java sería:

```
[especificador_acceso] [static] [final] [transient] [volatile]
    tipo nombreVariable [= valor_inicial];
```

## 6.4. Demostración de llamadas a funciones y procedimientos

Los métodos son funciones que pueden ser invocadas dentro de la clase o por otras clases. La implementación de un método consta de dos partes, una declaración y un cuerpo. La declaración en Java de un método se puede expresar sintéticamente como:

```
tipoDevolucion nomMetodo( [listado_argumentos] ) {
    bloqueMetodo
    }
```

En lenguaje C++, el método podría definirse en el interior de la definición de la clase, sin embargo también puede colocarse la declaración completa del método fuera de la clase, constituyéndose en una función inline. En Java, la definición completa del método debe quedar dentro de la definición de la clase y no se admite la posibilidad de métodos inline, por lo tanto, Java no suministra al programador distinciones entre métodos normales y métodos inline.

Los métodos pueden poseer cuantiosos atributos en el momento de declararlos, incluyendo el control de acceso, si es estático o no estático, etc. La sintaxis utilizada para hacer que un método sea estático y su interpretación, es semejante en Java y en C++. Por ello, la estructura usada para aplicar el control de acceso y su interpretación, es muy diferente en Java de C++.

El listado de argumentos no es necesario, tanto en Java como en C++, y en los dos supuestos puede obiarse a su mínima palabra que son dos paréntesis, sin parámetro alguno en su interior. No necesariamente, C++ soporta el término void para reflejar que la lista de argumentos está vacía, en Java no se utiliza. Los parámetros, o argumentos, se usan para enviar información al cuerpo del método. El juego de instrucciones de la declaración de un método, es el que se muestra en el bloque de código fuente con los items opcionales y requeridos:

```
especificadordeAcceso static abstract
    final native synchronized tipoDevolucion nomMetodo( listado_argumentos )
    throws listadoExcepciones
```

– especificadordeAcceso, establece si otros objetos pueden acceder al método y cómo consiguen hacerlo. Está soportado en Java y en C++, pero la sintaxis e interpretación es considerablemente diferente.

– static, sirve para indicarque los métodos serán accedidos sin que sea necesario instanciar un objeto del tipo que define la clase. C++ y Java son iguales en el soporte de esta peculiaridad.

– abstract, sirve para indicar que el método no se define en la clase, sino que se encuentra declarado ahí en una subclase sobreescrito. C++ además permite esta peculiaridad con una sintaxis desigual a Java, pero con análoga interpretación.

– final, sirve para evitar que un método sea sobreescrito.

– native, son métodos diseñados es otro lenguaje. Java perimte en la actualidad C y C++.

– synchronized, se utiliza en el soporte de multihilo tanto de java como de C++.

– listado_argumentos, es la lista no obligatoria de parámetros que se permiten enviar al método.

– throws listadoExcepciones, señala las excepciones que puede crear y manipular el método.

## 6.5. Empleo de llamadas a funciones y procedimientos incluidos en las clases

### Una declaración, innumerables llamadas

Cuando declaramos el método se desarrolla una sola vez en el bloque del código, sin embargo el número de llamadas al método podría ser ilimitado. Por ejemplo, el siguiente programa pretende elaborar una tabla con el valor del factorial de los veinte primeros números naturales:

```
/**
* Demostracion de la funcion factorial
*/
public class PruebaTablaSacaFactorial {
public static void main (String [] args){
for (int k=1; k<=20; k++)
System.out.println(k + "! = " + factorial(k));
}
public static int factorial (int n) {
int aux = 1;
for (int i = 2; i<=n; i++)
aux *= i; // aux = aux * i
return aux;
}
}
```

El resultado de la ejecución del programa anterior por pantalla es el siguiente:

```
$>java PruebaTablaSacaFactorial
1! = 1
2! = 2
3! = 6
4! = 24
```

```
5! = 120
6! = 720
7! = 5040
8! = 40320
9! = 362880
10! = 3628800
11! = 39916800
12! = 479001600
13! = 1932053504
14! = 1278945280
15! = 2004310016
16! = 2004189184
17! = -288522240
18! = -898433024
19! = 109641728
20! = -2102132736
$>
```

Podemos comtemplar que el valor del factorial calculado para enteros superiores a 12 es incorrecto. Las razones es que el valor del factorial de un número crece muy deprisa con el operando.

Así 12! es 479.001.600, aproximándose al límite superior del intervalo para reflejar del tipo primitivo int (2.147.483.647). El factorial de 13 (6.227.020.800) desborda este límite y genera un error de cálculo por desbordamiento "overflow" del intervalo de representación. El problema se puede resolver de nuevo colocando un tipo primitivo con un intervalo de numérico mayor, como puede ser de tipo long:

```
/**
* Nueva demostracion de la funcion factorial
*/
public class PruebaTablaSacaFactorial2 {
public static void main (String [] args) {
for (int k=1; k<=24; k++)
System.out.println(k + "! = " + factorial(k));
}
public static long factorial (int n) {
long aux = 1;
for (int i = 2; i<=n; i++)
aux *= i; // aux = aux * i
return aux;
}
}
```

### 6.5.1. Llamadas calificadas y no calificadas (instancia actual)

Las llamadas calificadas manejan lo que se denomina con la alfabeto del punto(.) porque el receptor de la llamada se identifica explícitamente; es la entidad o la expresión (x) que aparece antes del punto.

Las llamadas no clasificadas no violan la propiedad denominada F2 en el principio de llamada de una característica; al igual que el las llamadas calificadas tiene un receptor el cual es la instancia actual del proceso.

Las llamadas calificadas como es la invocación a métodos o atributos del objeto se efectuan usando el símbolo -> de la flecha:

```
int edad = homero->edad();
obtener e invocar métodos de una clase Java
Enlace publicado el 22 octubre, 2013 | Dejar comentario
```

```
Method[] metodos = primeraClase.getMethods();
```

En lenguaje Java se puede acceder a los métodos de una clase usando lo que se entiende como “Java Reflection”. Desde una clase, accederemos a todos los métodos que posea y estén definidos como “públicos”.

### Como tomar todos los métodos de una clase

Desde un objeto “Class” obtendremos un listado con todos los métodos definidos como públicos de la forma siguiente:

```
Class primeraClase = ..... // Obtenemos la clase
Method[] metodos = primeraClase.getMethods();
```

Con este proceso obtendremos un array que posee la refencia a cada uno de los objetos “Method” definidos como públicos en la clase.

### Obtener un método del que sabemos su nombre

También podríamos pretender conseguir un método específico del que conocemos su nombre. Para ello debemos hacer:

```
Class primeraClase = ..... // obtener la clase
```

De esta manera almacenaremos en “metodo” una referencia al Method “nom_metodo” que está dentro de la clase “primeraClase”.

```
Method metodo = primeraClase.getMethod('nom_metodo');
```

Si por algún motivo el método no existiera en la clase se dispararía un excepción del tipo “NoSuchMethodException”.

### Cómo conocer que tipo de parámetros posee un método

Si tenemos un objeto “Method” cargado, obtendremos que tipo de parámetros tiene utilizando el siguiente método:

```
Method primerMetodo = primeraClase.getMethod(“testeo”);
Class[] parametros = primerMetodo.getParameterTypes();
```

Con esto tendríamos un array con una referencia al tipo de parámetro de cada parámetro que estuviese definido en el método “primerMetodo”.

### Procedimiento para llamar un método

Si poseemos un objeto de tipo “Method” podemos crear una invocación al método que referencia de la siguiente manera:

```
String datosEvalua = “extrae”;
Method primerMetodo = primeraClase.getMethod(“testeo”);
```

### Invacar al método con un parámetro

```
Object result = primerMetodo.invoke(objeto,datosEvalua);
```

De esta manera lo que se haría es una invocación al método “testeo”, que está declarado como público en la clase “primeraClase” y le estaríamos pasando un parámetro “datosEvalua” en la llamada. El método “testeo” se ejecutaría sobre el objeto “objeto” preliminarmente creado, guardando el resultado devuelto por el propio método. Es decir, haciendo esto es como si estuviéramos haciendo:

```
objeto.testeo(datosEvalua);
```

### Llamada al método con N parámetros

Si el método requiriese más parámetros correspondería añadirlos a continuación del objeto, separados por comas, de la siguiente manera:

```
Object& result = primerMetodo.invoke(objeto,parametro1,parametro2,parametro3,parametro4);
```

Que equivaldría a:

```
objeto.testeo(parametro1,parametro2,parametro3,parametro4);
```

Llamada al método sin parámetros

Si el método no recibe ningún parámetro deberemos hacer la invocación de la siguiente forma:

```
Object result = primerMetodo.invoke(objeto);
```

Que sería como poner a:

```
objeto.testeo();
```

El método “invoke” posibilita enviar tantos parámetros como se requieran. El número y tipo de parámetros depende de cómo esté declarado el método al que se está llamando.

## 6.5.2. Paso de parámetros

Más parámetros

Por otro lado, el número de parámetros o argumentos de los métodos puede ser 0, 1, 2… En el siguiente ejemplo, el método producto devuelve el producto de dos valores enteros, a y b, dados como parámetros o argumentos:

```
/**
* Demostracion de la funcion producto
*/
public class MultiplicaClase {
public static void main (String [] args) {
System.out.println("Tabla de multiplicar del 5");
for (int i=0; i<=10; i++)
System.out.println("5 x " + i + " = " + producto(5,i));
}
public static int producto (int a, int b) {
return a*b;
}
}
```

Al compilar e rodar el programa obtendremos un resultado en linea de mandato así:

```
$>java MultiplicaClase
Tabla de multiplicar del 5
5 x 0 = 0
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
```

## Return y void

En muchos momentos, no es necesario que el método estático tenga que retornar un valor al terminar su ejecución. En este supuesto, el tipo de dato que tiene que indicar en la cabecera de declaración del método es el tipo “void” y la sentencia “return” no viene a continuación de ninguna expresión.

Sintaxis:

```
return;
```

En el siguiente código se incluye un ejemplo de método que no devuelve un valor (de tipo void):

```
/**
* Demostracion del metodo tabla
*/
public class MultiplicaClase2 {
public static void main (String [] args){
tabla(4); // poner los parámetros de las distintas tablas a calcular del 4 y del 7
tabla(7);
}
public static void tabla (int n) { // de tipo void
System.out.println(“Tabla de multiplicar del numero :” + n);
for (int i=0; i<=10; i++)
System.out.println(n + “ x “ + i + “ = “ + producto(n,i));
return; // No retorna ningun valor
}
public static int producto (int a, int b) {
return a*b;
}
}
```

Proceso de ejecutar desde linea de mandato y salida correspondiente por pantalla:

```
$>java MultiplicaClase2
Tabla de multiplicar del numero: 4
4 x 0 = 0
4 x 1 = 4
4 x 2 = 8
4 x 3 = 12
4 x 4 = 16
4 x 5 = 20
4 x 6 = 24
4 x 7 = 28
4 x 8 = 32
4 x 9 = 36
4 x 10 = 40
Tabla de multiplicar del numero: 7
7 x 0 = 0
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

Al no haber sentencia return dentro del método, se ejecuta si cortes y sigue hasta el final del método y luego se retorna la secuencia de ejecución al lugar dónde se llamó al método.

### 6.5.3. Los atributos de la clase




Los atributos establecen el estado que tiene un objeto. Estos pueden ser tipo de datos primitivos u objetos de cualquier clase.

Ejemplo:

La clase persona tiene como atributos la altura, el color de ojos y la edad.

Ejemplo en Java:

```
public class Persona {

public int altura;
public String colorDeOjos
public int edad;

}

Utilizacion:

Persona cayetano = new Persona();
mario.altura = 183;
mario.colorDeOjos = “marrones”;
mario.edad = 30;
```

## Atributos de clase frente a atributos de instancia

Los atributos de instancia son adaptables a un solo objeto. Establecen el estado en el que se halla un objeto.

Para explicar mejor la diferencia,hablemos sobre un ejercicio de ejemplo:

El atributo altura en la clase Persona es de instancia, ya que cada persona tendrá su propia altura.

Ejemplo en Java:

```
public class Persona{

public int altura;

}
// Luego:

Persona cayetano = new Persona();
mario.altura = 184;
Persona maria = new Persona();
maria.altura = 152;
```

## Atributos de clase

Los atributos de clase son comunes a todos los objetos de la clase que están instanciados. Al estar definido en la clase no hace falta instanciar la clase para utilizarlo. Las constantes se suelen definir como atributos de clase.

Para explicar mejor la diferencia, hablemos sobre un ejercicio de ejemplo:

El atributo numeroDeOjos en la clase Persona es de clase, debido a que todas las instancias de la clase persona tendrán igual cantidad de ojos.

Ejemplo en Java:

```
public class Persona{

public static int numeroDeOjos = 2;

}

//Luego:

Persona cayetano = new Persona();
Persona maria = new Persona ();
mario.numeroDeOjos                // es igual a 2
maria.numeroDeOjos                // es igual a 2
Persona.numeroDeOjos              // es igual a 2
```

Por ello este 2 es siempre el mismo, no se instancia un numeroDeOjos para cada objeto, siempre es el mismo 2 de numeroDeOjos. Si rectifico y cambiamos esa cantidad de ojos, se actualiza para todas las instancias del objeto:

```
Persona.numeroDeOjos = 3;
```

Entonces:

```
cayetano.numeroDeOjos             // es igual a 3
mario.numeroDeOjos                // es igual a 3
maria.numeroDeOjos                // es igual a 3
```

# UD6
## Lo más importante

- Las medidas permiten detectar tendencias y anticipar problemas y, por lo tanto, permiten establecer un mejor control de las aplicaciones que conlleve a la mejora de la calidad.

- No hay nada mejor en programación que tener un buen hábito de trabajo y de escribir código de manera efectiva, limpia.

- Un buen patrón de codificación cuando se trata de nombres de métodos es el patrón de nombres de métodos revelador de intenciones.

- Los métodos pequeños son tan preferibles como las clases pequeñas y por razones similares.

- Por favor, usa los comentarios. Las personas que siguen luego de usted (o incluso usted mismo,seis meses más adelante) le agradecerán.

- La complejidad es el número de posibles caminos lógicos que tiene cada componente, es decir, las sentencias selectivas (if, case), sentencias iterativas (while, for), lanzamiento y tratamiento de excepciones (throw, catch), etc. A mayor complejidad, un sistema es más difícil de modificar y de probar.

- El código fuente y el estilo que se utiliza en su desarrollo influyen directamente en la eficiencia de la aplicación, puesto que pueden afectar a la velocidad de ejecución de la aplicación, así como al uso de memoria de la misma.

– Bloques sin llaves. Se deberia evitar el uso de bloques if, if..else o for sin llaves (‘{‘). El uso de llaves ayuda a clarificar el codigo y lo hace mas comprensible.

– Variables cortas. Se recomienda evitar el uso de variables con un nombre muy corto, si dichos nombres no aportan un significado al lector en funcion del contexto en el que se declaran.

– La refactorizacion y la habilidad para escribir codigos de prueba son las aptitudes mas importantes para que los programadores nuevos aprendan.

# UD6
# Autoevaluación

1. ¿Por qué medir el software?
    a. Las medidas permiten detectar tendencias y anticipar problemas.
    b. No es necesario ni imprescindible.
    c. Lo importante es terminarlo pronto y rápido.
    d. Java es muy potente y no hay riesgo en ese sentido.

2. Un programa que sea considerado por volumen muy grande (unidad es cada método):
    a. Lineas de código por unidad = 50–100.
    b. Lineas de codigo por unidad: > 100.
    c. Lineas de código por método 10–50.
    d. No es importante el tamaño por unidad.

3. Un programa que sea considerado por complejidad (if, case, while…) tener un riesgo moderado:
    a. Complejidad por unidad: >50.
    b. Complejidad por unidad: (10–20].
    c. No es importante la cantidad de bucles y condicionales que tenga.
    d. Complejidad por unidad: (20–50].

4. ¿La eficiencia en una aplicación en Java medirá?
   a. Que los usuarios sepan manejarlo bien.
   b. Comportamiento temporal, buenos tiempos de respuesta, utilización de recursos.
   c. Que no tenga errores.
   d. Buena programación, rapidez de ejecución, buen precio.

5. ¿Que nombre de método es el más eficiente?
   a. Aa().
   b. Alumnotas_03().
   c. AlumnoNotasTrimestre().
   d. ALUMNOTRIMESTRESUSNOTAS().

6. Import es una palabra reservada, ¿además es un mandato para?
   a. Referencia a otras clases.
   b. Importar un .java dentro del código fuente.
   c. Llamar a métodos.
   d. Especifica que una clase implementa una interfaz.

7. ¿Que tipo de operadores son =, +=, -=, *=, /=?
   a. Operadores de comparación.
   b. Operadores de asignación.
   c. Operadores de signo unitarios.
   d. Operadores de desplazamiento a nivel de bit.

8. ¿Qué término inglés tiene la conversión explícita de tipos en Java?
   a. Casting.
   b. Convert.
   c. Transfor.
   d. Java lo realiza implícitamente.

9. El lenguaje Java soporta las estructuras de control if–else, switch–case:
   a. Toma de decisión.
   b. Bucles.
   c. Miscelánea.
   d. Ninguna de las anteriores.

10. ¿Que son los Parámetros formales?
   a. Son los argumentos que aparecen en la llamada a un método.
   b. Contienen los valores que se le pasan al método.
   c. Son los valores concretos que tienen unos parámetros en un momento dado.
   d. Son los argumentos que aparecen en la cabecera del método.

Área: informática y comunicaciones

# UD7

## Estructura de la información

7.1. Enumeración de datos simples: Numéricos (enteros y reales), lógicos, carácter, cadena de caracteres, puntero o referencia a memoria

7.2. Datos estructurados: arrays

- 7.2.1. Listas enlazadas, pilas y colas
- 7.2.2. Estructuras
- 7.2.3. Ficheros
- 7.2.4. Otras estructuras complejas: tablas hash e introducción a los árboles y grafos

7.3. Mecanismos de gestión de memoria

- 7.3.1. Uso de la gestión automática de memoria
- 7.3.2. Construcción y destrucción de objetos
- 7.3.3. Objetos inalcanzables
- 7.3.4. Recolección de «basura»
- 7.3.5. Métodos constructores y destructores

## 7.1. Enumeración de datos simples: Numéricos (enteros y reales), lógicos, carácter, cadena de caracteres, puntero o referencia a memoria

Java es un lenguaje parecido a C en su sintaxis no obstante ambos pertenecen al grupo que cumplen el paradigma POO. Esta similitud es la mejor herramienta para los programadores, ya que facilita en gran manera su transición a Java. Desgraciadamente, tantos parecidos hacen que no se subsane en algunas discrepancias que son trascendentales. La terminología utilizada en estos lenguajes, a veces es la misma, pero hay grandes diferencias subyacentes en su significado. Java igualmente detalla sus tipos primitivos, elimina cualquier tipo de punteros y tiene tipos referencia mucho más claros.

Todo este maremágnum de terminología provoca cierta consternación, así que los párrafos que siguen pretenden esclarecer lo que realmente significan los términos que se utilizan.

Todos los tipos básicos de datos se conocen ya considerablemente como: datos base, integrados, primitivos e internos; que son parecidos en C, C++ y Java; aunque Java abrevia un poco su uso a los desarrolladores creando que el reconocimiento de tipos sea mucho más rígido. Además, Java añade los tipos boolean y hace indispensable el uso de este tipo booleano en sentencias condicionales.

### Referencias en Java

Las referencias en Java no son punteros como en C++. Son identificadores de instancias de las clases. Es decir que referencia a un objeto de una clase específica. Anda más es necesario saber, por supuesto que hace ni su implementación. Vamos a tener en cuenta en una referencia como si se fuera de la llave electrónica de la habitación de un hotel. Veremos cómo usar en este ejemplo del Hotel para comprobar el uso y la utilización que puede hacerse de las referencias en Java. En primer lugar se diseña la clase Suite, que está implementada en el fichero mediante instancias de la cual se levantará el Hotel:

```
public class Suite {
    private int numSuite;
    private int numCamas;

    public Suite() {
        Suite( 0 );
        }

    public Suite( int numeroHab ) {
        Suite( numeroHab );
        }

    public Suite( int numeroHab,int camas ) {
        Suite( numeroHab );
        camas( camas );
        }

    public synchornized int Suite() {
        return( numSuite );
        }

    public synchronized void Suite( int numeroHab ) {
        numSuite = numeroHab;
        }

    public synchronized int camas() {
        return( camas );
        }

    public syncronized void camas( int camas ) {
        numCamas = camas;
        }
      }
```

El código anterior sería el alma de la aplicación. Ahora se diseña el Hotel creando Habitaciones y estableciéndole a cada una de ellas su llave electrónica; tal como muestra el código siguiente:

```
public class HotelBahia {
    public static void main( String args[] ) {
        Suite llaveHabitac1;              // paso 1
        Suite llaveHabitac2;

        llaveHabitac1 = new Suite( 222 ); // pasos 2, 3, 4 y 5
        llaveHabitac2 = new Suite( 1144,3 );
//      ^^^^^^^^^^   ^^^^^^^^^^^^^^^  ^^^^^^
//          A          B y             D        C
        }
    }
```

Con este código fuente de ejemplo para explicar el proceso, indicar que se dividen las acciones en los cinco pasos necesarios para poder entrar en la habitación del hotel. Aunque no se incluye, se puede también considerar el caso de la necesidad de un cerrajero, para que ante la pérdida de la llave se pueda abrir la puerta; y que, en el caso particular de este hotel virtual Java, sería el garbage collector, que recicla la habitación una vez que se hayan perdido todas las llaves. Es solo un mero matiz realista para hacerlo más completo.

El inicio está en crear la llave, definir la variable referencia, por defecto nula.

Los otros pasos se definen en una sola instrucción Java. La estructura B en el código fuente indica al Director del Hotel que ya dispone de una nueva habitación. La estructura C invoca al decorador para que comprobada la habitación según unos patrones determinados, para que no desentonen unas habitaciones con otras y no se cambien las señas de imagen del hotel. El cifrado electrónico que posibilita entrar en la habitación se genera en la estructura D, una vez conocido el interior de la habitación y ya se programa en la llave en la estructura A.

Si se abandonamos por un instante el ejemplo real y nos detenemos en lo que ocurre en la ejecución del código, se observa que el operador new busca espacio para una instancia de un objeto de una clase predefinida y setea la memoria a los valores adecuados. A continuación llama al método constructor de la clase, pasándole los argumentos necesarios. El operador new retorna una referencia a sí mismo, que es directamente asignada a la variable referencia.

Puede haber distintas llaves para una misma habitación:

```
. . .
Suite llaveHabitac3, llaveHabitac4;
llaveHabitac3 = llaveHabitac1;
llaveHabitac4 = llavehabitac2;
```

Así se logran copias de las llaves. Las habitaciones en sí mismas no se han tocado en este proceso. Así que, ya hay dos llaves para la habitación 342 y otras dos para la habitación 364.

Una llave podrá ser programada para que abra la puerta de una habitación concreta 342 por ejemplo, y podemos cambiar el código de apertura electrónica y así poder abrir otras como la 364. De esta manera cambiaremos la habitación anterior de una pareja de novios (342) por otra suite del ático (364).

```
llaveHabitac3 = llaveHabitac2;
```

Ahora hay una llave para la habitación 342 y tres para la habitación 364; manteniendo una llave para cada habitación en la conserjería, para poder utilizarla como llave maestra, en el caso de que alguien pierda su llave propia.

Cualquier persona con la llave de una habitación puede hacer cambios en ella, y las camareras que tengan llave de esa misma habitación, no sabrán que se ha cambiado hasta que vuelvan a entrar en la habitación. Por ejemplo, si que quita una de las camas de la habitación, entrando en ella con la llave maestra:

```
. . .
llaveHabitac2.camas( 2 );
```

Ahora cuando los inquilinos entren en la habitación podrán comprobar el cambio realizado:

```
. . .
llaveHabitac4.printData();
```

### Referencias y arrays

Java permite trabajar con arrays de tipos primitivos o de clases. En Java tiene un privilegio bastante importante y se usan muchísimo. Por ejemplo en el caso anterior de las llaves del Hotel.

Para ampliar el hotel construyendo una planta de habitaciones, se creará un juego de llaves maestras y luego se construirán las habitaciones:

```
public class HotelBahia2 {
    // Número de habitaciones por pasillo
    public static final int habPorPasillo = 12;

    public static void main( String args[] ) {
        Suite llavesMaestras[];                 // paso 1
        llavesMaestras = new Suite[ habPorPasillo ]; // pasos 2-5
//      ^^^^^^^^^^^^^^^   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
//          A                 B,      C,          D y E
```

```
        int numPiso = 1;
        for( int i=0; i < habPorPasillo; i++ )        // pasos 6-9
            llavesMaestras[ i ] = new Suite( numPiso * 100 + i,
                ( 0 == (i%2)) ? 2 : 1 );
        for( int i=0; i < habPorPasillo; i++ )        // pasos 10-11
            llavesMaestras[i].printData();
        }
    }
```

Cada avance en el ejercicio es similar al que ya se ha visto antes. El paso 1 indica que el juego de llaves maestras es un montón de llaves de habitaciones.

Los pasos 2 a 5 son, en este caso, la parte principal. En lugar de crear una habitación, el Director decide construir un grupo contiguo de habitaciones. El número de llaves se detalla entre corchetes y todas se crean en blanco.

Los pasos 6 a 9 son semejantes a los pasos 2 a 5 del ejemplo anterior, exclusive en este caso todas las llaves pasan a integrar parte del juego maestro. Los números de piso se dan en miles para que cuando se creen las habitaciones, todas posean el mismo formato. Además la mayoría de las Suites de número par poseen una sola cama, mientras que las Suites impares tendrán dos camas.

Los pasos 10 y 11 permiten obtener información de cada una de las habitaciones del hotel.

## Referencias y listas

Entre estudiantes se piensa que como Java no dispone de punteros, resulta exagerado de complejo el construir listas enlazadas, árboles binarios y grafos. Java dispone de distribuciones de datos de esos tipos y además suministra otras muchas como: mapas, conjuntos, tablas Hash, diccionarios, etc., que en el JDK 1.2 están muy bien diseñadas y siguen la nomenclatura al uso de la OOP. No obstante, en los párrafos siguientes se explica que quien así piensa está bastante equivocado, porque incluso de forma ramplón se pueden construir estas estructuras de datos sin demasiado ardor.

El paquete de la lista simple se dispone de dos clases. Cada componente de la lista es un NodoListaSuite, LlaveHotelMaestra.java:

```
public class LlaveHotelMaestra {
    private N LlaveHotelMaestra siguiente;
    private LlaveHotelMaestra anterior;
    private Object datos;
    // . . .
    }
```

Cada LlaveHotelMaestra contiene una referencia a su nodo preliminar en la lista y una referencia al nodo que le sigue. Además contiene una referencia genérica a cualquier clase que se use para facilitar acceso a los datos que el usuario proporcione.

La lista enlazada, LlaveHotelMaestra.java, contiene un nodo principio–fin y un contador para el número de nodos en la lista:

```
public class LlaveHotelMaestra {
    public LlaveHotelMaestra PrincipioFin;
    private int numNodos;
    // . . .
    }
```

El nodo especial PrincipioFin es sencillo, para abreviar el código. El contador se usa para perfeccionar los casos más habituales.

Hay que revisar pues el código del HotelBahia3, que será prácticamente el mismo que en el caso de los arrays:

```
public class HotelBahia3 {
    // Número de habitaciones por Pasillo
    public static final int habPorPasillo = 12;

    public static void main( String args[] ) {
        ListaEnlazada llaveHotelMaestra;          // paso 1
        llaveHotelMaestra = new ListaEnlazada();  // pasos 2-5

        int numPiso = 1;
        for( int i=0; i < habPorPasillo; i++ ) // pasos 6-9
            llaveHotelMaestra.insertAt( i,
                new Suite( numPiso * 100 + i,
                ( 0 == (i%2)) ? 2 : 1 );
        for( int i=0; i < habPorPasillo; i++ ) // pasos 10-12
            ( (Suite)llaveHotelMaestra.getAt(i) ).printData();
        }
    }
```

Exponiendo el código fuente anterior, hay que comentar que el paso 1 es la llave maestra de la lista. Está concretando con una lista genérica; es decir, una lista de llaves que cumple la convención que se ha establecido. Se podría apresurar el tiempo de compilación metiendo la lista genérica ListaEnlazada dentro de una ListaEnlazadaSuite.

Los pasos 2 a 5 son parejos a los del primer ejercicio. Se crean y se setea una nueva ListaEnlazada, que se usará como juego de llaves maestras.

Los pasos 6 a 9 son similares a los del ejercicio anterior con arrays, pero con distinta sintaxis. En Java, los arrays y el operador [ ] son internos del lenguaje. Como Java no permite la sobrecarga de operadores por medio del usuario, solo puede utilizarse en su forma normal.

La ListaEnlazada aporta el método insertAt() que recoge el índice en la lista, donde el nuevo nodo ha de ser fijado, como argumento primero. El segundo argumento es el objeto que será almacenado en la lista. Los pasos 10 a 12 provocan la misma salida que los pasos 10 y 11 del ejemplo con arrays. El paso 10 recoge la llave del juego que se fija en el método getAt(). Ahora, el sistema desconoce qué datos contiene la llave, ya que el contenido de la habitación es genérico. Pero el programa sí sabe lo que hay en la lista, así que avisa al sistema haciendo un moldeado a la llave de la habitación este casting creará un chequeo en tiempo de ejecución por parte del compilador, para certificarse de que se trata de una Suite. El paso 12 usa la llave para imprimir la información.

## Tipos de datos simples




Los tipos de datos en Java y en otros lenguaje de programación, constituyen como los métodos de almacenamiento disponibles para representar información, además de la manera en que dicha información será interpretada.

Para definir una variable en memoria deberá expresarse mostrando su tipo de variable y su identificación que la identificará de forma única. La sintaxis para declarar variables es la siguiente:

```
TipoVariableSimple Identific1, Identific2;
```

Esta sentencia muestra al compilador que guarde memoria para dos variables del tipo simple TipVariableSimple con nombres, identific1 e identific2.

Los tipos de datos en Java se fragmentarse en dos categorías: "simples y compuestos". Los simples son tipos nucleares que no se derivan de otros tipos, como los enteros, de coma flotante, booleanos y de carácter. Los tipos compuestos se apoyan en los tipos simples, e incluyen las cadenas, las matrices y tanto las clases como las interfaces, en general.

Cada tipo de datos simple aguanta un conjunto de literales que le pueden ser fijados, para darles valor. En este apartado se explican los tipos de datos simples (o primitivos) que presenta Java, así como los literales que soporta (sintaxis de los valores que se les puede asignar).

### Tipos de datos enteros

Son utilizados para especificar números enteros con signo. Hay cuatro tipos:

- Byte → 1 Byte (8 bits)
- Short → 2 bytes (16 bits)
- int → 4 bytes (32 bits)
- long → 8 bytes (64 bits)

### Literales enteros

Son elementales en la programación en Java y muestran tres formatos:

- Decimal: Los literales decimales aparecen como números frecuentes sin ninguna anotación especial.
- Hexadecimal: Los hexadecimales surgen con un 0x o 0X inicial, notación parecida a la utilizada en C y C++.
- Octal: Los octales se muestran con un 0 inicial en primer lugar delante de los dígitos. Por ejemplo, un literal entero para el número decimal 12 se refleja en Java como 12 en decimal, como 0xC en hexadecimal, y como 014 en octal.

Los literales enteros se acumulan por defecto en el tipo int 4 bytes con signo, o si se programa con números muy grandes, con el tipo long 8 bytes con signo, añadiendo una L o l al final del número.

La declaración de variables enteras es muy natural. Un ejemplo de ello sería:

```
long numeroLargo = 0xC; // Por defecto vale 12
```

### Tipos de datos en coma flotante

Se emplean para simbolizar números con fracciones. Hay dos tipos de coma flotante: float y double. El primero reserva almacenamiento para un número de precisión simple de 4 bytes y el segundo lo hace para un numero de precisión doble de 8 bytes.

- Tipo float → 4 Byte (32 bits)
- Tipo double →8 Bytes (64 bits)

### Literales en coma flotante

Simbolizan números decimales con partes fraccionarias. Aparecen representados con notación estándar (751,48) o científica (2.34577e2).

De manera predefinida son del tipo double de 8 bytes. Se conoce la opción de emplear un tipo más reducido el tipo float de 4 bytes, indicándolo con una F o f al final del número.

La presentación de variables de coma flotante es muy parecida a la de las variables enteras.

Por ejemplo:

```
double numeroPi = 314.16e-2 ; // Aproximadamente
float termometroTemp = (float)36.6; // Paciente sin fiebre
```

Se efectúa un moldeado a temperatura, porque todos los literales con decimales por defecto se consideran double.

## Tipos de datos boolean

Se usa para acumular variables que simbolizan dos estados, que serán descritos por los valores true y false. Representan valores biestado, descendientes de la denominada "Algebra del bool".

- **Literales booleanos**

  Java emplea dos términos diferentes para cada estado posible: "true" para Verdadero y "false" para Falso. En lenguajes como C o C++ el valor de falso se representa con un 0 numérico, y verdadero el número 1. Para expresar un dato del tipo booleano se emplea el término reservado "boolean":

```
boolean facturaPagada = true; //
```

- **Tipos de datos carácter**

  Se emplea para guardar símbolos Unicode simples. Como el grupo de caracteres Unicode contiene valores de 16 bits, el tipo de dato char se guarda en un entero sin signo de 16 bits.

  Java distingue entre matrices de caracteres y cadenas sin embargo esto no ocurre en C++.

- **Literales carácter**

  Simbolizan un solo carácter de la tabla de signos Unicode 1.1 y se muestran dentro de un par de comillas simples. De forma similar que en C/C++. Los caracteres especiales de control y no imprimibles se muestran con una barra invertida ('\') seguida del código carácter.

| Descripción | Representación | Valor Unicode |
|---|---|---|
| **Caracter Unicode** | \udddd | |
| **Numero octal** | \ddd | |
| **Barra invertida** | \\ | \u005C |
| **Continuación** | \ | \ |

| Descripción | Representación | Valor Unicode |
|---|---|---|
| Retroceso | \b | \u0008 |
| Retorno de carro | \r | \u000D |
| Alimentación de formularios | \f | \u000C |
| Tabulación horizontal | \t | \0009 |
| Línea nueva | \n | \u000A |
| Comillas simples | \’ | \u0027 |
| Comillas dobles | \” | \u0022 |
| Números arábigos ASCII | 0–9 | \u0030 a \u0039 |
| Alfabeto ASCII en mayúsculas | A–Z | \u0041 a \u005A |
| Alfabeto ASCII en minúsculas | a–z | \u0061 a \u007A |

Las variables de tipo “char” se definen de la siguiente manera:

```
char letraMayuscula = ’G’; // Importante poner las ’ ’
char letraW = ’\u0057’; // Letra ’W’
```

## Conversión de tipos de datos

En Java se permite convertir el tipo de una variable ú objeto en otro distinto al original con el que fue definido. Este proceso se denomina «conversión», «moldeado» o «tipado»._La conversión se lleva a cabo poniendo el tipo destino entre paréntesis, a la izquierda del valor que vamos a convertir de la manera siguiente:

```
boolean facturaPagada = true; //
```

La función read retorna un valor int, que se convierte en un char debido a la conversión (char), y el valor resultante se acumula en la variable de tipo carácter f. El tamaño de los tipos que queremos convertir es muy importante. No todos los tipos se convertirán de manera positiva. Por ejemplo, al convertir un long en un int, el compilador borra los 32 bits superiores del long de 64 bits, de forma que acoplen en los 32 bits del int, con lo que si tienen información valiosa, esta se perderá.

Por ello se crea la norma de que "en las conversiones el tipo destino siempre debe ser igual o mayor que el tipo fuente":

| Tipo origen | Tipo destino |
|---|---|
| **byte** | Double, float, long, int, char, short |
| **short** | Double, float, long, int |
| **char** | Double, flaot, long, int |
| **long** | Double, float |
| **float** | double |

Las principales características que Java no hereda de C++ son:

– Punteros: Las direcciones de memoria son la característica más fuerte de C++. El improcedente uso de los punteros provoca la mayoría de los errores de colisión de memoria, errores muy difíciles de detectar. Además, casi todos los virus que se han escrito aprovechan la capacidad de un programa para acceder a la memoria volátil (RAM) utilizando punteros. En Java, no existen punteros, evitando el acceso directo a la memoria volátil.

– Asignación de memoria: Con la función "malloc" de C, se aplica un número especificado de bytes de memoria retornando la dirección de ese conjunto. Con la función "free" se devuelve un bloque establecido al sistema para que lo use. Si por casualidad se olvida de invocar a free para liberar un bloque de memoria, se están limitando los recursos del sistema, ralentizando progresivamente los programas. Si por otro lado realizamos un free en un puntero ya libre, ocurrirá algo inesperado. Más tarde C++ añadió new y delete, que se usan de forma análoga, siendo todavía el programador, el responsable de liberar el espacio de memoria. Java no tiene funciones malloc ni free.

El operador new se usa para crear un espacio de memoria para un objeto. Con new no se obtiene una dirección de memoria en lugar de llo se obtiene un descriptor al objeto del bloque. La memoria real asignada a ese objeto se podría mover a la vez que el programa está funcionando, pero sin necesidad que preocuparse de ello.

En el supuesto que no haya referencia alguna de ningún objeto, la memoria llenada queda disponible para que la reutilice el resto del sistema sin necesidad de llamar a free o delete. A esto se le denomina como recogida de basura. El recolector de basura rueda siempre que el sistema esté libre, o cuando una asignación solicitada no encuentre asignación suficiente.

– Conversión de tipos insegura: Los maleables de tipo "casting" son un mecanismo importante de C y C++ que posibilitan modificar el tipo de un puntero. Esto necesita extremo cuidado puesto que no hay nada previsible para saber si la conversión es correcta en tiempo de ejecución. En Java se puede hacer una examinación en tiempo de ejecución de la compatibilidad de tipos y enviar una excepción cuando falla.

## 7.2. Datos estructurados: arrays

El lenguaje Java, a diferencia de C++, tiene un tipo array. En C++, un array es simplemente un conjunto de posiciones secuenciales de memoria a las que se puede acceder mediante un índice o punteros. Este proceso no acarrea ningún control sobre ese conjunto de posiciones de memoria.

En Java, al ser un tipo de datos verdadero, se dispone de argumentos exhaustivos del correcto manejo del array; por ejemplo, de la comprobación de sobrepasar los términos definidos para el array, en evitación de desbordamiento o corrupción de memoria.




En Java hay que declarar un array antes de poder usarlo. Y en la declaración hay que insertar el nombre del array, y el tipo de datos que se van a almacenar en él.

La sintaxis general para definir e instanciar un array es:

```
tipoDeComponentes[] nomArray = new  tipoDeComponentes[tamañoDelArray];
```

Se pueden declarar en Java arrays de cualquier tipo:

```
char calendario[];
int mesArray[];
```

Incluso se pueden construir arrays de arrays:

```
int tabla[][] = new int[7][9];
```

La definición de un array no localiza, como pasa con los demás objetos en Java , memoria para albergar los datos. En su puesto, únicamente localiza memoria para almacenar una referencia al array. La memoria necesaria para almacenar los datos que forman el array se buscará en memoria dinámica a la hora de instanciar y crear realmente el array.

Para crear un array en Java hay dos métodos básicos. Crear un array vacío:

```
int calendar[] = new int[50];
```

También se puede construir ya el array con valores desde el comienzo:

```
String nombres[] = {
    “Lucas”,”Pepe”,”Santi”,”Mario”
    };
```

Esto que es equivalente a:

```
String alumno[];
nombre = new String[4];
alumno [0] = new String( “Lucas” );
alumno [1] = new String( “Pepe” );
alumno [2] = new String( “Santi” );
alumno [3] = new String( “Mario );
```

En Java no se permite crear arrays estáticos en tiempo de compilación:

```
int unidades[34]; // dará un error en tiempo
                  // de compilación
```

No se puede cubrir un array sin especificar el tamaño con el operador new:

```
int unidades[];
for( int a=0; a < 235; a++ )
    lista[a] = a;
```

Con las sentencias anteriores paralelamente se declara el nombre del array y se reserva memoria para contener sus datos. Pero no es necesario combinar estos procesos. Se puede establecer la sentencia de declaración del array y posteriormente, otra sentencia para asignar valores a los datos.




En Java en el momento que se instancia un array, podemos tener acceso a los elementos de ese array empleando un índice, de forma parecida a la que podemos acceder en otros lenguajes de programación. No obstante, Java no soporta el acceder a los elementos de un array utilizando punteros.

```
    unidadesPalet[5] = 6;
    miVariable = unidadesPalets[5];
```

Como en C y C++, los índices de un array siempre empiezan en 0.

Todos los arrays en Java están compuestos de una función miembro como es length(), que se puede usar para extraer la longitud del array.

```
int unidades[][] = new int[10][3];
unidades.length;    /* 10 */
unidades[0].length; /* 3 */
```

Además, en Java no se puede completar un array sin expresar el tamaño con el operador new. El código que se muestra no sería válido:

```
int unidades[];
for( int a=0; a < 9; a++ )
    unidades[a] =a;
```

En lenguaje Java, todos los arrays son estáticos. Para convertir un array en el equivalente a un array dinámico en C/C++, se usa la clase Vector, que permite operaciones de inserción, borrado, etc. en el array.

Otro aspecto a tratar sobre esta unidad es de que Java puede crear arrays multidimensionales, que se distinguirían como arrays de arrays. Aunque esta es una característica común a muchos lenguajes de programación, en Java, por el contrario, los arrays secundarios no precisan ser todos del mismo tamaño. En el ejemplo, se define e instancia un array de dígitos con dos dimensiones, con un tamaño de inicio de la primera dimensión de 3. Mientras que las casillas de las dimensiones secundarias de cada uno de los arrays es 2, 3 y 4, individualmente.

También se pueden observar dentro de los arrays, como es el hecho de que cuando se define un array de dos dimensiones, no es forzoso reflejar el tamaño de la dimensión secundaria a la hora de la definición del array, sino que puede declararse posteriormente el tamaño de cada uno de los subarrays. Además observamos que el efecto de entrar a elementos que se hayan fuera de los límites del array; Java blinda el programa contra este tipo de errores de programación.

En C / C++, si se pretende acceder y guardar datos en un componente del array que está fuera de límites, se consigue, que los datos se guarden en una porción de memoria que no sea parte del array. En Java, si se pretende realizar esto, el sistema creará una excepción, tal como se vé en el ejemplo. La excepción únicamente hace que el programa finalice, pero se podría tomar esa excepción e implementar un controlador de excepciones "exception handler" para rectificar automáticamente el error, sin que sea necesario frenar la ejecución del programa.

La ejecución del ejemplo dará esta salida por consola:

```
%java javaDemoArray
00
012
0246
Acceso a un componente fuera de limites
java.lang.ArrayIndexOutOfBoundsException: at javaDemoArray.main(javaDemoArray)
```

Explicando otros usos de arrays en Java. Se trata, en este caso, de que Java permite la asignación de un array a otro. Igualmente, hay que extremar las precauciones cuando se hace esto, porque lo que en realidad está sucediendo es que se está creando una copia de la referencia a los mismos datos en memoria. Y, poseer dos referencias a los mismos datos no puede ser una buena idea, porque los resultados pueden desorientar. La salida por consola que se realiza tras la ejecución del ejemplo es:

```
%java javaDemoArray
Resultado del primerArray
0 1 2
Resultado del segundoArray
0 1 2
--> Cambiamos un valor en el primerArray
Resultado del primerArray
0 10 2
Resultado del segundoArray
0 10 2
```

## Los array como almacén de datos

Hay diversas características que difieren a los arrays de otras colecciones pero sin duda hay dos que las diferencian de cualquier tipo: eficiencia y tipo. El array es la forma más eficiente que Java suministra para almacenar y acceder a una secuencia de objetos. El array es una escueta secuencia lineal, que hace que el acceso a los componentes sea muy rápido, aunque hay que pagar un alto precio por esta velocidad y es que en el momento se crea un array su tamaño es fijado y no se puede modificar en toda la vida del objeto.




Se puede concretar la construcción de un array de tamaño predeterminado y luego, ya en tiempo de ejecución, crear otro más grande, mover todos los objetos al nuevo y eliminar el obsoleto. Esto es lo que hace la clase Vectorpero debido a la carga que supone esta flexibilidad, un Vector es menos eficiente que un array, en cuestiones de velocidad.

La clase vector en C++ desconoce el tipo de los objetos que posee, y tiene un inconveniente cuando se la compara con los arrays en Java: el operador [] de la clase vector de C++ no realiza un chequeo de límites, así que uno se puede pasar del final (aunque es posible saber el tamaño del vector invocando al método at() para realizar la comprobación de límites, si se desea). En Java, independientemente de que se esté utilizando un array o una colección, siempre hay comprobación de límites, y el sistema lanzará una excepción en caso de que se intente el acceso a un elemento que se encuentra fuera de los límites. Así pues, la razón de que C++ no compruebe límites es la velocidad, y con esta sobrecarga hay que vivir siempre en Java, porque todas las veces realizará la comprobación de que no se acceda a un lugar fuera del array o la colección.

Los otros formatos de colecciones útiles en Java: “Vector, Stack y Hashtable”; podrían contener diferentes tipo de objeto, sin que sea necesario de que sea de un tipo definido. Esto es así porque disponen a sus elementos como si fueran Object, la clase raíz de todas las clases Java. Esto es bueno desde la óptica de que se diseña solamente un solo repertorio, y cualquier tipo de objeto puede ser

almacenado en el. Pero aquí es donde los arrays vuelven a ser más eficientes que las colecciones genéricas, porque cuando se crea un array hay que indicar el tipo de objetos que va a contener. Esto quiere decir que ya en tiempo de compilación se procesan chequeos para que no se guarde en el array ningún objeto de tipo distinto al que está destinado a contener, ni que se pueda extraer un objeto distinto. Por ello, Java controla de que no se mande un mensaje inapropiado a un objeto, ya sea en tiempo de compilación como en tiempo de ejecución.

Igualmente, tanto por eficacia como por razonamiento de tipos, es mejor utilizar un array siempre que se pueda. Sin embargo, cuando se trata de resolver un problema más general, los arrays pueden ser muy prohibitivos y es entonces cuando hay que solicitar a otro tipo de almacenamiento de datos.

Las colecciones de clases manipulan solamente los identificadores, handles, de los objetos. Un array, por lo tanto, puede crearse para contener tipos básicos directamente, o también identificadores de objetos. Es posible utilizar las clases correspondientes a los tipos básicos, como son Integer, Double, etc., para poner tipos básicos internamente en una colección. El poner una cosa ú otra es cuestión de eficacia, porque es mucho más rápida la creación y acceso en un array de tipos básicos que en uno de objetos del tipo básico.

Desde luego, si se está utilizando un tipo básico y se necesita la elasticidad que brinda una colección de expandirse cuando sea preciso, el array no sirve y habrá que recurrir a la colección de objetos del tipo básico. Se puede creer que un vector especializado en cada uno de los tipos básicos puede ser igual de eficiente que un array, pero no es así, Java no permite mas que un tipo de genérico de vector, en el que se puede insertar de todo.

Los arrays en Java (vectores, matrices, hipermatrices de más de dos dimensiones) se tratan como objetos de una clase predestinada. Los arrays son objetos, pero con ciertas características propias.

Los arrays podrían asignarse a objetos de la clase Object y los métodos de Object podrían utilizarse con arrays.

Diferentes características importantes de los arrays son:

– Los arrays se diseñan con el operador new luego el tipo y número de componentes.

– Podemos acceder al número de elementos de un array con la variable miembro expresada length como ejemplo, agenda.length.

– Se llega a las casillas de un array con los corchetes [] y un índice que varía de 0 a length-1.

– Se permiten construir arrays de objetos de distintos tipos. Un array de objetos es un array de referencias que hay que comenzar llamando al operador new.

- Los componentes de un array se resetean al valor por defecto del tipo correspondiente. Como cero para datos numéricos, false para boolean, el carácter nulo para char, null para Strings y para referencias.
- Como todos los objetos, los arrays se envian como argumentos a los métodos por referencia.
- Se podrían definir arrays anónimos por ejemplo, crear un array como argumento actual en la invocación a un método.

Pasos en la Inicialización de arrays:

- Los arrays se inicializan con valores entre llaves {...} separados por comas.
- Además los arrays de objetos se inicializan con varias invocaciones a new dentro de unas llaves {...}.
- Si se asemejan dos referencias a un array no se copia el array, sino que se posee un array con dos nombres, enfocando al mismo y único objeto.
- Diseño de una referencia a un array. Se puede de dos formas:

```
double[] x; // recomendable
double x[];
```

- Crea el array con el operador new:

```
mes = new double[31];
```

- Las dos fases d y e se pueden unir en una sola:

```
double[] mes = new double[31];
```

Luego se presentan algunos ejemplos de creación de arrays:

```
// crear un array de 31 dígitos, que desde el principio se inicializa a cero

int notas[] = new int[10];

// crea arrays inicializado con disitntos datos

int notas[] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
String notas[] = {“Muydeficiente”, “Deficiente”, “Suficiente”, “Bien”, “Notable”, “Sobresaliene”, “Matricula”};

// array de 31 objetos

Calendario listaDias[] = new Calendario[31]; // de momento hay 25 referencias a null

for( i = 0 ; i < 31;i++)
listaDias[i] = new Calendario(...);
// array sin nombrar
obj.metodo(new String[]={“Lunes01”, “Martes02”, “Miercoles03”});
```

## Arrays bidimensionales

En Java los arrays bidimensionales se crean de un modo muy similar al de C++ (con reserva dinámica de memoria). En Java una matriz es un vector de vectores fila, o más preciso un vector de referencias a los vectores fila. Con este boceto, cada fila podría tener un número de elementos diferente.

Una matriz se puede construir directamente en la forma, int [ ][ ] aula = new int[3] [4]; o bien podriamos crearla de modo dinámico facilitando los siguientes pasos:

– Crea la referencia marcando con un doble corchete que es una referencia a matriz, int[ ][ ] mat;

– Crea el vector de referencia a las filas, aula = new int[nfilas][];

– Reserva memoria para los vectores correspondientes a las filas, for (int i=0; i<nfilas; i++);

```
mat[i] = new int[numerocolums];
// crear una matriz 6x6
// se inicializan a cero
double mat[][] = new double[6][6];
int [ ][ ] b = {{1, 2, 3},
{4, 5, 6}, };
int curso = new[3][]; // se crea el array de referencias a arrays
curso[0] = new int[5];
curso[1] = new int[4];
curso[2] = new int[8];
```

En el caso de una matriz bienes, bienes.length es el número de filas y bienes[6]. length es el número de columnas (de la fila 0). Los arrays bidimensionales pueden contener tipos primitivos de cualquier tipo ú objetos de cualquier clase.

### Suma de dos matrices cuadradas

Al sumar dos matrices de iguales dimensiones, se consigue otra matriz en la que sus componentes son las sumas de los correspondientes elementos de las matrices estante1 y estante2, es decir:

```
armarioij=estante1ij+estante2ij
```

En primer lugar, se crea una función miembro estática denominada suma. Interiormente, se crea una matriz temporal total, con la misma dimensión de las matrices que se sumarán en la operación, y se almacenan en sus elementos el resultado de la suma de las matrices estante1 y estante2. Por último, la función suma retorna la matriz resultado.

```
public static Matriz suma(Matriz estante1, Matriz estante2){
    Matriz resultado=new Matriz(a.n);
    for(int i=0; i< a.n; i++){
        for(int j=0; j< a.n; j++){
            resultado.x[i][j]=estante1.x[i][j]+ estante2.x[i][j];
        }
    }
    return resultado;
}
```

Observamos ahora como se invoca a la función que suma dos matrices.

```
    double[][]estante1={{1, 2, 3},{4,5,6},{7,8,9}};
    Matriz estante1=new Matriz(estante1);
    double[][]estante2={{1, 0, -1},{2,1,3},{-1, 0, 2}};
    Matriz estante2=new Matriz(estante2);
    Matriz result=Matriz.suma(estante1, estante2);
    System.out.println("matriz "+result);
```

## Producto de dos matrices

Ahora veremos que para multiplicar dos matrices es bastante más complicado que sumar dos matrices de las mismas dimensiones. Se pueden multiplicar dos matrices de dimensiones “m x n” y “n x q”, dando como resultado una matriz de dimensiones “m x q”. En este apartado nos ceñiremos exclusivamente a matrices cuadradas de dimensión n.

Los componentes clm se logran multiplicando los elementos alm de la fila l por los elementos anm de la columna m, y sumando los resultados.

La codificación se realiza empleando un tripe bucle for, guardando en los elementos de la matriz local resultado la suma de los productos de la fórmula anterior.

```
public static Matriz multiplicar(Matriz estante1, Matriz estante2){
   Matriz resultado=new Matriz(a.n);
   for(int l=0; l<a.n; l++){
      for(int m=0; m<a.n; m++){
         for(int n=0; n<a.n; n++){
            resultado.x[l][m]+= estante1.x[m][n]* estante2.x[n][m];
          }
       }
    }
   return resultado;
}
```

### 7.2.1. Listas enlazadas, pilas y colas

Hay diferentes diseños de List, siendo ArrayList la que debe ser la elección por defecto, en caso de no tener que usar las peculiaridades que presentan las demás implementaciones.

#### List (interfaz)

El ordenamiento es la faceta más importante de una Lista, consiguiendo que los elementos siempre se mantengan en una secuencia concreta. La Lista añade una serie de métodos a la colección que posibilitan la inserción y eliminar elementos en medio de la Lista. En la Lista Enlazada se podría crear un "ListIterator" para moverse entre los componentes en ambas direcciones.

#### ArrayList

Es una Lista creada en un Array. Se usa en lugar de vector como almacén de objetos de tipo general. Es posible acceder aleatoriamente y muy rápido a los elementos, sin embargo realiza con mucha lentitud las operaciones de insertar y borrar elementos de la Lista. Se usa un "ListIterator" para movernos hacia atrás y hacia delante en la Lista, pero no para insertar y borrar componentes.

#### LinkedList

Facilita un buen acceso en serie, permite insertar y borrar elementos de la Lista rápidamente. Pero es muy tardio el acceso aleatorio, si lo comparamos con ArrayList. Posee además de los métodos getFirst(), getLast(), addLast(), removeFirst() y removeLast(), sin definir en ningún interfaz o clase base y que posibilitan usar la ListaEnlazada como una Pila, una Cola o una Cola Doble.

Este mandato soporta mayoría de las acciones que se efectúan en las Listas, como moverse con un Iterator, cambiar componentes, ver los efectos del manejo de la Lista y efectuar operaciones sólo posibles a las Listas Enlazadas.

En testBasico() y moverIter() las invocaciones se efectúan únicamente para enseñar la sintaxis correcta, y aunque se toma el valor devuelto, no se usa para nada. En otros supuestos, el valor retornado no es capturado, porque no se utiliza con asiduidad.

## Sets

Set posee concretamente el mismo interfaz que Collection, y no hay ninguna funcionalidad extra, como en el supuesto de las Listas. Un Set es exactamente una Colección, pero tiene utilizada en un entorno determinado, que es adecuado para el uso de la herencia o el polimorfismo. Un Set sólo permite que exista una instancia de cada objeto.

Entre las diferentes aportaciones de Set, deben emplearse HashSet en general, a no ser que se necesiten las peculiaridades asignadas por una de las distintas implementaciones que se lleven a cabo.

## Set (interfaz)

El conjunto de componentes que se añada a un Set será único, ya que el otro supuesto no se sumará porque el Set no soporta almacenar elementos duplicados. Los elementos incorporados al Conjunto deben tener diseñado el método equals(), en aras de aplicar comparaciones para quitar duplicados. Set tiene similar interfaz que Collection, y no garantiza el ordenamiento en que aparezcan almacenados los objetos que posea.

## HashSet

Es la elección más corriente, a excepción de Sets que serán muy pequeños. Debe tener declarado el método hashCode().

## ArraySet

Es como un Set incrustado en un Array. Esto es de utilidad para Sets pequeños, especialmente los que son creados y destruidos con frecuencia. En estos pequeños Sets, la creación e iteración absorve muchos menos recursos que en el caso del HashSet. En cambio, el rendimiento es muy malo en el supuesto de Sets con gran cantidad de elementos.

## TreeSet

Es un Set organizado y ordenado, guardado en un árbol balanceado. En este supuesto es muy fácil extraer una secuencia ordenada a partir de un Set de este tipo. El ejercicio se limita a enseñar aquellas cosas que son particulares de los Sets.

```
import java.util.*;
   public class ArbolLista {
    public static void pruebaGrafica( Set dia ) {
arbol.fill( dia );
        arbol.fill( dia );
        arbol.fill( dia );
        arbol.print( dia ); // No permite Duplicados!

        // Se añade otro Set al anterior
        dia.addAll( dia );
        dia.add( “uno” );
        dia.add( “uno” );
        dia.add( “uno” );
        arbol.print( dia );

        // Buscar ese componente
        System.out.println( “dia.contains(\”uno\”): “+dia.contains( “uno” ) );
    }

    public static void main( String args[] ) {
        PruebaGrafica( new HashSet() );
        PruebaGrafica( new ArraySet() );
         }
    }
```

Como se inserta algún valor duplicado al Set, a la hora de lanzarlos por pantalla, se puede comprobar que solamente se permite una instancia de cada valor. Cuando se arranca el ejercicio, se observa también que el orden que tiene el HashSet es distinto del que presenta el ArraySet, ya que cada uno tiene una forma de guardar los elementos para la posterior recuperación. El ArraySet contiene los elementos ordenados, sin embargo el HashSet usa sus propias funcionalidades para que las búsquedas sean rápidas. Cuando el programador crea sus propias pautas de estructuras de datos, debe prestar especial atención ya que un Set requiere alguna forma de poder contener el orden de los elementos del conjunto, como se muestra en el ejemplo.

Las declaraciones de los métodos equals() y hashCode() son similares a las de ejemplos. Se debe definir equals() en ambos supuestos, mientras que hashCode() solo es requerido si la clase corresponde a un HashSet, que debe ser la primera opción a la hora de insertar un Set.

### Pilas y colas en Java

Vamos a explicar un ejemplo de colas y pilas en Java:

– Pilas: En primer lugar definimos la pila como último en entrar, primero en salir, cuando insertamos a la pilas usamos puhs y cuando sacamos de la pila usamos pop. El ejemplo más claro de pila es una torre de CD`S, el ultimo CD en entrar es el primero en salir; y el primer Cd en entrar es el ultimo en salir.

  Ahora vamos con el código de una pila en Java:

```
import java.util.*;
public class PilaCdrom extends LinkedList{

public void push (Object cdrom){ //creamos el método push donde por parametro
this.addFirst(cdrom); //Insertamos el objeto en la posición 1 de la pila
}
public Object pop(){ //Creamos el método pop el cual elimina el ultimo objeto agregado a la pila
return this.removeFirst();
}
}
```

Ahora vamos a implementar el código de la pila.

Este código es muy sencillo agregamos 5 números y eliminamos el ultimo:

```
public class EjemploPilas {
public static void main (String args[]){

PilaCdrom pilacdrom = new PilaCdrom ();
pilacdrom.push(1);
pilacdrom.push(2);
pilacdrom.push(3);
pilacdrom.push(4);
pilacdrom.push(5);
System.out.println(pilacdrom);
pilacdrom.pop();
System.out.println(pilacdrom);
}
}
```

– Colas: El primer componente en entrar es también el primero en salir. Un ejemplo bastante corriente en nuestra vida diaria es cuando haces un fila en taquilla del Cine, el primer cliente en entrar es el primer en ser atendido y por ello será el primer en salir nuestro código de la cola:

```
import java.util.*;

public class ColaCine extends LinkedList {

public void push (Object ticket){
this.addFirst(ticket);
}
```

```
public Object pop(){
return this.removeLast();
}
}
```

Ahora vamos a teclear este código en un ejerccio similar al anterior de la pila. Vamos a insertar ahora cinco números.

```
public class CineMiramar {

public static void main (String args []){

ColaCine colacine = new ColaCine ();

colacine.push(1);
colacine.push(2);
colacine.push(3);
colacine.push(4);
colacine.push(5);
System.out.println(colacine);

colacine.pop();

System.out.println(colacine);
}
}
```

Podemos observar en la Pila el numero que salía era el ultimo que entraba en ese caso era el numero 5; sin embargo en la Cola el primer número en entrar es el primero en salir y en este caso el número que sale es el 1.

## 7.2.2. Estructuras

### Almacenamiento de datos

Los arrays y las estructuras de datos como listas, colas, pilas, etc. son entidades de informática abstracta que permiten almacenar datos en un lenguaje de programación como Java, objetos y tipos primitivos int, double, char, boolean, etc…

Los arrays son seguramente, la estructura más empleada para almacenar y ordenar datos dentro de la programación, debido a que las instrucciones para acceder a sus datos es muy amigable para el programador. Por ejemplo, si tenemos un array de enteros, y queremos sumar el segundo número de un array con el cuarto, simplemente usamos la notación [ ] para acceder a ellos:

```
int suma = numeros[1] + numeros[3];
```

En otro caso si queremos llenar un array con todas las letras minúsculas, desde la a la z, tecleáremos el siguiente código:

```
for (int letrasmin=65; <=90; letrasmin++)
[letrasmin-65]=chart(letrasmin);
```

Conversión por Cast.

Y así, minúsculas sería igual a {‘a’, ‘b’, ‘c’, ‘d’,…, ‘z’}.

Si queremos enseñar gráficamente lo que un array corresponde en la memoria, podríamos observar el siguiente esquema:

| | | | | | | |
|---|---|---|---|---|---|---|
| DATOS - → | objeto0 | objeto1 | objeto2 | objeto3 | objeto4 | objeto5 |
| Índices _ → | 0 | 1 | 2 | 3 | 4 | 5 |

Y si queremos acceder a un objeto en el array, únicamente hacemos alusión a su puntero/índice y usamos la notación [ ].

Pero esta manera de almacenar tiene ciertas desventajas, por ejemplo:

– Los arrays tienen una capacidad de almacenamiento fijada y no alterable, lo que se transforma realmente en un problema si queremos almacenar datos sin saber cuántos almacenaremos en total, o si la cantidad de datos es variable.

– Los programadores emplean arrays grandes, derrochando espacio de memoria que nunca se utiliza. Si queremos registrar a los clientes que han realizado alguna compra en nuestra tienda cada día, podríamos crear un array de una capacidad estimada de 100 espacios para cada día. Sin embargo en el supuesto que en una ocasión sólo entraran a comprar 22 clientes, 78 espacios del array son desperdiciados, y, por lo tanto, espacio en la memoria es malgastado. Peor aún, que entraran 150 personas a comprar y que nuestro array no fuera capaz de almacenar a todos los clientes. En estos casos, los arrays no son la herramienta indicada.

– La inclusión de un objeto al principio del array, sin sobrescribir el primer espacio, se torna mas complicada, pues debemos desplazar en un espacio a la derecha a todo el resto de los datos. Es decir, los arrays no manipulan los datos de forma ágil y dinámica. Para solventar este problema, haremos uso de una nueva forma de almacenamientos: las estructuras datos basadas en punteros, con su principal exponente, la Lista.

## 7.2.3. Ficheros

Un fichero es un conjunto de datos organizados que son almacenados en medios de almacenamiento y pueden ser usados por las aplicaciones. La manera en que un ordenador organiza, da nombre, guarda y maneja los ficheros y gestiona se conoce como “sistema de archivos”, siempre depende del Sistema Operativo y del medio de almacenamiento como disco duro, disco externo, memorias USB, etc.

La peculiaridad principal de los archivos es el Nombre y su extensión, El nombre es la identificación del archivo y su extensión indica el tipo de archivo al que corresponde, la extensión generalmente es de tres caracteres, por ejemplo:

– trabajo.doc → es un archivo tipo documento de Microsoft Word.

– cancion.mp3 →es un archivo de música.

– mensualidades.xls →es un archivo de hoja de cálculo de Microsoft Excel.

– Fichero.txt → es un archivo de texto.

### Serialización de un objeto

Para que un programa pueda convertir un objeto en un cúmulo de bytes y pueda luego rescatarlos, el objeto necesita ser serializable. Para que un objeto sea serializable es necesario que implemente la interfaz Serializable, como la interfaz Serializable no tiene métodos, es muy facil implementarla, basta con un implements Serializable.

#### Las clases FIleInputStream y FileOutputStream

Gran parte de la entrada y salida de los programas se efectua basándose en archivos y para ello Java.io entrega dos clases. Una para ficheros de entrada, FileInputStream, y otra para ficheros de salida, FileOutputStream. La diferencia entre ficheros de entrada y de salida es fundamental, pues situaciones como querer leer de un fichero de salida o a la inversa pueden generar en errores.

#### Las clases ObjectInputStream y ObjectOutputStream

Son como tamices de sucesión que pueden leer y escribir objetos de una fuente de datos orientada a bytes. Únicamente tiene coherencia si los datos guardados son objetos.

El método readObject devuelve un object de los datos de la entrada.

Para controlar las excepciones mediante try catch, opcionalmente podemos controlar mediante throws Exception en la función que utilizaremos, a continuación se muestra un programa donde se hace uso de las clases nombradas mediante listas enlazadas.

```
import java.io.*;
public class Fichas implements Serializable{
private String nombre;
private int anotacion;
Fichas enlace;
Fichas(String n,int no)
{
nombre=n;
anotacion=no;
enlace=null;
}
String getNombre()
{
return nombre;
}
int getAnotacion()
{
return Anotacion;
}
}
```

Siguiendo con el hilo del ejemplo anterior, en este supuesto práctico observamos como utilizamos el fichero externo:

```
import java.io.*;
public class Lista{
Fichas cab;
File ficheroalumnos=new File("d:\\notas.not");
void cargar()throws Exception
{
if(ficheroalumnos.exists())
{
FileInputStream fis=new FileInputStream(ficheroalumnos);
ObjectInputStream ois=new ObjectInputStream(fis);
cab=(Fichas)ois.readObject(); //cargamos el archivo al objeto alumno
fis.close();
ois.close();
}
Else{
ficheroalumnos.createNewFile();
}
void insertar(String n,int no)
{
if(cab==null)
```

```
cab=new Nodo(n,no);
else
{
Fichas temp=new Fichas(n,no);
temp.enlace=cab;
cab=temp;
}
}
void mostrar()
{
Fichas aux=cab;
while(aux!=null)
{
System.out.println(aux.getNombre()+" "+aux.getNota());
aux=aux.enlace;
}
}
void guardar()throws Exception
{
FileOutputStream fos=new FileOutputStream(f);
ObjectOutputStream oos=new ObjectOutputStream(fos);
oos.writeObject(cab); //guardamos el objeto en el archivo
//se guarda los cambios realizados en la lista
fos.close();
oos.close();
}
}
```

En este supuesto hacemos lo mismo, partimos de un caso anterior y terminamos el ejercicio con el Menú que sigue con el hilo del ejemplo anterior:

```
import java.util.*;
public class menu {
public static void main(String[] args) throws Exception {
Scanner en=new Scanner(System.in);
Lista universitarios=new Lista();
universitarios.cargar();
int opc=10;
while(opc!=3)
{
mostrarMenu();
opc=en.nextInt();
switch(opc)
{
case 1: System.out.println("Teclea Nombre y Nota");
universitarios.insertar(en.next(),en.nextInt());
break;
case 2: universitarios.mostrar();
break;
case 3: universitarios.guardar();
break;
}
}
}
static void mostrarMenu()
{
System.out.println("--------Menu-------");
System.out.println("1. insertar");
System.out.println("2. mostrar");
System.out.println("3. Grabar y salir");
}
}
```

## 7.2.4. Otras estructuras complejas: tablas hash e introducción a los árboles y grafos

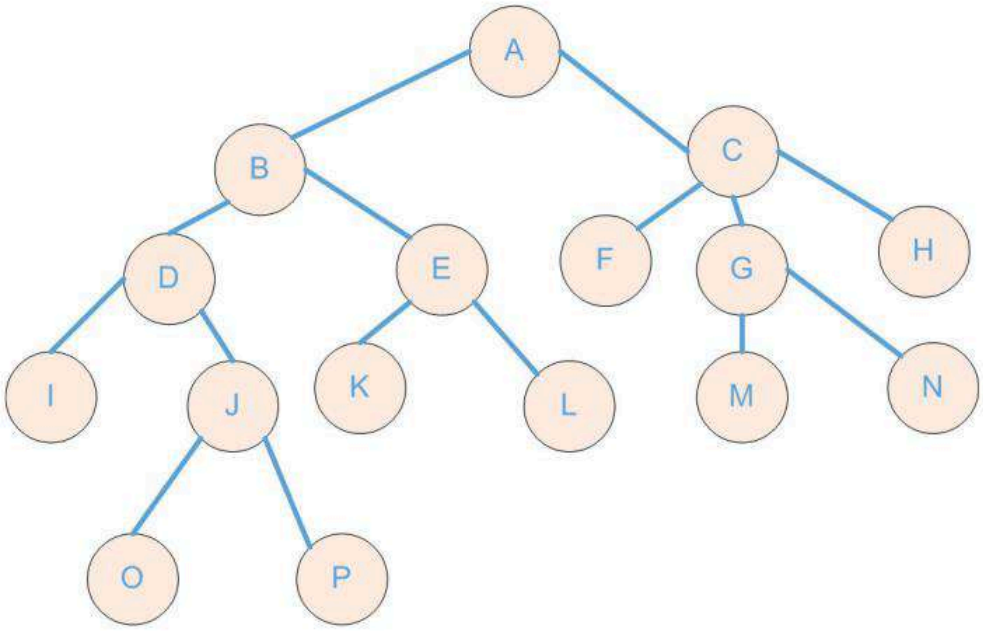
### Conceptos básicos

Los árboles son una de las estructuras de datos no secuenciales más utilizada. Sirve para representar estructuras de información jerárquicas y direcciones o etiquetas de una manera organizada.

Dentro de la informática, los árboles tienen muchas aplicaciones, como, por ejemplo:

- Ordenar tablas de símbolos en compiladores
- Reflejar tablas de decisión
- Asignación de bloques de memoria de distinto tamaño
- Organizar
- Localzar y buscar
- Solucionar y definir juegos
- Testear teoremas

Como objetos científicos y en el campo de las matemáticas los árboles también han sido estudiados con profundidad.




*Esquema de la representación de un árbol con raíz hacia arriba con un Grafo*

Los árboles posibilitan representar situaciones de la vida cotidiana en diferentes ámbitos como son:

– Organización de una empresa

– Árbol genealógico de familias

– Mapas mentales y de toma de decisiones

– Definición de eventos deportivos

Para dar una definición clara de árbol, diremos que un árbol es un conjunto finito A de uno o más nodos tal que:

– Existe un punto especial llamado la raíz del árbol

– Los puntos restantes están fraccionados en m > 0 conjuntos disjuntos A1,…,Am y cada uno de estos conjuntos es a su vez un árbol. Los árboles A1,…,Am son llamados subárboles de la raíz.

Cualquier punto es la raíz de un subárbol que consiste de él y los puntos debajo de él. Esto se deriva de la definición recursiva de árbol presentada.

Un árbol puede representarse gráficamente de muchas formas. Algunas de ellas son por medio de:




La forma más normal de representar a los árboles es por medio de un grafo con la raíz hacia arriba.

Cada vértice o nodo del árbol puede tener un nombre y puede tener una información asociada; un arco es una conexión entre dos vértices.

Una rama en un árbol es una lista de vértices distintos en los que vértices sucesivos están conectados por arcos en el árbol. Una propiedad que define a los árboles es que existe exactamente una rama entre la raíz y cada uno de los otros nodos en el árbol.

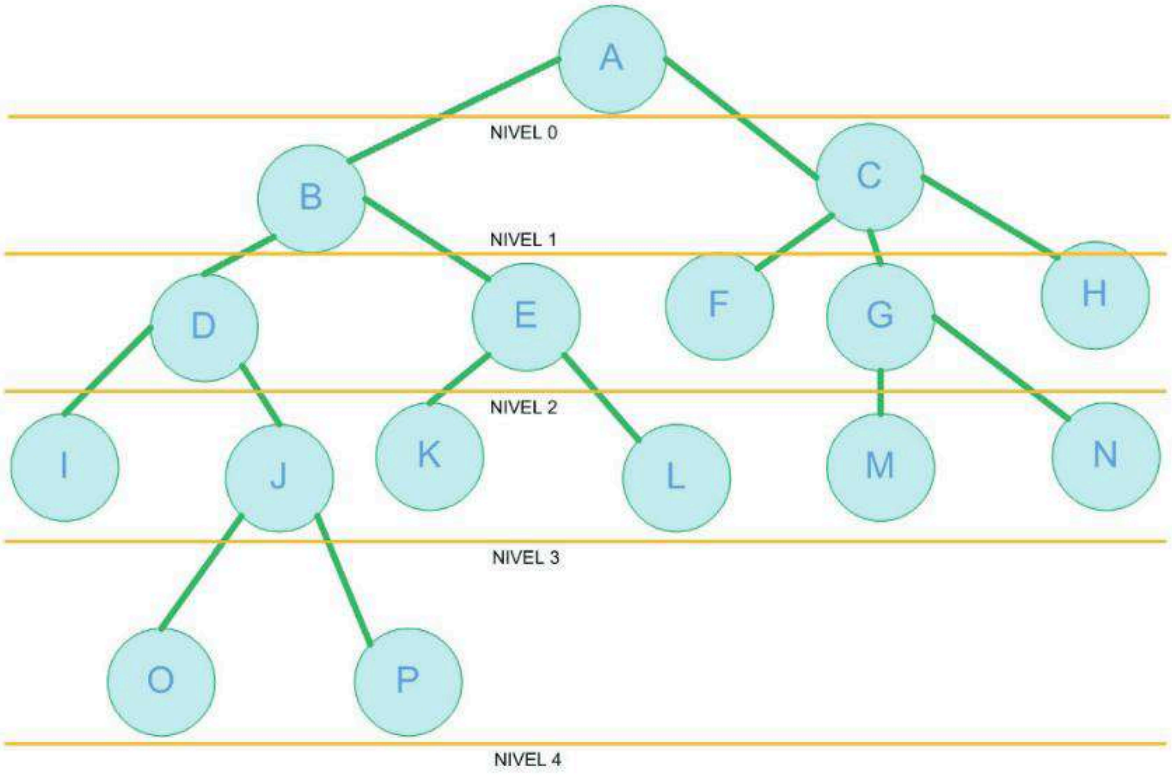
La longitud de una rama es el número de nodos de la rama menos uno. Por tanto, hay una rama de longitud cero de cualquier nodo a si mismo.

Se dice que un nodo Y está abajo de un nodo X, si X está en la rama de Y a la raíz.

Además, cada nodo, excepto la raíz, tiene exactamente un nodo arriba de él, que es llamado su padre; los nodos que están exactamente abajo de él son llamados sus hijos.

El número de hijos que cuelgan de un nodo es llamado el grado del nodo. El grado de un árbol es el grado máximo de los nodos del árbol. Un nodo de grado cero es llamado hoja, es decir, no tiene hijos.

Por ejemplo, considera el siguiente árbol:




*Esquema de un árbol con raíz hacia arriba con un Grafo con nodos de Grado*

Los nodos de grado 0 son: I, P, K, L, F, M, N, O, y H es decir son las hojas del árbol.

El único nodo de grado 1 es J.

Los nodos de grado 2 son A, B, D y E.

Los nodos de grado 3 son C y G.

No hay nodos de grado mayor, por ello el grado del árbol es 3.

Los nodos de un árbol pueden fragmentarse en niveles; el nivel de un nodo es el número de nodos en el camino de él a la raíz. La altura de un árbol es el nivel máximo de todos los nodos en el árbol, es decir, la distancia máxima de la raíz a cualquier otro nodo.

## 7.3. Mecanismos de gestión de memoria

### La JVM (máquina virtual Java)

La máquina virtual de Java —o JVM en sus siglas en inglés— es el anclaje "físico" de nuestro maravilloso lenguaje multiplataforma con el Sistema Operativo correspondiente y, como tal, ocupa y utiliza memoria del sistema. Bien, esto es evidente pero, lo que es menos conocido, es como está implementada esta gestión de memoria.

La JVM se arranca reservando, por defecto, 64 MB de memoria para trabajar que va ampliando según necesidades. Sin embargo, estas sucesivas ampliaciones automáticas llegan a un límite. Podemos gestionar nosotros mismos estos límites —tanto por arriba como por abajo— gracias a unos parámetros con los que se puede acompañar al comando de arranque de la JVM.

Esto es vital si presentimos que nuestro programa va a necesitar una gran cantidad de memoria para su correcto funcionamiento y queremos evitar un OutOfMemoryError o si, por el contrario, nuestra aplicación compite con otras de mayor prioridad en un sistema y necesitamos que utilice solo una parcela de memoria.

## Parámetros de gestión de memoria

Antes de nada, es importante tener en cuenta que los parámetros de gestión de memoria son, en la mayoría de los casos, específicos de la máquina virtual. Así, el parámetro -Xlp es exclusivo de la máquina virtual implementada por IBM y se puede utilizar exclusivamente con esta.

Aunque hay un diferencias en la práctica de implementaciones de JVMs, realmente, solo 3 o 4 son utilizadas en un porcentaje explicativo y, en cualquier caso, siempre habrá que probar nuestra aplicación con la implementación de referencia, la de SUN, para evitar posibles problemas de compatibilidades. Por eso, nos centraremos en los parámetros que se puedan utilizar con las maquinas virtuales de SUN.

Los parámetros de gestión de memoria se identifican por un sistema de prefijos que indican la naturaleza de dichos parámetros, así y según SUN:

– Los parámetros que comienzan con -X no son estándar, no se garantiza su implementación en todas las implementaciones de la JVM y pueden cambiar en futuras versiones del JDK.

– Los parámetros que comienzan con -XX no se consideran estables (¿¿??) y no se recomienda su uso para usuarios sin un conocimiento especifico.

Los principales parámetros de gestión de memoria son:

– Xms < tamaño > asigna la cantidad inicial de memoria.

– Xmx < tamaño > asigna la cantidad máxima de memoria.

– Xss < tamaño > asigna el tamaño máximo de pila de cada hilo de Java.

Es decir, si quisiéramos arrancar nuestra aplicación con una cantidad de memoria asignada de 64 MB y queremos que llegue a utilizar un GB, por ejemplo, tendríamos que lanzar un script de la siguiente manera:

```
java -Xms64m -Xmx1024m “nombre de nuestra aplicación”
```

Es importante conocer en profundidad que implican y para que se utilizan cada uno de estos parámetros:

– Los parámetros Xms y Xmx establecen el tamaño inicial y máximo de la cantidad de memoria que la JVM utilizará para ejecutar programas Java pero, esto no implica que nuestro proceso Java ocupe en memoria un tamaño máximo como el establecido en el parámetro Xmx. Como se ha indicado, la JVM es un proceso en si mismo y por ello requiere memoria para cargar código y datos. Por ello, un proceso Java puede incluso ocupar más espacio en memoria que el determinado en Xmx.

– El parámetro Xss sirve para establecer cual es la cantidad de memoria máxima que podrá usar un hilo realizado dentro de la maquina virtual. Para aquellos que no estén familiarizados con la gestión de hilos, pondremos un ejemplo de sencilla comprensión: imaginemos una aplicación web desarrollada en Java. Este programa web poseerá sus datos comunes y ocupará X espacio en memoria. Por cada petición a nuestra página web, se creará un hilo de ejecución independiente que tendrá sus propios datos. Por ejemplo, la información de un formulario de alta en nuestra web y que no serán compartidos por el resto de hilos que rueden en este momento nuestra web no es monousuario puede tener miles de usuarios concurrentes. En este supuesto, toda la memoria que requiera cada uno de los hilos se suma para el total de memoria global determinado por el parámetro Xmx pero, además cada hilo chequea que no supere el máximo de memoria que establece el parámetro Xss. Si se supera, se lanza un StackOutOfMemoryError.Un caso concreto donde se puede dar un StackOutOfMemoryError es a la hora de usar cálculo en procesos XSLT por lo que, con seguridad tengamos que subir el tamaño máximo de pila por hilo si usamos este tipo de tecnología.

### Parámetros avanzados de gestión de la memoria

Hemos observado los parámetros más elementales de gestión de memoria pero hay muchos más. Vamos a enfocar en dos de los más importantes o, lo que es lo mismo, de los que pueden solucionar problemas tanto de eficiencia como de funcionamiento.

El problema es que, antes de poder asignar cada comando, tendremos que profundizar mucho más para entender que parámetros vamos a aplicar.

## Gestión de memoria y recolector de basura

En este punto se toca uno de los puntos más críticos acerca del funcionamiento de la maquina virtual de la plataforma Java, la gestión de memoria. Es necesario que la ocupación en memoria de la que haga uso la maquina virtual sea la mínima posible y que además las tareas relativas a dicha gestión de memoria sean lo más rápidas posibles.

Uno de los apartados más importantes en la gestión de memoria es el denominado "recolector de basura" que es el mecanismo cuya responsabilidad es de chequear la memoria ocupada para ver si está siendo usada por ella y poder liberar en caso de que no sea así.

Es importante comprender desde una visión global de la gestión de memoria en los sistemas informáticos, los distintas técnicas que se emplean y los problemas que se pueden presentar. Posteriormente se tratarán las dos tareas principales de gestión de memoria:

– Reserva de memoria.

– Recolector de basura.




La reserva de memoria es la operación por la cual se toma una porción de la memoria virtual libre en el sistema operativo para uso de una aplicación.

Existen varias técnicas conocidas entre ellas:

– Técnica de First Fit.

– Técnica de Buddy System.

– Técnica de Suballocators.

Es necesario conocer como la recolección de basura, acarrea un uso automático de la memoria. Esto quiere decir, que el programador no se tiene que preocupar por liberar la memoria de un objeto cuando este no se este utilizando sino que es una función automática. Actualmente hay muchísimos algoritmos distintos de recolección de basura entre los cuales hay que comentar los siguientes:

- Recolector de marcas
- Contador de referencias de 1 bit
- Recolector conservador
- Recolector incremental
- Contadores de referencias
- Recolector de copias
- Recolectores de traza

### 7.3.1. Uso de la gestión automática de memoria

#### La memoria en Java

Es un aspecto muy importante y que en programación tiene siempre un papel fundamental entre los aplicativos, programadores, los programas implementados donde la responsabilidad de la gestión de la memoria de los programas caía en el programador. En C, es frecuente usar las instrucciones tipo “malloc” y “calloc” para reservar memoria dinámicamente y la función dispose para liberar la memoria que hemos reservado en nuestro programa para que pueda ser reutilizada.

Una gran ventaja que aportó el lenguaje Java es la gestión dinámica de la memoria. Con la programación en Java nos olvidamos de gestionar la memoria y toda la responsabilidad recae en el lenguaje. Se definirán variables en el orden que se vayan construyendo en la memoria **heap** y se liberará automáticamente estos datos cuando no se requiere más por un proceso de la máquina virtual de Java (JVM) llamado Garballe Collector (GC).

### Tipos de memoria

En Java podemos hablar de tres zonas de memoria:

- **Zona de datos**, donde se almacenan las instrucciones del programa, las clases con sus métodos y constantes. Esta zona es inalterable durante todo el tiempo de ejecución del programa.

- **La memoria heap**, lugar en el que se guardan las variables que se crean en los programas con la instrucción "new". Esta zona de memoria es dinámica e irá aumentando conforme se vayan creando objetos desde nuestro programa.

- **La memoria stack**, donde se almacenan los métodos y las variables locales que se crean dentro de los mismos. Esta zona de memoria es estática y no cambia durante el desarrollo del programa. Cada vez que un método se llama se coloca al inicio de la pila y se almacenará información como el número de la línea que se está ejecutando así como todos los valores de las variables locales. Los Threads o Hilos tienen su propia pila de llamadas. Un fallo que nos puede suceder es que al arrancar nuestro programa obtengamos la excepción Java.lang.OutOfMemoryException por un StackOverflow. La causa sería por ejemplo por estar realizando una invocación recursiva y no hayamos implementado adecuadamente la condicional de parada.

### Ventajas y desventajas de la gestión dinámica

Una pequeño problema es quizá que en de la gestión de memoria en Java, nosotros no controlamos donde queremos establecer nuestra memoria. Asignar memoria en el **heap** es más costoso y también liberar dicha memoria, por eso, a veces es deseable almacenar un objeto en la memoria **stack**.

Entre las mejoras con otros lenguajes, en Java con la gestión dinámica no tenemos que preocuparnos por establecer y desasignar memoria, ya que es un proceso totalmente dinámico. Asimismo la JVM internamente utiliza una optimización llamada análisis de escape (en inglés scape analysis) que realiza comprobaciones para saber si un objeto se utiliza sólo en un thread o un método y crearlo en la memoria **stack**, incrementando el rendimiento de nuestros programas.

### Configuración de tamaño de memoria en la JVM

La JVM asigna el tamaño inicial y el tamaño máximo para la reserva de la memoria **heap** cuando arranca, en este caso nosotros podemos modificar esos parámetros si lo creemos oportuno. Por ejemplo, a veces nos podría dar una excepción Java.lang.OutOfMemoryException en nuestro servidor J2EE (Tomcat, Weblogic…) ya que puede que una aplicación consuma demasiada memoria y el tamaño que puede llegar a alcanzar la memoria **heap** es muy pequeña. Igualmente conseguiremos modificar el tamaño máximo de la memoria **heap** que jamás deberá exceder el tamaño de la memoria física de la máquina.

– Para indicar el tamaño mínimo que tiene la memoria **heap** utilizamos el parámetro: Xms1024m. Establecemos el tamaño mínimo del **heap** en 1024 MB.

– Para indicar el tamaño máximo que tiene la memoria **heap** utilizamos el parámetro: Xmx1800m. Establecemos el tamaño máximo del **heap** en 1800 MB.

### "Write once, run anywhere"

Una vez visto cómo se gestionará la memoria en Java, tendremos que repasar un punto que a mi modesto entender es muy necesario en la plataforma Java, la compilación y ejecución de los programas.

Las bondades de un lenguaje en un título. Es el título es un eslogan con gancho para promocionar el lenguaje Java y hace referencia al soporte multiplataforma de Java. Porque en Java escribimos un programa una vez y lo compilamos a bytecodes. Dicho código compilado nos vale para ejecutar el programa en diferentes plataformas (móviles, Linux, Windows…). Para ello sólo necesitamos diferentes máquinas virtuales (JVMs) que traduzcan los bytecodes al código máquina de la plataforma oportuna.

Ya desde las primeras versiones de Java la JVM era simplemente un mero intérprete de bytecodes. Esto hacía que los programas se ejecutaran con lentitud, ya que se tenía que interpretar y ejecutar los bytecodes a la vez sin realizar ninguna optimización.




En las versiones actuales de la JVM se ejecutan los programas ajustando la interpretación de bytecodes y la compilacion JIT o Just–In–Time. Estas siglas significan la Java Virtual Machine examina bytecodes a medida que se van interpretando e identifica los bytecodes que ruedan con más frecuencia.

Estos bytecodes son convertidos a lenguaje máquina correspondiente por el compilador JIT. Cuando la JVM localice de nuevo estos puntos activos nuevamente, ejecutará directamente el código máquina disminuyendo el tiempo de ejecución del programa y aumentando el rendimiento.

### Gestión automática de memoria

Aunque hemos tocado este punto ya en esta Unidad, es bueno recordar que en la gestión automática de memoria, se puede considerar realmente como un servicio, una parte de un lenguaje de programación o una extensión del mismo que automáticamente recicla la memoria que un programa no va a volver a usar. Los gestores automáticos (también conocidos como recolectores de basura o simplemente recolectores) constantemente hacen su trabajo reciclan aquellos objetos que no son asequibles desde ninguna variable del programa, es decir objetos a los cuales no apunta ningún puntero.

Las principales ventajas de la gestión automática de memoria son:

– El programador se libera de todas las tareas de gestión de memoria.

– Las divergencias entre distintos módulos son claras.

– Hay muy escasos fallos relacionados con la memoria.

– El manejo de memoria suele ser más eficiente.

Y solo escasas desventajas de este método son:

– La memoria que es alcanzable desde algún fragmento del programa, se mantiene aún cuando no vaya a ser usada más.

– Los gestores automáticos de memoria tienen limitada disponibilidad.

Se conocen muchas formas de implementar recolectores automáticos de memoria. La mayor parte de los lenguajes de programación actuales y aprovechan este tipo de recolectores: Java, Basic, Dylan, Erlang, Haskell, JavaScript, Lisp, ML, Modula–3, Perl, Postcript lenguaje, Prolog, Phyton, Scheme,…

### 7.3.2. Construcción y destrucción de objetos

Para el ejemplo que veremos de código fuente, si queremos disponer de espacio en la memoria para archivar la información de por ejemplo de alumnos tendremos que crear un objeto con el patrón (clase) alumno.

Los objetos se crean con el término reservado "new". En la siguiente instrucción se detalla un ejemplo de declarar un objeto:

```
primero =  new alumno();
```

Establecemos un objeto de la clase alumno que referenciamos con la variable primero.

La variable tendrá que haberse declarado antes, y la declaramos como una variable del tipo alumno, ya que la vamos a emplear para guardar información de objetos del tipo alumno.

Un programa con la declaración de la variable y la creación del objeto sería:

```
public class aula1{
public static void main(String[] args){
    alumno primero;         // 1
    primero  = new alumno();   //  2
    primero.nombre = "Juanjo"; // 3
    primero.apellido = "Lucas Miró"; // 4
 System.out.println(primero.nombre + " " + primero.apellido);
}
}
```

## Asignación de direcciones

La única dirección especifica que se puede asignar a un objeto directamente es null. Se trata de un valor especial que se entiende que es el de ninguna parte de la memoria. El siguiente programa no dará ningún error de compilación:

```
public class aula5{
public static void main(String[] args){
    alumno primero, otro = null;
    otro.nombre = “Miguel”; // 2
    otro.apellido = “Muñoz Lopez”;
}
}
```

Sin embargo revelará el error en ejecución al intentar, en la instrucción 2, guardar un String en un lugar de memoria nulo e inútil.

El valor null se usa para inicializar variables de objetos, y para dejar de manifiesto que un objeto no se va a utilizar más y está listo para que el recolector de memoria recicle el espacio.

También resulta muy útil para comprobar si una variable está sin utilizar:

```
if (otro == null) .....
```

## Destrucción de objetos

En la instrucción 4 del siguiente bloque de código Java, se deja abandonado sin ninguna referencia al objeto y estará listo para ser destruido.

```
public class Aula2{
public static void main(String[] args){
    alumno primero = null;
    primero  = new alumno();
    primero.nombre = "Juanjo";
    primero = null;  // 4
}
}
```




En lenguaje Java, el desarrollador no tiene que molestarse por la gestión de la memoria. Los objetos se destruyen solos, por medio del recolector de memoria, sin que el desarrollador tenga que solicitarlo.

En diferentes lenguajes de Java, como el Pascal o C, en los que la memoria se emplea dinámicamente para varios menesteres, si que debe liberarse esta explícitamente.

El recolector de memoria absorve recursos de la máquina pero, merece el esfuerzo. Lo que está claro que la memoria no se estropea por defectos en la programación y gestión de la memoria.

### 7.3.3. Objetos inalcanzables

El lenguaje Java posee muchas ventajas y características de gran utilidad para el programador, entre otras cosas:

– En el momento que el compilador e intérprete de Java son portables a una nueva y diferente plataforma, un programa Java puede rodar en tal plataforma sin cambios.

– El compilador hace varias comprobaciones, tal como comparar los tipos de los argumentos que se envían a los métodos para asegurar que se ajustan a los tipos de los parámetros formales.

El intérprete hace muchas verificaciones durante el tiempo de ejecución, tal como asegurar que el índice de un array no salga de sus límites, o que una referencia a un objeto no sea nula.

No son posibles ni "fugas de memoria" (memory leak, un bloque de memoria que todavía se considera reservado en memoria aun cuando no haya referencias a él) ni "apuntadores colgantes" (dangling pointers, una relación a un lugar de memoria que ha sido liberado), ya que el recolector de basura "garbage collector" de Java recopila toda la memoria reservada y ocupada por objetos inalcanzables. No es preciso un procedimiento free(), susceptible a fallos.

El manejo automático de la memoria por el recolector de memoria, atendiendo a los objetos y datos alojados en la misma y su posterior liberación en el momento que ya no son requeridos, es lo que se toca en este apartado y se entienden como Objetos Inalcanzables.




Java emplea un recolector de basura automático para el manejo de memoria en el ciclo de vida del objeto. El programador define cuándo se crean los objetos, y el tiempo de ejecución es responsable de la recopilación de la memoria una vez que los objetos ya están usándose.

En el momento que no hay referencias a un objeto considerándose el objeto inalcanzable éste tendrá derecho a ser liberados automáticamente por el recolector de basura. Similar a una pérdida de memoria pueden producirse si el código de un programador contiene una referencia a un objeto que ya no es necesaria, en términos generales cuando hay objetos que ya no son importantes, estos se guardan en contenedores que están todavía usándose. Si los métodos de un objeto que ya no existe se invocan, una «excepción de puntero nulo» es lanzada.

Por otro volviendo al asunto de Java como modelo de gestión de memoria, es que los programadores se ahorraría la carga de tener que realizar la gestión de memoria manual. En algunos lenguajes de memoria para la creación de objetos es implícitamente asignado en la pila, o asignados de forma explícita y cancela la asignación del bloque. De cualquier manera la responsabilidad de gestionar la memoria reside en el programador. Teniendo en cuenta que la recolección de elementos no impide que haya pérdidas de memoria, aquellos en los que la referencia a la memoria no se utilizó.

La recolección de basura puede acontecer en cualquier momento. Lo perfecto sería que se originará cuando un programa está inactivo. Se garantiza que se activará si no hay suficiente memoria libre en el bloque para asignar un nuevo objeto, lo que puede provocar que un programa se tenga que detener momentáneamente.

Es necesario conocer que las variables de tipos primitivos de Java no son objetos. Los valores de tipos primitivos se acumulan directamente en los campos (para objetos) o en la pila (para los métodos) y no en el bloque como para los objetos. Esta fue una decisión consciente por parte de los diseñadores de Java por motivos de rendimiento. Por ello, Java no fue considerado como un lenguaje de programación orientado a objetos puro. Sin embargo, a partir de Java 5.0, consiente a los programadores autoboxing proceder como si los tipos primitivos son instancias de clases de su envoltura.

### 7.3.4. Recolección de «basura»

Habitualmente es bastante complicado el poder añadir un recolector de basura a este tipo de sistemas, aunque existe una técnica nombrada “Conservative Garbage Collection” CGC puede ser usada en estos casos. El inconveniente habitual con este tipo de lenguajes, es que no están preparados de proveer al recolector de basura con información fidedigna de la estructuras albergadas en la memoria, por lo que el recolector de basura no podrá conocer lo que es un puntero y lo que no lo es. Un recolector de basura conservador confía que cualquier dato puede ser un puntero, de esta forma el recolector hace una búsqueda en memoria de un dato que pueda parecerle un puntero y efectúa una copia de dicho puntero en una zona de memoria donde podrá impedir que sea reciclado.

Sugerir que dado que el recolector no conoce con seguridad que localizaciones de memoria contienen punteros, no puede ser combinado con algoritmos copying collectors. El algoritmo copying collector requiere conocer donde están los punteros, para poder actualizarlos cuando los bloques en memoria han sido movidos. Como este algoritmo parece muy sencillo y parece que puede conferir que grandes cantidades de memoria subsistan sin reciclar, en la práctica no es así sobre todo si se optimiza el algoritmo con diferentes refinamientos.

### Garbage collector

El Garbage Collector o recolector de basura es un mecanismo automático de baja prioridad que se ejecuta dentro de la Java Virtual Machine. Su función es de limpiar aquella memoria del **heap** que ya no se utiliza y por tanto, podría ser utilizada por otros programas. Por ello es necesario que el sistema sepa que un objeto podrá ser limpiado cuando desde el **stack** ninguna variable haga referencia al mismo.

Si necesitamos conocer cuando se arranca el Garbage Collector debemos emplear el parámetro "-verbose:gc" antes de arrancar nuestra Java Virtual Machine.

El proceso Garbage Collector además se ejecuta manualmente desde un programa Java si lo vemos conveniente y si creemos que en un momento establecido, hemos podido dejar muchos objetos sin referenciar. En el siguiente ejercicio enseño mediante un pequeño programa Java cómo se puede calcular la memoria libre en un momento determinado en la JVM y cómo ejecutar el Garbage Collector manualmente desde un programa:

```
package es.memo.test;

import java.util.ArrayList;
import java.util.List;

public class TestMemo {
 private static final long Mega = 1024L * 1024L;

 public static long bytesToMega(long bytes) {
  return bytes / Mega;
 }

 public static void main(String[] args) {
// Voy llenando memoria HEAP con objetos que luego no tendrán referencia
```

```
// Esta memoria será liberado por la GC de JAVA
  for (int i = 0; i <= 1000000; i++) {
   Item item = new Item("1", "Rutina de liberación de Memoria", 2);
   item = null;
  }

  // Obtenemos la runtime de Java
  Runtime runtime = Runtime.getRuntime();
  // Ejecutamos el proceso de Garbage Collector
  runtime.gc();

  // Calculamos la memoria máxima y la memoria disponible
  long memory = runtime.totalMemory() - runtime.freeMemory();
  long memoriaMax = runtime.maxMemory();
  System.out.println("Memoria máxima en bytes: " + memoriaMax);
  System.out.println("Memoria máxima en Megabytes: "
    + bytesToMega(memoriaMax));
  System.out.println("Memoria utilizada en bytes: " + memory);
  System.out.println("Memoria utilizada en Megabytes: "
    + bytesToMega(memory));
 }
}
```

### 7.3.5. Métodos constructores y destructores

Para crear los objetos hemos utilizado con la instrucción “new” el nombre de la clase como un método.

Toda clase tiene ese método, por defecto, para fabricar los objetos. Aunque, se pueden programar métodos de creación específicos para cada objeto. A estos métodos de creación de los objetos se les conoce como constructores. El constructor básicamente es un método con el nombre de la clase y para declararlo no se especifica el tipo de dato que retorna sino a la referencia a un objeto.

El constructor puede poseer parámetros que se pueden utilizar para rellenar datos del objeto.

El constructor sólo usa con la instrucción new.

Por ejemplo:

```
public class calendario {
    int dia;
    int mes;
    int ano;
    public calendario(int d,int m, int a){
        dia = d;
        mes = m;
        ano = a;    }
    public void imprime(){
            System.out.println(dia+”/” +mes+”/”+ano);}
 }
```

Y que se utilizaría del siguiente modo:

```
public class dianace{
public static void main(String[] args){
    estudiante uno =  new estudiante();
    uno.nacimiento = new fecha(10,9,1974);
    uno.nacimiento.imprime();
}
}
```

En el constructor se suman todas las acciones que se necesiten realizar cada vez que se crea el objeto.

Si por ejemplo, se quiere añadir en el constructor de la clase estudiante la construcción del objeto fecha, de modo que no tenga que crearse de manera independiente:

```
public class estudiante{
    String nombre, apellido ;
    Int numalum;
    double teoria,practicas;
    String NotasOrdi,NotasExtra;
    fecha nacimiento;
    public estudiante (){
        nacimiento =  new fecha(1,1,2001);
}
}
```

Que se utilizaría:

```
public class dianace{
public static void main(String[] args){
    estudiante uno =  new estudiante();
    uno.nacimiento.imprime();
}
}
```

Los constructores, como todos los métodos pueden ``sobrecargarse". Quiere decir que podrán diseñarse distintos métodos con el mismo nombre pero con diferente contenido, siempre que se distingan por el tipo de parámetros que toman.

En la siguiente definición de la clase estudiante, se incluyen tres constructores diferentes:

```
public class estudiante{
String nombre, apellido,notasOrdi,notasExtra ;
 int numalum; double teoria,practicas; fecha nacimiento;
public estudiante ()
{ nacimiento = new fecha(1,1,2001);}
public estudiante (String n, String a, int ni){
 nombre = n; apellido = a; numalum = ni;
 nacimiento = new fecha(1,1,2001);   }
public estudiante (String n, String a, int ni,
```

```
                int fd, int fm, int fa){
 nombre = n; apellido = a; nia = ni;
 nacimiento = new fecha(fd,fm,fa);   }
public void imprime(){
 System.out.println(nombre+" "+apellido+" "+ numalum +" ("+
 nacimiento.dia+"/"+nacimiento.mes+"/"
 +nacimiento.agno+")");
}
}
```

Que se utiliza en el programa:

```
public class fichaalumno{
public static void main(String[] args){
 estudiante uno =  new estudiante ();
 estudiante otro = new estudiante ("Lucas","Perez",27722);
 estudiante tercero =  new estudiante ("Ignacio"," Lopez
Briant",22223,10,3,1970);
    uno.imprime(); otro.imprime();tercero.imprime();
}
}
```

Con la salida:

```
null null 0 (1/1/2001)
Juan Perez 27722 (1/1/2001)
Ignacio Lopez Briant 22223 (10/3/1970)
```

Puede verse en la consola como las variables de los objetos que no se inicializan, quedarán de forma automática determinada con el valor null o con el valor que posea por el tipo de dato almacenado.

# UD7
## Lo más importante

- Las referencias en Java no son punteros como en C++. Son identificadores de instancias de las clases.Es decir que referencia a un objeto de una clase específica.

- Java permite trabajar con arrays de tipos primitivos o de clases. En Java tiene un privilegio bastante importante y se usan muchísimo.

- Los tipos de datos en Java y en otros lenguaje de programación, constituyen como los métodos de almacenamiento disponibles para representar información, además de la manera en que dicha información será interpretada.

- Para definir una variable en memoria deberá declararse indicando su tipo de variable y su identificación que la identificará de forma única.

- Los tipos de datos en Java pueden dividirse en dos categorías: simples y compuestos.

- Los tipos compuestos se basan en los tipos simples, e incluyen las cadenas, las matrices y tanto las clases como las interfaces, en general.

- Tipos de datos Boolean se usa para almacenar variables que presenten dos estados, que serán representados por los valores true y false.

- En Java es posible transformar el tipo de una variable u objeto en otro diferente al original con el que fue declarado.

– Los Punteros no los hereda Java: Las direcciones de memoria son la característica más poderosa de C++. El inadecuado uso de los punteros provoca la mayoría de los errores de colisión de memoria, errores muy difíciles de detectar.

– En Java hay que declarar un array antes de poder utilizarlo. Y en la declaración hay que incluir el nombre del array y el tipo de datos que se van a almacenar en él.

# UD7
## Autoevaluación

1. Los tipos de datos en Java pueden dividirse en:
    a. Simples y compuestos.
    b. Abstractos y Concretos.
    c. Numéricos y literales.
    d. Binarios y Hexadecimales.
2. Tipos de datos enteros se usan para representar:
    a. Cadenas y matrices.
    b. Números enteros con signo.
    c. Decimales y Hexadecimales.
    d. Números naturales sin signo.
3. De los Literales caracter que representan un único caracter /r significa?
    a. Retroceso.
    b. Retorno de carro.
    c. Continuación.
    d. Linea nueva.

4. La conversión de tipos de datos se pone:
   a. Tipo destino entre paréntesis, a la izquierda del valor que queremos convertir.
   b. Se pone al final de la instrucción junto al ;.
   c. Lo realiza cada método de tipo de datos.
   d. Cada fila es un campo.

5. En Java para trabajar con los array primero:
   a. Hay que declarar un array antes de poder utilizarlo, incluir el nombre y el tipo de datos.
   b. Solo declararlo con new.
   c. La declaración del array localiza, y reserva memoria para contener los datos.
   d. Ninguna de las anteriores.

6. ¿Que función miembro posee Java que se utiliza para conocer la longitud de un array?.
   a. miArray().
   b. ListaArray().
   c. Array().
   d. length().

7. Para crear un array con 5 elementos y unos nombres como datos:
   a. String nombres[] = {“Juan”,”Pepe”,”Pedro”,”Maria”,”Andrea”};.
   b. Int Array()={“Juan”,”Pepe”,”Pedro”,”Maria”,”Andrea”};.
   c. int lista[] = new int[“Juan”,”Pepe”,”Pedro”,”Maria”,”Andrea”];.
   d. Ninguna de las anteriores.

8. List (interfaz) es un elemento:
   a. Lista, asegurando que los elementos siempre se mantendrán en una secuencia concreta.
   b. Es una Lista volcada en un Array.
   c. Permite un acceso aleatorio muy rápido a los elementos.
   d. Permite inserciones y borrado de elementos de en medio de la Lista muy rápidas.

9. ¿Cuál no es una aplicación de los arboles?
    a. Probar teoremas.
    b. Ordenar.
    c. Almacenar información jeráquica.
    d. Organizar tablas de símbolos en compiladores.

10. La forma más común de representar a los árboles es por medio de un grafo con la raíz hacia arriba:
    a. Se dice que un nodo Y está encima de un nodo X, si X está en el camino de Y a la raíz.
    b. Cada vértice o nodo del árbol puede tener un nombre y puede tener una información asociada.
    c. Y un nodo de grado uno es llamado hoja, es decir, no tiene hijos.
    d. Con la longitud de un camino es el número de nodos del camino más uno.

Área: informática y comunicaciones

# UD8

## Lenguajes de programación orientados a objetos

8.1. Análisis del lenguaje de programación orientado a objetos y paradigma orientado a objetos

- 8.1.1. Lenguajes de programación orientados a objetos
- 8.1.2. Lenguajes de programación basados en objetos
- 8.1.3. Lenguajes de programación que utilizan objetos

8.2. Comparación entre los lenguajes de programación orientados a objetos más habituales. Características esenciales

8.3. Librerías de clases

- 8.3.1. Definición de su estructura
- 8.3.2. Creación y utilización

## 8.1. Análisis del lenguaje de programación orientado a objetos y paradigma orientado a objetos

La programación Orientada a objetos (POO) es una manera específica de programar, más próxima a como formularíamos las cosas en la vida real. Muy a diferencia de otros lenguajes o paradigmas. En esta Unidad trataremos de ver y expresar todo lo relativo a los lenguajes de programación que cumplen dichos requisitos y paradigma.

Con la POO necesitamos empezar a discurrir las cosas de una manera distinta, para conseguir desarrollar nuestros programas en términos de objetos, propiedades, métodos y otros elementos y sintaxis que iremos viendo rápidamente para aclarar conceptos, y dar una buena base que posibilite soltarnos a partir de ahora con este tipo de programación. La programación Orientada a objetos (POO) es una manera diferente de programar, más cerca a como pensaríamos las cosas en la vida real que frente a otros tipos de programación.

Nuevamente vuelvo a repetir la idea tan importante de que con la POO tenemos que aprender a pensar las cosas de una manera distinta, y dentro de todos los lenguajes existentes, son los lenguajes orientados a objetos, los más predominantes en la actualidad, los que más oportunidades laborales y de proyectos en estos momentos ofrecen y Java copa dichos interese industriales y empresariales, entre sus características se definen las siguientes:

### Algunas definiciones de interés

- **Abstracción:** dentro del contexto cada componente denominado objeto se usa como modelo de un componente abstracto que puede realizar una labor, comunicar y modificar su estado, y comunicarse con otros objetos en el sistema sin desvelar detalles de cómo se implementan dichas características. Los procesos, las funciones o los métodos pueden además ser abstraídos y cuando lo están, una variedad de técnicas son requeridas para ampliar una abstracción. Es una propiedad muy importante ya que permite que no necesitemos conocer los entramados internos, solo su uso.

- **Encapsulamiento:** Con esta característica que permite aunar todos los componentes que se pueden considerar como propiedad a una misma entidad, al mismo nivel de abstracción. Esto permite aumentar la fusión de los elementos del sistema.

- **Ocultación:** esta propiedad permite poseer aislado del exterior a cada objeto, y cada tipo de objeto exhibe solo una interfaz a otros objetos que determinada cómo permiten interactuar con los objetos de la clase. Esta peculiaridad protege a las propiedades de un objeto contra su edición por quien no tenga privilegios a acceder a ellas, únicamente los propios métodos internos del objeto tendrán acceso a su estado. Esto asegura que otros objetos no pueden cambiar el estado interno de un objeto de formas imprevistas, eliminando con la ocultación todo despropósito de efectos secundarios e interacciones inesperadas. Algunos lenguajes relajan esto, permitiendo un acceso directo a los datos internos del objeto de una manera controlada y limitando el grado de abstracción. La aplicación entera se reduce a un agregado o crucigrama de objetos.

- **Polimorfismo:** esta propiedad del lenguaje Java, habla de la capacidad de poseer un comportamiento distinto, ligado a objetos distintos, y pueden incluso compartir el idéntico nombre, al invocarlos por ese nombre se usará el comportamiento correspondiente al objeto que se esté empleando. Las referencias y las colecciones de objetos podrán poseer objetos de tipos diferentes, y la invocación de un comportamiento en una referencia llevará a producir el comportamiento adecuado para el modelo real del objeto referenciado. Cuando esto sucede en tiempo de ejecución, esta peculiaridad final se conoce como asignación dinámica. Otros lenguajes suministran medios más estáticos en tiempo de compilación de polimorfismo, como las plantillas y la sobrecarga de operadores de C.

- **Herencia:** Esta es una propiedad muy importante y ventajosa, ya que habla sobre que las clases no están incomunicadas, sino que se relacionan entre sí, constituyendo una jerarquía de clasificación. Los objetos heredarán las características y como se comportarán las clases a las que corresponden. La herencia gestiona, permite el polimorfismo y el encapsulamiento consintiendo a los objetos ser creados y definidos como tipos especializados de objetos ya constituidos. Estos pueden compartir y extender)su comportamiento sin tener que volver a implementarlo. Es posible realizarse normalmente colocando los objetos en clases y estas en jerarquías que muestran un comportamiento común. Cuando un objeto hereda de más de una clase se dice que hay herencia múltiple.

Clasificación

Los lenguajes de programación hoy en día se clasifican de acuerdo a varios factores, entre ellos el perfil de abstracción, otro factor es según la forma de ejecutarse y finalmente según el paradigma de desarrollo que tienen cada uno de ellos y estos pueden ser:

## Según su nivel de abstracción

Por su nivel o perfil de abstracción diremos que son aquellos que se aproximan al funcionamiento de un ordenador.

- **Lenguajes Maquina:**

  Están desarrollados en lenguajes perfectamente y de primera mano entendibles por el ordenador, ya que sus instrucciones son bloques de cadenas binarias compuestas de dígitos 0 y 1. Además permite la posibilidad de cargar y transportar un programa a la memoria, sin que tenga que haber una traducción posterior lo que supone una velocidad de ejecución superior, solo que con poca fiabilidad y dificultad de verificar y poner a punto los programas. Y por supuesto su complejidad es una característica.

- **Lenguajes de bajo nivel:**

  Los lenguajes de bajo nivel son lenguajes de programación que se aproximan mucho al funcionamiento de un ordenador. El lenguaje de más bajo nivel sin duda el código máquina. Conjunto de instrucciones con gran nivel electrónico y de microprocesadores. A éste le sigue el lenguaje ensamblador, ya que al programar en ensamblador se acceden a los registros de memoria del ordenador de manera directa.

- **Lenguajes de medio nivel:**

  Existen ciertos lenguajes de programación que se consideran por expertos como lenguajes de medio nivel como puede ser C, al tener ciertas características que los aproximan a los lenguajes de bajo nivel pero teniendo, al mismo tiempo, varias puntualizaciones que lo hacen un lenguaje más ligado al humano considerado de alto nivel.

- **Lenguajes de alto nivel:**

  Este tipo de lenguajes de alto nivel son muy fáciles de implementar y muy visuales, están formados por elementos de lenguaje natural. Programan con estos lenguajes los ordenadores parecen entender un lenguaje natural, aunque lo hacen en realidad de una forma rígida y cuadriculada. En muchas ocasiones nos complicamos algo la vida siendo todo mucho más sencillo en la programación con estos lenguajes.

## Según la forma de ejecución

- **Lenguajes compilados:**

  Está claro que un programa que se diseña y desarrolla en un lenguaje de alto nivel, necesariamente tiene que convertirse en una traducción a un código que pueda comprenderlo el ordenador. Los programas que traducen estos bloques de código y que pueden realizar esta operación se denominan "Compiladores". Con normalidad generan muchas líneas de código máquina compilado por cada estructura y bloque del programa fuente. Se requiere un proceso de de compilación siempre antes de procesar los datos de un programa. La función primordial de los compiladores es reconvertir un programa diseñado en un lenguaje concreto, a un conjunto de nemotécnicos que comprenda el ordenador y entienda del lenguaje máquina con código binario. Al usar un lenguaje compilado, el programa desarrollado jamás se ejecuta mientras haya errores, solo si se ha compilado sin errores el programa, entonces si arrancará y podremos usarlo.

- **Lenguajes interpretados:**

  Este tipo de lenguajes no traducen el programa fuente y almacenan de forma completa el código objeto, que se origina durante la compilación, para usarse en la fase de producción, en estos lenguajes el desarrollador sólo cargará el programa fuente en el ordenador y los datos que serán procesados. Es viable y se puede además utilizar una posibilidad distinta a la de los compiladores para traducir lenguajes de alto nivel. Un programa intérprete almacenado en el sistema operativo del disco, o insertado de forma perenne dentro de la máquina, traduce cada instrucción del programa fuente en lenguaje de máquina según se vaya necesitando durante el proceso de los datos. No se graba el código objeto para utilizarlo posteriormente.

  Por ejemplo, durante el procesado reiterativo de los pasos de un ciclo, cada instrucción del ciclo tendrá que volver a ser interpretado cada vez que se ejecute el ciclo, lo cual hace que el software sea más lento en periodo de ejecución porque se va revisando el código en momento de ejecución pero más rápido en tiempo de diseño porque no se tiene que estar compilando a cada momento el código completo. La próxima vez que se utilice una instrucción, se le debe interpretar otra vez y traducir a lenguaje máquina. El intérprete elimina la urgencia de efectuar un repaso de compilación posteriormente de cada variación del programa cuando se quiere anudar funciones o anular errores; no obstante es patente que un programa objeto compilado con anticipación, deberá ejecutarse con mucha mayor celeridad que uno que se debe interpretar a cada paso durante su elaboración.

### Según el paradigma de programación OOP

No es superior uno que otro, acaso que cada uno tiene ventajas y desventajas. Dependiendo del contexto un paradigma resulta más apropiado que otro. Un paradigma de programación simboliza un enfoque particular o filosofía para la construcción del software. Atendiendo al paradigma de programación, se pueden encasillar los lenguajes en:

El paradigma imperativo o por procedimientos, es considerado el más común y está representado, por ejemplo, por **C** o por **Basic**. Aunque Java rompe totalmente con ello al no ser nada procedimental. La estructura basa en procedimientos está quedando relegada y obsoleta por otros paradigmas. El paradigma funcional está representado por la familia de lenguajes **Lisp** en particular Scheme, ML o Haskell.

El paradigma orientado a objetos. Permite que la representación orientada a objetos mejore la estructura de los datos y por lo tanto se ha aplicado a diferentes paradigmas como Imperativo Secuencial, Funcional, etc. No obstante, la manipulación no queda esencialmente afectada y por lo tanto el paradigma inicial tampoco a pesar de ser re–orientado a objetos. Por ello puede seleccionarse la forma pura de estos paradigmas al momento de desarrollar, en la práctica es frecuente que se mezclen, dando lugar a la programación multiparadigma. Actualmente el paradigma de programación más usado debido a diferentes conveniencias respecto a sus anteriores, es la programación orientada a objetos. Aunque evolucionan muchísimo los lenguajes orientados a eventos y basados en objetos.

#### Lenguajes orientados a objetos

| | | |
|---|---|---|
| ActionScript | Delphi | Oz |
| Ada | Harbour | Perl (soporta herencia múltiple) |
| C++ | Eiffel | PHP (en su versión 5)Python |
| C# | Java | Ruby |
| VB.NET | JavaScript | Smalltalk |
| Visual FoxPro | Lexico (en castellano) | Magik (SmallWorld) |
| Clarion | Objective–C | |
| | Ocaml | |

## Ventajas

Los lenguajes orientados a objetos poseen las siguientes ventajas:

– Construcción de modelos.

– Relacionar el sistema al mundo real.

– Facilita el sustento del software.

– Fomenta la reutilización y extensión del código.

– Permite producir sistemas más complicados.

– Facilita la creación de programas visuales.

– Agiliza el desarrollo de programa.

– Facilita el trabajo en equipo.

Lo interesante de la Poo es que proporciona conceptos y herramientas con las cuales se modela y representa el espacio real tan literalmente de cualquier manera factible.

Algunas de las utilidades que representa dicho paradigma.

– **Flexibilidad:** Partimos de la premisa que mediante la definición de clases instauramos módulos independientes, a partir de los cuales podemos definir nuevas clases, entonces podemos pensar en estos módulos como conjuntos de componentes con los cuales podemos construir diferentes programas.

– **Reusabilidad:** Desde el instante que hemos precisado a la entidad persona para utilizarla en una aplicación de negocios, por nombrar un ejemplo, y deseamos fabricar a continuación una aplicación, digamos de deporte, en donde requerimos precisar a la misma entidad persona, no es deseable volver a escribir la definición para la entidad persona. Por medio de la reusabilidad, podemos utilizar una clase definida previamente en las aplicaciones que nos sea conveniente. Es claro que la flexibilidad con la que se definió la clase va a ser fundamental para su reutilización.

– **Mantenibilidad:** Las clases que conforman una aplicación, vistas como módulos independientes entre sí, son fáciles de mantener sin afectar a los demás componentes de la aplicación. Si en la vida real el programa toma un tamaño considerable de clases y paquetes diferentes, será igual, fácil de mantener y administrar.

– **Extensibilidad:** Esta es una ventaja que gracias a la modularidad y a la herencia, una aplicación diseñada bajo el guía de la orientación a objetos puede ser cómodamente desplegable para cubrir necesidades de aumento de la aplicación.

### Desventajas

A pesar de que las ventajas de la programación orientada a objetos superan a las exclusiones de la misma, podemos hallar algunas facetas y apartados no queridos en ésta.

- **Limitaciones para el programador:** Aunque la tecnología orientada a objetos no es novedad, un gran porcentaje de programadores no poseen todavía grandes conocimientos de dicha tecnología. En otras palabras, la lógica de la programación estructurada sigue siendo predominante en la totalidad de los desarrolladores de software, en muchos casos es complicado pasar de un lenguaje a otros tras haber programado, después de haber revisado de forma breve los principios de la programación orientada a objetos, está muy claro que en ésta se requiere una lógica de pensamiento totalmente diferente a la lógica comúnmente utilizada para la programación estructurada. Lo bueno es que es fácil de aprender y cambiar de chip.

- **Tamaño muy grande en las aplicaciones resultantes:** Aunque la gran mayoría de los equipos incluso a nivel particular y de estudiantes, no solo en terreno industrial y de empresas, cuentan con grandes capacidades tanto de almacenamiento como de memoria lo suficientemente buena como para ejecutar la mayoría de las aplicaciones que puedan desarrollarse con la tecnología orientada a objetos, no obstante existen casos en los que lo anterior no se cumple. Una de las desventajas de la programación orientada a objetos es que cuando se heredan clases a partir de clases existentes se heredan de forma tácita todos los miembros de dicha clase aun cuando no todos se requieran, lo que produce aplicaciones muy grandes y enormes que no siempre encajan en los sistemas con los que se disponga.

- **Velocidad de ejecución:** Velocidad de ejecución: Esto tiene que ver, en cierto modo, con el punto anterior, una aplicación redundantemente pesada en muchas veces es más lenta de ejecutar que una aplicación conformada únicamente por los módulos ineludibles.

Podemos asegurar que los lenguajes de programación orientados a objetos tratan a los programas como conjuntos de objetos que se ayudan entre ellos para efectuar actividades, entendiendo como objeto a las entidades que contienen datos y permitiendo que los programas sean más fáciles de redactar, sustentar y reutilizar.

Por otra parte la POO programación orientada a objetos, representa un gran progreso en la programación, entre sus primacías más importante se hallan un notable aumento de la productividad del programador y de la robustez de los programas, personalmente el lenguaje orientado a objetos que más está creciendo es Eiffel, además de sus posibilidades como lenguaje de orientación a objetos, soporta la metodología de diseño por contratos. Otro lenguaje bastante bien diseñado y de uso en aumento es Ruby. También es recomendable, y muy simple para comenzar, el lenguaje Python.

### 8.1.1. Lenguajes de programación orientados a objetos

Un poco de historia sobre los lenguajes Oop, Simula en el año 1967 fue aceptado como el primer lenguaje que posee las características principales de un lenguaje orientado a objetos. Fue gestado para realizar programas de simulación, en donde los fines eran la representación de la información más importante. Otro lenguaje como Smalltalk entre 1972 y 1980 es tal vez el prototipo canónico, y con el que gran parte de la teoría de la programación orientada a objetos se ha gestado.

Entre los lenguajes orientados a objetos se destacan los siguientes:

| | | |
|---|---|---|
| ABAP | Eiffel | Ruby |
| ABL | Fortran 90/95 | Self |
| ActionScript | Java | Smalltalk |
| ActionScript 3 | JavaScript | Magik (SmallWorld) |
| Ada | Lexico | Vala |
| C++ | Objective–C | VB.NET |
| C# | Ocaml | Visual FoxPro |
| Clarion | Oz | Visual Basic 6.0 |
| Clipper | R | Visual DataFlex |
| D | Pauscal [en español] | Visual Objects |
| Object Pascal | Perl | XBase++ |
| Gambas | PHP | Lenguaje DRP |
| Genie | PowerBuilder | Scala |
| Harbour | Python | |

Muchos de estos lenguajes de programación no son considerados puros orientados a objetos, sino que son mezcla y fundidos de lenguajes que combinan la POO con otros paradigmas.

Un gran número de lenguajes ha contribuido a la proliferación de las reglas orientadas a objetos. Las versiones precedentes de Lisp en la década de los cincuenta y Simula en la década de los sesenta, sentaron las semillas para el nacimiento de Smalltalk que al comienzo de los setenta supuso el establecimiento definitivo de la filosofía orientada a objetos. Smalltalk–80, la versión más popular de Smalltalk, y la formación de extensiones orientada a objetos de dos lenguajes populares, C y Pascal (últimamente se ha adherido Visual Basic), han acelerado rápidamente la implantación de la Poo dentro de lo que hoy se conoce ingeniería del software.

Los lenguajes orientados a objetos (LOO) se dividen en dos grandes Grupos:

– Puros: Son los que solo permiten realizar programación orientada a objetos. Ej: smalltalk, Java, Eiffel y Actor principalmente.

– Hibridos: Son los que permiten la POO con la programacion estructurada y son aquellos que añaden las propiedades OO a las características intrínsecas de tipo procedimental. Entre los lenguajes orientados a objetos híbridos se destacan Objetive–C, Object–Pascal, Turbo Pascal (versiones 5.0, 6.0 y 7.0) y C++ (versiones Turbo C++, Microsoft C/C++ 7.0, Borland C++ 3.1, Zortech C++ 3.1, Visual C++, Symantec C++ 6.0, etc.).

Existen tres pilares en la Programación Orientada a Objetos (OOP), los cuales son:

– **Encapsulación:** cunado varios miembros de una clase, están ocultos al cliente y otros son visibles ya sean miembros dato o métodos. En este contexto, el cliente, el que utiliza la clase, es el programador. Es decir, algunos miembros pueden ser utilizados por el programador, mientras otros son usados internamente por los métodos de la clase.

– **Herencia:** si una clase X que expresa un conjunto de miembros dato y operaciones, puede ser la clase base de una clase Y. Si Y es una subclase de X, hereda los miembros dato y operaciones de su superclase, X.

– **Polimorfismo:** tenemos una clase que expresa un conjunto de operaciones, éstas pueden ser implementadas por esa misma clase, con lo que se dice que tienen método. Las clases cliente de esta clase base pueden implementar las operaciones previamente declaradas. Partiendo de esta premisa el cliente es una clase que provee su propia versión a la operación declarada en la clase base.

  Supongamos que tenemos dos clases, X e Y. X es la clase base, e Y es una subclase de X. X declara una operación Imprimir (), la implementa con un método, y la clase Y provee su propia versión del Imprimir (). Supongamos que tenemos un puntero o referencia a un objeto de clase X, sin embargo realmente ese puntero enfoca hacia un objeto de clase Y, y se realiza una invocación a la operación Imprimir (), el programa ejecuta el método dotado por Y, no por X.




La diferencia primordial entre la Programación Orientada a Objetos y la Programación Basada en Objetos, es que la última no posee la característica de herencia. Puede poseer encapsulación e realizar de alguna forma una suerte de polimorfismo, pero no tiene herencia.

El lenguaje más popular de esta situación es VBA, Visual Basic for Applications (previo a Visual Basic .NET). Este lenguaje es basado en objetos, ya que sus clases pueden utilizar encapsulación y polimorfismo, pero no la herencia. En vez de herencia, se utiliza el modelo de componentes, más conocido como COM (Component Object Model). Este modelo es una de las bases de la tecnología ActiveX.

### 8.1.2. Lenguajes de programación basados en objetos

Un objeto es un concepto que asocia datos y procesos a una misma entidad, conservando una fuerte relación entre las estructuras de datos, y las actuaciones en respuesta a mensajes, dejando visible sólo la interfaz de comunicación con el exterior. La aproximación orientada a objetos se caracteriza por la idea central, el objeto.

De esta forma, el software se organiza como una colección de objetos que representan abstracciones del mundo real. Podemos decir que todo lo que nos rodea es un objeto (cada mesa, cada compañero, cada monitor…).

La naturaleza del desarrollo orientado a objetos reside en la caracterización y la organización de entidades en el dominio de la aplicación, y no de la representación última en un lenguaje de programación.

Es una concepción independiente de cualquier lenguaje.

Se atienden propósitos de conceptos de máximo nivel, no implementaciones que corresponden a un nivel inferior. El paradigma es absolutamente aplicable a todas las fases y elementos que forman parte de la ingeniería del software: análisis, diseño, programación y bases de datos.

Como resultado, de poner esta forma de pensar en práctica es:

– Se halla la posibilidad de constituir directamente las entidades del mundo real en los escenarios informáticos, sin necesidad de falsearlas.

– Se facilita muchísimo la reutilización y modificaciones del software realizado y enormemente escalable.

– Es posible trabajar con entornos de desarrollo muy potentes y de todos los ámbitos industriales y sociales.

– Permite ampliamente trabajar en equipo, desarrollando en paralelo módulos para una aplicación con un esfuerzo mínimo en coordinación.

De esta manera y resumiendo un poco lo expuesto, el modelo orientado a objetos contribuye al diseño de algoritmos con una aportación importante, en cuanto a la abstracción, generalización e interacción, y en cuanto a la sencillez de los procedimientos de adaptación y de extensión de software que permite.

Entre las razones que hacen tan atractiva la orientación a objetos figuran:

– Sus conceptos sencillos concernientes a las entidades que aparecen en el mundo real. La sencillez del modelo que emplea los mismos componentes fundamentales para expresar de manera uniforme el análisis, el diseño y la implementación de un sistema. La enorme capacidad de ajuste e integración de sus modelos, que permite la reutilización y reformas diversas, incluso durante el proceso de desarrollo, y el mantenimiento. La posibilidad de aumentar las oportunidades de reutilización de componentes de software en proyectos distintos.

– Se conocen un grupo de puntos a tratar que ayuden de guía a la hora de componer un sistema empleando orientación a objetos:

– Identificar los objetos que intervienen en él.

– Unir en clases a todos los objetos que tengan características y comportamiento comunes.

– Identificar los datos y operaciones de cada una de las clases.

– Emparejar las relaciones que puedan existir entre las clases.

– Los lenguajes de programación que sí admiten herencia son los que apoyan la Programación Orientada a Objetos. Algunos de estos lenguajes son: C, Object Pascal (el que utiliza Borland Delphi), y todos los que están basados en la plataforma .NET.

## Visual Basic

El lenguaje de programación es el super utilizado **Basic**. Visual Basic es una magnífica herramienta de programación que permite construir aplicaciones propias para Windows. Pero solo para este entorno. Ni para Linux, Unix ú otros sistemas operativos. Este entorno de programación permite crear ventanas, botones, menús, acceso a bases de datos etc. de una forma fácil e intuitiva Visual Basic es un lenguaje de programación visual, también llamado lenguaje de cuarta generación.

Esto significa, que un gran número de tareas se ejecutan sin escribir código, simplemente con operaciones gráficas efectuadas con el ratón sobre la pantalla con entorno gráfico GUI.

Visual Basic es un programa basado en objetos, sin emabrgo no con orientación a objetos como C o Java.

La diferencia radica en que Visual Basic usa objetos con sus propiedades y métodos, pero está desprovisto de los componentes de herencia y polimorfismo propios de los verdaderos lenguajes orientados a objetos como Java y C.

Algunos componentes de este lenguaje como son los Formulario o fichas en Visual Basic es una ventana. Es también un componente, llamado contenedor porque contiene normalmente otros controles.

Control es cada uno de los elementos gráficos de una aplicación típica de Windows: ventanas (forms), cajas de diálogo, botones, barras de desplazamiento, etc.

## Objetos y propiedades

Dentro de Visual Basic hay componentes como son los formularios y los distintos controles que se pueden emplear, son realmente entidades genéricas de las que pueden existir y crearse varios ejemplares concretos en cada programa.

Aunque en Visual Basic se le puedan considerar a estos diferentes componentes genéricos que forman este lenguaje como clases y objetos, es un lenguaje más basado en objetos. En el paradigma de programación orientada a objetos en Visual Basic deberíamos denominarlo como lenguaje “basado en objetos”, se entiende como clase a estas entidades genéricas, mientras que se llama objeto a cada ejemplar de una clase determinada. El “TextBox” es una clase.

Los TextBox1, TextBox2, TextBox3 de mi código fuente, son objetos de la clase “TextBox”.

Cada objeto posee un grupo de propiedades y pueden asignarse en tiempo de diseño y en tiempo de ejecución.

En tiempo de diseño se establece la propiedad Name del Form, simplemente escribiendo en la ventana de propiedades. En tiempo de ejecución, se establece la propiedad Text de un TextBox, con una línea de código del tipo:

```
objRespuesta.Text = “buenos días “ + objNombre.Text
```

Para acceder a una propiedad en “tiempo de ejecución” sería de la siguiente manera:

```
NombreObjeto.NombrePropiedad
```

En este lenguaje un objeto posee también propiedades, tiene métodos funciones. Los métodos son acciones que podrían efectuar los objetos correspondientes.

Por ejemplo:

```
objNombre.SetFocus
```

En este lenguaje pueden existir métodos como “SetFocus” de la clase TextBox que realiza la acción de colocar el cursor en el objeto.

Se puede ver como la notación correspondiente a un método es la misma que para una propiedad. Como conclusión decir que el desarrollo con Visual Basic significa entre otras cosas, el desarrollo e implementación de las propiedades y métodos de los diferentes objetos controles.

### 8.1.3. Lenguajes de programación que utilizan objetos

La POO fragmenta a la generalidad de programadores en dos conjuntos:

- **Productores de componentes**. Son desarrolladores unidos y trabajando en equipo, que crean componentes, los someten a pruebas de depuración y proveen una interfaz de comunicación de forma que el producto llega al usuario en perfecto estado para su implementación. La interfaz proporcionada impide conocer el interior del componente, lo cual permite que los cambios internos que sufran no afecten a los consumidores de dichas componentes.
- **Consumidores de componentes**. El objetivo último del consumidor de componentes (programador) es conocer los diferentes módulos que existen en el mercado y usarlos convenientemente para conseguir el objeto deseado.




El paradigma de programación más moderno y como tendencia actual, es la Programación Orientada a Aspectos OPA. Es una de las últimas ideas en Informática y puede ayudar a traspasar la barrera de complejidad de las aplicaciones cuando se tratan ciertos aspectos del sistema como la seguridad, el control de errores y las transacciones.

#### La familia orientada a objetos

La filosofía del paradigma orientado a objetos envuelven todo el contexto de la programación, dando paso a los lenguajes, Smalltalk, Java C, C# además de que se han expandido a muchos ámbitos que integran el desarrollo de software.

En el campo de análisis y desarrollo de software, durante la década de los años 80, aparecieron distintas metodologías como: Jsd, Oosd, Omt. Posteriormente, ya en la década de los años 90, comenzaron a brotar emergentes metodologías de desarrollo orientado a objetos, descritas como de segunda generación, de las que forman parte Fusión, Ooa/d o Medea. En esta segunda generación se aprovechan las estructuras y sistemas de las primeras: identidades Crc, modelo de clases de Omt, para producir metodologías más completas donde se incorporan asimismo métricas y lenguajes formales de especificación.

Se puede entender el resurgimiento de una tercera generación, surgida en la segunda mitad de los años 90, que se caracteriza por la combinación de varios modelos existentes, Booch, Rumbaugh y Jacobson, derivando en UML (Lenguaje Unificado de Modelado), metodología establecida como estándar en 1997 y adoptada en Métrica 3.

En estos momentos, se comete un esfuerzo de unificación que ejerce primordialmente sobre las anotaciones de las técnicas utilizadas para el modelado orientado a objetos, consiguiendo espacio lo que se denomina técnicas de desarrollo ágil, en un intento de simplificar la gran complejidad y diversidad de metodologías existentes. En esta línea está el método XP de Programación extrema, Dsdm.

En el entorno de la arquitectura de sistemas de información y con el objetivo de facilitar el desarrollo de sistemas basados en arquitectura multi–nivel, surgen estándares como CORBA para la implementación de sistemas de objetos distribuidos, J2EE propuesta de Oracle para hacer más fácil el diseño, desarrollo, ensamblaje, distribución y despliegue de sistemas distribuidos multi–nivel basado en elementos, como el motor de desarrollo DCOM de Microsoft que posibilita comunicaciones entre objetos en diferentes ordenadores y distantes a través de Internet.

## 8.2. Comparación entre los lenguajes de programación orientados a objetos más habituales. Características esenciales

Comentando sobre diversos lenguajes de programación actuales:

### Ruby

Ruby es un lenguaje de programación ideado por el programador nipón Yukihiro Matsumoto: comenzó a trabajar en él en 1993 y lo presentó públicamente en 1995 como el mejor amigo del programador. Le puso el nombre de Ruby (en español rubí) como broma, aludiendo al lenguaje de programación Perl (perla).

Las bases que guiaron a su inventor fueron la productividad y la diversión para el programador y una buena interfaz de usuario, ya que consideraba que el diseño de programas necesita resaltar las necesidades humanas más que las de la máquina.

Ruby mezcla una estructura inspirada en Python y Perl, con características de programación orientada a objetos, donde la mayoria de las estructuras de datos son objetos, lo que quiere decir que se les pueden establecer propiedades y acciones a toda información y código.

Comparte igualmente operatividad con otros lenguajes de programación como Lisp, Lua, Dylan. Ruby es un lenguaje de programación interpretado y su implementación oficial es distribuida bajo licencia de software libre. Sigue el principio de la menor sorpresa, lo que simboliza que el lenguaje debe sobrellevarse de tal manera que minimice la confusión de los usuarios experimentados, simplificando el trabajo de programación.

Su desarrollo une partes de diferentes lenguajes: Perl, Smalltalk, Eiffel y Ada, para fundar un nuevo lenguaje que incorporara tanto la programación funcional de un paradigma de programación declarativa basado en la utilización de funciones aritméticas, como la programación imperativa que es un grupo de instrucciones que le indican a la ordenador cómo realizar una tarea. Ruby ha sido descripto como un lenguaje de programación multi–paradigma, lo que, como se ha dicho, quiere decir que es compatible con otros lenguajes.

Su formato de lenguaje y de estructura es parecido a Perl o Python, pero con la ventaja de ser más elástico, ya que soporta a sus usuarios modificarlo libremente. Las partes esenciales de Ruby pueden ser quitadas o redefinidas: se puede agregar una funcionalidad a partes ya existentes e incluso puede hacerse en cualquier otro lenguaje. Esta básicamente orientado a no restringir al desarrollador.

Este lenguaje cuenta con miles de seguidores en todo el planeta, por ser completamente libre y también por la fama alcanzada por sus aplicaciones, particularmente por la plataforma con la que cuenta para encaminar adelante desarrollos web con un mínimo de configuraciones. Si necesitas conocer cómo piensan los internautas de este lenguaje de programación, puedes entrar en su página www.ruby-lang.org, en la que encontrarás manuales, tutoriales y referencias que te serán útiles a la hora de iniciarte en este lenguaje denominado Ruby.

## Scratch

La inclinación a nivel universal es el desarrollo de lenguajes de programación cada vez más asequibles, fáciles de implementar y que potencien la cultura participativa, abierta y libre. Los impulsores de este movimiento sostienen que hace falta una mayor democratización de estos lenguajes para que cada usuario pueda adaptar los programas a sus propias necesidades. Pero para que esto sea posible es importante ampliar y mejorar el acceso a las tecnologías y lograr una mayor alfabetización en materia de programación.

Ellos aseguran que al igual que las matemáticas o los idiomas, los lenguajes de programación asimismo deben ser una enseñanza más en el currículo educativo: con ese espíritu nació el lenguaje de programación Scratch.

Scratch fue concebido como una herramienta didáctica destinada a enseñar programación en exclusivamente e informática en general a los alumnos de nivel primario y secundario.

Es una aplicación informática basada en la nube, aunque también se puede instalar en ordenador Linux, Mac y Windows, que permite que los niños exploren y experimenten con los conceptos de programación de ordenadoras por medio del uso de una sencilla interfaz gráfica. Si bien es aún un desarrollo bastante joven —apareció en el 2007—, ha tenido una gran popularidad. Fue desarrollado por The Lifelong Kindergarten group, un área dedicada a los proyectos educacionales del Mit, el Instituto de Tecnología de Massachussets, en los Estados Unidos. En Málaga hay un centro especializado en formación en Scratch es ICHTON y lleva varios años celebrando unos campamentos tecnológicos, de la mano del MIT en Estados Unidos como centro colaborador aquí en España.




Los desarrolladores de Scratch comprendieron que con la expansión de los ordenadores y sus aplicaciones, los alumnos se interesarían (cada vez en edades más tempranas) en la programación como medio creador de diferentes utilidades, aplicaciones y juegos.

Scratch es un lenguaje basado en objetos. Con Scratch, prácticamente todo se lleva a cabo interactuando el joven programador/a, interactivamente arrastrando y soltando elementos con el mouse, y modificando con el teclado únicamente valores numéricos o textos. Admite multimedia, eventos, sonidos, animaciones, y todo tipo de bucles, condicionales, sintaxis de lógica de conceptos de programación, variables, constantes, ciclos, temporizadores, contadores, retornos etc.

Es un software libre, que se puede instalar y redistribuir libremente en cualquier ordenador sin importar cuál sea su sistema operativo:

Windows, Mac OS X o Linux. Aunque es un proyecto de código abierto, su desarrollo es cerrado: el código fuente se consigue libremente pero la aplicación se desarrolla en el MIT.

El nombre con el que se fundó “Scratch” es relativo de la técnica de scratching (arañar), una técnica utilizada por los DJs de hip–hop y música electrónica, que consiste en mover un disco de vinilo hacia adelante y hacia atrás sobre el plato del tocadiscos para crear un efecto parecido al de rayar el disco, para construir ritmos y frases melódicas.

La semejanza con el scratching musical es la fácil reutilización de piezas: en Scratch todos los objetos, gráficos, sonidos y secuencias de comandos pueden ser cómodamente importados a un nuevo proyecto y combinados, permitiendo a los principiantes obtener resultados rápidos y motivándolos a nuevos intentos.

Scratch se utiliza en muchos países y en entornos diferentes, no obstante ha sido especialmente pensado para academias, locales, centros comunitarios y hogares. El público al que está dirigido comprende la franja de los 8 a los 16 años de edad, sin embargo es incluso utilizado por usuarios de otras edades por sus características amigables que facilitan el diseño de sobre todo juegos, historias interactivas, animaciones, música y arte, para publicar en la Web. Cuenta para ello con numerosos elementos disponibles como imágenes, sonidos y textos.

Igualmente admite incorporar las propias grabaciones o archivos de los usuarios. En el caso de los sonidos, cuenta con una grabadora incorporada en el entorno para poder registrarlo y luego utilizarlos para crear nuevos.

Scratch potencia el desarrollo del pensamiento creativo y del trabajo en equipo, ya que favorece el intercambio entre usuarios estimulando el aprendizaje colaborativo.

Algunas de las ventajas de Scratch son:

Se halla disponible en diferentes idiomas, entre ellos el español. La interfaz gráfica consiente crear y utilizar un escenario con múltiples fondos y objetos móviles programables.

En lugar de escribir instrucciones (códigos), permite programar con el mouse, arrastrando bloques autoencajables. Posibilita explorar principios básicos de programación sin las complicaciones de sintaxis que tienen otros lenguajes.

A medida que los usuarios crean sus programas, aprenden el manejo de conceptos como: control de flujo, iteración (repetición o ciclos), condicionales, procedimientos, hilos múltiples y eventos. Permite controlar y combinar diferentes formatos (gráficas, texto, sonido). Facilita la manipulación de objetos.

Para empezar a trabajar en Scratch solo hace falta ingresar en su página, suscribirse a la comunidad que ya cuenta con más de 1 millón de usuarios y ponerse a explorar. Pueden ingeniar sus propios proyectos o intervenir en alguno de los más de mil que se encuentran en desarrollo. Antes de hacerlo, les sugerimos leer la guía de inicio.

En la web puedes descubrir una sección exclusiva para profesores y educadores, con herramientas, artículos de interés y material educativo para iniciarse en este lenguaje y sus bondades.

### Rankin de lenguajes de programación en 2014

Según nuestro análisis los 10 lenguajes de programación más utilizados en el mundo en 2014 fueron:

1. C
2. Java
3. C++
4. Shell
5. Javascript
6. PHP
7. Perl
8. Python
9. SQL
10. C#

## Según Ranking Tiobe

En el ranking de mes de Septiembre de 2014, no obstante se mantiene a C y Java por encima en los primeros puestos. Tiobe expresa que en número generales alcanzan la cantidad más baja desde que se usa el índice. Motivo que se justifica por el uso cada vez más representativo de lenguajes de propósito específico en contraposición a estos lenguajes más generales.

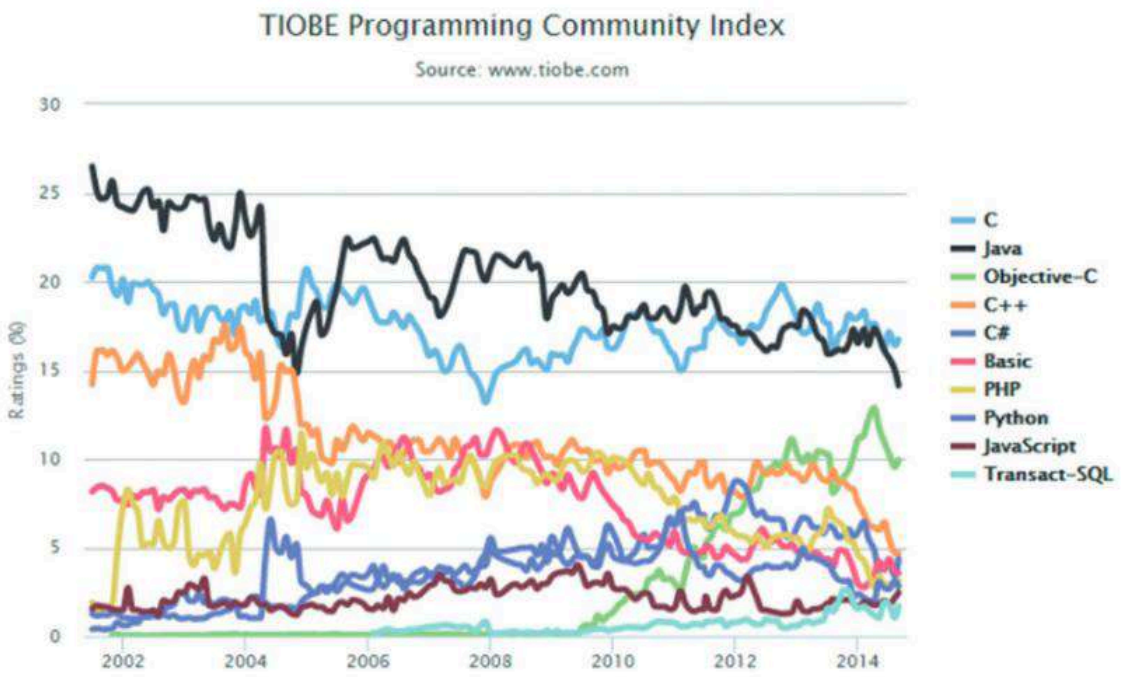



*Gráfico con el Ranking de lenguajes de programación según Tiobe en 2014*

El índice **Tiobe** es fundamentalmente un indicador de popularidad. No es estándar de facto en el tema de evaluar el interés por cada lenguaje, que lleva haciéndolo y perfeccionando su algoritmo desde 2001 con más de 150 lenguajes catalogados. Se fundamenta en cálculos sobre el número de programadores cualificados en cada lenguajes, oferta formativa y suministradores de terceros que trabajan sobre ellos. También tiene en cuenta las búsquedas relativas que se realizan en Google, Bing, Yahoo, Wikipedia, Amazon, Youtube y Baidu.

| Sep 2014 | Sep 2013 | Change | Programming Language | Ratings | Change |
|---|---|---|---|---|---|
| 1 | 1 | | C | 16.721% | -0.25% |
| 2 | 2 | | Java | 14.140% | -2.01% |
| 3 | 4 | ↑ | Objective-C | 9.935% | +1.37% |
| 4 | 3 | ↓ | C++ | 4.674% | -3.99% |
| 5 | 6 | ↑ | C# | 4.352% | -1.21% |
| 6 | 7 | ↑ | Basic | 3.547% | -1.29% |
| 7 | 5 | ↓ | PHP | 3.121% | -3.31% |
| 8 | 8 | | Python | 2.782% | -0.39% |
| 9 | 9 | | JavaScript | 2.448% | +0.43% |
| 10 | 10 | | Transact-SQL | 1.675% | -0.32% |
| 11 | 11 | | Visual Basic .NET | 1.532% | -0.31% |
| 12 | 12 | | Perl | 1.369% | -0.32% |
| 13 | 13 | | Ruby | 1.281% | -0.10% |
| 14 | - | ⇈ | Visual Basic | 1.272% | +1.27% |
| 15 | 14 | ↓ | Delphi/Object Pascal | 1.157% | +0.26% |
| 16 | 26 | ⇈ | F# | 0.990% | +0.49% |
| 17 | 15 | ↓ | Pascal | 0.893% | +0.01% |
| 18 | - | ⇈ | Swift | 0.852% | +0.85% |
| 19 | 19 | | MATLAB | 0.818% | +0.18% |
| 20 | 17 | ↓ | PL/SQL | 0.809% | +0.13% |

*Tabla con el Ranking de lenguajes de programación según Tiobe en 2014*

Para hacer un análisis correcto de estos puestos debemos tener en cuenta la antigüedad de cada uno de los lenguajes. Sin duda los más nuevos deberán contar con más tiempo de pruebas, de desarrollo y también de difusión. Y muy importante la usabilidad actual según programadores y usuarios.

Enfocaremos en solo dos lenguajes para compararlos, elegidos por su similitud y por ser actualmente de los más usados.

La sintaxis de Java resulta bastante familiar a los programadores de C, debido mayoritariamente a que Java proviene de C. Sin embargo Java pretende mejorar a C en muchos aspectos (sobre todo en los aspectos orientados a objeto del lenguaje), sin embargo prohíbe varias de las tareas por las que C fue tan extendido. Se observa que las diferencias han sido diseñadas como mejoras del lenguaje, ya que uno de los aspectos más reprochado (y defendido) de C es su capacidad para producir cosas “no orientadas a objetos”, así como acceder a los recursos de las máquinas (lo que le permitía infectar sistemas, siendo uno de los lenguajes más difundidos entre los programadores de virus). En este apéndice pretendemos enseñar aquellas diferencias significativas, para que los programadores familiarizados con C puedan desarrollar en Java, conociendo sus potenciales y limitaciones.

### Elementos similares

– Libro de bibliotecas (paquetes): Como no existen macros como #includes se utiliza import.

– Operador de herencia: La herencia en Java se especifica con la palabra clave extends,en lugar del operador:: de C.

– Constantes: En lugar de const de C las variables constantes se declaran como static y final.

### Elementos equivalentes

– Miembros estáticos: No es necesaria declaración previa (fuera de las clases) de los miembros estáticos (static).

– Métodos inline:En Java no existen por ello hay que meter el cuerpo de los métodos junto a su definición, no obstante los compiladores Java suelen intentar expandir en línea, como si fueran métodos inline los métodos declarados como final.

– Métodos virtuales: Todos los métodos no estáticos (static) se consideran virtual en Java.

– Forward: No existe en Java. Sencillamente se llama el método, y el compilador se encarga de averiguarlo.

– Clases anidadas: Aunque Java no permite anidar clases, sí que se puede modelizar este concepto mediante los paquetes Java y la composición.

– No existen “amigos” (friend): Es su lugar se considera amigas a todas las clases y elementos que componen un paquete.

### Elementos anidados

– Comentarios: Java soporta los dos tipos de comentarios de C, e incorpora un tercero, con la sintaxis /**comentario */, para la documentación automática.

– Operador >>>: Java añade este operador para desplazamientos a la derecha con signo.

– Iniciación: Con las variables de tipo simple, y las referencias a objeto se reinician a un valor 0 semejante, y null en las referencias a objeto.

– Referencia super: En Java super hace referencia a la superclase (clase padre) de la clase actual. Dicha clase solamente puede ser una, porque Java solo soporta herencia simple.

### Elementos suprimidos

- goto: No existe en Java, aunque con break y continue, fundidos con etiquetas de conjunto, se puede suplir.
- Condiciones: Deben utilizarse expresiones booleanas y nunca números.
- Argumentos por defecto: Java no los permite.

### Diferencias por el tipo de diseño entre Java y C

#### Tipos de datos

- Tipos simples: permite los mismos que C, sumando boolean (true/false), y mejorando el tipo char, para soportar caracteres Unicode de 16 bits.
- Punteros: En Java no existen punteros, admitiendo así programación segura. En su posición se permiten las referencias a objeto, que pueden ser reasignadas igual que si fueran un puntero a objeto de C.
- Vectores: Son objetos de sólo lectura, con un método length() para devolver su longitud, y que envian una excepción por si se prentede acceder a un elemento fuera del vector.
- Clases: Todo debe de estar incluído en clases; no existen enumeraciones (enum) ni registros (struct).
- Elementos globales: No existen variables o funciones globales, aunque se puede utilizar static para simularlas.

#### Diseño de las clases

- Cuerpo de los métodos: Las estructuras de las clases deben de estar codificados en las definiciones de las clases. No se pueden separa como se hace en C mediante ficheros de cabecera (“h”).
- Constructores: Aunque existen constructores, como en C, no existen constructores copia, puesto que los argumentos son pasados por referencia.
- Destructores: No existen destructores en Java, puesto que tiene recolección automática de basura, sin embargo en su lugar se pueden registrar métodos finalize(), que serán ejecutados cuando el recolector de basura de Java destruya el objeto.

– Arbol de herencia: En Java todas las clases se relacionan en un único árbol de herencia, en cuya cúspide se encuentra la clase Object, con lo que todas las clases heredan de ella. En C sin embargo se pueden declarar clases que no tengan padre.

– Herencia no restrictiva: Al heredar, no se puede rebajar las ocultaciones del padre: En C sí se podría ampliar la visibilidad de uno de los elementos heredados. En todo caso sí se puede limitar.

– Herencia simple de clases: No existe la herencia múltiple de clases. Aún así se puede implementar una herencia múltiple utilizando interfaces, ya que ellas sí la soportan. Sobrecarga de métodos: Es precisamente igual que la de C.

– Sobrecarga de operadores: No existe. En los objetos String el operador + y += se permiten para la comparación de cadenas.

### Nuevos diseños

– Plantillas (Templates): En Java no se soportan plantillas p clases genéricas de C, aunque existen una serie de clases en la API de Java que tratan objetos genéricos (clase Object) como Vector o Stack.

– Interfaces (interface): Son como clases abstractas con métodos abstractos, y que soportan herencia múltiple, usando el término reservado implements.

– Diferente gestión de excepciones: Todas las excepciones de Java heredan de la clase “Throwable”, que las dota de una interfaz común.

### Diferencia de ejecución

**Aspectos modificados:**

– Cadenas: Las cadenas entrecomilladas se convierten en objetos String, no en vectores estáticos de caracteres.

– Instanciación: Los objetos se crean en el método (new) y en absoluto en la pila (malloc), y los arquetipos simples no permiten new ,excepto los vectores de tipos simples, iguales que los de C.

– Intérprete: Los intérpretes Java son más lentos que los de C, aunque esta diferencia se está reduciendo con lo compiladores JIT(Just In Time) para Java que están apareciendo en el mercado.

– Biblioteca estándar: En C existían librerías de plataformas para ejecutar cosas como, trabajo en red, conexión a bases de datos, uso de diferentes hilos de control, uso de objetos distribuídos o compresión. Java incorpora bibliotecas estándar multiplataforma para todas estas labores en su Api.

– Excepciones por fallos de descriptores: Java lanza excepciones cuando hay errores en el acceso a un descriptor, permitiendo al programador gestionar dichos errores, y recuperar al programa de ellos. Gestión de errores al compilar: Además revisar el lanzamiento de excepciones en momento de compilación, comprueba el cumplimiento del lanzamiento de excepciones por los métodos sobreescritos.

### Aspectos eliminados

– Preprocesador: en Java no existe preprocesador, y las macros (#include, #define…) como en C no existen.

– Acceso directo al hardware: En Java es restringido, para eso permite la utilización de métodos nativos, escritos para la plataforma normalmente C. Por ello los applets no pueden usar estos métodos nativos, sólo las aplicaciones Java pueden hacerlo.

### Aspectos introducidos

– Multiples hilos de control (multithreading): Java soporta el uso de múltiples hilos de control y la ejecución en paralelo (y sincronizada) de múltiples tareas, mediante la clase Thread.

– Applets Java: Este tipo de programas son seguros, distribuíbles por Internet y ejecutables por los navegadores, aunque tienen restricciones (como la escritura en disco).

– Extracción automática de documentación: Un nuevo tipo de comentario (/** com_doc*/) deja a los programadores extraer de manera automática comentarios de sus fuentes, generando automáticamente documentación estandarizada.

– JavaBeans: Gracias a esta biblioteca se permite crear elementos visuales multi–plataforma, algo impensable en C.

En este apartado se va a confrontar Java con otros lenguajes de programación orientados a objeto.

## Comparación de los tipos de datos

### Tipos de datos simples (primitivos)

Java es similar a C en la estructura de tipos de datos elementales con algunas livianas modificaciones.

En Java se diferencia claramente los tipos de datos primitivos y clases, aunque existen unas clases especiales wrappers que pueden modificar los tipos de datos primitivos.

Los tipos de datos primitivos (o simples) pueden ser numéricos, booleanos o caracteres.

### Datos numéricos

Existen cuatro tipos de datos numéricos: byte de 1 byte, short de 2 bytes, int de 4 bytes, y los long de 8 bytes. El tipo más corriente y usado de los cuatro es el tipo int. El byte viene a sustituir el tipo char de C, ya que Java introduce una interpretación diferente al tipo de datos char.

Las principales diferencias con C son:

– No se halla un tipo sin signo “unsigned” para los números en Java.

– Los tipos numéricos reales son el float 8 bytes y el double 16 bytes.

– Los números que utilizan coma flotante por ejemplo 213.934566 son considerados double por defecto, y habrá que realiza un moldeado casting expresamente para que sean float.

### Caracteres

Los formatos de datos tipo carácter en Java se apoyan en los de C que a su vez son heredados de C. Los caracteres son Unicode de 2 bytes. Los caracteres Unicode son valores de 2 bytes sin signo, con lo que se puede obtener hasta un rango de 65535 caracteres diferentes, que son convenientes para los diferentes lenguajes y sistemas de representación universales. El carácter como tipo de datos del lenguaje Java proviene del tradicional C. Hay que señalar que los caracteres en C eran de tan solo 1 byte, con lo que en Java podremos representar muchos más caracteres que en C.

### Datos Booleanos

En Java se crear y utilizan para las variables con valores Verdadero/Falso o Sí/No, en definitiva, valores con dos estados posibles. Una variable booleana puede poseer los valores true verdadero o false falso. Son similares a los de C, sin embargo, y a diferencia de C estas variables no podrán convertirse a datos numéricos, y es un tipo de datos básico.

### Operadores relacionales y aritméticos

Se utilizan en Java los mismos operadores que C, con la variación de >>>.

Elemento entendido como desplazamiento sin signo y la utilización del operador _ para la concatenación de varias cadenas de caracteres.

### Vectores

Los vectores en Java, a diferencia de C, son una variedad de objetos. Un vector es un objeto real con una representación en tiempo real. Se pueden declarar y guardar vectores de cualquier tipo, y almacenar incluso vectores de vectores para granjearse matrices (vectores con varias dimensiones). En este último aspecto no existe diferencia con C.

### Cadenas

Son objetos del lenguaje Java, no existen como seudoarrays de caracteres cadenas como era el supuesto de lenguaje C. Concurren dos componentes de cadenas de objetos, las que se obtienen de la clase String, para cadenas de tan solo lectura y las que se adquieren de la clase Stringbuffer para cadenas que se pueden alterar.

### Otras características

En este apartado se va a comparar Java con los lenguajes C y Smalltalk como primer lenguaje que presentaba un modelo de objeto.

| Características | Java | SmallTalk | C |
|---|---|---|---|
| **Sencillez** | Si | Si | No |
| **Robustez** | Si | Si | No |
| **Seguridad** | Si | Un poco | No |
| **Interpretado** | Si | Si | No |
| **Dinamicidad** | Si | Si | No |
| **Portabilidad** | Si | Algo | No |

### Sencillez

Java tiene una simplicidad que no tiene C aunque sí Smalltalk. Porque una de las razones de la fundación de Java es la de conseguir un lenguaje parecido a C pero reduciendo los errores más comunes de la programación, algo que se logra con mucho éxito puesto que Java reduce un 50% los errores que se realicen en C entre los que destacan:

| |
|---|
| Borrado del mecanismo de punteros y de las referencias. |
| Ya no aparecen los registros struct, heredados del paradigma de estructuras. |
| Ya no se puede usar ni la definición de tipos typedef ni la de macros #define. |
| Ya no es necesario liberar memoria free o delete. |

De todas formas, lo que Java hace, en realidad, es la eliminación de palabras reservadas, y la utilización de un intérprete bastante pequeño.

### Robustez

Java ejecuta comprobaciones en busca de dificultades tanto en tiempo de compilación como en tiempo de ejecución, lo que hace que se detecten errores lo antes posible, habitualmente en el ciclo de desarrollo. Algunas de estas verificaciones que hacen que Java sea un lenguaje robusto son:

| |
|---|
| Comprobación del código de byte. |
| Control exaustivo de excepciones y errores. |
| Gestión de punteros y de límites de vectores. |

Se aprecia una clara diferencia con C lenguaje que no realiza ninguna de estas comprobaciones.

### Seguridad

En Java están prohibidos los accesos ilegales a memoria, algo que sí se permitía en C. Esto es algo muy importante puesto que este tipo de problema puede producir la propagación de virus y otras clases de programas dañinos por la red. El código Java pasa muchos tests antes de ejecutarse en un ordenador. El código se envia a través de un examinador de código de byte que evidencia el formato de los fragmentos de código y aplica un probador de teoremas para detectar fragmentos de código ilegal, código que falsea punteros, viola derechos de acceso sobre objetos o intenta cambiar el tipo o clase de un objeto.

Distintas enseñanzas que podemos tomar de los códigos de byte si traspasan la comprobación sin generar ningún mensaje de error son:

| |
|---|
| El código no generará desbordamiento de operandos en la pila. |
| El tipo de los parámetros de todos los códigos de operaciones es conocido y correcto. |
| No ha pasado ninguna conversión ilegal de datos, tal como convertir enteros en punteros. |
| Comprobación acceso a los campos de un objeto se sabe si es legal mediante las palabras reservadas public, prívate y protected. |
| No hay ningún intento de sobrepasar las reglas de acceso y seguridad establecidas. |

Por estas peculiaridades, y por no permitir Java la utilización de métodos de un programa sin los privilegios del kernel del sistema operativo, la obligatoriedad de autentificar por clave pública para la realización de modificaciones, por todo ello es considerado Java un lenguaje seguro. Todo esto no lo incorporan ni C ni Smalltalk, por lo que Java es de los tres considerado como seguro.

### Lenguaje interpretado

Java es un lenguaje en el que el código puede ser ejecutado directamente, por ello no es un lenguaje interpretado. Esto es una característica que sí que tiene Smalltalk, y no C. No obstante, y no obstante en teoría se consumen menos y hay una reducción de recursos estando los lenguajes interpretados, el actual compilador que existe es bastante lento, y menos rápido que C. Habitualmente no es vital para la aplicación tampoco demasiado apreciable por el usuario, y aún más esta diferencia se está reduciendo con los nuevos compiladores Jit (Just In Time).

### Dinamicidad

Para la consecución de un mayor provecho de la tecnología orientada a efectos, Java no intenta enlazar todos los módulos que comprenden una aplicación inclusive el tiempo de ejecución. Esta característica ya es contemplada por Smalltalk, pero no C, que enlaza todos los módulos cuando se compila.

### Portabilidad

Un programa Java puede ser ejecutado en diferentes entornos y ambientes operativos, algo inalcanzable para C.

## 8.3. Librerías de clases

En este apartado tocaremos diferentes ejercicios con librerías de clases muy conocidas y potentes como AWT.

AWT viene de X Window Toolkit para Java, donde X puede ser cualquier cosa: Abstract, Alternative, Awkward,; parece que Sun se decanta por Abstracto. Es una biblioteca de clases Java para el desarrollo de Interfaces de Usuario Gráficas. Es muy potente y utulizada en todo el mundo. La versión del AWT Sun proporcionado con el Jdk se desarrolló en solamente dos meses y es la parte más endeble de todo lo que representa Java como lenguaje. El entorno que ofrece es demasiado elemental, no se han tenido en cuenta las ideas de entornos gráficos innovadores. Quizá la presión de tener que lanzar algo urgentemente al mercado haya tenido mucho que ver en la pobreza de Awt.

Javasoft, se unió con otras empresas para mejorar la versión de AWT, y para asegurarse que los elementos que desarrolla para generar interfaces gráficas sean fácilmente transportables entre plataformas, se ha unido como decía anteriormente con IBM y Lighthouse Design para implantar un conjunto de clases que proporcionen una impresión visual atrayente y sean más fáciles de usar por el programador. Esta recopilación de clases son las Java Foundation Classes (Jfc), que están formadas por cinco juegos de clases, al menos hasta este momento: Awt, Java 2d, Accesibilidad, Arrastrar y Soltar y Swing.

AWT, contiene componentes del AWT que existían en la versión 1.1.2 del JDK y en los que se han incorporado en versiones posteriores:

– Java 2D es un grupo de clases gráficas.

– Accesibilidad, suministra clases para facilitar el uso de ordenadores y tecnología informática a disminuidos, como lupas de pantalla, y medios multimedia para personas con discapacidad.

– Tecnología Arrastrar y Soltar Drag and Drop, son clases en las que soporta Glasgow, que es la nueva generación de los JavaBeans.

– Swing, es la parte más importante y la que más desarrollada está en estos momentos. Proporciona una serie de elementos muy bien definidos y explicados de forma que su presentación visual es independiente de la plataforma en que se ejecute el applet o la aplicación que utilice estas clases. Swing simplemente extiende el AWT añadiendo un conjunto de componentes, JComponents, y sus clases correspondientes que son bastantes. Hay un bloque de componentes de Swing que son análogos a los de AWT, y algunos de ellos implementan la arquitectura MVC Modelo–Vista–Controlador, además Swing además proporciona otros widgets nuevos como árboles, pestañas, etc.

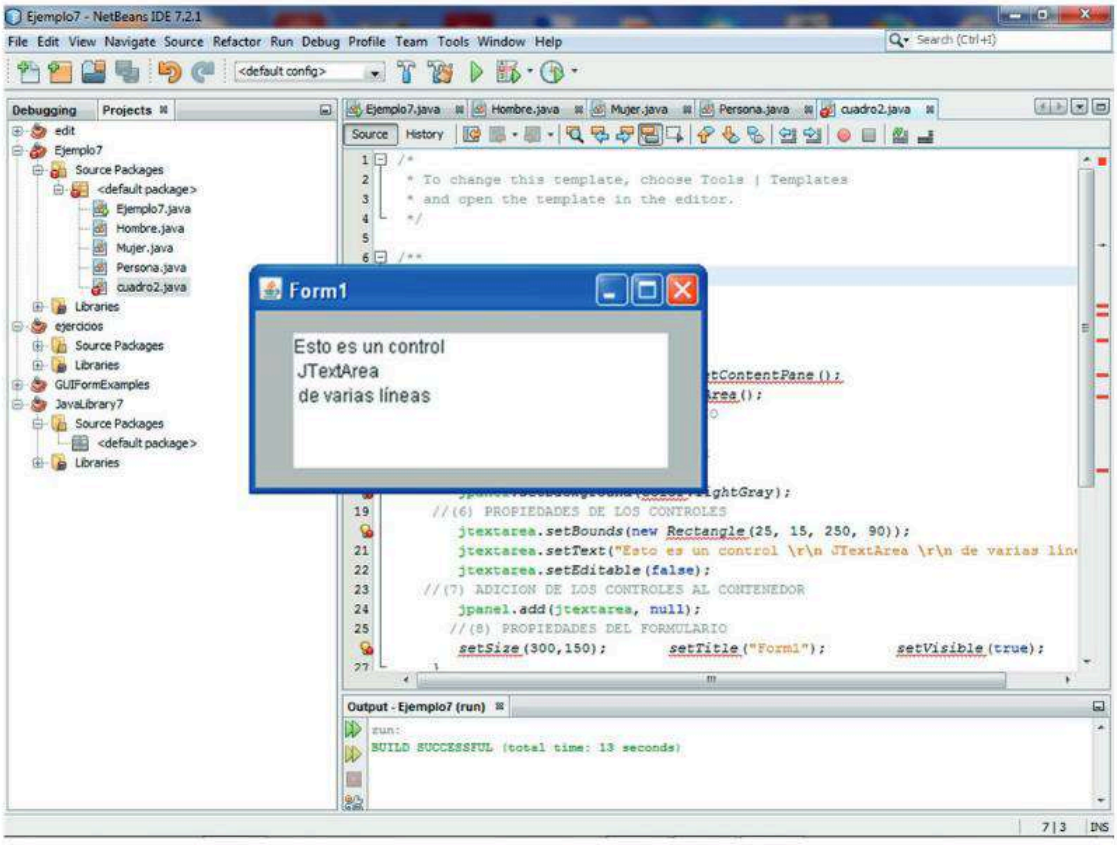
Sin duda este tipo de componentes de Java y otros muchos más que por extensión no se puede tocar en esta unidad, reportan al lenguaje unas potencialidades enormes y facilidad a la hora de programar con Java. Clases cada vez más documentadas, elementos gráficos e interfaz GUI de manejo con ratón muy interactivos y potentes, sin dejar atrás el componente industrial y profesional que Java reporta.

Como pequeño ejemplo de uso de esta librería, usaremos el componente JtextArea de demsotración:

JTextArea es un componente para usar modo gráfico de ventanas, que a diferencia de JTextField, este sólo nos permitía ingresar una línea de texto. Para mostrar su uso reproducimos en el siguiente código:

```
// paquete
import java.awt.*;
import javax.swing.*;
 // formulario
public class Aplicativo extends JFrame {
     // componentes del formulario
    JPanel jpanel = (JPanel) this.getContentPane();
    JTextArea jtextarea = new JTextArea();
     // constructor del formulario
    public Aplicativo() {
    // propiedades del contenedor
        jpanel.setLayout(null);
       jpanel.setBackground(Color.lightGray);
    // propiedades de los componentes
```

```
        jtextarea.setBounds(new Rectangle(30, 20, 255, 95));
        jtextarea.setText("Esto es un componente \r\n JTextArea \r\n de
varias líneas");
        jtextarea.setEditable(false);
    // adicion de los componentes al contenedor
        jpanel.add(jtextarea, null);
       // propiedades del formulario
        setSize(260,90);       setTitle("Formulario principal");
setVisible(true);
     }
    // metodos del formulario
    public static void main(String arg[]) {  new Aplicativo();  }
}
```




*Ejemplo en NetBeans de compilación de código con Paquete de librerías de Java con un JTextArea de AWT*

En este apartado vamos a añadir, que en lugar del componente JTextField estamos usando el componente JtextArea. Al tratarse de un componente para ingresar un texto de varias líneas, la forma de cambiar de línea es a través de los caracteres de escape \r\n, donde \r significa “ir al inicio de la línea” y \n significa “cambio de línea”. Esto se puede observar en la instrucción siguiente: jtextarea.setText(“Esto es un componente \r\n JTextArea \r\n de varias líneas”);

El método setText, muestra el texto en el componente JTextArea.

Indicar que si deseamos capturar el texto ingresado en un componente JTextArea se puede usar el método getText, igual que para el componente JTextField. Se utiliza de la siguiente manera:

```
String s = jtextarea.getText().
String s = jtexfield.getText().
```

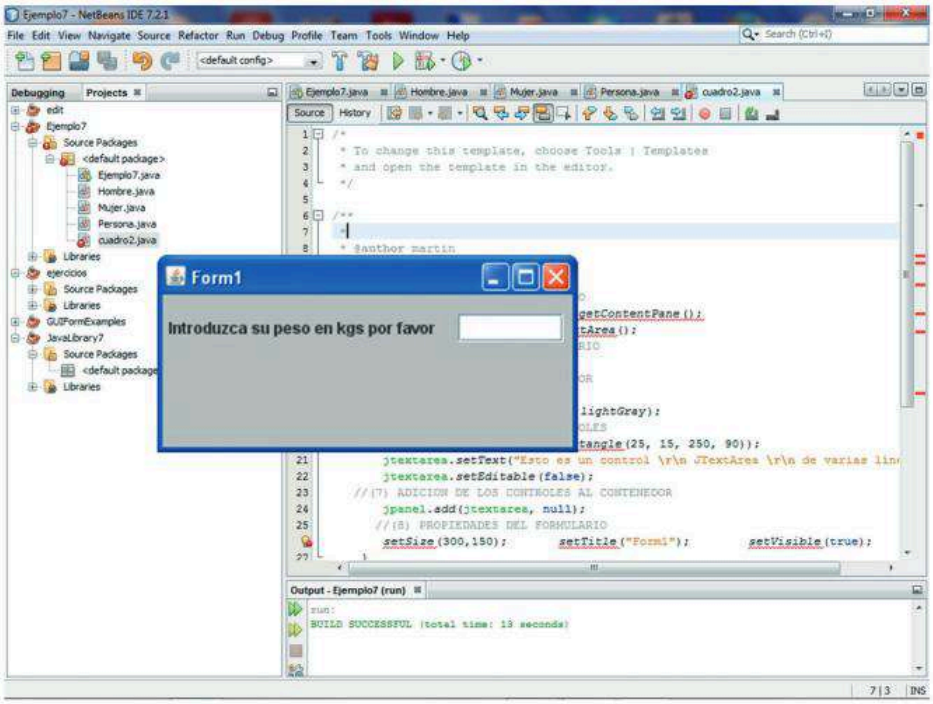
### Instrucción mostrar con Java. JLabels

Un JLabel sirve para insertar etiquetas que el usuario no podrá modificar en un formulario. Ejemplo:

```
// paquete
import java.awt.*;
import javax.swing.*;
 // formulario
public class Aplicativo extend s JFrame {
    // componentes del formulario
    JPanel jpanel = (JPanel) this.getContentPane();
    JLabel jlabel = new JLabel();
    JTextField jtextfield = new JTextField();
     // constructor del formulario
    public Aplicativo() {
        // propiedades del contenedor
        jpanel.setLayout(null);
       jpanel.setBackground(Color.lightGray);
```

```
        // propiedades de los componentes
         jlabel.setBounds(new Rectangle(15, 25, 230, 31));
         jlabel.setText("Introduzca su peso en kgs por favor");
         jtextfield.setBounds(new Rectangle(235, 25, 90, 31));
       // adicion de los componentes al contenedor
         jpanel.add(jlabel, null);
         jpanel.add(jtextfield, null);
       // propiedades del formulario
         setSize(260,90);        setTitle("Formulario Principal");
setVisible(true);
    }
    // metodos del formulario
    public static void main(String arg[]) { new Aplicativo();  }
}
```

El resultado que obtendremos es algo así:




*Ejemplo en NetBeans de compilación de código con Paquete de librerías de Java con un Jlabel de AWT*

En cuanto a las propiedades de los Jlabels, su manejo es esencialmente semejante que el de los Jtextfield. Mostraremos a continuación una relación de métodos propios de un Jlabel. Estos procedimientos se usan para transformar las propiedades de los Jlabels. Algunas están plasmadas en el ejemplo enseñado arriba y otras no. Vamos a ver un ejemplo para lograr un uso adecuado:

- jlabel.setFont (new java.awt.Font("Dialog", 3, 16)); El método setFont se usa para cambiar la fuente de letra a emplear. Si nos fijamos se le pasa el objeto new Java.awt.Font("Dialog",3,16) , que representa a la fuente de letra. El valor "Dialog" que se pasa como argumento es el nombre de la fuente, que podemos cambiar a cualquier otro tipo de fuente como "Verdana", "Arial" etc. El valor 1 indica el estilo negrita para la fuente, podemos cambiarla a 2 estilo normal o 3 estilo cursiva. El valor 16 se refiere al tamaño de la letra que podemos también cambiar a nuestro gusto para que sea más grande o más pequeña.

- jlabel.setText ("Inserta un entero positivo"); El método setText se usa para asignar el texto que va a mostrar la etiqueta JLabel. Observamos que el texto se le pasa como argumento.

- jlabel.setBounds (new Rectangle(140, 159, 135, 30)); El método setBounds se usa para posicionar el componente JLabel en una posición determinada dentro del formulario. Esta posición está indicada por el objeto new Rectangle(140, 159, 135, 30), que se le pasa como argumento, donde 140, 159 son respectivamente las coordenadas x,y dentro del formulario y 135 es el ancho y 30 la altura del componente JLabel.

- jlabel.setForeground (Color.red); El método setForeground se utiliza para modificar el color de letra del texto empleado en la etiqueta. El argumento Color.red indica el color del texto que en este caso es rojo, pero podemos modificar por otro color, como Color.green para verde, Color.black seria negro, Color.white si lo quereos en blanco, etc.

- jlabel.setHorizontalAlignment (SwingConstants.right); El método setHorizontalAlignment se utiliza en la alineación del texto internamente en el componente Jlabel. El argumento SwingConstants.right indica que el texto se debe alinear a la derecha, SwingConstants.left sería para alinear a la izquierda y SwingConstants.center alineación central.

- jlabel.setBorder (BorderFactory.createLineBorder(Color.red, 2)); El método setBorder se utiliza para colocar un borde a la etiqueta JLabel. Las propiedades del borde están reflejadas por el objeto BorderFactory. createLineBorder(Color.red, 2), que se le envia como argumento. Color. red indica el color rojo del borde y 2 es el grosor del borde. Contra mayor sea el valor más ancho será el borde.

Podemos escribir un Label y establecer distintos mensajes y propiedades.

Para mostrar un mensaje compuesto por distintas partes en un JLabel, usaremos el símbolo de concatenación +. Como la siguiente sintaxis:

```
jlabel.setText(mensage1 + mensage2);
```

El símbolo + permite que se lance el contenido insertado en las variables mensage1 y mensage2.

Otra forma interesante de mostrar datos es a través de la ventana de Dialogo:

```
JOptionPane.showMessageDialog(ventana,"Mensaje en
pantalla","Titulo Principal" ,JOptionPane.information_message);
```

En el momento que se llama a la función JOptionPane.showMessageDialog, se muestra una ventana independiente donde aparece un mensaje. Igualmente podemos configurar JOptionPane para que aparezcan distintos botones. Por ejemplo, si el mensaje es: "¿Quiere Cancelar?" los botones podrían ser "Sí", "No", o "Aceptar" y "Cancelar". Permiten múltiples posibilidades como dejar que el usuario escriba o seleccione un valor de un combo.

Otro ejemplo con AWT:

Los objetos Frame son muy usados en el diseño de GUI:

```
import java.awt.*;
/* Frame vacio. Para finalizar pulsar Ctrl-C */
public class CuadroCentral extends Frame {
public static void main(String args[]) {
CuadroCentral sf = new CuadroCentral();
sf.setSize(150, 150);
sf.setVisible(true);
}
}
```

## Tipos de componentes

En el siguiente árbol se facilita la relación que hay entre todas las clases que tiene AWT para crear interfaces de usuario, asignando la jerarquía de Clases e Interfaces:

| Clases |
| --- |
| Adjustable |
| BorderLayout |
| CardLayout |
| CheckboxGroup |
| Color |

| Clases | | |
|---|---|---|
| Component | Button | |
| | Canvas | |
| | Checkbox | |
| | Choice | |
| | Container | |
| | Panel | Applet |
| | ScrollPane | |
| | Window | Dialog |
| | | FileDialog |
| | | Frame |
| | Label | |
| | List | |
| | Scrollbar | |
| | TextComponent | |
| | TextArea | |
| | TextField | |
| Cursor | | |
| Dimension | | |
| Event | | |
| FlowLayout | | |
| Font | | |
| FontMetrics | | |

| Clases | | |
|---|---|---|
| Graphics | | |
| GridLayout | | |
| GridBagConstraints | | |
| GridBagLayout | | |
| Image | | |
| Insets | | |
| MediaTracker | | |
| MenuComponent | MenuBar | |
| | MenuItem | |
| | CheckboxMenuItem | |
| | Menu | PopMenu |
| MenuShorcut | | |
| Point | | |
| Polygon | | |
| PrintJob | | |
| Rectangle | | |
| Toolkit | | |

| Interfaces |
|---|
| LayoutManager |
| LayoutManager2 |
| MenuContainer |
| Shape |

### 8.3.1. Definición de su estructura

Para definir la estructura de un paquete de clases, veo oportuno comentar el paquete java.util, es uno de los principales paquetes que usaremos cuando estemos programando con el lenguaje Java, y junto al paquete java.lang es uno de los más utilizados en cualquier tipo de desarrollo informático basado en Java. Este paquete, sobre el que hablaremos a continuación, es un conjunto bastante grande tanto de interfaces como de clases de diversos tipos, que en conjunto podrían llamarse "utilidades".

#### Descripción del paquete java.util

En principio el paquete java.util engloba un gran volumen de clases e interfaces, algunos son muy importantes y usados en base a nuestra experiencia. Evidentemente podremos aprovechar la documentación oficial (instalada en nuestro equipo o disponible vía internet) para poder consultar por nuestra cuenta el resto de interfaces y clases del paquete que deseemos.

Podemos describir brevemente cada interface en Java y el conjunto de clases para implementar según nos hagan falta.

Las principales interfaces serán:

| Interfaces en java.util |
|---|
| Collection |
| Comparator |
| Iterator |
| List |
| Map |
| Set |
| SortedMap |
| SortedSet |

Mientras que las principales clases serán:

| Clases en java.util |
| --- |
| Calendar |
| Date |
| HashMap |
| HashSet |
| LinkedList |
| StringTokenizer |
| TreeMap |
| TreeSet |

### Breve descripción de las interfaces de java.util

A continuación comentaré resumidamente cada una de las interfaces descritas.

#### Collection

Algunas colecciones pueden admitir duplicados de elementos dentro de ellas, otras no. Algunas colecciones pueden tener los elementos ordenados, otras no. Este interfaz es la raíz de todas las interfaces relacionadas con colecciones de elementos. El Java Development Kit no proporciona ninguna implementación de esta interfaz y son respectivamente sus sub–interfaces las que efectuarán sus métodos como son por ejemplo la interfaz Set o List.

#### Comparator

Muy parecida a la interfaz Comparable del paquete java.lang, recordar que definía el método “compareTo” nos implementa métodos para poder comparar 2 elementos y poder ordenarlos los métodos compare y equals.

### Iterator

Nos explica los métodos que tenemos que implementar para ejecutar un iterator sobre una colección de elementos. Este nos obligaba a poseer un método llamado iterator(), que nos devolvía un objeto de la interfaz Iterator. Debemos entender y recordar como elemento para recorrer objetos internamente, incluidos arrays de elementos.

### List

Nos permite definir el comportamiento de una lista de elementos. Esta lista habitualmente permite elementos duplicados o iguales y se accede principalmente por su posición o por su secuencia.

### Map, set, sortedmap y sortedset

Un mapa no puede albergar elementos duplicados, para un mismo elemento con una determinada clave, el componente siempre tendrá que devolver el mismo valor. La interfaz Map, nos definirá el comportamiento de un objeto que mapea o asigna a una clave un determinado valor.

### SortedMap

Parecida a esta interfaz es SortedMap, únicamente cambian en que los elementos del mapa están ordenados, haciendo por tanto en principio que las consultas o búsquedas sean más rápidas y eficientes que si no estuvieran ordenados.

La interfaz Set es la que nos definirá un conjunto de elementos. Del mismo modo que la anterior no admite elementos duplicados, ni nulos. Y al igual que la anterior también tenemos la versión SortedSet que es un grupo de elementos, pero ordenados, con las ventajas que aporta la ordenación que hemos comentado anteriormente.

Sobre las principales interfaces del paquete java.util, diré que se trata de un conjunto de interfaces que nos van a permitir hacer programas bastante más avanzados, ya que nos facilitan por ejemplo el tener un conjunto de elementos ordenados, una lista de usuarios donde podemos consultar sus nombres por ejemplo, o un mapa donde a cada usuario lo podemos localizar por una clave de identificación.

Breve descripción de las principales clases en el paquete java.util

### Calendar

Es una clase abstracta, necesariamente tenemos que implementar diversos métodos para construir el calendario, pero normalmente se utiliza la subclase de esta GregorianCalendar para utilizar un calendario gregoriano, que es el calendario de uso más habitual en la mayoría de países. En el ejercicio usamos Calendar:

```
Calendar hoy = Calendar.getInstance();
System.out.println("Hoy es… " + hoy.getTime());
```

### Date

Es una clase que permite reflejar un momento dado en el tiempo con precisión de milisegundos.

### Hashmap, hashset, treemap y treeset

Son las principales clases que respectivamente implementan las interfaces para Map, Set, SortedMap y SortedSet respecivamente.

### Linkedlist

Es una clase que incorpora la interfaz List junto con ArrayList. Es enlazada lo que permite el progreso por una lista tanto en un sentido como en otro, acceder al primer elemento y al último elemento directamente o la inclusión de elementos null.

### Stringtokenizer

Dentro del paquete java.util, es una clase muy utilizadas. Esta clase permite extraer un string en tokens. Un token es una porción de un string más grande, al cual le decimos por qué carácter debemos de fraccionarlo. Por ejemplo si tenemos el string "Un avión azul y rápido" y deseamos obtener los tokens separados por un espacio en blanco, tendríamos como resultado 5 tokens que son: "Un", "avión", "azul", "y", "rápido").

### Sobre las clases del paquete java.util

Indicar que el paquete java.util suministra un conjunto de clases muy interesantes y de gran potencialidad. Cuando se programa mucho nos damos cuenta de la gran herramienta que son las bibliotecas Java, agradecemos que facilitan muchísimo el desarrollo de código a los programadores. Disponer de buenas bibliotecas ha sido uno de los factores que ha contribuido al éxito mundial del lenguaje Java. Sobre todo Java permite que cualquier tipo de aplicación por grande que sea, que no hay que comenzar desde 0 a desarrollar el software, ya que disponemos de clases y librerías muy potentes para ayudarnos.

## 8.3.2. Creación y utilización

### Estructura de directorios para utilizar librerías (“packages”)

Para usar los “Packages” en Java es vital organizar en una estructura de directorios que lleven la misma secuencia de las librerías que queremos implementar, observar el diseño:




El esquema anterior demuestra una estructura para las librerías del dominio com.introvot, se debe mencionar que no existe ninguna restricción en el número de directorios anidados, esto es, es posible que existan librerías de 5 o 10 niveles de profundidad, está sub–clasificación se va realizando en base a los requerimientos de las librerías y es dentro de cada sub–directorio donde son colocadas las distintas Clases.

Además de la generación de estos directorios, se debe incluir un calificador de la librería en el código fuente de cada Clase:

```
package com.introvot.votaciones;
```

La linea anterior sería incluida en todos los programas pertenecientes a la librería ("Package") com.introvot.votaciones, debe mencionarse que este calificador debe ser la primer declaración en todo programa Java (Salvo no vaya a ser incluido en librerías); recuerde que además de este calificador el programa debe estar colocado en la estructura de directorios correspondiente, lo anterior es sumamente importante al llevarse a cabo la compilación para la generación de Byte–Code.

### Código fuente pantalla.Java

La siguiente Clase define dos métodos que pueden ser utilizados para imprimir resultados a pantalla:

La siguiente Clase debe ser colocada bajo la estructura de directorios com/introvot/votaciones, en base a su sistema operativo (Linux o Windows) por favor genere dicha estructura para colocar esta Clase:

```
package com.introvot.votaciones;

 public class Pantalla {

      public static void votoNulo(String s) {
        System.out.print(s);
       }

      public static void votoValido(String s) {
        System.out.println(s);
       }

   }
```

– Primeramente se declara la librería (“package”) a la que será asignada el programa a través del vocablo package.

– Se define la Clase con los respectivos métodos votoNulo y votoValido.

– Recuerda que este programa debe residir dentro del directorio com/intro-vot/votaciones/.

– Observa que esta Clase no posee el método principal main.

Para compilar esta Clase basta ejecutar el comando:

```
$ javac Pantalla.java
```

# UD8
## Lo más importante

- Con la POO tenemos que aprender a pensar las cosas de una manera distinta, para escribir nuestros programas en términos de objetos, propiedades, métodos y otras cosas que veremos rápidamente para aclarar conceptos y dar una pequeña base que permita soltarnos un poco con este tipo de programación.

- Un paradigma de programación representa un enfoque particular o filosofía para la construcción del software. No es mejor uno que otro, sino que cada uno tiene ventajas y desventajas. Dependiendo de la situación un paradigma resulta más apropiado que otro.

- Mantenibilidad: Las clases que conforman una aplicación, vistas como módulos independientes entre sí, son fáciles de mantener sin afectar a los demás componentes de la aplicación.

- Por medio de la reusabilidad podemos utilizar una clase definida previamente en las aplicaciones que nos sea conveniente. Es claro que la flexibilidad con la que se definió la clase va a ser fundamental para su reutilización.

- De esta forma, el software se organiza como una colección de objetos que representan abstracciones del mundo real. Podemos decir que todo lo que nos rodea es un objeto (cada mesa, cada compañero, cada monitor…).

- La diferencia entre la Programación Orientada a Objetos y la Programación Basada en Objetos, es que la última no posee la característica de herencia. Puede tener encapsulación e implementar de alguna forma una suerte de polimorfismo, pero no tiene herencia.

– Múltiples hilos de componente (multithreading) Java permite la utilización de múltiples hilos de componente y la ejecución en paralelo (y sincronizada) de múltiples tareas, mediante la clase Thread.

– Java realiza verificaciones en busca de problemas tanto en tiempo de compilación como en tiempo de ejecución, lo que hace que se detecten errores lo antes posible, normalmente en el ciclo de desarrollo.

– El código Java pasa muchos tests antes de ejecutarse en una máquina. El código se pasa a través de un verificador de "código de byte" que comprueba el formato de los fragmentos de código.

– El código no produce desbordamiento de operandos en la pila.

# UD8
## Autoevaluación

1. No es una cualidad de los lenguajes Orientados a Objetos:
   a. Herencia.
   b. Polimorfismo.
   c. Ocultación.
   d. Compactado.

2. ¿Cuál no es una ventaja de un lenguaje orientado a objetos?
   a. Relacionar el sistema al mundo real.
   b. Agiliza el desarrollo de software.
   c. Fomenta la reutilización y extensión del código.
   d. No permite crear entornos visuales y usables.

3. No es un lenguaje Orientado a Objetos:
   a. Pascal.
   b. Java.
   c. C++.
   d. Phyton.

4. Es un lenguaje puro de programación orientada a objetos:
   a. Visual C++.
   b. Ruby.
   c. Visual Foxpro.
   d. Java.

5. La diferencia entre la Programación Orientada a Objetos y la Programación Basada en Objetos:
   a. Encapsulamiento.
   b. La primera utiliza el modelo de componentes, más conocido como COM.
   c. Sus clases pueden utilizar encapsulación, polimorfismo y herencia.
   d. La última no posee la característica de herencia.

6. Que elementos No se tiene como similares entre Java y C?
   a. Libro de bibliotecas (paquetes).
   b. Existen macros en ambos.
   c. Operador de herencia.
   d. Constantes.

7. ¿Qué elementos No se tiene como equivalentes entre Java y C?
   a. Miembros estáticos.
   b. Métodos inline.
   c. Métodos virtuales.
   d. Forward existe en ambos.

8. No son Diferencias por el diseño de las clases entre Java y C:
   a. Cuerpo de los métodos.
   b. Constructores.
   c. Vectores.
   d. Arbol de herencia.

9. No son Diferencias de Ejecución entre Java y C:
    a. Cadenas.
    b. Instanciación.
    c. Preprocesador.
    d. Intérprete.

10. ¿Que características posee Java por encima frente a Smalltalk?
    a. Sencillez.
    b. Robustez.
    c. Seguridad.
    d. Interpretado.

Área: informática y comunicaciones

# UD9

## Implementación del paradigma utilizando un lenguaje de programación orientado a objetos

9.1. Elección del lenguaje

9.2. Enumeración de los tipos de aplicaciones

9.3. Herramientas de desarrollo

9.4. Tipos de datos y elementos básicos característicos del lenguaje. Instrucciones

9.5. Estudio y utilización de las clases básicas incluidas en la librería de clases

9.6. Definición de clases

9.7. Construcción de métodos. Sobrecarga

9.8. Construcción de atributos

9.9. Construcción de la interfaz de la clase

9.10. Construcción de clases incluyendo relaciones de agregación/composición y asociación

9.11. Construcción de clases con herencia

9.12. Construcción de clases con herencia múltiple

9.13. Definición de clases abstractas

9.14. Construcción de clases con herencia incluyendo polimorfismo

9.15. Empleo de excepciones

9.16. Gestión de eventos

9.16.1. Eventos, fuentes y auditores de eventos

9.16.2. Tipos de eventos. Mecanismos de gestión de eventos

9.16.3. Librerías de clases asociadas

9.17. Empleo de hilos

9.17.1. Fundamentos

9.17.2. Creación

9.17.3. Prioridad

9.17.4. Comunicación

9.17.5. Sincronización

9.17.6. Estados

9.17.7. Creación y ejecución de hilos en el lenguaje

9.17.8. Librerías de clases asociadas

9.17.9. Programación multihilo

9.18. Definición y análisis de programación en red

9.18.1. Aplicación cliente servidor

9.18.2. Sockets

9.19. Acceso a bases de datos desde las aplicaciones. Librerías de clases asociadas

## 9.1. Elección del lenguaje

### Pensar en objetos

¿Como poner el pensamiento enfocando en objetos?, puede ser y es una ardua tarea. Pero que sea complicado no quiere decirse que no se pueda, todo se consiguen con práctica y creando un hábito de pensamiento. Por el contrario, pensar en objetos representa la mayor sencillez que uno podría suponer del mundo de la programación. Pensar en objetos, es simple… aunque lo simple, no necesariamente signifique sencillo.

### Y ¿qué es un objeto?

Estamos rodeados de objetos en todos nuestros aspectos de la vida, todo son objetos. Cada rincón de nuestra casa son objetos, el mundo de la informática igual, todo son objetos dentro de esa pantalla de ordenador, es simple. Olvida los detalles formales como te decía en la informática y todo lo que nos rodea. Simplemente, olvida todo y concéntrate en lo que sigue. To explicaré de manera amena:

Un objeto es una cosa. Y, si una cosa es un sustantivo, entonces un objeto es un sustantivo.




Mira a tu alrededor y encontrarás decenas, cientos de objetos. Tu ordenador es un objeto. Tú, eres un objeto. Tu llave es un objeto. El ratón que mueves constantemente al lado del ordenador, es otro objeto. Tu mascota también es un objeto.

Piensa que cuando pensamos en objetos, todos los sustantivos son objetos.

Facil ¿Verdad?, solo concéntrate en este momento en pensar la vida en objetos.

Ahora ¿deberiamos comenzar a explicar las cualidades de un objeto?

Describir un objeto, es simplemente mencionar sus cualidades. Las cualidades son adjetivos. Si no sabes que es un adjetivo. Diremos brevemente que un adjetivo es una cualidad del sustantivo.

Entonces, para describir «la manera de ser» de un objeto, debemos preguntarnos ¿cómo es el objeto? Toda respuesta que comience por **“el objeto es”,** seguida de un adjetivo, será una cualidad del objeto.

Algunos ejemplos:

– **El objeto es** gris

– **El objeto es** grande

– **El objeto es** pesado

Ahora, imagina que te encuentras frente a tu primito de 7 años (primito: objeto niño que no hace más que preguntas de cosas que doy por entendidas de forma implícita). Y cada vez que le dices las cualidades de un objeto al molesto primito, éste te pregunta: —¿Qué es…? —seguido del adjetivo con el cuál finalizaste tu frase. Entonces, tu le respondes diciendo **“es un / una”** seguido de un sustantivo. Te lo muestro con un ejemplo:

– El objeto es gris. ¿Qué es gris? Un color.

– El objeto es grande. ¿Qué es grande? Un tamaño.

– El objeto es esbelto. ¿Qué es esbelto? Un aspecto.

Estos sustantivos que responden a la pregunta de tu primito, pueden pasar a formar parte de una locución adjetiva que especifique con mayor precisión, las descripciones anteriores.

– El objeto es **de color** gris.

– El objeto es **de tamaño** grande.

– El objeto es **de aspecto** esbelto.

Podemos asegurar entonces y todo esto, debido a tu primito de 7 años, que una peculiaridad, es un atributo (deducido de característica atribuible a un objeto) y que entonces, un objeto es un sustantivo que posee caracteres, cuyas peculiaridades lo describen.

Veámoslo más gráficamente:

| Objeto (sustantivo) | Atributo (locución adjetiva) | Cualidad del atributo (adjetivo) |
|---|---|---|
| (el) Objeto | (es de) color | Gris |
| (el) Objeto | (es de) tamaño | Grande |
| (el) Objeto | (es de) aspecto | Esbelto |

Pero algunos objetos, también se componen de otros objetos…

Además de peculiaridades (frase adjetiva consecuente de un adjetivo), los objetos tienen otras cosas. Estas otras cosas, son aquellas pseudo–cualidades que en sucesión de responder a ¿cómo es el objeto? responden a ¿cómo está compuesto el objeto? o inclusive, todavía más simple "¿qué tiene el objeto?".

La respuesta a esta pregunta, estará dada por la expresión "el objeto tiene…", seguida de un adverbio de cantidad (uno, multiples, abundantes, algunos, unas cuantas) y un sustantivo.

Algunos ejemplos:

– El objeto **tiene algunas** uñas

– El objeto **tiene una** trompa

– El objeto **tiene unos cuantos** colmillos

Los elementos del objeto, igualmente integran los atributos de ese objeto. Solo que estos atributos, son poco particulares: son otros objetos que poseen sus propias peculiaridades. Es decir, que estos atributos–objeto incluso responderán a la pregunta ¿cómo es/son ese/esos/esas? seguido del atributo–objeto (sustantivo).

Ampliemos el ejemplo para que se entienda mejor:

– El objeto tiene algunos colmillos. ¿Cómo son es**os** colmillos?

– Los colmillos son de color blanco

– Los colmillos son de longitud 150 cms

– El objeto tiene una trompa. ¿Cómo es es **a** trompa?

– La trompa es de grosor medio

– La trompa es de longitud 200 cms

– El objeto tiene unas cuantas orejas. ¿Cómo son es **as** orejas?

– Las orejas son de forma redondeada

– Las orejas son de tamaño pequeño

Pongámoslo más gráfico:

| Objeto (sustantivo) | Atributo–objeto (sustantivo) | Atributos (locución adjetiva) | Cualidades de los atributos (adjetivo) |
|---|---|---|---|
| (el) Objeto | (tiene algunas) orejas | (de) forma | Redondeada |
| (el) Objeto | (tiene algunas) orejas | (de) tamaño | Grande |
| (el) Objeto | (tiene unos cuantos) colmillos | (de) color | Blanco |
| (el) Objeto | (tiene unos cuantos) colmillos | (de) longitud | 150 cms |
| (el) Objeto | (tiene una) trompa | (de) grosor | Medio |
| (el) Objeto | (tiene una) trompa | (de) longitud | 200 cms |

Entonces, podemos deducir que un objeto puede recoger dos tipos de atributos:

– Los que responden a la pregunta ¿cómo es el objeto? con la expresión El objeto es… + adjetivo atributos definidos por características.

– Los que responden a la pregunta ¿qué tiene el objeto? con la expresión El objeto tiene… + sustantivo, que como decíamos anteriormente son atributos establecidos por las características de otro objeto. Veámoslo además, más gráficamente:

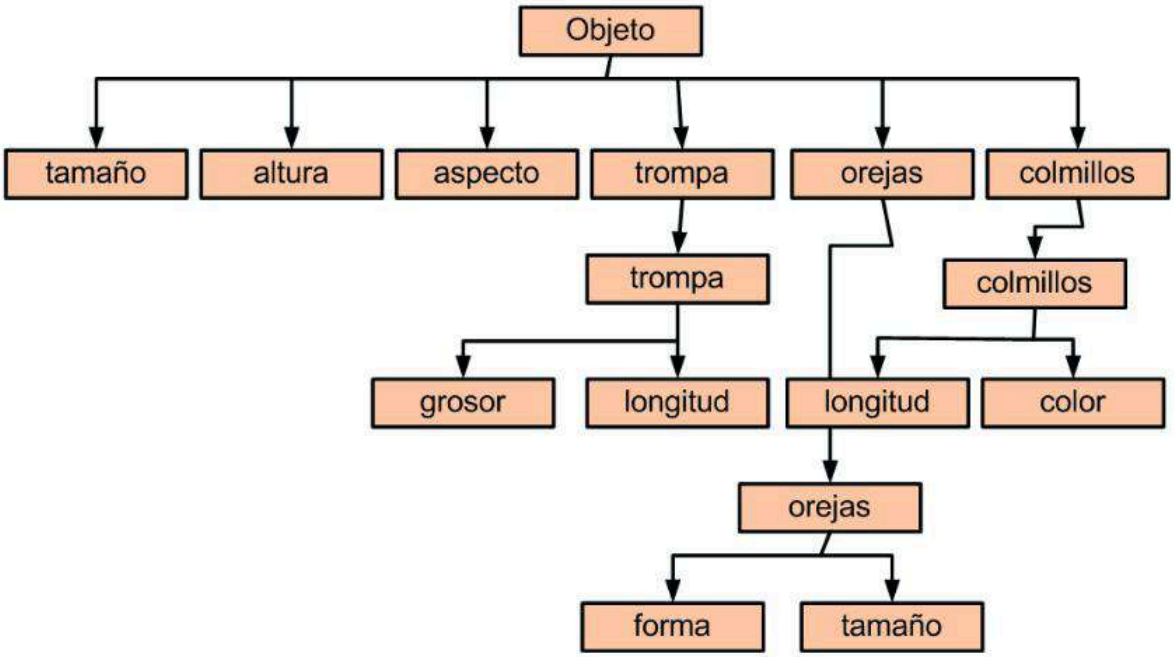



*Diagrama Objeto Elefante y sus atributos*

Viendo el gráfico anterior, tenemos lo siguiente: Un objeto (sustantivo) al cual hemos descrito con tres atributos (adjetivos) y otros tres atributos–objeto (sustantivos) los cuáles son a la vez, otros tres objetos (sustantivos) con sus atributos (adjetivos) correspondientes. ¿Es fácil verdad? Ahora, compliquemos todo un poco.

Igualmente hay objetos que comparten características con otros objetos. Como nuestro Objeto, es perfectamente semejante a un nuevo objeto. El nuevo objeto que estamos viendo, tiene totalmente todas las características que nuestro primer objeto, en otras palabras, tiene los mismos atributos. Pero aún, tiene algunas más. Por ejemplo, este nuevo objeto, igualmente de los atributos de nuestro primer objeto, tiene unas patas. Es decir, que las características de nuestro nuevo objeto, serán todas las del objeto original, más una nueva: patas. Repasemos las características de nuestro nuevo objeto:

– El nuevo objeto es **de color** gris.
– El nuevo objeto es **de tamaño** grande.
– El nuevo objeto es **de aspecto** esbelto.
– El nuevo objeto **tiene algunas** uñas
– El nuevo objeto **tiene una** trompa
– El nuevo objeto **tiene unos cuantos** colmillos
– El objeto tiene algunos colmillos. ¿Cómo son es**os** colmillos?
– Los colmillos son de color blanco
– Los colmillos son de longitud 150 cms
– El objeto tiene una trompa. ¿Cómo es es **a** trompa?
– La trompa es de grosor medio
– La trompa es de longitud 200 cms
– El objeto tiene unas cuantas orejas. ¿Cómo son es **as** orejas?
– Las orejas son de forma redondeada
– Las orejas son de tamaño pequeño

**(Nuevas características)**

– El nuevo objeto tiene unas patas. ¿Cómo son esas patas?
– La patas son de grosor mediaño
– La patas son de tamaño 150 cms
– Las patas tiene 3 uñas. ¿Cómo son esas uñas?
– Las uñas son de longitud 30 cms
– Las uñas son de forma alargada
– Las uñas son de grosor mediano

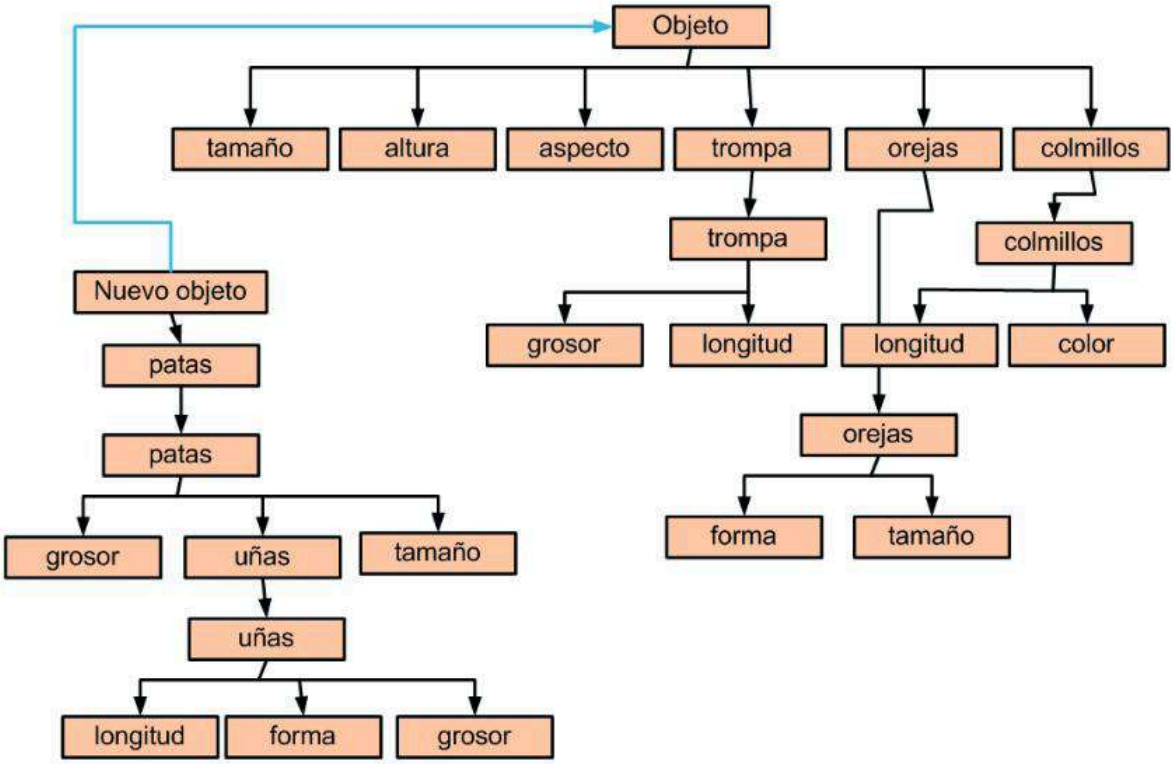



*Diagrama Objeto Elefante y sus atributos y objeto creado a partir de otro*

Veamos todas las características de este nuevo, en un gráfico.

Con mucha facilidad, hemos observado como nuestro nuevo objeto es una especie de objeto original ampliado. Hay que comentar que el nuevo objeto, es exactamente igual al objeto original (comparte todos sus atributos) pero posee nuevas características.

Está claro además, que el objeto original y el nuevo objeto, son dos objetos diferentes ¿cierto? No obstante, el nuevo objeto es un sub–tipo del objeto original.

Ahora sí, la dificultad ha crecido aún más.

Los objetos, también tienen la capacidad de hacer cosas

Hemos observado y explicado sobre las cualidades de nuestros objetos. Pero de lo que no hemos hablado, es de aquellas cosas que los objetos «pueden hacer», es decir, «cuáles son sus capacidades».

Los objetos tiene la capacidad de realizar acciones. Las actividades, son verbos. Es decir, que para saber las aptitudes de un objeto, debes preguntarte ¿qué puede producir el objeto? y la respuesta a esta pregunta, estará dada por todas aquellas que comiencen por la expresión “el objeto puede” seguida de un verbo en infinitivo.

Algunos ejemplos:

– El objeto original **puede** caminar

– El nuevo objeto (además) **puede** correr

Tendremos que añadir que acciones pueden hacer los objetos.

Si observas el gráfico anterior, notarás que el nuevo objeto, no solo tiene los mismos atributos que el objeto original, sino que igualmente, además puede llevar a cabo las mismas actividades que éste. Sencillo, cierto?

### Objetos y más propósitos: la parte difícil

Si entendiste todo lo anterior, inmediatamente viene la parte difícil. ¿viste que esto de pensar en objetos viene a colación de la programación orientada a objetos? Bueno, la parte difícil es que en la programación, todo lo que acabamos de contemplar, se denomina de una forma particular. Pero, la exposición es la misma que te di antes.

| Cuando en el documento… | En la programación se denomina… | Y con respecto a la programación orientada a objetos es… |
|---|---|---|
| Hablamos de objeto | Objeto | Un elemento |
| Hablamos de atributos (o cualidades) | Propiedades | Un elemento |
| Hablamos de acciones que puede realizar el objeto | Métodos | Un elemento |
| Hablamos de atributos–objeto | Composición | Una técnica |
| Vemos que los objetos relacionados entre sí, tienen nombres de atributos iguales (por ejemplo: color y tamaño) y sin embargo, pueden tener valores diferentes | Polimorfismo | Una característica |
| Hablamos de objetos que son sub–tipos (o ampliación) de otros | Herencia | Una característica |

Java es un lenguaje de tamaño reducido en cuanto que todo el paquete de desarrollo que incluirá bibliotecas de clases, los programas de compilación, interpretación, depuración, etc., aplicaciones. Mediante este pequeño software se proporciona diferentes funcionalidades facilitando:

– La comunicación.

– El acceso/proceso de la información.

– El desarrollo cognitivo.

– La realización de todo tipo de aprendizajes.

– La adaptación y autonomía ante el entorno.

– Ocio.

– Instrumentos de trabajo, posibilidades de realizar actividades laborales.

Java es un lenguaje portable. Y es un lenguaje de plataforma independiente tanto a nivel de código fuente como a nivel binario. Podemos redactar código Java en una tablada y marcharnos a otra con la garantía de que esta además entenderá el código sin necesidad de tener que reescribirlo.

Por otro lado, los ficheros binarios Java terminados de la compilación, conocidos por bytecodes, podrán ejecutarse desde cualquier plataforma sin necesidad de ninguna recopilación. Los bytecodes equivalen a código maquina no obstante sin ser específicos de ningún microprocesador. Para ejecutar los programas en Java se hacen dos operaciones, la compilación obteniendo los bytecodes (usando el compilador javac.exe) y la interpretación de estos desde cada plataforma con el interprete Java.

El Kit de desarrollo de Java (JDK) contiene las herramientas y librerías necesarias para crear y ejecutar applets y aplicaciones en Java.

A continuación se listas algunas de las utilidades que se pueden encontrar en el JDK:

– javac. Es el compilador de Java. Se encarga de resumir el código fuente escrito en Java a bytecode.

– java. Es el intérprete de Java. Ejecuta el bytecode a partir de los ficheros class.

– appletviewer. Es un visor de applets. En la generalidad de las veces puede utilizarse en lugar de un Navegador Web.

– javadoc. Se utiliza para originar documentación en formato Html a partir de el código fuente Java y los comentarios que contiene.

– javap. Es un desensamblador de Java.

– jar. Es una herramienta utilizada para trabajar con los ficheros Jar.

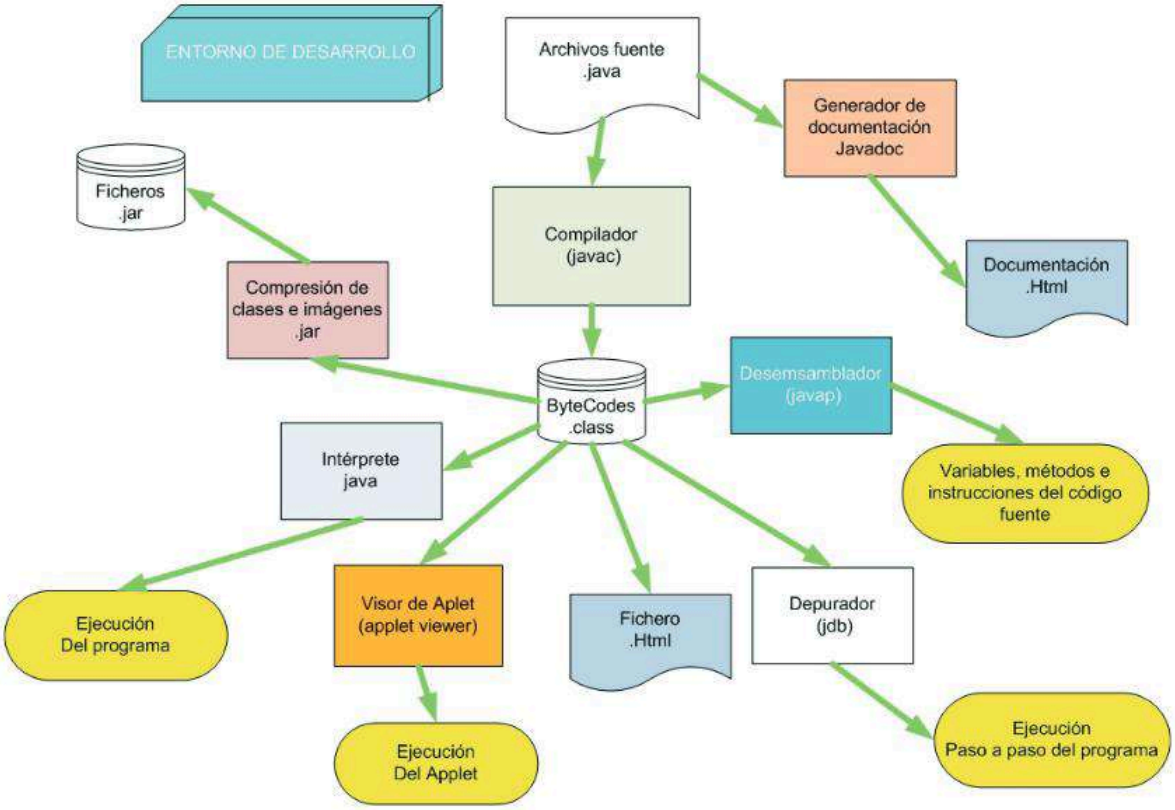



*Diagrama del entorno de desarrollo de Java*

Java es un lenguaje multitarea eso quiere decir que soporta arrancar varios programas Java a la vez en el kernel del sistema diferente que exista o bien en el navegador. Se pueden crear aplicaciones en las que pueden ocurrir varias cosas al mismo tiempo. Esta característica es soportada por ciertos sistemas operativos como UNIX o Windows.

La programación en Java es compilada en bytecode, el cuál es ejecutado por la máquina virtual Java. Usualmente se usa un compilador Jit. El lenguaje es parecidos a C y C++, no obstante su arquetipo de objetos es más sencillo, y fue influenciado además por Smalltalk, y Eiffel.

Java Specification Request (Jsr) propone encajar tipos y métodos generales en el lenguaje de programación Java. Desde los comienzos de Java, los desarrolladores han tratado de agregarle los tipos genéricos al lenguaje. Los elementos universales han sido empleados por años en otros lenguajes de programación y hoy forman parte de las últimas versiones de Java.

El paquete JDK contiene el entorno de desarrollo de Java de Sun, ya adquirida por Oracle. Sirve para desarrollar programas Java y proporciona el entorno de ejecución necesario para ejecutar dichos programas. El JDK en dos tipos, un binario precompilado y un paquete fuente.

Al compilar el código fuente del JDK, también necesita descargar la versión binaria para construir el JDK. Necesitarás descargar un total de cuatro ficheros para completar la construcción de las fuentes: jdk-1_7_0_02-linux-i586.bin; jdk-1_7_0-src-jrl.zip; jdk-1_7_0-bin-jrl.zip; y jdk-1_7_0-mozilla_headers-unix.zip, jdk-6-rc-windows-i586.

Características de Java

### Simple

Tiene las características de un lenguaje potente pero, sin:

- Aritmética de punteros.
- Liberación de memoria.
- Reduce en un 50% los errores más comunes de programación al omitir las características nombradas más arriba.
- Orientado a objetos: Exige trabajar con una orientación a objetos pura. No permite programación procedural.
- De construcción neutral: esto quiere decir que el compilador compilará el código a un archivo objeto de formato indiferente totalmente independiente de la arquitectura del hardware de la máquina en que se ejecutará.
- Robusto: permite y soporta la realización de comprobaciones para situar errores y problemas tanto en tiempo de compilación, como en periodo de ejecución.
- Distribuido: este lenguaje Java se ha implementado con grandes capacidades de interconexión Tcp/ip. Existen librerías de clases para operar perfectamente al nivel de protocolos como http y ftp.

El Sistema es la Máquina Virtual Java (JVM) y las librerías correspondientes que conlleva, que también nos posibilitan acceder directamente al hardware de la máquina. APIs de Java que también entren en contacto directo con el hardware y serán dependientes de la máquina como:

– Java 2D: gráficos 2D y manejo de imágenes.

– Java Media Framework: Componentes críticos en el tiempo: audio, video…

– Java Animation: Animación de elementos 2D.

– Java Telephony: Unificación con telefonía.

– Java Share: Intercambio entre aplicaciones multiusuario.

– Java 3D: Manejo Gráficos 3D y su diseño.

### Seguro

La seguridad en Java tiene dos fases. En el lenguaje, características como los punteros (no existen en Java) o el casting subyacente que hacen los compiladores de C y C++ se eliminan para alertar el acceso ilegal a la memoria. Cuando se usa Java para crear un navegador, se combinan las características del lenguaje con defensas de sentido común aprovechadas al propio navegador.

Si los byte–codes han pasado la verificación sin generar ningún mensaje de error, entonces podemos estar seguros de que:

– El código no llega hasta el desbordamiento de operaciones en la pila.

– El tipo de los parámetros de los códigos de operación son acreditados y correctos.

– No ha sucedido ninguna traducción ilegal de datos, tal como convertir enteros en punteros.

– El acceso a los campos de un objeto se sabe que es legal: public, private, protected.

– No hay ningún intento de transgredir las reglas de acceso y seguridad instauradas.

### Portable

Java aplica otros estándares de portabilidad para mejorar el desarrollo. Los enteros son siempre enteros y además, enteros de 32 bits en complemento a 2.

## ¿Como compilar un programa Java?

El lugar en el directorio donde se encuentran las herramientas Java tiene que estar bajo path del sistema.

Teclear “javac nombrefichero.java”.

Obtendremos tantos ficheros .class como clases existen en el fichero del código fuente.

## ¿Como ejecutar un proyecto Java?

Teclear “java nombre clase” El archivo nombreclase.class debe estar en un directorio comprendido en el Classpath Classpath es una variable de entorno que indica el trayecto por defecto en el que están las clases Generalmente el directorio actual está en el Classpath por lo que lo más sencillo es ejecutar el intérprete en el mismo directorio en el que está el archivo .class

Un proyecto en Java podrá construirse utilizando varias clases. En el caso más simple se utilizará una sola clase. Esta clase contiene el programa, rutina o método principal: main() y en éste se insertan las sentencias del programa principal. Estas sentencias se separan entre sí por puntos y coma.

La estructura de un programa simple en Java es la siguiente:

```
public class MiClasePrincipal {
public static void main(String[] args) {
instruccion_1;
instruccion_2;
// ...
instruccion_N;
}
}
```

Como primer ejemplo sencillo de programa escrito en Java se va a utilizar uno que muestra un mensaje por la pantalla del ordenador. Por ejemplo, el programa Hola.java:

```
/**
* La clase hola construye un programa que
* muestra un mensaje en pantalla
*/
public class Hola {
public static void main(String[] args) {
System.out.println("Hola, ");
System.out.println("me llamo Lucas");
System.out.println("Hasta luego");
}
}
```

En un programa en Java todo se organiza dentro de las clases. En el ejemplo Hola es el nombre de la clase principal y del archivo que tiene el código fuente. Todo el software escrito en Java tiene un método main o principal que tiene un bloque de código fuente que son un conjunto de instrucciones. En Java los conjuntos o bloques de sentencias se indican entre llaves { }. En el caso anterior, el conjunto de sentencias se simplifica a tres sentencias, que son invocadas a dos métodos predefinidos en Java (print y println) que permiten visualizar texto por el dispositivo de salida de datos por defecto (la pantalla).

Por el momento y hasta que se explique con detalle el concepto de clase, los ejemplos de programa que se utilizarán constarán de una sóla clase en la que se declara el método main. Este método es el punto de arranque de la ejecución de todo programa en Java.

## Comentarios

Los comentarios se usan mucho para poder documentar las líneas de código que escribimos. No se procesan ni tiene papel activo en el programa, pero su utilidad es enorme, ya que nos permite tanto a nosotros mismos como a otros programadores entender bien que estamos picando y con que sentido. Es habitual escribir código y dentro de un mes no sabemos ni lo que hacia dicho código, para ello los comentarios nos permiten esclarecer y documentar todo el código fuente. Así, el compilador ignora todo lo que se incluya entre la secuencia de caracteres // y el final de la línea. Por ejemplo:

```
// Ejemplo de comentario estilo C++, llega al final de la línea La pareja de caracteres / hay que insertarla sin dejar ningún espacio en blanco entre ellos
```

Otra forma de comentario es el que se utiliza en el lenguaje de programación C: el compilador ignora todo lo que se incluya entre las secuencias de caracteres /* y */. Por ejemplo:

```
/* Comentario formato C, el final lo reseña la marca */
```

El comentario con // es más fácil de teclear si sólo ocupa una línea, pero si ocupa varias, entonces el más sencillo es el de /*… */. Los comentarios pueden incluir cualquier carácter válido en Unicode y no pueden anidarse.

Estos dos formatos de comentario se emplean para los denominados comentarios de implementación. Los comentarios de implementación se utilizan en el programa fuente para resaltar código o para aclarar un desarrollo en particular.

Java tiene un tercer tipo de comentario “doc comments” que posibilita la creación de documentación técnica en formato HTML al emplear algunas herramientas de documentación como javadoc inmersa en el Kit de Desarrollo de Java. Los comentarios para crear la documentación se usan mucho para describir la especificación del código, desde una óptica independiente para ser leido por desarrolladores que no tengan necesariamente el código fuente a la vista. Debe evitarse emplear los comentarios para dar panorámicas del código y facilitar información que no facilite la propia lectura del código. Ejemplos de este tipo de comentarios:

```
/** Comentario estilo javadoc, se incluye
automaticamente en documentacion HTML */
/**
* Muchos programadores
* suelen incluir un asterisco
* al principio de cada linea del
* comentario para facilitar su lectura
*/
```

Los comentarios se usan para describir claramente las instrucciones y el código que picamos. Es muy importante utilizar siempre comentarios da igual el tamaño de los programas en Java que escribamos, o clases de programadores/as que usemos. La discusión de decisiones de diseño que no sean obvias triviales es apropiada, pero debe evitarse la duplicidad de información dada por el propio código. Es muy común que los comentarios redundantes se desactualicen. En general, se debe evitar incluir comentarios que puedan quedarse obsoletos conforme el código se desarrolle. Por ejemplo, dentro de un comentario no debería incluirse información acerca de cómo se ha construido un archivo fuente determinado o en que directorio se haya el código.

A veces la frecuencia de los comentarios refleja una peor calidad de código. Cuando se sienta la necesidad de añadir un comentario, debe considerarse la reescritura del código para hacerlo más claro.

### Conformidad para los comentarios al inicio de los programas

Todos los ficheros fuente deben empezar con un comentario estilo Java que liste el nombre de la clase, información de la versión, fecha y el aviso de copyright:

```
/*
* Nombre de la clase
*
* Información de la version
*
* Fecha
*
* Aviso de Copyright
*/
```

### Identificadores

Estos componentes indentificadores existen en el código fuente no obstante no ya en el programa objeto. Los identificadores son nombres que se les asignan a variables, métodos, clases… en el código fuente de un programa. Todo nuevo identificador que se use en un programa Java debe definirse antes a su utilización. Las normas para la construcción de un identificador empleando el lenguaje de programación Java son las siguientes:

Los identificadores siempre empiezan por una letra, un carácter de subrayado (_) o un carácter de dólar ($). No es bueno utilizar el carácter $, ya que el compilador los utiliza de forma interna para crear identificadores propios.

Los siguientes caracteres pueden ser también dígitos. Pero no pueden emplearse espacios en blanco u otros caracteres como el signo de interrogación (?) o el signo del tanto por ciento (%).

No hay límite máximo de caracteres. En los identificadores del código fuente de un software en Java se distinguen las mayúsculas de las minúsculas. Por ejemplo, factura, Factura y FACTURA son tres identificadores diferentes. Puedes insertar caracteres Unicode, con lo que se pueden emplear secuencias de escape /uxxxx para representar estos símbolos.

No puede emplearse el identificador de una variable o cualquier otro elemento del código fuente del programa para otro ya existente en el mismo bloque. Excepción: variable miembro y local con el mismo identificador.




Existe una serie de palabras reservadas que no pueden emplearse como identificadores por el programador en el código fuente para otros usos. En concreto, el término double se usa para declarar un tipo de dato real y el término for se emplea para construir un tipo determinado de bucle.

Abstract do implements protected throw boolean double import public throws break else instance of rest transient byte extends int return true case false interface short try catch final long static void char finally native strictfp volatile class float new super while const* for null switch continue goto* package synchronized default if private this.

Algunos de estos identificadores reservados no tienen todavía uso en la implementación actual del lenguaje Java. En concreto, los identificadores marcados con un asterisco * no se utilizan actualmente. Por otro lado aparecen los nombres de métodos reservados en Java cuyo significado y utilización se explicará más adelante cuando se menciones la clase predefinida Object.

Identificadores de métodos reservados en Java

clone equals finalize getClass hashCode notify notifyAll toString wait

Aunque la forma de escribir los identificadores en Java no está normalizada, a continuación se dan algunas recomendaciones para la elección de estos identificadores:




Es conveniente manejar identificadores autoinformadores para guiar al usuario que accede al código fuente de un programa sobre lo que representan, sin necesidad de usar los comentarios en el propio código fuente. Por ejemplo, para almacenar el valor del radio de una esfera podría declararse una variable con el identificador radio, radioesfera o radiodeunaesfera.

Si el identificador se compone de más de una palabra, éstas se agrupan y después de la primera el resto comienza con una letra mayúscula, como por ejemplo: esImporte o abonoFactura.

El signo de subrayado se admite en cualquier identificador pero por convención sólo se emplea para separar voces en identificadores en constantes, por convención las constantes se escriben totalmente en mayúsculas y esto dificulta la separación de los nombres. Sin embargo, para un valor constante que almacena un valor numeral que representa el número máximo de estudiantes con los que trabaja el programa se podría declarar el identificador de la constante NUMERO_MAXIMO_ESTUDIANTES.

### Convenciones para la legibilidad del programa fuente

Aunque la definición exacta de la indentación espacios vs tabuladores, no se detalla en el lenguaje Java, se recomienda utilizar una secuencia de cuatro espacios como unidad de indentación. Por otro lado se recomienda evitar las lineas de más de 80 caracteres en el código fuente ya que no se manejan correctamente por muchos terminales y herramientas de edición.

## 9.2. Enumeración de los tipos de aplicaciones

### Plataformas

Una plataforma es el entorno de hardware o software en el cual se ejecutan los programas. En general, la mayoría de las plataformas se describen como una combinación de hardware y sistema operativo. Algunas de las plataformas más populares son Windows, Solaris, Linux y MacOS.

Dicha plataforma Java es como se denomina al entorno originario de Sun Microsystems, capaz de ejecutar aplicaciones desarrolladas usando el lenguaje de programación Java. La plataforma no es un hardware específico o un sistema operativo, sino más bien una máquina virtual encargada de la ejecución de las aplicaciones, y un grupo de bibliotecas que ofrecen una enorme funcionalidad.

Java es una plataforma virtual que provee:

El lenguaje Java: como lenguaje de programación.

El Java Virtual Machine: Una máquina virtual con su propio set de instrucciones.

La plataforma Java incluye en sus distintas versiones:

– Plataforma Java, Edición Estándar (Java Platform, Standard Edition), o Java SE.

– Plataforma Java, Edición Empresa (Java Platform, Enterprise Edition), o Java EE.

– Plataforma Java, Edición Micro (Java Platform, Micro Edition), o Java ME para móviles.

En cuanto a entornos y ámbito de sus aplicaciones:

– La Java Platform API: Es una interfaz muy potente para la programación de aplicaciones.

– JavaCard: JavaCard se usa para desarrollar aplicaciones en Java que puedan ser usadas en tarjetas inteligentes.

– EmbeddedJava: Sirve para desarrollar aplicaciones relacionadas con el acceso a redes, concretamente con la entrega de servicios bajo demanda, como máquinas de "Vending" venta en la calle, dispensadores de gasolina, automóviles, de tal forma que el servicio se efectúe a través de la red. La máquina virtual no requiere más de 500 Kbytes.

– Javaphone: Son una librería de clases para desarrollo de aplicaciones en teléfonos. Se utiliza en diferentes tipos nuevos de teléfonos.

– Teléfonos inteligentes: conjugarán comunicaciones por voz, fax, correo electrónico, comunicación técnica radio, paginación, acceso a Internet, planificación de tareas al modo de los smartphone y muchas otras funciones que los teléfonos móviles, smartphone. Teléfonos con pantalla para Internet. Son pantallas de vídeo y opcionalmente teclados que permiten comunicaciones personales en Internet.

– Java Tv: Televisión digital interactiva brinda una plataforma para generar programas Java para controlar la televisión y los "set–top boxes" construidos para el entretenimiento digital, de manera que sean independientes de la tecnología latente de emisión.

## 9.3. Herramientas de desarrollo

### Descripción general de NetBeans

NetBeans es un software de desarrollo multi–lenguaje muy usado por programadores de todo tipo desde nóveles como por profesionales.

NetBeans es un software libre de uso gratuito, fue desarrollado por la propia compañía de Java, SUN, y ahora Oracle, se puede descargar e instalar de manera sencilla junto con el paquete de desarrollo Java SE.

Es una estructura para elaborar programas de escritorio para lenguaje Java y un conjunto de herramientas muy potentes integradas (Ide) para la Plataforma NetBeans. Está estudiada como plataforma, para que posibilite que las aplicaciones sean desarrolladas a partir de un conjunto de componentes de software, denominados módulos. Un módulo es como un pack en un archivo Java que contiene clases de Java escritas para interactuar con las APIs de NetBeans. Existen módulos gratuitos y de pago para casi cualquier tipo de programa que queramos desarrollar.

Los aplicativos desarrollados pueden ser ampliados agregándole nuevos módulos. Como los módulos pueden ser desarrollados independientemente, las aplicaciones establecidas en la plataforma NetBeans pueden ser extendidas cómodamente por otros desarrolladores de software. Es un proyecto de código abierto de gran éxito con una gran base de usuarios. Con NetBeans 6.01 , 6.5 y 7.2.1 Se dio soporte a frameworks comerciales como son Struts, Hibernate.

La Plataforma NetBeans es modular, ampliable y escalable empleada como una distribución de integración para implantar aplicaciones de escritorio. Programadores independientes y empresas asociadas del sector de desarrollo de software, proporcionan extensiones adicionales que se integran sencillamente en el entorno y que pueden también utilizarse para desarrollar sus propias herramientas y soluciones.

La plataforma ofrece servicios comunes a las aplicaciones de escritorio, permitiéndole al desarrollador enfocarse en la lógica específica de su aplicación.

## IDE NetBeans

Es un IDE–Comunmente indicar que es una herramienta ideada para desarrollar, escribir, compilar, depurar y ejecutar programas. Está escrito en Java y se puede emplear para otros lenguajes de programación. Contempla un número importante de módulos para extender el IDE NetBeans. El IDE NetBeans es un producto sin restricciones de uso.

El NetBeans IDE soporta el desarrollo de todos los tipos de aplicación Java (J2SE, web, EJB y aplicaciones móviles). Entre sus características se encuentra un sistema de proyectos basado en Ant, control de versiones y refactoring.

### Característica general de la plataforma NetBeans

Entre las características de NetBeans de la plataforma están:

- Administración de las interfaces de usuario (ej. menús y barras de herramientas)
- Administración de las configuraciones del usuario
- Administración del almacenamiento (guardando y cargando cualquier tipo de dato)
- Administración de ventanas
- Framework basado en asistentes (diálogos paso a paso)

### Característica NetBeans 7.2.1

NetBeans IDE es mucho más que un Java IDE. NetBeans IDE como una herramienta de desarrollo modular para un amplio desarrollo de tareas. La base del IDE incluye un avanzado editor, debugger e integración de portafolios, control de versiones y únicas características que se desarrollan en colaboración.

### IDE–wide QuickSearch

Herramienta para emplear el acceso directo Ctrl–i para hacer una búsqueda de contexto total, sets de ayuda, y de todos los proyectos abiertos. Localizará términos no solo en archivos, tipos y símbolos, pero también todo lo relacionado con acciones de menú, opciones de paneles y documentación.

### Plugin Manager

Sirve para trabajar con el administrador de Plugins desde el menu Herramientas para añadir, borrar o actualizar el set de características para Java SE, Java EE, Java ME, JavaFX, incluso para UML.

### Personaliza tus proyectos

Sirve para aplicar propiedades y compilar dependencias entre programas, proyectos, y usa librerías relativas para hacer proyectos compartibles.

Guarda varias configuraciones del proyecto para diferentes etapas de desarrollo. El IDE crea binarios ejecutables y carpetas de archivos en un número de formatos incluyendo JAR, WAR, EAR, NBM, JNLP Web Start, y carpetas ZIP, preparados para distribuir.

### Plantillas y ejemplos de aplicaciones completas

Se puede construir una forma abierta proyectos basados en una existente compilación de scripts y códigos, o iniciar proyectos desde una plantilla. El IDE viene con plantillas y ejemplos de proyectos para aplicaciones Java SE, aplicaciones moviles, aplicaciones web, aplicaciones Ruby and Ruby sobre Rails enterprise aplicaciones, aplicaciones JavaFX, NetBeans plugins, Groovy, PHP, aplicaciones C y C++.

### Bases de datos y servicios

Tiene soporta para bases de datos y servidores web, además como servicios y enterprise JavaBeans (EJBs). Se inicia y detiene los servidores, y drag and drop servicios web y EJBs dentro de proyectos.

### Listas de tareas

NetBeans chequea por si solo el código, código fuente y sus líneas con fallos al compilar, reparaciones rápidas, y warnings de estilo. Conexiones para un bug en bases de datos como. Issuezilla, Bugzilla, java.net, Scarab y reporta listas de cuestiones.

### Múltiples monitores

Desacopla algún editor para arrastrar y soltar en cualquier lugar del IDE. Esto llegara a su vez en ventanas independientes se puede mover en una segunda pantalla. Para reintegrar dentro de la venta del ID, arrastra y suelta de regreso.

### Proyectos en grupos

El entorno soporta y permite definir sets de proyectos relacionados. Esto permite abrirlos y cerrarlos rápidamente en un solo paso para grupos de desarrolladores conectados.

### Tecnologías de soporte

Incluye nuevas características como la integración, un marco de cooperación equiparable a java.net permite proyectos opensource; Soporte nativo para Maven; integración con Jira, Bugzilla; integración con Hudson y Glassfish. Esta relase incluso ofrece mejoras para Java, php, Ruby, Javascript, Groovy, C/c++ y más.

El IDE NetBeans ha ganado últimamente el Premio al entorno integrado de desarrollos para Windows, Mac, Linux, y Solaris. El proyecto es un IDE open–source y una aplicativo que permite a los desarrolladores crear rápidamente aplicaciones web, enterprise, desktop, y mobiles usando la plaforma Java, PHP, JavaScript y Ajax, Ruby y Ruby on Rails, Groovy y Grails, y C/C++,soporta además una diversa seleccion de plugins.

### Información de las características NetBeans IDE 7.2

NetBeans IDE 7.2 está integrado con el proyecto Kenai, un entorno colaborativo para desarrolladores de proyectos host open–source. Con Kenai y el IDE NetBeans, un equipo de desarrolladores podrá crear proyectos, revisarlos, editarlos, debuguear, construirb, todo a través de una interfaz fácil de usar.

La versión también soporta Maven; GlassFish, Java, PHP, Ruby, Groovy y C/C++. soporta JavaScript 1.7, Ruby Remote Debugging, y la integracion de Java ME SDK 3.0.1.

Otra buena herramienta de desarrollo es Eclipse

### Aplicación Java visual con Eclipse

Eclipse es una plataforma de software de Código abierto independiente de una plataforma para desarrollar Aplicaciones. Esta plataforma, es usada para desarrollar entornos integrados de desarrollo IDE, como el IDE de Java —Java Development Toolkit JDT—.

Este entorno Eclipse fue desarrollado originalmente por IBM como el sucesor de VisualAge. Luego fue implementado por la Fundación Eclipse, una organización sin ánimo de lucro que promueve una comunidad de progrmadores de software libre y un paquete de productos complementarios, capacidades y servicios.

El paso siguiente es instalar JDK y el IDE Eclipse gratuito y uy usado en el mundo de la programación. Se irán viendo los pasos correspondientes para su exitosa instalación.

Cuando termines tendrás todo listo para empezar a probar sus primeras líneas de código.

Lo primero que tienes que saber y comprobar que windows es de 32 o 64 bits.

Las versiones de windows son las siguientes:

| Java SE Development Kit 6 Update 25 | | |
|---|---|---|
| **Product / File Description** | **File Size** | **Download** |
| Linux x86 - RPM Installer | 76.85 MB | jdk-6u25-linux-i586-rpm.bin |
| Linux x86 - Self Extracting Installer | 81.11 MB | jdk-6u25-linux-i586.bin |
| Linux x64 - RPM Installer | 77.06 MB | jdk-6u25-linux-x64-rpm.bin |
| Linux x64 - Self Extracting Installer | 81.36 MB | jdk-6u25-linux-x64.bin |
| Solaris x86 - Self Extracting Binary | 81.00 MB | jdk-6u25-solaris-i586.sh |
| Solaris x86 - Packages - tar.Z | 136.67 MB | jdk-6u25-solaris-i586.tar.Z |
| Solaris SPARC - Self Extracting Binary | 85.96 MB | jdk-6u25-solaris-sparc.sh |
| Solaris SPARC - Packages - tar.Z | 141.11 MB | jdk-6u25-solaris-sparc.tar.Z |
| Solaris SPARC 64-bit - Self Extracting Binary | 12.24 MB | jdk-6u25-solaris-sparcv9.sh |
| Solaris SPARC 64-bit - Packages - tar.Z | 15.58 MB | jdk-6u25-solaris-sparcv9.tar.Z |
| Solaris x64 - Self Extracting Binary | 8.49 MB | jdk-6u25-solaris-x64.sh |
| Solaris x64 - Packages - tar.Z | 12.25 MB | jdk-6u25-solaris-x64.tar.Z |
| Windows x86 **windows 32 bits** | 76.66 MB | jdk-6u25-windows-i586.exe |
| Windows x64 **windows 64 bits** | 67.27 MB | jdk-6u25-windows-x64.exe |

*Pantalla de versiones de JDK para descargar de Java*

Hay que elegir la versión que más se adapte a nuestro sistema, luego descargaremos a nuestro ordenador el fichero comprimido de instalación y elegir la ruta adecuada como por ejemplo ("C:\Program Files\Java")

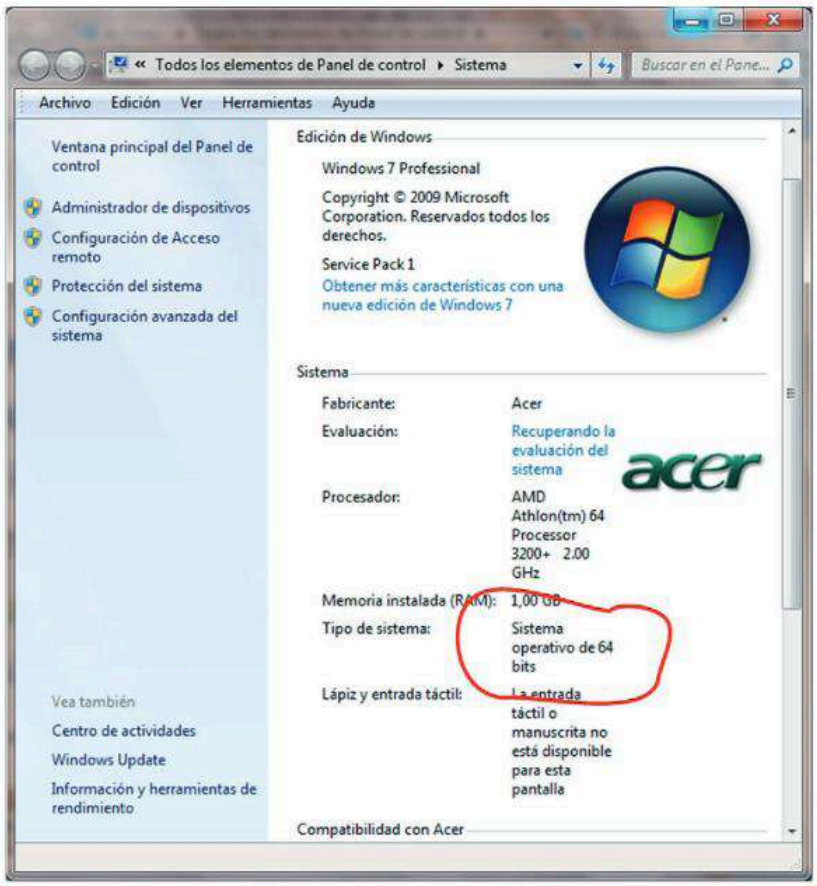
La variable de entorno son muy importantes incluirlas bien en el sistema para que desde la consola de Windows puedan compilar y ejecutar archivos Java.

Para agregar una variable de entorno en Windows 7 debemos clickar en los siguientes botones empezando por el botón **Inicio** hasta Panel de control:

Panel de control → Sistema → Configuración avanzada del sistema → Variables de entorno

Un shortcut o tecla rápida para acceder al Sistema es Tecla Windows + Tecla Pausa.

La pantalla de sistema es la siguiente:




*Pantalla de versión de sistema operativo que tenemos*

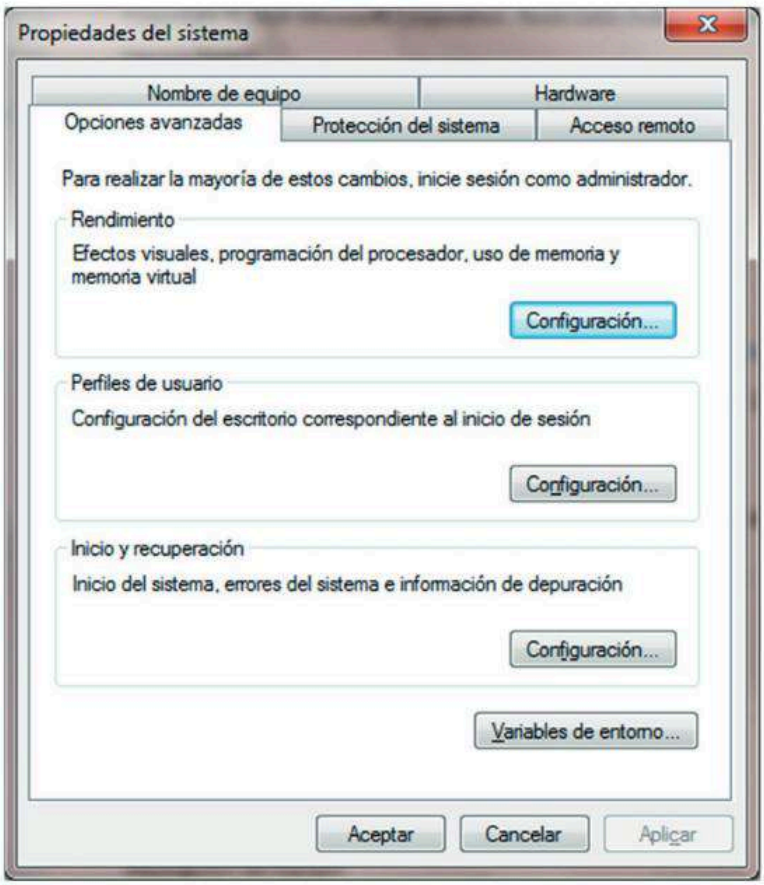
Al acceder a la Configuración avanzada del sistema vemos la siguiente pantalla:

En ella aparecen diferentes secciones todas ellas muy importantes dentro de nuestra configuración del sistema operativo Windows 7.

Haremos click en el último botón de la parte inferior donde aparece Variables de entorno. Los cambios que hagamos en esta sección afectarán una vez reiniciemos el sistema.

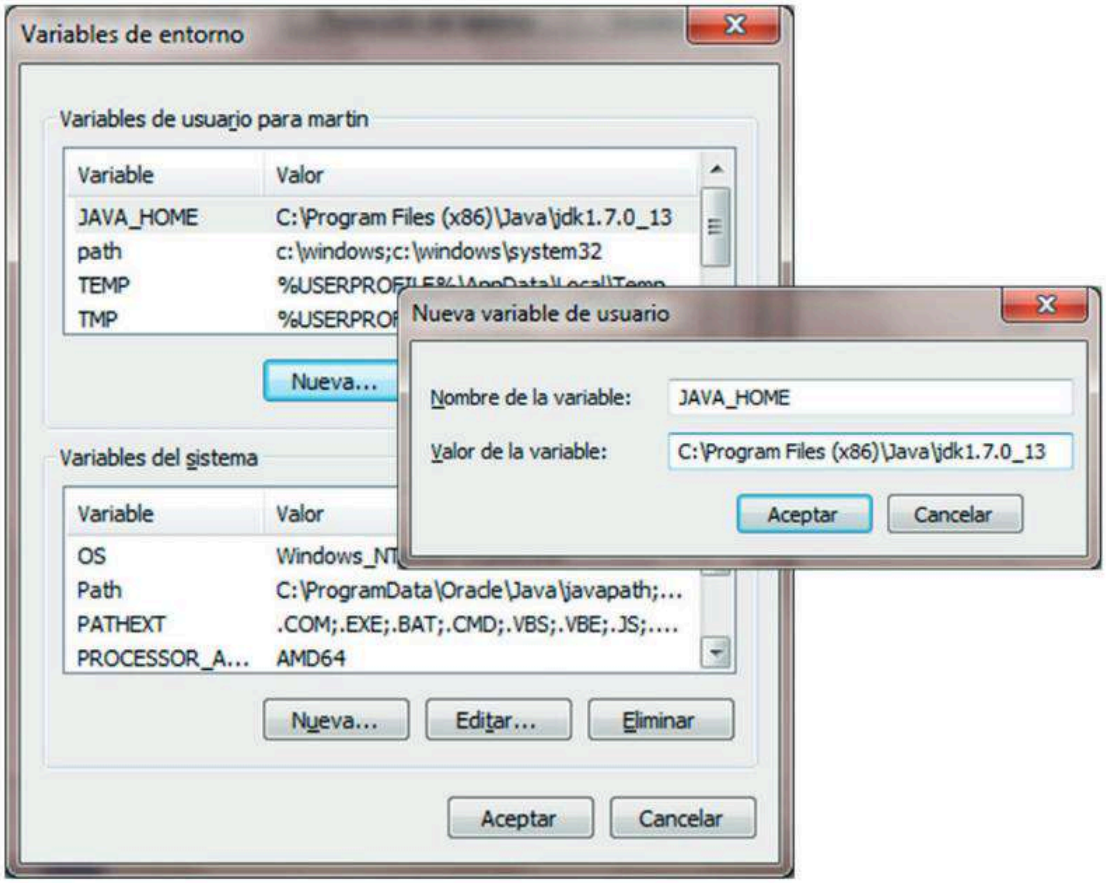
La sección de variables de entorno es muy importante para configurar nuestro entorno de programación, ya que ayudará a poder compilar y que el sistema localice aquellos ficheros relacionados con la compilación que estén bajo estas variables del sistema.

Lo primero es que localice el compilador **javac** de Java por ahí debemos empezar, sino tendremos errores precisamente con el PATH del entorno del Runtime de Java y su estructura de carpetas.




*Pantalla de entrar en variables de entorno*

Crear una nueva variable de entorno es el siguiente paso, para ello presionaremos en el botón “Nueva”, como muestra la imagen nos sale una nueva ventana donde especificamos el nombre de la variable y el valor de la variable. En el nombre pueden poner lo que quieran, es una simple referencia que utilizaremos luego (les recomiendo “JAVA_HOME”). El valor deberá ser sí o sí la ruta de acceso a la carpeta donde instalamos el JDK en nuestro sistema. Les debería quedar algo similar a la imagen. Al presionar aceptar verán la nueva variable dentro de la lista de “Variables del Sistema” tal cuál figura en la imagen.



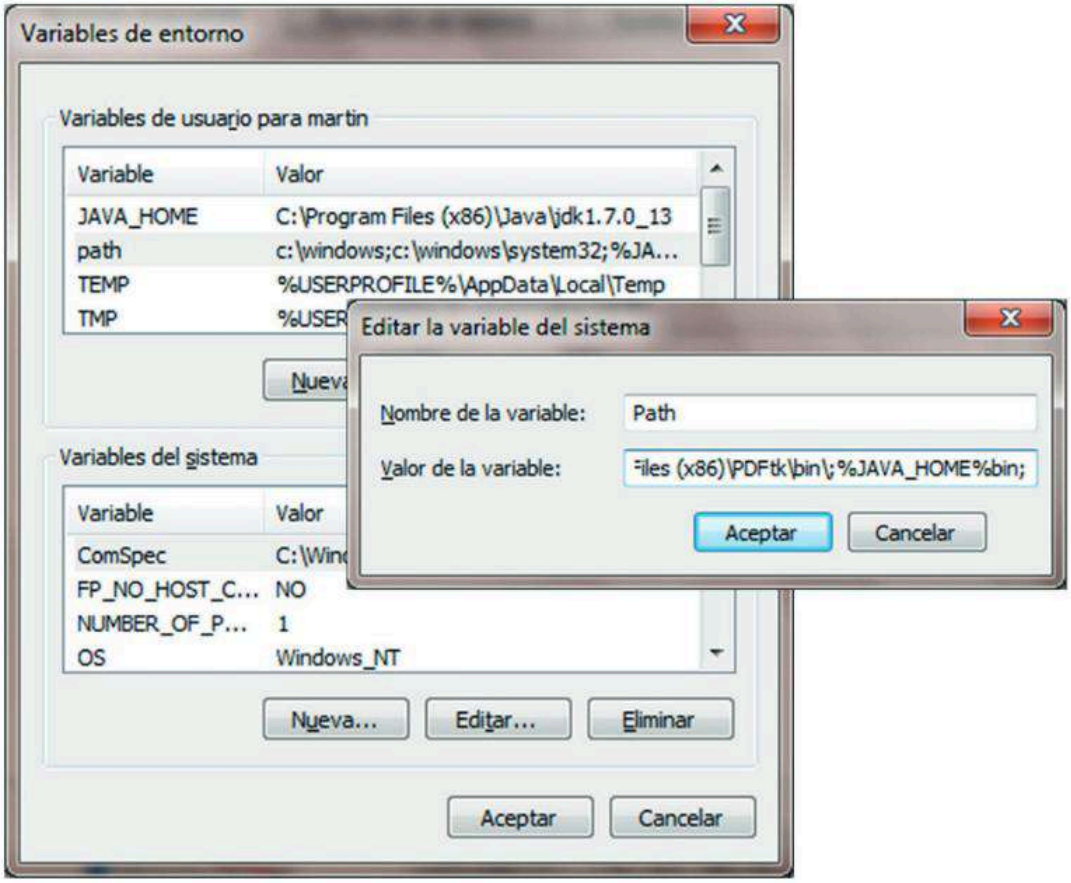

*Pantalla de actualizar variables de entorno y JAVA_HOME*

Para cambiar la variable de sistema ya existente “Path” . Para ello elegimos la variable de la lista de variables y presionamos el botón “Editar”. Esto abrirá una ventana emergente donde podremos ver el valor de la variable y editarlo.

Para terminar con este sencillo proceso de instalar Eclipse, y agregamos lo siguiente:

```
%JAVA_HOME%\bin;
```

Siendo “JAVA_HOME” el nombre de la variable que se añadieron anteriormente. Presiona aceptar en ambas pantallas y reiniciar el equipo para que active dichos cambios y los asimile Windows 7.



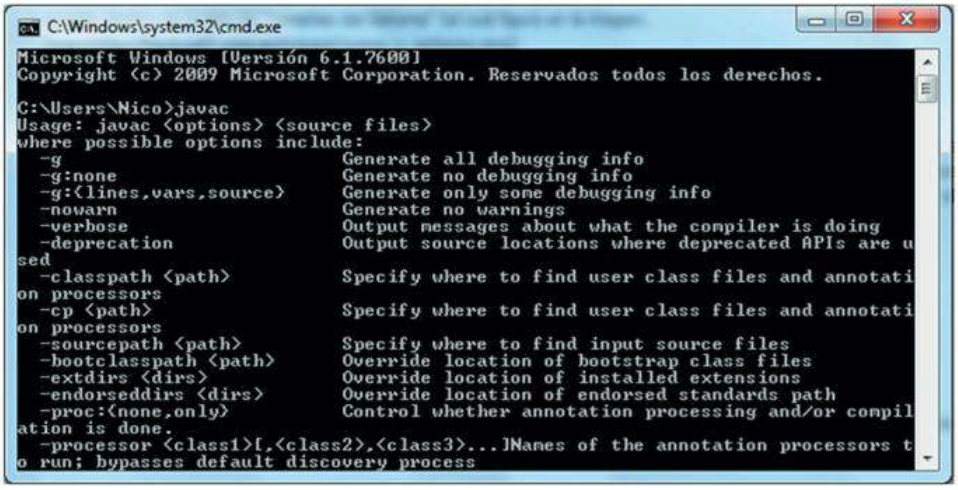

*Pantalla de insertar variables de entorno JAVA_HOME en la ruta PATH*

Para comprobar que la variable de entorno funcione y señal que se insertó correctamente, realizaremos este nuevo paso:

Vamos a realizar lo siguiente: la consola de Windows (es importante abrirla después de cambiar las variables para que el cambio se vea reflejado en la instancia abierta de la consola).

Tecla Windows + R escribimos **"**cmd**"** y presionamos **enter**

Cuando tengamos la consola abierta, dando igual que ruta tengamos en el prompt del sistema, tecleareremos el comando "javac" y seguidamente pulsaremos Enter.

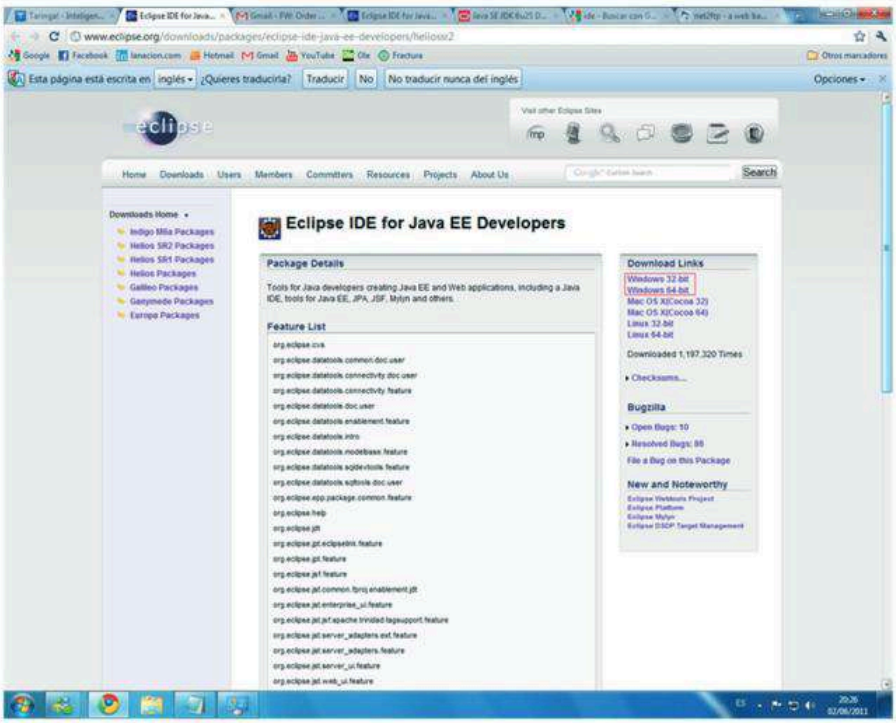



*Pantalla de ejecutar javac en la consola*

Con este paso ya tenemos bien instalado la máquina.. Ahora vamos con la herramienta de desarrollo.

En el navegador podemos buscar por Eclipse Helios y descargamos gratuitamente la versión disponible para nuestras características de equipo y sistema operativo.

Es muy importante que la versión de Java que descargamos y ahora con la misma versión de 32 o 64 bits elegiremos Eclipse.

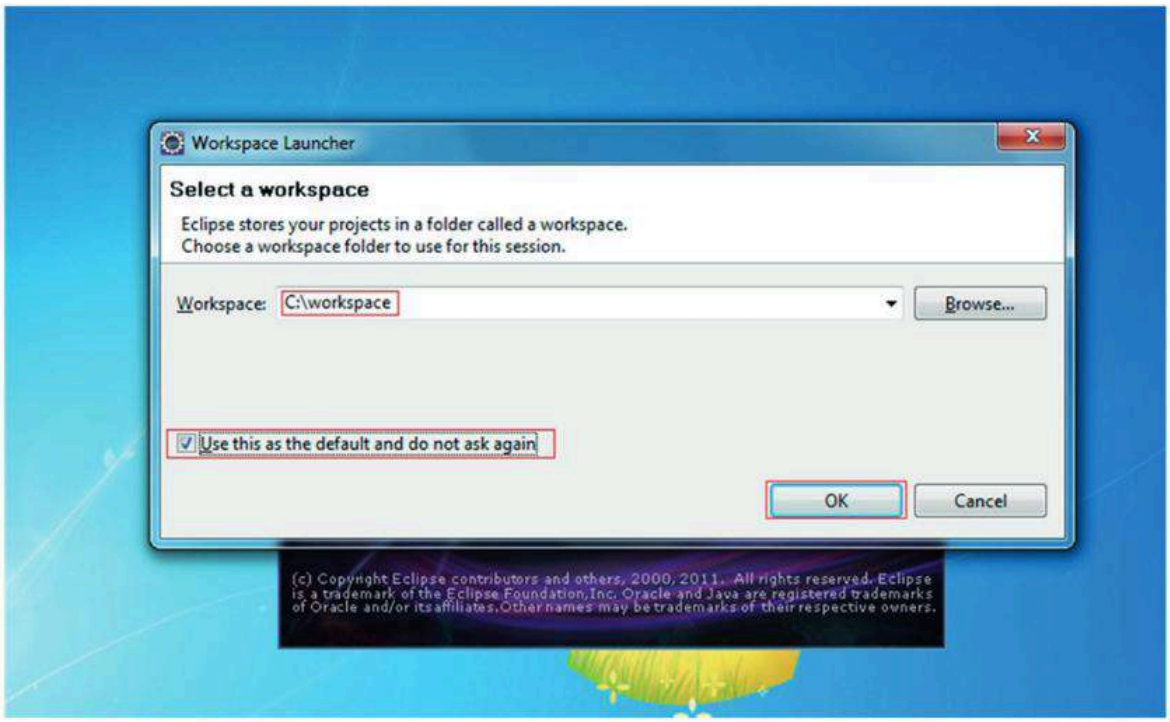



*Pantalla de descargar Eclipse de su Web oficial*

Una vez que hemos seleccionado la versión para descargar el software, nos lleva a una segunda pantalla donde seleccionamos el servidor desde donde se hará la bajada del programa.

En el momento que hayas acabado de descargar el archivo verás que es un zip y no tiene un instalador. Esto se debe a que el programa no hace falta instalarlo, lo colocas donde quieras que funcione, te recomiendo en la ruta " *C:\eclipse*".

Una vez que cargue te aparacerá la siguiente pantalla donde configuraremos el workspace área de trabajo.

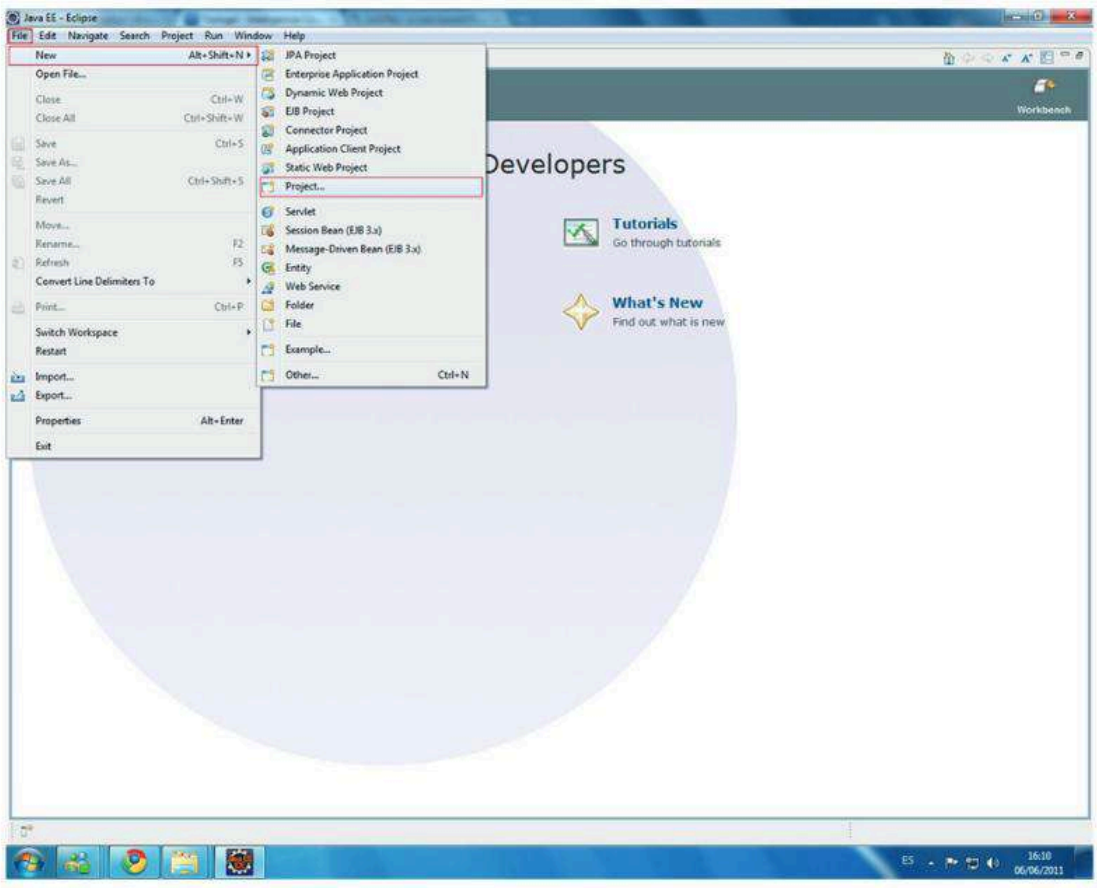



*Pantalla de selección en Eclipse espacio de trabajo*

El workspace es la carpeta donde Eclipse guarda todos sus proyectos. Les recomiendo que utilicen la dirección C:/workspace y tilden la opción "Use this as the default and do not ask again" (utilizar como default y no volver a preguntar) almacenando el espacio de trabajo de los proyectos que hagamos en Eclipse.

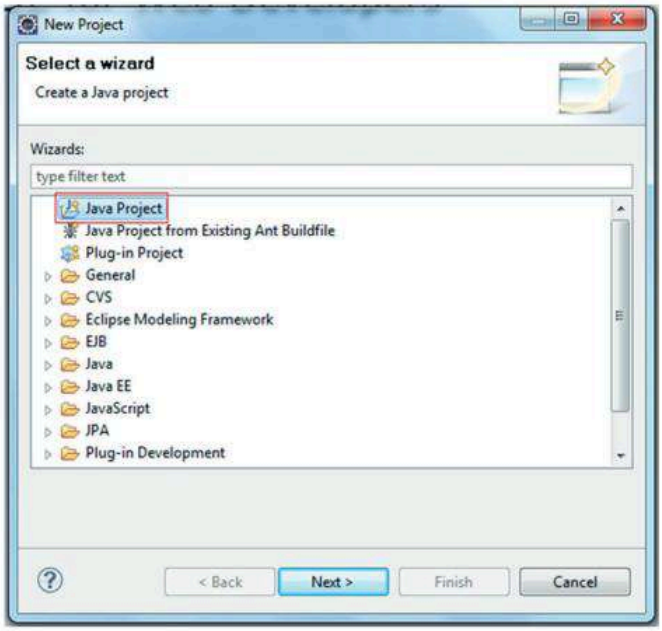
Una vez que concretemos el workspace, el programa terminará de cargar. Con el programa ya cargado crearemos nuestro primer proyecto, elegir un nombre y especificar que se tratará de un proyecto Java.

File → New → Project…



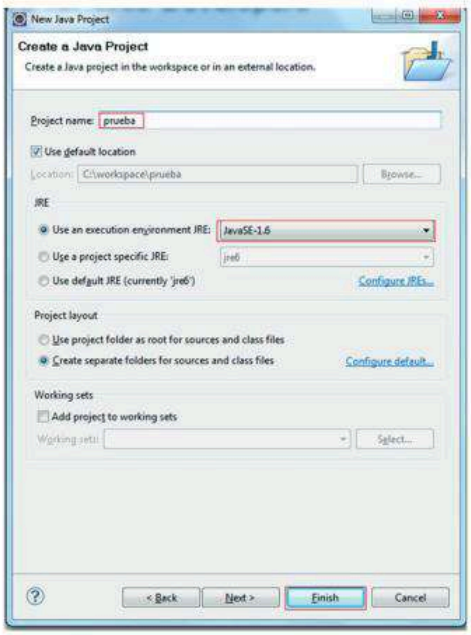

*Pantalla de Eclipse funcionando por defecto en Inglés y creación de New Project*

Seleccionamos en Java Projecto de toda la lista de proyectos y hacemos clicke en "Next →".




*Pantalla de Eclipse donde seleccionamos tipo de proyecto vamos a crear*

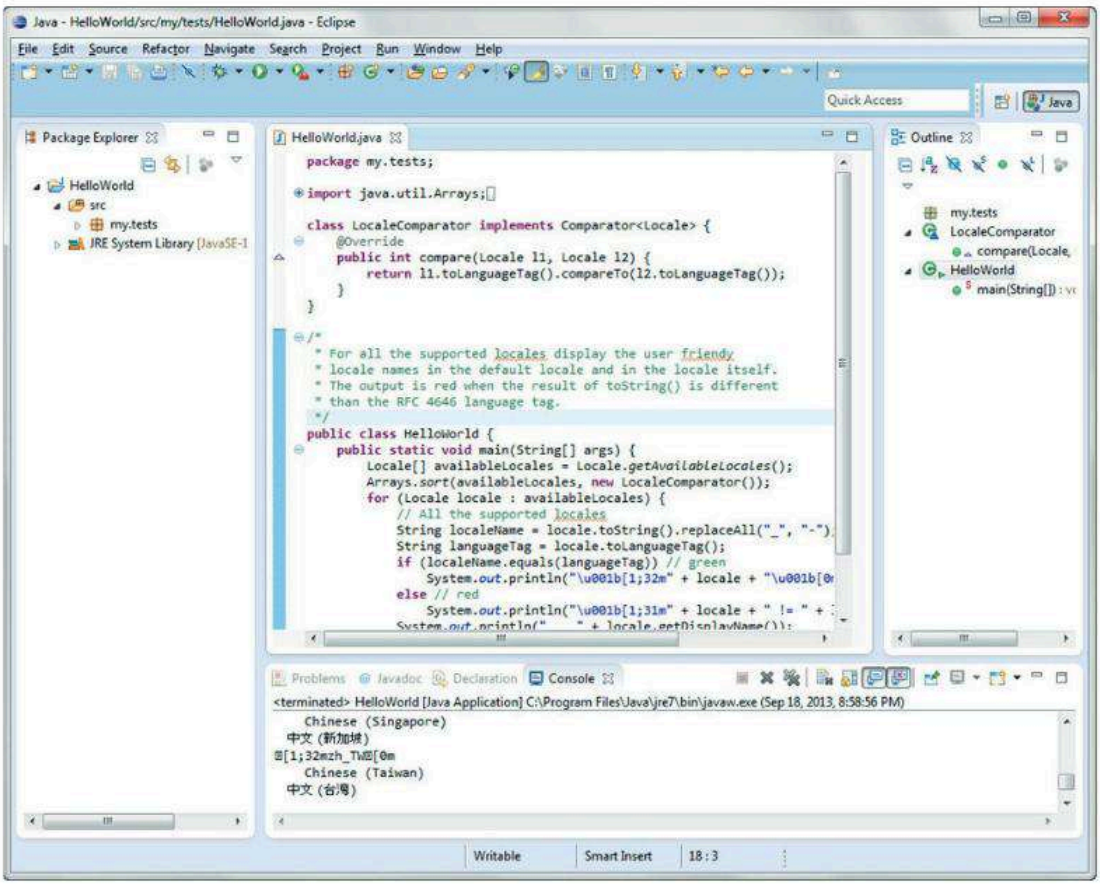
La última pantalla se elige un nombre para nuestro proyecto y definimos el JDK que se utilizará. Como realizamos con éxito toda la instalación del JDK en los pasos anteriores con las rutas que les indicamos al programa cargará el jdk "**JavaSE–1.7**". Le damos a Finalizar y ya tendremos nuestro proyecto preparado para empezar a trabajar y programar.




*Pantalla de Eclipse donde damos nombre a nuestro proyecto y JDK que usará*

Al finalizar podemos inducar si queremos aplicar la perspectiva de Java o de otro lengujae diferente. Las perspectivas son como configuraciones definidas en Eclipse para programar.

La perspectiva de Java es la siguiente:

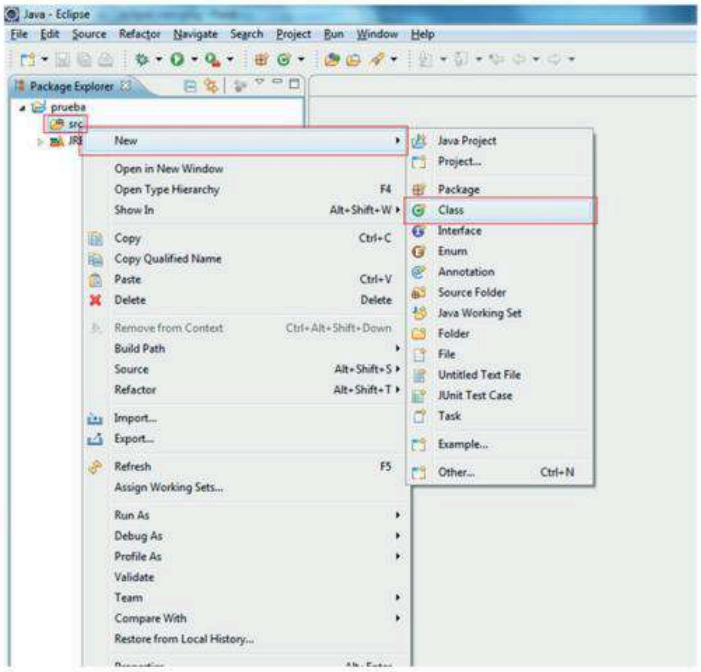



*Pantalla de Eclipse principal con todas las reas de trabajo, consola, paquetes y clases, area de programación y código*

En principio se ha configurado todo en condiciones para comenzar a programar Java con Eclipse.

Hacemos click derecho en el paquete “src” source/fuente en el explorador de paquetes y seleccionamos:

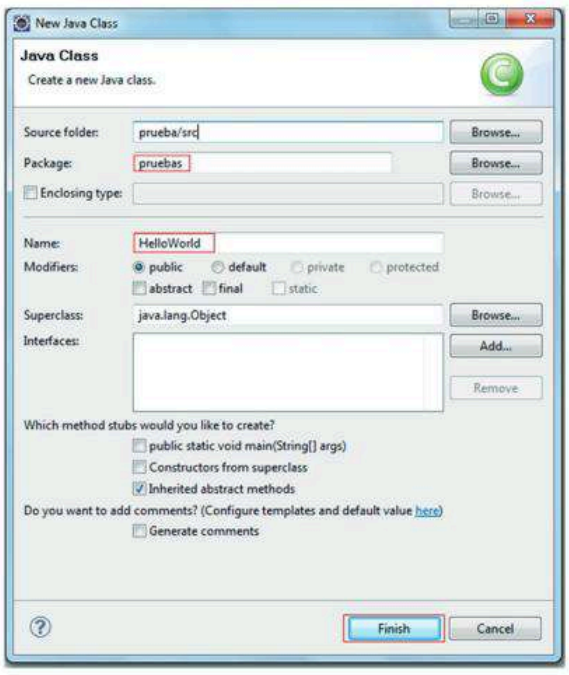
New → Class




*Pantalla de Eclipse donde crearemos una nueva clase*

En el siguiente paso, se abrirá una ventana para especificar el nombre de la clase.

El nombre de clase será "HolaMundo" que es el típico ejemplo que nunca puede faltar, también especificamos el paquete pruebas donde pondremos la clase.




*Pantalla de Eclipse donde damos nombre a la nueva clase*

Sabías que

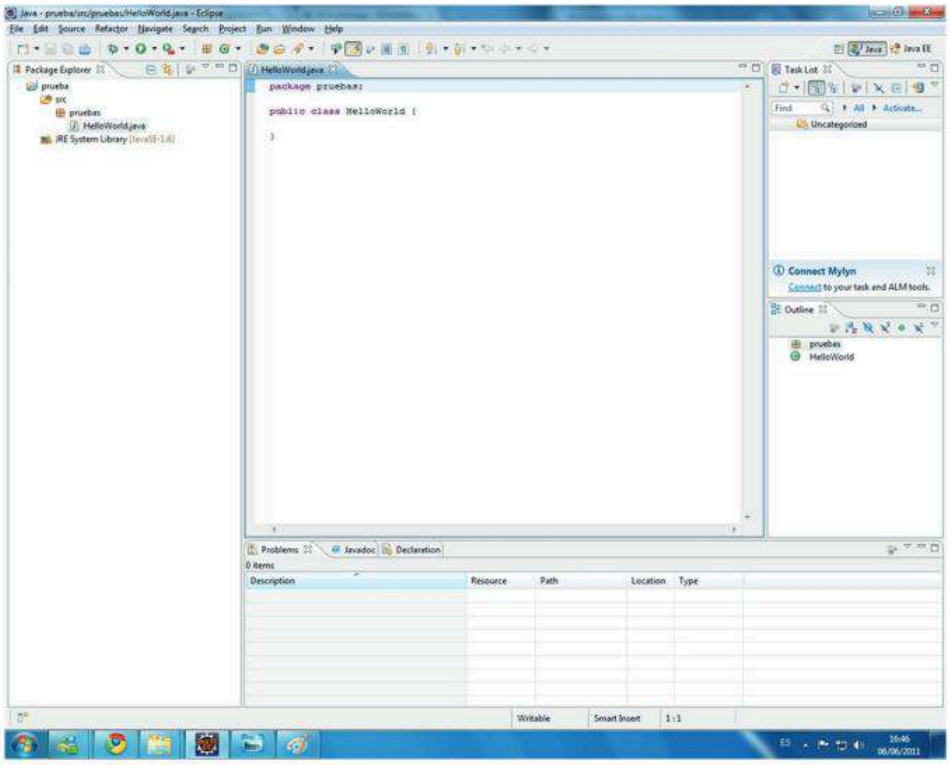
Por convención de Java los nombres de las clases se escriben con camel case. Ejemplo: StringBuffer, ensayoPrueba, HerramientaPesaje…

En Eclipse tendremos en cuenta que automáticamente crea el archivo .java con el mismo nombre que la clase, pero si creamos manualmente con el bloc de notas un archivo Java, la clase y el archivo deben tener el mismo nombre sino dará error al compilar.

Ejemplo: Si tu clase se llama Auto entonces el archivo deberá ser Auto.java

Si la declaración de la clase es “public class Zinnat{…}” el archivo no puede llamarse zennat.java, deberá ser sí o sí Zinnat.java

Al presionar “Finish” Eclipse creará el paquete “pruebas” y la clase “HolaMundo”, van a ver algo similar a la siguiente pantalla.




*Pantalla de Eclipse. Habremos creado un paquete y la clase HelloWord*

En este punto ya llegamos al principal objetivo de este punto que era instalar JDK y Eclipse.

Sigo un toque para que vean como compilar y ejecutar un main aunque sea.

Agreguen en el archivo **HolaMundo.java** lo siguiente y guardarlo (**CTRL + S** , si no lo guardan verán un asterisco al lado del nombre):

```
package pruebas;


public class HolaMundo {


public static void main(String[] args){
System.out.println("Hola Zinnat!!!" ) ;
}
}
```

Este Hola Mundo es mejor como ejemplo:

```
class HolaMundoOO
{
public void imprimeHola()
{
System.out.println (" Hola Mundo ");
}
}
```

```
class HolaMundo
{
public static void main(String[] args)
{
HolaMundoOO miHola = new HolaMundoOO();
miHola.imprimeHola();
}
}
```

### Programa HolaMundo (versión OO)

La nueva clase HolaMundo incluye un método no estático (necesita un objeto para ser ejecutado) denominado imprimeHola

Creamos una nueva clase únicamente para almacenar el método main

Creamos una instancia de la clase HolaMundo a través del operador new

Llamamos al método de instancia imprimeHola

Para ejecutar la clase y ver el resúltado, seleccionamos la clase que queremos ejecutar y presionamos el botón play o el acceso directo **CTRL + F11**

Si nosotros le especificamos en Eclipse donde se encuentra la fuente de Java (es un zip que tiene todo el código fuente de las librerías de Java) podrán ver el código de las clases que utilizan. Es bastante útil para entender las funciones o ver el javadoc (comentarios que explican que hace cada función, parámetros que recibe, retorna, etc.) rápidamente desde la IDE.

Cuando utilizamos funciones para auto–completar métodos o sobreescribirlos, si Eclipse tiene el source unido mostrará los nombres de las variables,es valioso para comprender que es cada una.

Una manera fácil de adjuntar la fuente es la siguiente:




Presionamos la tecla CTRL y pasamos el ratón sobre la palabra String, como es otra clase se cambiará en un link que al pulsarlo nos lleva al código de la clase String.

## Notepad++

Existen dos opciones para descargar el programa.

– Formato ZIP (v 6.6.9): Contiene los archivos mínimos para ejecutar Notepad++.

– Instalador Ejecutable (v 6.6.9): Más apropiado para cualquier usuario ya que podemos elegir lenguaje, componentes opcionales, etc ..

En caso de que instalemos en formato ZIP, tendremos que descomprimir el archivo en un directorio y ejecutar notepad++.exe. En caso de actualizar desde otra versión anterior la cosa funciona de manera diferente.

### Idiomas

Desde la versión v2.3 puede soportar distintos lenguajes mediante un archivo XML, traducido del original english.xml. La versión actual es la 6.6.9

Despues de instalar Notepad++ con el instalador puede ser que no haga falta, pero si lo ha hecho de la otro manera, tendremos que seguir este procedimiento:

– Descarga el archivo de idioma que quieras y renómbralo por nativelang.xml

– Abre la ventana de diálogo Ejecutar (Inicio/Ejecutar) y escribe lo siguiente: %APPDATA%\Notepad++

– Copia nativeLang.xml en el directorio que se abrirá.

– Inicia Notepad++

Los lenguajes disponibles para Notepad++ los puedes ver en la siguiente dirección: http://notepad-plus.sourceforge.net/es/site.htm en la parte de Archivos de Idiomas.

## Plugins

El plugin que queramos instalar (en formato DLL) hay que colocarlo en el directorio Notepad++\plugins, una vez instalado podremos usarlo mediante el menu “Plugins”.

Os dejo un listado de una serie de plugins que me parecen interesantes (como siempre, dejar comentarios si conoceis alguno mejor que me haya dejado en el tintero):

– Function List: Nos ofrece una lista de funciones acorde con el lenguaje de programación que utilicemos en ese momento. Los lenguajes soportados son: C, C++, Resource File, Java, Assembler, MS INI File, HTML, Javascript, PHP, ASP, Pascal, Python, Perl, Objective C, LUA, Fortran, NSIS, VHDL, SQL, VB y BATCH.

– Explorer Plugin: Es un explorador de archivos para incluirlo en el programa.

– Colour Picker: Traduce el color que selecciones en hexadecimal.

– Compare Plugin: Nos muestra las diferencias entre 2 archivos.

## Uso del autocompletado

Soporte esta opción de autocompletado desde la versión 2.0 y ya estamos en la versión 6.6.9 Para utilizarlo presionamos Ctrl+Espacio para mostrar la lista y asi poder agregar su propio archivo de API en el directorio.

SU_DIR_DE_NPP\plugins\APIs (donde SU_DIR_DE_NPP es el directorio en que tenga instalado Notepad++).

Puedes necesitar algunos de estos archivos de API:

– VB.api

– C.api

– Java.api

– Javascript.api

– Flash.api

- Python.api
- Smarty.api
- Coldfusion.api
- Ruby.api
- C#.api
- Fortran.api
- Pascal.api




Notepad++ es un programa para editar código fuente de cualquier lenguaje de programación. Como tiene soporte para una gran cantidad de lenguajes, interesará no sólo a los desarrolladores de webs, sino en general a toda la comunidad de programadores.

Es de estos editores que poseen ayudas ventajosas para programar, como resaltado de colores, posibilidad de trabajar en varios documentos a la vez, menús contextuales, auto–completar código, etc. Es un gran programa usado por los programadores.

Es un proyecto nacido a partir de un editor de código para desarrolladores llamado Scintilla, Por decirlo de alguna manera, Notepad++ es una distribución de Scintilla, pero con bastantes contribuciones adicionales.

Se distribuye sólo para Windows y está desarrollado en C++, utilizando directamente el API de win32, lo que hace que sea veloz y con un archivo de descarga pequeño.

Entre las características que ofrece Notepad++ podemos destacar:

- Colores en el código para más de 40 lenguajes de programación distintos, entre los que se incluyen todos los que un programador de internet podría tocar, como HTML, Javascript, ASP, SQL, PHP, CSS, Pitón, Ruby, y por supuesto **Java**.

- Muy liviano y poco pesado respecto a los recursos del sistema necesarios. Es ultrarápido.

- Impresión a color de los códigos.

- Permite definir el resaltado de sintaxis para nuevos lenguajes de programación que necesite el usuario.

- Autocompletado de código, también configurable por el usuario o extensible por medio de descargas bajo demanda del programador.

- Multi–Documento permitiendo tener abiertos bastantes ficheros simultaneos.

- Multi–Vista, podemos tener más de una vista de un mismo código, como por ejemplo dos versiones del mismo documento para comparar código editado.

- Permite realizar acciones de Buscar / Reemplazar utilizando incluso expresiones regulares para definir los patrones a reemplazar.

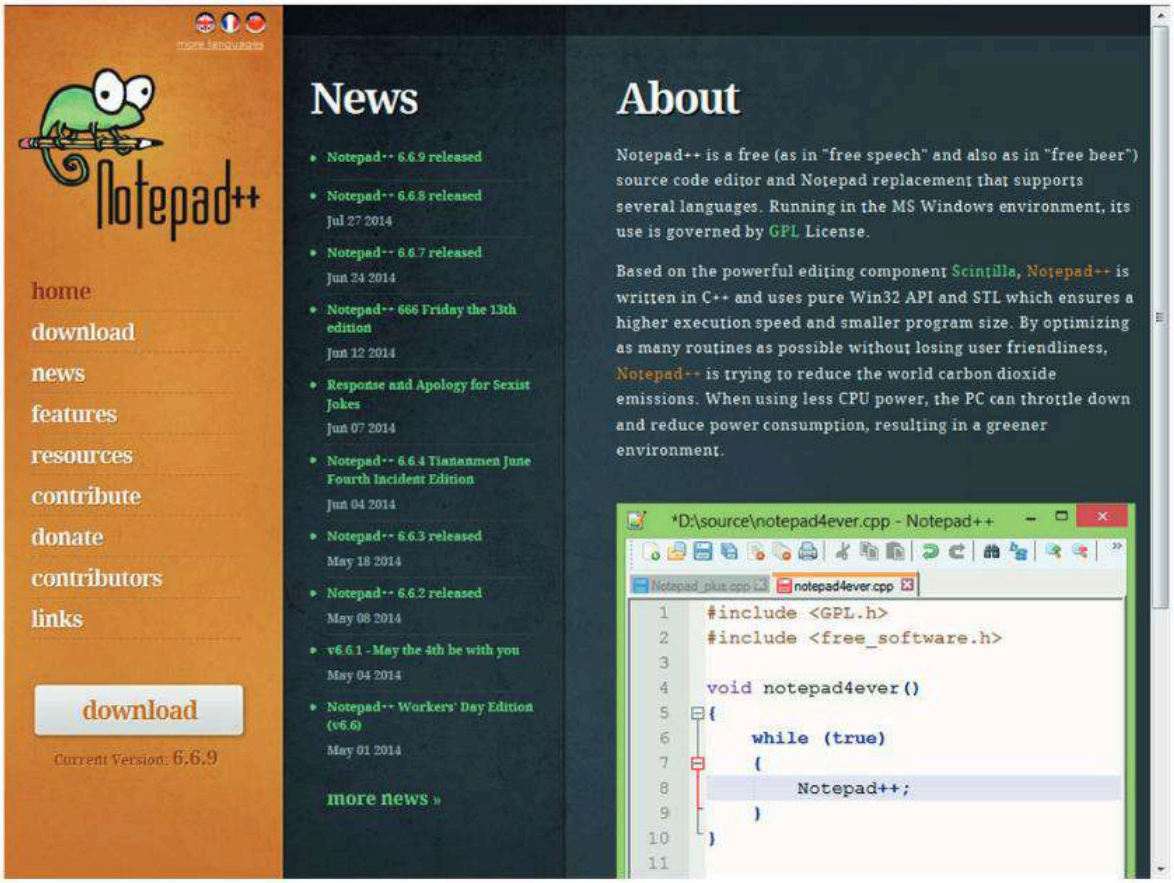
- Detección automática del estado del documento, que puede ayudarnos en caso que queramos guardar un archivo que había sido modificado por otro usuario o programa.

- Otras utilidades como Zoom, soporte para varios idiomas, puntos de marca, resaltado de paréntesis u sangría, creación de macros, etc.

El software se puede descargar y instalar con un programa instalador, además se puede obtener el código fuente. En la web oficial también se ofrecen multitud de plugins que pueden ser interesantes o necesarios para trabajar en una gran variedad de entornos.

### Preparando el entorno de desarrollo. Descargar e instalar Notepad++

Para comenzar a arreglar el entorno de desarrollo para poder trabajar con **Java**. Debemos descargar el editor de código Notepad++ e instalarlo. Para ello entramos en la dirección siguiente: http://notepad-plus-plus.org.




*Pantalla de Notepad++ para descargar desde su Web Oficial*

Cuando la web haya finalizado de cargar, haz click en el apartado download del menú. Encuentra el enlace, en la página web que nos aparecerá, bajamos la versión Installer, por ejemplo Notepad++ v6.9.6.2 Installer. El archivo tiene un nombre similar a npp.6.6.9.Installer.exe.

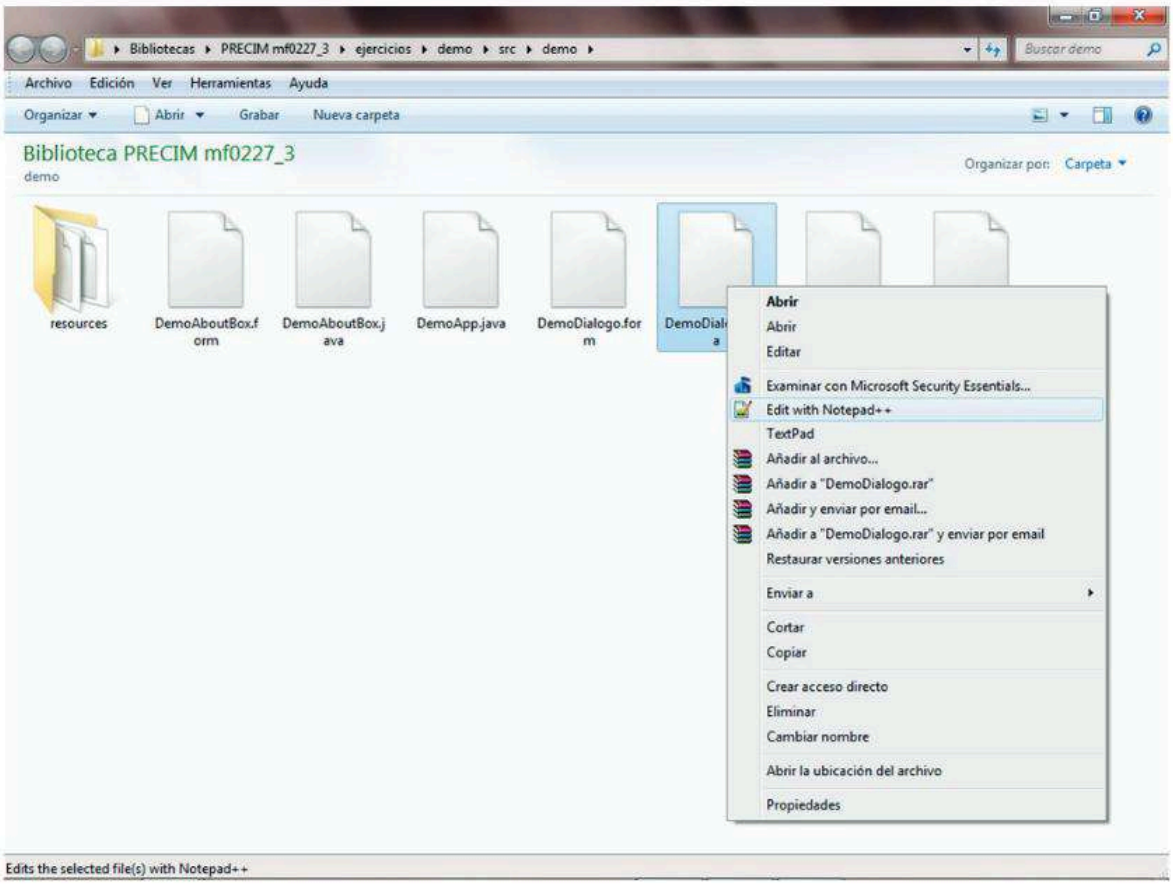
Una vez descargado el programa, lo localizamos en nuestro disco duro el archivo, lo ejecutamos (click sobre el archivo, botón derecho del ratón, ejecutar como administrador) para instalarlo en nuestro ordenador. Para instalar haz simplemente doble click sobre el archivo. El proceso de instalación es muy simple y el asistente nos guiará con más detalle.

Cuando instalemos el software, ya podremos trabajar con el editor de texto Notepad++, con el que abriremos y editaremos cualquier fichero HTML. Podremos acceder al programa desde Inicio → Programas → Notepad++ o desde el icono de acceso directo en el escritorio si se ha creado.

## Ejemplo: comenzar a usar Notepad++

Desde el explorador de archivos de Windows, nos fijamos dentro de las carpetas donde guardemos nuestros trabajos y ejercicios de **Java**.

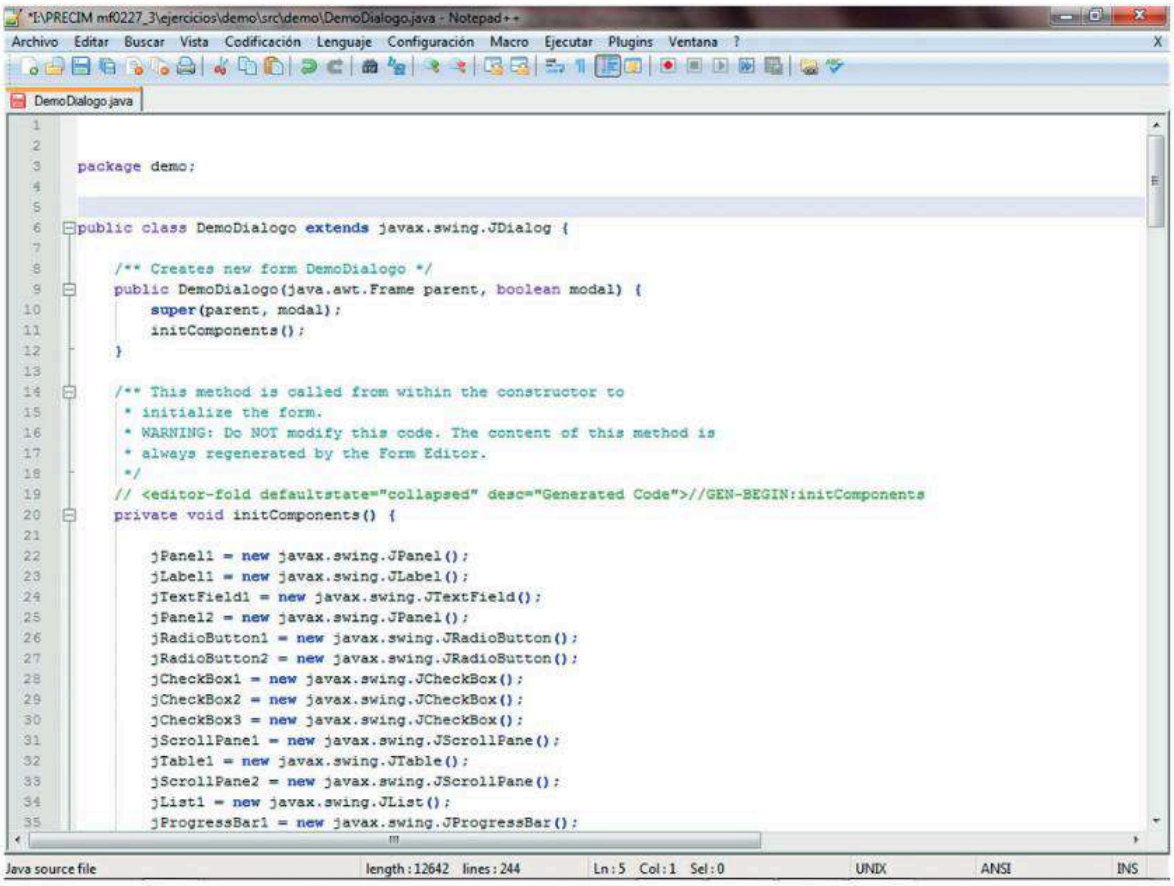
Una vez dentro de la carpeta correspondiente, entramos en ella, en nuestro caso es en la propia del curso donde están todos los trabajos y código fuente. Podemos marcar el fichero correspondiente y con el botón derecho del ratón y al mostrar una ventana emergente del sistema operativo, nos fijamos que tenemos integrado un elemento nuevo que es poder trabajar y abrir los ficheros con **Notepad++**. Otra posibilidad es abrimos Notepad++ y desde dentro pulsamos en Archivo → Abrir. Como nombre de archivo escibimos DemoDialog.java (este nombre de archivo es solo ejemplo). Ya tenemos cargado el código fuente del fichero Java.




*Pantalla de Windows donde tenemos integrado Notepad++*

En el menú desplegado seleccionamos “Edit with Notepad++”.

De una forma u otra, habremos abierto el archivo Java con nuestro editor Notepad++.




*Pantalla de Notepad++ donde hemos abierto un fichero DemoDialog.java*

En **Notepad++** disponemos sin ser un potente IDE como **NetBeans** o **Eclipse** de diferentes funciones, herramientas muy necesarias y útiles para programar en Java.

Aplicar colores diferentes en función de la estructura del código fuente, métodos, instrucciones, clases, comentarios y demás objetos de los ficheros.

Ahora escribimos el siguiente código **Java** y guardamos el archivo.

```
class HolaMundoOO
{
public void imprimeHola()
{
System.out.println (“ Hola Mundo “);
}
}
class HolaMundo
{
public static void main(String[] args)
{
HolaMundoOO miHola = new HolaMundoOO();
miHola.imprimeHola();
}
}
```

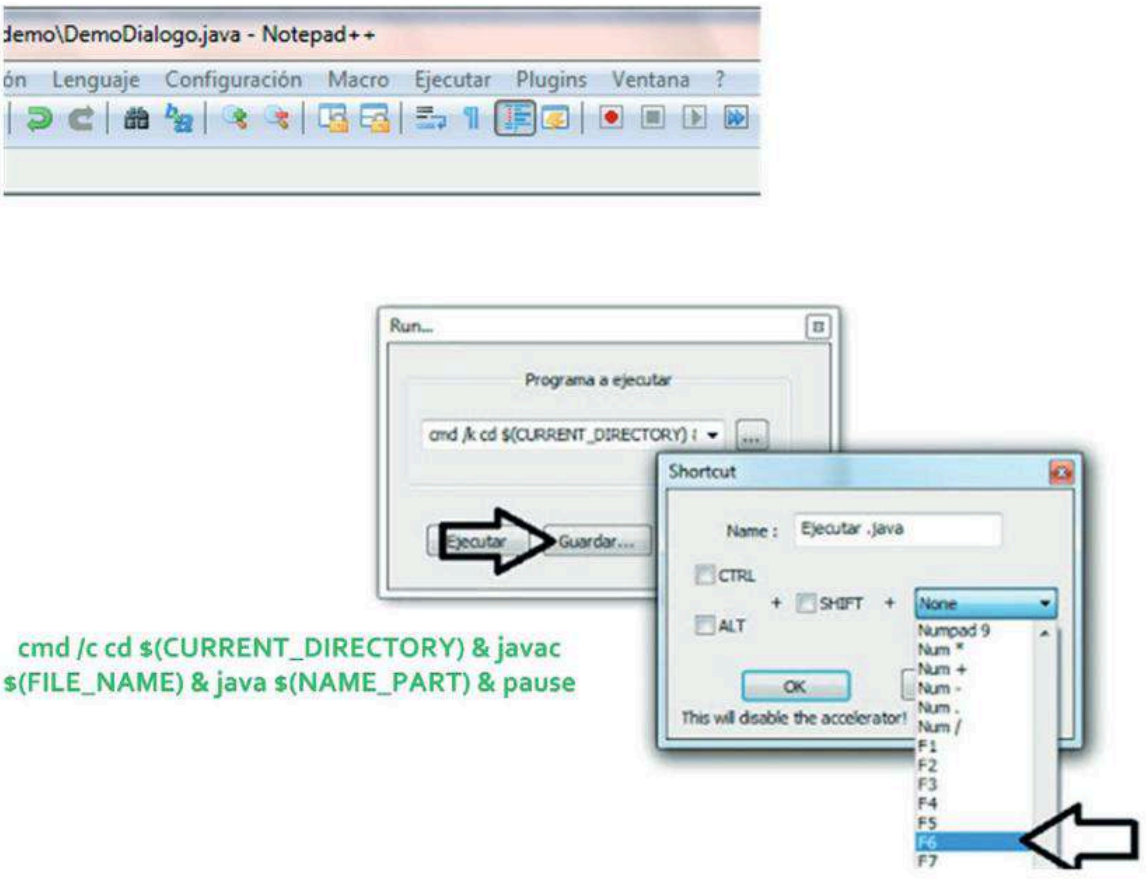
Para comenzar a compilar y ejecutar nuestros programas dentro de **Notepad++** a diferencia de algnos IDE que con solo presionar un botón es algo muy cómodo. Notepad++, es un editor de código ligero y potente que tiene soporte para muchísimos lenguajes, no permite compilar y ejecutar nuestro archivo .java con tan solo pulsar un botón, pero en cambio nos da herramientas para poder hacerlo nosotros mismos.

Para ello entraremos en el editor Notepad++, en la barra de opciones de la parte superior, haremos click en Ficha Ejecutar y seguidamente en Ejecutar…, la primera opción de la lista de opciones que aparecerá.

Seguidamente aparecerá una ventana pequeña donde habrá que introducir la invocación al programa a ejecutar. En este cuadro introduciremos lo siguiente:

```
cmd /c cd $(CURRENT_DIRECTORY) & javac $(FILE_NAME) & java $(NAME_PART) & pause
```

Al teclear el mandato, pinchamos en Guardar…, y en la nueva ventana escribimos el nombre que elijamos a esta sentencia, elegimos la combinación de teclas que queremos pulsar para ejecutar nuestra instrucción, pulsamos OK y ya solo nos queda cerrar la primera ventana llamada Run… clickando en Cancelar o en el aspa de arriba a la derecha.




*Pantalla de Notepad++ donde ejecutaremos el compilador javac con parámetro*

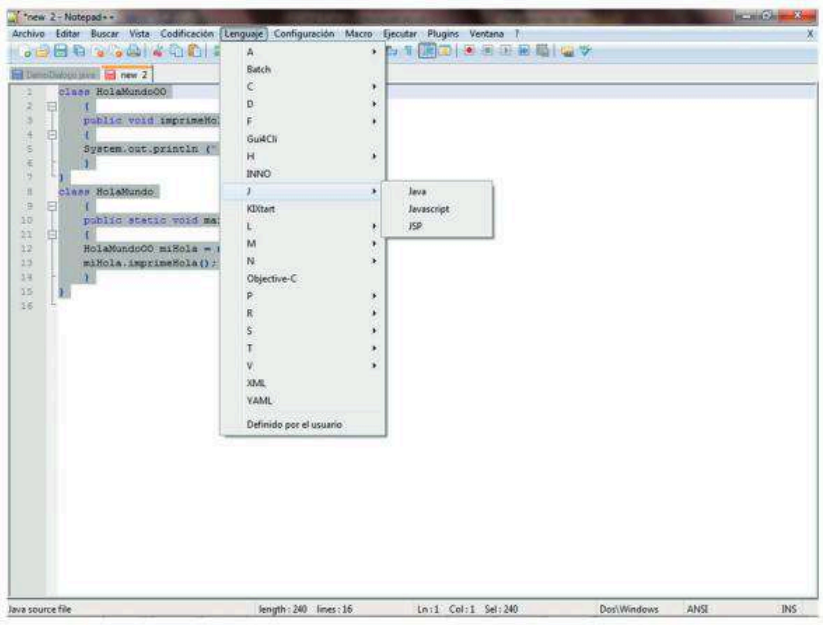
Podemos usar a partir de ahora la tecla F12 siempre que queramos compilar.

Como se ve en la imagen anterior, podemos elegir la combinación de teclas que queramos, yo señalo en la imagen la que yo mismo utilizo, pero se podría utilizar la que queráis, siempre y cuando no esté utilizada ya.

Desde este momento con pulsar F12 o la combinación que hayáis puesto se abrirá un CMD que se encargará de compilar y ejecutar automáticamente nuestro archivo .java.

Es necesario para disponer siempre de nuestro **Notepad++** adaptado a **Java**, configurar en la opción de lenguaje, para que lenguaje queremos trabajar y desarrollar, en nuestro caso como es **Java** se lo indicaremos desde la opción:

LENGUAJE >> J >> JAVA




*Pantalla de Notepad++ donde configuramos trabajar con lenguaje Java*

Tras ver **Notepad++** en este punto desde su instalación y configuración en **Java**, se puede dar por concluido este punto de Herramientas de desarrollo. **NetBeans**, **Eclipse** y **Notepad++** como componentes de software necesarios y potentes para desarrollar en **Java**, ahora es el usuario quien elige que programa prefiere.

## 9.4. Tipos de datos y elementos básicos característicos del lenguaje. Instrucciones

### Tipos de datos

| Tipo | Características | Utilización |
|---|---|---|
| **byte** | 8 bits–complemento a 2 | |
| **short** | 16 bits–complemento a 2 | |
| **int** | 32 bits–complemento a 2 | |
| **long** | 64 bits–complemento a 2 | |
| **float** | 32 bits–IEEE 754 | |
| **double** | 64 bits–IEEE 754 | |
| **boolean** | Valores true o false | |
| **char** | 16 bits–carácter | |
| **String** | Se trata como una clase<br>No se pueden cambiar los caracteres que lo forman | String s;<br>s = “Cadena”<br>s = new String(“Cadena”); |
| **Arrays** | Se distinguen tres operaciones: definir el array, dar tamaño al array y asignar elementos al array<br>Realiza comprobaciones exhaustivas del correcto funcionamiento del array<br>Comienzan por cero | Definición: Int[ ] lista;<br>Dar tamaño: lista = new int[10];<br>Asignación: lista[0] = 6;<br>Asignación objetos: listaS[0]=new String(“c”);<br>Matrices: int tabla[ ][ ] = new int[4][5];<br>Definición y asignación: int [ ] Lista ={ 1, 2, 3 }; |

### Características para utilizarlos

Los tipos de datos int, long, byte, short, float, double, boolean y char son definidos como tipos primitivos, con lo que quiere decir que no son clases. Si queremos trabajar con los tipos primitivos como clases existen clases contenedoras que se definen con nombres similares pero empezando con mayúsculas como por ejemplo Integer, Character, Boolean, Double, y otros más.

## Operadores

Los operadores que disponemos en Java se representan en la siguiente tabla:

Entre los tipos disponibles están los Aritméticos, los Relacionales, Los Condicionales, los Manejo de bits, los Operadores no asignados, los Conversores de tipos y los operadores ternarios.

| Tipo | Operador | Descripción |
|---|---|---|
| **Aritméticos** | +, -, *, / y % | Suma, Resta, Multiplicación, División y Resto |
| | ++, -- | Incremento, Decremento |
| **Relacionales** | <, <= | Menor que, Menor o igual que |
| | >, >= | Mayor que, Mayor o igual que |
| | ==, != | Distinto de |
| **Condicionales** | & | AND |
| | \| | OR |
| | && | AND condicional |
| | \|\| | OR condicional |
| | ! | NOT |
| **Manejo de bits** | >>, << | Desplaz a la derecha, desplaz a la izquierda |
| | >>> | Desplaz sin signo |
| | &, \|, ^ | AND, OR, XOR |
| | ~ | Complemento a |
| **Operadores no asignados** | = | Operador de asignación básico |
| | Operador= (+=, -=, *=, /=, %=, &=, \|=, ^=, <<=, >>=, >>>=) | A operador= B equivale a A = A operador B |
| **Conversión de tipos** | (Tipo) variable o expresión | |
| **Operador ternario** | Expresion ¿ sentencia1 : sentencia2 | if expresion<br>then sentencia1<br>else sentencia2 |

## Estructuras de control

Las estructuras de control que disponemos en Java se representan en la siguiente tabla:

Entre los tipos disponibles están las Bifurcaciones, los Bucles, El Manejo de Excepciones y el Control de flujo.

Todas las estructuras de control son muy conocidas en un lenguaje de programación como Java, además de usarse muchísimo en todos los programas que hagamos en este lenguaje.

| Tipo | Estructura | Sintaxis |
|---|---|---|
| **Bifurcaciones** | if… else | if (condicion) { ... }<br>else { ... } |
| | switch | switch (variable)<br>{<br>case n1: ... break;<br>case n2: ... break;<br>default: ... break;<br>} |
| **Bucles** | for | for (var=min; var<max; inc) { ... }<br>for (Object ó : miArray)<br>{ System.out.println(o); } |
| | while | while (condicion) {... } |
| | do… while | do<br>{ ... }<br>while (condicion) |
| **Manejo de excepciones** | try… catch… finally | try { ... }<br>catch (excepción) { ... }<br>finally { ... } |
| | throw | throw excepción; |
| | throws | type NombreMetodo (argumentos)<br>throws excepciones { ... } |
| **Control de flujo** | break | break [etiqueta]; |
| | continue | continue [etiqueta]; |
| | return | return expresión; |

## 9.5. Estudio y utilización de las clases básicas incluidas en la librería de clases

### Manejo de imágenes

En Java es muy sencillo insertar una imagen a sus elementos y hacer que las aplicaciones guarden una mejor apariencia.

Para agregar una imagen en cualquier elemento debemos de usar el método "SetIcon" del componente en el que queremos insertar una imagen, este método toma un objeto de tipo "ImageIcon" por lo que debemos de construir un nuevo ImageIcon con un constructor, que recibirá un String que representa la dirección en la que se encuentra la imagen que deseamos añadir, para poder usar imágenes que se situan en la misma carpeta que la clase que creamos, tenemos que utilizar el método "getResource" de la clase, este método nos retorna la dirección del archivo que se localice en el directorio en el que está guardada la clase.

También se puede utilizar la clase Image para conseguir distintas acciones que sirvan para manejo de imágenes, y para ello usaremos un Toolkit.

Un Toolkit es un conjunto de herramientas que nos permiten obtener imágenes, obtener el tamaño de la pantalla, entre otras cosas.

Si necesitamos construir un Toolkit usaremos el método getDefaultToolkit() de la clase Toolkit. Una vez que ya tenemos creamos un Toolkit, podemos utilizarlo para crear una nueva imágen mediante el método getImage() , que recibe la dirección en la que se encuentra el archivo, implementando el método getResource() de la clase, para conseguir ficheros que se encuentran en el mismo directorio que esta. Una vez que ya creamos el objeto Image podremos usar el constructor de ImageIcon que usa una imagen para ponerle el ícono al botón.

Partiendo de lo antes definido brevemente y al ver la necesidad que tienen los expertos del Instituto Especial Stephen Hawking, en la educación de sus niños y que para ello utilizan tarjetas con dibujos, libros con imágenes coloridas, etc.,

### Java Sound

Es un API muy usada en Java para implementaciones multimedia y su presentación en Applets y aplicaciones. En el supuesto con el sonido, Java Sound API permite capturarlo, su tratamiento, el almacenamiento y la presentación de sonido, tanto en formato MIDI como sonidos muestreados, aunque no se pueden tratar todos los formatos existentes de almacenamiento y transmisión de sonido.

Este API se compone de 4 paquetes:

– javax.sound.sampled

– javax.sound.sampled.spi

– javax.sound.midi

– javax.sound.midi.spi

EL paquete javax.sound.sampled, tiene las clases vitales para el control del sonido muestreado, esto incluye capturarlo, su mezcla, sampleado y la reproducción de audio, facilitando algún control y efecto sobre el sonido así como interfaces para el almacenaje.

En el paquete javax.sound.midi tenemos las interfaces de síntesis, secuenciamiento y transporte MIDI.

Otros paquetes como javax.sound.sampled.spi y javax.sound.midi.spi facilitan una interfaz para los desarrolladores basados en las interfaces anteriores.

En Java la reproducción de audio es un proceso muy sencillo, lo que tenemos que hacer es utilizar la clase AudioClip, que posee tres métodos:

– play(), se utiliza si queremos reproducir una vez el fichero como parámetro.

– loop(), para reproducir indefinidamente el fichero como parámetro.

– stop(), para detener el sonido que se está reproduciendo.

Para arrancar un objeto AudioClip usamos el método getAudioClip, de la clase Applet, de la siguiente forma:

```
getAudioClip(getDocumentBase(),"archivo.wav")
```

Los ficheros .wav son formas de onda puros.

Formatos de audio

Los distintos objetos de la clase AudioFormat especifican el formato de una señal de audio, como el número de canales, el número de bits por muestreo, frecuencia Hz de muestreo, el tamaño de la trama de voz y su frecuencia.

Junto a la anterior además está la clase AudioFileFormat para especificar el formato de un fichero de audio, esta clase posibilita emplear las clases AudioImputStream y AudioOutputStream, que heredan de ImputStream y OutputStream consecutivamente, permitiendo usar los métodos estándar para la lectura y escritura del audio en archivos.

## Reproducción de sonido

Primeramente antes de tocar la reproducción de sonidos con objetos SourceDataLine mencionaremos los objetos Clip. Un objeto de esta clase está diseñado para guardar el audio grabado de principio a fin, es la mejor solución cuando se conoce de antemano el tamaño de la señal a almacenar y es única, así como cuando se desea repetir un sonido varias veces por ejemplo en un bucle. El ejemplo típico de uso es la reproducción de sonido de ficheros no muy grandes, en este caso se leerá el contenido del fichero en un clip y luego se reproducirá, el control es más manejable y los recursos utilizados son menores. Sin embargo en los supuestos donde el audio es continúo o en ficheros de un gran tamaño es recomendable usar los otros tipos de DataLine SourceDataLine o TargetDataLine con el fin de no ocupar mucha memoria del sistema.

La forma más sencilla de reproducir sonidos es por medio del método play():

```
play( Url directorioSonido,String archivoSonido );
play( Url unaURLdelSonido );
```

Asignando una Url normal al método play() es la carpeta donde está el fichero Html. Se podrá acceder a esa ubicación invocando el método getDocumentBase() de la clase Applet.

```
play( getDocumentBase(),"trompeta.au" );
```

Para que carge el archivo y lo reproduzca por los altavoces, el fichero de la clase y el fichero trompeta.au deberían estar en el mismo directorio.

### Reproducción repetitiva

Si queremos cargar un clip de sonido, se usará el método getAudioClip():

```
AudoClip sonido;
sonido = getAudioClip( getDocumentBase(),"guitarra.au" );
```

En el momento que se carga el clip de sonido, se pueden usar tres métodos distintos:

– sonido.play(); para reproducir el clip de sonido.

– sonido.loop(); para iniciar la reproducción del clip de sonido y que entre en un bucle de reproducción, es decir, en una repetición automática del clip.

– sonido.stop(); para parar el clip de sonido que se halle en ese momento en reproducción.

Son métodos de clase muy usados en entornos multimedia donde programamos en Java utilidades muy interesantes cargando diferentes sintonias y efectos musicales en nuestros programas de manera sencilla y que resulta de gran valor en el desarrollo de herramietas didacticas de cursos y entornos de personas y niños/as con diferentes discapacidades.

## Paquetes de Java

Los paquetes agrupan en ficheros únicos clases e interfaces. Los nombres de los paquetes son términos separados por puntos y se guardan en directorios que coinciden con esos nombres.

EL uso de paquetes en nuestro código fuente amplia la potencialidad con herramientas que aportan valor añadido a nuestros programas.

Un ejemplo, de ficheros que contienen código fuente Java: Applet.java, AppletContext.java, AppletStub.java, AudioClip.java contienen en su código la siguiente línea: package java.applet;

Y las clases que se obtienen de compilar de los ficheros anteriores, que se encontrarán con un nombre parecido a nombre_clase.class, en el directorio que tengamos bajo Path en : java/applet

Es muy necesario documentarse bien sobre el uso y contenidos de los paquetes para aplicar sus clases y métodos con las propiedades adecuadas. Es sin duda un factor muy importante y potente que aporta Java como lenguaje de programación OOP.

### Import

Los paquetes de clases se insertan en nuestro código con el término clave import, indicando el nombre del paquete como ruta y nombre de la clase. Se pueden asignar varias clases utilizando un asterisco.

```
import java.Date;
import java.awt.*;
```

Si un fichero fuente Java no contiene ningún package, se pone en el paquete por defecto sin nombre. Es decir, en el mismo directorio que el fichero fuente, y la clase puede ser cargada con la instrucción import:

```
import MiprimeraClaseEjemplo;
```

## Paquetes de Java

El lenguaje Java suministra paquetes que contienen herramientas, utilidades, un sistema de entrada/salida general y comunicaciones.

– java.applet posee clases desarrolladas para usar con applets. Hay una clase denominada Applet y tres interfaces: AppletContext, AppletStub y AudioClip.

– java.awt Este paquete "Abstract Windowing Toolkit" más conocido como Awt entre programadores contiene clases para crear widgets y componentes GUI Interfaz Gráfico de Usuario. También Incluye las clases Button, Checkbox, Choice, Component, Graphics, Menu, Panel, TextArea y TextField.

– java.io El paquete de entrada/salida conlleva clases de acceso a ficheros como: FileInputStream y FileOutputStream.

– java.lang Este paquete incluye las clases del lenguaje Java propiamente dicho como: Object, Thread, Exception, System, Integer, Float, Math, String, etc.

– java.net Este paquete da soporte a las conexiones del protocolo Tcp/ip y también incluye las clases Socket, Url y URLConnection.

– java.util Este paquete es como un cajón de sastre con gran variedad de clases útiles para muchas aplicaciones en programación. Se incluyen, entre otras, Date (fecha), Dictionary (diccionario), Random (números aleatorios) y Stack (pila FIFO).

## Diagrama de clases

Ahora explicaremos las características de cada clases, interfaces, colaboraciones y relaciones de dependencia y generalización.

Una clase está simbolizada por un rectángulo que dispone de tres apartados, el primero para indicar el nombre, el segundo para los atributos y el tercero para los métodos. Cada clase debe poseer un nombre único, que las diferencie de las otras.

Un atributo simboliza propiedades de la clase que se pueden encontrar en todas las instancias de la clase. Los atributos pueden representarse solo mostrando su nombre, mostrando su nombre y su tipo, e incluso su valor por defecto.

Un método es la aplicación de un servicio que posea dicha clase, que enseña un comportamiento común a todos los objetos.

## Clases útiles

Esta tabla describe una serie de clases que serán de gran utilidad para los usuarios/as. Tan solo bastaria con documentarnos bien internamente sobre las bondades y usos de dichas clases para aplicarlas y sacarles provecho.

| | |
|---|---|
| **Wrappers** | Son clases credas para complementar los tipos primitivos ya que no son objetos, insertando al valor en si especificado como una variable miembro de la clase con una serie de métodos para implementar su uso. Los wrappers definidos son Byte, Short, Integer, Long, Float y Double |
| **java.lang.Math** | Suministra métodos para hacer operaciones matemáticas, así como algunas constantes como PI o E.<br>Ej. Math.random() |
| **Colecciones** | Es toda una colección como un conjunto de objetos que se agrupan, cualquier colección se identifica por el interfaz Collection.<br>Son colecciones las clases java.util.Vector, java.util.HashSet… y también son colecciones los interfaces java.util.List, java.util.Map…<br>– Collection: especifica métodos para manejar una colección genérica de elementos.<br>– Set: Colección de elementos que no admite repetición.<br>– SortedSet: es un Set para ordenar según un criterio establecido.<br>– List: soporta elementos repetidos y cuida el orden de inserción.<br>– Map: Conjunto de pares, clave/valor, sin reiteraciones de claves.<br>– SortedMap: es un Map con ordenación según un criterio marcado.<br>– Iterator: Interfaz de soporte usado para recorrer una colección y para borrar elementos.<br>– ListIterator: interfaz de soporte que permite recorrer List en ambos sentidos.<br>– Comparable: interfaz de soporte que define el método compareTo() que permite ordenar las diferentes colecciones según un orden natural.<br>– Comparator: interfaz de soporte que declara el método compare() y se utiliza en lugar de Comparable: cuando se desea ordenar objetos no estándar o sustituir a dicha interface. |

| | |
|---|---|
| java.awt | AWT son las siglas de Abstract Windows Toolkit y es un paquete Java que permite crear interfaces gráficas. Para construir una interfaz, se necesitan al menos tres elementos, un contenedor, unos componentes y un modelo de eventos.<br><br>El contenedor es la ventana o parte de la ventana donde se colocan los componentes. Los elementos son combo, menús, botones, barras de desplazamiento, cajas y áreas de texto, etc.<br><br>El modelo de eventos es la herramienta que nos posibilita controlar las acciones del usuario sobre los componentes, de modo que cada vez que el usuario hace una acción se genera un evento y AWT creará un objeto de la clase de evento correspondiente derivada de AWTEvent. El evento será luego gestionado por un objeto que conocerá el componente que lo ha tomado.<br><br>De forma que se distinguen dos tipos básicos de objetos, los que reciben los eventos (event source) y los que manejan los eventos (event listener).<br><br>Los objetos event source deben 'registrar' los objetos que habrán de gestionar sus eventos, mientras que los objetos event listener deben implementar los métodos adecuados para manejar un cada evento. La forma más sencilla de hacerlo es implementar el interfaz Listener de forma adecuada. |
| Threads | Los threads como hebras o hilos de ejecución posibilitan organizar los recursos del ordenador de forma que puedan haber varios programas o varias partes del mismo programa rodando en paralelo. Un hilo de ejecución hará las acciones indicadas en el método run(). |
| Exceptions | Las excepciones y su tratamiento como manera de capturarlas, acción de respuesta a las mismas. Cabe nombrar que una excepción no es igual que un error, ya que que las excepciones son recuperables, sin embargo los errores no lo son, de hecho las excepciones están agrupadas en una jerarquía que parte de la clase java.lang.Exception y los errores está organizados a partir de la clase java.lang.Error.<br><br>Es bueno comentar la posibilidad que tiene el programador de lanzar excepciones throw cuando se crea conveniente al detectar un dato en formato incorrecto para el programa, por ejemplo. Ésto se haría mediante objetos de dicha clase Exception. |

Ejemplos de código con diferentes clases:

```
//fichero: HolaJava1.java
public class HolaJava1 extends javax.swing.JComponent {
// se trata de una clase que es un JComponent.

  public static void main(String[] args) {
   javax.swing.JFrame f = new javax.swing.JFrame("HolaJava1");
   f.setSize(300, 300);
   // añade un componente al contenedor.
   f.getContentPane().add(new HolaJava1( ));
   f.setVisible(true);
  }

  public void paintComponent(java.awt.Graphics g) {
   // repinta el objeto cada vez que es necesario
   g.drawString("Hola, Java!", 125, 95);
  }
}
```

Otro ejercicio muy didáctico donde se trabaja con las clases y los paquetes explicados. Recordar la razón de los import y el objetivo de Constructor.

```
//fichero: HolaJava2.java
// import más complejo
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
```

```
public class HolaJava2 extends JComponent implements
MouseMotionListener {
// es una clase componente que va a responder a los eventos
relacionados con el movimiento del ratón

  // Coordenadas del mensaje
  int messageX = 125, messageY = 95;
  String theMessage;

  public HolaJava2(String message) {
   //constructor
   theMessage = message;
   // establece que el objeto que va a manejar los eventos del ratón va a
ser él mismo.
   addMouseMotionListener(this);
  }

  public void paintComponent(Graphics g) {
   g.drawString(theMessage, messageX, messageY);
  }

  public void mouseDragged(MouseEvent donde) {
   //para el evento drag and drop se llama a este método.
   // Almacena la posición del ratón y muestra el mensaje.
   messageX = donde.getX( );
   messageY = donde.getY( );
   repaint( );
  }

  public void mouseMoved(MouseEvent donde) {}
 // no se necesita por ello no hace falta implementarlo
```

```
  public static void main(String[] args) {
   JFrame venAbre = new JFrame("HolaJava2");
   // Hace que la aplicación termine cuando se cierre la ventana.
   // WindowAdapter se utiliza para crear listeners para los eventos de la
ventana              venAbre.addWindowListener(new WindowAdapter( ) {
    public void windowClosing(WindowEvent we) { System.exit(0); }
   });
venAbre.setSize(300, 300);
 ven.Abre.getContentPane( ).add(new HolaJava2("Hola, Java!"));
 venAbre.setVisible(true);
  }
}
```

```
//fichero: HolaJava4.java
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

// esta clase es una hebra, es la segunda forma de crear hebras, con
Runnable
public class HolaJava4 extends JComponent
  implements MouseMotionListener, ActionListener, Runnable {

  // Coordenadas del mensaje
  int messageX = 129, messageY = 105;
  String theMessage;
```

```
  JButton theButton;

  int colorIndex; // Índice del Color que tiene seleccionado del array.
  static Color[] someColors = { Color.black, Color.red,
  Color.green, Color.blue, Color.magenta };

  // variable booleana para decir si parpadea ó no.
  boolean blinkState;

  // el constructor crea además de todo lo anterior en nueva hebra y la
inicializa.
  // esta hebra se va a encargar de mostrar y ocultar el mensaje con
una ejecución
  // separada a la ejecución principal de este programa.
  public HelloJava4(String message) {
   theMessage = message;
   theButton = new JButton("Cambiar Color");
   setLayout(new FlowLayout( ));
   add(theButton);
   theButton.addActionListener(this);
   addMouseMotionListener(this);
   Thread hilo = new Thread(this);
   hilo.start( );
  }

 // según el valor de la variable blinkState, cambia el color a uno nuevo
ó al mismo color del fondo.
  public void paintComponent(Graphics gradiente) {
   gradiente.setColor(blinkState ? getBackground() : currentColor( ));
   gradiente.drawString(theMessage, messageX, messageY);
  }
```

```
  public void mouseDragged(MouseEvent donde) {
   messageX = donde.getX( );
   messageY = donde.getY( );
   repaint( );
  }

  public void mouseMoved(MouseEvent donde) {}

  public void actionPerformed(ActionEvent donde) {
   // nos asesguramos que se ha pulsado el botón
   if (donde.getSource( ) == theButton)
     changeColor( );
  }

  synchronized private void changeColor( ) {
   // Se pasa al siguiente color (se cambia el índice).
   if (++colorIndex == someColors.length)
    colorIndex = 0;
   setForeground(currentColor( )); // Se usa el nuevo color.
   repaint( ); // Se dibuja de nuevo para que podamos ver el cambio de
color.
  }

  synchronized private Color currentColor( ) {
   return someColors[colorIndex];
  }

 // este método es el que lanza la hebra separada e irá repintando cada
medio seg el mensaje
  public void run( ) {
```

```
    try {
     while(true) {
      blinkState = !blinkState; // Toggle blinkState.
      repaint( ); // Show the change.
      Thread.sleep(500);
     }
    }
    catch (InterruptedException ie) {}
   }

   public static void main(String[] args) {
    JFrame f = new JFrame("HolaJava4");
    // Hace que la aplicación termine cuando se cierre la ventana.
    f.addWindowListener(new WindowAdapter( ) {
     public void windowClosing(WindowEvent we) { System.exit(0); }
    });
    f.setSize(300, 300);
    f.getContentPane( ).add(new HolaJava4("Hola, Java!"));
    f.setVisible(true);
   }
}
```

Algunas clases algo más complejas y externas implementadas por diferentes programadores y fabricantes de software. Entre clases libres y de pago existen una enorme variedad. En el ejercicio anterior se ha tratado de mostrar los usos de semejantes clases como las que podamos encontrarnos o adquirir por módicos precios para ser implementadas en nuestros proyectos y aplicaciones Java.

## 9.6. Definición de clases

En este apartado se describirá de manera clara la forma habitual de especificar y crear una clase, con los atributos y métodos, así como las distintas posibilidades con las podemos contar para crearlas y como luego su uso e instanciación de un objeto de dicha clase.

En general, la forma usual de definir una clase es:

```
[visibilidad] [modificador] class NombreMiClase [herencia]
[implementación] {
   // declaración de variables y métodos
 sentencias….
instrucciones  ...
}
```

La explicación para cada sección del código fuente de la clase, es la siguiente:

La visibilidad es el ambiente en el que es conocida una clase, por ejemplo determina desde qué paquetes o clases es posible crear objetos de dicha clase y trabajar con ellos.

Con public indicamos que la clase puede ser vista desde cualquier otro paquete o clase. En cambio con package la clase solo puede ser accedida dentro del mismo paquete. Es la opción por defecto si no se especifica nada.

Podemos utilizar el modificador abstract para precisar un tipo de clases de las que no se consiguen crear objetos. Su utilidad es que otras clases deriven o hereden (extends) de ella, proporcionándoles un marco o modelo a seguir y una serie de métodos de utilidad general.

Es muy viable poseer métodos totalmente definidos dentro de una clase abstract y además tener métodos abstract.

La herencia, es simple solo entre una clase hija y otra padre, se utilza usando la palabra reservada extends más la clase de la que se hereda clase padre. Ejemplo class Circulo extends Figura. Debemos recordar que mediante la herencia alcanzamos que la clase hijo disponga de las mismas variables y funciones que la clase padre, las cuales puede redefinir. Además, dicha clase podrá poseer otras variables y métodos propios.

```
package calculafactors2;
public class calcfactorial {
private int numero;
private double volcadoresult;
public calcfactorial(int num){
this.numero=num;
this.volcadoresult=1;
}
public double retornaFactorial(){
for (int i=1; i<=this.numero;i++){
this.volcadoresult*=i;
}
return this.volcadoresult;
}
}
```

## 9.7. Construcción de métodos. Sobrecarga

### Métodos (funciones)

Los métodos son como funciones ya que no pueden denominarse como tal funciones, descritas dentro de una clase, y la manera general de declararlos es:

```
[visibilidad] [modificador] Clase_ValorRetorno nombredeMétodo
( [Clase_parametro1 nombreParametro1, ..., Clase_parametroN
nombreParametroN] ) {
    instrucciones;
}
```

La sección visibilidad será public, package, private o protected, con el mismo significado que en el caso de los atributos.

Los parámetros del método se envian continuamente por valor, aunque se modifiquen dentro del mismo, su valor no se verá afectado fuera de él. En el momento que se pasan referencias, como arrays ú objetos, la referencia tampoco podrá cambiar, pero sus valores los elementos del array o los atributos del objeto, si que podrán hacerlo.

Respecto a los parámetros de los métodos hay que indicar que son optativos, de modo que si no tienen, basta con dejar los paréntesis vacíos.

Los métodos además se pueden declarar con el modificador abstract, al igual que las clases. Su utilidad es precisar un modelo de método para las clases que hereden de aquella en la que estén definidos. Si una clase tiene un método abstract, entonces la clase deberá ser abstract.

Los métodos poseen visibilidad directa de las variables miembro de la clase en la que se definieron, quiere decir que pueden hacer referencia a ellas.

Dentro de los métodos se pueden declarar variables locales que tendrán como ámbito dicho método al finalizar las eliminará el recolector de basura.

Se usa la palabra **this** para tener referencia a algún componente de la clase misma en la que se encuentra el método. Del mismo modo, para hacer referencia a una variable o método de la clase padre de en la que esté el método en caso de que exista, se utiliza el término **super**.

Por ello el dato a devolver es único y se retornará utilizando la instrucción return en el interior del cuerpo del método, terminando el mismo de forma inmediata.

También existen métodos de clase, de manera similar a las variables y se definen de la misma manera colocando static y que solo tienen sentido a nivel de clase. Por ejemplo método que aumenta el número de objetos creados de una clase.

Se conoce un tipo de métodos que deben definirse para todas las clases para eludir errores, son los denominados constructores y son como funciones que tienen como fin inicializar los atributos de un objeto a la hora de construirlo. Tienen que tener el mismo nombre que la clase en la que se definen y no poseen valor de retorno. Pueden existir varios, pero con distinto número de parámetros. Los constructores son llamados cuando inicializamos un objeto con la instrucción NEW.

Los constructores se pueden invocar con mismo nombre de la clase y los parámetros pertinentes desde dentro de otros constructores o métodos de clase, incluido de clases hijas utilizando super, pero no pueden ser llamados desde otro tipo de métodos.

Si no se define ninguno, Java crea uno automáticamente en el que inicializa las variables con sus valores por defecto.

Es viable declarar varios métodos con el mismo nombre dentro de la misma clase, aunque con distinto número o tipo de parámetros, es lo que se denomina en Java como sobrecarga de un método y con ella se implementa la propiedad polimorfismo en Java.

De la misma manera, es viable sobreescribir los métodos heredados en la clase hija, es la conocida como redefinición.

## 9.8. Construcción de atributos

### Atributos (variables miembro)

Para expresar un atributo de una clase, lo declaramos como cualquier variable dentro del cuerpo de la clase, la forma es:

```
[visibilidad] TipodeClase nombredetributo;
```

La visibilidad será **public, package, private** o **protected**, siendo public y package iguales a los comentados para la definición de clases y private y protected son validos para asegurar que los atributos solo se puedan acceder desde dentro de la misma clase desde los métodos de ésta.

Se conocen un tipo de variables denominadas “variables de clase” que se crean anteponiendo el término reservado static y que poseen como característica que son variables comunes a todos los objetos, pero que únicamente tienen sentido y solo se usan a nivel de clase. Por ejemplo un contador de objetos diseñados de una clase. Se usarían para declarar constantes que pueden usar todos los objetos de la misma clase.

Finalmente ver las variables definidas con la palabra clave Final, que son aquellas que no pueden modificar ni variar su valor a lo largo de la ejecución son equivalentes a las constantes.

En el siguiente ejercicio se muestran varios bloques de código dnde se muestra el procedimidneto apra crear unos atributos dentro del cuerpo de la clase.

```
// Este ejercicio crea un atributo para el título de un libro.
attr = new BibliotecaLibros(libroestante);
attr.setNombre(“Título del Libro”);
attr.setDescripcion(“Creación de Empresa Digitales.”);
attr.setTipo(Constante.varchar);
attr.setTamano(120);
attr.add();
// Este parte define un atributo para el número de páginas de un libro.
attr = new BibliotecaLibros (libroestante);
        attr.setNombre(“Número de páginas del libro”);
      attr.setDescripcion(“Almacenamos el nº de páginas del libro en concreto.”);
attr.setType(Constante.int);
        attr.setMin((short) 0);
        attr.setMax((short) 100000);
    attr.add();
```

## 9.9. Construcción de la interfaz de la clase

Para expresar una clase que implemente un interfaz ya definido se usa la palabra reservada implements La implementación se refiere a las posibilidades que ofrecen los interfaces, usando el nombre del interfaz o interfaces separados por comas a implementar. Ej. class Cuadrado implements Interfaz Figura. Recordamos nuevamente que un interfaz es un conjunto de declaraciones de funciones vacías, se podría decir, unidas bajo un nombre al igual que una clase.

La siguiente linea muestra la definición de un interfaz sería:

```
[visibilidad] interface NombredeInterfaz [herencia] [implementación] {
   // definición de métodos   ...
Sentencias ....
}
```

Lo mismo que sucede con las clases, un interfaz puede heredar con la sentencia “extends” de otro ú otros acción que no pueden hacer las clases. Del mismo modo, también podrá implementar con “implements” a otro ú otros.

Los interfaces se pueden declarar en archivos con su nombre y solo podrán ser public o package al igual que las clases.

### Objetos

Los objetos son instancias de las clases, cada objeto es diferente de los demás, aunque sean de la misma clase, por lo que tendrá valores propios para sus atributos que determinarán un estado diferente de los demás.

Para construir un objeto lo declaramos como de una clase y con posterioridad lo inicializamos invocando al constructor, por ejemplo:

```
MiClase miObjeto;
miObjeto = new MiClase();
```

Para acceder a los atributos y los métodos que posee el objeto de los que ofrece la clase y cuando pueda según la visibilidad declarada, suficiente con usar el operador ' . ', por ejemplo:

```
miObjeto.x = 10;
miObjeto.calcularY(20);
```

Ahora en esta sección tocaremos los packages, su manejo y su utilidad en Java.

A un conjunto de clases relacionadas de alguna forma, se le denomina package es similar a una librería en otros lenguajes. Todas las clases del mismo package estarán en el mismo directorio, de forma que se establece una relación directa entre package y directorio.

Un usuario podrá construir sus propios packages picando el código fuente al principio del fichero con la clase a incluir la instrucción:

```
package nombredePackage;
```

De manera genérica, estos nombres se pondrán en minúscula y tendrán varios términos separados por puntos, que se corresponden normalmente con la estructura de directorios en la que se encuentra el package, a partir del path que indica la variable **classpath**.

Para utilizar las clases incluidas en un package (las públicas), se usa la instrucción:

```
import nombredePackage.[*|NombredeClase.class];
```

Pudiendo elegir una clase concreta o todas las clases del package con ‘ * ‘.

Esto se debe hacer explícitamente para todos los que queramos importar, aunque sean sub–packages de otros, ya que esto último no se hace de manera automática.

## 9.10. Construcción de clases incluyendo relaciones de agregación/composición y asociación

Respecto a la interacción de nuestros objetos es necesario conocer, los dos términos que cómo mínimo utlizaremos cuando intentamos descifrar la manera en que tus objetos interactuarán son Asociación y Composición.

### Asociación

La asociación se podría definir como el momento en que dos objetos se unen para trabajar juntos y así, alcanzar una meta.

Un punto a tomar muy en cuenta es que ambos objetos son independientes entre sí, es muy importante entender qué implicación tiene este concepto. Para validar la asociación, la frase *“usa un”*, debe tener sentido:

– El ingeniero usa una computadora.

– El cliente usa tarjeta de crédito.

### Composición

En caso contrario, la composición es un tipo de relación dependiente en dónde un objeto más complejo es conformado por objetos más pequeños. En esta situación, la frase “*tiene un*”, debe tener sentido:

– El auto tiene llantas.

– El portátil tiene un teclado.

El siguiente código Java, ayuda a compredner como implementar las Asociación como modo de interacción con muestros objetos.

### Cómo implementar Asociación

Construiremos la relación: El cliente usa tarjeta de crédito.

```
public class Cliente {

    private int id;
    private String Nombre;
    private String Apellidos;
    private TarjetaCredito tarjetaCredito;

    public Cliente() {
        //las instrucciones que el  construtor realice
    }

    public void setTarjetaCredito (TarjetaCredito  tarjetaCredito) {
        this.tarjetaCredito = tarjetaCredito;
    }
}
```

La explicación viene más adelante para darles oportunidad que hagan sus propias comparaciones.

### Cómo implementar Composición

Diseñaremos la relación: El portátil tiene un teclado.

```
public class Portatil {
    private String fabricante;
    private String modelo;
    private String servicioTag;
    private Teclado teclado = new Teclado();
```

```
    public Portatil() {
      //Las instrucciones correspondientes de lo que el constructor vaya a realizar
    }

}
```

De manera similar aunque estimando que hay cierta diferencia: Podremos diseñar un objeto de tipo Cliente y asignarle una TarjetaCredito más tarde mediante el método setTarjetaCredito.

Pero si creamos un objeto Portatil, desde el principio sabemos que poseerá un teclado ya creado, puesto que la variable de referencia teclado es declarada e inicializada al mismo tiempo.

Llamaremos a las clases Cliente y Portatil, clases contenedoras.

De ambos casos podemos deducir que:

En la asociación:

– Cliente está independientemente relacionado con TarjetaCredito, debido a que el cliente puede existir sin necesidad de tener asignada una tarjeta de crédito.

– Se puede asignar o retirar la tarjeta de crédito, sin que la existencia del Cliente se vea afectada No debería verse afectada, esto significa que Cliente no debe fallar si no hay un TarjetaCredito presente.

En la composición:

– Los objetos que constituyen clase contenedora, deben existir desde el principio. (También pueden ser creados en el constructor, no sólo al momento de declarar las variables como se muestra en el ejemplo).

– No hay momento en que la clase contenedora pueda existir sin alguno de sus objetos componentes. Por lo que la existencia de estos objetos no debe ser abiertamente manipulada desde el exterior de la clase.

## Tiempo de vida de un objeto

Para que quede más clara la diferencia entre Asociación y Composición, debemos comprender también, lo que es el tiempo de vida de un objeto. Se define como el tiempo que transcurre desde que un objeto es creado hasta que se destruye. Aplicando esto a la asociación, conseguimos que los tiempos de vida de ambos objetos se cruzan mientras están trabajando juntos, esto es, mientras se encuentran asociados, pero no significa que se hayan creado al mismo tiempo.

En el ejemplo del cliente, primero se cosntruye cliente, después la tarjeta de crédito y luego viene la asociación. o incluso se puede crear antes la tarjeta de crédito. Sus tiempos de vida se cruzan sólo mientras la tarjeta de crédito está asociada al cliente.

En la composición, tanto los objetos componentes como la clase contenedora, nacen y mueren al mismo tiempo. Esto es, tienen el mismo tiempo de vida.Al ser una relación demasiado dependiente, si cualquier objeto muere, se lleva consigo a todos los demás. En el ejemplo de la portátil, si mi teclado se descompone, mi laptop ya no es funcional.

No es lógico ni real que puedo reemplazar el teclado, y que puedo seguir trabajando con el mismo portátil y un teclado distinto. Tienes que analizarlo de la siguiente forma:

Si a tu coche, se le pincha una llanta, podrás reemplazarlas siempre y cuando lo tengas estacionado, Es como modificar el código de la clase, el sistema no está en funcionamiento. Pero ¿Qué ocurre si tu coche está en marcha?, ¿puedes cambiarla al vuelo e impedir que el auto se detenga? No, entonces tú coche deja de cumplir su objetivo en ese instante y para fines prácticos, ya no vale.

Sucede lo mismo con los objetos ya construidos, no puedes reemplazarles elementos sobre la marcha y al vuelo porque no existe mecanismo alguno en la declaración de la clase, que lo permita.

## ¿Asociación o Composición?… Depende

Existen supuestos en que será complicado extraer qué tipo de relación usar cuando ambas encajan:

– Un coche tiene ruedas.

– Un coche usa ruedas para dar circular.

Así que debes tomar en cuenta qué tanta flexibilidad te daría implementar una u otra.

Desde el punto de vista de fabricante de coches, necesito tener control sobre cada una de las piezas que conforman mis coches; así, si alguna pieza sale defectuosa, puedo reemplazarla antes que mi producto llegue al mercado. Me conviene la asociación.

Pero desde el punto de vista de Consumidor final, Si mis ruedas se pinchan, pues tiro el coche entero y me compro uno nuevo. Lo vería como composición. Cuando es totalmente fuera de lugar por ello mcuhas veces es muy complicado extraer las relaciones adecuadamente.

## 9.11. Construcción de clases con herencia

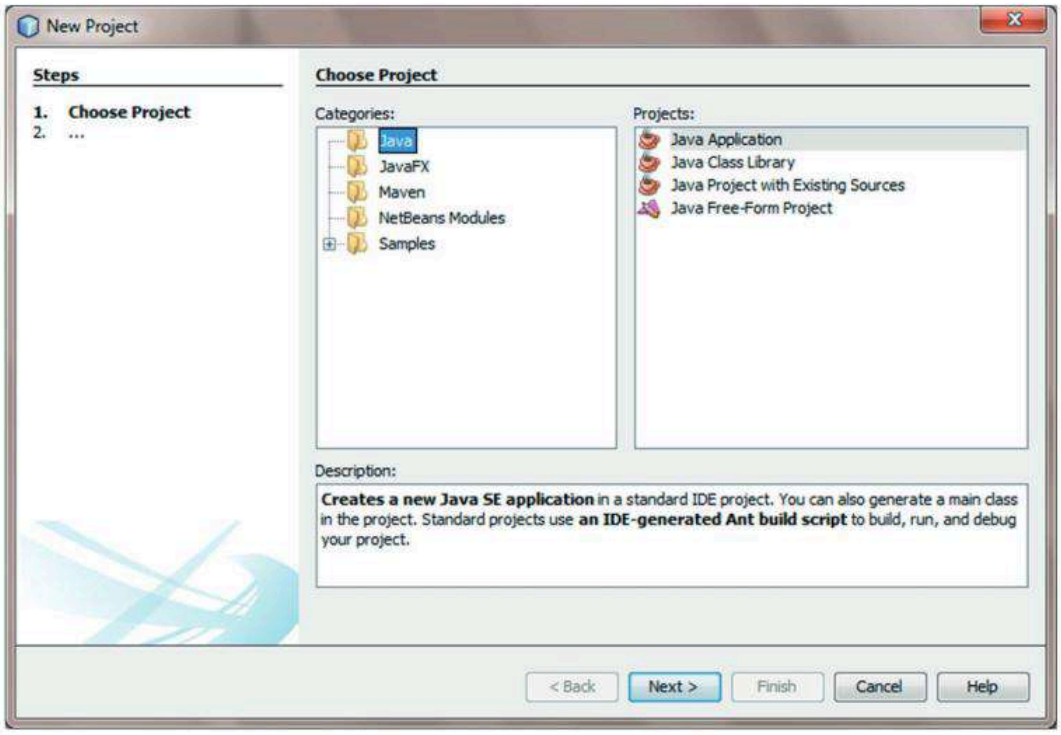
Para entenderlo vamos a realizar un ejercicio con NetBeans.

– Crear un nuevo proyecto NetBeans

  · Seleccionamos opción File → New Project o pulsar Ctrl+Shift+N. El diálogo New Project aparece.

  · En la sección Choose Project seleccionar Java en la sección Categories y Java Application en la sección Projects.

  · Pulsar en Next.




*Pantalla de NetBeans creando una nueva aplicación Java*

En la sección Name and Location, del campo Project Name, escribir Complices. Click Finish.

Create a new project.

Observar que aparece el nodo del proyecto Complices se crea en la sección Projects del IDE NetBeans y que se muestra Main.java en la ventana del editor del IDE.

– Escribir Persona.java

Hacer click con el botón derecho en el proyecto Complices y seleccionar New → Java Class. Observar que aparece el cuadro de diálogo New Java Class con la sección Name and Location.

En el campo Class Name escribir Persona.

En el campo Package, escribir complices o seleccionar complices desde el menú desplegable.

Click en Finish.

Creación de Persona.java

Observar que se genera y muestra en la ventana del editor Persona.java.

Modificar Persona.java como se muestra en el texto. Notar que la clase Persona tiene dos campos, nombre y direccion, y los métodos getter y setter para ellos.

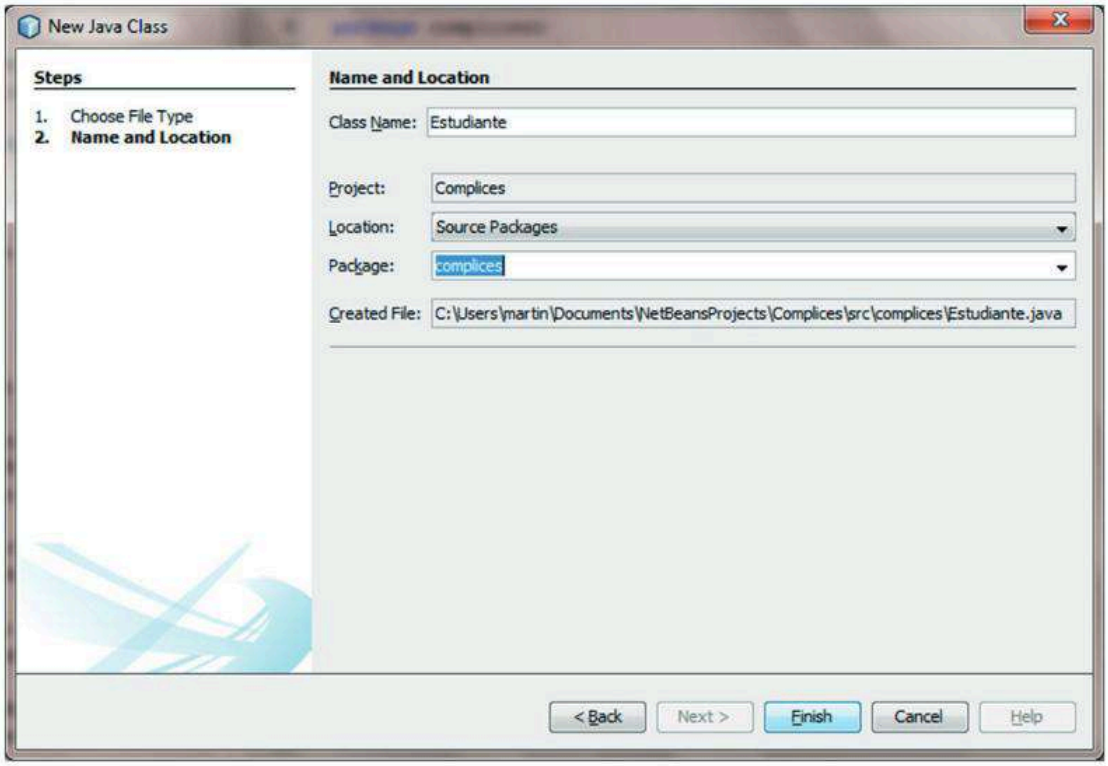
```
/*
 * Para cambiar esta Plantilla elegir Herramientas  | Plantillas (Templates)
 * y posteriormente abria la Plantilla en el Editor de Netbeans
 */
package Complices;
public class Persona {
```

```
    private String nombre;
  private String direccion;
  public String getNombre() {
    return nombre;
  }
  public void setNombre(String nombre) {
    this.nombre = nombre;
  }
  public String getDireccion() {
    return direccion;
  }
  public void setDireccion(String direccion) {
    this.direccion = direccion;
  }
}
```

Este bloque de código de ejemplo se compilará con el nombre de Persona.java que es el mismo nombre que le damos a la clase.

– Escribir Estudiante.java. Notar que la clase Estudiante es una subclase de la clase Persona. Hacer click con el botón derecho en el proyecto Complices y seleccionar New → Java Class. Observar que aparece el cuadro de diálogo New Java Class con la sección Name and Location. En el campo Class Name escribir Estudiante En el campo Package, escribir Complices o seleccionar Complices a partir del menú desplegable.

  Click en Finish.




*Pantalla de NetBeans creando una nueva clase Estudiante*

- Observar que se genera y muestra en la ventana del editor Estudiante.java.
- Modificar Estudiante.java como se muestra en texto Notar que la clase Estudiante tiene dos campos, escuela y grado, y métodos getter y setter para ellos.

```
package Complices;


public class Estudiante extends Persona {


    private String Escuela;

    private double Grado;
```

```
    public String getEscuela() {
        return Escuela;
    }

    public void setEscuela(String Escuela) {
        this.Escuela = Escuela;
    }

    public double getGrado() {
        return grado;
    }

    public void setGrado(double grado) {
        this.grado = grado;
    }
}
```

Este bloque de código de ejemplo se compilará con el nombre de Estudiante.java que es el mismo nombre que le damos a la clase.

– Escribir EstudianteInternacional.java. Notar que la clase EstudianteInternacional es una subclase de la clase Estudiante.

  · Hacer click con el botón derecho en el proyecto Complices y seleccionar New → Java Class.

  · Observar que aparece el cuadro de diálogo New Java Class con la sección Name and Location.

  · En el campo Class Name escribir EstudianteInternacional.

- En el campo Package, escribir Complices o seleccionar Complices a partir del menú desplegable.
- Click en Finish.
- Observar que se genera y muestra en la ventana del editor EstudianteInternacional.java.
- Modificar EstudianteInternacional.java como se muestra en texto. Notar que la clase EstudianteInternacional tiene un campo, pais, y métodos getter y setter.

```
package Complices;

public class EstudianteInternacional extends Estudiante {

    private String Pais;

    public String getPais() {
        return Pais;
    }

    public void setPais(String Pais) {
        this.Pais = Pais;
    }

}
```

EstudianteInternacional.java

– Escribir Profesor.java. Notar que la clase Profesor es una subclase de la clase Persona.

  · Hacer click con el botón derecho en el proyecto Complices y seleccionar New → Java Class.

  · Observar que aparece el cuadro de diálogo New Java Class con la sección Name and Location.

  · En el campo Class Name escribir Profesor.

  · En el campo Package, escribir Complices o seleccionar Complices a partir del menú desplegable .

  · Click Finish.

  · Observar que se genera y muestra en la ventana del editor Profesor.java.

  · Modificar Profesor.java como se muestra en el ejercicio. Notar que la clase Profesor tiene un campo, Especialidad, y métodos getter y setter para su gestión.

```
package Complices;
public class Profesor extends Persona {

    private String Especialidad;

    public String getEspecialidad() {
        return Especialidad;
    }
    public void setEspecialidad(String Especialidad) {
        this.Especialidad = Especialidad;
    }
}
```

Profesor.java

– Modificar Complices.java.

Modificar Complices.java como se muestra en el ejercicio. El cambio es para crear objetos de las clases Persona, Estudiante, EstudianteInternacional, y Profesor.

```
Package Complices;
public class Complices {

    public static void main(String[] args) {

        Persona Persona1 = new Persona();
        Persona1.setNombre("Antonio Perez");

        Estudiante Estudiante1 = new Estudiante();
        Estudiante1.setNombre("Benito");
        Estudiante1.setEscuela("Loyola High");

       EstudianteInternacional EstudianteInternacional1 =   new EstudianteInternacional();
        EstudianteInternacional1.setNombre("Juan Morales");
        EstudianteInternacional1.setEscuela("Lexington High");
        EstudianteInternacional1.setPais("Alemania");

        Profesor Profesor1 = new Profesor();
        Profesor1.setNombre("Francisco Sánchez");
        Profesor1.setEspecialidad("Física Nuclear");

        System.out.println("Mostrando datos de cada instancias.....");
        System.out.println("  Persona1.getNombre() = " + Persona1.getNombre());
        System.out.println("  Estudiante1.getNombre() = " + Estudiante1.getNombre());
        System.out.println("  EstudianteInternacional1.getNombre() = " +
              EstudianteInternacional1.getNombre());
        System.out.println("  Profesor1.getNombre() = " + Profesor1.getNombre());

    }
}
```

Complices.java

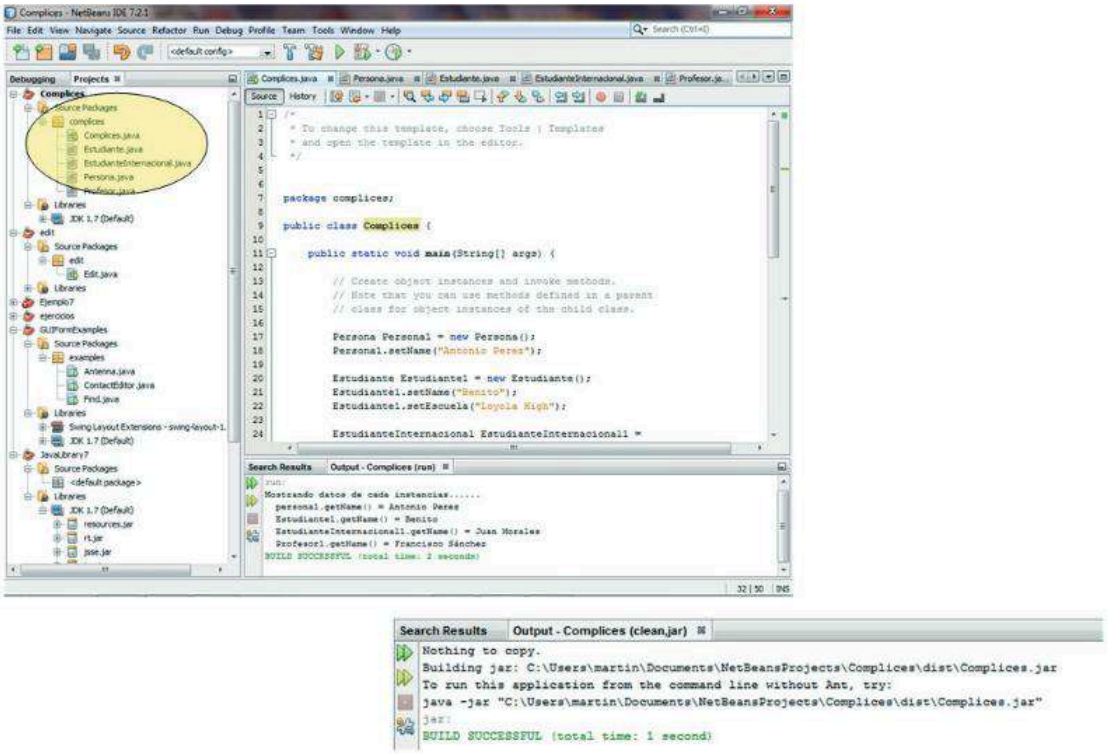
– Compilar y ejecutar el programa.

  · Hacer click con el botón derecho en el proyecto Complices y seleccionar Run.

  · Observar el resultado en la ventana Output de NetBeans.

```
Mostrando datos de cada instancias.....
  persona1.getNombre() = Antonio Perez
  Estudiante1.getNombre() = Benito
  EstudianteInternacional1.getNombre() = Juan Morales
  Profesor1.getNombre() = Francisco Sánchez
BUILD SUCCESSFUL (total time: 2 seconds)
```

*Resultado de la ejecución de la aplicación*




*Pantalla de NetBeans programando y complicando un programa con Herencia*

## 9.12. Construcción de clases con herencia múltiple

Los lenguajes como C++ soportan el concepto de herencia múltiple: en cualquier punto de la jerarquía, una clase puede heredar de una o más clases. El lenguaje Java soporta solo la herencia simple, que quiere decir que podrá usarse la palabra clave “extends” con una sola clase. Por ello, la jerarquía de clases para cualquier clase Java dada consistirá siempre en un línea recta hacia java.lang.Object.

Sin embargo, el lenguaje Java soporta la implementación de interfaces múltiples en una sola clase, lo que le da un método alternativo de tipos para la herencia simple. Una genial respuesta a las resoluciones de problemas con herencia múltiple.

Con este ejemplo explicaremos la implementación de unas herencia con interfaz en Java para crear la solución de lo que seria una herencia múltiple en lenguajes que si lo soportan como C++.

Construiremos una clase ArrayRatio que declare un atributo de tipo double[ ] y que implemente una interfaz llamada Estadisticas. El contenido de esta interfaz es el siguiente:

```
public interface Estadisticas {
double minimo();
double maximo();
double sumatorio();
}
```

Siguiente paso definir una clase final Rango que aumente la declaración de métodos estáticos de la clase Math y que implemente un interfaz llamado Extremos compilado con el siguiente código fuente:

```
public interface Extremos {
int min(int [ ] abcisa);
int max(int [ ] abcisa);
double min(double [ ] abcisa);
double max(double [ ] abcisa);
}
```

Diseñaremos además un interfaz Relaciones y con posterioridad una clase que la implemente que añada estos métodos:

```
// Retorna verdadero si abcisa es mayor que barras
boolean SiesMayor(Object barras) ;
// Retorna verdadero si abcisa es menor que barras
boolean SiesMenor(Object barras) ;
// Retorna verdadero si abcisa es igual que barras
boolean SiesIgual(Object barras) ;
```

### Los pasos que seguiremos en NetBeans

Clase ArrayRatio que declara un atributo de tipo double[ ] e implementa la interfaz Estadisticas.

```
/**
* Clase ArrayRatio
*
public class ArrayRatio implements Estadisticas {
double [ ] valor;
public void asignar() {
for (int i=0; i<valor.length; i++) {
valor[i] = Math.random();
}
}
```

```
public double minimo() {
double menor = valor[0];
for (int i=0; i<valor.length; i++) {
if (menor>valor[i]) { menor=valor[i]; }
}
return menor;
}
public double maximo() {
double mayor = valor[0];
for (int i=0; i<valor.length; i++) {
if (mayor<valor[i]) { mayor=valor[i]; }
}
return mayor;
}
public double sumatorio() {
double suma =0.0;
for (int i=0; i<valor.length; i++) {
suma+=valor[i];
}
return suma;
}
public void imprimir() {
for (int i=0; i<valor.length; i++) {
System.out.println("x["+i+"]="+valor[i]);
}
}
}
```

Ejemplo de uso de la clase ArrayRatio:

```
/**
* Programa PruebaArrayRatio
*/
public class PruebaArrayRatio {
public static void main (String [] args) {
ArrayRatio x = new ArrayRatio();
x.valor = new double[5];
x.asignar();
x.imprimir();
System.out.println("Minimo : " + x.minimo());
System.out.println("Maximo : " + x.maximo());
System.out.println("Sumatorio: " + x.sumatorio());
}
}
```

Ejemplo de salida por pantalla de la ejecución del código anterior:

```
$>java PruebaArrayRatio
x[0]=0.6257870038198547
x[1]=0.6362298582780569
x[2]=0.1032244210543346
x[3]=0.7885078556272651
x[4]=0.3466167914826589
Minimo : 0.1032244210543346
Maximo : 0.7885078556272651
Sumatorio: 2.5003659302621704
```

Al crear la Clase final Rango que ampliará las declaraciones de métodos estáticos de la clase y que implementa la interfaz llamada Extremos.

```
public final class Rango implements Extremos {
public int min(int [ ] abcisa) {
int menor = abcisa [0];
for (int i=1; i< abcisa.length; i++) {
if (menor> abcisa[i]) { menor= abcisa[i]; }
}
return menor;
}
public int max(int [ ]abcisa) {
int mayor = abcisa[0];
for (int i=1; i< abcisa.length; i++) {
if (mayor< abcisa[i]) { mayor= abcisa[i]; }
}
return mayor;
}
public double min(double [ ] abcisa) {
double menor = abcisa[0];
for (int i=1; i< abcisa.length; i++) {
if (menor> abcisa[i]) { menor= abcisa[i]; }
}
return menor;
}
public double max(double [ ] abcisa) {
double mayor = abcisa[0];
for (int i=1; i< abcisa.length; i++) {
if (mayor< abcisa[i]) { mayor= abcisa[i]; }
}
return mayor;
}
}
```

Ejemplo de uso de la clase Rango:

```
/**
* Programa PruebaRango
*/
public class PruebaRango {
public static void main(String [] args) {
// Para emplear los metodos de la clase Rango es necesario
// crear una instancia de la clase Rango
Rango aux = new Rango();
int [ ] n = new int[12];
for (int i=0; i<n.length; i++) {
n[i] = (int) (100*Rango.random()+1);
System.out.println("n["+i+"] = "+n[i]);
}
System.out.println("Minimo : " + aux.min(n));
System.out.println("Maximo : " + aux.max(n));
}
}
```

Ejemplo de salida por pantalla de la ejecución del código anterior:

```
$>java PruebaRango
n[0] = 49
n[1] = 27
n[2] = 67
n[3] = 101
```

```
n[4] = 98
n[5] = 102
n[6] = 58
n[7] = 3
n[8] = 99
n[9] = 38
n[10] = 74
n[11] = 88
Minimo : 3
Maximo : 102
```

## 9.13. Definición de clases abstractas

Se denominan clases abstractas las que tienen al menos un método abstracto sin implementar, sin código. Para ello:

– Define la estructura de una establecida abstracción, sin implementar totalmente cada método. Las clases y métodos abstractos se especifican con la palabra clave “abstract”.

– No pueden acarrear el modificador abstract:

  · Los constructores

  · Los métodos estáticos

  · Los métodos privados

```
public abstract class Figura {...}
```

No podemos crear objetos de una clase abstracta.

- Podrían existir referencias a clases abstractas.
- Aunque enfocarían a objetos de clases derivadas de la clase abstracta.

```
Figura fig = new Rectangulo(2,3);
```

– Sí podemos heredar de una clase abstracta.

– En una clase abstracta puede haber:

  - Metodos abstractos
  - Métodos no abstractos

  Las clases abstractas se emplean para definir clases con implementaciones parciales.

  Distintos métodos pueden no estar sin implementación pero sí declarados

– El objetivo de implementar parcialmente es dar una interfaz homogénea a todas las clases derivadas de una clase base abstracta

  Igualmente en los casos en los que la clase base no tiene la necesaria información como para poder implementar el método.

## Métodos abstractos

Métodos que se han declarado pero sin implementación en las clases abstractas:

```
abstract tipodeResultado nombre (listadodeParametros);
```

- Se definen con el término reservado abstract

  Las clases que heredan de la clase abstracta deben implementar los métodos abstractos de la superclase

- O serán abstractas igualmente como las anteriores

```
abstract class Plano {
double dimension1;
double dimension2;
Plano(double dimension1, double dimension2){
this.dimension1 = dimension1;
this.dimension2 = dimension2;
}
abstract double area();
}
class Rectangulo extends Plano {
Rectangulo(double dimension1, double dimension2){
super(dimension1,dimension2);
}
double area(){
// area de Rectangulo
return dimension1*dimension2;
}
}
```

## 9.14. Construcción de clases con herencia incluyendo polimorfismo

Un interfaz no es simplemente una manera más clara de mencionar a una clase abstracta, su mira es mucho más amplia que eso. Como un interfaz no tiene ninguna implementación, es decir, no hay ningun tipo de almacenamiento asociado al interfaz, no hay nada que impida la combinación de varios interfaces. Esto tiene mucha utilidad en programación ya que hay supuestos en que es necesario que un objeto X sea a1 y b1 y c1. En C++ es donde se denomina como herencia multiple y no es sencillo de realizar que cada clase pueda tener su propia implementación. En Java, se puede hacer, pero solamente una de las clases puede poseer implementación, con lo cual los problemas que aparecen en C++ no ocurren con Java en el momento de combinar varios interfaces.

En una clase derivada, el programador no está forzado a tener una clase base sea esta abstracta o concreta, es decir, una sin métodos abstractos. Si heredamos a partir de una clase no interfaz, solo se puede heredar de una. Los demás componentes base deben ser interfaces. Todos los nombres de interfaces se ponen después del término implements y separados por comas. Podemos tener todos los interfaces que queramos y cada uno será un tipo independiente. El siguiente ejercicio, enseña una clase en concreto que al combinar con varios interfaces para generar una nueva clase.

```
import java.util.*;

interface Encesta {
    void encestar();
    }
interface Corre {
    void correr();
    }
interface Pasa {
    void pasar();
    }
class Accion {
    public void encestar() {}
    }
```

```
class Heroe extends Accion implements Encesta,Corre,Pasa {
    public void correr() {}
    public void pasar() {}
    }

public class Puede {
    static void a( Encesta x ) {
        x.encestar();
        }
    static void b( Corre x ) {
        x.correr();
        }
    static void c( Pasa x ) {
        x.pasar();
        }
    static void d( Accion x ) {
        x.encestar();
        }

    public static void main( String args[] ) {
        Jugador i = new Jugador();
        a( i ); // Trata al Jugador como Encesta
        b( i ); // Trata al Jugador para Corre
        c( i ); // Trata al Jugador para Pasa
        d( i ); // Trata al Jugador como Accion
        }
    }
```

Se puede observar que Heroe combina la clase concreta Accion con los interfaces Encesta, Corre y Pasa. Cuando se combina la clase concreta con interfaces de este modo, la clase debe ir la primera y luego los interfaces; en caso contrario, el compilador dará un error.

La autentificación, signature, para encestar() es la misma en el interfaz Encesta y en la clase Accion, y encestar() no está proporcionado con la definicion de Jugador. Si se quiere crear un objeto del nuevo tipo, la clase debe tener todas las definiciones que necesita, y aunque Jugador no proporciona una definicion explicita para encestar(), la definición es proporcionada automáticamente por Accion y así es posible crear objetos de tipo Jugador.

En la clase Puede del ejercicio, hay cuatro métodos que toman como argumentos los distintos interfaces y la clase concreta. Cuando un objeto Jugador es creado, puede ser pasado a cualquiera de estos métodos, lo que quiere decir que se está creando un upcasting al interfaz relacionado. Debido a la manera en que han sido definidos los interfaces en Java, funcionan correctamente sin ninguna dificultad ni esfuerzo extra para del programador.

Los motivos más importantes par que existan interfaces en el ejemplo anterior es el poder realizar un upcasting a más de un tipo base. Pero, hay una segunda razón que es la misma por la que se usan clases abstractas, impedir que el programador tenga que crear un objeto de esa clase y establecerle además que solo es un interfaz. Y esto plantea la cuestión de qué se debe utilizar pues, un interfaz o una clase abastacta.

Un interfaz aporta las bondades de una clase abstracta y, además, los beneficios propios del interfaz, es posible crear una clase base sin la declaración de ningún método o variable miembro, por lo que se deberían utilizar mejor los interfaces que las clases abstractas. De hecho, si queremos tener una clase base, primeramente elegiremos ser un interfaz, y usar clases abstractas únicamente si es necesario tener alguna variable miembro o algun método implementado.

## 9.15. Empleo de excepciones

El uso de excepciones en Java es un mecanismo del lenguaje que posibilita gestionar errores y escenarios excepcionales. Como el tratamiento de excepciones es un pilar importante del lenguaje, los programadores saben como lanzarlas y capturarlas, aunque como todos los aspectos fundamentales en programación es vital conocer el propósito por el cual tenemos a disposición este mecanismo, además de hacer un buen uso de esta funcionalidad del lenguaje Java.

## Excepciones

Una excepción en Java es un error o situación de excepción que se origina durante la ejecución de un programa. Varios ejemplos de errores y situaciones excepcionales son:

- Leer un fichero que no existe
- Acceder al valor N de una relación que posee menos de N elementos
- Enviar/recibir datos por la red en el momento se pierde de conexión.
- Intentar imprimir y desconectarse la misma o quedarse sin papel.
- Guardar un fichero y dar un error a la mitad.

Estas excepciones son mero ejemplo de la cantidad enorme de suspuestos, que hay que controlar y gestionar en programación. Por suerte la gestión en Java de las mismas son de gran ayuda y nos permite un control muy automatizado.

### Solución habitual

- La solución habitual consiste en que los métodos devuelvan un valor en el que indique si en su ejecución se ha producido algún tipo de incidencia que pudiera acarrear errores en el futuro.
- Por ejemplo la función “fopen” de C++ se usa para abrir un fichero en el disco. Si este fichero no puede abrirse la función retorna el valor null.

### Desventajas de la solución habitual

- El encargado de invocar al método debe recordar de recoger el valor devuelto.
- El código fuente muchas veces puede llegar convertirse en una sucesión de comprobaciones de posibles situaciones erróneas.
- El encargado de invocar al método puede desvocarse por el error y necesite pasárselo a métodos de niveles superiores.

## Ventajas de las excepciones

– Si no se quiere las excepciones no pueden ser obviadas.

– El lenguaje provee de construcciones del tipo try–catch–finally que facilitan la escritura del código en presencia de excepciones y evita tener que incluir sentencias condicionales cada vez que se llama a un método que puede generar una excepción.

– Los métodos que se desborden por una excepción pueden pasarla a niveles superiores con el mandato throws.

Ejemplo Piscina:

```
class PiscinaOlimpica
{
private int nivel;
public final int NIVEL_MAX_AGUA;
public PiscinaOlimpica(int max)
{
if (max<0) max=0;
NIVEL_MAX_AGUA =max;
}
public int getNivel()
{ return nivel; }
public void vaciar(int cantidad)
{ nivel=nivel-cantidad; }
public void llenar(int cantidad)
{ nivel=nivel+cantidad; }
}
```

Ejemplo Piscina con excepciones:

```
class PiscinaOlimpica
{
private int nivel;
public final int NIVEL_MAX_AGUA;
public PiscinaOlimpica(int max)
{
if (max<0) max=0;
NIVEL_MAX_AGUA =max;
}
public int getNivel()
{ return nivel; }
public void vaciar(int cantidad) throws Exception
{
if (nivel-cantidad < 0)
throw new Exception();
else nivel=nivel-cantidad;
}
public void llenar(int cantidad) throws Exception
{
if (nivel+cantidad > NIVEL_MAX_AGUA)
throw new Exception();
else nivel=nivel+cantidad;
}
}
```

## Características generales

Las clases predeterminadas para Excepciones se incluyen en el paquete java.lang

– Throwable

  · Muestra la funcionalidad básica de aquello que pude ser lanzado en un programa como excepción.

  · Dos constructores: uno sin parámetros y otro incluiremos una cadena de texto que describa el error producido.

– Error

  Clase destinada a especificar problemas graves o condiciones anómalas que no tendrían que suceder normalmente, por lo que no es fundamental que los programas se ocupen de gestionarlos.

– Exception

  · Es la clase de las excepciones que puede arrojar un programa, por lo que es la clase que más importa al desarrollador.

  · Tanto Exception como sus subclases sin meternos con "RuntimeException" son excepciones chequeadas, en el supuesto de que el compilador compruebe que se lanza una excepción en un método, esta debe ser capturada por el propio método o incluida en su condición throws.

– RuntimeException

  · Las RuntimeException pueden suceder en cualquier lugar del programa, normalmente, de manera muy numerosa. Por esa cuestión el coste de chequear obligatoriamente si ha ocurrido una RuntimeException es mayor, que el propio beneficio que se produce por las capturas obligatorias.

  · De esta forma Java soporta que la captura de estas excepciones sea totalmente opcional.

## Excepciones personalizadas

```
class Piscina_Chequeo extends Exception
{
int nivel;
public Piscina_Chequeo (String descripcion, int valor)
{
super (descripcion);
nivel=valor;
}
}
class PiscinaOlimpica
{
private int nivel;
public final int NIVEL_MAX_AGUA;
public PiscinaOlimpica(int max)
{
if (max<0) max=0;
NIVEL_MAX_AGUA =max;
}
public int getNivel()
{ return nivel; }
public void vaciar(int cantidad) throws Piscina_Chequeo
{
if (nivel-cantidad < 0)
throw new Piscina_Chequeo (“Alerta. Vaciado excesivo”, nivel-cantidad);
else nivel=nivel-cantidad;
```

```
}
public void llenar(int cantidad) throws Piscina_Chequeo
{
if (nivel+cantidad > NIVEL_MAX_AGUA)
throw new Piscina_Chequeo (“Alerta. Llenado excesivo”,
nivel+cantidad);
else nivel=nivel+cantidad;
}
}
```

Captura de excepciones con try–catch:

```
class PiscinaFigueroa
{
public static void mantenimPiscina(PiscinaOlimpica P)
{
try
{
for (int i=1;i<=3;i++)
{
P.llenar((int)(Math.random()*100));
System.out.println (“llenado...” + P.getNivel());
P.vaciar((int)(Math.random()*100));
System.out.println (“vaciado...” + P.getNivel());
}
```

```
}
catch (Piscina_Chequeo e)
{
System.out.println(e.toString() + ‘ ‘ + e.nivel);
}
System.out.println(“El nivel de la piscina es…” + P.getNivel());
}
public static void main (String [ ] args)
{
PiscinaOlimpica P = new PiscinaOlimpica(100);
MantenimPiscina(P);
}
}
```

*En este ejercicio se aplica una gestión de excepciones con Try–catch*

Múltiples sentencias catch:

```
try
{ … }
catch (Excepcion_1 identificador_1)
{ … }
catch (Excepcion_2 identificador_2)
{ … }
finally
{ … }
```

throws como alternativa al try–catch:

```
class PiscinaFigueroa
{
public static void mantenimPiscina(Piscina P) throws Piscina_Chequeo
{ for (int i=1;i<=3;i++)
{
P.llenar((int)(Math.random()*100));
System.out.println ("llenado..." + P.getNivel());
P.vaciar((int)(Math.random()*100));
System.out.println ("vaciado..." + P.getNivel());
}
System.out.println("El nivel de la piscina es..." + P.getNivel());
}
public static void main (String [ ] args)
{ PiscinaOlimpica P = new PiscinaOlimpica(100);
try
{ mantenimPiscina(P); }
catch (Piscina_Chequeo e)
{ System.out.println(e.toString() + ' ' + e.nivel); }
}
}
```

*En este ejercicio se sustituye sentencias throws como alternativa a Try–catch*

## Aserciones

Son una forma de comprobar el código fuente para verificar precondiciones, no variantes de bucle, estructuras posteriores a condiciones, etc.

– Formato

  · assert expresión_booleana;

  · assert expresión_booleana : expresión_2;

– Funcionamiento

  · La condición booleana se evalúa y en supuesto de no ser verdadera se lanza la excepción AssertException.

  · expresión_2 es una expresión que retorna un valor y no puede ser void. Este dato es convertido a cadena String y enviado al constructor de AssertException para comprobar **más en** profundidad la aserción.

Ventajas

– Introducir aserciones en nuestros programas facilita la depuración de errores.

– Además las aserciones pueden quitarse una vez finalizado el periodo de pruebas, y no afectar al rendimiento del sistema.

– Recomendación

  En las precondiciones de métodos se recomienda no usar aserciones, en su lugar excepciones NullPointerException, IllegalArgumentException porque dan **más información que un** genérico AssertException.

– Utilización

  · "assert" es utilizada desde la versión 1.4 de Java por lo que al compilar con el JDK 1.4 es necesario hacer lo siguiente:

    javac –source 1.4 fichero.java (con el JDK 1.5 JDK y 1.6 no es necesario).

  · Al interpretar código Java con aserciones hay dos posibilidades.

    › java –ea fichero. Con parámetro (–ea para enable assertions) activa las aserciones.

    › java –da fichero. Con parámetro (–da para disable assertions) desactiva las aserciones (por defecto).

Ejemplo:

```
switch(palo)
{
case Palo.rombos:
…
break;
case Palo.picas:
…
break;
case Palo.treboles:
…
break;
case Palo.corazones:
…
break;
default:
assert false : palo;
}
```

## 9.16. Gestión de eventos

Los eventos aprueban al programa interactuar con el usuario. Además que se utiliza el ratón o el teclado, se crea un evento que representa de una manera exacta lo que ha ocurrido.

En Java distinguimos dos clases de eventos:

– Eventos generales o simples: son eventos producidos por una acción general. Por ejemplo, pulsar una tecla, mover el ratón…etc. En la tabla siguiente se muestran los eventos estándar para el teclado y el ratón:

| Eventos estándar para teclado y ratón | | |
|---|---|---|
| ID del evento | Método | Descripción |
| KEY_PRESS | keyDown(Event,int) | Pulsar una tecla |
| KEY_RELEASE | keyUp(Event,int) | Soltar una tecla |
| KEY_ACTION | keyDown(Event,int) | Pulsar una tecla de función |
| KEY_ACTION_RELEASE | keyUp(Event,int) | Soltar una tecla de función |
| MOUSE_DOWN | mouseDown(Event,int,int) | Pulsar el ratón |
| MOUSE_UP | mouseUp(Event,int,int) | Movimiento hacia arriba del ratón |
| MOUSE_MOVE | mouseMove(Event,int,int) | Movimiento del ratón |
| MOUSE_ENTER | mouseEnter(Event,int,int) | Movimiento de entrada del ratón |
| MOUSE_EXIT | mouseExit(Event,int,int) | Movimiento de salida del ratón |
| MOUSE_DRAG | mouseDrag(Event,int,int) | Arrastrar el ratón |

– Eventos específicos: son los que más se utilizan con más frecuencia en AWT. Partimos de la premisa de un caso en el que tengamos varios botones. Si cada botón, al ser pulsado, no generase un evento único ¿cómo sabriamos que botón se ha pulsado?. Por este motivo, existen los llamados eventos específicos, en el modo de que poseen más información que los eventos generales sobre el elemento que ha disparado el evento.

Los eventos generados por componentes se muestran en la siguiente tabla:

| Eventos generados por componentes | | |
|---|---|---|
| Componente | ID del evento | Descripción |
| Button | ACTION_EVENT | El usuario ha hecho click. |
| Check Box | ACTION_EVENT | El usuario ha pulsado una casilla de activación. |
| Choice | ACTION_EVENT | El usuario ha seleccionado un elemento. |
| Component | GOT_FOCUS | Obtiene el foco de entrada. |
| Component | KEY_ACTION | El usuario ha pulsado una tecla de función. |
| Component | KEY_ACTION_RELEASE | El usuario ha soltado una tecla de función. |
| Component | KEY_PRESS | El usuario ha pulsado una tecla. |

| Eventos generados por componentes | | |
|---|---|---|
| Componente | ID del evento | Descripción |
| Component | KEY_RELEASE | El usuario ha soltado una tecla. |
| Component | LOST_FOCUS | Se ha perdido el foco de entrada. |
| Component | MOUSE_ENTER | El ratón ha entrado en un componente. |
| Component | MOUSE_EXIT | El ratón ha salido de un componente. |
| Component | MOUSE_DOWN | Se ha pulsado un botón del ratón. |
| Component | MOUSE_UP | Se ha soltado un botón del ratón. |
| Component | MOUSE_MOVE | El usuario ha movido el ratón. |
| Component | MOUSE_DRAG | Se arrastró el ratón. |
| List | ACTION_EVENT | El usuario ha hecho doble click sobre una lista de elementos. |
| List | LIST_SELECT | El usuario ha seleccionado un elemento de la lista. |
| List | LIST_DESELECT | El usuario ha anulado la selección de un elemento de la lista. |
| Menu Item | ACTION_EVENT | Se ha seleccionado un elemento del menú. |
| ScrollBar | SCROLL_LINE_UP | El usuario ha solicitado un desplazamiento ascendente de una línea. |
| ScrollBar | SCROLL_LINE_DOWN | El usuario ha solicitado un desplazamiento descendente de una línea. |
| ScrollBar | SCROLL_PAGE_UP | El usuario ha solicitado un desplazamiento ascendente de una página. |
| ScrollBar | SCROLL_PAGE_ DOWN | El usuario ha solicitado un desplazamiento descendente de una página. |
| ScrollBar | SCROLL_ABSOLUTE | El usuario ha solicitado un cambio absoluto. |
| TestField | ACTION_EVENT | Se ha pulsado la tecla “Return”. |
| Window | WINDOW_DESTROY | La ventana se ha destruido. |
| Window | WINDOW_ICONIFY | La ventana se ha transformado en icono. |
| Window | WINDOW_DEICONIFY | El icono se ha transformado en ventana. |
| Window | WINDOW_MOVED | Se ha movido la ventana. |

Todos estos tipos de eventos son objetos instanciados con la clase java.awt.Event.

### 9.16.1. Eventos, fuentes y auditores de eventos

Si queremos manejar eventos en Java, a los desarrolladores se le ofrece 2 elecciones:

#### Desarrollar un manejador de eventos

En el momento que se genera un evento UI, el mecanismo de manejo de eventos de AWT lo envia al nivel justamente anterior en la jerarquía de componentes. La manera de gestionar estos eventos es implementarlo como un método handleEvent().

```
public boolean handleEvent (Event e) {
  switch (e,id) {
    case Event.ACTION_EVENT: if (e.target instanceof Button) {
                              .....  // este case se inserta un evento de botón
                             }
                             return true;
                            if (e.target instanceof MenuItem) {
                              .....  // en la condicional hemos metido es un evento de
menú
                             }
                             return true;
   case Event.MOUSE_MOVE:  // este case insertamos un evento de movimiento de ratón
                        ......
       return true;
       .....
       .....
    default: super.handleEvent(e);  // el evento no se maneja aquí, de
            break;                  // manera que la responsabilidad se le
   }                                // pasa a la superclase
}
```

Enviar los eventos a un método action().

Dicho método action() tiene el siguiente formato:

```
public boolean action (Event e,Object obj) {
    .....
    .....
}
```

Esto significa que los eventos UI se envian al método action(), unido a un parámetro Object adicional que conlleva información sobre el evento. Esta información tiene dependencia del elemento UI que genere el evento.

Aunque no todos los componentes UI generarán eventos action, por lo que si necesitamos atrapar eventos que no esten en la tabla anterior deberemos construir un método handleEvent(), de propósito general.

Sin embargo, para los componentes que sí llaman un método action(), es recomendable declarar dicho método, de modo que al consultar el parámetro Object podemos encontrar información concreta acerca del evento.

Ejemplo:

```
public boolean action (Event e,Object obj) {
  if (e.target instanceof Button) {   // la condición es para un evento
button
      .....
  }
  if (e.target instanceof MenuItem) {  // la condición es para un evento de
menú
      .....
  }
  return true;
}
```

Las variables de instancia que poseen los objetos Event contenidos en java.awt.Event son:

- ID: identificador del evento.
- target: es el objetivo de un evento, es decir, de donde proviene el evento. Por ejemplo, de un cuadro de texto, botón, de una barra de desplazamiento…, etc.
- when: instante en el que se generó el evento.
- x,y: coordenadas en las que se generó el evento.
- key: tecla pulsada. Es muy útil para eventos de teclado.
- arg: argumento de tipo Object, como argumento asociado a un evento. Por ejemplo, el nombre del botón seleccionado.
- modifiers: estado de las teclas modificadoras cuando se genera un evento.

### 9.16.2. Tipos de eventos. Mecanismos de gestión de eventos

A partir de la versión 1.1 de Java, se diseño un nuevo sistema de gestión de eventos. El motivo por el cual apareció este nuevo sistema es que cuando teníamos una interfaz de usuario muy amplia y queriamos controlar los eventos, había que sacar comparativas de los nombres de los distintos elementos para así distinguirlos bien. Estas comparaciones se realizaban por medio de los nombres de los componentes, que son Strings , lo que disminuía considerablemente el rendimiento de la aplicación.

En la versión 1.1 se implementará una clase base denominada EventObject y a partir de ella se asignarán una serie de clases como:

- java.awt.AWTEvent
- java.awt.event.ActionEvent
- java.awt.event.AdjustmentEvent
- java.awt.event.ComponentEvent

– java.awt.event.ContainerEvent

– java.awt.event.FocusEvent

– java.awt.event.InputEvent

– java.awt.event.PaintEvent

– java.awt.event.WindowEvent

– java.awt.event.ItemEvent

– java.awt.event.TextEvent

Existen dos niveles de eventos: eventos de bajo nivel y eventos de alto nivel.

### Eventos de bajo nivel

Estos eventos se producen cuando el usuario pulse alguna tecla o efectúe algún movimiento con el ratón, o también cuando se modifique o visualice algún componente en la pantalla.

Las clases asociadas son:

– ComponentEvent: se generará cuando el usuario mueva o redimensione un elemento.

– FocusEvent: se va a producir cuando el usuario modifica el foco de un elemento.

– KeyEvent: cuando el usuario pulsa una tecla.

– MouseEvent: se produce cuando el usuario realiza un movimiento o haga click con el ratón.

– ContainerEvent: cuando se añaden o borren elementos en el contenedor.

– WindowEvent: se produce cuando se haga algún tipo de operación con la ventana.

A continuación vamos a ver los métodos para KeyEvent y MouseEvent:

| Métodos de KeyEvent | |
|---|---|
| getKeyChar () | Retorna el carácter asociado con la tecla que produjo el evento |
| getKeyCode() | Retorna el código de la tecla que produjo el evento |
| getKeyModifiers() | Retorna una cadena que indica el modificador de la tecla, por ejemplo “Shift” |
| getKeyText(int) | Retorna una cadena que indica el tipo de tecla pulsada, por ejemplo F1, indicando que se trata de una tecla de función |
| isActionKey() | Retorna se la tecla es una tecla de “acción” |

| Métodos de MouseEvent | |
|---|---|
| getClickCount() | Retorna el número de clics de ratón asociados con el evento |
| getX() | Retorna la posición x cuando se genera el evento |
| getY() | Retorna la posición y cuando se genera el evento |
| isPopupTrigger() | Retorna si es o no un evento de tratón del tipo pop–up menú para esta plataforma |
| translatePoint(int posx,int posy) | Mueve la posición de las coordenadas del evento a posx y posy |

### Eventos de alto nivel

Son aquellos que tienen que ver con la semántica de los componentes. Así por ejemplo, se generarán cuando se pulse un botón o cuando se se cambie el texto de un campo de texto.

Las clases asociadas a este nivel son:

– ActionEvent: Cuando se realiza alguna acción sobre el elemento, como puede ser la pulsación de un botón.

– AdjustmentEvent: se encaja algún valor de una elemento.

– ItemEvent: se ha modificado el estado de algún elemento que pertenece al componente, como puede ser un elemento de una lista desplazable

– TextEvent: el contenido de texto de algún componente ha cambiado.

## 9.16.3. Librerías de clases asociadas

### Las clases de escucha

Para poder capturar tanto los eventos de bajo nivel, como los de alto, Java proporciona las clases de escucha Listeners que son son interfaces. En cada tipo de evento existirá una clase de escucha. Cada clase de escucha posee la declaración de un conjunto de métodos, a los que se llamarán dependiendo del evento producido.

En la siguiente tabla se mostrarán las clases de escucha con sus métodos abstractos:

| Clases | Métodos | Descripción |
|---|---|---|
| **ActionListener** | actionPerformed(ActionEvent e) | Ejecuta algún mandato |
| **AdjustmentListener** | adjustamentValueChanged(AdjustamentEvent e) | Encaja algún valor |
| **ComponentListener** | componentHidden(ComponentEvent e) | El elemento se oculta |
| | componentMoved(ComponentEvent e) | El elemento se mueve |
| | componentResized(ComponentEvent e) | El elemento se redimensiona |
| | componentShown(ComponentEvent e) | El elemento se visualiza |
| **ContainerListener** | componentAdded(ContainerEvent e) | Se añade un elemento el contenedor |
| | componentRemoved(ContainerEvent e) | Se elimina un elemento del contenedor |
| **FocusListener** | focusGained(FocusEvent e) | El elemento obtiene el foco |
| | focusLost(FocusEvent e) | El elemento pierde el foco |
| **ItemListener** | ItemStateChanged(ItemEvent e) | Se modifica el estado de algún elemento del componente, como puede ser la elección de alguna casilla de verificación |
| **KeyListener** | keyPressed(KeyEvent e) | Se ha presionado una tecla |
| | keyReleased(KeyEvent e) | Se ha soltado la tecla |

| Clases | Métodos | Descripción |
|---|---|---|
| | keyTyped(KeyEvent e) | Se ha tecleado un carácter |
| **MouseListener** | mouseClicked(MouseEvent e) | Se ha pulsado el botón del ratón |
| | mouseEntered(MouseEvent e) | El puntero del ratón ha entrado en el elemento |
| | mouseExited(MouseEvent e) | El puntero del ratón ha salido del elemento |
| | mousePressed(MouseEvent e) | Se ha presionado un botón del ratón |
| | mouseReleased(MouseEvent e) | Se ha soltado un botón del ratón |
| **MouseMotionListener** | mouseDragged(MouseEvent e) | Se está desplazando el ratón con el botón pulsado |
| | mouseMoved(MouseEvent e) | El puntero del ratón ha cambiado de posición |
| **TextListener** | textValueChanged(textEvent e) | El contenido del texto del elemento ha cambiado |
| **WindowListener** | windowActivated(WindowEvent e) | La ventana ha sido activada |
| | windowClosed(WindowEvent e) | Se ha cerrado la ventana |
| | windowClosing(WindowEvent e) | Se ha solicitado cerrar la ventana |
| | windowDeactivated(WindowEvent e) | La ventana ha sido desactivada |
| | windowDeiconified(WindowEvent e) | Cuando se restaura la ventana a su tamaño original o se maximiza |
| | windowIconofied(WindowEvent e) | Se ha minimizado la ventana |
| | windowOpened(WindowEvent e) | Se ha abierto la ventana |

Tras haber creado los elementos, debemos indicar qué clase de escucha implementaremos a cada componente.

Deberemos utilizar la siguiente línea de código:

```
nombreComponente.addClaseEscucha( );
```

Donde ClaseEscucha es el nombre de la clase de escucha según los nombres especificados en la tabla anterior.

Además tendremos que indicar que la clase en la que se situan los componentes implementa la interfaz siguiente:

```
class Prueba1 implements ClaseEscucha {
        ...
}
```

Un gran inconveniente que tienen las interfaces es que hay que definir todos sus métodos abstractos en las clases que las implementan. Así, si una de nuestras clases implementa la interfaz WindowListener, tendrá que implementar todos los métodos asociados, aún cuando sólo utilicemos uno de ellos. El resto de los métodos tendrán una implementación vacía. Por esta razón surgen las clases adaptadoras. Estas clases adaptadoras se encargan de implementar todos los métodos del la clase de escucha. Por ello necesitamos definir los métodos que vayan a servirnos para gestionar eventos , sin que nos tenga que preocupar el resto.

Indicaremos como en la siguiente linea, que nuestra clase es una subclase del adaptador:

```
class Prueba2 extends Adaptador{
    ...
}
```

Sólo las clases que tienen más de un método tienen adaptador, y son los siguientes:

| |
|---|
| ComponentListener posee ComponentAdapter |
| ContainerListener posee ContainerAdapter |
| FocusListener posee FocusAdapter |
| KeyListener posee KeyAdapter |
| MouseListener posee MouseAdapter |
| MouseMotionListener posee MouseMotionAdapter |
| WindowListener posee WindowAdapter |

Es necesario usar adaptadores de cara a la vida futura del programa, ya que si algún día Sun empresa creadora de Java ú Oracle su nueva propietaria, decide añadir un nuevo evento a una clase de escucha y nosotros tenemos implementado en el proyecto la interfaz de escucha, deberemos implementar el nuevo método, ya que si no el compilador nos dará un error. Sin embargo si nuestra aplicación utiliza adptadores, nosotros podrmos olvidar el nuevo evento ya que Oracle será la encargada de facilitar la nueva sobrecarga vacía para el método asociado al nuevo evento.

## 9.17. Empleo de hilos

Una de las mayores bondades del entorno de ejecución Java es que posibilita manejar programas que tiene múltiples procesos denominados “Threads”. La ventaja es construir aplicaciones que realicen a la vez diferentes tareas y que se están rodando varias cosas diferentes en memoria principal. Por ejemplo, una aplicación multithreaded que gestiona unas oficina será capaz de recibir datos por la red, realizar cálculos y a la vez, aceptar la entrada del usuario. Aunque en realidad sólo se ejecutará un thread a la vez, el sistema operativo que ejecuta el programa va ejecutando durante cierto tiempo un thread y luego saltará al siguiente thread, pero al usuario le dará la impresión de que se están ejecutando todos a la vez.Para comprender cómo se codifican los threads, vamos a ver un ejemplo que contiene un solo proceso. En este ejemplo, el código se ejecuta de forma lineal hasta que se llega a la última línea, entonces el proceso termina.

```
public class Saludos {
  // métodos
  public void saluda (String quien) {
   System.out.println (quien + " ha saludado al entrar");
  }
    // main
  public static void main (String args[ ]) {

   Saludos invitado = new Saludos();
   invitado.saluda ("comensal1");
   invitado.saluda ("comensal2");
   invitado.saluda ("comensal3");
   System.out.println ("Todo el mundo ha entrado y saludado");
 }
}
```

La ejecución comienza y se crea una instancia de la clase Saludos. Luego se llama tres veces al método saluda(). Cada llamada se ejecuta y luego se devuelve el control al programa principal. La última sentencia del main() escribe "Todo el mundo ha entrado y saludado" y entonces el programa termina. Al incluir threads en este bloque de código, se romperá la linealidad de la ejecución. El programa se dividirá en fragmentos y cada uno de ellos se ocupará de escribir el mensaje en pantalla.

```
public class Saludos implements  Runnable {
  // metodos
  public void run () {
   System.out.println (Thread.currentThread().getNombre() + " ha
saludado al entrar");
  }
  // main
  public static void main (String args[ ]) throws InterruptedException {
   int ind = 0;
   Saludos invitado = new Saludos();
```

```
    // Se crea el primer thread
    Thread unThread = new Thread (invitado, “comensal1”);
    // Se crea el segundo thread
    Thread otroThread = new Thread (invitado, “comensal2”);
    unThread.start(); // se inicia el primer thread
    otroThread.start(); // se inicia el segundo thread
    // Cuerpo del programa principal
    while (unThread.isAlive() || otroThread.isAlive()) ind++;
    // Se ejecuta después de que los threads hayan terminado
    System.out.println (“El valor de ind es “ + ind );
    System.out.println (“Todo el mundo ha entrado y saludado”);
   }
}
```

En este momento el programa inicia dos threads que se ejecutan concurrentemente a la vez que el programa principal. Trás construir los threads, se invoca al método start() de cada uno, informando al intérprete de Java que inicie ese thread. El método main() responde de resetear cada thread y de ordenar en que momento debe concluir. El proceso es necesario porque el programa tiene que saber cuando es seguro arrancar la línea de código:

```
System.out.println (“El valor de ind es “ + ind );
```

De otro modo, el programa podría finalizar antes de que los threads hayan termiando, dejándolo bloqueado. Para gestionar esta incidencia, se ha colcoado un bucle que incrementa el valor de una variable mientras los threads están ejecutándose.

```
while (unThread.isAlive() || otroThread.isAlive()) ind++;
```

Al compilar y ejecutar el bloque de código, se puede apreciar que el valor de la variable ind es distinto en cada ejecución del mismo. Esta variable guarda el número de veces que se ejecuta el bucle dede el momento que se ejecutan los threads. El hecho de que este valor cambie, muestra como Java ejecuta los programas que se dividen en procesos distintos.

### 9.17.1. Fundamentos

Lo más importante de la programación con "threads" es saber cuándo se necesita usarlas. Los threads pueden ayudar mucho a la programación y que se ejecute más rápido aunque puede recargar y frenar la ejecución del mismo. Únicamente debe usarse cuando se requiera que dos o más procesos se ejecuten a la vez. Por ejemplo, en un entorno de ventanas, puede hacerse que se abran varias ventanas a la vez en el supuesto de que varias operaciones están ocurriendo al mismo tiempo y necesiten mostrarse.

La idea de que cuando una aplicación se está ejecutando, los distintos procesos no se ejecutan realmente al mismo tiempo. Es el sistema operativo quien se debe ocupar de gestionar los ciclos de procesador para que existea multitarea real y la máquina virtual Java también maneja la gestión del procesador para controlar qué ejecuciones deben ocurrir y en qué orden.

Metodo "main" para ejecutar los Threads de la clase proceso:

```
public static void main(String[ ] args) {
        // Método Main para procesar los hilos de ejecución

        Procedimiento hilo1 = new Procedimiento("Hilo 1");
        Procedimiento hilo2 = new Procedimiento("Hilo 2");
        hilo1.setMensaje("Este es el mensaje del hilo 1");
        hilo2.setMensaje("Mensaje hilo 2");

        hilo1.start();
        hilo2.start();

    }
```

Ejemplo con clase Procedimiento:

```
public class Procedimiento extends Thread{

    String mensaje;
    public Procedimiento(String msg)
    {
        super(msg);
    }

    public void run()
    {
        for(int i =0;i<20;i++)
        {
            System.out.println(mensaje);
        }
       System.out.println("Este procedimiento ha finalizado:"+this.getNombre());
    }

    public void setMensaje(String msj)
    {
        this.mensaje = msj;
    }

}
```

### 9.17.2. Creación

Antes de crear un thread, se deberá inicializar una clase que sea la autorizada de manejar el thread. Esto puede llevarse a cabo de dos maneras: derivando una clase, haciendo una subclase de la clase Thread o implementando un interface.

#### Heredando de thread

La forma más obvia de crear un thread es haciendo una subclase de la clase Thread que proporciona la librería Java. Esta cercanía permitirá sobreescribir los métodos de dicha clase para hacer las funciones que sean necesarias en el programa. Se muestra la sintaxis para crear una clase derivando de la clase Thread:

```
[visibilidad] [modificador] class NombredeClase extends Thread
   // declaración de variables y métodos
   …
}
```

Veámoslo utilizando este ejemplo:

```
public class Compra extends Thread {
  // métodos
  public void run () { System.out.println (“ha comprado “);}

  // main
  public static void main (String args[ ]) {

   int idt = 0;
```

```
    Compra vendedor = new Compra();

    // Se crea el primer thread
    Thread unThread = new Thread (vendedor);

    // Se crea el segundo thread
    Thread otroThread = new Thread (vendedor);

    unThread.start(); // se inicia el primer thread
    otroThread.start(); // se inicia el segundo thread

    // Bucle while principal
    while (unThread.isAlive() || otroThread.isAlive())
     idt++;

    // Se ejecuta después de que los threads hayan terminado
    System.out.println ("Todos los clientes han comprado");

   }
}
```

La clase Compra viene de la clase Thread y sobreescribe el método run() determinado en esta clase al heredar Compra de Thread, tiene todos los métodos de Thread como start() y run(). En el ejemplo se mantienen las versiones originales de los métodos de la clase Thread excepto en el caso del método run(), que ha sido redefinido. Este método es el que le dice al thread qué operaciones tiene que ejecutar una vez que éste ha sido iniciado, por eso habitualmente este método debe ser sobrecargado.

## Implementando Runnable

Cuando se diseña la jerarquía de clases para la aplicación, hay veces que es dificil descender de la clase Thread porque las clases ya desciendan de otras, por ejemplo. En estos casos, se puede implementar el interface Runnable para resolver el problema. La sintaxis es:

```
[visibilidad] [modificador] class NombredeClase [extends SuperClase]
implements Runnable
   // declaración de variables y métodos
   …
}
```

La bondad de esta forma es que se puede construir una nueva clase que desciende de otra clase y además usar los métodos definidos por el interface Runnable.

```
public class Saludos implements Runnable {
  public void run () {
   System.out.println (Thread.currentThread().getNombre() + " te ha
saludado");
  }

  public static void main (String args[ ]) throws InterruptedException {
   int idg = 0;
   Saludos invitado = new Saludos();
   // Se crea el primer thread
   Thread unThread = new Thread (invitado, "comensal1");
```

```
    // Se crea el segundo thread
    Thread otroThread = new Thread (invitado, “comensal2”);
    unThread.start(); // se inicia el primer thread
    otroThread.start(); // se inicia el segundo thread
    // Cuerpo del programa principal
    while (unThread.isAlive() || otroThread.isAlive()) idg++;
    // Se ejecuta después de que los threads hayan terminado
    System.out.println (“El valor de idg es “ + idg );
    System.out.println (“Todos los invitados te han saludado”);
   }
}
```

### 9.17.3. Prioridad

#### Prioridades

Cuando se procesa un thread, éste se introduce automáticamente en una cola, en la que debe esperarse hasta que le llegue el turno de ejecutarse. Este proceso de espera se llama bloqueo.

En la cercania FIFO (primero en entrar primero en salir), el thread que está el primero de la cola espera hasta que finalice el que está rodando y entonces empezará su ejecución.

En la diseño de las prioridades, si un thread tiene una prioridad mayor que otro le permutará su lugar dentro de la cola. Este proceso continua hasta que se localice un thread con una prioridad mayor o igual. De igual manera que los threads que están al final de la cola no se ejecutan hasta que los de alta prioridad finalicen. El supuesto más común es el caso del thread “Garbage Collector” de Recolección de Basura, que tiene la prioridad más baja.

Para gestionar la prioridad de un thread, la clase Thread aporta las variables: max_priority, norm_priority y min_priority.

Para establecer y captar la prioridad de un thread se usan los métodos setPriority() y getPriority().

El planificador es la porción de la máquina virtual que se autoriza para resolver qué hebra ejecuta en cada momento. La especificación de la máquina virtual no fuerza al uso de ningun algoritmo particular en la planificación de hebras. Por ello, es problemático de cada implementación en concreto decidir si implementan por sí misma el planificador, o descansar en el sistema operativo subyacente. De todas maneras sí se fuerzan, ciertas peculiaridades que debe tener el planificador. En concreto, todos los hilos tienen una prioridad, de modo que el planificador dará más ventaja a los hilos con mayor prioridad que a las de menor. Sin embargo, es obligatorio que exista expropiación entre hilos de misma prioridad, de modo que si hay varios hilos con igual prioridad todos rueden en algún momento. La especificación de la máquina virtual, no obliga a la expropiación de hilos de una prioridad mayor si este hilo que va a pasar a ejecutarse es de prioridad menor. No se garantiza que hilos o hebras como se sobreentienda de prioridad baja pasen a ejecutarse si existe alguna hebra de mayor prioridad.

Un sencillo ejercicio que puede usarse para comprobarse es el siguiente:

```
public class CompruebaPrior implements Runnable {
int numero;
CompruebaPrior(int c) { numero = c; }
public void run() {
while (!parar) {
system.out.println(numero);
cont++;
}
}
public static void main(String[ ] args) {
Thread nuevohilo;
for (int c = 0; c < 10; c++) {
nuevohilo = new Thread(new CompruebaPrior(c));
if (c == 0) nuevohilo.setPriority(Thread.max_priority);
nuevohilo.start();
}
}
}
```

Si la máquina virtual que arranque este ejercicio no hace expropiación en hilos de mayor prioridad para ejecutar hilos de menor prioridad, solo se ejecuta el hilo número 0, por lo que sólo se enseñará ese número en la salida estandar. Si efectua expropiación, otras hebras se ejecutarán, aunque, un menor número de veces que la hebra 0. Del código anterior se pueden deducir algunas constantes y funciones definidas en la clase Thread para controlar la prioridad. Se definen dos funciones:

– setPriority(int): asigna la prioridad del hilo. Podría ocurrir la generación de una excepción de seguridad si el hilo que pide el cambio de prioridad de otra no está autorizada a hacerlo.

– getPriority(): retorna la prioridad del hilo. Para asignar los posibles valores en el parámetro de setPriority o como resultante de getPriority, la clase Thread asigna tres constantes estáticas a la clase:

– MAX_PRIORITY (= 10): es el valor que indica la prioridad máxima.

– MIN_PRIORITY (= 1): es el valor que indica la mínima prioridad.

– NORM_PRIORITY (= 5): es el valor que indica la prioridad normal, la que posee el hilo construido durante el arranque de la máquina virtual y que se encarga de arrancar la función main().

### 9.17.4. Comunicación

Otra clave para el éxito y la primacía de la utilización de diferentes hilos de ejecución en una aplicación, o aplicación multithreaded, es que pueden relacionarse entre sí. Se pueden trazar hilos para usar objetos comunes, que cada hilo puede manejar autónomamente de los otros hilos de ejecución.

El modelo clásico de comunicación de hilos de ejecución es un ejemplo productor/consumidor. Un hilo produce una salida, que otro hilo usa sea lo que sea esa salida. Entonces se crea un creador, que será un hilo que irá extrayendo caracteres por su salida; y se creará además un consumidor que vá recolectando los caracteres que vaya tomando el productor y un monitor que controla el proceso de sincronizar entre los hilos de ejecución. Funciona como una tubería, al insertar en el productor caracteres en un extremo y los lee el consumidor en el otro extremo, con el monitor siendo la propia tubería.

## Productor

El productor ampliará la clase Thread, y en este código de ejemplo se detalla:

```
/* creando la clase que funcionará como Productor denominada FabricaLetras */
class FabricaLetras extends Thread {
    private TuboLetras tuboLetras;
    private String alfabeto = "abcdefghijklmnopqrstuvwxzyABCDEFGHIJKLMNOPQRSTUVWXYZ";

    public FabricaLetras( TuboLetras t ) {
        // Mantiene una copia propia del objeto compartido
        tuboLetras = t;
        }

    public void run() {
        char c;

        // Mete 10 letras en la tuboLetras
        for( int i=0; i < 10; i++ )
            {
            c = alfabeto.charAt( (int)(Math.random()*26 ) );
            tuboLetras.lanzar( c );
            // Imprime un registro con lo añadido
            System.out.println( "Lanzadas las letras "+c+" por el tubo de letras." );
            // Espera un poco antes de añadir más letras
            try {
                sleep( (int)(Math.random() * 100 ) );
            } catch( InterruptedException e ) {;}
            }
        }
    }
```

Notar que se creará una instancia de la clase Tuberia, y que se usa el método tuberia.lanzar() para que se vaya construyendo la tubería, en principio de 10 caracteres.

## Consumidor

Ahora se reproduce el código del consumidor, que también extenderá la clase Thread:

```
/* creando la clase que funcionará como Consumidor denominada LeeLetras */

class LeeLetras extends Thread {
   private TuboLetras tuboLetras;

   public LeeLetras( TuboLetras t ) {
      // Mantiene una copia propia del objeto compartido
      tuboLetras = t;
      }

   public void run() {
      char c;

      // Consume 10 letras del tubo de letras
      for( int i=0; i < 10; i++ )
          {
          c = tuboLetras.recoger();
          // Imprime las letras retiradas
          System.out.println( "Recogida de letras"+c );
          // Espera un poco antes de coger más letras
          try {
              sleep( (int)(Math.random() * 2000 ) );
          } catch( InterruptedException e ) {;}
          }
      }
   }
```

En este supuesto, de igual manera que en el supuesto de ejemplo productor, se cuenta con un método en la clase TuboLetras, tuboLetras.recoger(), para manipular y gestionar la información.

## Monitor

El productor de información clase FabricaLetras y el consumidor representado en la clase LeeLetras, queda por ver únicamente qué es lo que hace la clase TuboLetras.

Lo que realiza la clase TuboLetras, es una tarea de supervisar las transacciones entre los dos hilos ejecutándose el productor y el consumidor. Los monitores, generalmente son elementos muy necesarios de las aplicaciones multihilo, porque soportan el flujo de comunicación entre los hilos.

```
class TuboLetras {
    private char buffer[] = new char[6];
    private int siguiente = 0;
    // Flags para conocer el estado del buffer
    private boolean estaLlena = false;
    private boolean estaVacia = true;

    // Método para extraer letras del buffer
    public synchronized char recoger() {
        // No se puede consumir si el buffer está vacío
        while( estaVacia == true )
            {
            try {
                wait(); // Se sale cuando estaVacia cambia a false
            } catch( InterruptedException e ) {
                ;
                }
            }
        // Decrementa la cuenta, ya que va a consumir una letra
        siguiente--;
        // Comprueba si se retiró la última letra
        if( siguiente == 0 )
            estaVacia = true;
        // El buffer no puede estar lleno, porque acabamos
        // de consumir
```

```
    estaLlena = false;
    notify();

    // Retorna la letra al thread consumidor
    return( buffer[siguiente] );
    }

// Método para añadir letras al buffer
public synchronized void lanzar( char c ) {
    // Espera hasta que haya sitio para otra letra
    while( estaLlena == true )
        {
       try {
           wait(); // Se sale cuando estaLlena cambia a false
        } catch( InterruptedException e ) {
            ;
            }
        }
    // Añade una letra en el primer lugar utilizable
    buffer[siguiente] = c;
    // Cambia al siguiente lugar disponible
    siguiente++;
    // Comprueba si el buffer está lleno
    if( siguiente == 6 )
        estaLlena = true;
    estaVacia = false;
    notify();
    }
}
```

En la clase TuboLetras se pueden contemplar dos características importantes: los miembros dato (buffer[ ]) son privados, y los métodos de acceso (lanzar() y recoger()) son sincronizados.

En el ejercicio anterior se prueba que la variable estaVacia es un semáforo, muy comunes en lenguajes de programación. La forma privada de los datos impide que el productor y el consumidor puedan acceder directamente a éstos datos. Si se permite el acceso directo de los hilos de ejecución a los datos, se podrían provocar problemas; si el consumidor intenta extraer datos de un buffer vacío, obtendrá excepciones superfluas, o se bloqueará el proceso.

Los métodos sincronizados de acceso frenan que los productores y consumidores dañen un objeto compartido. Mientras el productor está añadiendo una letra a la tubería, el consumidor no la puede retirar y viceversa. Esta sincronización es vital para mantener la integridad de cualquier objeto compartido. No puede ser lo mismo adaptar la clase en vez de los métodos, porque esto significa que nadie puede acceder a las variables de la clase en paralelo, sin embargo al sincronizar los métodos, sí podemos acceder a todas las variables que son de fuera de los métodos y que corresponden a la clase.

Se puede sincronizar inclusive variables, para hacer alguna función determinada sobre ellas, como por ejemplo:

```
sincronized( parada ) {
    // aquí se comienza el código fuente
    // los threads que intenten acceder a parada se pararán
    // y lanzarán una InterruptedException
    }
```

El método notify() invocado al final de cada método para acceder notifica a cualquier proceso que esté en modo espera por el objeto, en ese momento el proceso que esperaba intentará acceder de nuevo al objeto. El método wait() posibilita que el hilo se quede esperando de que le llegue un notify(), ya sea enviado por el hilo de ejecución o por el sistema.

Como ya disponemos de un productor, un consumidor y un objeto compartido, necesitaremos una aplicación que ejecute los hilos y que logre que todos comuniquen con el mismo objeto que están compartiendo.

```
class ConectaCarga {
    public static void main( String args[ ] ) {
        TuboLetras t = new TuboLetras();
        FabricaLetras p = new FabricaLetras( t );
        LeeLetras c = new LeeLetras( t );

        p.start();
        c.start();
        }
    }
```

Compilando y ejecutando esta aplicación que es el método Main, se puede comprobar el modelo que se ha desarrollado en pleno funcionamiento.

### Monitorización del productor

Los programas que hacen de productor y consumidor frecuentemente usan sistema de monitorizar remotamente, que deja al consumidor contemplar el hilo del productor con interacción con un usuario o con otra parte del sistema. En el supuesto de una red, un conjunto de hilos de ejecución productores pueden colaborar cada uno en un ordenador diferente. Los productores pueden imprimir documentos, almacenando una entrada en un registro log. Un consumidor o varios consumidores pueden gestionar el registro y crear por la noche un informe de la actividad de impresión del día anterior.

Otro ejemplo, a menor escala puede ser el uso de varias ventanas en un ordenador. Una ventana se puede usar para la entrada de datos (el productor), y otra ventana interactuando cona esa información (el consumidor).

## 9.17.5. Sincronización

### Compartiendo Objetos

Cuando en una aplicación conviven varios threads rodando a la vez, aparece la necesidad de limitar el acceso a ciertas partes del código donde el acceso simultáneo de varios procedimientos puede provocar un mal funcionamiento de la aplicación.

### Sincronización

Java utiliza la idea de monitores para concordar el acceso a los datos. Un monitor es un lugar privado y protegido en el que todos los recursos protegidos tienen los mismos bloqueos. Existe solo una llave para desbloquear todos los bloqueos dentro del monitor y los threads deben obtener esa llave para entrar en el monitor y acceder a los recursos protegidos. Si demasiados threads requieren entrar en el monitor al mismo tiempo, porque sólo un thread tiene la via/llave, los otros deberán esperar fuera hasta que el thread que tiene la llave termine y devuelva la llave a la maquina virtual Java. Una vez que el thread tiene la llave del monitor, podrá acceder a todos de los recursos que controla el monitor tantas veces como quiera, siempre que posea la llave. No obstante, si el thread quiere acceder a los recursos controlados por otro monitor, debe tomar la llave de ese otro monitor. En un momento en concreto, un thread puede disponer varias llaves y diversos threads pueden poseer diferentes llaves al mismo tiempo. En el momento que varios threads están esperando recíprocamente las llaves de otros puede producirse el efecto de interbloqueo o deadlock.

En Java, los recursos protegidos por los monitores son fracciones del programa en forma de métodos o grupos de sentencias bajo “llaves”. La palabra clave synchorized se utiliza para indicar que el siguiente método o bloque de sentencias esta coordinado por un monitor. Cuando se usa el modificador “synchronized” en un método, el objeto donde está contenido el método sólo será accedido por un thread a la vez. Cuando un thread ingresa en una parte de código sincronizada, es frenado hasta que el thread que está al principio de él finalice la ejecución de ese código.

### Wait y Notify [Métodos]

Cuando ejecutemos un thread dentro de un método sincronizado y que en la mitad del método se requiere tomar cierta información que proporciona otro thread. Por ejemplo, el primer thread puede abrir un fichero, el siguiente thread una vez abierto, puede entrar en el método y escribir algo en el fichero y para terminar, el primer thread hará las funciones de limpieza oportunas y cerrará el fichero. Si partimos de la premisa que estamos en un método sincronizado, sólo se posibilitará que un thread lo arranque. Para remediar esta supuesto,

se usa el método wait(), que provoca que el siguiente thread entre dentro del método sincronizado. El thread que ha deajdo el método, podrá volver a él cuando se invoque al método notify() desde dentro del método que nos ocupa, volviendo a ejecutar el método desde el punto donde lo dejó.

```
public synchronized void run() {
  int d = 0;

  while (d < 100) {
   System.out.println (Thread.currentThread().getNombre() + " " + d);
   d++;
   if (d == 50) {
    try {
    if (Thread.currentThread.getNombre().equals("Hilo1")) {
      this.wait();
     }
    }
    catch (InterrumptedException e) {
     System.out.println ("Falló Error");
    }
   }
  }

  if (Thread.currentThread().getNombre().equals("Hilo2"))
   this.notify();
}
```

En este ejemplo, el thread con nombre "Hilo1" contará hasta 50. Entonces abandonará el método y se lo dejará al segundo thread. El thread "Hilo2" contará hasta 99 y notificará al primero para que vuelva a ejecutar el método desde el punto donde lo dejó.

### 9.17.6. Estados

En el momento de ciclo de vida de un hilo, éste puede tener varios estados.

#### Nuevo Thread

La siguiente instrucción creará un nuevo hilo de ejecución pero no lo ejecuta, lo deja en el estado de Nuevo Thread:

```
Thread MicontrolThread = new FabricaLetrasThread();
Thread MicontrolThread = new Thread( new UnaClaseThread,"hiloA" );
```

Cuando un hilo está en este estado, es simplemente un objeto Thread vacío. El entorno no ha preparado ningún recurso para él. Únicamente desde esta situación podrá arrancarse llamando al método start(), o detenerse finalmente, llamando al método stop(); la invocación a cualquier otro método no tiene explicación y lo único que creará será generar una excepción de tipo IllegalThreadStateException.

#### Ejecutable

En este supuesto el hilo estará en modo ejecutable. Ahora comprueba las dos líneas de código que se expresan a continuación:

```
Thread MicontrolThread = new FabricaLetrasThread();
MicontrolThread.start();
```

La llamada al método start() crea los recursos del sistema precisos para que el hilo puede ejecutarse, los junta a la lista de procesos disponibles para ejecución del sistema y llama al método run() del hilo de ejecución. En este instante se encuentra en el estado ejecutable del esquema. Y este estado es Ejecutable y no En Ejecución, porque cuando el hilo está aquí no esta corriendo. Muchos ordenadores tienen únicamente un procesador lo que hace dificil que todos los

hilos estén corriendo al mismo tiempo. Java tiene un tipo de scheduling o lista de procesos, que posibilita que el procesador sea compartido entre todos los procesos o hilos que aparecen en la lista. Por ello, se puede considerar que este estado es realmente un estado En Ejecución, porque la impresión que produce ante el usuario es que todos los procesos se ejecutan al mismo tiempo.




Cuando el hilo se encuentre en este estado, todas las instrucciones de código que estén dentro del mismo bloque declarado para el método run(), se ejecutarán de manera secuencial.

### Parado

El hilo de ejecutándose entra en modo Parado cuando se invoca al método suspend(), cuando se llama al método sleep(), cuando el hilo está bloqueado en un proceso de "entrada / salida" o cuando el hilo utiliza un método wait() para estar esperando a que se efectúe una determinada condición. Cuando suceda cualquiera de las cuatro eventos anteriores, el hilo estará Parado.

Por ejemplo, en el bloque de código siguiente:

```
Thread FabricaLetras = new FabricaLetrasThread();
MiControlThread.start();
try {
    MiControlThread.sleep( 10000 );
} catch( InterruptedException e ) {
    ;
    }
```

La línea de código que invoca al método sleep():

```
MiControlThread.sleep( 20000 );
```

Esta sentencia hace que el hilo se duerma durante 20 segundos. Durante todo ese tiempo, incluso cunado el procesador estuviera completamente libre, MiControlThread no rodaría. Después de esos 20 segundos. MiControlThread volverá a estar en estado Ejecutable y en este momento el procesador puede hacerle caso cuando se encuentre disponible.

Hay una manera específica de volver a estado Ejecutable para los cuatro modos de entrada en estado Parado.Cada forma de rescatar ese estado es exclusiva; por ejemplo, si el hilo ha sido puesto en espera, una vez transcurridos los milisegundos que se indiquen, él solo se activará y volverá a estar en estado Ejecutable. Inovcar al método resume() mientras esté el hilo durmiendo no serviría para nada.

Los métodos usados para recuperar el estado Ejecutable, en la medida de la forma de entrar al estado Parado del hilo, son los siguientes:

– Cuando un hilo está dormido, agotado el lapso de tiempo.

– Cuando un hilo de ejecución está suspendido, tras una invocación a su método resume().

– Cuando un hilo está obstaculizado en una E/S, una vez que el mandato de entrada/salida concluya su ejecución.

– Cuando un hilo está en espera por una condición, todas las veces que la variable que controla esa condición cambie debe invocarse al método notify() o notifyAll().

### Muerto

Un hilo de ejecución puede morirse de dos maneras: por causas naturales o porque lo maten con sentencia stop()). Un hilo morirá cuando finaliza de forma habitual su método run(). Por ejemplo, en el siguiente bloque de código, el bucle while es un bucle finito hace la iteración 30 veces y termina:

```
public void run() {
    int i=0;
    while( i < 30 ) {
        i++;
        System.out.println( “i = “+i );
        }
    }
```

Un hilo que tenga el método run(), morirá tranquilamente tras completarse el bucle y run() finalice.

Por otro lado se puede matar en cualquier momento un hilo, invocando a su método stop(). En el trozo de código siguiente:

```
Thread MiControlThread = new FabricaLetrasThread();
MiControlThread.start();
try {
    MiControlThread.sleep( 15000 );
} catch( InterruptedException e ) {
    ;
    }
MiControlThread.stop();
```

Se crea y ejecuta el hilo MiControlThread, se duerme durante 15 segundos y en el momento de despertarse, la llamada a su método stop(), lo mata.

El método stop() manda un objeto "ThreadDeath" al hilo de ejecución que quiere matar. Cuando un hilo es parado de esta manera, muere asíncronamente. El hilo muere en el instante en que recibe ese objeto ThreadDeath.

Los "Applets" usaran el método stop() para matar a todos sus hilos cuando el navegador con soporte Java en el que se están ejecutando le indica al applet que se detengan, por ejemplo, cuando se minimiza la ventana del navegador o cuando se cambia de página.

#### El método isAlive()

El interfaz de programación de la clase Thread lleva el método isAlive(), que retorna true si el hilo ha sido iniciado con start()) y no se ha muerto con stop(). Por ello, si el método isAlive() retorna "false", sabemos que estamos delante de un nuevo Nuevo Thread o ante un thread Muerto. Si devuelve "True", se sabe que el hilo se encuentra en estado Ejecutable o Parado. No se podemos diferenciarlo entre Nuevo Thread y muerto, ni entre un hilo Ejecutable o Parado.

### 9.17.7. Creación y ejecución de hilos en el lenguaje

Un Thread posee un nacimiento, una vida y una muerte. De acuerdo a estos estados posibles, un thread puede perseguir diversas opciones dependiendo del objetivo que se quiera conseguir. Las diferentes etapas de un thread las determina un grupo de métodos establecidos que pueden sobreescribirse para ejecutar las tareas pertinentes.

En este punto describimos cada una de estas fases, así como los métodos y estados destacados.

#### Creación

Para crear un nuevo thread debemos llamar al constructor de la clase como se muestra en el ejemplo siguiente:

```
public void start () {
  if (miControlThread == null) {
   miControlThread = new Thread(this);
   miControlThread.start();
  }
}
```

Desde el instante que creamos la instancia tenemos un nuevo thread en estado "Creado". En este estado el thread es llanamente un objeto thread vacío. Todavía no se ha guardado ningún recurso del entorno para él. Cuando un thread se mantiene en Creado, lo único que podemos hacer es iniciarlo. Si invocamos a otro método saltará una excepción del tipo IllegalThreadStateExcepcion.

### Comienzo

Vemos el método que se enseña en el siguiente ejemplo:

```
public void start () {
  if (miControlThread == null) {
     miControlThread = new Thread(this);
     miControlThread.start();
  }
}
```

El método start() creará los recursos necesarios para que el thread se pueda ejecutar, gestiona cuando debe iniciar el thread y lanza el método run(). Cuando el método start() termina, el thread pasa al estado Runnable. Si tomamos en cuenta que muchos ordenadores sólo disponen de un procesador, es muy complicado ejecutar a la vez todos los threads que están en este estado. Por esta razón, la máquina virtual Java debe generar un sistema de planificación para que todos los threads compartan el procesador. Por eso un thread en estado Runnable efectivamente se halla en espera de su turno para ser ejecutado en la unidad central de proceso.

### Run [Método]

En este método se definen las acciones que deben realizarse durante la vida del thread. Por ejemplo:

```
public void run() {
  Thread Corriente = Thread.CurrentThread();
  while (miControlThread == Corriente) {
   try {
    Thread.sleep(60);
   }
   catch (InterruptedException e) {
  }
  System.out.println ("por fortuna sigo vivo");
  }
}
```

El método run() se ejecutará hasta que la circunstancia del bucle miControlThread == Corriente sea verdad. Por ello, necesitamos comprender solo que esta condición permitirá que el thread termine correctamente.

### Runnable (y Not Runnable) [Estados]

Un thread cambia al estado Not Runnable cuando suceden uno de estos eventos:

– Se invoca al método sleep().

– El thread invoca al método wait() para estar esperando a que se satisfaga alguna condición.

– El thread está bloqueado en un acción de entrada/salida.

En el ejercicio, el thread miControlThread pasa al estado Not Runnable en el momento que el método run() llama al método sleep().

```
public void run() {
   Thread Corriente = Thread.CurrentThread();
   while (miControlThread == Corriente) {
    try {
     Thread.sleep(60);
    }
     catch (InterruptedException e) {
    }
   System.out.println ("por fortuna sigo vivo del calambre");
  }
}
```

Durante los 60 milisegundos que el thread está "dormido" sleep(), no rodará aunque el procesador esté disponible. Cuando haya transcurrido ese tiempo, el thread volverá al estado Runnable otra vez y se ejecutará cuando el procesador esté disponible.

Los threads pasan de nuevo al estado Runnable cuando suceda uno de estos eventos:

– Cuando el thread está dormido, pasarán los milisegundos especificados.

– Cuando el thread está en la espera de alguna condición, otro objeto informará a dicho thread de que esa condición será cambiada llamando al método notify() o notifyAll().

– Si el thread está obstaculizado en una operación de E/S, dicha operación debe finalizar.

### Sleep [Método]

El método Sleep liberará al thread del control del microprocesador, en una cantidad de tiempo especificado en el parámetro. La sintaxis de este método será:

```
Thread.sleep(milisegundos);
```

En el método run() del ejercicio visto anteriormente, se llamará al método sleep() para posibilitar que se ejecuten otros threads desde el momento que miControlThread está durmiendo.

## Cómo se para un Thread

Un thread sucumbe de forma natural cuando su método run() concluye. En el ejemplo anterior la condición para el que bucle termine y con él, el método run() es miThread == corriente. Esta condición indica que el bucle concluirá cuando el thread que está actualmente en ejecución no sea igual a miThread.

```
public void run() {
  Thread Corriente = Thread.CurrentThread();
  while (miControlThread == corriente) {
   try {
    Thread.sleep(60);
   }
    catch (InterruptedException e) {
   }
   System.out.println (“Sigo vivo trás otro calambre”);
  }
 }
```

Cuando se precise frenar el thread, alguien debe invocar a al método sleep, wait(), stop() también se pueden utilizar aunque en algunas versiones de Java están en desuso, que parará la hebra de forma temporal. De esta manera se le está diciendo al bucle del método run() que debe parar un tiempo. En estas paradas se podrá modificar alguna variable de control del bucle, que es la que deberá inducir que el metodo run() finalice de forma definitiva y por lo tanto la hebra desaparezca.

IsAlive [Método]

Este método retornará true si el thread ha sido iniciado y no está parado. Si el método retorna false, sabremos que el thread está en estado "Creado" o en estado "Muerto". Si devuelve true el thread estará en estado "Runnable" o "Not runnable".

Yield [Método]

Este método induce que el thread que se está elaborando en un momento dado se mueva al final de la cola para permitir que el siguiente thread sea procesado.

```
public void run() {
  while (miControlThread != null) {
   System.out.println ("Sigo vivo todavía…");
   yield();
  }
}
```

## 9.17.8. Librerías de clases asociadas

### La clase Thread

Es la clase que encapsula todo el control necesario sobre los hilos de ejecución llamados threads. Hay que diferenciar claramente un objeto Thread de un hilo de ejecución o thread. Esta distinción resulta compleja de ver, aunque se puede simplificar si se considera al objeto Thread como el panel de control de un hilo de ejecución thread. La clase Thread es la única manera de gestionar el comportamiento de los hilos y por ello se vale de los métodos que se explican a continuación:

### Métodos de Clase

Estos son los métodos estáticos que deberán ser llamados de forma directa en la clase Thread.

#### currentThread()

Este método retorna el objeto thread que simboliza al hilo de ejecución que está rodando actualmente.

#### yield()

Este método actua para que el intérprete modifique el contexto entre el hilo actual y el siguiente hilo ejecutable en disposición. Es una forma de garantizar que los hilos de menor prioridad no sufran inanición.

#### sleep(long)

El método sleep() hace que el intérprete, coloque al hilo en curso a dormir un tiempo en milisegundos que refleje el parámetro. Una vez transcurridos esos milisegundos, el hilo vuelve a estar factible para su funcionamiento. Los relojes vinculados a la mayoria de los intérpretes de Java no podrán conseguir precisión mayor de 10 milisegundos, por más que se permita reflejar hasta nanosegundos en la llamada alternativa a este método.

### Métodos de Instancia

Estos son algunos de los métodos de la clase Thread, los demás correspondne a áreas en el que el estándar de Java no está terminado, y podría quedar obsoletos en las sucesivas versiones del JDK.

#### start()

Este método solicita al motor de Java que construya un argumento del hilo del sistema y proceda a ejecutarlo. De esta forma, el método run() de este hilo será invocado en el nuevo contexto del hilo. Hay que tener cuidado de no invocar al método start() muchas veces sobre un hilo concreto.

#### run()

El método run() forma el cuerpo de un hilo en ejecución. Este es el único método del interfaz Runnable. Es invocado por el método start() una vez que de que el hilo apropiado del sistema se haya inicializado. Siempre que el método run() retorne el control, el hilo actual se parará.

### stop()

Este método genera que el hilo se pare de manera completa. A menudo constituye una manera brusca de detener un hilo, especialmente si este método se ejecuta sobre el hilo en curso. En tal caso, la línea inmediatamente posterior a la llamada al método stop() no llegará a ejecutarse nunca, ya que el contexto del hilo muere anteriormente a que stop() retorne el control. Una manera más fina de parar un hilo es usar alguna variable que ocasione que el método run() finalice de manera ordenada. En realidad, nunca se debería acudir al uso de este método.

### suspend()

El método suspend() es diferente de stop(). suspend() toma el hilo y hace que se pare su ejecución sin destruir el hilo de sistema subyacente, ni el estado del hilo anteriormente en funcionamiento. Si la ejecución de un hilo se suspende, podrá invocarse a resume() en el mismo hilo para lograr que se ejecute de nuevo.

### resume()

El método resume() se usa para revivir un hilo suspendido. No hay garantías de que el hilo comience a rodar inmediatamente, ya que puede haber un hilo de mayor prioridad en ejecución, pero resume() ocasiona que el hilo vuelva a ser un candidato a ejecutarse.

### setPriority(int)

Este método setPriority() pone al hilo la prioridad asignada por el valor pasado como parámetro. Hay bastantes constantes determinantes para la prioridad de la clase Thread, tales como Min_Priority, Norm_Priority y Max_Priority, que toman los valores 1, 5 y 10, de forma individual. Puede establecerse que la mayor parte de los procesos a nivel de usuario deberían tomar una prioridad en torno a Norm_Priority. Las tareas en segundo lugar, como una “entrada / salida” a red o el nuevo dibujo de la pantalla, deberían tener una prioridad cercana a Min_Priority. Con las tareas a las que se fije la máxima prioridad, en torno a Max_Priority, hay que ser especialmente cuidadosos, porque si no se hacen llamadas a sleep() o yield(), se puede provocar que el intérprete Java quede totalmente fuera de control.

### getPriority()

Este método devuelve la prioridad del hilo ejecutándose en el momento, que es un valor situado entre uno y diez.

### setName(String)

Este método crea una identificación sobre el hilo con un nombre "mnemónico" o nemotécnico. De esta manera se ayuda a la depuración de programas multihilo. El nombre mnemónico aparecerá en todas las líneas de trazado que se muestran cada vez que el intérprete Java lanza excepciones no capturadas.

### getName()

Este método devuelve el valor actual, de tipo cadena, asignado como nombre al hilo en ejecución mediante setName().

## Creación de un Thread

Hay maneras de conseguir hilos de ejecución threads en Java. Una es implementando el interfaz Runnable, la otra es mediante la clase Thread.

La construcción del interfaz Runnable es la forma normal de crear hilos. Los interfaces aportan al programador una forma de agrupar el trabajo de infraestructura de una clase. Se utilizan para diseñar los requerimientos comunes al grupo de clases que implementará. El interfaz define el trabajo y la clase, o clases, que implementan el interfaz para ejecutar dicho trabajo. Los diferentes conjuntos de clases que construyan el interfaz deberán seguir las mismas reglas de funcionamiento.

Hay diversas diferencias entre interfaz y clase, que ya son conocidas. Primero, un interfaz solamente puede contener métodos abstractos y/o variables estáticas y finales (constantes). Las clases, por otro lado, pueden construir métodos y poseer variables que no sean constantes. Segundo, un interfaz no puede implementar cualquier tipo de método. Una clase que construya un interfaz debe implementar todos los métodos definidos en ese interfaz. Un interfaz puede extenderse de otros interfaces y, a diferencia de las clases, podrá extenderse de múltiples interfaces. Sin embargo, un interfaz no puede ser instanciado con el operador new; la siguiente instrucción no está permitida:

```
Runnable a = new Runnable();   // No está permitido
```

El primer método de crear un hilo de ejecución es llanamente extender la clase Thread:

```
class MiControlThread extends Thread {
    public void run() {
        . . .
        }
```

El ejercicio anterior creas una nueva clase “MiControlThread” que extiende la clase Thread y sobreescribe el método Thread.run() por su misma construcción. El método run() es el lugar donde se realiza toda la función de la clase. Extendiendo la clase Thread, se heredará los métodos y variables de la clase padre. En este supuesto, únicamente se podrá extender o derivar una vez de la clase padre. Esta restricción de Java puede ser solventada a través de la implementación de Runnable:

```
public class MiControlThread implements Runnable {
    Thread t;
    public void run() {
        // Se Ejecuta el thread una vez que se creó
        }
    }
```

En este caso crearemos una instancia de Thread en primer lugar y antes que el sistema pueda rodar el proceso como un hilo. Además, el método abstracto run() está definido en el interfaz Runnable y tiene que ser implementado. La discrepancia entre estos métodos es que este último es más flexible. En el ejemplo anterior, aún está la posibilidad de extender la clase MiControlThread, si fuese preciso. La mayoría de las clases que se crean y que necesitan ejecutarse como un hilo, implementan el interfaz Runnable, ya que posiblemente extenderán varias de sus funciones a otras clases.

Estudiar como el interfaz Runnable está creando alguna cosa cuando la tarea se está ejecutando. Solamente posee métodos abstractos, con lo cual es una clase para dar idea sobre la implementación de la clase Thread. De hecho, si se observan los fuentes de Java, se puede demostrar que solamente tiene un método abstracto:

```
package java.lang;
public interface Runnable {
    public abstract void run() ;
    }
```

Como se comprueba, un interfaz aporta un diseño para las clases que vayan a ser implementadas. En el caso de Runnable, mejora la definición del método run(), por ello, la mayoria del trabajo se realiza en la clase Thread. Un apreciación un poco más profunda a la definición de la clase Thread da una visión de lo que realmente está sucediendo.

```
public class Thread implements Runnable {
    ...
    public void run() {
        if( actividad != null )
            actividad.run() ;
            }
        }
    ...
    }
```

De este bloque de código se extrae que la clase Thread también implemente el interfaz Runnable. actividad.run() se garantiza que la clase con que actua la clase que va a ejecutarse como un hilo no sea nula y arranca el método run() de esa clase. Cuando esto ocurra, el método run() de la clase hará que corra como un hilo.

A continuación se presenta un ejemplo que implementa el interfaz Runnable para crear un programa multihilo.

```
class javapromulhil1 {
    static public void main( String args[ ] ) {
        // Se crean dos  objetos Thread
        Thread hiloA = new Thread( new MiHilo(),"actividad1" );
        Thread hiloB = new Thread( new MiHilo(),"actividad2" );

        // Se invocan los dos hilos, para que arranque su ejecución
        hiloA.start();
        hiloB.start();

        // Aquí se espera la ejecución unos cinco segundos y se captura la
        // posible excepción que genera el método, pero no se hace
        // nada en el caso de que se produzca
        try {
            Thread.currentThread().sleep( 5000 );
        }catch( InterruptedException e ){}

        // Lanza información sobreel Thread ó hilo principal
        // del programa
        System.out.println( Thread.currentThread() );

        // Se para la ejecución de los dos hilos
        hiloA.stop();
        hiloB.stop();
        }
    }
```

Otro ejercicio aclarativo:

```
class NoHaceNada {
// Esta clase existe únicamente para que se herede
// MiControlHilo, para impedir que esta clase sea capaz de heredar la clase
// Thread, y se pueda realizar el interfaz Runnable en su
// lugar
}

class MiControlHilo extends NoHaceNada implements Runnable {
    public void run() {
        // lanza en pantalla información sobre este hilo en particular
        System.out.println( Thread.currentThread() );
        }
    }
```

Como se puede observar, el programa define una clase MiControlHilo que extiende a la clase NoHaceNada e implementa el interfaz Runnable. Se redefine el método run() en la clase MiControlHilo para presentar información sobre el hilo.

La simple razón de extender la clase NoHaceNada es enseñar un ejemplo de situación en que hay que extender alguna otra clase, además de implementar el interfaz.

En el ejercicio se observa el mismo programa, pero en este caso se extiende la clase Thread, en lugar de implementar el interfaz Runnable para construir el programa multihilo.

```
class javarunmulhil {
    static public void main( String args[ ] ) {
        // Se crean dos nuevos objetos Thread
        Thread hiloA = new Thread( new MiHilo(),"actividad1" );
        Thread hiloB = new Thread( new MiHilo(),"actividad2" );

        // Se lanzan los dos hilos, para que comiencen su ejecución
        hiloA.start();
        hiloB.start();

        // Aquí se frena la ejecución un segundo y se capta la
        // posible excepción que genera el método, aunque no se hace
        // nada en el caso de que se produzca
        try {
            Thread.currentThread().sleep( 5000 );
        }catch( InterruptedException e ){}

        // Presenta información acerca del Thread ó hilo principal
        // del programa
        System.out.println( Thread.currentThread() );

        // Se para la ejecución de los dos hilos
        hiloA.stop();
        hiloB.stop();
        }
    }

class MiControlHilo extends Thread {
    public void run() {
        // Lanza en pantalla información sobre este hilo en concreto
        System.out.println( Thread.currentThread() );
        }
    }
```

En el ejercicio la nueva clase “MiControlHilo” extiende la clase Thread y no implementa el interfaz Runnable directamente la clase Thread implementa el interfaz Runnable, por lo que indirectamente MiHilo también está implementando ese interfaz. El resto del programa es similar al anterior.

Y todavía se puede crear un ejemplo más sencillo, utilizando un constructor de la clase Thread que no precisa parámetros, tal como se presenta en el ejercicio. En los ejemplos anteriores, el constructor utilizado para Thread requeria dos parámetros, el primero un objeto de cualquier clase que pueda implementar el interfaz Runnable y el segundo una cadena que lanza el nombre del hilo, este nombre es diferente del nombre de la variable que enfoca al objeto Thread.

```
class javaprinhil {
    static public void main( String args[ ] ) {
        // Se crean dos nuevos objetos Thread
        Thread hiloC = new MiControlHilo();
        Thread hiloD = new MiControlHilo();

        // Se lanzan los dos hilos, para que comiencen a ejecutarse
        hiloC.start();
        hiloD.start();

        // en este punto se retarda la ejecución un segundo y se capta la
        // posible excepción que crea el método, aunque no se hace
        // nada en el caso de que se efectue
        try {
            Thread.currentThread().sleep( 5000 );
        }catch( InterruptedException e ){}

        // Lanza información acerca del Thread ó hilo principal
```

```
        // del programa
        System.out.println( Thread.currentThread() );

        // Se para la ejecución de los dos hilos
        hiloC.stop();
        hiloD.stop();
        }
    }

class MiControlHilo extends Thread {
    public void run() {
        // Lanza en pantalla datos sobre este hilo en concreto
        System.out.println( Thread.currentThread() );
        }
    }
```

Las lineas de instrucción en este ejemplo para instanciar objetos Thread, son mucho más sencillas, siendo el programa, esencialmente, el mismo de los ejemplos anteriores.

### Arranque de un Thread

Las aplicaciones ruedan main() tras ejecutarse. Este es el motivo de que main() sea el lugar habitual para crear y arrancar otros hilos. La línea de código:

```
temporizador1 = new TestTh( “Thread 1”,(int)(Math.random()*4000) );
```

Este ejemplo construye un nuevo hilo de ejecución. Los dos argumentos pasados representan el nombre del hilo y el tiempo que se desea que espere antes de imprimir el mensaje.

Al tener control directo sobre los hilos, hay que arrancarlos explícitamente. En el ejemplo con:

```
temporizado1.start();
```

start(), en realidad es un método oculto en el hilo de ejecución que llama a run().

### Manipulación de un Thread

Si todo fue ok en la formación del hilo, temporizador1 debería tener un thread, que gestionaremos en el método run().

Así dentro de run(), se comienzan las sentencias de rodaje como en otros programas. run() sirve como rutina main() para los hilos; cuando run() termina, también lo hace el hilo. Las funciones y actividades que necesitamos que haga el hilo de ejecución debe estar incluido dentro de run(), para ello cuando se dice que un método es Runnable, es obligatorio escribir un método run().

En la siguiente linea, el método se espera unos segundos de manera aleatoria pasada a través del constructor:

### sleep(retardo);

El método sleep() simplemente envia al hilo ejecutándose que duerma durante los milisegundos indicados. Se debe usar sleep() en el momento que queramos retardar la ejecución del hilo. sleep() no gasta recursos del sistema en el momento que el hilo duerme. De esta manera otros hilos pueden seguir funcionando. Una vez hecho el retardo, se lanza el mensaje “Hola Mundo!” con el nombre del hilo y el retardo.

### Suspensión de un Thread

Es bastante utilizado si necesitamos eliminar la ejecución de un hilo sin tomar un límite de tiempo. Si, por ejemplo, queremos crear un "Applet" con un hilo de animación, posiblemente nos permitiremos la opción de frenar la animación hasta que queramos continuar. No se trata de finalizar la animación, sino desactivarla. Para estas gestiones de los hilos ejecutándose se puede utilizar el método suspend().

```
temporizador1.suspend();
```

Con este método no detenemos ni abortamos la ejecución completamente. El hilo es suspendido sin tiempo concreto y de manera indefinida y para volver a activarlo de nuevo se requiere realizar una invocación al método resume():

```
temporizador1.resume();
```

Este método retorna verdadero (true) en caso de que el hilo temporizador1 esté vivo, esto quiere decir, que ya se haya invocado a su método run() y no se ha parado con un stop() ni haya finalizado el método run() mientras se ejecuta.

```
temporizador1.isAlive();
```

En el ejemplo podemos hacer una parada incondicional, al encontrarse todos los hilos vivos. Pero si a un hilo de ejecución, que puede no estar vivo, se le llama su método stop(), se originará una excepción. En este caso, en los que el estado del hilo no se conoce de primera mano es donde se necesita el uso del método isAlive().

Si se necesita, se puede comprobar si un hilo está vivo o no; considerando vivo un hilo que ha comenzado y no ha sido detenido.

### Parada de un Thread

Como elementos de control que requerimos sobre los hilos de ejecución es el método stop() el último a tratar. Es usado para finalizar la ejecución de un hilo:

```
temporizador1.stop();
```

Esta llamada no destruye el hilo, sino que detiene su ejecución. La ejecución ya no se puede reanudar con temporizador1 .start(). Cuando se desasignen las variables que se usan en el hilo, el objeto Thread (creado con new) estará marcado para suprimirlo y el "garbage collector" se ocupará de liberar la memoria que utilizaba.

En el ejemplo, no se requiere detener rotundamente el hilo de ejecución. Simplemente se le deja terminar. Los programas más complejos necesitarán un control sobre cada uno de los hilos que lancen, el método stop() puede utilizarse en esas situaciones.

## 9.17.9. Programación multihilo

### Inicialización de una hebra

Antes de que se use un thread, deberá este inicializar y construir una instancia de la clase Thread. La mejor manera de hacerse es usando el constructor de la clase Thread. La forma más simple de este constructor es:

```
Thread identificador = new Thread();
```

Otras variaciones de este constructor son:

```
Thread identificador = new Thread(ReferenciaAObjeto);
Thread identificador = new Thread(Nombre);
Thread identificador = new Thread(ReferenciaAObjeto, Nombre);
```

El parámetro ReferenciaAObjeto marca la referencia al objeto de la clase que implementa el método run() y que es el que debe rodarse cuando el thread comience su ejecución. El parámetro Nombre se utiliza para asignar un nombre a cada uno de los threads por si se quiere distinguir. Por ejemplo:

```
Thread unControlThread = new Thread (invitado, “comensal1”);
Thread unControlThread = new Thread (invitado);
```

Como solamente puede iniciarse un Thread cada vez, el ordenamiento en que se invocan los threads no tiene por qué definir necesariamente el orden en el que finalizan. Volvemos al ejercicio Saludos para enseñar cómo es difícil de predecir el orden de ejecución de los threads.

```
public class Saludos implements Runnable {
  // Métodos
  public void run () {
   System.out.println (Thread.currentThread().getName() +  " te ha ha saludado");
  }

  // Main
  public static void main (String args[ ]) throws InterruptedException {

   int comida = 0;
   int baile = 0;

   Saludo invitado = new Saludo();

   // Se crea el primer thread
   Thread unThread = new Thread (invitado, "comensal1");

   // Se crea el segundo thread
   Thread otroThread = new Thread (invitado, "comensal2");

   unThread.start(); // se inicia el primer thread
   otroThread.start(); // se inicia el segundo thread
```

```
    // Cuerpo del programa principal
    while (unThread.isAlive() || otroThread.isAlive()) {

     // bucle contador del primer thread
     if (unThread.isAlive())
      comida++;

     // bucle contador del segundo thread
     if (otroThread.isAlive())
      baile++;
    }

    // Se arranca después de que los threads hayan finalizado
    System.out.println ("El valor de comida es " + comida );
    System.out.println ("El valor de baile es " + baile );

    System.out.println ("Todo el mundo ha saludado");

   }
}
```

Como ambos threads se forman usando el mismo objeto, se envia un string para distinguir un thread del otro. En el método run(), el método getName() se utiliza para lanzar el nombre del thread que se está ejecutando. En este ejercicio se ha insertado un contador para cada threads. El planificador de la máquina virtual Java es el responsable de indicar qué threads pueden arrancar y cuales deberán esperar en la cola. Esta decisión se podrá tomar de dos formas: por "Prioridad" o "FIFO"First in, First Out. Por "orden de llegada a la cola".

## 9.18. Definición y análisis de programación en red

Java contiene un grupo de clases que posibilitan usar los URLs (Uniform Resource Locators). Un URL es una dirección electrónica que permite localizar una información en Internet especificando:

– El nombre del protocolo que permitirá leer la información. Por ejemplo HTTP.

– El nombre del servidor que proporciona la información. Por ejemplo www.nic.es

o bien una dirección IP directamente.

– El nombre del fichero en el servidor. Por ejemplo /java/data/acceso.html

Java es seguramente el lenguaje con más provenir en cuanto a la programación para Internet.

Con certeza, podría evolucionar hasta el punto de que el navegador no interpretara las applets de Java, sino que él forme un conjunto de applets que se descarguen de Internet según se vayan necesitando. Por ello en todo momento podríamos estar rodando la última versión del navegador.

No obstante pensando un poco más innovador, podríamos plantearnos conectarnos a servidores que nos cobraran por el uso de sus programas hojas de cálculo, procesadores de texto… en función del tiempo de uso, trabajando siempre con la última versión del mismo, en lugar de invertir nuestro dinero en actualizaciones. En este sentido el futuro ya ha llegado con el concepto de Cloud computing y **Java**.

### Programación cliente–servidor

Java es un lenguaje de programación nada nuevo y en competencia con otros lenguajes por su uso. Que aunque resuelve algunos de los problemas típicos de este tipo de programación basada en la web que crean en Internet, en la Telaraña Mundial, en el World–Wide–Web, en la nube (In the Cloud). Pero, en esencia, Internet es un sistema Cliente–Servidor gigante. La idea primaria de un sistema cliente–servidor es que debe haber un sitio donde se centraliza la información, que se desea distribuir bajo demanda a un conjunto de personas o máquinas.




La clave de esta concepción está en que si se produce un cambio modificaciones en la información del sistema servidor central, inmediatamente es propagada a los receptores–clientes de la información, a la parte remota.

El gran incoveniente se plantea cuando hay únicamente un servidor que tiene enganchados a muchísimos clientes, y el rendimiento principal del sistema cae de forma exponencial al incremento del número de usuarios/as. Todos conocemos el sufrimiento de asesores y empresarios con el conocido software Tributario en periodos de renta.

La operatividad de la Web desde sus comienzos seguía este patrón de funcionamiento. Se solicita una información a un servidor y éste envía de vuelta un fichero que será interpretado por el navegador "el cliente" que lo formateará para visualizarlo en la máquina cliente. El software navegador ha sido el primer gran paso adelante en la expansión de Internet, ya que posibilitaba visualizar un mismo fichero en plataformas distintas sin hacerle cambio alguno; porque su objetivo principal es la visualización de archivos, no la interactividad con el usuario, ni la posibilidad de trabajar con programas en la parte del usuario, que es el segmento cliente del sistema.

Al lenguaje HTML se le fue apareciendo diferentes motores de ejecución y software diferente para potenciar las ventajas de procesamiento en el servidor. Con HTML5 por ejemplo y otros ciertos mecanismos básicos de entrada de datos, como botones, cajas de selección, campos de texto, y algunos otros; aunque la carga que añadian al sistema se volcaba del todo dentro del lado del servidor, por ello, si existían muchos usuarios/as enganchados, el bloqueo del servidor era completo. Por suerte aparecieron algunas alternativas a los CGI, para que se pudiera liberar al servidor de tanta carga y tráfico y que el cliente realizase también operaciones.

## Plug–ins

Son bloques de código que se incorporan al navegador y le indican diferentes usos y aplicaciones. Hay características que se añaden a los navegadores a través de plug–ins que son muy rápidas y eficientes, pero el problema es que escribir un plug–in no es una tarea trivial ni tampoco es algo que haya que hacer en el proceso de construcción de un Site. Además los plug–ins son herramientas óptimas para programadores muy expertos que permiten incorporar características nuevas al navegador sin necesidad de que el vendedor conceda su permiso.

## Scripts

Un lenguaje script permite incrustar código fuente para la programación del lado cliente, directamente en la página HTML, y el plug–in que interpreta ese lenguaje se activará automáticamente cuando se cargue en el navegador. Estos bloques de código en general son muy simples y sencillos, porque se cargan muy rápidamente ya que van insertados en la página web que envía el servidor. El incoveniente es que el código del desarrollador está expuesto a la vista de cualquiera, aunque tampoco es muy importante ya que no se pueden hacer programas grandes con un lenguaje script.

Estos lenguajes o metalenguajes se utilizan basicamente para potecniar los interfaces gráficos de las páginas, que disponen de elementos gráficos y pueden llegar a resolver el 80% de los problemas que se plantean en la programación de la parte cliente del sistema cliente–servidor; y, además, son lenguajes mucho más sencillos de aprender e implementar soluciones con ellos que recurrir a Java o ActiveX.

El lenguaje script más usado es “JavaScript”, que no tiene nada que ver con Java, también se pueden realizar scripts en la parte cliente con Visual Basic. Es muy potente y cómodo de utilizar.

## Java

**Java** es el nuevo enfoque de los lenguajes de programación para internet. Se puede hacer bien cualquier tipo de software de gestión y tenerlo in the Cloud para ser usado por decenas o cientos de usuarios. **Java** dice un poco adiós a los programas que se instalaban desde Cd–rom o disco duro, todo está cambiando radicalmente. ¿Y qué pasa con el 20% de los problemas que no pueden resolver los lenguajes script? La respuesta más común, si se hiciese una encuesta, sería Java, y no solamente porque sea un poderoso lenguaje de programación enfocado a la seguridad, multi–plataforma e internacional, sino porque Java está siendo constantemente extendido para proporcionarle nuevas funcionalidades y librerías de tod tipo que resuelven problemas que son complicados en la programación tradicional como sistemas en real time, el acceso a bases de datos, el uso de multihilo, la programación de redes y la programación distribuida, y además porque Java, a través de los applets, permite la programación de la parte cliente.

Un “Applet” es un pequeño software que rueda solamente bajo un navegador y es descargado automáticamente como parte de una página Web, al igual que cualquier gráfico, y cuando se activa, ejecuta un programa. Además, proporciona una manera a través de la cual se puede distribuir software al cliente desde el servidor, en el instante en que el cliente requiera ese software, y no antes, con lo cual siempre poseerá el cliente la última versión de ese programa, se actualice cuando se actualice. Por ello, tal como se diseñó Java, el desarrollador construye su programa solo una vez, y ya estará listo para ser ejecutado en todas las plataformas que dispongan de un navegador con soporte Java.

Con **Java** se realiza en el cliente antes y después de hacer peticiones al servidor. Permite mandar una petición a través de Internet mientras el usuario no haya termiando de introducir los parámetros correctos de esa petición, que estará chequeando el cliente, sin obligación de tener que consultar constántemente al servidor; con ello se consigue seguridad de datos y velocidad de respuesta ante el usuario, una reducción general en el tráfico de red y una gran descarga de trabajo para el servidor.

Una de las bondades de los applets sobre los scripts es que están en forma compilada, con lo cual el código fuente no es visible; aunque se puede descompilar sin demasiadas complicaciones, pero vamos, no está del todo accesible. También, un applet podría comprimir distintos módulos, usando ficheros *JAR*, impidiendo múltiples conexiones con el servidor, ya que en una sola se obtienen descargados todos los elementos necesarios.

### ActiveX

En estos elementos de programación basados en internet, el competidor directo de Java es "ActiveX", de Microsoft, aunque su punto de vista es muy distinto. ActiveX es originalmente una solución orientada a entornos Windows, aunque parece ser que ahora está siendo desarrollado para ser multiplataforma. Microsoft Internet Explorer soporta ActiveX directamente y otros navegadores como Opera, Chrome y Firefox lo soporta a través de plug–in. Por lo tanto, ActiveX no está constreñido a un solo lenguaje de programación, sino que se pueden desarrollar componentes ActiveX en el lenguaje que domine el programador, ya sea C++, Visual Basic o el Delphi de Borland.

La programación y posterior procesamiento de componentes ActiveX en nuestros ordenadores, con la descarga de estos pequeños programas es muy subceptible de trer dolor de cabeza y problemas de seguridad, ya que son componentes muy inseguros. Los usuarios puede bajarse un componente ActiveX que haga destrozos en su máquina.




Esto no quiere decir nada ya que existen ActiveX geniales y muy bien programados para diferentes acciones y aplicaciones. En el momento de solicitar una página Web, se reciben ficheros gráficos, el código de la página con o sin scripts, código Java precompilado y componentes ActiveX.

Con respecto a Java, hay diversas opiniones respecto a la posibilidad de bloqueo a que un “Applet” no pueda escribir en el disco local del cliente; porque, que pasa si el usuario quiere crear una base de datos local, o quiere guardar cualquier tipo de información para cuando se encuentre desconectado de Internet. Los applets con firma, que poseen una clave pública para cheqeuar que el applet viene del lugar idoneo y no se ha podido manipular, es sin duda una de las mejores soluciones. Ya desde los comienzos con Java 1.1 se disponía de un sistema para insertar firmas digitales, para evitar y controlar los Applets si fuera requerido.

### 9.18.1. Aplicación cliente servidor

Los sockets son basicamente formas en las que podemos interconectar dos o mas programas mediante el medio internet. En Java se utilizan para poder crear conexiones utilizando basicamente una IP/hostname y un puerto (port) para establecer la conexión. Para aprender podemos utilizarla para conenctar 2 programas por medio de Internet.

El modelo mas corriente y didáctico de los sockets consta de 2 simples programas, un servidor y un cliente. Basicamente el programa servidor comienza a “escuchar” en un puerto determinado (nosotros lo especificamos), y posteriormente el programa que la hace de “cliente” debe conocer la ip o nombre de [dominio / hostname] del servidor y el puerto que esta escuchando, conociendo estos parámetros se solicita establecer una conexión con el servidor. Es aqui cuando el servidor acepta esa conexión y se puede decir que estos programas estan conectados, de este modo pueden intercambiar información.

El cliente debería saber tanto el puerto a usar como la IP o dominio existente del servidor, mientras el servidor solo debería saber el puerto de conexión.

A continuación os dejo este código de un servidor muy simple Chat realizado en Java, lo que hace es escuchar el puerto 5557, cuando un cliente se conecta este manda un mensaje para confirmar al cliente, luego el cliente al enviar su primer mensaje, el servidor manda un segundo mensaje y se cierra la conexión. Además el de un cliente muy simple solo para que se aprecie las ventajas ente uno y otro, de como se solicita la conexión al servidor, el cliente únicamente envía un mensaje de texto pero no recibe información.

Código fuente Java para el Servidor:

```
/**
* Código Java del Servidor de chateo.
 */
package chat.servidor;

import java.net.ServerSocket;
import java.net.Socket;

import javax.swing.DefaultListModel;

public class SalaServidorChat
{
   /** Lista en la que se almacena toda la conversacion */
   private DefaultListModel conversacion = new DefaultListModel();

   public static void main(String[ ] args)
   {
      new SalaServidorChat();
   }

   public SalaServidorChat()
   {
      try
       {
         ServerSocket socketServidor = new ServerSocket(5557);
          while (true)
           {
             Socket chatero = socketServidor.accept();
             Runnable nuevoChatero = new HiloDeChatero(conversacion, chatero);
              Thread hilo = new Thread(nuevoChatero);
              hilo.start();
           }
      } catch (Exception e)
       {
         e.printStackTrace();
       }
   }
}
```

Código fuente Java para el Cliente:

```
/**
* Cliente de un chat
package chat.cliente;

import java.net.Socket;

import javax.swing.JFrame;
import javax.swing.WindowConstants;

public class SalaClienteChat
{
    /** Socket con el servidor del chat */
    private Socket socket;

    /** Panel con la ventana del cliente */
    private PanelCliente panel;

    public static void main(String[ ] args)
    {
        new SalaClienteChat();
    }
    public SalaClienteChat()
    {
       try
        {
```

```
            creaYVisualizaVentana();
            socket = new Socket("192.168.1.15", 5557);
            ControlCliente control = new ControlCliente(socket, panel);
        } catch (Exception e)
        {
            e.printStackTrace();
        }
    }
    private void creaYVisualizaVentana()
    {
        JFrame v = new JFrame();
        panel = new PanelCliente(v.getContentPane());
        v.pack();
        v.setVisible(true);
        v.setDefaultCloseOperation(WindowConstants.exit_on_close);
    }
}
```

## 9.18.2. Sockets

### Fundamentos

Los sockets son unos módulos de comunicación, entre procesos de diferentes ordenadores de una red. Con más profundidad comentar que un socket es un enlace de comunicación por el cual un proceso puede emitir o recibir información.

Los sockets deben conseguir usar el protocolo de streams TCP "Transfer Contro Protocol" y el de datagramas UDP "User Datagram Protocol".

Utilizan una serie de directivas para establecer el punto de comunicación, para conectarse a una máquina remota en un determinado puerto que esté disponible, para escuchar en él, para leer o escribir y publicar información en él, y finalmente para desconectarse.

Con todas las directivas se puede crear un sistema de diálogo muy completo.

### Funcionamiento genérico

El sistema es el siguiente: Un servicio que en un lenguaje como Java, se ejecuta sobre un ordenador concreto y tiene un socket que responde en un puerto específico. El servidor únicamente espera, escuchando a través del socket a que un ordenador cliente haga una petición.

Desde la óptica del lado cliente: el cliente conoce el nombre de host de la máquina con la IP en la cual el servidor se encuentra ejecutando un proceso en Java y el número de puerto en el cual el servidor está conectado. Para realizar una petición de conexión, el cliente intenta encontrar al servidor en la máquina servidora en el puerto especificado.

En el proceso el equipo servidor admite la conexión. Al mismo tiempo de aceptar, el servidor obtiene un nuevo socket sobre un puerto diferente. Esto se debe a que necesita un nuevo socket y en consecuencia, un numero de puerto diferente, para seguir atendiendo al socket original para peticiones de conexión mientras atiende las necesidades del cliente que se ha conectado.

Por la parte del cliente, si la conexión es admitida, el socket se instaura de forma cómoda y puede usarlo para comunicarse con el servidor. Es importante darse cuenta que el socket en el cliente no está utilizando el número de puerto usado para realizar la petición al servidor. En lugar de éste, el cliente asigna un número de puerto local a la máquina en la cual está siendo ejecutado.

Ahora el cliente y el servidor pueden comunicarse haciendo cualquier tipo de cosas mediante la conexión de sockets.

### Java Sockets

El paquete java.net incorpora la clase Socket, la cual construye una de las partes de la comunicación bidireccional entre un programa Java y otro programa en la red.

La clase Socket se encuentra en la zona más alta de una construcción dependiente de la plataforma, escondiendo los detalles de cualquier sistema. Usando la clase java.net.Socket en vez de emplear código fuente nativo, los programas Java mantienen comunicación por la red de una forma totalmente independiente de la plataforma.

Además java.net contiene la clase "ServerSocket", la cual construye un socket el cual los servidores pueden utilizar para oír y aceptar peticiones de conexión de clientes.

Nuestro objetivo será conocer cómo utilizar las clases Socket y ServerSocket.

Como queremos conectar a través de la Web, la clase URL y clases relacionadas "URLConnection, URLEncoder" son seguramente más apropiadas que las clases de sockets. Las clases URL son una conexión a un nivel más alto a la Web y manejan bien como parte de su implementación interna los sockets.

## Modelo de comunicaciones con Java

El modelo de sockets más simple es:

– El servidor establece un puerto y espera durante un cierto tiempo a clientes que establezcan la conexión. Cuando el cliente solicite una conexión, el servidor abrirá la conexión socket con el método accept().

– El cliente implanta una conexión con el ordenador host desde el puerto que se designe en puerto#

– El cliente y el servidor mantienen comunicación con "InputStream" y "OutputStream" Apertura de Sockets. Si queremos programar el software cliente el socket se abre de la forma:

```
Socket ConexionCliente;
ConexionCliente = new Socket( "equipo", numeroPuerto "5557" );
```

En la sintaxis "equipo" es el nombre de la máquina en donde estamos intentando abrir la conexión y numeroPuerto es el puerto (suele usarse un rango entre 5000–9999) del servidor que está operativo sobre el cual nos queremos conectar. Cuando se elige un número de puerto, debemos tener en cuenta que los puertos en el rango "0 a 1023" están siempre reservados para usuarios con muchos privilegios (superusuarios o root). Estos puertos son usados por los servicios estándar del sistema como email, telnet, ftp o http. Para los nuevos programas a desarrollar, se recomienda comenzar usando un puerto por encima del 1024.

En el ejemplo anterior de SalaChat no se usan excepciones por esclarecer mejor el ejercicio, pero es muy recomendable capturar excepciones al programar con sockets. El mismo ejemplo quedaría como:

```
Socket ConexionCliente;
try {
ConexionCliente = new Socket( “equipo”,”5557” );
} catch( IOException e ) {
System.out.println( e );
}
```

Si estamos programando un servidor, la manera de abrir el socket es la que muestra el siguiente ejemplo:

```
Socket ConexionServidor;
try {
ConexionServidor = new ServerSocket( numeroPuerto );
} catch( IOException e ) {
System.out.println( e );
}
```

Si queremos implementar un servidor necesitamos crear un objeto socket desde el ServerSocket para que esté a la escucha de las conexiones que le puedan solicitar clientes y puedan aceptar dichas conexiones:

```
Socket socketServidor = null;
try {
socketServidor = ConexionServidor.accept();
} catch( IOException e ) {
System.out.println( e );
}
```

Creación de streams

### Creación de streams de entrada

En el bloque de código de la parte “Cliente”, se puede utilizar la clase DataInputStream para crear un stream de entrada, que esté listo a recibir todas las solicitudes que el servidor le mande.

```
DataInputStream puertauno;
try {
puertauno = new DataInputStream( miControlCliente.getInputStream() );
} catch( IOException e ) {
System.out.println( e );
}
```

La clase DataInputStream permite leer líneas de texto y tipos de datos primitivos de Java de un modo versátil. Tiene métodos para leer todos esos tipos como: read(), readChar(), readInt(), readDouble() y readLine(). Tendremos que usar la función que estimemos necesaria dependiendo del tipo de dato que esperamos recibir del servidor.

En el lado del “Servidor”, además emplearemos DataInputStream, pero en este supuesto para recibir las entradas que se produzcan de los clientes que se hayan enganchado:

```
DataInputStream puertados;
try {
puertados = new DataInputStream( socketServicio.getInputStream() );
} catch( IOException e ) {
System.out.println( e );
}
```

### Creación de streams de salida

Podremos construir un stream de salida en el lado del cliente para mandar datos al socket del servidor usando las clases PrintStream o DataOutputStream:

```
PrintStream salidauno;
try {
salidauno = new PrintStream( miControlCliente.getOutputStream() );
} catch( IOException e ) {
System.out.println( e );
}
```

La clase PrintStream posee métodos para la representar textualmente datos primitivos de Java. Sus métodos write() y println() tienen una especial relevancia en este apartado.

Sin embargo, para mandar datos al servidor podemos utilizar además DataOutputStream:

```
DataOutputStream salidados;
try {
salidados = new DataOutputStream( miControlCliente.
getOutputStream() );
} catch( IOException e ) {
System.out.println( e );
}
```

La clase DataOutputStream soporta escribir tipos de datos primitivos de Java, bastantes de sus métodos escriben un tipo de dato primitivo en el stream de salida. De todos esos métodos, el más útil quizás sea writeBytes().

En el lado del "Servidor", podemos utilizar la clase PrintStream para enviar información al cliente:

```
PrintStream salidatres;
try {
salidatres= new PrintStream( socketServicio.getOutputStream() );
} catch( IOException e ) {
System.out.println( e );
}
```

Pero además podríamos usar la clase DataOutputStream como en el caso de mandar información desde el cliente.

## Cierre de Sockets

Es importante desconectar y cerrar los canales de entrada/salida que se estuvieran abiertos durante la ejecución de la aplicación.

En la parte del Cliente:

```
try {
salidauno.close();
puertauno.close();
miControlCliente.close();
} catch( IOException e ) {
System.out.println( e );
}
```

Y en la parte del servidor:

```
try {
salidauno.close();
puertauno.close();
socketServidor.close();
miServidor.close();
} catch( IOException e ) {
System.out.println( e );
}
```

## Clases útiles en comunicaciones

### Socket

Esta clase proporciona métodos para la “Entrada/Salida” a través de streams que hacen la lectura y escritura a través de sockets muy sencilla.Es el objeto básico en toda comunicación a través de Internet, bajo el protocolo TCP.

### ServerSocket

Este objeto no realiza el servicio, sino que crea un objeto Socket en función del cliente para efectuar toda la comunicación a través de él.Es un objeto utilizado en los programas servidor para escuchar las solictudes que realicen los clientes conectados a ese servidor.

### DatagramSocket

Aunque la comunicación por estos sockets es bastante rápida ya que no hay que perder tiempo estableciendo la conexión entre cliente y servidor.La clase de sockets datagrama puede utilizarse para crear datagramas no fiables como sockets UDP protocolo sin seguridad ni control de errores.

### DatagramPacket

Clase que conlleva un paquete datagrama con información de paquete, longitud, dirección IP y dígitos de puerto.

### MulticastSocket

Esta clase sirve para construir “Multicast” de las clase socket datagrama. Esto quiere decir múltiples “clientes/servidores” pueden enviar por multicast, conjunto de direcciones IP y que comparten el mismo número de puerto.

### NetworkServer

Esta clase sirve para aplicar métodos en la creación de un servidor.

### NetworkClient

Una clase sirve para aplcar métodos utilizados en la creación de un cliente.

### SocketImpl

Es un Interface para crear modelo de comunicación.Si vamos a desarrollar una aplicación con requerimientos especiales de comunicaciones, como pueden se la implementación de un cortafuegos, o acceder a equipos especiales como un lector de código de barras o un GPS, necesitaremos nuestra propia clase Socket.

## Ejemplo de uso

Vamos a proceder a estudiar un ejemplo donde se incluyen clases para manejo de Sockets y otras comunicaciones en red.

## Programa cliente

La aplicación cliente se conectará a un servidor mostrando el nombre del ordenador y el número puerto en el que el servidor está configurado.

Una vez conectado, lee una cadena del servidor y la escribe en la pantalla:

```
import java.io.*;
import java.net.*;
class ConexiondeCliente {
static final String host = “192.168.1.16”;
static final int puerto=5557;
public ConexiondeCliente( ) {
try{
Socket skCliente = new Socket( host , Puerto );
InputStream aux = skCliente.getInputStream();
DataInputStream flujo = new DataInputStream( aux );
System.out.println( flujo.readUTF() );
skCliente.close();
} catch( Exception e ) {
System.out.println( e.getMessage() );
}
}
public static void main( String[] arg ) {
new ConexiondeCliente();
}
}
```

Tenemos que crear el socket denominado skCliente, al que se le detallan el nombre de host (host) y el número de puerto (port) en este ejercicio como constantes. Se vincula el flujo de datos de dicho socket obtenido mediante getInputStream), que es asociado a un flujo (flujo) DataInputStream de lectura secuencial. De dicho flujo capturamos una cadena readUTF(), y la lanzamos por pantalla (System.out). Cerramos el socket, una vez finalizadas las operaciones, mediante el método close(). Se incluye excepción para captar posibles fallos tanto de los flujos de datos como del socket.

### Programa Servidor

La aplicación servidor se asigna en un puerto concreto el que sea dentro de los rangos que conocemos de uso, esperando algunas conexiones, a las que tratará mediante un segundo socket. Cada vez que se presenta un cliente, le saluda con una frase "Hola cliente Nº». Este servidor sólo atenderá hasta tres clientes, y después termina de ejecutarse, pero es común utilizar bucles infinitos while(true) en los servidores, para que puedan atender llamadas constántemente.

Tras atender cuatro clientes, el servidor deja de ofrecer su servicio:

```
import java.io.* ;
import java.net.* ;
class ConexionServidor {
static final int puerto=5557;
public ConexionServidor( ) {
try {
ServerSocket skServidor = new ServerSocket(puerto);
System.out.println("Te escucho por el puerto " + puerto );
for ( int numCli = 0; numCli < 3; numCli++; ) {
Socket skCliente = skServidor.accept(); // Crea objeto
System.out.println("Sirvo al cliente " + numCli);
OutputStream aux = skCliente.getOutputStream();
```

```
DataOutputStream flujo= new DataOutputStream( aux );

flujo.writeUTF( “Bienvenido cliente “ + numCli );
skCliente.close();
}
System.out.println(“Muchísismos clientes por hoy”);
} catch( Exception e ) {
System.out.println( e.getMessage() );
}
}
public static void main( String[ ] arg ) {
new ConexionServidor();
}
}
```

Se usa clase ServerSocket (skServidor) para esperar la conexión en un puerto determinado (port), y un objeto de la clase Socket (skCliente) que sirve para negociar una conexiones con pc clientes.

Usamos un “bucle for” y la variable “numCli” se controla los clientes a tres, así cada vez que en el puerto de este servidor aparezca un cliente, se gestiona y se aumenta el contador. Para atender a los clientes se utiliza accept() de la clase ServerSocket, que es una rutina que crea un nuevo Socket (skCliente) para atender a un cliente que se ha conectado a ese servidor.

Como siguiente punto asociamos al socket construido (skCliente) con un flujo (flujo) de salida DataOutputStream, en el que se escribe el mensaje a mandar. El control de las excepciones es muy corto en el ejercicio, solamente captura y lanza el mensaje que incluye la excepción con getMessage().

## Ejecución

Como el diseño de los sockets está diseñada para trabajar con ordenadores en red, en sistemas operativos multitarea como Mac, Windows y Linux se puede chequear el buen funcionamiento de un programa de sockets en un ordenador local. Podemos poner el servidor en una ventana, obteniendo lo siguiente:

```
>java ConexionServidor
Escucho el puerto 5557
En otra ventana se lanza varias veces el programa cliente, obteniendo:
>java Conexiondecliente
Bienvenido cliente 1
>java Conexiondecliente
Hola cliente 2
>java Conexiondecliente
Hola cliente 3
>java Conexiondecliente
connection refused: no further information

En paralelo en la ventana del servidor se ha lanzado lo siguiente:
Sirvo al cliente 1
Sirvo al cliente 2
Sirvo al cliente 3
Muchísimos clientes por hoy
```

En el momento que se lanza la cuarta conexión de cliente, el servidor ya ha desconectado la conexión, con lo que se lanza una excepción. Es interesante ver que tanto el cliente como el servidor pueden leer o escribir del socket. Los sistemas de comunicación podrán ser reescritos modificando la implementación de los sockets, con el uso de las clases abstractas que posee el paquete java.net.

## 9.19. Acceso a bases de datos desde las aplicaciones. Librerías de clases asociadas

Jdbc (Java Database Connect) es para Java lo que Odbc(Open Database Connect) es para Windows. Windows define el estándar Odbc consistente en un conjunto de herramientas de conexión que cualquier driver o fuente ODBC debe ser capaz de entender y manipular. Los programadores que a su vez deseen escribir programas para manejar bases de datos genéricas en Windows utilizan las llamadas Odbc.

Con Jdbc pasa lo mismo: Jdbc es una descripción de un conjunto de clases y métodos de operación que facilitan a cualquier programa Java conectar a sistemas de bases de datos de forma homogénea. Naturalmente al igual que Odbc, la aplicación de Java debe tener acceso a un driver Jdbc adecuado. Este controlador es el que construye la funcionalidad de todas las clases de acceso a datos y suministra la forma de comunicar entre la API Jdbc y la base de datos real.

Es necesario Jdbc, incluso existiendo Odbc, debido a que Odbc es un interfaz diseñado en lenguaje C, haría que los programas Java dejasen atrás perdiendo su estructura portable. Odbc tiene el problema de que se ha de instalar de maneral manual en cada ordenador; a la inversa que los drivers Jdbc, que como se escribieron en Java son dinámicamente instalables, portables y seguros.

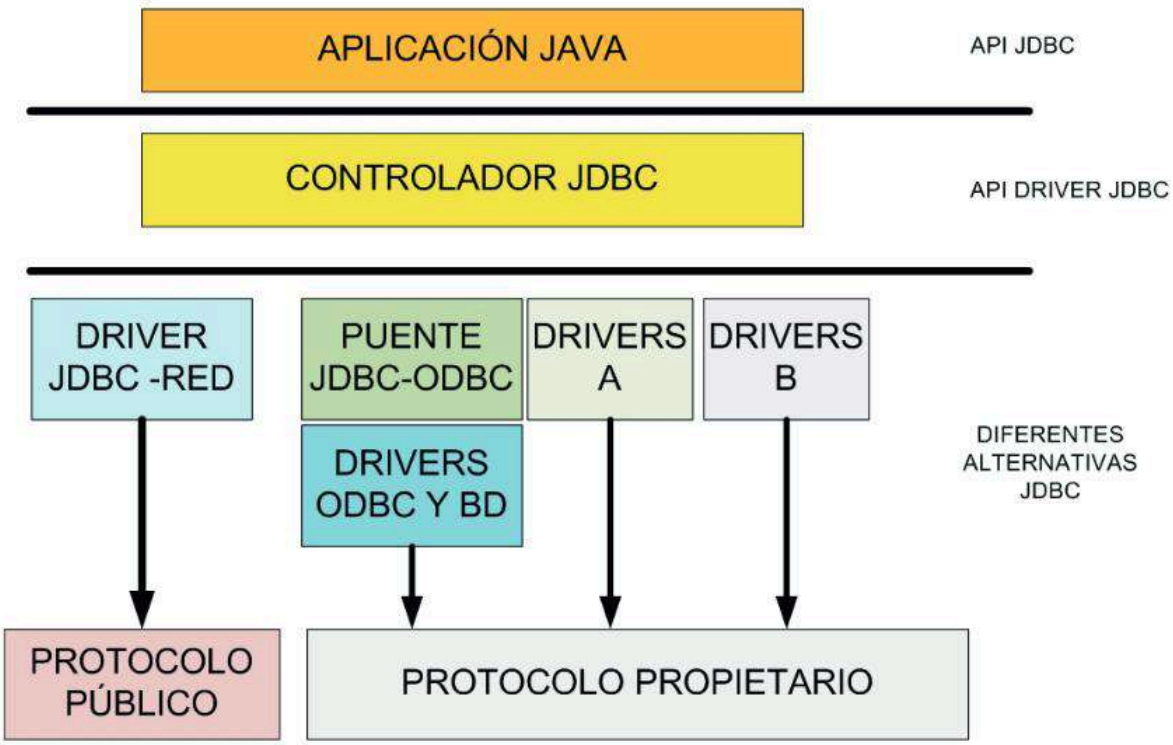



*Diagrama de estructura componentes JDBC y drivers de BBDD*

Toda la conexión distante y remotamente con bases de datos de Java se basa en sentencias "Sql", por lo que necesita obligatoriamente una formación correcta de "Sql" para trabajar correctamente con sentencias de operación de bases de datos. Existen entornos de desarrollo Java permiten trabajar en modo gráfico y más fácil. Aunque para usar directamente el JDK se haga imprescindible. La especificación Jdbc necesita que algunos driver Jdbc seas compatibles con al menos el nivel de entrada de Ansi Sql.

## Acceso de JDBC a bases de datos

El API Jdbc puede trabajar con dos modelos de conexión con Bases de Datos, el estandar de dos capas y el de tres capas.

### Modelo de dos capas

La conexión entre la aplicativo **Java** o "Applet" que rueda en el navegador Firefox, Opera, Explorer o Chrome, se basa en el modelo de dos capas de conexión a base de datos.

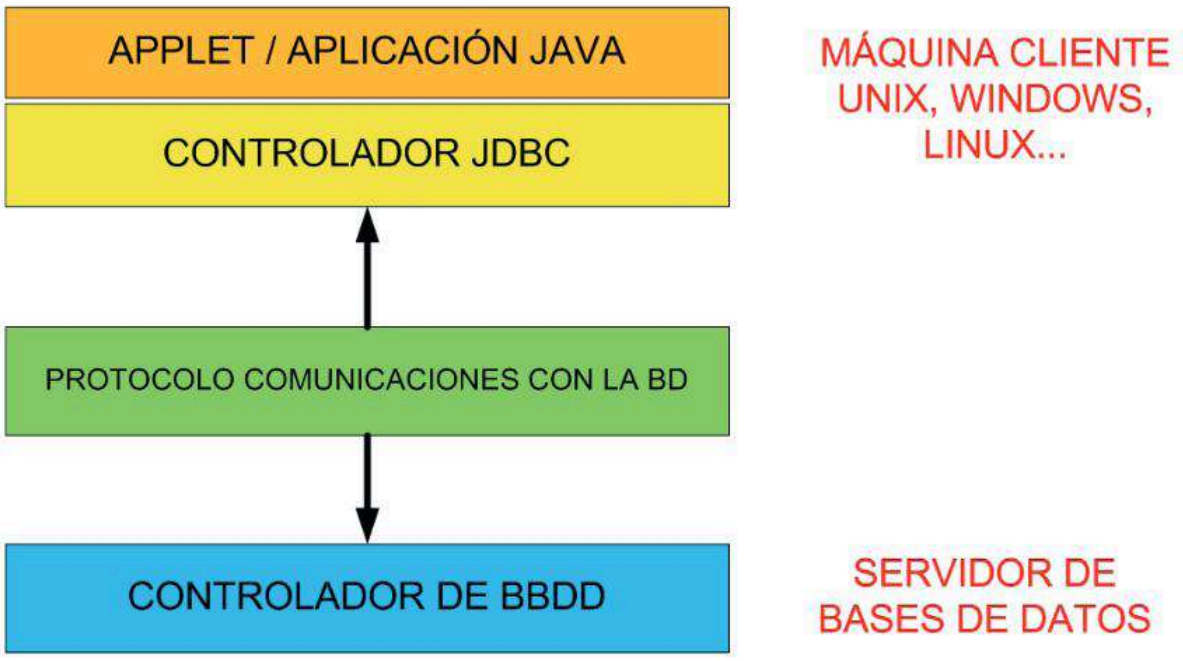



*Diagrama de estructura Modelo de dos capas componentes JDBC*

En este modelo el driver Jdbc para enganchar con la base de datos, debe residir en el sistema local. La base de datos están en el servidor y se conecta a ella mediante la red local o internet. Esta es la disposición muy conocida "Cliente/Servidor" el software cliente manda sentencias "Sql" a la base de datos remota, ésta las procesa y manda los resultados de retorno a la aplicación.

## Modelo de tres capas

En este modelo de acceso a las bases de datos, los mandatos son enviados a una capa intermedia entre "Cliente y Servidor", que es la que se encarga de enviar las sentencias "Sql" a la base de datos y recoger el resultado desde la base de datos. En este caso el usuario no tiene contacto directo, ni a través de la red, con la máquina donde reside la base de datos.

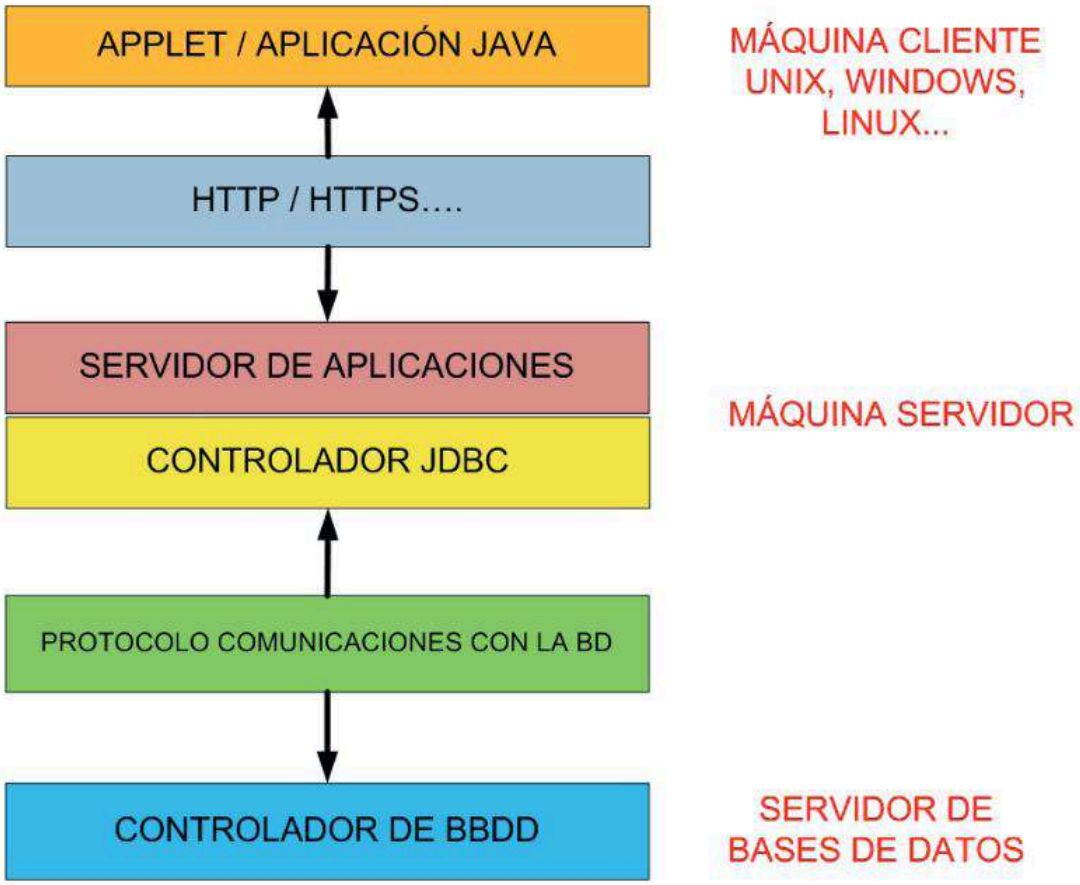



*Diagrama de estructura Modelo de tres capas componentes JDBC*

Gracias a este modelo tres capas, que tiene una pequeña ventaja relativa al control de las operaciones existentes, debido a un paso entremedias a la hora de conectar y procesar información, en la base de datos remota. Los drivers Jdbc no necesitan estar situados en la máquina cliente, lo cual libra al usuario de la instalación de cualquier tipo de driver.

## Tipos de drivers

Un driver jdbc podrá pertenecer a distintas categorías según su manera de actuar.

### Puente JDBC–ODBC

La primera categoría de drivers es la usada por Sun inicialmente luego por Oracle en sus versiones más actualizadas, para presentar Jdbc y consiste en crear un puente entre Jdbc y Odbc. Este driver convierte todas las llamadas Jdbc a llamadas Odbc y realiza la conversión correspondiente de los resultados.

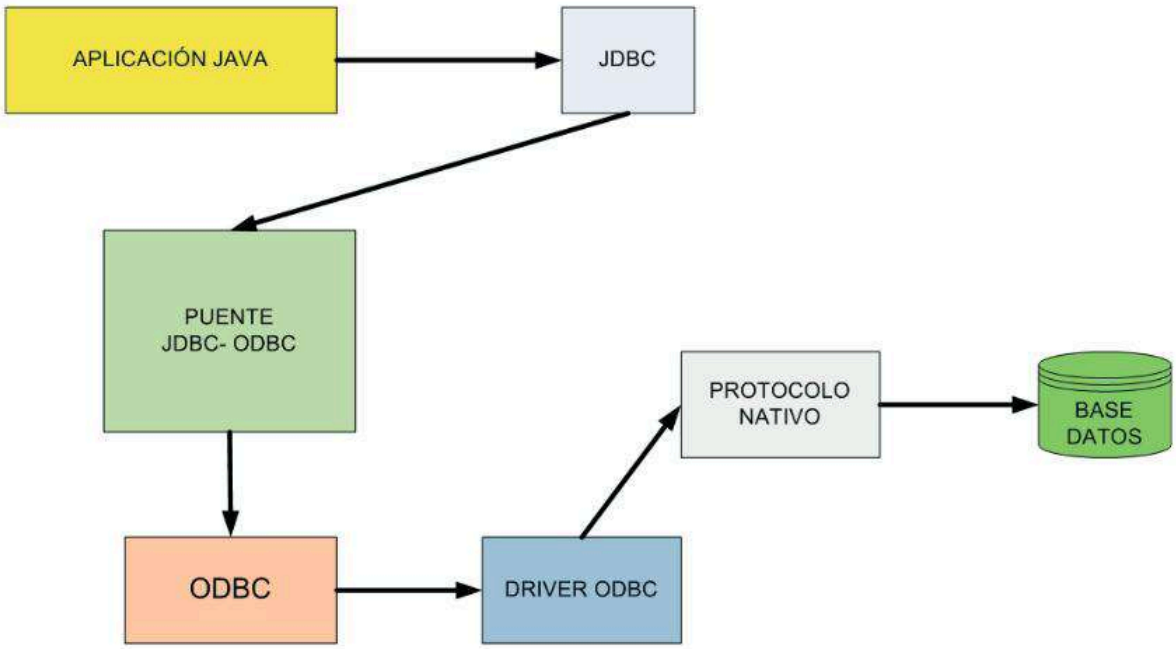



*Diagrama de estructura puente JDBC y ODBC de acceso a BBDD*

Este driver se entrega con el JDK, ya que Java dispone de acceso instantaneo a todas las fuentes de bases de datos posibles, sin tener que hacer configuraciones adicionales aparte de la que ya existe. No obstante, tiene dos inconvenientes muy grandes; por un lado, la mayoría de los drivers Odbc a su vez convierten sus llamadas a invocaciones a una librería nativa del fabricante Ddms, con lo cual la lentitud del driver JDBC–ODBC se eleva demasiado, al llevar dos capas de más que no añaden funcionalidad alguna; y por otra parte, el puente JDBC–ODBC demanda una instalación Odbc ya existente y configurada.

En la unidad todos los ejercicios están referidos al puente JDBC–ODBC y a la conectividad Jdbc con servidores "Sql" comerciales. Se puede trabajar muy bien trabajando sin necesidad de adquirir paquetes comerciales, aunque también se puede utilizar mySQL, se recomienda MySQL o PostGres, que es un sistema de base de datos relacional que une las estructuras clásicas con los conceptos de programación orientada a objetos.

Drivemanager es la clase que se necesita para cargar todos los drivers JDBC. Un programa podrá utilizar "DriverManager" para obtener un objeto de forma de conexión, "Connection" con una base de datos. La conexión se asigna usnado una sintaxis basada en Url, de la manera como sigue:

```
jdbc:protocolo//servidor:puerto/base de datos
```

Para entenderlo mejor, diremos que si usamos MySQL el nombre del protocolo será "Mysql". En algunas ocasiones es necesario identificar aún más el protocolo. Por el contrario si se usa el puente jdbc–odbc no es bastante con jdbc:odbc, ya que podrían coexistir múltiples drivers ODBC, y en este caso, hay que especificar aún más, mediante jdbc:odbc:fuente de datos.

Con el objeto "Connection", podemos crear todo tipo de elementos y componentes como sentencias, statements, ejecutables. Cada sentencia podría retornar un valor null o diversos datos, que se devuelven como objetos de tipo ResultSet.

La tabla siguiente presenta lista de clases e interfaces relacionados con bases de datos:

| Clase/Interface | Descripción |
|---|---|
| **Driver** | Posibilita conectarse a una base de datos cada gestor de base de datos necesita un driver distinto |
| **DriverManager** | Posibilita gestionar todos los drivers instalados en el sistema |

| Clase/Interface | Descripción |
|---|---|
| DriverPropertyInfo | Aporta diversa información acerca de un driver |
| Connection | Asigna una conexión con una base de datos. Una aplicación puede tener más de una conexión a más de una base de datos |
| DatabaseMetadata | Aporta información acerca de una Base de Datos, como las tablas que contiene, etc. |
| Statement | Posibilita ejecutar sentencias SQL sin parámetros |
| PreparedStatement | Posibilita ejecutar sentencias SQL con parámetros de entrada/TD> |
| CallableStatement | Posibilita ejecutar sentencias SQL con parámetros de entrada y salida, típicamente procedimientos almacenados |
| ResultSet | Posee las filas o registros obtenidos al ejecutar un SELECT |
| ResultSetMetadata | Posibilita obtener información sobre un ResultSet, como el número de columnas, sus nombres, etc. |

El ejercicio sirver para ver como conectar con una tabla en el servidor de una base de datos, usando un puente Jdbc–Odbc y la fuente del servidor es Sql Server. Las pocas líneas de establecimiento de conexión con el servidor son las más problemáticas y siempre nos prodcue mucho gasto de tiempo el encontrar bien los parámetros, de contraseña, servidor, ip en algunos casos.

Si da errores puede ocurrir al comienzo seria interesante activar las llamadas traza Odbc en el panel de control del sistema operativo Windows 7 o el que tengamos Linux, Mac etc… La aplicación siguiente es un ejemplo en donde se aplica el esquema anterior, crea una tabla y rellena algunos datos iniciales.

```
import java.sql.*;

class enganchaBbdd {
  static public void main( String[] args ) {
    Connection conexion;
    Statement sentencia;
    ResultSet resultado;

    System.out.println( “Iniciando programa.” );

    // Se carga el driver JDBC-ODBC
    try {
      Class.forName( “sun.jdbc.odbc.JdbcOdbcDriver” );
   } catch( Exception e ) {
      System.out.println( “No se pudo cargar el puente JDBC-ODBC.” );
      return;
    }

    try {
      // Se establece la conexión con la base de datos
      conexion = DriverManager.getConnection( “jdbc:odbc:Tutorial”,””,”” );
      sentencia = conexion.createStatement();
      try {
         // Se elimina la tabla en caso de que ya existiese
         sentencia.execute( “DROP TABLE AGENDA” );
      } catch( SQLException e ) {};
```

```
      // Esto es código SQL
      sentencia.execute( "CREATE TABLE CONTACTOS ("+
        " NOMBRE VARCHAR(15) NOT NULL, " +
        " APELLIDOS VARCHAR(30) NOT NULL, " +
        " CUMPLE DATETIME) " );
      sentencia.execute( "INSERT INTO CONTACTOS " +
        "VALUES('JOSE','GONZALEZ','03/15/1973')" );
      sentencia.execute( "INSERT INTO CONTACTOS " +
        "VALUES('PEDRO','GOMEZ','08/15/1961')" );
      sentencia.execute( "INSERT INTO CONTACTOS " +
        "VALUES('GONZALO','PEREZ', NULL)" );
    } catch( Exception e ) {
      System.out.println( e );
      return;
    }
  System.out.println( "Creacion finalizada." );
  }
}
```

El siguiente bloque de código muestra una conexión típica con Excepción construida. El import trae toda la funcionalidad de la clase para Jdbc con la primera instrucción.

```
import java.sql.*;
```

Las siguientes líneas son las que cargan el puente Jdbc–Odbc, mediante el método forName() de la clase Class.

```
try {
  Class.forName( “sun.jdbc.odbc.JdbcOdbcDriver” );
  } catch( Exception e ) {
    System.out.println( “No pudo cargarse el puente jdbc-odbc.” );
    return;
  }
```

DriverManager se preocupa de leer todos los drivers Jdbc compatibles, aunque no está de más colocar siempre dicho pequeño código por lo que es mejor asegurarse. El método forName() localiza, lee y enlaza una clase determinada. Para los drivers Jdbc, la sintaxis de forName() es “nombreEmpresa.nombreBaseDatos.nombreDriver”, y el driver deberá estar ubicado en el directorio “nombreEmpresa\nombreBaseDatos\nombreDriver.class” a partir del directorio indicado por la variable de entorno **classpath**. En este caso se indica que el puente jdbc–odbc que se desea leer es precisamente el de Sun. o el de Oracle “oracle.jdbc.odbc.JdbcOdbcDriver»

Si por cualquier motivo no es posible conseguir cargar JdbcOdbcDriver.class, salta la excepción y se sale del programa. La carga del driver también se puede especificar desde la línea de comandos al lanzar la aplicación:

```
java –D jdbc.drivers=sun.jdbc.odbc.JdbcOdbcDriver miPrograma
```

DriverManager proporcionará una conexión para una fuente de datos Odbc. El parámetro jdbc:odbc:Tienda3 especifica que la intención es acceder a la fuente de datos con nombre Tienda3, Data Source Name o DSN, en la terminología ODBC.

```
conexion = DriverManager.getConnection("jdbc:odbc:Libreria","","" );
```

El segundo y tercer parámetro son el nombre del usuario y la clave con la cual se intentará la conexión. En este caso el acceso es libre, para acceder como administrador del sistema en el caso de un servidor MySQL se usa la cuenta system o administrator, cuya cuenta de acceso no tiene clave definida; en caso de acceder a un servidor con Microsoft Access, la cuenta del administrador es admin y también sin clave predefinida. Esta es la única línea que con seguridad habrá de cambiar el desarrollador para chequear sus aplicaciones. getConnection soporta además una manera con un único parámetro, el URL de la base de datos, que debe asignar toda la información de conexión necesaria al driver Jdbc correspondiente. Para el caso Jdbc–Odbc, podría usarse correctamente la sentencia siguiente:

```
DriverManager.getConnection ( "jdbc:odbc:SQL;UID=sa;PWD=" );
```

Seguidamente trás obtener la conexión, en la siguiente línea:

```
sentencia = conexion.createStatement();
```

Se solicitará que aporte un objeto de tipo "Statement" que podrá arrancar instrucciones a través de esa conexión. Por ello posee de los métodos execute String sentencia para ejecutar una petición SQL que no retorna datos o executeQuery String sentencia, para ejecutar una consulta SQL. Este último método devuelve un objeto de tipo ResultSet.

Desde el momento que disponemos del objeto Statement ya podremos enviar consultas y ejecutar instrucciones contra el servidor. Todo lo demás es realmente instrucciones de lenguaje SQL insertadas y enviadas en la línea:

```
sentencia.execute( “DROP TABLE CONTACTOS” );
```

Se ejecuta DROP TABLE CONTACTOS para borrar cualquier tabla existente anteriormente. Puesto que este ejemplo es una aplicación «de instalación» y es posible que la tabla CONTACTOS no exista, dando como resultado una excepción, se aísla la sentencia.execute() mediante un try y un catch.

La línea siguiente ejecuta una sentencia SQL que crea la tabla CONTACTOS con tres campos: NOMBRE, APELLIDOS y CUMPLE. De ellos, únicamente el tercero, correspondiente al cumpleaños, es el que puede ser desconocido, es decir, puede contener valores nulos.

```
sentencia.execute( “CREATE TABLE CONTACTOS (“+
        “ NOMBRE VARCHAR(15) NOT NULL, “ +
        “ APELLIDOS VARCHAR(30) NOT NULL, “ +
        “ CUMPLE DATETIME) “ );
```

Y finalmente en las líneas siguientes se ejecutan instrucciones INSERT para cumplimentar con datos la tabla. En todo momento se ha puesto adecuadamente un try… catch exterior para capturar cualquier excepción que pueda llevarse a cabo por las sentencias. En general, para java.sql está definida una clase peculiar para excepciones que es SQLException. Se captará una excepción de este tipo en el momento que suceda algún error de proceso de Jdbc, incluidos si es de nivel Jdbc, como si es a nivel inferior Odbc o ya de protocolo.

Por ejemplo, si en lugar de CAYETANO en la línea correspondiente a la última inserción en la Base de Datos, se intenta insertar un nombre nulo (Null), se activará una excepción.

SQLException con el mensaje de advertencia

```
[Microsoft][ODBC SQL Server Driver][SQL Server]Attempt to insert the
value NULL into column DIRECCION, table 'master.dbo.UNIVERSIDAD;
column does not allow nulls. INSERT fails.
```

En el caso de Microsoft Access sería:

```
[Microsoft][ODBC Microsoft Access 97 Driver] The field 'AGENDA.
NOMBRES' can't contain a Null value because the Required property for
this field is set to True. Enter a value in this field.
```

Es porque la columna NOMBRE esté definida como NOT NULL, hace que no pueda quedarse vacía.

Con la base de datos ya funcionando, podemos acceder a ella y consultar Query y Reports de todo tipo, de acuerdo a alos campos y registros que tengamoa almacenados en la base de datos. Se utilizará executeQuery en lugar de execute para obtener resultados. Se sustituyen las líneas que contenían esa sentencia por:

```
resultado = sentencia.executeQuery( "SELECT * FROM CONTACTOS" );
while( resultado.next() ) {
  String nombre = resultado.getString( "NOMBRE" );
  String apellidos = resultado.getString( "APELLIDOS" );
  String cumple = resultado.getString( "CUMPLE" );
  System.out.println( "El aniversario de D. " + nombre + " "
    + apellidos + ", se celebra el " + cumple );
  }
```

En este caso, en la primera línea se utiliza executeQuery para obtener el resultado de SELECT * FROM CONTACTOS. Mediante resultado.next() la posición se situará en el siguiente elemento del resultado, o bien sobre el primero si todavía no se ha utilizado. La función next() devuelve true o false si el elemento existe, de forma que se puede iterar mediante while (resultado.next()) para tener acceso a todos los elementos.

En las líneas siguientes se utilizan los métodos getxxxx() de resultado para tener acceso a las diferentes datos. El acceso podrá realizarse por el nombre de la columna, como en las dos primeras líneas, o por su ubicación relativa, como en la última línea. Además de getString() están disponibles getInt(), getLong(), getBoolean(), getByte(), getDouble(), getFloat(),getNumeric(), getObject(), getShort(), getDate(), getTime() y getUnicodeStream(), cada uno de los cuales retorna la columna en el formato oportuno.

Se recomienda cerrar las conexiones finalmente, una vez que hayamos trabajado con ellas mediante sentencia.close() o conexion.close(). Este comportamiento se puede cambiar mediante el método Connection.setAutoCommit(). Conectar con una base de datos remota en un servidor en Java con componetens Jdbc no es difícil. Ló único complicado en elegir bien los drivers y controaldores, tener siempre a mano las contraseñas y la url del servidor donde residen nuestras bases de datos.

# UD9
# Lo más importante

- Podemos escribir código Java en una plataforma y marcharnos a otra con la garantía de que esta también entenderá el código sin necesidad de tener que reescribirlo.
- El Kit de desarrollo de Java (JDK) contiene las herramientas y librerías necesarias para crear y ejecutar applets y aplicaciones en Java.
- javac. Es el compilador de Java. Se encarga de convertir el código fuente escrito en Java a bytecode.
- java. Es el intérprete de Java. Ejecuta el bytecode a partir de los archivos class.
- Existen unas librerías y clases Java 3D para Gráficos 3D y su manipulación.
- El directorio en el que se encuentran las herramientas Java debe estar en el path del sistema.
- Un programa en Java puede construirse empleando varias clases. En el caso más simple se utilizará una única clase. Esta clase contiene el programa, rutina o método principal: main() y en éste se incluyen las sentencias del programa principal.
- NetBeans es un entorno de desarrollo multilenguaje muy utilizado tanto por programadores nóveles como por profesionales.
- Eclipse es una plataforma de software de Código abierto independiente de una plataforma para desarrollar Aplicaciones. Esta plataforma, es usada para desarrollar entornos integrados de desarrollo (IDE), como el IDE de Java (Java Development Toolkit JDT).

– Notepad++ es un programa para editar código fuente de cualquier lenguaje de programación. Como tiene soporte para una gran cantidad de lenguajes, interesará no sólo a los desarrolladores de webs, sino en general a toda la comunidad de programadores.

– Java Sound Es un API utilizado ór Java para la utilización de multimedia y su presentación en Applets y aplicaciones.

– Los paquetes de clases se cargan con la palabra clave import, especificando el nombre del paquete como ruta y nombre de clase. Se pueden cargar varias clases utilizando un asterisco. import java.Date; import java.awt.*;

– java.awt–AWT son las siglas de Abstract Windows Toolkit y es la parte de Java que se permite construir interfaces gráficas. Para construir una interfaz, se necesitan al menos tres elementos, un contenedor, unos componentes y un modelo de eventos.

– Para declarar un atributo de una clase, basta con declararlo como cualquier variable dentro del cuerpo de dicha clase, la forma sería: [visibilidad] TipoClase nombreAtributo;

– Exception, Es la clase base de aquellas excepciones que puede lanzar un programa, por lo que es la clase que más interesa al programador.

– El lenguaje provee de construcciones del tipo try–catch–finally que facilitan la escritura del código en presencia de excepciones y evita tener que incluir sentencias condicionales cada vez que se llama a un método que puede generar una excepción.

– Un applet es un miniprograma que corre solamente bajo un navegador y es descargado automáticamente como parte de una página Web, al igual que cualquier gráfico, y cuando se activa, ejecuta un programa. Este es el interés, proporciona una forma a través de la cual se puede distribuir software al cliente desde el servidor.

– Los sockets son basicamente formas en las que podemos interconectar 2 (o mas) programas mediante el uso de la internet. En Java se utilizan para poder crear conexiones utilizando basicamente una IP/hostname y un puerto para establecer la conexión.

# UD9
## Autoevaluación

1. Dentro de las utilidades que se pueden encontrar en el JDK: Javac es?
   a. Es el compilador de Java.
   b. Es el intérprete de Java.
   c. Es un visor de applets.
   d. Es un desensamblador de Java.

2. ¿Como ejecutar un programa Java?
   a. Teclear “java fichero” aplicacion1.java.
   b. Teclear “java fichero” aplicacion1.jar.
   c. Teclear “java nombre clase” . El fichero nombreclase.class dentro ruta.
   d. Ninguna de las anteriores.

3. ¿Qué afirmación es verdadera?
   a. NetBeans es una herramienta con licencia para poder programar.
   b. NetBeans como Eclipse son IDE para programar en Java.
   c. Eclipse ha quedado en desuso desde su última versión en el mercado.
   d. El editor Notepad ++ es muy complicado para programar en Java.

4. ¿Qué expresión es verdadera?
   a. Si la declaración de la clase es «public class Zinnat{…}» el archivo puede llamarse zennat.java.
   b. Java es abierto al nombre de los ficheros que definan la clase y el archivo en la ruta del disco duro.
   c. Si tu clase se llama Auto entonces el archivo deberá ser Auto.java.
   d. Ninguna de las anteriores.

5. ¿Que es falso con respecto a características de Notepad++?
   a. Muy liviano y poco pesado respecto a los recursos del sistema.
   b. Multi–Vista, lo que significa que puedes tener más de una vista de un mismo código.
   c. No permite Coloreado de código.
   d. Autocompletado de código, también configurable por el usuario.

6. ¿Cuál de las siguientes expresiones es falsa?
   a. Los paquetes permiten agrupar clases e interfaces.
   b. Los paquetes de clases se cargan con la palabra clave import.
   c. Un fichero fuente Java no puede compilarse sin ningún package.
   d. Se pueden cargar varias clases utilizando un asterisco.

7. “El ingeniero usa una computadora”, es una manera de interactuar entre objetos por:
   a. Composición.
   b. Asociación.
   c. Adicción.
   d. Ninguna de las anteriores.

8. “El portátil tiene un teclado”, es una manera de interactuar entre objetos por:
   a. Composición.
   b. Asociación.
   c. Adicción.
   d. Ninguna de las anteriores.

9. ¿Qué expresión relativa a Composición es falsa?

   a. Tanto los objetos como la clase contenedora, tienen el mismo tiempo de vida.

   b. Si cualquier objeto muere, se lleva consigo a todos los demás.

   c. Tanto los objetos como la clase contenedora, tienen diferentes tiempo de vida.

   d. Es una relación demasiado dependiente entre objetos.

10. Las clases y métodos abstractos se definen con la palabra clave abstract, pueden llevar el modificador abstract:

   a. Los constructores.

   b. Los métodos estáticos.

   c. Los métodos privados.

   d. Ninguna de las anteriores.

Área: informática y comunicaciones

# Glosario

- **Término:** Definición.
- **Abstracción:** En el contexto cada componente denominado objeto sirve como modelo de un componente abstracto que puede realizar una labor, informar y cambiar su estado, y comunicarse con otros objetos.
- **Atributos:** Los parámetros o variables que definen el estado de un objeto se denominan atributos.
- **Boolean:** Tipo de dato que sólo consigue tomar los valores verdadero o falso.
- **Clase:** Una clase en una especie de plantilla o modelo que describe como se crean ciertos tipos de Objeto. Cada vez que se construye un Objeto de una Clase, se crea una instancia de esa Clase("instance"). Una Clase es una colección de Objetos similares y un Objeto es una instancia de una Clase.
- **Catch:** Captura las excepciones formadas por las sentencias try.
- **Complejidad:** La complejidad es el número de posibles vías lógicas que tiene cada componente, es decir, las sentencias selectivas (if, case), sentencias iterativas (while, for), lanzamiento y tratamiento de excepciones (throw, catch…).
- **Encapsulamiento:** Con esta propiedad que permite aunar todos los componentes que pueden considerarse pertenecientes a una misma entidad, al mismo nivel de abstracción.

- **Extends:** Indica que una clase es derivada de otra o de una interfaz.

- **Extensibilidad:** Esta es una ventaja que gracias a la modularidad y a la herencia, una aplicación diseñada bajo el paradigma de la orientación a objetos puede ser fácilmente extensible para cubrir necesidades de crecimiento de la aplicación.

- **Flexibilidad:** Partimos de la premisa que mediante la definición de clases instauramos módulos independientes, a partir de los cuales podemos definir nuevas clases.

- **Herencia:** Esta es una propiedad muy importante y ventajosa, ya que habla sobre que las clases no están aisladas, sino que se relacionan entre sí, formando una jerarquía de clasificación. Los objetos heredan las propiedades y el comportamiento de diferentes las clases a las que pertenecen.

- **import:** Referencia a otras clases.

- **Mantenibilidad:** Las clases que conforman una aplicación, vistas como módulos independientes entre sí, son fáciles de mantener sin afectar a los demás componentes de la aplicación. Si en la vida real el programa toma un tamaño considerable de clases y paquetes diferentes, será igual, fácil de mantener y administrar.

- **Métodos:** Las acciones que pueden realizar los objetos se denominan métodos.

- **Objeto:** Los objetos son las cosas físicas y conceptuales que encontramos en el universo alrededor de nosotros. Hardware, software, documentos , seres humanos, los conceptos son todos los ejemplos de los objetos. Todo lo que nos rodea son objetos.

- **Ocultación:** Esta propiedad permite poseer aislado cada objeto del exterior, y cada tipo de objeto expone solo una interfaz a otros objetos que específica cómo permiten interactuar con los objetos de la clase.

- **Oop:** Este paradigma de programar orientado a objetos no es una visión conceptual nueva, sus inicios y técnicas de programación se iniciaron a principios de los 70. Se puede explicar brevemente programación orientada a objetos (OOP) como una técnica ó manera de programar con un lenguaje que utiliza objetos como bloques esenciales de construcción.

- **Private:** Especificador de acceso que indica que un método o variable sólo puede ser accesible desde la clase en la que está declarado.

- **Polimorfismo:** Esta propiedad del lenguaje Java, habla de la capacidad de poseer comportamientos diferentes, asociados a objetos distintos, y pueden incluso compartir el mismo nombre.

- **Protected:** Especificador de acceso que muestra que un método o variable sólo puede ser accesible desde la clase en la que está declarado (o una subclase de la clase en la que está declarada u otras clases del mismo paquete).

- **Package:** Declara un paquete Java.

- **Public:** Especificador de acceso utilizado para clases, interfaces, métodos y variables que indican que un tema es accesible desde la aplicación (o desde donde la clase defina que es accesible).

- **Reusabilidad:** Por medio de la reusabilidad podemos utilizar una clase definida previamente en las aplicaciones que nos sea conveniente. Es claro que la flexibilidad con la que se definió la clase va a ser fundamental para su reutilización.

- **Robustez:** Java ejecuta comprobaciones en busca de dificultades tanto en tiempo de compilación como en tiempo de ejecución, lo que hace que se detecten errores lo antes posible.

- **Static:** Indica que una variable o método es un método de una clase (más que estar limitado a un objeto particular).

- **This:** Se refiere al objeto actual en un método o constructor.

- **Void:** Especifica que un método no devuelve ningún valor.

- **Volumen:** El volumen de las aplicaciones hace mención al tamaño de las mismas medido en líneas de código, lo que afecta directamente a su mantenibilidad.

Área: informática y comunicaciones